A READING THROUGH TECHNOLOGY AND ECONOMICS

Blockchain for Lawyers

A self-study guide to blockchains that a legal scholar can read to the end with high-school mathematics. It explains the subject, from hashes and signatures to fee markets and stablecoins, in the language of technology and economics, and marks the places where legal questions arise.

Edited by **Bowon Kwon**, Presiding Judge, Daegu District Court

Contents · Parts 0–6 in 30 chapters, with Appendices A–F
Interactive figures · 12 (screen edition; the print edition shows a still image in the same place)
Notebooks · 21 Colab notebooks (standard library only; numpy in Chapter 25 alone)
Reference date for figures · 2026-10-25

Printing · Ctrl+P and save as PDF; every collapsible box opens.
Offline · This file opens with nothing but the fonts folder beside it. The interactive figures run without a connection; links open when you are online.

This book is an academic self-study guide. It is not investment advice or legal advice. The full notice is in the colophon (Appendix F.4).

Edition 1.0-en · November 14, 2026 · Compiled with generative AI. It may contain errors, and corrections will continue.

Contents

Chapter 1. The problem a blockchain solves: preventing double spending without a trusted third party 10

Learning objectives 10

1. A world where copying is free 11

2. Three kinds of ledger 12

3. Why it is hard: order and majority 14

4. The price: slowness, cost, and no "done" 15

5. A map of this book 16

6. Summary 17

Sources for this chapter 17

Chapter 2. A 30-year prehistory: from electronic cash to the white paper 19

Learning objectives 19

1. Timeline: two rules before reading 20

2. 1976: splitting the key in two 23

3. 1982 / 1983: Chaum's electronic cash, anonymity gained and the bank left in place 23

4. 1980 / 1987: Merkle's trees; 1982: the Byzantine generals 25

5. 1991 / 1993: Haber and Stornetta's chain, which fixed the order, but 26

6. 1997: Hashcash, putting a price on computation 26

7. 1998: b-money, money sketched and consensus left blank 27

8. 2005: bit gold, a registry of title and time-stamping 27

9. 2008: the white paper, an assembly 28

10. 2013 / 2014: Ethereum, programs on the ledger 29

11. 2014 to 2023: milestones after the white paper, and the later chapters of this book 29

12. Summary 30

Sources for this chapter 30

Chapter 3. How to read this book: markers, boxes, interactive figures, notebooks, and three paths 35

Learning objectives 36

1. Three levels of marker 36

2. Three boxes 36

3. Interactive figures and notebooks 37

4. Three paths 39

5. Notes to the reader: figures, the reference date, marks 40

6. Summary 41

Sources for this chapter 42

Chapter 4. Hash functions: one-way functions that take a fingerprint 43

Learning objectives 43

1. What failure this technology grew out of 43

2. Intuition: a fingerprint, and what a fingerprint cannot do 44
3. Code: one character changes everything 46
4. [Intermediate] The birthday paradox: collision resistance is only half as strong as the number of bits 50
5. What is SHA-256? 52
6. Summary: half of "tamper-proof" 54
Sources for this chapter 54

Chapter 5. Public-key signatures: splitting authority between two keys 56
Learning objectives 56
1. What failure this technology grew out of 57
2. The intuition: a signature is approval, not encryption 57
3. [Intermediate] Reading the signing formula: ECDSA 60
4. Code: doing all of it on a toy curve 62
5. Summary 68
Sources for this chapter 68

Chapter 6. Merkle trees: vouching for the whole with a single root 70
Learning objectives 70
1. What failure this technology grew out of 70
2. The intuition: a tournament bracket 71
3. [Intermediate] The length of a proof: why $\log_2 n$ 73
4. Bitcoin's Merkle tree: two implementation features and one weakness 77
5. Summary: what the root vouches for 79
Sources for this chapter 79

Chapter 7. Blocks and the chain: a structure that makes tampering costly 82
Learning objectives 82
1. What failure this technology grew out of 83
2. What a block is made of 83
3. Why tampering shows up in a cascade 84
4. Code: a toy chain 87
5. Summary 90
Sources for this chapter 91

Chapter 8. Proof of work: difficulty, mining, and the longest chain 93
Learning objectives 93
1. What failure this technology grew out of 94
2. Intuition: a lottery 94
3. [Intermediate] Formulas: probability of success, expected number of attempts, difficulty 95
4. Code: a hand miner 97
5. The "longest chain" and the share q 103
6. Summary 104

Chapter 9. The probabilistic nature of finality: what "confirmations" mean 107
Learning objectives 107
1. What an attacker can and cannot do 108
2. [Intermediate] The formula in §11 of the white paper, in two stages 109
3. Code: reproducing the white paper's tables, and going beyond them 111
4. "Done" is a probability 118
5. Summary 119
Sources for this chapter 119

Chapter 10. Proof of stake: collateral instead of energy 121
Learning objectives 121
1. What failure this technology grew out of 122
2. Intuition: deposits, votes, and punishment 122
3. The current rules: numbers from the specification 125
4. Code: stake-weighted selection and slashing arithmetic 126
5. Proof of work and proof of stake: different kinds of cost 129
6. Summary 130
Sources for this chapter 130

Chapter 11. UTXOs and accounts: two ways a ledger records "balances" 133
Learning objectives 133
1. What failure this technology grew out of 134
2. UTXO: a heap of unspent outputs 134
3. Accounts: recording state for each address 136
4. [Intermediate] Code: two state transition functions 137
5. Comparing the two models, and the two shapes of a "mistaken transfer" 139
6. Summary 140
Sources for this chapter 140

Chapter 12. Anatomy of a real block: unpacking the 80 bytes of the genesis block by hand 142
Learning objectives 142
1. Where the data came from 142
2. Six slots for 80 bytes 143
3. Recomputing the hash and matching it to the published value 145
4. [Intermediate] Unpacking the transaction: the coinbase and the sentence inside it 146
5. Compared with a block from 2026 149
6. Summary: closing Part 1 150
Sources for this chapter 150

Chapter 13. The Ethereum Virtual Machine and gas: a ledger that runs code 152
Learning objectives 152
1. What failure this technology grew out of 153
2. Intuition: accounts, code, transactions, messages 153
3. Gas: how to stop a program that does not stop 156

4. Solidity for reading: the smallest contract 159
5. Code: imitating state transitions and running out of gas with a toy EVM 160
6. Summary: a program called a "contract" 166
Sources for this chapter 166

Chapter 14. Token standards: what ERC-20 and 721 decide and what they leave undecided 171
Learning objectives 171
1. What failure this technology grew out of 172
2. Intuition: coins, tokens, and a single ledger 173
3. ERC-20: the six functions a fungible token promises 174
4. ERC-721: a ledger of things that are each different 176
5. What a token points to: on-chain rights and off-chain promises 178
6. Code: imitating the two ledgers and recomputing the numbers the standards wrote down 179
7. Summary 186
Sources for this chapter 186

Chapter 15. Decentralized exchanges and AMMs: price formation without an order book 190
Learning objectives 190
1. What failure this technology grew out of 191
2. Intuition: a pool of two tokens and one product 192
3. Formulas: constant product, fee, execution price 193
4. Supplying liquidity: fee income and impermanent loss are the two pans of one scale 195
5. A spot price is not an oracle: the bridge to Chapters 16 and 17 197
6. Code: a constant-product pool, the slippage curve, and the two pans of the scale 198
7. Summary 202
Sources for this chapter 202

Chapter 16. The oracle problem: bringing facts from outside the chain into it 205
Learning objectives 205
1. What failure this technology grew out of 206
2. Why the chain does not know the outside 207
3. The ideal oracle, and why it cannot be had 208
4. Three branches of design: what has to be trusted 209
5. At which layer does manipulation happen? 211
6. Summary 213
Sources for this chapter 213

Chapter 17. MEV and transaction ordering: the power to set the order 216
Learning objectives 216
1. What failure this technology grew out of 217
2. The intuition: slip in front, slip in behind, slip in on both sides 218
3. Who sets the order: auctions and producers 219
4. The response, not prohibition but a division of labor: proposer–builder separation 221

5. Code: running the sandwich in pool units 223
6. Summary 227
Sources for this chapter 227

Chapter 18. Scalability: layer 2 and rollups 230
Learning objectives 230
1. What failure this technology grew out of 231
2. The structure of a rollup: execution outside, data and disputes inside 232
3. Optimistic rollups: correct until proven wrong 234
4. Zero-knowledge rollups: a proof of correctness attached every time 236
5. Comparing the two approaches: who has to be honest? 237
6. Trust does not disappear; it moves 238
7. Summary: closing Part 2 239
Sources for this chapter 240

Chapter 19. The functions of money and the "zero intrinsic value" equilibrium 245
Learning objectives 245
1. The question this chapter tries to answer 246
2. The functions approach: a checklist, not a proof 246
3. [Intermediate] The equation of exchange and its limits 247
4. The "zero intrinsic value" equilibrium: three models 249
5. Code: the listing of equilibria in numbers 254
6. What the models say and what they do not 257
7. Summary 258
Sources for this chapter 259

Chapter 20. The security budget: from rewards to fees 264
Learning objectives 264
1. The question this chapter answers 265
2. The two parts of the reward: subsidy and fees 266
3. Budish's three equations: security is a flow 268
4. How long an attack takes and what it costs: Proposition 3 and Table II 271
5. An attack-cost calculator: the inputs are hash rate, unit price, and time 274
6. [Advanced] A world where the subsidy has reached 0: Carlsten et al. 278
7. [Advanced] When a stock is at stake: specialized capital and collapse 280
8. Summary 281
Sources for this chapter 281

Chapter 21. The fee market: from auctions to EIP-1559 286
Learning objectives 286
1. The question this chapter answers 287
2. The first-price auction: what was wrong with it 287
3. EIP-1559: a toll set by formula 288

4. Three kinds of incentive compatibility: who can cheat? 292
5. [Advanced] The blob fee market: the same principle, a different resource 297
6. Summary 299
Sources for this chapter 300

Chapter 22. The economics of token design: issuance rules, governance, incentives 305
Learning objectives 305
1. The question this chapter tries to answer 306
2. Issuance: how much, and by what rule 306
3. Distribution: who receives it at the start 307
4. Governance: who changes the rules 311
5. How "tokenomics" is used as a marketing term 314
6. Summary 315
Sources for this chapter 315

Chapter 23. Game theory: selfish mining and equilibrium 318
Learning objectives 318
1. The question this chapter tries to answer 319
2. The white paper's argument as a payoff table 319
3. Conventional wisdom: "honest mining is a Nash equilibrium" 320
4. Selfish mining: the strategy of hiding blocks 321
5. Other places in the same frame: the coordination game, the cooperative equilibrium, undercutting 326
6. Summary 330
Sources for this chapter 331

Chapter 24. Stablecoins: design types and the structure of a bank run 334
Learning objectives 334
1. The question this chapter tries to answer 335
2. Four types: what backs the promise 336
3. Why runs happen: the economics of "no questions asked" 338
4. [Advanced] Diamond–Dybvig: a run is an equilibrium 339
5. Carrying it over to stablecoins: what is the same and what is different 342
6. Evidence: two runs 347
7. Summary 350
Sources for this chapter 350

Chapter 25. Econometric tests for bubbles: what they measure and what they cannot 355
Learning objectives 355
1. The question this chapter tries to answer 356
2. Two assumptions the definition of a bubble requires 356
3. [Intermediate] The ADF regression and the explosive coefficient 357
4. [Advanced] GSADF, the minimum window, date-stamping, and critical values 360
5. What to take as the fundamental value: three answers 364

6. The literature's findings in one table: sample, period, assumptions, and critical values together 366
7. What the test says and what it does not 369
8. Summary 370
Sources for this chapter 370

Chapter 26. Four cases: Mt.Gox, The DAO, Terra-Luna, FTX 375
Learning objectives 376
1. How this chapter reads the cases: the diagnostic table and the scope of "the blockchain itself" 376
2. Mt.Gox (2014): a mismatch between the books and the chain 378
3. The DAO (2016): a flaw in the contract code and the decision that followed 384
4. Terra-Luna (2022): the reflexivity of the design and the issuer's statements 386
5. FTX (2022): misuse of customer assets and the absence of internal controls 388
6. The four cases in one table 390
7. The scope of "the blockchain itself did not break" 392
8. Summary 393
Sources for this chapter 393

Chapter 27. The bridge between technology and law (1): why this book does not go deep, the map of scholarly positions, a survey by jurisdiction 402
Learning objectives 403
27.1 Why this book does not treat the law in depth 403
27.2 The map of scholarly positions on legal nature: positions matched against technical facts 405
27.3 A survey by jurisdiction 412
Summary 422
Sources for this chapter 423

Chapter 28. The bridge between technology and law (2): an overview by issue, and where to read further 433
Learning objectives 434
28.1 An overview by issue 434
28.2 Where to read further 448
Summary 450
Sources for this chapter 451

Chapter 29. The quantum transition: signatures break, hashes stand, and the vulnerable coins 456
Learning objectives 456
1. Two algorithms, two targets: an asymmetry 457
2. Resource estimates: two kinds of qubit, five assumptions 460
3. What is exposed: three places where a public key appears 463
4. Transition proposals: output types, signatures, sunset 471

6. "As of 2026-10": what in this chapter will date 477
7. Summary 478
Sources for this chapter 478

Chapter 30. The naming problem: 가상자산, crypto-asset, digital asset, 暗号資産, cryptoasset 487
Learning objectives 487
1. Why the name matters 488
2. The definitional provisions of five jurisdictions, in the original 488
3. Reading the provisions on top of one another: what goes in and what is left out 497
4. What the name decides: jurisdiction, taxation, criminal law 499
5. Three cases in which the name changed within one jurisdiction 502
6. Summary 504
Sources for this chapter 504

Appendix A. Trend notes (as of 2026-10): index (entries from Parts 1–6 and Part 0) 508
Appendix B. Common misconceptions: a verdict table 516
Sources 522

Appendix C. Concordance of technical and statutory terms 523
C.1 The defined terms of the five jurisdictions (Chapter 30, Section 2) 523
C.2 Exclusion lists (the table in Chapter 30, Section 3) 527
C.3 Which words the technical terms reach 528
Sources 533

Appendix D. Timeline, 1976–2023 (the still version of the clickable timeline W11) 534
1. 29 milestones 534
2. Clickable timeline (screen edition) 536
Sources 537

Appendix E. Mathematical supplement 538
Concordance: which box points here 538
E.1 The Merkle–Damgård construction: chaining compression functions 539
Sources 540

Appendix F. Glossary · list of interactive figures · list of notebooks · colophon 541
F.1 Glossary 541
F.2 List of interactive figures 545
F.3 List of notebooks 546
F.4 Colophon 548
Sources 549

PART 0 · A MAP

Chapter 1. The problem a blockchain solves: preventing double spending without a trusted third party

KEY TAKEAWAY

Copying a digital file costs nothing. So to make digital "money" you have to stop **the same money from being spent twice** (double spending), and until now the answer has always been to trust "one party that keeps the ledger": a bank, a mint, a payment company. A blockchain is a way of doing the same job without that one party. It makes every transaction public and writes down which transaction came first in a single history that **a majority agrees on by computation**. The price is slowness and cost. A record is made once every 10 minutes, it takes electricity and equipment, and a record is never "done"; the probability of reversing it only gets smaller. A lawyer should take one thing from this chapter. A blockchain is "a way for many parties to agree on and maintain a single record in a world where copying is free," and so every legal question in this book starts from "there are many copies of that record, so which one is the original?"

Learning objectives

1. Explain why "double spending" is the central difficulty of digital cash.
2. Distinguish a central ledger, a distributed ledger and a blockchain by "who writes the ledger" and "who decides which record came first."
3. Derive the price a blockchain pays (slowness, cost, probabilistic finality) from the structure of its solution.
4. Know which piece of this problem each of the six parts of this book takes up.

What a reader of the [Basic] paragraphs alone takes away. A blockchain is "a way for many parties to agree on and maintain a single record in a world where copying is free," and the price is slowness and cost.

1. A world where copying is free

You cannot hand one banknote to two people at the same time. There is one piece of paper, and when you hand it over it leaves your hand. A file is different. Make a "100-won file" and send it to A, and it stays on your computer just as it was, so you can send it again to B. Both are genuine, since not a single bit differs. This is the problem of **double spending**, and every attempt to build digital cash ran into it. The blind-signature electronic cash that David Chaum proposed at a 1982 conference succeeded in having the bank issue electronic notes that it had signed without the bank knowing **to whom** it had issued them. Yet the eleventh of the thirteen steps in that system's payment procedure reads: "(11) Bank adds note to comprehensive list of cleared notes and stops if note already on list."[1] It is **the bank** that checks whether the same note arrives twice. Anonymity was achieved, but the guard against double spending was still a single bank.

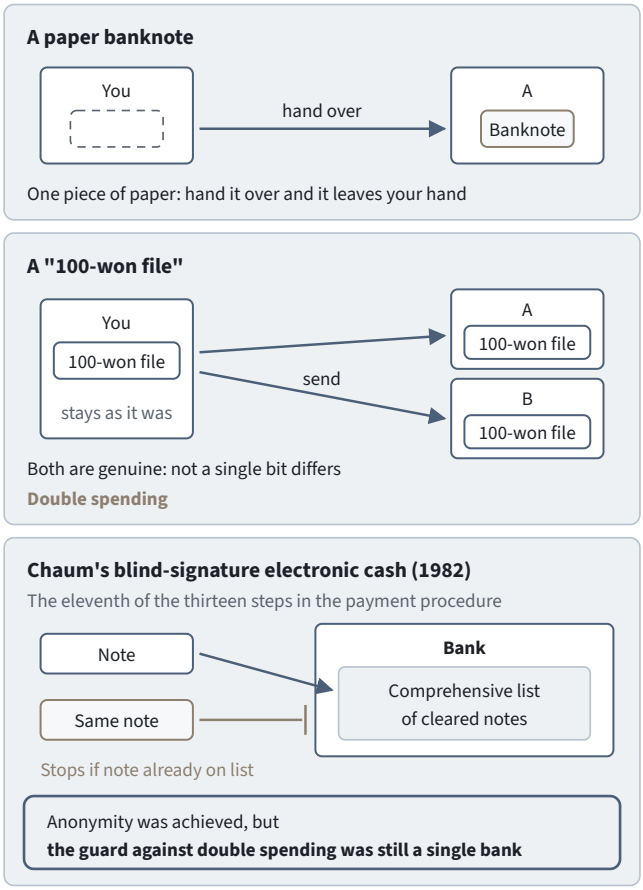


Figure 1-1. A paper banknote leaves your hand when you hand it over, but a "100-won file" sent to both A and B is genuine in both places. In Chaum's system the bank holds a comprehensive list of cleared notes and stops if the same note arrives twice (the eleventh of thirteen steps).

The 2008 white paper states the problem like this: "We define an electronic coin as a chain of digital signatures. ... The problem of course is the payee can't verify that one of the owners did not double-spend the coin. A common solution is to introduce a trusted central authority, or mint, that checks every transaction for double spending." And the problem with that solution: "the fate of the entire money system depends on the company running

the mint, with every transaction having to go through them, just like a bank."[2] Chaum's bank is exactly that mint. A signature proves **who authorized** a payment (Chapter 5), but it cannot prove **that the same money was not authorized twice**. That is not a problem of signatures. It is a problem of **order**.

2. Three kinds of ledger

We sort three kinds by "who writes the ledger" and "who decides which transaction came first."

	CENTRAL LEDGER	DISTRIBUTED LEDGER	BLOCKCHAIN
Who writes the ledger	One party (a bank or mint)	Many parties, each with a copy	Many parties, each with a copy
Who decides what came "first"	That one party	**Undecided** (if the copies diverge, whose is right?)	The chain with the most computation behind it, agreed by computation among a majority of participants
What must be trusted	The honesty and competence of that one party	The honesty of every holder of a copy	That honest participants hold a majority of the computing power
Example	Chaum's bank (1983); today's bank account	The first protocol of Wei Dai's b-money (1998)	Bitcoin (2009–)

The middle column matters. The idea of many parties sharing a ledger existed before blockchains. The first protocol of Wei Dai's b-money in 1998 says that "every participant maintains a (seperate) [sic] database of how much money belongs to each pseudonym. These accounts collectively define the ownership of money ...."[3] But what if the copies differ? If Alice broadcasts to everyone the message "give X to K_B" and **at the same time** broadcasts the message "give X to K_C," some participants receive the first message first and some receive the second first. The two ledgers diverge, and the first protocol has no rule for deciding which is right. Dai himself called it "impractical," writing that it "makes heavy use of a synchronous and unjammable anonymous broadcast channel," and in the second protocol he hands the keeping of the accounts to some of the participants, which he calls "servers." Once again there is someone who has to be trusted.[4] The 2014 Ethereum white paper sums up this history in one sentence: "In 1998, Wei Dai's b-money became the first proposal to introduce the idea of creating money through solving computational puzzles as well as decentralized consensus, but the proposal was scant on details as to how decentralized consensus could actually be implemented."[5]

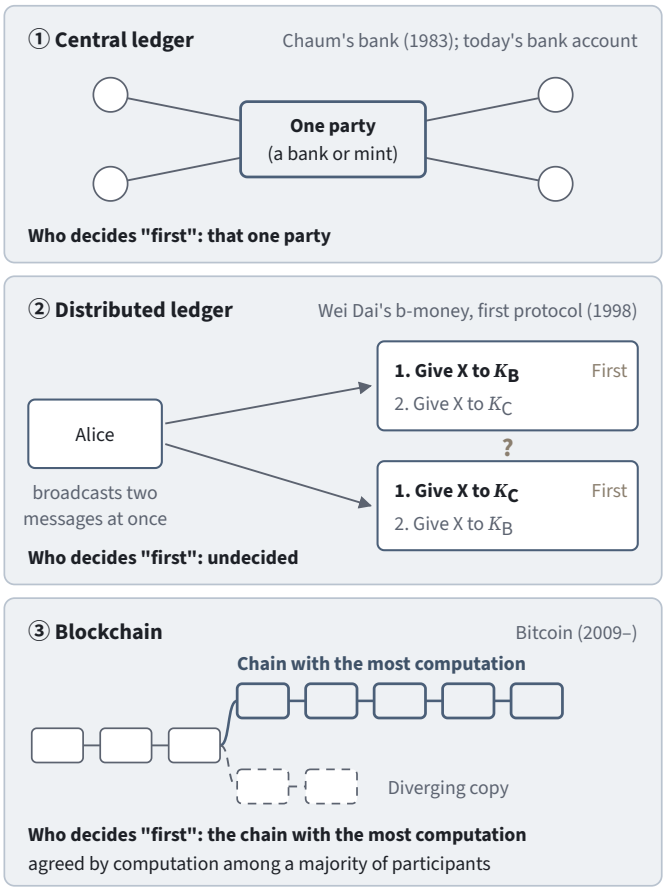


Figure 1-2. The row of the table above on who decides what came "first," drawn as a figure. In a central ledger one party decides. In a distributed ledger, when Alice broadcasts two messages at once, participants receive them in different orders and nothing is decided. In a blockchain the chain with the most computation behind it decides.

The third column supplies those details. The last paragraph of §2 of the white paper states the problem precisely: "We need a way for the payee to know that the previous owners did not sign any **earlier** transactions. For our purposes, the earliest transaction is the one that counts, so we don't care about later attempts to double-spend. The only way to confirm the absence of a transaction is to be aware of all transactions. ... To accomplish this without a trusted party, transactions must be **publicly announced** [1], and we need a system for participants to agree on a **single history** of the order in which they were received. The payee needs proof that at the time of each transaction, the majority of nodes agreed it was the first received."[6] All three elements are there: **publicity** (every transaction to everyone), **a single history** (one order) and **majority agreement** (who decides). Part 1 of this book takes apart, one at a time, the components that produce those three elements, namely hashes (Chapter 4), signatures (Chapter 5), Merkle trees (Chapter 6), the chain (Chapter 7) and proof of work (Chapter 8), and in Chapter 9 it shows why "majority agreement" is given only as a probability.

3. Why it is hard: order and majority

Why "a majority agrees" is not easy had already been shown by a 1982 paper, the "Byzantine Generals Problem" of Leslie Lamport, Robert Shostak and Marshall Pease. Generals who can communicate only by messenger must agree on a common plan of action, but some of them are traitors who may pass on different messages to different generals. Is there an algorithm by which the loyal generals reach agreement? The paper's answer is conditional: "It is shown that, using only oral messages, this problem is solvable if and only if more than two-thirds of the generals are loyal; so a single traitor can confound two loyal generals. With unforgeable written messages, the problem is solvable for any number of generals and possible traitors."[7] Signatures (Chapter 5) supply "unforgeable written messages." Then isn't the problem solved? The Ethereum white paper explains why not. The consensus research before it "assumed that all participants in the system were known, and produced security margins of the form "if N parties participate, then the system can tolerate up to N/4 malicious actors". The problem is, however, that in an anonymous setting such security margins are vulnerable to sybil attacks, where a single attacker creates thousands of simulated nodes on a server or botnet and uses these nodes to unilaterally secure a majority share."[8] On an open network where you do not know who the participants are, there is no way to count "one person, one vote." If not people, what should be counted? The white paper's answer is computation: "Proof-of-work is essentially one-CPU-one-vote."[9] That is Chapter 8, and that is why a blockchain takes electricity.

The Ethereum white paper calls Bitcoin "a first-to-file system": "if one entity has 50 BTC, and simultaneously sends the same 50 BTC to A and to B, only the transaction that gets confirmed first will process. There is no intrinsic way of determining from two transactions which came earlier, and for decades this stymied the development of decentralized digital currency. Satoshi's blockchain was the first credible decentralized solution."[10] There is no **intrinsic** way to decide what came "first." So what a blockchain does is not to discover "first" but to **make** it. It bundles transactions into blocks, links the blocks into a chain, and **defines** the "order" as the chain with the most computation behind it. Chapter 7 looks at that chain and Chapter 8 at that computation.

4. The price: slowness, cost, and no "done"

Not trusting one party is paid for in three ways. **Slowness.** The white paper writes "If we suppose blocks are generated every 10 minutes," and says that difficulty adjustment works by "targeting an average number of blocks per hour."[11] Compared with a bank server that processes each item at once, one batch every 10 minutes is slowness by design. It buys time for everyone to agree on the same history. **Cost.** "The steady addition of a constant of amount [sic] of new coins is analogous to gold miners expending resources to add gold to circulation. In our case, it is CPU time and electricity that is expended."[12] That electricity is the material of the "structure that makes tampering expensive" in Chapter 7, and Chapter 20 looks at where that cost is paid from. **No "done."** The white paper writes that the probability of an attacker catching up with the chain "diminishes exponentially as subsequent blocks are added." It diminishes, but it never becomes 0.[13] A bank transfer has a "point of irrevocability" fixed by rules, but a blockchain has no such moment, only a process in which the probability of reversal gets smaller. Chapter 9 computes that probability, and 28.1(c) notes how it fails to line up with the legal concepts of "performance" and "completion."

> **? A LAWYER'S QUESTION**
>
> A blockchain ledger exists in **thousands of copies**, and none of them carries a mark saying "original." That does not matter while everyone holds the same bytes. But if one node's copy differs from another's by one block (as Chapter 7 shows, this happens by design, and the following blocks bring the copies back together), which one is "the" ledger at that moment? When a party in litigation submits a printout saying "this is what is recorded on the chain," which copy, at what point in time, is it? The evidentiary concepts of original, copy and authenticity appear to presuppose a picture in which there is one "original" and copies reproduce it. In a ledger whose original is **decided afresh at every moment by agreement**, what do those concepts point to? If the chain is reorganized and a block drops out of history, is a printout containing that block a forgery, or a record that was once true? This book does not answer; it passes the question to Part 5. → Chapter 28

5. A map of this book

The six parts divide the pieces of this problem among themselves.

PART	WHAT IT ASKS	CHAPTERS
Part 0 · A map	What the problem is (Chapter 1), where its solution came from (Chapter 2), and how to read this book (Chapter 3)	1–3
Part 1 · Technical foundations	The components that produce "publicity, a single history, majority agreement": hashes, signatures, Merkle trees, the chain, proof of work, probabilistic finality, proof of stake, the two state models, and one real block	4–12
Part 2 · Smart contracts and applications	What happens when **code runs** on that ledger: the virtual machine and gas, tokens, automated markets, the problem of bringing in outside facts, the power to set the order, scaling	13–18
Part 3 · Economics	How money and people move on top of it: theories of money, the funding of security, fees, token design, game theory, stablecoins, measuring bubbles	19–25
Part 4 · Reading through cases	What broke: four cases on the same diagnostic table	26
Part 5 · Law	Where technical and economic facts enter legal judgment: why the law is not treated in depth, the map of scholarly positions, the survey by jurisdiction, the overview by issue	27–28
Part 6 · Open problems	The day signatures break and the coins left behind (Chapter 29), five names for the same thing (Chapter 30)	29–30

Each chapter in Parts 1 and 2 begins with "What failure this technology grew out of." The failure this chapter has shown, that order could not be decided without one party to trust, is the starting point, and Chapter 2 looks at the 30 years in which that failure was filled in piece by piece.

6. Summary

The difficulty of digital cash comes from the fact that copying is free. A signature proves "who authorized it" but not "whether the same money was authorized twice," and the latter is a problem of **order**. Until now the order was decided by one party that held the ledger. Merely sharing the ledger among many parties cannot decide who is right when the copies diverge (the distributed ledger), and on a network where you do not know who the participants are you cannot even count "one person, one vote" (the Sybil attack). A blockchain makes transactions public to everyone and **defines "first" by computation**: the chain with the most work behind it is the history. The price is the slowness of 10 minutes, the cost of electricity and equipment, and, in place of "done," "a shrinking probability of reversal." The question a lawyer should take from this chapter is one. In a ledger whose original is decided afresh at every moment by agreement, what is "the original"? The rest of this book checks, one by one, the **factual premises** that go into that question.

Sources for this chapter

- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — double spending and the mint (§2, p. 2), proof of work and "one-CPU-one-vote" (§4, p. 3), incentive and cost (§6, p. 4), the 10-minute assumption (§7, p. 4), the exponential decline of the probability (§4 and §11).
- David Chaum, "Blind Signatures for Untraceable Payments," *Advances in Cryptology: Proceedings of Crypto 82*, pp. 199–203 (Springer, 1983) — the 13 steps of the payment procedure and the list of cleared notes (p. 202). (scan, read from the images).
- Wei Dai, "b-money," http://www.weidai.com/bmoney.txt, 1998 — the two protocols. Accessed and checked word for word on 2026-10-25.
- Leslie Lamport, Robert Shostak & Marshall Pease, "The Byzantine Generals Problem," *ACM TOPLAS* 4(3): 382–401 (1982) — abstract (p. 382).
- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," official PDF at ethereum.org (2014; first draft 2013) — first-to-file (p. 1), History (p. 4).
- This chapter has no code or notebook (by design). The word "money" is used only as a technical term, and nothing is said about value.

Notes

1. David Chaum, "Blind Signatures for Untraceable Payments," in D. Chaum, R. L. Rivest & A. T. Sherman (eds.), *Advances in Cryptology: Proceedings of Crypto 82*, pp. 199–203 (Springer, 1983), p. 202, "Untraceable Payments System." The same page: "The critical concept is that the bank will sign anything with its private key, but anything so signed is worth a fixed amount, say $1." (scan from the IACR archive with no text layer; read directly from the page images).
2. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), p. 2, §2, Transactions. Original: "We define an electronic coin as a chain of digital signatures." / "The problem of course is the payee can't verify that one of the owners did not double-spend the coin. A common solution is to introduce a trusted

central authority, or mint, that checks every transaction for double spending. After each transaction, the coin must be returned to the mint to issue a new coin, and only coins issued directly from the mint are trusted not to be double-spent. The problem with this solution is that the fate of the entire money system depends on the company running the mint, with every transaction having to go through them, just like a bank."

3. Wei Dai, "b-money," http://www.weidai.com/bmoney.txt (1998; accessed and checked word for word on 2026-10-25). Original: "In the first protocol, every participant maintains a (seperate) [sic] database of how much money belongs to each pseudonym. These accounts collectively define the ownership of money, and how these accounts are updated is the subject of this protocol." The document carries no date; 1998 is the date given in reference [1] of the Nakamoto white paper (Chapter 2).

4. Same text. Original: "The first one is impractical, because it makes heavy use of a synchronous and unjammable anonymous broadcast channel." / "In the second protocol, the accounts of who has how much money are kept by a subset of the participants (called servers from now on) instead of everyone." / "Since the servers must be trusted to a degree, some mechanism is needed to keep them honest."

5. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform" (official version posted at ethereum.org, 2014; first draft 2013), p. 4, "History."

6. Nakamoto (2008), p. 2, §2. Emphasis added. Original: "We need a way for the payee to know that the previous owners did not sign any earlier transactions. For our purposes, the earliest transaction is the one that counts, so we don't care about later attempts to double-spend. The only way to confirm the absence of a transaction is to be aware of all transactions. In the mint based model, the mint was aware of all transactions and decided which arrived first. To accomplish this without a trusted party, transactions must be publicly announced [1], and we need a system for participants to agree on a single history of the order in which they were received. The payee needs proof that at the time of each transaction, the majority of nodes agreed it was the first received."

7. Leslie Lamport, Robert Shostak & Marshall Pease, "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems* 4(3): 382–401 (July 1982), p. 382, abstract.

8. Buterin (2014), p. 4, "History." Original: "The protocols assumed that all participants in the system were known, and produced security margins of the form "if N parties participate, then the system can tolerate up to N/4 malicious actors". The problem is, however, that in an anonymous setting such security margins are vulnerable to sybil attacks, where a single attacker creates thousands of simulated nodes on a server or botnet and uses these nodes to unilaterally secure a majority share."

9. Nakamoto (2008), p. 3, §4, Proof-of-Work. Original: "The proof-of-work also solves the problem of determining representation in majority decision making. If the majority were based on one-IP-address-one-vote, it could be subverted by anyone able to allocate many IPs. Proof-of-work is essentially one-CPU-one-vote. The majority decision is represented by the longest chain, which has the greatest proof-of-work effort invested in it."

10. Buterin (2014), p. 1. Original: "Bitcoin as an application can be described as a first-to-file system: if one entity has 50 BTC, and simultaneously sends the same 50 BTC to A and to B, only the transaction that gets confirmed first will process. There is no intrinsic way of determining from two transactions which came earlier, and for decades this stymied the development of decentralized digital currency. Satoshi's blockchain was the first credible decentralized solution."

11. Nakamoto (2008), p. 4, §7: "If we suppose blocks are generated every 10 minutes ..."; p. 3, §4: "To compensate for increasing hardware speed and varying interest in running nodes over time, the proof-of-work difficulty is determined by a moving average targeting an average number of blocks per hour. If they're generated too fast, the difficulty increases."

12. Nakamoto (2008), p. 4, §6, Incentive.

13. Nakamoto (2008), p. 3, §4: "We will show later that the probability of a slower attacker catching up diminishes exponentially as subsequent blocks are added."; p. 7, §11: "Given our assumption that p > q, the probability drops exponentially as the number of blocks the attacker has to catch up with increases."

PART 0 · A MAP

Chapter 2. A 30-year prehistory: from electronic cash to the white paper

KEY TAKEAWAY

Bitcoin is less an invention than an **assembly**. The eight references in the 2008 white paper come close to being its parts list: the hash tree of 1980, three time-stamping papers from 1991, 1993, and 1997, a 1999 design for a time-stamping service, Hashcash (announced 1997, paper 2002), b-money of 1998, and a textbook of probability theory. One part the white paper does not cite is public-key cryptography from 1976, yet the white paper's "chain of digital signatures" stands on it. For each part it is clear **what it solved and what it did not**, and each unsolved problem called for the next part. Chaum's electronic cash gained anonymity but left the bank in place. Haber and Stornetta's chain fixed the order but left open "who writes the chain?" Hashcash put a price on computation but was not money. b-money and bit gold sketched money but had no details of consensus. The white paper filled those gaps in one system, and the history after it (Ethereum, The DAO, automated markets, the collapse of two exchanges, the definitional provisions of various jurisdictions) is left to the later chapters of this book. What a lawyer should take away is not the dates but the **lineage**. If you know where each part came from, then when the single word "blockchain" points at ten different claims at once, you can tell which one is meant.

Learning objectives

1. Say what electronic cash, time-stamping, Hashcash, b-money, and bit gold each solved and failed to solve.
2. Trace, following the white paper's references, how the 2008 white paper assembled those parts.
3. Know which chapter of this book covers each milestone after the white paper (Ethereum, The DAO, automated markets, the forks, the two exchanges, the definitional provisions).
4. Know the rules for dates (conference year and publication year, posting date and displayed date) and read the timeline with them.

What a reader of the [Basic] paragraphs alone takes away. Bitcoin is less an invention than an **assembly**, and each of the assembled parts was already in the academic literature.

1. Timeline: two rules before reading

The table below contains only milestones whose **originals this book opened and checked**. Dates follow two rules. First, for a conference paper the conference year and the publication year can differ. Chaum's paper was presented at "Crypto 82" and came out as a book in 1983, and Merkle's tree signature was presented at "CRYPTO '87" and came out in 1988. When this book names the conference it uses the conference year, and when it gives a bibliographic citation it uses the publication year. Second, for a web document the posting date and the date displayed on screen can differ. Nick Szabo's bit gold post was posted on December 29, 2005, and the screen now shows December 27, 2008 (the footnote gives the method of checking). Where a date could not be settled, it is not asserted; that fact is recorded instead. The static version of this table is Appendix D, and the clickable timeline, which takes you to the relevant chapter when you click an event, is interactive figure W11. Items from 2024 on (spot ETFs, stablecoin legislation, proposals for a quantum transition) go stale quickly, so they sit in Appendix A rather than in the main text (the updating rule in the notes to the reader in Chapter 3).

YEAR	MILESTONE	WHAT IT SOLVED	WHAT IT LEFT OPEN	CHAPTER
1976	Diffie and Hellman, public-key cryptography	Signatures and secret communication without sharing a key	Who the "signer" is, and whether signed money has already been spent, are questions outside the signature	5
1980 / 1987	Merkle, hash tree	Vouching for many items with a single root	— (a component)	6
1982	Lamport, Shostak, and Pease, the Byzantine generals	Conditions for agreement in a group that includes traitors	Presupposes a closed group whose participants are known	1, 8
1982 / 1983	Chaum, electronic cash with blind signatures	Anonymous electronic notes: the bank does not know to whom it issued them	The bank does the double-spending check	1, 2
1991 / 1993	Haber and Stornetta (and Bayer), time-stamp chains and trees	Fixing the order of documents with a hash chain, compressing it with a tree	Who watches the service that writes the chain?	4, 6, 7
1997	Back, Hashcash	Putting a price on computation that is verified with a single hash	A stamp, not money: no transfer or storage of value	8
1998	Dai, b-money	A sketch of money created by computation, with everyone keeping the ledger	Whose word counts when the ledgers diverge	1, 8
2005	Szabo, bit gold	A sketch linking proof of work to time-stamping and a registry of title	How to "distribute" the registry and the time-stamping	8

YEAR	MILESTONE	WHAT IT SOLVED	WHAT IT LEFT OPEN	CHAPTER
2008-10-31	Nakamoto, white paper released	The parts above in one system: public, a single history, a majority by computation	Probabilistic finality, cost, governance	1, 7, 8, 9
2009-01-03	Genesis block	The system actually starts running	—	12
2014 (draft 2013)	Buterin, Ethereum white paper	Running arbitrary programs on the ledger	The cost of programs that do not halt (gas), defects in code	13
2014-02-28 · 2014-04-24	Mt.Gox: petition for civil rehabilitation · bankruptcy proceedings commenced	—	Mismatch between the exchange's books and the chain	26
2016	The DAO: sale, attack, hard fork	—	The relation between code and governance decisions	14, 22, 26
2017-07-25	SEC, report on The DAO	Finding that securities law applies to the sale of tokens	Which tokens are securities	26, 27
2019-03-15 · 2020-06-11	Tokyo District Court (東京地方裁判所) and Tokyo High Court (東京高等裁判所), criminal judgments against Mt.Gox's representative director	—	Criminal assessment of an exchange's manipulation of its books	26
2020-03	Uniswap v2 white paper	An automated market without an order book	Who the addressee of the rules is in a market without an operator	15
2021	London fork (EIP-1559) · Taproot	Redesign of the fee market · addition of a signature scheme	Public-key exposure comes back (Chapter 29)	21, 5, 29
2022-05 · 2022-09 · 2022-11	Terra collapse · Ethereum's move to proof of stake · FTX bankruptcy filing	—	Reflexivity of the design · consensus based on collateral · misuse of customer assets	24, 26, 10
2023-02-03	Court of Appeal of England and Wales, *Tulip Trading*	—	Developers' fiduciary duties	27, 28
2023-06-09	MiCA published in the Official Journal	A single EU definition and regulation	Does the definition keep up with the technology?	27, 30



Figure 2-1. Clickable timeline, 1976–2023. The same 29 milestones as the table in Chapter 2, Section 1 and Appendix D. Select an item to go to the chapter that treats the event from its original text. Dates are given at the precision confirmed (year, month or day), and estimated dates are marked separately. There is no price axis. — In the HTML edition you can change the values here (HTML edition, Chapter 2, Section 1).

2. 1976: splitting the key in two

The 1976 paper by Whitfield Diffie and Martin Hellman announces a revolution in the first sentence of its introduction: "WE STAND TODAY on the brink of a revolution in cryptography." Low-cost digital hardware had made it possible to use cryptographic devices "in such commercial applications as remote cash dispensers and computer terminals," and such applications call for "new types of cryptographic systems which minimize the necessity of secure key distribution channels and supply **the equivalent of a written signature**."[1] "[T]he equivalent of a written signature" is the digital signature of Chapter 5, and, as Chapter 1 showed, the white paper defines a coin as "a chain of digital signatures." But a signature proves only "who approved." Whether a signature has been used twice is a matter outside the signature, and among the materials this book reads, the earliest to treat that outside matter as a problem of electronic cash is Chaum.

3. 1982 / 1983: Chaum's electronic cash, anonymity gained and the bank left in place

At the opening of his 1982 conference paper, Chaum writes that electronic payments "may have a substantial impact on personal privacy as well as on the nature and extent of criminal use of payments," and proposes a system that addresses both concerns together: third parties should be unable to determine the payee, time, or amount of payments; individuals should nonetheless be able to provide proof of payment or, in exceptional circumstances, to determine the identity of the payee; and it should be possible to stop the use of payment media reported stolen.[2] The tool is the **blind signature**. Chaum compares it to an envelope lined with carbon paper: "Writing a signature on the outside of such an envelope leaves a carbon copy of the signature on a slip of paper within the envelope." The bank signs the outside of the envelope and the payer opens the envelope and takes out the signed note, so the bank cannot link a note it signed to the person from whom it later comes back.[3] As Chapter 1 showed, to prevent double spending that system has the bank keep **a comprehensive list of the notes it has cleared** and stop when the same note arrives twice. Anonymity was obtained by cryptography; the policing of double spending was still left to **the ledger of one institution**. When the Ethereum white paper of 2014 wrote of this generation that "The anonymous e-cash protocols of the 1980s and the 1990s, mostly reliant on a cryptographic primitive known as Chaumian blinding, provided a currency with a high degree of privacy, but the protocols largely failed to gain traction because of their reliance on a centralized intermediary," it was pointing at that spot.[4]

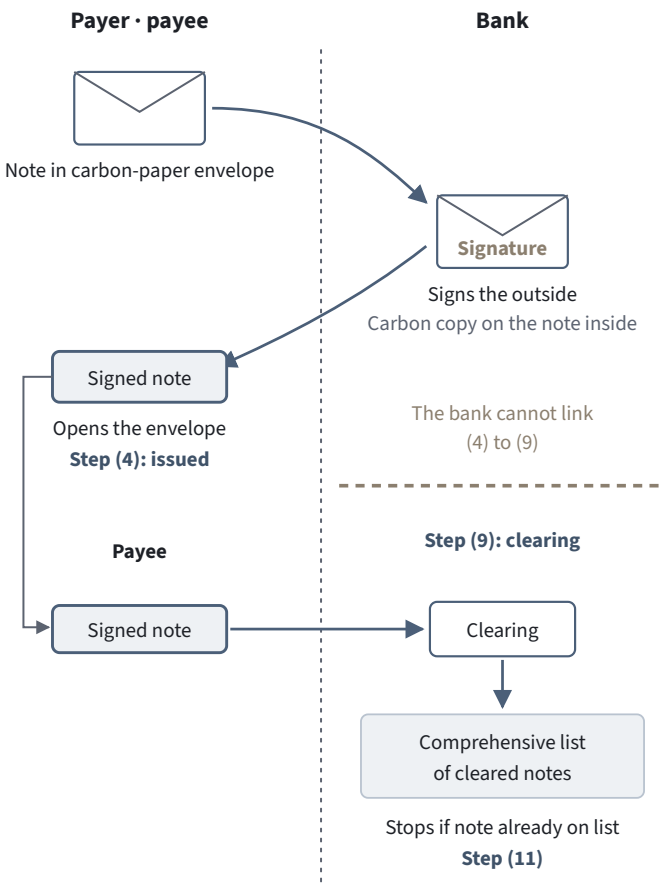


Figure 2-2. The flow of a blind signature, using the analogy of the carbon-paper envelope. Because the bank signs only the outside of the envelope, it cannot link the note issued in step (4) to the note presented for clearing in step (9), and double spending is prevented only by checking against the comprehensive list in step (11), seen in Chapter 1. The step numbers are those of Chaum's paper; the other steps are drawn without numbers.

4. 1980 / 1987: Merkle's trees; 1982: the Byzantine generals

Both parts came from places that had nothing to do with money. Ralph Merkle's hash tree is a device for vouching for many items with a single hash. The white paper cites three sources when it uses the Merkle tree ([7][2][5]), and Merkle's own among them is his paper at the 1980 IEEE Symposium on Security and Privacy.[5] That paper describes "tree authentication" as a way to confirm one entry in a list of public keys without receiving the whole list: by splitting the list in half and hashing, over and over ("divide and conquer"), one entry can be confirmed with only a single root and $\log_2 n$ intermediate values. The detailed discussion, however, is deferred to another manuscript of his, "A certified digital signature."[6] When the 1993 paper by Bayer, Haber, and Stornetta introduces trees, it too notes that "Such hash trees were previously used by Merkle [Merk 80] for a different purpose, to produce authentication certificates for a directory of public enciphering keys."[7] Merkle's 1987 conference paper (published 1988) applies the same idea to signatures and builds a system in which "the signature size increases logarithmically as a function of the number of messages signed." This is the root of the "proof length $\log_2 n$" shown in Chapter 6.[8] The Byzantine Generals Problem (1982) appeared in Chapter 1. It formalized, through the metaphor of generals, the conditions under which a group containing traitors can reach agreement (deferring the rigorous proofs to the same authors' 1980 paper), and showed that with signatures the problem can be solved whatever the number of traitors. But the result was for a group whose generals are **known**, and the road to an open network, where anyone can create fake generals, was laid only in 2008.

5. 1991 / 1993: Haber and Stornetta's chain, which fixed the order, but

This is the paper Chapter 7 read closely. In the abstract of their 1991 paper, Stuart Haber and W. Scott Stornetta set out the problem this way: "The problem is to time-stamp the **data**, not the medium. We propose computationally practical procedures ... so that it is infeasible for a user either to back-date or to forward-date his document, even with the collusion of a time-stamping service."[9] The first example in the introduction is **intellectual property**: "it is sometimes crucial to verify the date an inventor first put in writing a patentable idea, in order to establish its precedence over competing claims."[10] The solution was **linking**, in which each certificate contains the hash of the certificate before it, and the 1993 paper bundled that chain into trees, adding a way "to achieve an exponential increase in the publicity obtained for each time-stamping event, while reducing the storage and the computation required in order to validate a given certificate."[11] And the first sentence of that paper's introduction, "Causality fixes events in time," was the key to Chapter 7. What this chain left open is "who writes the chain?" The 1991 paper wrote that its second solution (several randomly chosen witnesses sign) "could be implemented without the need for a centralized TSS at all,"[12] but those witnesses were still drawn from within a known "client pool." How do you decide who "writes" in an open network? Computation fills that blank.

6. 1997: Hashcash, putting a price on computation

On March 28, 1997, Adam Back announced on the cypherpunks mailing list "A partial hash collision based postage scheme": "The idea of using partial hashes is that they can be made arbitrarily expensive to compute (by choosing the desired number of bits of collision), and yet can be **verified instantly**."[13] It was a **stamp**, meant to curb spam e-mail and the abuse of anonymous remailers. The sender burns CPU looking for an input whose hash begins with a run of 0s, and the recipient checks it with a single hash. The 2002 paper that set the scheme out in full classified this cost function as a "non-interactive, publicly auditable, trapdoor-free cost function with unbounded probabilistic cost,"[14] and §4 of the white paper takes it over as is: "a proof-of-work system similar to Adam Back's Hashcash." Chapter 8 deals with that lottery. What Hashcash left open is clear. It was a stamp, not money. Once used, it is done, and value is neither transferred nor stored. The 2002 paper itself put at the end of its list of applications "a minting mechanism for Wei Dai's b-money electronic cash proposal," which means that the next part, one that would turn the stamp into money, was already out.[15]

7. 1998: b-money, money sketched and consensus left blank

Dai's b-money is a short web document with no date on it. Reference [1] of the white paper gives "1998," and Back's 2002 paper gives "Nov 1998." This book uses only 1998 and does not give a month or day. The first protocol appeared in Chapter 1. What is interesting is the **creation** of money: "Anyone can create money by broadcasting the solution to a previously unsolved computational problem. The only conditions are that it must be easy to determine how much computing effort it took to solve the problem and the solution must otherwise have no value, either practical or intellectual."[16] In effect, Hashcash's stamp became the issuance of money. And Dai himself points to the most problematic part of his proposal: "This part of the protocol requires that all of the account keepers decide and agree on the cost of particular computations. Unfortunately because computing technology tends to advance rapidly and not always publicly, this information may be unavailable, inaccurate, or outdated …."[17] He patches this with auctions in an appendix, but "whose word counts when the ledgers diverge?" and "who are the account keepers?" are still blank. The one thing the white paper took from b-money is that "transactions must be publicly announced," and it attached [1] at exactly that spot.

8. 2005: bit gold, a registry of title and time-stamping

Szabo's bit gold is a blog post of December 29, 2005. Its statement of the problem is different from the start: "our money currently depends on trust in a third party for its value. As many inflationary and hyperinflationary episodes during the 20th century demonstrated, this is not an ideal state of affairs." Szabo's view is that precious metals have "an unforgeable scarcity due to the costliness of their creation" and so gave money a value largely independent of third parties, but "you can't pay online with metal." So he sketches "a protocol whereby unforgeably costly bits could be created online with minimal dependence on trusted third parties, and then securely stored, transferred, and assayed with similar minimal trust."[18] Three of the seven steps foreshadow the system that came later: "(3) The proof of work is securely timestamped. This should work in a distributed fashion, with several different timestamp services so that no particular timestamp service need be substantially relied on. (4) Alice adds … to a distributed property title registry for bit gold. Here, too, no single server is substantially relied on to properly operate the registry. (5) The last-created string of bit gold provides the challenge bits for the next-created string."[19] Proof of work (Hashcash), the time-stamp chain (Haber and Stornetta), and a **registry of title** came into one picture, along with a chain in which the last string is the input to the next. What is blank is how the "distributed fashion" of (3) and (4) actually works, and Szabo himself wrote that "The main limits to the security of the scheme are how well trust can be distributed in steps (3) and (4) …."[20] The same post introduces RPOW (reusable proofs of work), a variant implemented by Hal Finney, of which the Ethereum white paper says that it "once again fell short of the ideal by relying on trusted computing as a backend."[21]

9. 2008: the white paper, an assembly

At 14:10 (U.S. Eastern time) on Friday, October 31, 2008, a message was posted to the cryptography mailing list: "I've been working on a new electronic cash system that's fully peer-to-peer, with no trusted third party." A list of five main properties follows: "The main properties: Double-spending is prevented with a peer-to-peer network. No mint or other trusted parties. Participants can be anonymous. New coins are made from Hashcash style proof-of-work. The proof-of-work for new coin generation also powers the network to prevent double-spending."[22] Of the five lines, the first four are the names of parts this chapter has passed through: do away with the mint that Chaum left in place, take over Dai's "publicly announced transactions," and make coins with Back's Hashcash. What is new is the fifth line. It makes the computation that creates money and the computation that fixes the order **the same computation**, and so answers at once Chapter 1's "who writes the ledger?" and the "who are the account keepers?" that Dai left blank. The one who writes is the one who did the computation, and the one who did the computation receives coins. The eight references of the white paper are the assembly drawing: [1] Dai's b-money (public announcement), [2]–[5] the time-stamping literature (Haber and Stornetta 1991 and 1997, Bayer et al. 1993, Massias et al. 1999), [6] Back's Hashcash, [7] Merkle 1980, and [8] Feller's probability textbook (the gambler's ruin of Chapter 9).[23] The system actually started running on January 3, 2009. Chapter 12 took apart the 80 bytes of that first block by hand. The time recorded in it, 1231006505, is 18:15:05 (UTC) on January 3, 2009, and the sentence "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks" sits in the coinbase.[24]

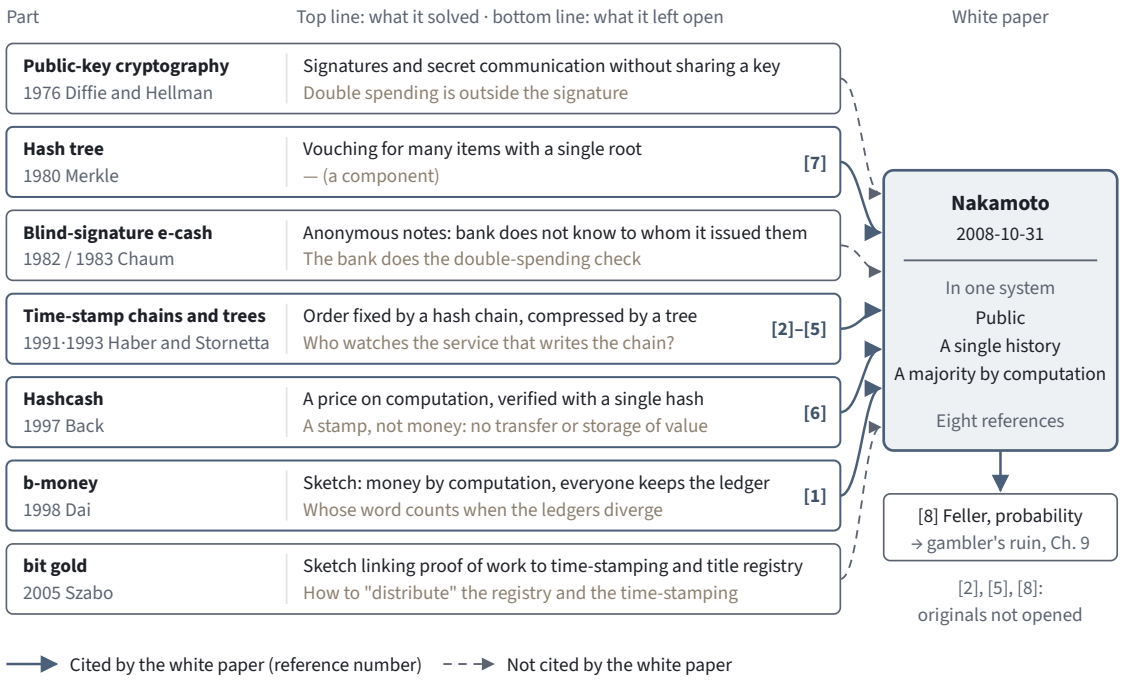


Figure 2-3. The parts that came together in the white paper, and what each solved or left open (the table in Section 1). The numbers beside the solid lines are the white paper's reference numbers; dashed lines mark what the white paper does not cite. This book did not open the originals of [2], [5], and [8].

10. 2013 / 2014: Ethereum, programs on the ledger

Five years after the white paper, Vitalik Buterin's Ethereum white paper reads Bitcoin as two inventions: "The first is the 'bitcoin', a decentralized peer-to-peer online currency that maintains a value without any backing, intrinsic value or central issuer. ... However, there is also another, equally important, part to Satoshi's grand experiment: the concept of a proof of work-based blockchain to allow for public agreement on the order of transactions." It then turns attention to "how the blockchain concept can be used for more than just money."[25] The white paper's "History" section sets down the very lineage this chapter has traced: Chaum, Dai, Finney, Byzantine consensus and the Sybil attack, and the assessment that "The innovation provided by Satoshi is the idea of combining a very simple decentralized consensus protocol ... with proof of work ...."[26] What Ethereum added is "a blockchain with a built-in fully fledged Turing-complete programming language," and its consequences (gas, which puts a price on programs that do not halt; ten-line "contracts"; defects in that code) are taken up in Chapters 13 and 26. The year is given as "2014 (draft 2013)." The cover of the official PDF says "(2014)," the text has the phrase "as of April 2014," and the first web-archive capture of the draft is dated December 19, 2013.[27]

11. 2014 to 2023: milestones after the white paper, and the later chapters of this book

The later chapters of this book each take up the subsequent history from the originals. Here we note only which chapter covers what.

- **February 2014, Mt.Gox.** The exchange filed for civil rehabilitation, and the mismatch between its off-chain books and the chain was where the problem lay. Chapter 26 reads the case through to the criminal judgments at first instance (2019) and on appeal (2020).
- **2016, The DAO.** The token sale (April 30 to May 28), the movement of assets by exploiting a defect in the code (June 17), and the hard fork (July 20). The report of the U.S. Securities and Exchange Commission of July 25, 2017, found that "DAO Tokens are securities," did not bring an enforcement action, and wrote that "The automation of certain functions through this technology, 'smart contracts,' or computer code, does not remove conduct from the purview of the U.S. federal securities laws."[28] This is how this book records the "ICO phase": not by size and price but **by the judgments of institutions**. Chapter 14, Chapter 22, Chapter 26, Chapter 27.
- **March 2020, the Uniswap v2 white paper.** The white paper calls v1 "an automated liquidity protocol based on a 'constant product formula'" and describes v2 as "a new implementation based on the same formula": a market without an order book. Chapter 15 looks at the formula, and Chapter 17 at the ordering problem on top of it.[29]
- **2021.** Ethereum's London fork (redesign of the fee market, Chapter 21) and the activation of Taproot on Bitcoin (Schnorr signatures, Chapter 5 and Appendix A-8). That Taproot brought public-key exposure back comes up again in Chapter 29.
- **2022.** The collapse of Terra in May (Chapter 24 and Chapter 26), Ethereum's move to proof

of stake in September (Chapter 10 and Appendix A-7), and FTX's bankruptcy filing in November (Chapter 26).

- **February 3, 2023, the Court of Appeal of England and Wales, *Tulip Trading*.** The judgment that treated whether Bitcoin developers owe fiduciary duties to owners as a "serious issue to be tried." Chapter 27 deals with it.
- **June 9, 2023, MiCA published in the Official Journal.** The EU's single definition, "crypto-asset," and its regulation. It applies from December 30, 2024. Chapter 30 sets that definitional provision beside those of four other jurisdictions.

Developments from 2024 on (spot exchange-traded funds, legislation in the United States, the United Kingdom, and Japan, proposals for Bitcoin's quantum transition, NIST's post-quantum standards) are placed in the Appendix A trend notes, with reference dates. They are not yet "history" but "the current state of affairs."

12. Summary

Each part solved something and left something open. Public-key cryptography in 1976 gave signatures, but double spending was outside the signature. Electronic cash in 1983 gave anonymity but left the bank. The chains and trees of 1991 and 1993 fixed the order but left the question of who writes. Hashcash in 1997 put a price on computation but was not money. b-money in 1998 and bit gold in 2005 sketched money and a registry but left the details of the "distributed fashion." The white paper of 2008 filled those gaps at once by making the computation that creates money and the computation that fixes the order the same computation, and its assembly drawing is its eight references. The later chapters of this book read the subsequent history from the originals. A lawyer should take one thing from this chapter: behind the word "blockchain" lie ten different inventions from different periods, one over another, and you have to read carefully to tell **which one** a given case or provision points to. The next chapter, Chapter 3, is on how to read this book.

Sources for this chapter

- Whitfield Diffie & Martin E. Hellman, "New Directions in Cryptography," *IEEE Trans. Inform. Theory* IT-22(6): 644–654 (1976) — p. 644.
- David Chaum, "Blind Signatures for Untraceable Payments," *Advances in Cryptology: Proceedings of Crypto 82*, pp. 199–203 (Springer, 1983) — pp. 199–203 (scan, read from the images).
- Ralph C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," *CRYPTO '87*, LNCS 293, pp. 369–378 (Springer, 1988) — p. 369.
- Ralph C. Merkle, "Protocols for Public Key Cryptosystems," *Proc. 1980 IEEE Symposium on Security and Privacy*, pp. 122–134 (DOI 10.1109/SP.1980.10006) — pp. 126, 127, 133. Copy from the author's site (a scan; the text layer is OCR, and quotations were checked against the page images).
- Leslie Lamport, Robert Shostak & Marshall Pease, "The Byzantine Generals Problem," *ACM TOPLAS* 4(3): 382–401 (1982).

- Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *J. Cryptology* 3(2): 99–111 (1991) — pp. 99, 110.
- Dave Bayer, Stuart Haber & W. Scott Stornetta, "Improving the Efficiency and Reliability of Digital Time-Stamping," *Sequences II*, pp. 329–334 (Springer, 1993) — pp. 1–2 of the authors' typeset version.
- Adam Back, "[ANNOUNCE] hash cash postage implementation," cypherpunks, 1997-03-28 (hashcash.org/papers/announce.txt); Adam Back, "Hashcash — A Denial of Service Counter-Measure," 2002-08-01 — pp. 1, 3, 7, 9, 10.
- Wei Dai, "b-money," http://www.weidai.com/bmoney.txt (1998). Accessed 2026-10-25.
- Nick Szabo, "Bit gold," *Unenumerated*, posted 2005-12-29 (settled from the Wayback snapshot of 2006-03-29; date currently displayed 2008-12-27). Accessed 2026-10-25.
- Satoshi Nakamoto, "Bitcoin P2P e-cash paper," cryptography@metzdowd.com, 2008-10-31; Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008) — §2, §4, §7, References.
- This book's bundled data, `data/genesis_block.meta.json` (reference date 2026-10-25) — the genesis nTime and the coinbase sentence.
- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform" (official ethereum.org PDF, 2014; draft 2013) — pp. 1, 4.
- SEC, Release No. 81207 (2017-07-25) — pp. 1–2.
- Hayden Adams, Noah Zinsmeister & Dan Robinson, "Uniswap v2 Core" (March 2020) — p. 1.
- The primary sources for the other milestones (Mt.Gox, the course of The DAO, the London fork, Taproot, Terra, the Merge, FTX, MiCA) are those of the chapters responsible for each (Chapters 5, 10, 21, 24, 26, 30) and of Appendix A. This chapter has no code or notebook (by design). The ICO, ETFs, and stablecoin legislation are recorded only as "institutional change," without size or price. Items from 2024 on are in Appendix A.

Notes

1. Whitfield Diffie & Martin E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory* IT-22(6): 644–654 (November 1976), p. 644: "WE STAND TODAY on the brink of a revolution in cryptography. The development of cheap digital hardware has freed it from the design limitations of mechanical computing and brought the cost of high grade cryptographic devices down to where they can be used in such commercial applications as remote cash dispensers and computer terminals. In turn, such applications create a need for new types of cryptographic systems which minimize the necessity of secure key distribution channels and supply the equivalent of a written signature." Emphasis added.
2. David Chaum, "Blind Signatures for Untraceable Payments," in D. Chaum, R. L. Rivest & A. T. Sherman (eds.), *Advances in Cryptology: Proceedings of Crypto 82*, pp. 199–203 (Springer, 1983), pp. 199–200: "The ultimate structure of the new electronic payments system may have a substantial impact on personal privacy as well as on the nature and extent of criminal use of payments." / "(1) Inability of third parties to determine payee, time or amount of payments made by an individual. (2) Ability of individuals to provide proof of payment, or to determine the identity of the payee under exceptional circumstances. (3) Ability to stop use of payments media reported stolen." (IACR scan with no text layer, read directly from the page images). Conference 1982, publication 1983.
3. Chaum (1983), p. 200, "Basic Idea": "The paper analog of a blind signature can be implemented with carbon paper lined envelopes. Writing a signature on the outside of such an envelope leaves a carbon copy of the

signature on a slip of paper within the envelope." p. 203: "Notice that by the blind signature property above, when the bank receives a note to be cleared from the payee in step (9) the bank does not know which payer the note was originally issued to in step (4)."

4. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform" (official version posted on ethereum.org, 2014; draft 2013), p. 4, "History."

5. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), p. 4, §7: "transactions are hashed in a Merkle Tree [7][2][5], with only the root included in the block's hash." Reference [7]: "R.C. Merkle, "Protocols for public key cryptosystems," In Proc. 1980 Symposium on Security and Privacy, IEEE Computer Society, pages 122-133, April 1980." The original runs to pp. 122–134 (p. 134 holds the end of the references and Figure 1).

6. Ralph C. Merkle, "Protocols for Public Key Cryptosystems," *Proc. 1980 IEEE Symposium on Security and Privacy*, pp. 122–134 (DOI 10.1109/SP.1980.10006), p. 126: "Fortunately, it is possible to selectively authenticate individual entries in the public file without having to know the whole public file by using Merkle's "tree authentication," [13]. The essence of tree authentication is to authenticate the entire public file by "divide and conquer."" Same page: "By applying this concept recursively, A can confirm Y_B in the public file knowing only R, $\log_2 n$ intermediate values, and Y_B itself." p. 127: "For a more detailed discussion the reader is referred to [13]." p. 133, reference [13]: "Merkle, R.C. A certified digital signature, to appear, CACM." A paper with the same title was presented at CRYPTO '89 and published in 1990: R. C. Merkle, "A certified digital signature," in G. Brassard (ed.), *Advances in Cryptology — CRYPTO '89*, LNCS 435, pp. 218–238 (Springer, 1990). This book confirmed that citation from reference [244] of Babbush et al. (2026) and from the citation of the same volume in the references of Haber and Stornetta (1991) (Appendix E), and did not open the original. This book therefore does not treat the 1980 paper as the place where the tree was first presented. Copy from the author's site (a scan; the text layer is OCR, and quotations were checked against the page images).

7. Dave Bayer, Stuart Haber & W. Scott Stornetta, "Improving the Efficiency and Reliability of Digital Time-Stamping," in R. Capocelli, A. De Santis & U. Vaccaro (eds.), *Sequences II: Methods in Communication, Security, and Computer Science*, pp. 329–334 (Springer, 1993), p. 2 of the authors' typeset version (§1): "We do this by building trees in place of the linked list of the linking solution, thus achieving an exponential increase in the number of observers. Such hash trees were previously used by Merkle [Merk 80] for a different purpose, to produce authentication certificates for a directory of public enciphering keys." (authors' typeset version, marked "March 1992," pp. 1–6). The publication year is 1993 (Crossref, OpenAlex, Springer, and white paper reference [4] agree), and no publication record supports 1992.

8. Ralph C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in C. Pomerance (ed.), *Advances in Cryptology — CRYPTO '87*, LNCS 293, pp. 369–378 (Springer, 1988), p. 369, abstract: "The signature system can sign an unlimited number of messages, and the signature size increases logarithmically as a function of the number of messages signed." Conference 1987, publication 1988.

9. Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *Journal of Cryptology* 3(2): 99–111 (1991), p. 99, abstract: "The problem is to time-stamp the data, not the medium. We propose computationally practical procedures for digital time-stamping of such documents so that it is infeasible for a user either to back-date or to forward-date his document, even with the collusion of a time-stamping service." Received 1990-08-19, revised 1990-10-26 (p. 99, footnote 1). Emphasis added.

10. Haber & Stornetta (1991), p. 99, §1: "For example, in intellectual property matters, it is sometimes crucial to verify the date an inventor first put in writing a patentable idea, in order to establish its precedence over competing claims."

11. Bayer, Haber & Stornetta (1993), p. 1 of the authors' typeset version, abstract: "By using trees, we show how to achieve an exponential increase in the publicity obtained for each time-stamping event, while reducing the storage and the computation required in order to validate a given certificate." First sentence of the introduction (p. 2): "Causality fixes events in time."

12. Haber & Stornetta (1991), p. 110, §8, Summary: "In the second solution, several members of the client pool must time-stamp the hash. The members are chosen by means of a pseudorandom generator that uses the hash of the document itself as seed. This makes it infeasible to choose deliberately which clients should and should not time-stamp a given hash. The second method could be implemented without the need for a centralized TSS at all."

13. Adam Back, "[ANNOUNCE] hash cash postage implementation," cypherpunks@toad.com, Fri, 28 Mar 1997 16:52:26 GMT (copy on the author's site, http://www.hashcash.org/papers/announce.txt, accessed 2026-10-25): "A partial hash collision based postage scheme / I've been talking about a partial hash collision based postage scheme on the crypto lists for the last few days. The idea of using partial hashes is that they can be made arbitrarily expensive to compute (by choosing the desired number of bits of collision), and yet can be verified instantly." **Date discrepancy:** the 2002 paper gives the first proposal as "May 1997" in both its abstract and its body, and gives reference [1] as "Adam Back. Hashcash, May 1997. Published at http://www.cypherspace.org/hashcash/", two months off from the 1997-03-28 of the announcement e-mail. This book writes "the cypherpunks announcement of March 1997" and records the discrepancy here. The copy of the announcement is the one on the author's site and was not checked against an independent archive. Emphasis added.

14. Adam Back, "Hashcash — A Denial of Service Counter-Measure," technical report, August 1, 2002, p. 3, §3: "Hashcash is a non-interactive, publicly auditable, trapdoor-free cost function with unbounded probabilistic cost." Abstract, p. 1: "Hashcash was originally proposed as a mechanism to throttle systematic abuse of unmetered internet resources such as email, and anonymous remailers in May 1997." Same page: "At the time of publication of [1] the author was not aware of the prior work by Dwork and Naor in [2] who proposed a CPU pricing function for the application of combatting junk email." This book did not open the original of Dwork and Naor (1992) and reports it only through Back's account.

15. Back (2002), p. 7, §7, Applications: "hashcash as a minting mechanism for Wei Dai's b-money electronic cash proposal, an electronic cash scheme without a banking interface [19]". Reference [19]: "Wei Dai. b-money. Published at http://www.eskimo.com/~weidai/bmoney.txt, Nov 1998." White paper §4 (p. 3): "we will need to use a proof-of-work system similar to Adam Back's Hashcash [6]".

16. Wei Dai, "b-money," http://www.weidai.com/bmoney.txt (1998; accessed and checked verbatim on 2026-10-25): "1. The creation of money. Anyone can create money by broadcasting the solution to a previously unsolved computational problem. The only conditions are that it must be easy to determine how much computing effort it took to solve the problem and the solution must otherwise have no value, either practical or intellectual."

17. Same document, Appendix A: "One of the more problematic parts in the b-money protocol is money creation. This part of the protocol requires that all of the account keepers decide and agree on the cost of particular computations. Unfortunately because computing technology tends to advance rapidly and not always publicly, this information may be unavailable, inaccurate, or outdated, all of which would cause serious problems for the protocol." White paper p. 2, §2: "To accomplish this without a trusted party, transactions must be publicly announced [1] ...".

18. Nick Szabo, "Bit gold," *Unenumerated* (blog), https://unenumerated.blogspot.com/2005/12/bit-gold.html (accessed and checked verbatim on 2026-10-25): "The problem, in a nutshell, is that our money currently depends on trust in a third party for its value. As many inflationary and hyperinflationary episodes during the 20th century demonstrated, this is not an ideal state of affairs." / "Precious metals and collectibles have an unforgeable scarcity due to the costliness of their creation." / "What's worse, you can't pay online with metal. Thus, it would be very nice if there were a protocol whereby unforgeably costly bits could be created online with minimal dependence on trusted third parties, and then securely stored, transferred, and assayed with similar minimal trust." **Posting date:** the URL path is 2005/12 and the date now displayed is "Saturday, December 27, 2008," but the date header of the original Wayback snapshot (web.archive.org/web/20060329122942id_/.../2005/12/bit-gold.html) is "Thursday, December 29, 2005," and the snapshot carries the full text. This book uses the displayed date of the preserved copy, 2005-12-29, as the posting date. How the displayed date of the current page came to be 2008-12-27 was not established.

19. Same post: "(3) The proof of work is securely timestamped. This should work in a distributed fashion, with several different timestamp services so that no particular timestamp service need be substantially relied on. (4) Alice adds the challenge string and the timestamped proof of work string to a distributed property title registry for bit gold. Here, too, no single server is substantially relied on to properly operate the registry. (5) The last-created string of bit gold provides the challenge bits for the next-created string." And: "Note that Alice's control over her bit gold does not depend on her sole possession of the bits, but rather on her lead position in the unforgeable chain of title (chain of digital signatures) in the title registry."

20. Same post: "The main limits to the security of the scheme are how well trust can be distributed in steps (3) and (4), and the problem of machine architecture which will be discussed below."

21. Buterin (2014), p. 4, "History": "In 2005, Hal Finney introduced a concept of "reusable proofs of work", a system which uses ideas from b-money together with Adam Back's computationally difficult Hashcash puzzles to create a concept for a cryptocurrency, but once again fell short of the ideal by relying on trusted computing as a backend." Szabo's post: "Hal Finney has implemented a variant of bit gold called RPOW (Reusable Proofs of Work). This relies on publishing the computer code for the "mint," which runs on a remote tamper-evident computer."

22. Satoshi Nakamoto, "Bitcoin P2P e-cash paper," cryptography@metzdowd.com, Fri Oct 31 14:10:00 EDT 2008, https://www.metzdowd.com/pipermail/cryptography/2008-October/014810.html (accessed and checked verbatim on 2026-10-25). The text matches the Nakamoto Institute copy.

23. Nakamoto (2008), p. 9, References (abbreviated transcription: URLs, subtitles, and the full names of conferences are shortened). [1] W. Dai, "b-money," 1998; [2] H. Massias, X.S. Avila & J.-J. Quisquater, "Design of a secure timestamping service with minimal trust requirements," 20th Symposium on Information Theory in the Benelux, May 1999; [3] S. Haber & W.S. Stornetta, "How to time-stamp a digital document," *Journal of Cryptology* 3(2): 99-111, 1991; [4] D. Bayer, S. Haber & W.S. Stornetta, "Improving the efficiency and reliability of digital time-stamping," *Sequences II*, pages 329-334, 1993; [5] S. Haber & W.S. Stornetta, "Secure names for bit-strings," 4th ACM CCS, pages 28-35, April 1997; [6] A. Back, "Hashcash - a denial of service counter-measure," 2002; [7] R.C. Merkle, "Protocols for public key cryptosystems," 1980 Symposium on Security and Privacy, pages 122-133, April 1980; [8] W. Feller, "An introduction to probability theory and its applications," 1957. Of these, this book did not open the originals of [2], [5], and [8].

24. This book's bundled data, `data/genesis_block.meta.json` and `data/README.md` (reference date 2026-10-25): nTime 1231006505 and the 69-byte ASCII sentence in the coinbase input script, 'The Times 03/Jan/2009 Chancellor on brink of second bailout for banks', which match Bitcoin Core v31.1 `src/kernel/chainparams.cpp` (commit 9be056a8), L70 and L134–L137. Chapter 12, NB12-1 to NB12-3. The time conversion is the output of NB12-1 in Chapter 12.

25. Buterin (2014), p. 1: "When Satoshi Nakamoto first set the Bitcoin blockchain into motion in January 2009, he was simultaneously introducing two radical and untested concepts. The first is the "bitcoin", a decentralized peer-to-peer online currency that maintains a value without any backing, intrinsic value or central issuer. ... However, there is also another, equally important, part to Satoshi's grand experiment: the concept of a proof of work-based blockchain to allow for public agreement on the order of transactions." / "And now, attention is rapidly starting to shift toward this second part of Bitcoin's technology, and how the blockchain concept can be used for more than just money." This book reproduces the statement that bitcoin "maintains a value" only as the white paper author's wording and does not deal with price.

26. Buterin (2014), p. 4: "The innovation provided by Satoshi is the idea of combining a very simple decentralized consensus protocol, based on nodes combining transactions into a "block" every ten minutes creating an ever-growing blockchain, with proof of work as a mechanism through which nodes gain the right to participate in the system." p. 1: "What Ethereum intends to provide is a blockchain with a built-in fully fledged Turing-complete programming language that can be used to create "contracts" that can be used to encode arbitrary state transition functions".

27. Cover of the official ethereum.org PDF: "By Vitalik Buterin (2014)"; text: "as of April 2014"; body text of ethereum.org/whitepaper/: "This whitepaper was published in 2014", meta description: "published in 2013"; first Wayback CDX capture of vitalik.ca/ethereum.html: 2013-12-19. The text of the draft was not opened.

28. SEC, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Release No. 81207 (July 25, 2017), pp. 1–2: "Based on the investigation, and under the facts presented, the Commission has determined that DAO Tokens are securities under the Securities Act of 1933 ("Securities Act") and the Securities Exchange Act of 1934 ("Exchange Act")." / "The Commission has determined not to pursue an enforcement action in this matter based on the conduct and activities known to the Commission at this time." / "The automation of certain functions through this technology, "smart contracts," or computer code, does not remove conduct from the purview of the U.S. federal securities laws." Sale period: "From April 30, 2016 through May 28, 2016" (p. 2). The dates of the attack and the fork are from Chapter 26 (Ethereum Foundation announcements, 2016-06-17 and 07-20).

29. Hayden Adams, Noah Zinsmeister & Dan Robinson, "Uniswap v2 Core" (March 2020), p. 1: "Uniswap v1 is an on-chain system of smart contracts on the Ethereum blockchain, implementing an automated liquidity protocol based on a "constant product formula" [1]."

PART 0 · A MAP

Chapter 3. How to read this book: markers, boxes, interactive figures, notebooks, and three paths

NOTICE

One thing should be made clear first. **This book does not tell you to buy or sell anything.** It is a study book that explains, in the language of technology and economics, how blockchains work and how money and people move on them. So it contains no price outlook, it carries no price charts as a rule, and it does not say that any exchange, wallet or token is good. Names appear only where the book records historical facts that are already settled.

Every figure carries a reference date and a source. This field changes fast, so things may have changed by the time you read this. The code is a toy for seeing the principles with your own eyes and was not built to move real assets. Do not use it as it is.

The editor is a sitting judge. This book is the editor's personal study record and has nothing to do with the institution the editor belongs to. On cases that are not yet over and on bills pending in the National Assembly, I have written only the facts and not my own views. Nor is this book legal advice.

The full notice is in the colophon (F.4 of Appendix F).

KEY TAKEAWAY

This book is written so that a legal scholar can read it to the end with nothing but high-school mathematics. The devices that keep that promise are **three levels of marker** ([Basic], [Intermediate], [Advanced]) and **collapsible boxes**: skip every [Advanced] passage and the argument of the text does not break. The "A lawyer's question" boxes only ask questions and pass the answers on to Part 5, and the interactive figures and notebooks are where you move the formulas and code of the text with your own hands. Three paths (basic, technical, economics and law) are given by chapter number, every figure carries the reference date (2026-10-25), and all the code is for study and is not to be used with real assets. What to take from this chapter is **the promise itself: "you can skip [Advanced] and the book does not break."**

Learning objectives

1. Know what the three levels of marker, the collapsible boxes, the lawyer's question boxes, and the interactive figures and notebooks are for.
2. Choose your own path among the three and follow it by chapter number.
3. Know what the reference date for figures, the "as of 2026-10" mark and the "original to be checked" mark mean.
4. Check your own environment with NB00.

What a reader of the [Basic] paragraphs alone takes away. The promise itself: "you can skip [Advanced] and the book does not break."

1. Three levels of marker

[Intermediate] and [Advanced] paragraphs carry a marker at the head. A paragraph without a marker is [Basic].

MARKER	ASSUMES	CONTAINS	IF YOU SKIP IT
[Basic]	High-school mathematics	Intuitions, figures, analogies, lawyer's questions, summaries	Do not skip it: this is the text
[Intermediate]	Algebra, probability calculations, reading short code	Working out formulas, tables, Python code and its output	The conclusions are restated in [Basic], so the argument does not break
[Advanced]	Group operations on elliptic curves, Poisson processes and random walks, deriving equilibria, proofs	The inside of the mathematics, corrections and reproductions of the original papers	It is collapsed: you need not open it

There is one rule. **The argument must carry on through the [Basic] sentences before and after an [Advanced] box alone.** That is why each chapter states, in one line below the learning objectives, "What a reader of the [Basic] paragraphs alone takes away." Read those lines together and you have the skeleton of the whole book. Each formula is followed by one line reading the symbols and one numerical example. For instance, below the formula for the attack's success probability in Chapter 9 comes a number such as "0.0002428 at $q = 0.1$, $z = 6$." You can read that number without reading the formula.

2. Three boxes

Collapsible boxes. In the screen edition you open one by clicking its title line; in the PDF it is set off by a gray background. [Advanced] formulas and long code go here. If the title carries "[Advanced]," you may skip the box.

A lawyer's question. A box marked "A lawyer's question" (with a question mark in a circle before the title) notes **the place where a legal question arises** from the technical and economic facts of the chapter. It states the question and not the answer. Each box ends with

an arrow to a section of Part 5 (27.2, 27.3, or (a)–(g) of 28.1), and in that section this book sets out **how far** the primary materials it has checked in the original **reach** and **where the gaps are**. There are twenty-three of them (Chapters 1, 4, 5, 7, 8, 9, 10, 11, 13, 14, 15, 16, 17, 19, 20, 21, 22, 23, 24, 25, 26, 29 and 30), and the "table matching boxes to issues" at the end of 28.1 gathers them all in one table. These boxes are the core of what this book can give a lawyer: not an exposition of legal doctrine, but **a bridge that shows where technical and economic facts enter legal judgment** (27.1).

Key takeaway. The box at the head of each chapter. It states "what you will be able to do after reading this chapter." It is not a summary but what this book's reader, a lawyer, can take away at once.

3. Interactive figures and notebooks

Interactive figures are control panels placed beside the formulas of the text in the screen edition (HTML). Move a variable and the chart changes. The initial state is always the same as the example in the text, so that you first confirm the text's numbers on screen and only then move the variables. The variables are **structural variables**: hash-power share, number of confirmations, collateral ratio, liquidity pool size, the share of holders demanding redemption. There is no variable for entering an asset's price or investment return, and no interactive figure of that kind was built. One addition is the attack-cost calculation in Chapter 20 (W6). It has fields in which the user enters the unit cost of the resources an attack consumes (the unit cost of hashing), and this book does not supply those values. In the PDF, a still image of the initial state takes the place of each interactive figure.

INTERACTIVE FIGURE	CHAPTER	VARIABLES YOU MOVE	INITIAL STATE
W1 Hash avalanche	4	Input string, bit to flip	One letter changed from "hello"
W4 Merkle proof	6	Number of transactions n, leaf to prove	$n = 8$, leaf 5
W3 Chain tampering demonstration	7	Number and content of the block to tamper with	5-block chain, block 3 tampered with
W2 Hand miner	8	Difficulty in bits	16 bits
W5 Double-spending probability	9	q, z	$q = 0.1, z = 0...10$ (the table in §11 of the white paper)
W12 Block header anatomy	12	Field selection, nonce	Genesis block
W7 AMM slippage	15	Pool size x, y, order Δx	1,000 : 1,000, order 100
W10 Sandwich attack	17	Victim's order, attacker's input, pool size, tolerance	The pool of Chapter 15, victim 100, attacker 100
W6 51% attack cost	20	Share q, unit cost of hashing, duration	Share 60%, $k + e = 12$
W9 Fee market	21	Gas used per block	Limit 60,000,000, target 30,000,000
W8 Stablecoin bank run	24	Share demanding redemption, composition of reserve assets, collateral ratio	Type B′, demand 40%
W11 Clickable timeline	2, Appendix D	Click	1976–2023

Notebooks are Colab files that carry the Python code of the text over unchanged. They are numbered with the chapter number (NB04, NB05, ..., NB29), and a code cell marked "NB09-2" in the text is the second cell of notebook NB09. Four rules. The "Output" blocks in the text carry over **exactly** the results of the editor's actual runs; they were not touched up by hand. There is no installation cell: only the standard library is used, the one exception being a numerical package in Chapter 25, and the reason is given in that chapter and in the notebook list (Appendix F). No external API is called: for public block data the code reads only snapshot files downloaded on the reference date (the genesis block, the hash rate, the types of exposed outputs), and if a file is missing it substitutes constants with the same values. And **all the code is for study.** It is a toy elliptic curve over a small prime, a hand miner at a low difficulty, a state-transition machine that moves no real assets. The code comments also say "not for real use." This book contains no code that creates or stores a real private key or sends a transaction.

Below is the environment check cell (NB00). All that the code in this book assumes is Python 3.13 and the standard library.

```
# NB00: environment check — the code in this book assumes only Python 3.13 and the standard
library (for study only)
import sys, hashlib, secrets, json, struct, math, random, decimal

major, minor = sys.version_info[:2]
print(f"Python {major}.{minor}  →", "OK (3.13 or later)" if (major, minor) >= (3, 13) else "3.13
or later is recommended — the output format of some cells may differ")
print("SHA-256 check:", hashlib.sha256(b"").hexdigest()[:16], "… == e3b0c44298fc1c14 …",
hashlib.sha256(b"").hexdigest().startswith("e3b0c44298fc1c14"))
print("Double SHA-256 (the function Chapter 12 uses):",
hashlib.sha256(hashlib.sha256(b"abc").digest()).hexdigest()[:16], "…")
print("secrets module (for the secret numbers of the toy signatures in Chapter 5):", "available"
if secrets.randbits(1) in (0, 1) else "?")
print("struct (byte parsing, Chapter 12), json (reading snapshots, Chapters 20 and 29), decimal
(summing amounts, Chapter 29):", "available")
try:
    import numpy  # used only by NB25 in Chapter 25 — every other chapter runs without it
    print("numpy:", numpy.__version__, "(needed only for NB25 in Chapter 25)")
except ImportError:
    print("numpy: not installed — install it only when you run NB25 in Chapter 25. Every other
chapter runs on the standard library.")
print()
print("All the code in this book is for study only. Do not use it with real assets, real private
keys or real transactions (not for real use).")
```

```
# Output — bcbook (Python 3.13), exactly as run
Python 3.13  → OK (3.13 or later)
SHA-256 check: e3b0c44298fc1c14 … == e3b0c44298fc1c14 … True
Double SHA-256 (the function Chapter 12 uses): 4f8b42c22dd3729b …
secrets module (for the secret numbers of the toy signatures in Chapter 5): available
struct (byte parsing, Chapter 12), json (reading snapshots, Chapters 20 and 29), decimal
(summing amounts, Chapter 29): available
numpy: 2.5.4 (needed only for NB25 in Chapter 25)

All the code in this book is for study only. Do not use it with real assets, real private keys
or real transactions (not for real use).
```

4. Three paths

You need not read everything. Here are three paths, by chapter number.

PATH	CHAPTERS	ASSUMES
Basic path	1·2·3 → 4·7·8·9·12 → 13·15 → 19·20·24 → 26 → 27·28 → 30	[Basic] paragraphs only. All formulas and code skipped
Technical path	1·3 → all of 4–12 → all of 13–18 → 20·21·23 → 29 → Appendix E	Up to [Intermediate], running the notebooks
Economics and law path	1·2·3 → 4·8·9 (concepts only) → 15·17 → all of 19–25 → 26 → 27·28 → 30 → Appendices B and C	Up to [Intermediate]; Chapter 25 including [Advanced]

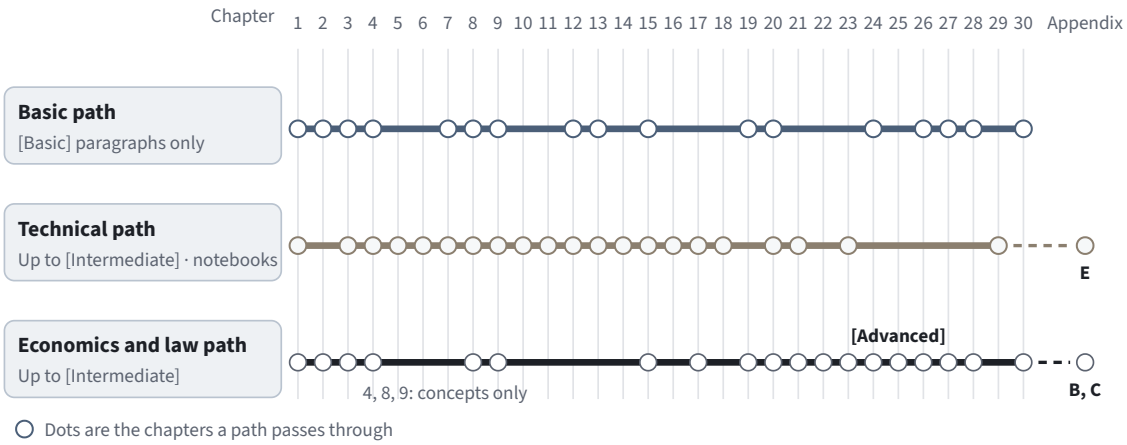


Figure 3-1. The three paths of the table above, redrawn as a route map with the chapter numbers as stations. The dots are the chapters a path passes through; the technical path continues to Appendix E, and the economics and law path to Appendices B and C.

The basic path passes through only five chapters of Part 1: hashes (4), the chain (7), proof of work (8), probabilistic finality (9), and one real block (12). Those five are enough to reach the meaning of "publicity, a single history, a majority by computation" (Chapter 1). The technical path reads all of Parts 1 and 2 and goes on to the quantum transition in Part 6, and the mathematical supplement in Appendix E serves as that path's footnotes. The economics and law path reads all of Part 3. Chapter 25 alone requires opening the `[Advanced]` boxes, because the point of the econometric tests for bubbles lies in the very way the critical values are computed. The chapters all three paths pass through are 1, 3, 4, 8, 9, 15 and 20.

5. Notes to the reader: figures, the reference date, marks

Reference date. Every figure in this book is as of the reference date, October 25, 2026 (Korea time; the actual observations were made on October 24, 2026, UTC). Block height, difficulty, hash rate, constants, the types of exposed outputs and the in-force status of provisions are those of that day, and the bundled snapshot files (`data/`) contain the raw values. There is one exception. The exposure tally in Chapter 29 is a snapshot made by a third party on October 19, 2026, downloaded on the reference date (Chapter 29, Section 3). Things may have changed since.

"As of 2026-10." Where a value that dates quickly had to go into the text, the sentence itself says "as of 2026-10," and the same item is entered in the Trend notes in Appendix A. When the edition changes, only Appendix A is replaced (the update cycle is half-yearly). The statements of principle in the text do not change. For example, "difficulty is adjusted every 2,016 blocks" is a principle, while "the difficulty on the reference date is about 1.4×10^{14}" is item A-4 of Appendix A.

"Original to be checked," "not verified," "reserved." This book did not write any bibliographic detail from memory, and it quotes in the footnotes only what was checked word for word against an opened original. What could not be checked is said to be so. The "Sources for this chapter" at the end of each chapter lists, along with the materials used, **what was not used and what could not be verified**. Marks such as "what the court

documents did not decide," "wording confirmed only through secondary reproduction" and "🟡 secondary tally" are of that kind. Those marks are not a weakness; they are the scale that shows how far this book can be trusted.

Footnotes and quotations. Quotations are **verbatim**. Two conventions of American usage apply to their punctuation: a comma or period that closes a quotation may sit inside the quotation marks even where the source has no such mark at that point, and a quotation set as a sentence of its own that stops before the end of the source sentence ends in an ellipsis. Passages from English-language sources stand in their original words, in the text or in the footnotes, and are not translated back from the Korean edition of this book. Passages from Korean and Japanese sources are given in the original, followed by a working gloss in English for reading only: the gloss is the editor's, it is not an official translation, and the original controls. Statutory provisions are reproduced as enacted (Chapter 30). If a gloss looks doubtful, read the original beside it, because translation is interpretation. Judgments are cited with the court, the date of the decision and the case number, and edited material such as headnotes and summaries of holdings is not quoted.

What this book does not do. It gives no price outlook (past price figures are used, with a reference date and source, only where they are needed to explain a method), and it does not recommend, compare or rate any asset, exchange, wallet or token. Names appear as examples only where the book records settled historical facts: judgments, official investigations, documents from insolvency proceedings, standards. The notices in three places, at the head of this chapter, at the opening of Part 4 and in the colophon, set down that promise.

Copyright. The text and the code of the interactive figures of this book are released under CC BY-NC 4.0: you may copy, adapt and redistribute them for noncommercial purposes on condition that you credit the source, give a link to the license, and indicate any changes you made (https://creativecommons.org/licenses/by-nc/4.0/). Copyright in the originals quoted in the footnotes remains with their respective holders, and the sources of the bundled snapshot data are given in the README of the data folder. The notice for the Solidity examples for reading in Part 2 is placed in the colophon (Appendix F).

6. Summary

The markers sort depth, the boxes step out of the way, the question boxes mark where the law comes in, and the interactive figures and notebooks put the formulas and code in your hands. Choose one of the three paths, check the reference date of each figure, and read every gloss against the original beside it. And you may skip `[Advanced]`. Whether that promise has been kept is the measure of whether this book was written well. Part 1 begins with the next chapter, Chapter 4.

Sources for this chapter

- The writing rules of this book (three levels of marker, collapsible boxes, the lawyer's question boxes, the range of variables in the interactive figures, "not for real use" in the code, marking of reference dates, the update cycle of the Trend notes, the license) were set by the editor.
- The list of interactive figures and their initial states are the same as the examples in the text of each chapter (Chapters 4, 6, 7, 8, 9, 12, 15, 17, 20, 21 and 24). The list of notebooks is in Appendix F.
- This book's bundled data, `data/README.md` (reference date 2026-10-25, accessed 2026-10-24 UTC): sources and formulas of the three snapshots.
- Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0), https://creativecommons.org/licenses/by-nc/4.0/.
- Code execution: Python 3.13 (conda environment `bcbook`), standard library only. Notebook NB00.

PART 1 · TECHNICAL FOUNDATIONS

Chapter 4. Hash functions: one-way functions that take a fingerprint

KEY TAKEAWAY

A hash function turns data of any length into a short, fixed-length "fingerprint." You cannot go back from the fingerprint to the original (one-wayness), you cannot produce a different original with the same fingerprint (collision resistance), and if one character of the original changes, the whole fingerprint changes (the avalanche effect). Half of the grounds on which blockchains are called "tamper-proof" come from here. The other half is the proof of work of Chapter 8. After this chapter you will be able to separate **what a hash value proves from what it does not prove**.

Learning objectives

After reading this chapter you will be able to do the following.

1. Tell one-wayness, collision resistance, and the avalanche effect apart.
2. Compute SHA-256 yourself with the Python standard library and confirm that a one-character difference in the input changes the entire output.
3. Explain, at the level of the technology, what a "document of which only the hash value was kept" can prove later, and where the limits of that proof lie.
4. **[Intermediate]** Work out with the birthday paradox why finding a collision in a 256-bit hash takes 2^{128} attempts.

What a reader of the [Basic] paragraphs alone takes away. A hash is a "fingerprint that cannot be reversed," and half of the grounds on which blockchains are called "tamper-proof" come from here. The other half is in Chapter 8.

1. What failure this technology grew out of

The oldest way to check that data has not been changed along the way is the **checksum**. You add up all the bytes and write down the last digits, then later add them up again and see whether the result is the same. That is enough to catch **accidental** errors, such as a single bit flipped by radio noise during transmission.

The problem is **intent**. Anyone can run addition backward. A person who has altered a document only has to adjust a byte, or append one, to make the last digits match. A checksum answers "did it change by mistake?" but not "did someone change it on purpose?" The code in Section 3 below shows this before your eyes.

So cryptographers looked for a different kind of function: one that is easy to compute but cannot be run backward, and for which no one can deliberately produce another input that gives the same value. In his 1987 paper Ralph Merkle defined a **one-way function** as "a function that is easy to compute, but difficult to invert. Given x, computing $y = F(x)$ is easy. Given y, determining any x such that $F(x) = y$ is hard." And he called one that takes an input of arbitrary length and produces a short output of fixed length a **one way hash function**.[1]

The time-stamping researchers Stuart Haber and W. Scott Stornetta, in their 1991 paper, defined a family of **collision-free hash functions**, which carries a similar but stronger requirement. These are functions that compress bit strings of arbitrary length to a fixed length of l bits such that (1) they are easy to compute and it is easy to pick one at random from the family, and (2) for a given h it is computationally infeasible to find distinct x, x' with $h(x) = h(x')$.[2] Why the two authors' paper matters in the prehistory of blockchains is something we see in Chapter 7.

2. Intuition: a fingerprint, and what a fingerprint cannot do

A person's fingerprint is far less information than the person, yet it picks the person out. You cannot reconstruct a person from a fingerprint, and you cannot make another person have the same fingerprint. A cryptographic hash function is a fingerprint for data. Put in a file of any length and out comes a value of a set length (for SHA-256, 256 bits, or 64 characters in hexadecimal).

The Secure Hash Standard (FIPS 180-4) of the U.S. National Institute of Standards and Technology (NIST) states in two sentences why these functions are called "secure": "The hash algorithms specified in this Standard are called secure because, for a given algorithm, it is computationally infeasible 1) to find a message that corresponds to a given message digest, or 2) to find two different messages that produce the same message digest. Any change to a message will, with a very high probability, result in a different message digest."[3] The standard's wording names two things, that finding a preimage is infeasible and that finding a collision is infeasible, and adds that a change alters the digest. Cryptography textbooks divide this into the three properties below; the table is this book's summary.

PROPERTY	MEANING	WHAT HAPPENS IF IT BREAKS
Preimage resistance, that is, one-wayness	Given only a hash value y, you cannot find an x with $h(x) = y$	A secret published only as its hash value (a password, a sealed document) is exposed
Second-preimage resistance	Given x, you cannot find a different x' that gives the same hash	A document already made public can be swapped for a different document with the same fingerprint
Collision resistance	You cannot find **any** two inputs $x \neq x'$ that give the same hash	A "two-faced" document can be prepared from the start

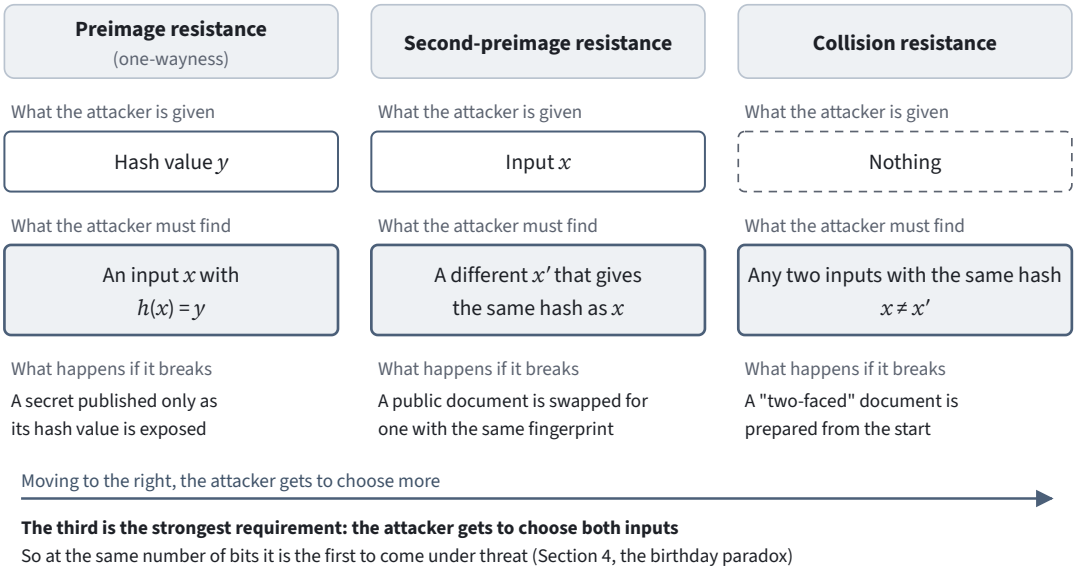


Figure 4-1. The three properties of a hash function, set side by side as what the attacker is given and what the attacker must find. Moving to the right, the attacker gets to choose more, and the third, collision resistance, is the strongest requirement.

The third is the strongest requirement, because the attacker gets to choose both inputs. So of the three, the third is the easiest to break, and at the same number of bits it is the first to come under threat. The birthday paradox in Section 4 below shows why, in numbers.

And there is a fourth property: the last sentence of the standard's wording, "Any change to a message will, with a very high probability, result in a different message digest." In practice it is stronger than that. Flip just one bit of the input and about half of the output bits flip. This is called the **avalanche effect**. Fingerprints are never "a little alike." They are either the same or completely different.

What a hash cannot do

A hash is not encryption. Encryption has to be reversible if you have the key. A hash cannot be reversed by anyone. Because of the name "cryptocurrency" it is easy to think that Bitcoin encrypts the contents of transactions, but Bitcoin's transactions and ledger are not encrypted. They are all public, and what is used there is hashes and signatures. This story continues in Chapter 5.

A hash also **does not vouch for the content.** A hash value says only "this bit string is the same as that bit string was then." It does not say whether the bit string is true, who wrote it, or when it was created. There is a limit that Haber and Stornetta had already recorded in 1991: "if any amount of time may pass between the creation of a document and when it is time-stamped, then no method can do more than forward-constrain the time at which the document itself was created. Thus, in general, time-stamping should only be considered as evidence that a document **has not been back-dated**."[4] Many services offer "proof of existence" by tying a hash to a chain, but what that proof points to is exactly this range.

> **? A LAWYER'S QUESTION**
>
> Only the hash value of a document was recorded on a public ledger, and years later it is submitted together with the original file. What does this hash value prove, and what does it not prove? "That bit string existed at that time," "that document was made at that time," and "the content of that document is true" are three different propositions. Which of the three do the evidence-law concepts of the original, the copy, and authenticity reach? → Chapter 28

3. Code: one character changes everything

[Intermediate] SHA-256 is included in `hashlib`, part of Python's standard library. All the code below is printed together with its results exactly as it was actually run. The environment is Python 3.13, and all the code in this book uses only the standard library.

```
# NB04-1: taking SHA-256 fingerprints — a one-character difference
import hashlib

def sha256_hex(s: str) -> str:
    """Encode a string as UTF-8 bytes and return its SHA-256 digest as 64 hexadecimal
characters."""
    return hashlib.sha256(s.encode("utf-8")).hexdigest()

for s in ["hello", "hellp", "Hello", "hello "]:
    print(f"{s!r:10} -> {sha256_hex(s)}")

def differing_bits(h1: str, h2: str) -> int:
    """Number of bits that differ between two hexadecimal digests (Hamming distance)."""
    return bin(int(h1, 16) ^ int(h2, 16)).count("1")

print()
print("hello vs hellp : differing bits", differing_bits(sha256_hex("hello"),
sha256_hex("hellp")), "/ 256")
print("hello vs Hello : differing bits", differing_bits(sha256_hex("hello"),
sha256_hex("Hello")), "/ 256")
print("hello vs 'hello ': differing bits", differing_bits(sha256_hex("hello"), sha256_hex("hello
")), "/ 256")
```

```
# Output — bcbook (Python 3.13), exactly as run
'hello'    -> 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
'hellp'    -> fdd7585e08c4e2afd71dcabdb4636c89d557a3f42db9e2040c8bbd1708aa4ce7
'Hello'    -> 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
'hello '   -> 5e3235a8346e5a4585f8c58562f5052b8fe26a3bb122e1e96c76784964dfc461

hello vs hellp : differing bits 131 / 256
hello vs Hello : differing bits 125 / 256
hello vs 'hello ': differing bits 139 / 256
```

[Intermediate] To the human eye the four inputs are almost the same, but the fingerprints are completely different. Close to half of the 256 bits flipped. This is the avalanche effect. To confirm that it is no accident, flip the bits one at a time and take the average.

Flipping one bit at a time: the average avalanche

```
# NB04-2: the avalanche effect — flip the input bits one at a time and average how much the
output changes
import hashlib

msg = b"The Times 03/Jan/2009 Chancellor on brink of second bailout for banks"
base = hashlib.sha256(msg).digest()

def flip_bit(data: bytes, i: int) -> bytes:
    """Flip the i-th bit (counting from 0) of a byte string."""
    b = bytearray(data)
    b[i // 8] ^= 1 << (i % 8)
    return bytes(b)

diffs = []
for i in range(len(msg) * 8):          # flip every bit of the input once (69 bytes × 8 = 552
bits)
    h = hashlib.sha256(flip_bit(msg, i)).digest()
    diffs.append(sum(bin(a ^ b).count("1") for a, b in zip(base, h)))

print("input bits flipped:", len(diffs))
print("output bits changed — min %d, max %d, mean %.1f (of 256 bits)" % (min(diffs), max(diffs),
sum(diffs) / len(diffs)))
```

```
# Output — bcbook (Python 3.13), exactly as run
input bits flipped: 552
output bits changed — min 101, max 150, mean 127.5 (of 256 bits)
```

[Intermediate] Whichever bit of the input is flipped, about 128 bits of the output change on average. That means the output looks like random numbers unrelated to the input. What this input sentence is will be revealed in Chapter 12.

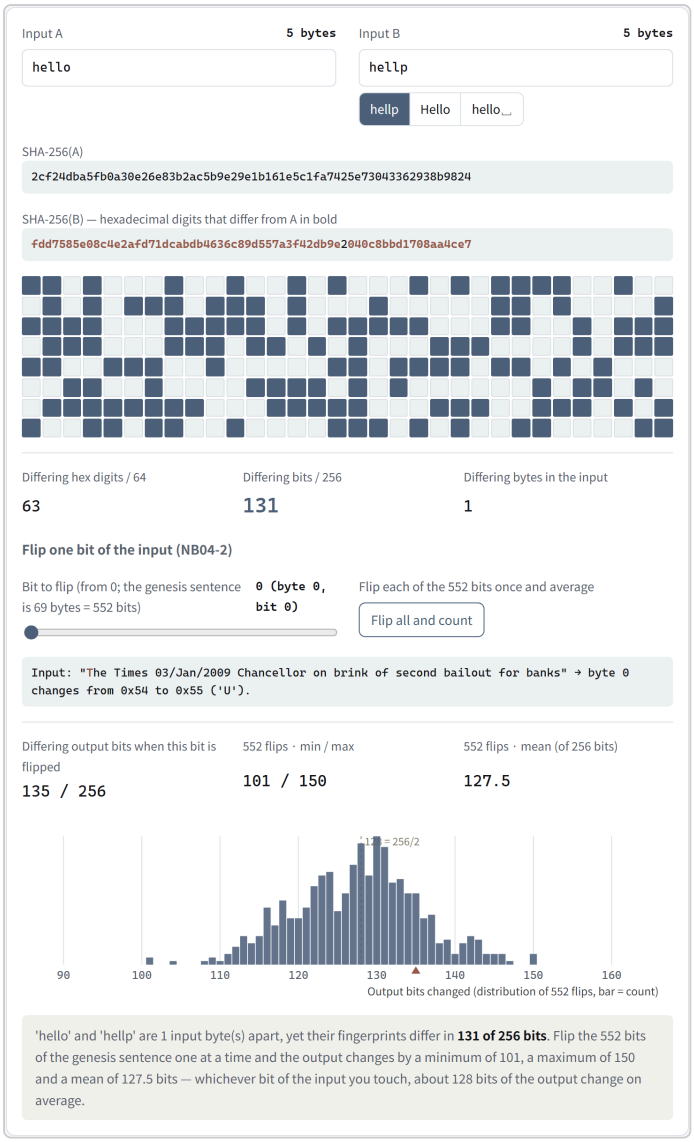


Figure 4-2. Hash avalanche. "hello" and "hellp" differ by one letter, yet their SHA-256 fingerprints differ in 131 of 256 bits. Flip any one bit of the input and about 128 bits of the output change on average (NB04-2: 552 flips, minimum 101, maximum 150, mean 127.5). — In the HTML edition you can change the values here (HTML edition, Chapter 4, Section 3).

Why a checksum will not do: a forgery exercise

[Intermediate] Now we confirm in code what Section 1 said. We compare a checksum that adds up the bytes and writes down the last digits (mod 256) with a "toy hash" that uses only the first 16 bits of SHA-256.

```
# NB04-3: forging a checksum is immediate; forging a hash takes exhaustive search
import hashlib

def checksum8(data: bytes) -> int:
    """Last digits of the byte sum (mod 256): a checksum for accidental errors."""
    return sum(data) % 256

original = b"Pay to Alice 100"
target = checksum8(original)
forged = bytearray(b"Pay to Bob   100 ")          # change the payee,
forged.append((target - checksum8(forged)) % 256)   # then append one byte to make the last
digits match
print("original checksum:", target)
print("forged checksum  :", checksum8(forged), "| forged:", bytes(forged))

def toy_hash16(data: bytes) -> int:
    """Toy hash that uses only the first 16 bits of SHA-256 (not for real use; for explanation
only)."""
    return int.from_bytes(hashlib.sha256(data).digest()[:2], "big")

t = toy_hash16(original)
attempts = 0
while True:
    attempts += 1
    candidate = b"Pay to Bob   100 #" + str(attempts).encode()
    if toy_hash16(candidate) == t:
        break
print("attempts to match the 16-bit toy hash:", attempts, "(expected 2^16 = 65,536)")
print("expected attempts to match all 256 bits: 2^256 ≈ %.1e" % 2**256)
```

```
# Output — bcbook (Python 3.13), exactly as run
original checksum: 220
forged checksum  : 220 | forged: b'Pay to Bob   100 k'
attempts to match the 16-bit toy hash: 27882 (expected 2^16 = 65,536)
expected attempts to match all 256 bits: 2^256 ≈ 1.2e+77
```

The checksum was matched at once by appending one byte. The 16-bit toy hash took tens of thousands of attempts, and that number doubles with every bit added. At 256 bits the expected number is $2^{256} \approx 1.2 \times 10^{77}$. Even running for 1 billion years at 10^{21} attempts per second, about the estimated hash rate of the whole Bitcoin network in 2026 (Chapter 8), comes to about 3×10^{37} attempts, short of the number needed by a factor of 10^{40}. The gap between those two lines is why "making a different document that matches the hash" is treated in practice as "impossible."

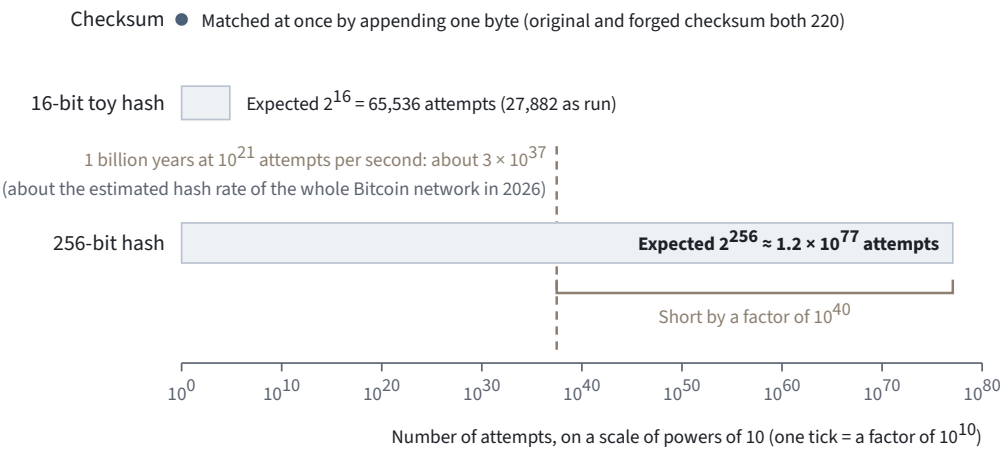


Figure 4-3. The number of attempts needed for forgery, placed on a scale of powers of 10. The figures are the output of NB04-3 and the calculation in the text, and 10^{21} per second is about the estimated hash rate of the whole Bitcoin network in 2026.

4. [Intermediate] The birthday paradox: collision resistance is only half as strong as the number of bits

[Intermediate] With 23 people in a class, the probability that two of them share a birthday is more than one half. That holds even though 23 days is only a small part of 365. The reason is that the number of pairs, "some two people," is $\binom{23}{2} = 253$. A hash collision is the same calculation.

[Intermediate] When the output space has size N and n values are drawn at random, the probability that some two of them are the same is approximately

$$P(\text{collision}) \approx 1 - e^{-n^2 / (2N)}$$

Reading the symbols: N is the number of possible hash values (2^{256} for SHA-256), and n is the number of inputs tried. The n at which this probability reaches $1 / 2$ is $n \approx 1.18\sqrt{N}$. A numerical example: for the 16-bit toy hash ($N = 65{,}536$), $n \approx 1.18 \times 256 \approx 302$, and for SHA-256, $n \approx 1.18 \times 2^{128}$. That is why people say that "the collision resistance of a 256-bit hash is 128 bits": half the number of bits.

[Intermediate] Merkle warned about this in his 1987 paper, calling it a "square root" attack: "The design of one way hash functions should be approached with caution: the most obvious approaches are sometimes vulnerable to 'square root' attacks."[5] The code below measures, 200 times over, how many values have to be drawn from the 16-bit toy hash before the first collision appears, and takes the average. The expected number up to the first collision is $\sqrt{\pi N / 2} \approx 1.25\sqrt{N}$, or about 320.8 for $N = 65{,}536$.

```
# NB04-4: the birthday paradox — how many draws until the first collision in the 16-bit toy hash
import hashlib, math

def toy_hash16(data: bytes) -> int:
    return int.from_bytes(hashlib.sha256(data).digest()[:2], "big")

def draws_until_collision(run: int) -> int:
    """Experiment number run: feed in distinct inputs one at a time and count until a hash
repeats for the first time."""
    seen = {}
    k = 0
    while True:
        k += 1
        h = toy_hash16(f"run{run}-item{k}".encode())
        if h in seen:
            return k
        seen[h] = k

results = [draws_until_collision(r) for r in range(200)]
N = 2**16
print("runs 200, output space N = %d" % N)
print("draws until first collision — mean %.1f, min %d, max %d" % (sum(results) / len(results),
min(results), max(results)))
print("theoretical expectation sqrt(pi*N/2) = %.1f" % math.sqrt(math.pi * N / 2))
print("n at which collision probability is 1/2: n ≈ 1.18*sqrt(N) = %.0f" % (1.18 *
math.sqrt(N)))
print("for SHA-256 (N = 2^256), 1.18*sqrt(N) ≈ %.2e" % (1.18 * math.sqrt(2**256)))
```

```
# Output — bcbook (Python 3.13), exactly as run
runs 200, output space N = 65536
draws until first collision — mean 325.1, min 19, max 754
theoretical expectation sqrt(pi*N/2) = 320.8
n at which collision probability is 1/2: n ≈ 1.18*sqrt(N) = 302
for SHA-256 (N = 2^256), 1.18*sqrt(N) ≈ 4.02e+38
```

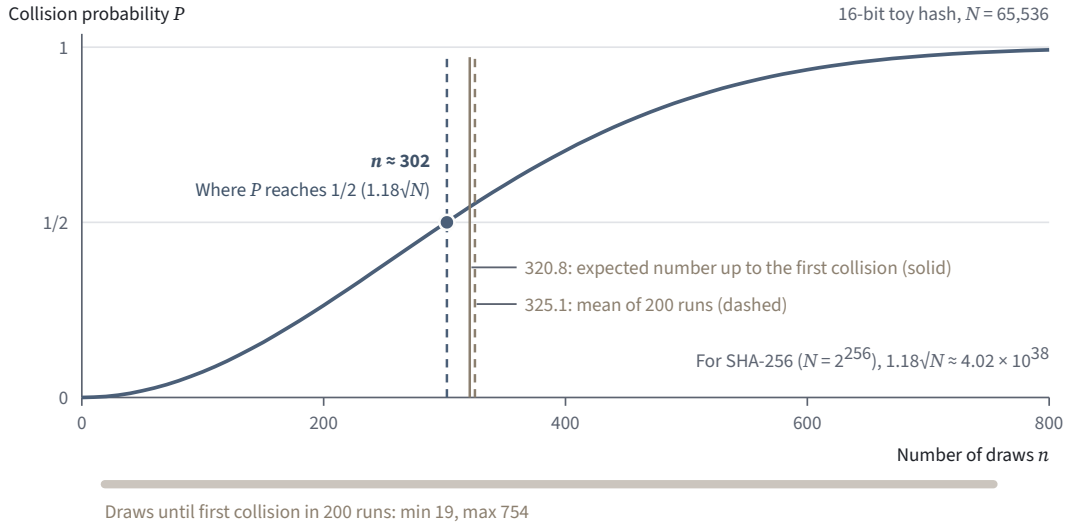


Figure 4-4. Probability of a collision against the number of draws n for the 16-bit toy hash ($N = 65{,}536$). The curve plots the formula in the text, $1 - e^{-n^2/(2N)}$, and the vertical line and figures are the output of NB04-4.

[Intermediate] The average lands near the theoretical value. At 16 bits a collision comes after 300 or so draws. At 256 bits it is on the order of 10^{38}. Same formula, different exponent.

5. What is SHA-256?

The hash function Bitcoin uses is SHA-256. It is an algorithm specified by the U.S. federal standard FIPS 180-4: it accepts any message shorter than 2^{64} bits, processes it in 512-bit blocks, computes with 32-bit words, and produces a 256-bit digest.[6] The standard is an August 2015 update of the 2012 edition, and NIST has said that in 2023 it began work on revising the standard. The current version this book consulted is the 2015 edition.

Bitcoin applies SHA-256 **twice, one on top of the other**. Both the hash of a block header and the transaction identifier (txid) are SHA256(SHA256(·)). To make an address, the public key is hashed once with SHA-256 and then shortened further with a 160-bit hash called RIPEMD-160.[7] The designer left no explanation of why it is done twice. The paper cited gives a reason in parentheses, that it is "doubled to prevent 'padding attacks'," but that is a third party's explanation, and this book records only the fact that SHA-256 is applied twice. Recomputing a double hash by hand and matching it against a published block hash is done in Chapter 12.

▾ **[Advanced] Inside SHA-256: a structure that chains compression functions together**

[Advanced] The standard divides the algorithm into two stages: **preprocessing** (padding the message, parsing it into 512-bit blocks, and setting the initial hash value) and **hash computation** (generating a message schedule for each block and iteratively updating the hash value with functions, constants, and word operations). The final hash value becomes the digest.[8] The key is "iteratively updating." A 256-bit state $H^{(i-1)}$ and a 512-bit message block $M^{(i)}$ go into the compression function to give a new state $H^{(i)}$, which goes in again along with the next block. The state is passed from block to block **like links in a chain**. Haber and Stornetta's 1991 paper states that Damgård gave the first formal definition of collision-free hash functions, and a constructive proof of their existence, on the assumption that one-way "claw-free" permutations exist, and it also cites Damgård's 1989 paper on a "design principle" for hash functions.[9] The name and the detailed development of this design principle, which chains compression functions together, are left to Appendix E.

[Advanced] Below, SHA-256 is rewritten in Python following the standard's specification. It takes fewer than 60 lines. We check that it gives the same values as `hashlib`, and for a two-block message we print the intermediate values to see how the state is passed from block to block. **This implementation is for study.** In real software, use a vetted library (not for real use).

```
# NB04-5: SHA-256 by hand — seeing the iterated structure of the standard's specification (for
study, not for real use)
import hashlib, struct

def icbrt(n: int) -> int:
    """Integer cube root (rounded down) — uses integer arithmetic only, so the constants carry
no floating-point error."""
    x = 1 << ((n.bit_length() + 2) // 3)
    while True:
        y = (2 * x + n // (x * x)) // 3
        if y >= x:
            return x
        x = y

PRIMES =
[2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,83,89,97,101,103,107,109,113,

127,131,137,139,149,151,157,163,167,173,179,181,191,193,197,199,211,223,227,229,233,239,241,25
1,
          257,263,269,271,277,281,283,293,307,311]
# constants K: first 32 bits of the fractional parts of the cube roots of the first 64 primes
(FIPS 180-4 §4.2.2) — the integer part of cbrt(p)·2^32, modulo 2^32
K = [icbrt(p << 96) & 0xFFFFFFFF for p in PRIMES]
# initial hash value H0: first 32 bits of the fractional parts of the square roots of the
first 8 primes (FIPS 180-4 §5.3.3)
import math
H0 = [math.isqrt(p << 64) & 0xFFFFFFFF for p in PRIMES[:8]]

def rotr(x, n): return ((x >> n) | (x << (32 - n))) & 0xFFFFFFFF

def compress(H, block):
    """Compression function: 256-bit state H and one 512-bit block → new state (FIPS 180-4
§6.2.2)"""
    w = list(struct.unpack(">16I", block))
    for t in range(16, 64):
        s0 = rotr(w[t-15], 7) ^ rotr(w[t-15], 18) ^ (w[t-15] >> 3)
        s1 = rotr(w[t-2], 17) ^ rotr(w[t-2], 19) ^ (w[t-2] >> 10)
        w.append((w[t-16] + s0 + w[t-7] + s1) & 0xFFFFFFFF)
    a, b, c, d, e, f, g, h = H
    for t in range(64):
        S1 = rotr(e, 6) ^ rotr(e, 11) ^ rotr(e, 25)
        ch = (e & f) ^ (~e & g)
        T1 = (h + S1 + ch + K[t] + w[t]) & 0xFFFFFFFF
        S0 = rotr(a, 2) ^ rotr(a, 13) ^ rotr(a, 22)
        maj = (a & b) ^ (a & c) ^ (b & c)
        T2 = (S0 + maj) & 0xFFFFFFFF
        h, g, f, e, d, c, b, a = g, f, e, (d + T1) & 0xFFFFFFFF, c, b, a, (T1 + T2) &
0xFFFFFFFF
    return [(x + y) & 0xFFFFFFFF for x, y in zip(H, [a, b, c, d, e, f, g, h])]

def sha256_manual(msg: bytes, trace=False) -> str:
    L = len(msg) * 8
    padded = msg + b"\x80" + b"\x00" * ((55 - len(msg)) % 64) + struct.pack(">Q", L)   #
§5.1.1 padding
    H = H0[:]
    for i in range(0, len(padded), 64):
        H = compress(H, padded[i:i+64])
        if trace:
            print(f"  state after block {i//64 + 1}, H^({i//64 + 1}) = {''.join(f'{x:08x}' for
x in H)}")
    return "".join(f"{x:08x}" for x in H)

for m in [b"", b"abc", b"hello"]:
    ok = sha256_manual(m) == hashlib.sha256(m).hexdigest()
    print(f"{m!r:9} by hand == hashlib ? {ok}")

two_blocks = b"a" * 100          # 100 bytes → two 512-bit blocks after padding
print("state changes for a 100-byte message (2 blocks):")
digest = sha256_manual(two_blocks, trace=True)
print("  final digest               =", digest)
print("  hashlib                    =", hashlib.sha256(two_blocks).hexdigest())
```

```
# Output — bcbook (Python 3.13), exactly as run
b''       by hand == hashlib ? True
b'abc'    by hand == hashlib ? True
b'hello'  by hand == hashlib ? True
state changes for a 100-byte message (2 blocks):
  state after block 1, H^(1) =
df5bb81ce81e0626fb45a8944fd40f31b25e6816d6d499c1ab90492900635e66
  state after block 2, H^(2) =
2816597888e4a0d3a36b82b83316ab32680eb8f00f8cd3b904d681246d285a0e
  final digest               =
2816597888e4a0d3a36b82b83316ab32680eb8f00f8cd3b904d681246d285a0e
  hashlib                    =
2816597888e4a0d3a36b82b83316ab32680eb8f00f8cd3b904d681246d285a0e
```

[Advanced] The state after block 1 becomes the input to block 2, and the state after block 2 becomes the digest unchanged. This structure of "handing the state on" has the same shape as the structure in Chapter 7 of "putting the hash of the previous block into the next block." In one sense, a blockchain is the internal structure of a hash function spread out at the scale of a network.

6. Summary: half of "tamper-proof"

Blockchains are often introduced as "tamper-proof ledgers." What this chapter has shown is half of that claim. Thanks to hash functions, **tampering is always revealed.** Change one character and the fingerprint changes, and you cannot slip in different content that gives the same fingerprint. But "revealed" and "prevented" are different things. What happens if someone puts forward the whole altered ledger again, **together with new fingerprints**? The device that makes that costly is the proof of work of Chapter 8, and the point that costly does not mean impossible is the subject of Chapter 9. Misconception ① in Appendix B ("blockchains are tamper-proof") is judged after all three chapters have been read.

Sources for this chapter

- NIST, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, August 2015. DOI 10.6028/NIST.FIPS.180-4. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf — definition (p. iv), two-stage structure (p. 3), table of parameters (Figure 1, p. 3), constants and initial values (§4.2.2, §5.3.3), padding (§5.1.1), SHA-256 computation (§6.2.2).
- Ralph C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in C. Pomerance (ed.), *Advances in Cryptology — CRYPTO '87*, LNCS 293, pp. 369–378. Springer, 1988. DOI 10.1007/3-540-48184-2_32 — definitions of the one-way function and the one way hash function, and the warning about square root attacks (p. 370).
- Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *Journal of Cryptology* 3(2): 99–111, 1991. DOI 10.1007/BF00196791 — definition of collision-free hash functions (pp. 101–102), the range of what a time-stamp proves (p. 106).
- Cyril Grunspan & Ricardo Pérez-Marco, "Double Spend Races," *International Journal of Theoretical and Applied Finance* 21(8): 1850053, 2018. DOI 10.1142/S021902491850053X; arXiv:1702.02867 — double SHA-256 (§2).
- Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin: Programming the Open*

Blockchain, 3rd ed. O'Reilly, 2023 (CC BY-SA 4.0, GitHub manuscript commit 94c9ba64) — construction of the address hash (chapter 4).

- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library modules `hashlib`, `struct`, and `math`. Notebook NB04.

Notes

1. Ralph C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in C. Pomerance (ed.), *Advances in Cryptology — CRYPTO '87*, LNCS 293 (Springer, 1988), p. 370: "A one-way function F is a function that is easy to compute, but difficult to invert. Given x, computing y=F(x) is easy. Given y, determining any x such that F(x)=y is hard." On the same page Merkle calls a one way function that accepts an arbitrarily large input and produces a small, fixed-size output a one way hash function.
2. Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *Journal of Cryptology* 3(2): 99–111 (1991), pp. 101–102 (§4.1, Hash).
3. NIST, *Secure Hash Standard (SHS)*, FIPS PUB 180-4 (August 2015), p. iv (Announcing section, 3. Explanation). DOI 10.6028/NIST.FIPS.180-4.
4. Haber & Stornetta (1991), p. 106 (§6.2, Time Constraints). Emphasis added.
5. Merkle (1988), p. 370. He goes on to give the example of a naive design that reduces 112 bits to 64 bits and is broken in about 2^{28} operations.
6. FIPS 180-4, Figure 1 (p. 3), "Secure Hash Algorithm Properties": SHA-256 — Message Size < 2^{64} bits, Block Size 512, Word Size 32, Message Digest Size 256. The planned revision is as stated in the Planning Note (2023-03-07) on the FIPS 180-4 page of NIST CSRC; when accessed on 2026-10-25, the current version was the 2015 edition.
7. Double SHA-256: Cyril Grunspan & Ricardo Pérez-Marco, "Double Spend Races," *International Journal of Theoretical and Applied Finance* 21(8): 1850053 (2018), §2 (p. 5 of arXiv:1702.02867v3): "The mining algorithm consists in performing the double SHA-256 of the block header ..."; the same sentence adds the reason in parentheses, "(doubled to prevent 'padding attacks')", and footnote 1 says that "a double hash SHA256(SHA256(header)) is computed". Hash for addresses: Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed. (O'Reilly, 2023), chapter 4: $A = \mathrm{RIPEMD160}(\mathrm{SHA256}(K))$ (GitHub manuscript, commit 94c9ba64, `ch04_keys.adoc`). That the transaction identifier is a double SHA-256 is confirmed directly from the genesis block data in Chapter 12.
8. FIPS 180-4, §1, Introduction (p. 3): "Each algorithm can be described in two stages: preprocessing and hash computation. Preprocessing involves padding a message, parsing the padded message into m-bit blocks, and setting initialization values to be used in the hash computation. The hash computation generates a message schedule from the padded message and uses that schedule, along with functions, constants, and word operations to iteratively generate a series of hash values."
9. Haber & Stornetta (1991), p. 102: "Damgård gave the first formal definition, and a constructive proof of their existence, on the assumption that there exist one-way 'claw-free' permutations [4]." Reference [4] is I. Damgård, "Collision-free hash functions and public-key signature schemes," *Eurocrypt '87*, LNCS 304 (1988), 203–217. Reference [5] of the same paper is I. Damgård, "A design principle for hash functions," *Crypto '89*, LNCS 435 (1990), 416–427, and it is this one that deals with the design principle of iterating a compression function (this book has not obtained the originals of either paper).

PART 1 · TECHNICAL FOUNDATIONS

Chapter 5. Public-key signatures: splitting authority between two keys

KEY TAKEAWAY

Bitcoin's transactions and ledger are not encrypted. Every transaction is public. What Bitcoin protects is not a secret but **the authority to spend**, and its tool is the digital signature. Only the holder of the private key can produce a signature, and anyone who knows the public key can verify it. The technical meaning of "having a coin" is only this: "being able to produce the signature that passes that coin to the next person." After this chapter you will be able to explain how private keys, public keys and addresses relate, and to follow by hand why the signing formula $s = k^{-1}(e + r\,d)$ has the shape it does.

Learning objectives

1. Draw the relation among private key, public key and address, and explain that a signature is "approval of content," not "encryption."
2. [Intermediate] Write the inputs and outputs of ECDSA signing and verification as a table, and confirm in code why every signature needs a new secret number k.
3. [Advanced] Do one point addition by hand on a toy elliptic curve over a small prime, and read the parameters of the real secp256k1 curve from the standards document.
4. Say what, technically, a person loses when a private key is lost.

What a reader of the [Basic] paragraphs alone takes away. The transactions and ledger of cryptocurrencies are not encrypted. What is protected is not a secret but **the authority to spend**.

1. What failure this technology grew out of

Traditional ciphers have one key. The same key locks and opens. So before two people can begin a secret correspondence, **they must first hand over that key safely.** But if there is a channel safe enough to carry the key, why not send the letter through it? The key-delivery problem was harder than the cipher itself. In the first paragraph of their 1976 paper, Whitfield Diffie and Martin Hellman wrote of two new requirements that commercial applications had created: "such applications create a need for new types of cryptographic systems which **minimize the necessity of secure key distribution channels** and **supply the equivalent of a written signature**."[1] Public-key cryptography is the invention that answered both at once.

The core of the idea is to split the key in two. One is made public (the public key) and one is kept hidden (the private key). The two keys are a mathematical pair, but the private key cannot be computed from the public key. For this to work you need an operation that is "easy in one direction and hard in the other." It has the same root as the one-wayness of Chapter 4. The difference is that nobody can reverse a hash, whereas in a signature scheme **only the holder of the private key** can produce a valid signature and nobody else can. The foundation is an operation that goes easily from private key to public key but cannot go back from public key to private key.

2. The intuition: a signature is approval, not encryption

A digital signature consists of the following three operations.

OPERATION	WHO	INPUTS	OUTPUT
Key generation	The owner	Random number	A pair: private key d and public key Q
Signing	Only the holder of the private key	Message M, private key d	Signature (r, s)
Verification	Anyone who knows the public key	Message M, signature (r, s), public key Q	True or false

The message stays in plain view. A signature does not hide the message. It **attaches** evidence that "the owner of the private key approved this message." If one bit of the message changes, verification fails. So a signature guarantees two things at once: who approved (authentication), and that the content has not changed since it was approved (integrity).

The Nakamoto white paper defines a coin as a chain of these signatures: "We define an electronic coin as a chain of digital signatures. Each owner transfers the coin to the next by digitally signing a hash of the previous transaction and the public key of the next owner and adding these to the end of the coin. A payee can verify the signatures to verify the chain of ownership."[2] A coin is not a thing but **a record of approvals**. "Having a coin" means having the private key that pairs with the public key written in the last slot of that record, and so

being able to write the next slot.

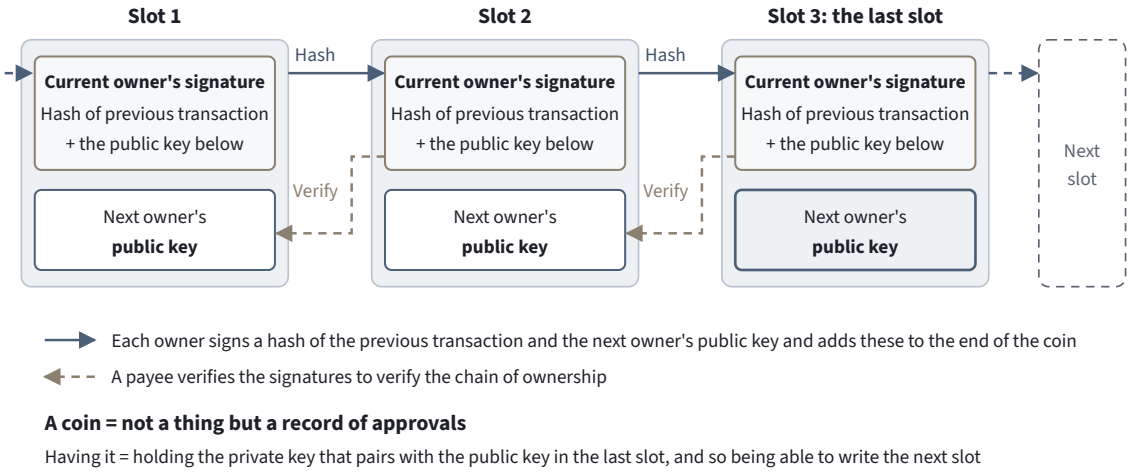


Figure 5-1. The definition in §2 of the Nakamoto white paper, redrawn in that sentence's own words. Each slot is the current owner's signature over the hash of the previous transaction and the next owner's public key, and the coin is this record of approvals.

From public key to address

A Bitcoin "address" is not the public key itself but a hash of the public key. The public key K is hashed with SHA-256, and the result is shortened by a second, 160-bit hash called RIPEMD-160: $A = \text{RIPEMD160}(\text{SHA256}(K))$.[3] These 20 bytes are written into the output of a transaction, and whoever wants to spend that money must present both the public key from which the hash was made and a signature. So an address that has only received money through such hash-type outputs (P2PKH, P2WPKH) has not yet revealed its public key. Other output types are different. The genesis output (P2PK) in Chapter 12 has the public key written out in full, and the Taproot output (P2TR) discussed below also carries a single public key directly. Why the distinction matters comes up again in Chapter 29. What a quantum computer threatens is coins whose public keys have been **exposed**.

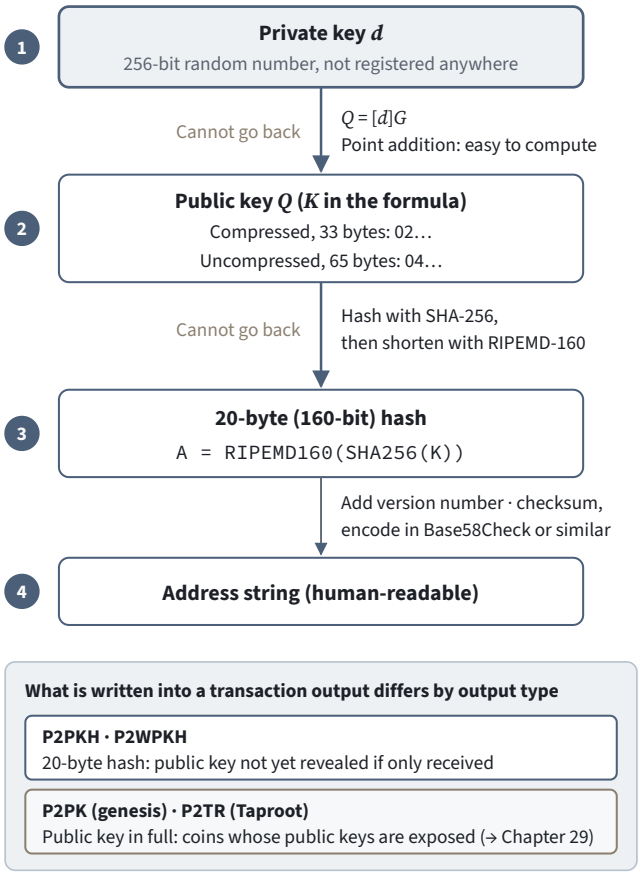


Figure 5-2. The road from private key to public key, and from public key to a 20-byte hash, runs one way only. The final encoding is a notation, not a hash, and what is written into a transaction output differs by output type.

The white paper advises using a new key pair for each transaction: "As an additional firewall, a new key pair should be used for each transaction to keep them from being linked to a common owner."[4] Most of what the software called a "wallet" does is generate and store these key pairs. This book does not name any wallet product.

If you lose the private key

A private key is a single large number. It is not registered anywhere, and if you forget it there is no one to recover it for you. The coins of a person who has lost the private key remain on the ledger. It is just that nobody can spend them. Conversely, if a private key leaks, anyone who has the leaked key can produce a valid signature that spends those coins. The ledger looks only at the validity of signatures, not at people.

> **? A LAWYER'S QUESTION**
>
> What has a person who lost the private key lost? The coins are still recorded on the ledger, and they sit at that person's address. Is what was lost possession (점유), ownership (소유), or a third thing called "accessibility" (접근 가능성)? If someone who stole the private key moves the coins, what has that person "acquired" (취득)? On a ledger that looks only at the validity of signatures, where does the concept of the "rightful holder" (정당한 권리자) sit? → Chapter 27

3. [Intermediate] Reading the signing formula: ECDSA

[Intermediate] The signature scheme Bitcoin has used from the start is ECDSA (Elliptic Curve Digital Signature Algorithm). It is a signature algorithm that runs on an elliptic curve, and the curve it uses is the one named secp256k1. The BIP-340 document records this in two sentences: "Bitcoin has traditionally used ECDSA signatures over the secp256k1 curve with SHA256 hashes for authenticating transactions. These are standardized …."[5] In this sentence, the document linked from "standardized" is SEC 1 (the signature algorithm), and the document linked from "secp256k1 curve" is SEC 2 (the curve parameters). The U.S. federal standard FIPS 186-5 also specifies the same ECDSA algorithm, but it leaves the choice of curves to a separate document (NIST SP 800-186), and secp256k1 is a curve specified by SEC 2. Below we read the steps of the algorithm as FIPS 186-5 describes them.

[Intermediate] First, the symbols. There is a base point G on the curve, and the point obtained by "adding" G to itself an integer k times is written $[k]G$. The order n of G is the smallest positive integer for which $[n]G$ is again the "zero point." The private key d is an integer with $1 \le d \le n-1$, and the public key is $Q = [d]G$. Computing Q from d is easy (a few hundred point additions), and finding d from Q is a hard problem **called the elliptic curve discrete logarithm problem**. This is the one-wayness of this signature.

Signature generation (FIPS 186-5 §6.4.1).[6]

STEP	WHAT IT DOES	FORMULA
1	Hash the message	$H = \mathrm{Hash}(M)$
2	Turn the hash into an integer	e: all of H if the bit length of n is at least the hash length, otherwise the leftmost $\log_2 n$ bits of H (for secp256k1 and SHA-256 both are 256 bits, so all of it)
3	Draw a secret number that is **new for each message**	$k, 0 < k < n$
4	Make a point on the curve	$R = [k]G, r = x_R \bmod n$
5	Compute the second component of the signature	$s = k^{-1}(e + r\,d) \bmod n$
6	Destroy k and k^{-1} at once	—
7	Output the signature	(r, s)

Reading the symbols: d is the private key, e is the message hash read as an integer, k is a one-time secret number used only for this signature, r is the x coordinate of $[k]G$ reduced modulo n, and s is the value that binds the three together. For a numerical example, see the toy curve ($n = 79$) in code NB05-2 below.

Signature verification (FIPS 186-5 §6.4.2).[7] The verifier knows neither d nor k. All it has are M, (r, s) and Q.

STEP	WHAT IT DOES	FORMULA
1	Check that r, s are in $[1, n-1]$	—
2	Hash the message the same way to get e	e
3	Make two coefficients	$u = e\,s^{-1} \bmod n, v = r\,s^{-1} \bmod n$
4	Rebuild a point	$R_1 = [u]G + [v]Q$
5	Check whether that point's x coordinate equals r	$r \stackrel{?}{=} x_{R_1} \bmod n$

[Intermediate] Why does this fit together? Since $Q = [d]G$, $R_1 = [u]G + [v\,d]G = [(e + r\,d)\,s^{-1}]G$. In signature generation $s = k^{-1}(e + r\,d)$, so $(e + r\,d)\,s^{-1} = k$. Therefore $R_1 = [k]G = R$, and its x coordinate equals r. The k and d that only the signer knows cancel each other out inside the formula, and the verifier checks the equation without knowing either of them. This is the essence of a public-key signature.

Use the secret number k twice and the private key is revealed

[Intermediate] Why the standard insists that k be generated anew for each signature and kept protected follows straight from the formula. If two messages e_1, e_2 are signed with the same k, r is the same, and since $s_1 - s_2 = k^{-1}(e_1 - e_2)$, $k = (e_1 - e_2)(s_1 - s_2)^{-1}$ can be computed from public information alone. Once k is known, the private key comes out as $d = (s_1 k - e_1)\,r^{-1}$. NB05-3 below actually does this on the toy curve. **The security of a signature rests on more than keeping the private key secret. It also rests on the quality of the new random number drawn for each signature.**

An ECDSA signature has two forms: malleability

[Intermediate] ECDSA has one more property. If (r, s) is a valid signature, then $(r, n - s)$ is also valid for the same message and the same public key. BIP-340 puts it this way: "ECDSA signatures are inherently malleable; a third party without access to the secret key can alter an existing valid signature for a given public key and message into **another** signature that is valid for the same key and message." It adds that Bitcoin has permitted only one of the two through a network policy.[8] When the signature changes, the identifier (txid) of the transaction that carries the signature also changes. One branch of the problem called "transaction malleability" comes from here, and we meet the term again in the account of Mt.Gox in Chapter 26.

Schnorr signatures and Taproot

[Intermediate] With the Taproot upgrade, activated in 2021, a second signature scheme entered Bitcoin: **the Schnorr signature** specified by BIP-340. It has a fixed length of 64 bytes, and BIP-340 lists three advantages over ECDSA: provable security, non-malleability, and linearity (the signatures of several participants can be combined into a single signature for the sum of their public keys).[9] BIP-341 specified a new output type (SegWit version 1) that uses these signatures and Merkle branches.[10] As of 2026-10-25 both documents have the status "Deployed." The Bitcoin Core source records the activation parameters of the Taproot soft fork as a signaling period from 2021-04-24 to 2021-08-11 and a minimum activation height of 709,632.[11] ECDSA has not gone away. The two schemes are used side by side.

4. Code: doing all of it on a toy curve

[Intermediate] The numbers in the real secp256k1 run to 78 digits, which makes them hard to follow by hand. So we first do exactly the same operations on a **toy curve** $y^2 = x^3 + 7 \pmod{97}$ over the prime $p = 97$. The shape of the equation $(x^3 + 7)$ is the same as secp256k1; only the size is small. All of the code below is **for study only and must never be used for real**. This curve has only 79 points, so a private key can be found by exhaustive search in under 1 second.

▾ [Advanced] What an elliptic curve is: what it means to "add" points

[Advanced] In his 1987 paper Neal Koblitz defines an elliptic curve as follows. An elliptic curve E_K over a field K of characteristic other than 2 or 3 is the set of solutions $(x, y) \in K^2$ of the equation $y^2 = x^3 + ax + b$ ($a, b \in K$, with no repeated root in the cubic on the right), together with one extra "point at infinity."[12] Over a finite field this "curve" does not look like a curve; it is a scattered set of points. Even so, there is a rule for "adding" two points to get a third, and the rule satisfies associativity, commutativity and the existence of inverses, so the points **form a group**. The point at infinity $\mathcal{O}$ is the zero (the identity element).

[Advanced] The addition rule (over the prime field $\mathbb{F}_p$, all coordinates $\bmod p$):

- If $P = (x_1, y_1)$, $Q = (x_2, y_2)$ and $P \neq \pm Q$, the slope is $\lambda = (y_2 - y_1)(x_2 - x_1)^{-1}$,

$x_3 = \lambda^2 - x_1 - x_2$, $y_3 = \lambda(x_1 - x_3) - y_1$.

- If $P = Q$ (doubling), $\lambda = (3x_1^2 + a)(2y_1)^{-1}$, and the rest is the same.
- If $Q = -P = (x_1, -y_1)$, then $P + Q = \mathcal{O}$.

Reading the symbols: $(\cdot)^{-1}$ is the multiplicative inverse modulo p ($a^{p-2} \bmod p$ by Fermat's little theorem), and a is the coefficient of the linear term in the curve equation ($a = 0$ for both secp256k1 and the toy curve). A numerical example: doubling $G = (1, 28)$ on the toy curve starts with $\lambda = 3 \cdot 1 \cdot (56)^{-1} \bmod 97$. The code below computes all of it.

[Advanced] The scalar multiple $[k]G$ is G added to itself k times, but when k has 78 digits you cannot add k times. Instead, write k out in binary and repeat "double, and add if the bit is 1" 256 times, and you are done. Computing it is that easy. In the other direction, no fast way is known to find k from $[k]G$ (on classical computers; for quantum computers see Chapter 29).

```
# NB05-1: toy elliptic curve y^2 = x^3 + 7 (mod 97) — point addition, doubling, scalar
multiplication (for study only, never for real use)
p, a, b = 97, 0, 7          # same shape as secp256k1 (a=0, b=7), only smaller
O = None                     # point at infinity (identity element)

def on_curve(P):
    if P is O: return True
    x, y = P
    return (y * y - (x ** 3 + a * x + b)) % p == 0

def inv(z): return pow(z, p - 2, p)              # multiplicative inverse (Fermat's little
theorem)

def add(P, Q):
    if P is O: return Q
    if Q is O: return P
    (x1, y1), (x2, y2) = P, Q
    if x1 == x2 and (y1 + y2) % p == 0: return O   # P + (-P) = O
    if P == Q: lam = (3 * x1 * x1 + a) * inv(2 * y1) % p
    else:      lam = (y2 - y1) * inv(x2 - x1) % p
    x3 = (lam * lam - x1 - x2) % p
    y3 = (lam * (x1 - x3) - y1) % p
    return (x3, y3)

def mul(k, P):                                    # double-and-add
    R = O
    while k:
        if k & 1: R = add(R, P)
        P = add(P, P)
        k >>= 1
    return R

points = [(x, y) for x in range(p) for y in range(p) if on_curve((x, y))]
print("number of points on the curve (including the point at infinity) n =", len(points) + 1)
G = min(points)                                   # take the point with the smallest x as the
base point
print("base point G =", G)
print("2G =", add(G, G), " 3G =", mul(3, G), " 79G =", mul(79, G), "(None means the point at
infinity)")
d = 23                                            # toy private key: never use for real
Q = mul(d, G)
print("private key d = %d → public key Q = [d]G = %s" % (d, Q))
found = next(k for k in range(1, 79) if mul(k, G) == Q)
print("d recovered from Q by exhaustive search: d =", found, "(instant with only 79 points; the
real curve is on the scale of 2^256)")
```

```
# Output — bcbook (Python 3.13), exactly as run
number of points on the curve (including the point at infinity) n = 79
base point G = (1, 28)
2G = (68, 81)  3G = (53, 38)  79G = None (None means the point at infinity)
private key d = 23 → public key Q = [d]G = (61, 28)
d recovered from Q by exhaustive search: d = 23 (instant with only 79 points; the real curve is
on the scale of 2^256)
```

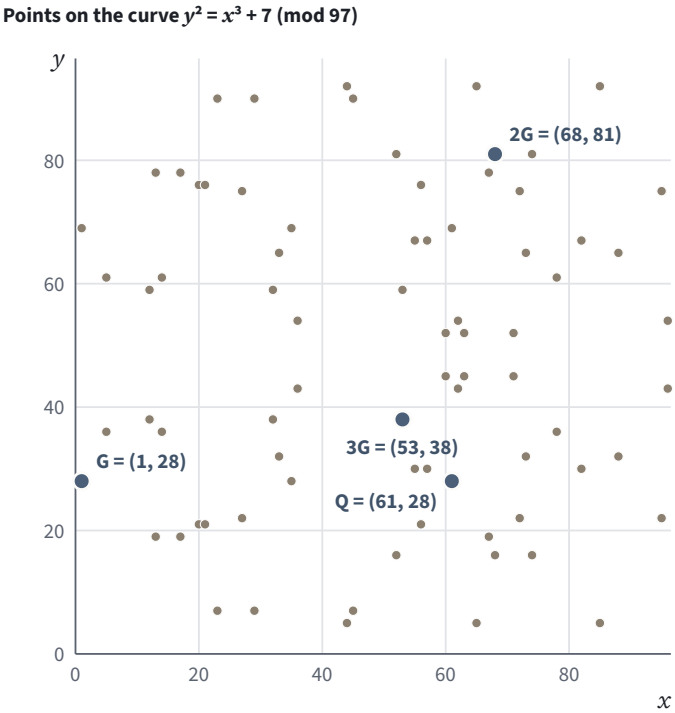


Figure 5-3. Points on the toy curve $y^2 = x^3 + 7 \pmod{97}$. The points were plotted by computing the solutions of the equation in the text; the coordinates of the labeled G, 2G, 3G and Q and the point count 79 (including the point at infinity) are the output of NB05-1. Over a finite field, the "curve" is a scattered set of points.

[Intermediate] Since the order 79 is prime, every point other than the point at infinity is a generator, and $[79]G = \mathcal{O}$. The same code runs on the real curve if you change only the numbers (NB05-4). Now we implement ECDSA on this group exactly as specified.

```
# NB05-2: toy ECDSA — signing and verification, failure when one bit changes, (r, n-s) also
valid (for study only, never for real use)
import hashlib
n = 79                                              # order of the base point G

def inv_n(z): return pow(z, n - 2, n)

def msg_int(M: bytes) -> int:
    """Message hash as an integer. Since n is small, reduce the SHA-256 result modulo n (a
simplification for illustration)."""
    return int.from_bytes(hashlib.sha256(M).digest(), "big") % n

def sign(d, M, k):
    e = msg_int(M)
    R = mul(k, G)
    r = R[0] % n
    s = inv_n(k) * (e + r * d) % n
    assert r != 0 and s != 0
    return (r, s)

def verify(Q, M, sig):
    r, s = sig
    if not (1 <= r <= n - 1 and 1 <= s <= n - 1): return False
    e = msg_int(M)
    u = e * inv_n(s) % n
    v = r * inv_n(s) % n
    R1 = add(mul(u, G), mul(v, Q))
    return R1 is not O and R1[0] % n == r

M = b"Pay 1 coin to Bob"
k = 17                                              # one-time secret number: fixed here for
illustration (in practice, a secure random number)
sig = sign(d, M, k)
print("message:", M, "| signature (r, s) =", sig)
print("normal verification      :", verify(Q, M, sig))
print("one character changed    :", verify(Q, b"Pay 2 coin to Bob", sig))
print("someone else's public key:", verify(mul(24, G), M, sig))
r, s = sig
print("(r, n-s) =", (r, (n - s) % n), "is also valid:", verify(Q, M, (r, (n - s) % n)), " ←
ECDSA malleability")
```

```
# Output — bcbook (Python 3.13), exactly as run
message: b'Pay 1 coin to Bob' | signature (r, s) = (17, 45)
normal verification      : True
one character changed    : False
someone else's public key: False
(r, n-s) = (17, 34) is also valid: True  ← ECDSA malleability
```

```
# NB05-3: use the secret number k twice, and the private key can be computed from two public
signatures (for study only)
M1, M2 = b"Pay 1 coin to Bob", b"Pay 5 coins to Carol"
k_reused = 31
(r1, s1), (r2, s2) = sign(d, M1, k_reused), sign(d, M2, k_reused)
e1, e2 = msg_int(M1), msg_int(M2)
print("the two signatures have the same r:", r1 == r2, "(r =", r1, ")")
k_rec = (e1 - e2) * inv_n(s1 - s2) % n
d_rec = (s1 * k_rec - e1) * inv_n(r1) % n
print("recovered k =", k_rec, "| recovered private key d =", d_rec, "| actual d =", d)
```

```
# Output — bcbook (Python 3.13), exactly as run
the two signatures have the same r: True (r = 52 )
recovered k = 31 | recovered private key d = 23 | actual d = 23
```

[Intermediate] The private key was recovered from two signatures alone. This happens because of the shape of the formula, not because the curve is small. It works the same way on the real curve.

The real curve: secp256k1

[Intermediate] The parameters of the real Bitcoin curve are set out in §2.4.1 of the SEC 2 document.[13] The prime $p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$, the curve $y^2 = x^3 + 7$, the compressed form of the base point G, `02 79BE667E …`, the order n, `FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFE BAAEDCE6 AF48A03B BFD25E8C D0364141`, and the cofactor $h = 1$. The code below puts these numbers into the functions of NB05-1 and checks three things: whether G is on the curve, whether $[n]G = \mathcal{O}$, and whether recovering y from the compressed form `02…` gives the same point as the uncompressed form `04…`. No external package is used. The numbers are larger; the operations are the same.

```
# NB05-4: the real secp256k1 parameters (SEC 2 §2.4.1), checked with the standard library alone
(for study only, never for real use)
import hashlib
p = 2**256 - 2**32 - 2**9 - 2**8 - 2**7 - 2**6 - 2**4 - 1
a, b = 0, 7
n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8
G = (Gx, Gy)
print("p in base 16:", hex(p).upper()[2:])
print("G is on the curve            :", on_curve(G))
print("[n]G is the point at infinity:", mul(n, G) is O)
# recover y from the compressed form 02||x: since p ≡ 3 (mod 4), y = (x^3+7)^((p+1)/4). The
prefix 02 means y is even.
y_cand = pow((Gx**3 + 7) % p, (p + 1) // 4, p)
y_even = y_cand if y_cand % 2 == 0 else p - y_cand
print("y recovered from the compressed form == y of the uncompressed form:", y_even == Gy)

d_toy = 1                                          # study private key 1 → the public key is G
itself. This is not real key generation.
Qx, Qy = mul(d_toy, G)
compressed = bytes([2 + (Qy & 1)]) + Qx.to_bytes(32, "big")
print("compressed public key for d=1:", compressed.hex())
h160 = hashlib.new("ripemd160", hashlib.sha256(compressed).digest()).digest()
print("RIPEMD160(SHA256(K)), 20 bytes:", h160.hex())
print("The first byte of an uncompressed 65-byte public key is 0x04; the public key in the
genesis output of Chapter 12 has this form.")
```

```
# Output — bcbook (Python 3.13), exactly as run
p in base 16: FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F
G is on the curve            : True
[n]G is the point at infinity: True
y recovered from the compressed form == y of the uncompressed form: True
compressed public key for d=1:
0279be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798
RIPEMD160(SHA256(K)), 20 bytes: 751e76e8199196d454941c45d1b3a323f1433bd6
The first byte of an uncompressed 65-byte public key is 0x04; the public key in the genesis
output of Chapter 12 has this form.
```

[Intermediate] The numbers in the standards document fit together inside the code. The process of applying two hashes to a 33-byte compressed public key to get the 20 bytes of address material is also shown as it is. The private key 1 is for study. Real wallets use a 256-bit random number as the private key, and this book contains no code that generates or

stores real keys.

5. Summary

The point of this chapter fits in three lines. Bitcoin does not encrypt; it signs. A coin is not a thing but a chain of signatures, and "having" it means "being able to produce the next signature." And the security of a signature rests on the secrecy of the private key and on the quality of the secret number drawn anew for each signature. The next chapter, Chapter 6, looks at the device that ties thousands of signed transactions into a single hash.

Sources for this chapter

- Whitfield Diffie & Martin E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory* IT-22(6): 644–654, November 1976. DOI 10.1109/TIT.1976.1055638 — the requirements of key delivery and signatures (p. 644).
- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. https://bitcoin.org/bitcoin.pdf — coin = chain of signatures (§2, p. 2), a new key for each transaction (§10, p. 6).
- NIST, *Digital Signature Standard (DSS)*, FIPS 186-5, February 2023. DOI 10.6028/NIST.FIPS.186-5 — ECDSA signature generation §6.4.1 (p. 24) and verification §6.4.2 (p. 25), the requirement to protect *k* (p. 24).
- Certicom Research, *SEC 2: Recommended Elliptic Curve Domain Parameters*, Version 2.0, January 27, 2010. https://www.secg.org/sec2-v2.pdf — secp256k1 parameters (§2.4.1, p. 9).
- Neal Koblitz, "Elliptic Curve Cryptosystems," *Mathematics of Computation* 48(177): 203–209, 1987. DOI 10.1090/S0025-5718-1987-0866109-5; Victor S. Miller, "Use of Elliptic Curves in Cryptography," *CRYPTO '85*, LNCS 218, pp. 417–426, Springer, 1986. DOI 10.1007/3-540-39799-X_31 — the original sources of elliptic curve cryptography.
- Pieter Wuille, Jonas Nick & Tim Ruffing, "Schnorr Signatures for secp256k1," BIP 340; Pieter Wuille, Jonas Nick & Anthony Towns, "Taproot: SegWit version 1 spending rules," BIP 341. https://github.com/bitcoin/bips (status Deployed, accessed 2026-10-25) — ECDSA malleability, the advantages of Schnorr signatures, Taproot outputs.
- Bitcoin Core v31.1, `src/kernel/chainparams.cpp` (commit 9be056a8a72b624dae9623b2f7bded92c2a21c91) — Taproot activation parameters.
- Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed., O'Reilly, 2023 (CC BY-SA 4.0, GitHub manuscript commit 94c9ba64) — address = RIPEMD160(SHA256(K)) (chapter 4).
- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library module `hashlib`. Notebook NB05.

Notes

1. Whitfield Diffie & Martin E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory* IT-22(6): 644–654 (November 1976), p. 644. Emphasis added.

2. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), p. 2 (§2, Transactions).

3. Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed. (O'Reilly, 2023), chapter 4: "Starting with the public key K, we compute the SHA256 hash and then compute the RIPEMD-160 hash of the result, producing a 160-bit (20-byte) number: A = RIPEMD160(SHA256(K))". GitHub manuscript commit 94c9ba64, `ch04_keys.adoc`. The human-readable address string is these 20 bytes with a version number and a checksum added, encoded in Base58Check or a similar scheme (same chapter).

4. Nakamoto (2008), p. 6 (§10, Privacy).

5. Pieter Wuille, Jonas Nick & Tim Ruffing, "Schnorr Signatures for secp256k1," BIP 340, Motivation section. The original: "Bitcoin has traditionally used ECDSA signatures over the secp256k1 curve with SHA256 hashes for authenticating transactions. These are standardized, but have a number of downsides compared to Schnorr signatures over the same curve". github.com/bitcoin/bips, file commit 2885f13d (2026-01-12), accessed 2026-10-25. The link on "standardized" points to SEC 1 v2.0 at secg.org, and the link on "secp256k1 curve" points to SEC 2 v2.0. FIPS 186-5 delegates the curves: "NIST-recommended curves for ECDSA are provided in SP 800-186" (FIPS 186-5, main text). This book did not check the list of curves in SP 800-186.

6. NIST, *Digital Signature Standard (DSS)*, FIPS 186-5 (February 3, 2023), §6.4.1 "ECDSA Signature Generation Algorithm," p. 24. Step 9 in the original: "Compute s = k^-1 · (e + r · d) mod n." Step 3 reads "per-message secret number k, 0 < k < n." On the same page: "A value k shall be generated at each invocation of the signature generation algorithm. The private key d and the per-message secret numbers k and k^-1 shall be protected from unauthorized disclosure and modification." DOI 10.6028/NIST.FIPS.186-5. Step 2 in the original: "If len(n) ≥ hashlen, set E = H. Otherwise, set E equal to the leftmost log2(n) bits of H." The table leaves out detailed steps (integer conversion, failure handling) and renumbers the steps.

7. FIPS 186-5, §6.4.2 "ECDSA Signature Verification Algorithm," p. 25. Steps 5 and 6 in the original: "Compute u = e · s^-1 mod n and v = r · s^-1 mod n." "Compute R1 = [u]G + [v]Q."

8. BIP 340, Motivation section, Non-malleability item and its footnote. Emphasis added. The footnote in the original: "If (r,s) is a valid ECDSA signature for a given message and key, then (r,n-s) is also valid for the same message and key. If ECDSA is restricted to only permit one of the two variants (as Bitcoin does through a policy rule on the network), it can be proven non-malleable under stronger than usual assumptions."

9. BIP 340, Abstract ("This document proposes a standard for 64-byte Schnorr signatures over the elliptic curve secp256k1.") and the Provable security, Non-malleability and Linearity items of the Motivation section. Header Status: Deployed.

10. Pieter Wuille, Jonas Nick & Anthony Towns, "Taproot: SegWit version 1 spending rules," BIP 341, Abstract: "This document proposes a new SegWit version 1 output type, with spending rules based on Taproot, Schnorr signatures, and Merkle branches." Status: Deployed. github.com/bitcoin/bips, file commit 24e96e87, accessed 2026-10-25.

11. Bitcoin Core v31.1, `src/kernel/chainparams.cpp` (commit 9be056a8), mainnet parameters: `DEPLOYMENT_TAPROOT` nStartTime 1619222400 ("April 24th, 2021"), nTimeout 1628640000 ("August 11th, 2021"), min_activation_height 709632 ("Approximately November 12th, 2021"). Accessed 2026-10-25.

12. Neal Koblitz, "Elliptic Curve Cryptosystems," *Mathematics of Computation* 48(177): 203–209 (January 1987), p. 203 (§2, Elliptic Curves). In the acknowledgment at the end of the introduction on the same page, Koblitz notes that V. S. Miller independently arrived at "some similar ideas." Miller's paper: Victor S. Miller, "Use of Elliptic Curves in Cryptography," *Advances in Cryptology — CRYPTO '85*, LNCS 218, pp. 417–426 (Springer, 1986). Miller proposed an elliptic curve analogue of Diffie–Hellman key exchange, and Koblitz proposed elliptic curve analogues of public-key systems over the multiplicative group of a finite field (Massey–Omura, ElGamal) (the abstract of each paper).

13. Certicom Research, *SEC 2: Recommended Elliptic Curve Domain Parameters*, Version 2.0 (Standards for Efficient Cryptography Group, January 27, 2010), §2.4.1 "Recommended Parameters secp256k1," p. 9. Hexadecimal values of p, $a = 0$, $b = 7$, G (compressed and uncompressed), n and $h = 1$. https://www.secg.org/sec2-v2.pdf, accessed 2026-10-25. The document permits reproduction on condition that it is credited as "Standards for Efficient Cryptography 2 (SEC 2)."

PART 1 · TECHNICAL FOUNDATIONS

Chapter 6. Merkle trees: vouching for the whole with a single root

KEY TAKEAWAY

Pair up thousands of transactions and hash each pair, pair up those hashes and hash them again, and keep going, and in the end a single hash is left. This one "root" stands in for all the transactions. Change one character of any transaction and the root changes. And to show that one transaction is in the set, you need not the whole set but **only $\lceil \log_2 n \rceil$ hashes**. For 4,000 transactions that is 12. This structure is why an 80-byte block header can represent all the transactions, and why a light wallet that has not downloaded the whole ledger can confirm that "my transaction is in that block."

Learning objectives

1. Draw the structure that summarizes a batch of transactions in a single root hash.
2. **[Intermediate]** Explain why an inclusion proof (Merkle proof) takes only $\lceil \log_2 n \rceil$ hashes, and build and verify a proof in code.
3. Know the two implementation features of the Merkle tree Bitcoin uses (double SHA-256, and duplicating the last item when the count is odd) and the weakness that resulted from them.
4. Separate what the phrase "a single hash vouches for the whole" guarantees from what it does not.

What a reader of the [Basic] paragraphs alone takes away. The Merkle tree is the device that lets you confirm "my transaction is in that block" without downloading everything. But the confirmation goes only as far as "it is in the block." Whether "that block is in the real chain" is a different question.

1. What failure this technology grew out of

Of the 1991 time-stamping schemes of Haber and Stornetta mentioned in Chapter 4, the "linking" scheme strung document hashes together **in a single line**. Each certificate carried information from the previous request, so that nobody could slip anything into the middle or take anything out. That chain is the subject of Chapter 7. But stringing things in a single line had a cost problem. Anyone who wants to check the time of a document has to follow the chain **one link at a time**, from that document to a certificate they trust. If there is one trustworthy certificate for every N documents, verification takes up to N steps.

In their 1993 paper Dave Bayer, Haber and Stornetta put the problem this way: "What if an immense flood of banal transactions want their time-stamps to become part of the historical record, but history just isn't interested?" Their answer was **a tournament**: "We propose to merge many unnoteworthy time-stamping events into one noteworthy event, using a tournament run by its participants. The winner can be easily and widely publicized. Each player, by remembering a short list of opponents, can establish participation in the tournament. We do this by building trees in place of the linked list of the linking solution, thus achieving an exponential increase in the number of observers."[1] They go on to state that such "hash trees" had been used earlier by Ralph Merkle in 1980 **for a different purpose**, namely to produce authentication certificates for a directory of public enciphering keys.[2] The Nakamoto white paper also cites that 1980 paper as the source of the Merkle tree.[3] So this book takes Merkle 1980 as the original source of the Merkle tree. That paper itself describes "tree authentication," which hashes the public file by dividing it in halves, as a way to confirm a single entry with only the root and $\log_2 n$ intermediate values, and it calls the path from the root to the entry, along which the information needed for that confirmation lies, the "authentication path"; but it leaves the detailed discussion to another manuscript, [13].[4] This book takes the 1980 paper as the original source because that is the source the white paper and Bayer et al. pointed to, not because the tree was first presented in that paper. The signature paper Merkle published in 1987 developed the same idea into "an infinite tree of one-time signatures." Each node has three one-time signatures, left, right and message, and is authenticated by `HASH(i)`, which binds the public values of those three with a one-way hash; the verifier works back from a node to its parent until reaching the root.[5]

2. The intuition: a tournament bracket

Suppose there are 8 transactions. Compute the hash of each transaction (8 of them). Pair them up, concatenate each pair, and hash again (4). Pair up and hash again (2). Finally, hash the two together (1). That 1 is the **Merkle root**, the root hash. It has the shape of an inverted tournament bracket: the transactions are the players, and the root is the winner.

This root is a fingerprint of all eight transactions. Change one character of any transaction and that transaction's hash changes, the hash one level up that it was paired into changes, and in the end the root changes. So instead of writing out all the transactions, the block header records **only the root**. The white paper says: "transactions are hashed in a Merkle Tree [7][2][5], with only the root included in the block's hash".[6] This is why the block header is 80 bytes no matter how many transactions there are (Chapter 7).

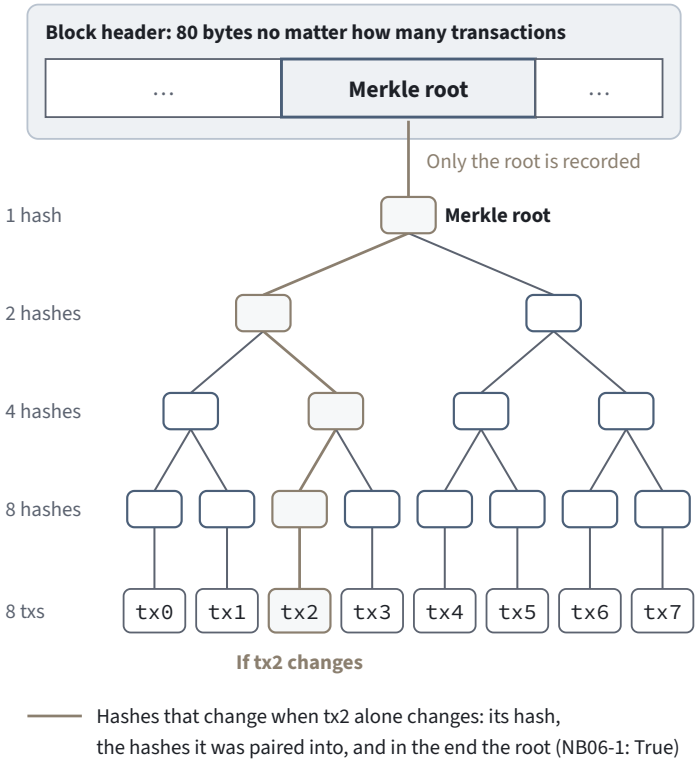


Figure 6-1. A Merkle tree of 8 transactions (the hash counts per level from NB06-1: 8, 4, 2, 1). If tx2 changes, everything from its hash up to the root changes, and only that one root goes into the 80-byte block header.

Inclusion proofs: you only need to remember your opponents

Now you want to show someone that the fifth transaction is in this block. There is no need to show all eight transactions. You only need to hand over **the hashes it was paired with** on the way from the fifth transaction's hash up to the root. For eight transactions that is three. The recipient uses those three to recompute the root and checks whether it matches the root they know. In the words of Bayer, Haber and Stornetta, a participant need only record, along with the document, "the $\lceil \log_2 N \rceil$ hash values that were directly combined with the document's hash value along the path to the root of the tree," together with the **handedness** that shows whether their own value went before or after when combined, and verification consists simply of "recomputing the root of the tree from this data."[7]

This is exactly the "Simplified Payment Verification (SPV)" of §8 of the white paper. A user who does not run a full node keeps only the block headers, obtains "the Merkle branch linking the transaction to the block it's timestamped in," and confirms that the transaction is in the block. The white paper states the limits right away: "He can't check the transaction for himself, but by linking it to a place in the chain, he can see that a network node has accepted it," and "the simplified method can be fooled by an attacker's fabricated transactions for as long as the attacker can continue to overpower the network."[8] A Merkle proof guarantees that "this transaction is in this block." The guarantee that "this block is in the correct chain" is the job of Chapter 8 and Chapter 9.

3. [Intermediate] The length of a proof: why $\log_2 n$

[Intermediate] The height of the tree is the number of times you pair up the n leaves until one is left, so it is $\lceil \log_2 n \rceil$. An inclusion proof takes one sibling hash per level from leaf to root, so its length is also $\lceil \log_2 n \rceil$.

$$\text{proof length} = \lceil \log_2 n \rceil \text{ hashes} = 32 \lceil \log_2 n \rceil \text{ bytes}$$

Reading the symbols: n is the number of transactions in the block, $\lceil \cdot \rceil$ means rounding up, and 32 is the number of bytes in one SHA-256 hash. A numerical example: if $n = 4{,}000$, $\lceil \log_2 4000 \rceil = 12$, and the proof is 12 hashes, 384 bytes. Instead of receiving all 4,000 transactions, the check is done with 384 bytes. If the number of transactions doubles, the proof grows by just one hash.

Root, proof, verification

```
# NB06-1: Merkle tree — computing the root, making an inclusion proof, verifying the proof
(general form: SHA-256 once)
import hashlib, math

def H(b: bytes) -> bytes:
    return hashlib.sha256(b).digest()

def merkle_levels(leaves):
    """Return the hash list of each level, from the bottom level up, given the list of leaf
hashes.
    If a level has an odd count, use the last one once more to make a pair (the same rule as
Bitcoin; see Section 4)."""
    levels = [leaves]
    while len(levels[-1]) > 1:
        cur = levels[-1]
        if len(cur) % 2 == 1:
            cur = cur + [cur[-1]]
        levels.append([H(cur[i] + cur[i + 1]) for i in range(0, len(cur), 2)])
    return levels

def merkle_proof(levels, index):
    """Inclusion proof for leaf number index: a list of (sibling hash, is the sibling on the
right) — the 'handedness' goes with it."""
    proof = []
    for level in levels[:-1]:
        if len(level) % 2 == 1:
            level = level + [level[-1]]
        sibling = index ^ 1                          # sibling's position: the right neighbor if
index is even, the left neighbor if odd
        proof.append((level[sibling], sibling > index))
        index //= 2
    return proof

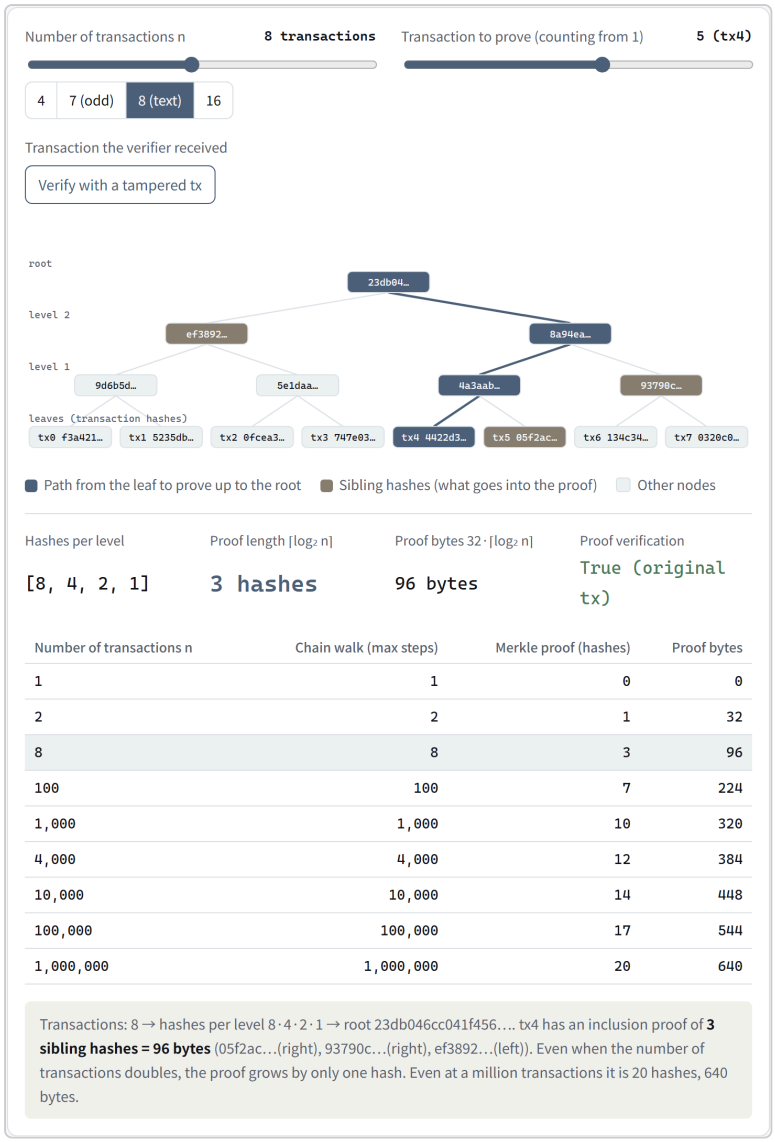
def verify_proof(leaf, proof, root):
    h = leaf
    for sibling, sibling_is_right in proof:
        h = H(h + sibling) if sibling_is_right else H(sibling + h)
    return h == root

txs = [f"tx{i}: Alice pays Bob {i} coin".encode() for i in range(8)]
leaves = [H(t) for t in txs]
levels = merkle_levels(leaves)
root = levels[-1][0]
print("8 transactions → hashes per level:", [len(l) for l in levels], "→ root", root.hex()[:16]
+ "…")

idx = 4                                              # the fifth transaction (4, counting from 0)
proof = merkle_proof(levels, idx)
print("length of the inclusion proof for tx4:", len(proof), "hashes =", 32 * len(proof),
"bytes")
print("verify proof (original tx4):", verify_proof(leaves[idx], proof, root))
print("verify proof (tampered tx4):", verify_proof(H(b"tx4: Alice pays Bob 40 coin"), proof,
root))

tampered = leaves[:]; tampered[2] = H(b"tx2: Alice pays Bob 200 coin")
print("does changing tx2 change the root:", merkle_levels(tampered)[-1][0] != root)
```

```
# Output — bcbook (Python 3.13), exactly as run
8 transactions → hashes per level: [8, 4, 2, 1] → root 23db046cc041f456…
length of the inclusion proof for tx4: 3 hashes = 96 bytes
verify proof (original tx4): True
verify proof (tampered tx4): False
does changing tx2 change the root: True
```



Number of transactions n	Chain walk (max steps)	Merkle proof (hashes)	Proof bytes
1	1	0	0
2	2	1	32
8	8	3	96
100	100	7	224
1,000	1,000	10	320
4,000	4,000	12	384
10,000	10,000	14	448
100,000	100,000	17	544
1,000,000	1,000,000	20	640

Transactions: 8 → hashes per level 8 · 4 · 2 · 1 → root 23db046cc041f456…. tx4 has an inclusion proof of **3 sibling hashes = 96 bytes** (05f2ac…(right), 93790c…(right), ef3892…(left)). Even when the number of transactions doubles, the proof grows by only one hash. Even at a million transactions it is 20 hashes, 640 bytes.

Figure 6-2. Merkle inclusion proof. In a tree of 8 transactions, the inclusion proof for the fifth transaction (tx4) is 3 sibling hashes, 96 bytes. When the number of transactions doubles, the proof grows by one hash. — In the HTML edition you can change the values here (HTML edition, Chapter 6, Section 3).

Proof length: chain versus tree

```
# NB06-2: how much does a proof grow as the number of transactions grows — a single-line chain
(N steps) versus a tree (log N)
import math
print(f"{'tx count n':>12} {'chain walk (max steps)':>22} {'Merkle proof hashes':>20}
{'bytes':>10}")
for n in [1, 2, 8, 100, 1_000, 4_000, 10_000, 100_000, 1_000_000]:
    k = math.ceil(math.log2(n)) if n > 1 else 0
    print(f"{n:>12,} {n:>22,} {k:>20} {32*k:>10,}")
```

```
# Output — bcbook (Python 3.13), exactly as run
  tx count n chain walk (max steps)  Merkle proof hashes      bytes
          1                       1                     0         0
          2                       2                     1        32
          8                       8                     3        96
        100                     100                     7       224
      1,000                   1,000                    10       320
      4,000                   4,000                    12       384
     10,000                  10,000                    14       448
    100,000                 100,000                    17       544
  1,000,000               1,000,000                    20       640
```

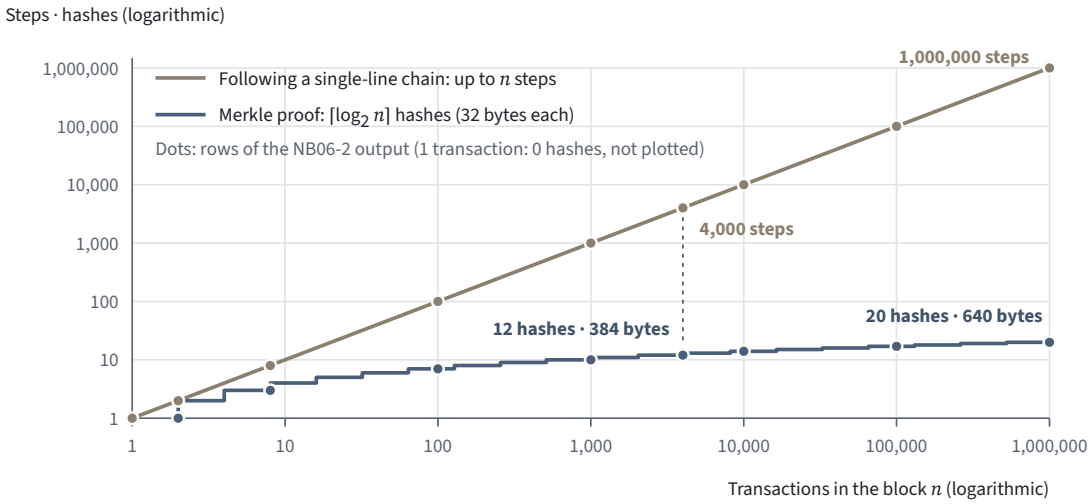


Figure 6-3. Number of verification steps against the number of transactions n. Both axes are logarithmic; the Merkle line is the graph of $\lceil \log_2 n \rceil$ from the text, and the dots are the output of NB06-2.

[Intermediate] Even at a million transactions the proof is 20 hashes, 640 bytes. This table is what Bayer, Haber and Stornetta meant when they wrote of reducing the cost "from N to log$N$." The pruning in §7 of the white paper comes from the same structure. When old transactions are discarded, the tree's branches are "stubbed off" and only the hashes at those spots are kept; the root stays the same, so the block hash is not broken.

4. Bitcoin's Merkle tree: two implementation features and one weakness

Bitcoin's tree has two implementation features. It hashes with SHA-256 **twice** (Chapter 4, Section 5; the general-form code above hashes once), and whenever the number of hashes at any level is **odd, it duplicates the last one** to make a pair (the general-form code above used this same rule).[9] Textbooks explain this by saying that a binary tree needs an even number of leaves, but this duplication made it possible for **different transaction lists to produce the same root**. A long comment in the Bitcoin Core source calls this a "serious flaw" and gives it the vulnerability number CVE-2012-2459. The transaction lists [1,2,3,4,5,6] and [1,2,3,4,5,6,5,6] produce the same root. This is because, in the list where the last two (5,6) are duplicated, the odd-duplication rule makes (F) and (F,F) come out as the same value.[10] If an attacker sends a block with such a duplicated list, a node receives a block "whose root is right but whose transactions are wrong," and if the node marks that block as permanently invalid it will end up rejecting the real block too. Core detects and blocks the case where two identical hashes end up being paired at the end of a list.

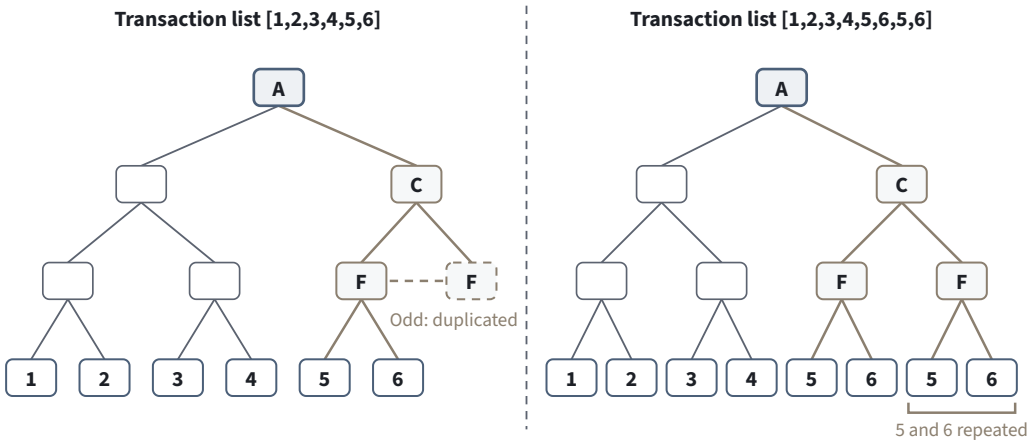


Figure 6-4. CVE-2012-2459. Because of the rule that duplicates the last hash on a level with an odd count, (F) and (F,F) both hash to C, so two different transaction lists produce the same root A (NB06-3).

```
# NB06-3: Bitcoin-style Merkle tree — double SHA-256, odd duplication, and the "same root" of
CVE-2012-2459
import hashlib

def H2(b: bytes) -> bytes:                          # Bitcoin's hash: SHA256(SHA256(x))
    return hashlib.sha256(hashlib.sha256(b).digest()).digest()

def bitcoin_merkle_root(txids):
    """Compute the root from a list of txids (internal byte order). If the count is odd,
duplicate the last one."""
    layer = list(txids)
    while len(layer) > 1:
        if len(layer) % 2 == 1:
            layer.append(layer[-1])
        layer = [H2(layer[i] + layer[i + 1]) for i in range(0, len(layer), 2)]
    return layer[0]

ids = [H2(bytes([i])) for i in range(1, 7)]        # fake txids for transactions 1-6
root_a = bitcoin_merkle_root(ids)
root_b = bitcoin_merkle_root(ids + ids[4:6])       # [1,2,3,4,5,6,5,6]
print("[1..6] root    :", root_a.hex())
print("[1..6,5,6] root:", root_b.hex())
print("same root for both lists:", root_a == root_b, " ← different transaction lists, same
root")

# in a block with only one transaction (the genesis block is one), root = that transaction's
txid
one = H2(b"only one transaction")
print("with 1 transaction, root == txid :", bitcoin_merkle_root([one]) == one, "(checked with
real data in Chapter 12)")
```

```
# Output — bcbook (Python 3.13), exactly as run
[1..6] root    : df316ddf151a6d7f65a868df97de2168ed674371119b038bb97c9926b3f7948d
[1..6,5,6] root: df316ddf151a6d7f65a868df97de2168ed674371119b038bb97c9926b3f7948d
same root for both lists: True  ← different transaction lists, same root
with 1 transaction, root == txid : True (checked with real data in Chapter 12)
```

[Intermediate] The other weakness is **64-byte transactions**. An inner node of the Merkle tree is the hash of 64 bytes, two 32-byte hashes concatenated, and some Bitcoin transactions also come to exactly 64 bytes when serialized. A 64-byte transaction can then be read either as a "leaf" or as an "inner node," which makes it possible to get an SPV verifier to accept an inclusion proof for a transaction that is not in the block. The Bitcoin Improvement Proposal BIP-54 ("Consensus Cleanup") contains a rule that removes this weakness by making 64-byte transactions invalid.[11] As of 2026-10-25 this proposal's status is "Complete": it is ready for adoption, but this is not the mark that it has been activated ("Deployed"). Whether it has been activated is updated in the trend notes of Appendix A. This is what someone who wants to use a Merkle proof as legal evidence needs to know. **The strength of a proof depends on the implementation details of the tree, those details have had known holes (one was blocked in the implementation, and a fix has been proposed for the other), and they can change.**

5. Summary: what the root vouches for

A single Merkle root is a fingerprint of all the transactions. If any transaction changes, the root changes, and that a transaction is included can be shown with $\lceil \log_2 n \rceil$ hashes. What this guarantees is that "this transaction is in a block with this root." The root does not tell you whether that block is in the chain everyone follows, or whether that chain will not be overturned. The next chapter, Chapter 7, looks at how the 80-byte headers that carry this root become a chain.

Sources for this chapter

- Dave Bayer, Stuart Haber & W. Scott Stornetta, "Improving the Efficiency and Reliability of Digital Time-Stamping," in *Sequences II: Methods in Communication, Security, and Computer Science*, pp. 329–334. Springer, 1993. DOI 10.1007/978-1-4613-9323-8_24 — the tournament analogy and the precedent of Merkle 1980 (p. 330), tree proofs and handedness (§3, pp. 3–4 of the authors' typeset version).
- Ralph C. Merkle, "Protocols for Public Key Cryptosystems," *Proc. 1980 IEEE Symposium on Security and Privacy*, pp. 122–134 (DOI 10.1109/SP.1980.10006) — the work this book takes as the original source of the Merkle tree. Pp. 126, 127, 133, 134. Copy from the author's website (a scan whose text layer is OCR; the quotations were checked against the page images) (the white paper's references give the pages as 122–133).
- Ralph C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," *CRYPTO '87*, LNCS 293, pp. 369–378. Springer, 1988. DOI 10.1007/3-540-48184-2_32 — the tree of one-time signatures (a later development).
- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — the Merkle tree and pruning (§7, p. 4), simplified payment verification (§8, p. 5), reference [7].
- Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed., O'Reilly, 2023 (CC BY-SA 4.0, GitHub manuscript commit 94c9ba64) — the odd-duplication rule and the CVE-2012-2459 sidebar (chapter 11).
- Antoine Poinsot & Matt Corallo, "Consensus Cleanup," BIP 54. https://github.com/bitcoin/bips/blob/master/bip-0054.md (status Complete, accessed 2026-10-25) — the 64-byte transaction weakness.
- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library modules `hashlib` and `math`. Notebook NB06.

Notes

1. Dave Bayer, Stuart Haber & W. Scott Stornetta, "Improving the Efficiency and Reliability of Digital Time-Stamping," in R. Capocelli, A. De Santis & U. Vaccaro (eds.), *Sequences II: Methods in Communication, Security, and Computer Science* (Springer, 1993), pp. 329–334. Quoted from p. 330 (§1, Introduction). In the authors' typeset version (dated March 1992, 6 pages) the passage is on p. 2.
2. Same article, p. 330. The original: "Such hash trees were previously used by Merkle [Merk 80] for a different purpose, to produce authentication certificates for a directory of public enciphering keys." [Merk 80] = R. C. Merkle, "Protocols for public key cryptosystems," *Proc. 1980 Symposium on Security and Privacy*, IEEE

Computer Society (April 1980).

3. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), p. 4 (§7) and p. 9 (reference [7]: R.C. Merkle, "Protocols for public key cryptosystems," In Proc. 1980 Symposium on Security and Privacy, IEEE Computer Society, pages 122–133, April 1980). The white paper cites [7], [2] and [5] together for the Merkle tree. The pages of the original are 122–134 (p. 134 holds the end of the references and Figure 1).

4. Ralph C. Merkle, "Protocols for Public Key Cryptosystems," *Proc. 1980 IEEE Symposium on Security and Privacy*, pp. 122–134 (DOI 10.1109/SP.1980.10006), p. 126. The original: "Fortunately, it is possible to selectively authenticate individual entries in the public file without having to know the whole public file by using Merkle's "tree authentication," [13]. The essence of tree authentication is to authenticate the entire public file by "divide and conquer."" On the same page: "By applying this concept recursively, A can confirm Y_B in the public file knowing only R, $\log_2 n$ intermediate values, and Y_B itself." It continues: "The information needed to authenticate Y_B, given that R has already been authenticated, lies along the path from R to Y_B and will be called the authentication path." P. 127: "For a more detailed discussion the reader is referred to [13]." Reference [13] on p. 133: "Merkle, R.C. A certified digital signature, to appear, CACM." A paper with the same title was presented at CRYPTO '89 and published in 1990: R. C. Merkle, "A certified digital signature," in G. Brassard (ed.), *Advances in Cryptology — CRYPTO '89*, LNCS 435, pp. 218–238 (Springer, 1990). This citation was confirmed from reference [244] of Babbush et al. (2026) and from the citation of the same volume in the references of Haber and Stornetta (1991) (Appendix E); the paper itself was not opened. Figure 1 (p. 134) draws the authentication path of Y_5 in a tree with 8 leaves. Copy from the author's website (a scan whose text layer is OCR; the quotations were checked against the page images).

5. Ralph C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," *CRYPTO '87*, LNCS 293 (Springer, 1988), pp. 373–375 (section heading "AN INFINITE TREE OF ONE-TIME SIGNATURES"). Each node has "a 'left' signature, a 'right' signature, and a 'message' signature" (p. 373), and the "one way hash total" of node i, $\mathrm{HASH}(i) = F(F(y[i, \text{left}, *]), F(y[i, \text{right}, *]), F(y[i, \text{message}, *]))$, binds the public values of that node's own three signatures (p. 374). Verification proceeds from node i back to i / 2 until it reaches the root (node 1, HASH(1) in the public file) (p. 375). This paper is a "later development," and its use differs from that of Bitcoin's transaction tree.

6. Nakamoto (2008), p. 4 (§7, Reclaiming Disk Space). The original: "transactions are hashed in a Merkle Tree [7] [2][5], with only the root included in the block's hash. Old blocks can then be compacted by stubbing off branches of the tree. The interior hashes do not need to be stored."

7. Bayer, Haber & Stornetta (1993), §3, Trees: pp. 3–4 of the authors' typeset version (this corresponds to pp. 331–332 of the published version, but since this book checked the published pagination only up to p. 330, it cites the pages of the authors' typeset version). The original: "To later establish priority, a participant need only record his own document, as well as the $\lceil \log_2 N \rceil$ hash values that were directly combined with the document's hash value along the path to the root of the tree. In addition, along with each combining hash value, the user needs to record its 'handedness,' indicating whether the newly computed value was placed before or after the combining hash value. Verification consists simply of recomputing the root of the tree from this data."

8. Nakamoto (2008), p. 5 (§8, Simplified Payment Verification). The original: "A user only needs to keep a copy of the block headers of the longest proof-of-work chain ... and obtain the Merkle branch linking the transaction to the block it's timestamped in. He can't check the transaction for himself, but by linking it to a place in the chain, he can see that a network node has accepted it ... the simplified method can be fooled by an attacker's fabricated transactions for as long as the attacker can continue to overpower the network."

9. Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed. (O'Reilly, 2023), chapter 11: "If there are an odd number of transactions to summarize, the last transaction hash will be duplicated to create an even number of leaf nodes ... Similarly, if there are an odd number of hashes to process at any level, the last hash is duplicated." GitHub manuscript commit 94c9ba64, `ch11_blockchain.adoc`.

10. Same chapter, sidebar "A Design Flaw in Bitcoin's Merkle Tree," which reproduces the Bitcoin Core source comment with light editing. The original: "the following merkle tree algorithm has a serious flaw related to duplicate txids, resulting in a vulnerability (CVE-2012-2459). The reason is that if the number of hashes in the list at a given level is odd, the last one is duplicated before computing the next level (which is unusual in merkle trees). This results in certain sequences of transactions leading to the same merkle root. ... The transaction lists [1,2,3,4,5,6] and [1,2,3,4,5,6,5,6] (where 5 and 6 are repeated) result in the same root hash A

(because the hash of both of (F) and (F,F) is C)."

11. Antoine Poinsot & Matt Corallo, "Consensus Cleanup," BIP 54 (Layer: Consensus (soft fork), Status: Complete, Assigned 2025-04-11), Motivation section. The original: "In computing a block's Merkle root, a transaction with exactly 64 bytes of non-witness data can be interpreted both as an intermediate node in the tree and as a leaf in the tree. This makes it possible to trick an SPV verifier into accepting an inclusion proof for a transaction that is not part of a block, by pretending a 64-byte block transaction is actually an inner node." github.com/bitcoin/bips, file commit 927b6de9 (2026-10-02), accessed 2026-10-25. The meaning of the status "Complete" follows BIP 3.

PART 1 · TECHNICAL FOUNDATIONS

Chapter 7. Blocks and the chain: a structure that makes tampering costly

KEY TAKEAWAY

A block is an 80-byte head (the header) plus a list of transactions. The header contains **the hash of the previous block's header**, so the blocks become a chain threaded together by hashes. Change even one character of any past block and that block's hash changes, the header of the next block, which points to it, becomes wrong, and so does the one after that. Tampering becomes a job of remaking every block that follows. But **with a hash chain alone, the remaking itself is free**. The toy chain in this chapter shows this. What makes "remaking" costly is the proof of work of the next chapter, and so the immutability of a blockchain is not a mathematical impossibility but **an economic cost**.

Learning objectives

1. Explain the six fields of the block header and how the "previous block hash" forms a chain.
2. Distinguish "tampering is impossible" from "tampering shows" and "the cost of tampering grows the further back it goes."
3. **[Intermediate]** Build a toy chain and confirm in code that altering a block in the middle breaks everything after it, and that without proof of work relinking all of it is possible at once.
4. Know what the hash chain was originally invented for (time-stamping).

What a reader of the [Basic] paragraphs alone takes away. The immutability of a blockchain is not a mathematical impossibility but **an economic cost**. This is the basis for misconception ① in Appendix B ("A blockchain cannot be tampered with").

1. What failure this technology grew out of

How do you prove that a document existed "at that time"? In their 1991 paper, Haber and Stornetta first examine the simplest answer, a "digital safety-deposit box." You send the document to a time-stamping service (TSS), and the service records the time it received it and keeps a copy. But this scheme came with four worries: privacy (the service sees the document), bandwidth and storage (large documents are costly), incompetence (the service may make a mistake or lose the copy), and **trust**: "nothing in this scheme prevents the TSS from colluding with a client in order to claim to have time-stamped a document for a date and time different from the actual one."[1] The first three are solved with hashes and signatures (Chapter 4, Chapter 5). The fourth remains: "But who will guard the guards themselves?"[2]

Their answer is **linking.** Into each certificate the service issues, it puts the information from the previous request (time, client number, hash) together with the hash of the previous linking information, and signs it. Then the service **cannot stamp a time later than the actual one** (no forward-dating). The certificate would have to contain bits from the requests that came in just before that time, and the service has not yet received them. **Nor can it stamp a time earlier than the actual one** (no back-dating). The certificates right after that earlier time would have to contain bits from the document, but those certificates have already been issued without them. Slipping a new document into a stream that has already been issued would require computing a collision of the hash function.[3] A chain in which each certificate points to the previous one. This is the skeleton of the blockchain, and §3 of the white paper, citing these papers, draws the same picture: "Each timestamp includes the previous timestamp in its hash, forming a chain, with each additional timestamp reinforcing the ones before it."[4]

In the first sentence of the introduction to their 1993 paper, Bayer, Haber and Stornetta set out the basis of this idea in a line: "Causality fixes events in time. If an event was determined by certain earlier events, and determines certain subsequent events, then the event is sandwiched securely into its place in history."[5] In their words, paper documents "have forensic qualities allowing them to be accurately dated and examined for signs of after-the-fact tampering," whereas with digital documents "tampering may not leave any tell-tale signs in the medium." The hash chain is a device that **artificially plants causality** in digital data.

2. What a block is made of

A Bitcoin block has two parts: an 80-byte **header** and **a list of transactions.** The transaction list makes up most of the block's size, and the header is 80 bytes however many transactions there are.[6] The white paper also says that "A block header with no transactions would be about 80 bytes."[7] The six fields of the header are as follows.[8]

SIZE	FIELD	MEANING
4 bytes	Version	Originally a version number. Its use has changed over time
32 bytes	Previous Block Hash	Hash of the header of the immediately preceding (parent) block
32 bytes	Merkle Root	Root of the Merkle tree of this block's transactions (Chapter 6)
4 bytes	Timestamp	Approximate time the block was created (Unix time)
4 bytes	Target (nBits)	Compact representation of the proof-of-work target (Chapter 8)
4 bytes	Nonce	An arbitrary number used for proof of work

A block's "name" is the hash of its header. The 32 bytes obtained by hashing the 80-byte header twice with SHA-256 are **the block hash**, or more precisely **the block header hash**. The transactions enter only indirectly, through the Merkle root.[9] And this very hash is written in the second slot of the next block's header. That is all there is to the "chain." The header of block B_k contains the hash of B_{k-1}, and the header of B_{k-1} contains the hash of B_{k-2}. It goes back like this to the genesis block.

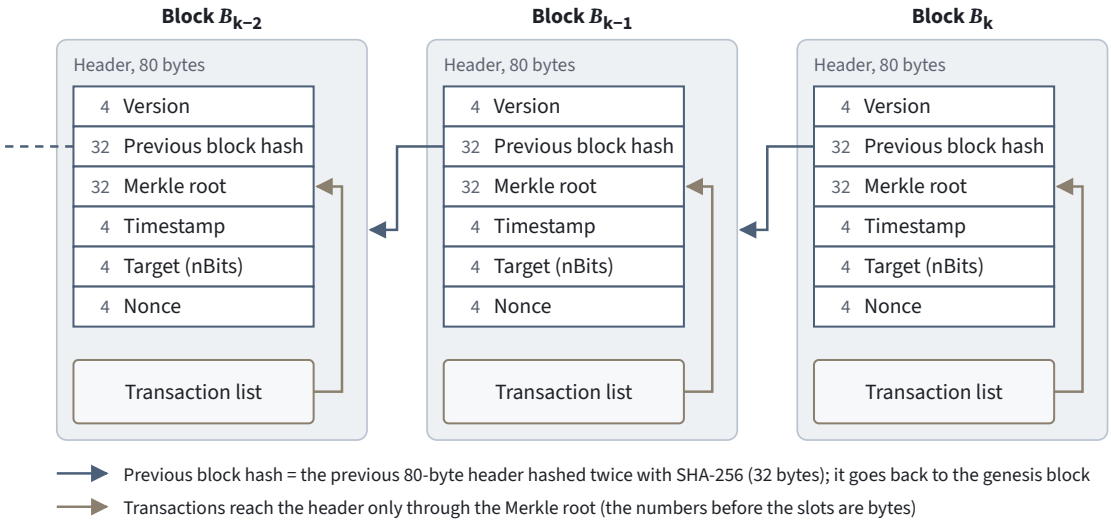


Figure 7-1. Three blocks and their links. The second slot of each block header holds the hash of the previous block's header, and the transaction list reaches the header only through the single Merkle root slot.

3. Why tampering shows up in a cascade

§4 of the white paper sums it up in two sentences: "Once the CPU effort has been expended to make it satisfy the proof-of-work, the block cannot be changed without redoing the work. As later blocks are chained after it, the work to change the block would include redoing all the blocks after it."[10] Two things have to be read apart here.

First, **showing.** Alter one character of a transaction in block B_k and its Merkle root changes, its header changes, and the hash of its header changes. But the header of B_{k+1} still has the old hash written in it, so B_{k+1} no longer points to B_k. Anyone can see that the chain has been broken. A hash chain alone is enough for this. Proof of work is not needed.

Second, **cost.** To relink the broken chain, you have to write the new hash into the header of

B_{k+1}, which changes the hash of B_{k+1}, so B_{k+2} has to be fixed too. All the way to the end. But **computing a hash costs little and is fast.** One hash per block is enough, so with a ledger that has only a hash chain, a tamperer can re-thread the whole thing in an instant and present a "new chain." That chain is perfectly consistent internally. All that shows is the fact that "it differs from the original chain," and the hash does not tell you which of the two is the original.

The "CPU effort" in the white paper's quotation is exactly the device that fills this gap. If for every block one had to **hash an astronomical number of times, changing the nonce,** until the header's hash meets some condition (as of 2026-10, about 6×10^{23} attempts expected per block; Chapter 8), then re-threading would take that much computation all over again. The more blocks are stacked on top, the more there is to re-thread. That is why the proposition "the cost of tampering grows the further back it goes" holds. This is the subject of the next chapter, and how large that cost is, and what happens if someone can pay it, we see in Chapter 9 and Chapter 20.

The "longest chain" rule

If two or more chains appear, which one is canonical? §5 of the white paper says: "Nodes always consider the longest chain to be the correct one and will keep working on extending it." If two nodes put out different blocks at the same time, each works on the one it received first but keeps the other branch too, and when the next proof of work is found and one branch becomes longer, the nodes that were working on the other branch switch to the longer one.[11] Current implementations read "longest" as "with the most accumulated proof of work." Meni Rosenfeld put it this way in his 2014 paper: "It is agreed that every node will consider the longest branch it is aware of as the valid chain (more precisely, the branch representing the most proof of work)."[12] Over a stretch where the difficulty is the same, the two coincide.

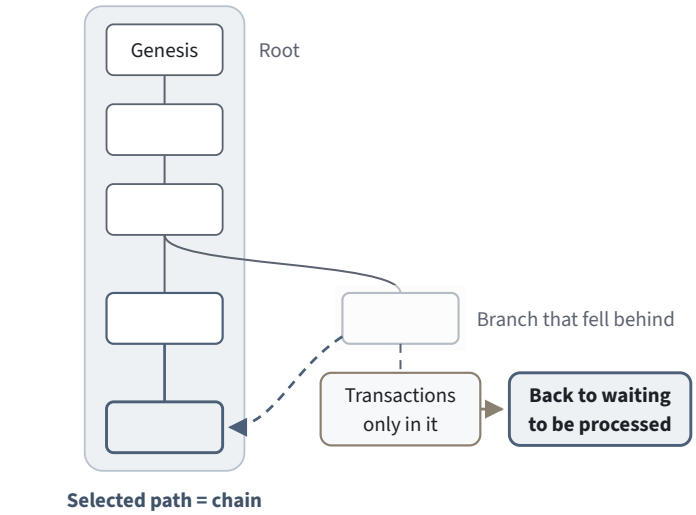


Figure 7-2. In a block tree rooted at the genesis block, the branches split, and the nodes move over to the side that grows longer when the next block is attached. The selected path is the chain, and transactions that were only in the branch that fell behind go back to waiting to be processed.

So the "real" blockchain is not registered anywhere. It is decided from moment to moment **by which branch the participants who follow the rules keep building on.** This is what lawyers find unfamiliar, and it is why, when a hard fork occurs, a situation arises in which both chains claim to be the "real" one (Chapter 23, Chapter 26).

> **? A LAWYER'S QUESTION**
>
> Someone says that a transaction containing their personal information has been recorded on a blockchain and asks for it to be deleted. The chain is designed so that it "cannot be altered," and altering it would require a great many participants to remake every block that follows. What does the right to deletion of personal information (개인정보 삭제권) end up demanding on a ledger that cannot be erased? Against whom is deletion to be demanded (삭제의 상대방), and what act, technically, does "deleting" refer to? Does a design that keeps personal information off the chain from the start and puts only a hash on it solve this problem, or postpone it? → Chapter 28

4. Code: a toy chain

Tampering and re-threading

[Intermediate] The toy chain below has no proof of work. It puts only the previous hash, the data and the time into the header and threads them with SHA-256. This lets you see "showing" and "free re-threading" separately. In Chapter 8, proof of work is attached to this class by adding one line.

```
# NB07-1: toy blockchain — a hash chain without proof of work. Tampering shows, but re-threading
is free (for study)
import hashlib, json

def H(s: str) -> str:
    return hashlib.sha256(s.encode()).hexdigest()

class Block:
    def __init__(self, index, prev_hash, data, timestamp):
        self.index, self.prev_hash, self.data, self.timestamp = index, prev_hash, data,
timestamp
    def header(self) -> str:                        # header = the part that goes into the chain
        return json.dumps([self.index, self.prev_hash, H(self.data), self.timestamp])
    def hash(self) -> str:                          # block hash = hash of the header (the data
enters only as H(data), in the place of the Merkle root)
        return H(self.header())

def build_chain(datas):
    chain = [Block(0, "0" * 64, datas[0], 1231006505)]            # genesis
    for i, d in enumerate(datas[1:], start=1):
        chain.append(Block(i, chain[-1].hash(), d, 1231006505 + 600 * i))
    return chain

def validate(chain):
    """Check that each block's prev_hash equals the actual hash of the previous block. Return
the number of the first wrong block (None if there is none)."""
    for i in range(1, len(chain)):
        if chain[i].prev_hash != chain[i - 1].hash():
            return i
    return None

datas = ["genesis", "A→B 1", "B→C 2", "C→D 3", "D→E 4", "E→F 5"]
chain = build_chain(datas)
print("number of blocks:", len(chain), "| valid?", validate(chain) is None)
for b in chain:
    print(f"  #{b.index} prev={b.prev_hash[:10]}… hash={b.hash()[:10]}… data={b.data!r}")

print("\n--- change the transaction in block 3 from '3' to '300' ---")
chain[3].data = "C→D 300"
bad = validate(chain)
print("first broken link: block #", bad, "(block 3 was changed, yet block 4 is reported wrong:
block 4's prev_hash is the old hash)")
for b in chain[3:]:
    print(f"  #{b.index} prev={b.prev_hash[:10]}… actual previous hash={chain[b.index-1].hash()
[:10]}…  match={b.prev_hash == chain[b.index-1].hash()}")

print("\n--- the tamperer rewrites the prev_hash of every later block ---")
for i in range(4, len(chain)):
    chain[i].prev_hash = chain[i - 1].hash()
print("re-threaded chain valid?", validate(chain) is None, "← without proof of work, relinking
the whole chain takes one hash per block")
```

```
# Output — bcbook (Python 3.13), exactly as run
number of blocks: 6 | valid? True
  #0 prev=0000000000… hash=475e9da5b5… data='genesis'
  #1 prev=475e9da5b5… hash=e9668fba8f… data='A→B 1'
  #2 prev=e9668fba8f… hash=dcdec3cf8e… data='B→C 2'
  #3 prev=dcdec3cf8e… hash=daa1496cf4… data='C→D 3'
  #4 prev=daa1496cf4… hash=0e85befdef… data='D→E 4'
  #5 prev=0e85befdef… hash=3c35f3caa0… data='E→F 5'

--- change the transaction in block 3 from '3' to '300' ---
first broken link: block # 4 (block 3 was changed, yet block 4 is reported wrong: block 4's
prev_hash is the old hash)
  #3 prev=dcdec3cf8e… actual previous hash=dcdec3cf8e…  match=True
  #4 prev=daa1496cf4… actual previous hash=a1b2cd168c…  match=False
  #5 prev=0e85befdef… actual previous hash=0e85befdef…  match=True

--- the tamperer rewrites the prev_hash of every later block ---
re-threaded chain valid? True ← without proof of work, relinking the whole chain takes one hash
per block
```

[Intermediate] Three things are visible. Tampering shows up **in the very next block** (alter block 3 and the link from block 4 breaks). If you rewrite everything after the break, the chain becomes "valid" again. And that job is done at once. What a hash chain guarantees is **consistency**, not **history**. Without a device that decides which of two consistent chains is the history, "tamper-proof" is an empty word.

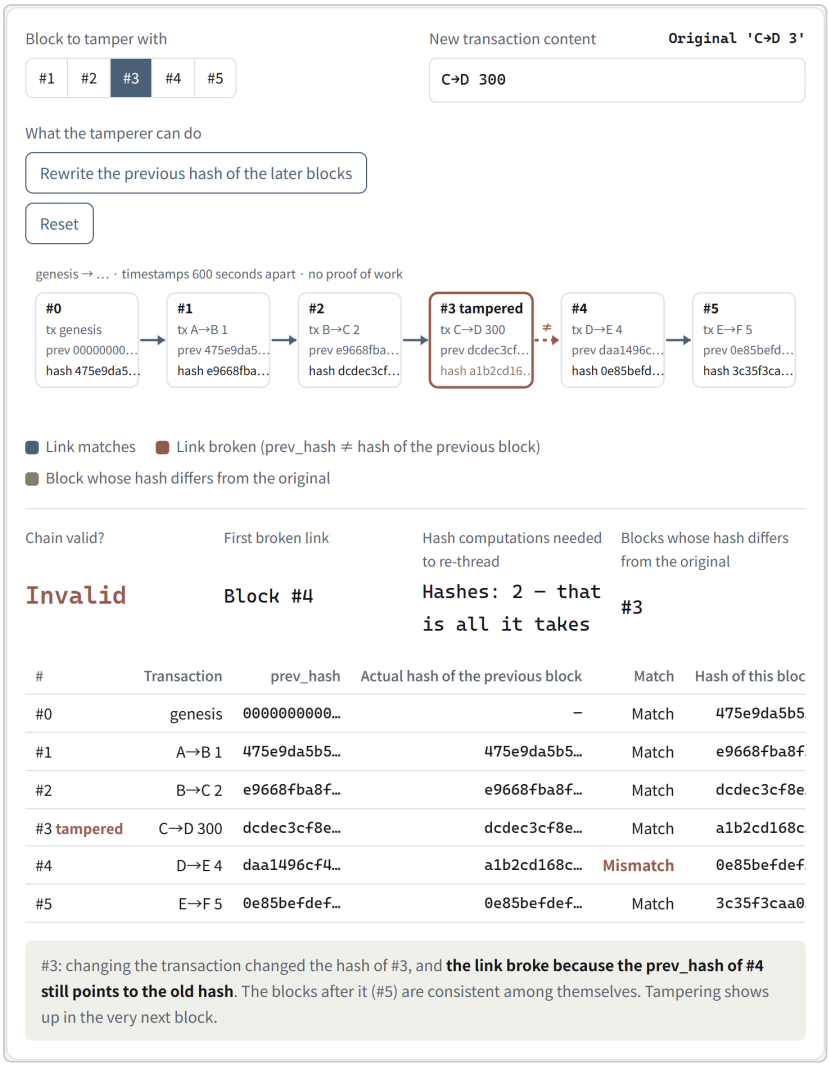


Figure 7-3. Chain tampering demonstration. Change the transaction in block 3 from "C→D 3" to "C→D 300" and the previous-hash field of block 4 no longer matches, so the link breaks. Without proof of work, re-threading everything after the break takes one hash per block. — In the HTML edition you can change the values here (HTML edition, Chapter 7, Section 4).

When both chains are "valid"

```
# NB07-2: when both chains are "valid" — hashes alone cannot tell which one is the original
original = build_chain(datas)
forged = build_chain([d if i != 3 else "C→D 300" for i, d in enumerate(datas)])
print("original chain valid?", validate(original) is None, "| forged chain valid?",
validate(forged) is None)
print("last block hash — original:", original[-1].hash()[:16], "… forged:", forged[-1].hash()
[:16], "…")
print("The two chains differ, but each is perfectly consistent internally. Hashes cannot answer
'which one is real?'")
print("What came in to answer this question is the proof of work (cost) of Chapter 8 and the
rule of the 'chain with the most accumulated work'.")
```

```
# Output — bcbook (Python 3.13), exactly as run
original chain valid? True | forged chain valid? True
last block hash — original: 3c35f3caa070a93e … forged: 63f530852e4eed45 …
The two chains differ, but each is perfectly consistent internally. Hashes cannot answer 'which
one is real?'
What came in to answer this question is the proof of work (cost) of Chapter 8 and the rule of
the 'chain with the most accumulated work'.
```

▾ [Advanced] The chain is a tree: forks and branch selection

[Advanced] "Chain" is the name for how things usually look. Since each block points to only one hash, that of the previous block, the structure is **a tree whose root is the genesis block**, and the chain is one path selected in that tree. Rosenfeld's description is exact: "The blocks hence form a tree, with the genesis block as the root, and each block being a child of the block it references. A branch in this tree is a path from a leaf block to the genesis block; each such branch represents one version of the history of Bitcoin transactions. Each branch must be internally consistent and can never include two conflicting transactions; however, the branches need not be consistent with one another …."[13] If two nodes find a block at almost the same time, the tree gets two branches (a fork), and when the next block is attached on top of one of them, that branch becomes longer and everyone moves over to it. The branch that fell behind drops out of the valid chain, and the transactions that were only in that branch go back to waiting to be processed. How often, and how deep, such reversals can happen is the mathematics of Chapter 9.

[Advanced] Viewed as a tree, the two chains of NB07-2 above are two branches that split at block 2. In a proof-of-work system, the accumulated work of the two branches is compared and the larger is chosen, and the work is computed from the difficulty target of each block. It is "weight," not "length."

5. Summary

A block is an 80-byte header and a list of transactions, and the "previous block hash" in the header forms the chain. Thanks to the chain, tampering with the past always shows. But if re-threading the chain to hide the tampering takes only hash computations, the cost is close to 0. The real content of the phrase "tamper-proof" is "re-threading is costly, and grows costlier the further back it goes," and the device that creates that cost is the proof of work of the next chapter. The one sentence to remember from this chapter is this. **A hash chain proves consistency, not history. History is proved by cost.**

Sources for this chapter

- Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *Journal of Cryptology* 3(2): 99–111, 1991. DOI 10.1007/BF00196791 — the four worries of the naive solution (p. 101), the linking scheme and its argument (pp. 103–104).
- Dave Bayer, Stuart Haber & W. Scott Stornetta, "Improving the Efficiency and Reliability of Digital Time-Stamping," *Sequences II*, pp. 329–334. Springer, 1993. DOI 10.1007/978-1-4613-9323-8_24 — "Causality fixes events in time" (p. 330).
- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — timestamp server (§3, p. 2), the work needed for tampering (§4, p. 3), the longest-chain rule (§5, p. 3), the 80-byte header (§7, p. 4).
- Meni Rosenfeld, "Analysis of Hashrate-Based Double Spending," arXiv:1402.2009, 2014 — the block tree and branch selection (§2, pp. 2–3).
- Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed., O'Reilly, 2023 (CC BY-SA 4.0, GitHub manuscript commit 94c9ba64) — the 80-byte header and its six fields, the block header hash (chapter 11).
- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library modules `hashlib` and `json`. Notebook NB07.

Notes

1. Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *Journal of Cryptology* 3(2): 99–111 (1991), p. 101 (§3, A Naive Solution). The original: "Trust. The fundamental problem remains: nothing in this scheme prevents the TSS from colluding with a client in order to claim to have time-stamped a document for a date and time different from the actual one."

2. Same article, p. 103 (epigraph of §5). It quotes Juvenal's "Sed quis custodiet ipsos Custodes?" and renders it as "But who will guard the guards themselves?"

3. Same article, p. 104 (§5.1, Linking). The original: "the TSS cannot forward-date a document, because the certificate must contain bits from requests that immediately preceded the desired time, yet the TSS has not received them. The TSS cannot feasibly back-date a document by preparing a fake time-stamp for an earlier time, because bits from the document in question must be embedded in certificates immediately following that earlier time, yet these certificates have already been issued. Furthermore, correctly embedding a new document into the already-existing stream of time-stamp certificates requires the computation of a collision for the hash function H."

4. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), p. 2 (§3, Timestamp Server). In this section the white paper cites [2-5], and [3] and [4] are Haber & Stornetta 1991 and Bayer, Haber & Stornetta 1993.

5. Dave Bayer, Stuart Haber & W. Scott Stornetta, "Improving the Efficiency and Reliability of Digital Time-Stamping," *Sequences II* (Springer, 1993), p. 330 (§1).

6. Andreas M. Antonopoulos & David A. Harding, *Mastering Bitcoin*, 3rd ed. (O'Reilly, 2023), chapter 11: "The block header is 80 bytes, whereas the total size of all transactions in a block can be..." GitHub manuscript commit 94c9ba64, `ch11_blockchain.adoc`.

7. Nakamoto (2008), p. 4 (§7).

8. *Mastering Bitcoin*, 3rd ed., chapter 11, table "The structure of the block header": Version 4 bytes / Previous Block Hash 32 / Merkle Root 32 / Timestamp 4 / Target 4 / Nonce 4. The byte order of each field and the actual

values are checked against the genesis block in Chapter 12.

9. *Mastering Bitcoin*, 3rd ed., chapter 11 (§Block Identifiers): "The primary identifier of a block is its cryptographic hash … made by hashing the block header twice through the SHA256 algorithm. The resulting 32-byte hash is called the block hash but is more accurately the block header hash, because only the block header is used to compute it." The transaction list is reflected in the header only through the Merkle root.

10. Nakamoto (2008), p. 3 (§4, Proof-of-Work).

11. Nakamoto (2008), p. 3 (§5, Network). The original: "Nodes always consider the longest chain to be the correct one and will keep working on extending it. If two nodes broadcast different versions of the next block simultaneously, some nodes may receive one or the other first. In that case, they work on the first one they received, but save the other branch in case it becomes longer. The tie will be broken when the next proof-of-work is found and one branch becomes longer; the nodes that were working on the other branch will then switch to the longer one."

12. Meni Rosenfeld, "Analysis of Hashrate-Based Double Spending," arXiv:1402.2009 (2014; first draft 2012-12-11), p. 3 (§2).

13. Rosenfeld (2014), p. 2 (§2). The original: "The blocks hence form a tree, with the genesis block as the root, and each block being a child of the block it references. A branch in this tree is a path from a leaf block to the genesis block; each such branch represents one version of the history of Bitcoin transactions. Each branch must be internally consistent and can never include two conflicting transactions; however, the branches need not be consistent with one another, and one branch can include a transaction which contradicts a transaction in another branch."

PART 1 · TECHNICAL FOUNDATIONS

Chapter 8. Proof of work: difficulty, mining, and the longest chain

KEY TAKEAWAY

Mining is not "solving a puzzle." It is **scratching lottery tickets**: changing the nonce in the block header, computing a hash each time, and looking for a nonce whose hash value falls below the target. The target sets the chance that a single attempt wins, and every 2,016 blocks the target is adjusted, "lowered if recent blocks came too fast and raised if they came too slowly," so that on average one block appears every 10 minutes. That is why the winning interval returns to 10 minutes even when machines get faster. The variable that matters for security is not any participant's **absolute computing speed** but its **share** q of the total. After this chapter you will be able to turn a difficulty number into a target and explain why "faster computers will break the blockchain" is wrong.

Learning objectives

1. Explain proof of work as "a lottery for a hash that meets a condition," and compute the probability of success and the expected number of attempts from the target.
2. Convert nBits (the compressed target) in a block header into the actual target and the difficulty.
3. Read the 2,016-block adjustment rule in the Bitcoin Core source, and know why the size of an adjustment has upper and lower limits (4x and ¼x).
4. Connect the "longest (heaviest) chain" rule with the proposition "share, not absolute speed."

What a reader of the [Basic] paragraphs alone takes away. Mining is a lottery, not a puzzle, and even when machines get faster the winning interval is brought back to 10 minutes. This is the basis for misconception ⑥ in Appendix B ("faster computers will break the blockchain").

1. What failure this technology grew out of

Proof of work was not invented for money. It was invented because of **spam**. Adam Back's Hashcash was first announced on the cypherpunks mailing list in March 1997,[1] and his 2002 technical report describes its purpose as "a mechanism to throttle systematic abuse of un-metered internet resources such as email, and anonymous remailers."[2] The idea is simple. To send one email message, the sender's computer must first carry out a computation lasting a few seconds. A few seconds per message is nothing, but for a million messages it is dozens of days. The recipient checks with a single hash that the computation was done.

That "computation" is grinding out hashes. Back defines the hashcash cost function as "finding partial hash collisions on the all 0 bits k-bit string 0^k." You hash over and over, changing part of the input each time, until you find an input whose first *k* output bits are all 0. The fastest method is to try blindly (brute force); the server does not need to issue a challenge (non-interactive); anyone can verify a token efficiently (publicly auditable); and even the server has no back door that lets it make tokens at lower cost (trapdoor-free).[3] Back also writes that the server must keep a "double spending database" of tokens already spent, to stop the same token from being used again.[4] The words "double spending" already appear in a 2002 anti-spam document.

In bringing this device over, Nakamoto changed two things. First, the cost was attached not to "one email message" but to "one block." Second, the size of the cost was **not fixed but left for the network to adjust by itself**. These are the sentences in §4 of the white paper: "To implement a distributed timestamp server on a peer-to-peer basis, we will need to use a proof-of-work system similar to Adam Back's Hashcash [6], rather than newspaper or Usenet posts." And: "To compensate for increasing hardware speed and varying interest in running nodes over time, the proof-of-work difficulty is determined by a moving average targeting an average number of blocks per hour. If they're generated too fast, the difficulty increases."[5] The non-interactive hashcash cost function, as written, has no device by which the network itself changes the amount of work *k*. The report takes up the idea that "it becomes possible to dynamically adjust the work factor required for the client based on server CPU load" only for the interactive variant, in which the server issues a challenge.[6] Bitcoin put the adjustment inside the **consensus rules**. This difference is what produces "10 minutes even when machines get faster."

2. Intuition: a lottery

Here is the definition in §4 of the white paper: "The proof-of-work involves scanning for a value that when hashed, such as with SHA-256, the hash begins with a number of zero bits. The average work required is exponential in the number of zero bits required and can be verified by executing a single hash."[7] The miner hashes the block header while changing its nonce (an arbitrary 4-byte number) through 0, 1, 2, and so on. If the hash meets the condition, the miner releases that header to the network. If not, the next nonce. That is all.

Why a "lottery" and not a "puzzle"? A puzzle gets solved faster if you think harder. Here there is nothing to think about. As we saw in Chapter 4, the output of SHA-256 looks like random numbers unrelated to the input, so which nonce will meet the condition **cannot be known before you try it.** One attempt is one lottery ticket, and the chance of winning is the same every time. To borrow Grunspan and Pérez-Marco's terms, the work starts from scratch with each new hash, so the time it takes to mine a block is **memoryless**. Having come up empty for 10 minutes does not raise the chance of winning within the next 1 minute.[8]

What sets the chance of winning is the **target**. "k leading 0s" means the same as "less than 2^{256-k} when the hash value is read as a 256-bit integer." Instead of the number of 0s, Bitcoin writes the target itself in the header (the nBits field) and requires that "the hash of the header ≤ the target." The smaller the target, the harder it is to win.

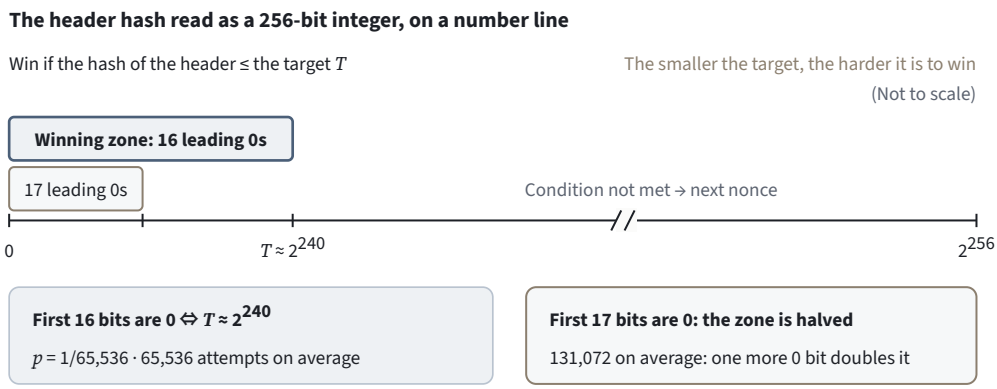


Figure 8-1. A number line of the header hash read as a 256-bit integer. Below the target T is the winning zone; one more leading 0 bit halves the zone and doubles the expected number of attempts. Not to scale.

3. [Intermediate] Formulas: probability of success, expected number of attempts, difficulty

[Intermediate] The probability that a single hash meets the condition is

$$p = \frac{T+1}{2^{256}} \approx \frac{T}{2^{256}}$$

Reading the symbols: T is the target (a 256-bit integer), and 2^{256} is the number of possible hash values. A numerical example: "the first 16 bits are 0" means $T \approx 2^{240}$ and $p = 2^{-16} = 1 / 65{,}536$. Each attempt is independent and has the same probability, so the number of attempts until the first success follows a **geometric distribution**, whose expected value is $1 / p$. With 16 leading zero bits that is 65,536 attempts on average, and with 17 it is 131,072. This is what the white paper calls "exponential": each additional 0 bit doubles it.

[Intermediate] **Difficulty** D is the target inverted so that people can read it easily. Taking Bitcoin's easiest target $T_{\max}$ (the value that the genesis block's nBits, `0x1d00ffff`, points to) as the reference,

$$D = \frac{T_{\max}}{T}$$

A numerical example: if $T = T_{\max} / 2$, then $D = 2$. "Difficulty 2" means "twice as hard as at genesis." At difficulty 1 the expected number of hashes for one block is $2^{256} / T_{\max}$, which is about 2^{32} (the code below gives the exact value). So **at difficulty D the expected number of hashes per block is about $D \cdot 2^{32}$**, and if blocks appear every 600 seconds on average, the hash rate of the whole network can be **estimated** at about $D \cdot 2^{32} / 600$ hashes per second. This is not a measurement but an estimate built on the "600-second assumption." Where the actual block interval differs from 600 seconds, the estimate is off by that much.

nBits: putting a 256-bit number into 4 bytes

[Intermediate] The target field in the header is only 4 bytes. A 256-bit number cannot be written there as it is, so a **compressed notation somewhat like floating point** is used. The first byte is the exponent, and the remaining three bytes are the mantissa (coefficient).

$$T = \text{coefficient} \times 256^{\,\text{exponent} - 3} = \text{coefficient} \times 2^{\,8(\text{exponent} - 3)}$$

Reading the symbols: the coefficient is the lower 3 bytes of nBits read as an integer, and the exponent is the first byte of nBits.[9] A numerical example: the genesis value `0x1d00ffff` has exponent `0x1d` = 29 and coefficient `0x00ffff` = 65,535, so $T_{\max} = 65535 \times 256^{26}$. On October 25, 2026, the nBits of the block at the tip of the chain (height 972,958) was `0x17020093`, which decodes to a difficulty of about 1.41×10^{14} (as of 2026-10; trend notes in Appendix A).[10]

The adjustment rule: what Bitcoin Core does

[Intermediate] The target is adjusted once every 2,016 blocks. The Bitcoin Core source is the rule itself. The mainnet parameters are the target timespan `nPowTargetTimespan = 14 * 24 * 60 * 60` (2 weeks) and the target spacing `nPowTargetSpacing = 10 * 60` (10 minutes), and the adjustment interval is their ratio, `DifficultyAdjustmentInterval() = nPowTargetTimespan / nPowTargetSpacing` = 2,016.[11] The adjustment function `CalculateNextWorkRequired` measures how long the last 2,016-block period actually took, clamps that time to ¼ of the target timespan if it is less than ¼ and to 4 times the target timespan if it is more than 4 times, and then sets the new target to

$$T_{\text{new}} = T_{\text{old}} \times \frac{\text{actual time taken}}{\text{target timespan}}$$

If the result is larger than the easiest target, `powLimit`, it is capped at that value.[12] (`powLimit` is `0x00000000ffff…ffff`; reduced to compact notation, it becomes `0x1d00ffff`, the same as the genesis target $T_{\max}$. This book calls both $T_{\max}$.) Reading the symbols: the target timespan is 1,209,600 seconds (14 days), and the actual time taken is the difference between the header times of the first and last blocks of the period. A numerical example: if a 2-week period ended in 1 week, blocks came twice as fast, so $T_{\text{new}} = T_{\text{old}} / 2$ and the difficulty doubles. Because of the limits, a single adjustment never raises difficulty by more than 4 times or lowers it below ¼. Even if the hash rate suddenly grows 100-fold, difficulty follows it in several steps of 4 times each.

[Intermediate] A textbook points out one detail. The "actual time taken" is the difference between the times of the first and last blocks of the period, and over a 2,016-block period

that difference covers **2,015 block intervals**. *Mastering Bitcoin* writes that because of this off-by-one in the original software the adjustment is biased toward higher difficulty by 0.05%.[13] Consensus rules are very hard to change once they are set, so even a small slip like this one stays.

A known weakness of the adjustment rule: the timewarp

[Intermediate] The time in a header is a value the miner writes, and it is only "a rough time." Because the adjustment rule depends on that time, cleverly chosen timestamps leave room to deceive the adjustment. BIP-54 ("Consensus Cleanup") explains the **timewarp bug** this way: "The timewarp bug makes it possible for a majority-hashrate attacker to arbitrarily lower mining difficulty, and therefore arbitrarily increase the block rate. In the worst case, an attacker can bring down the difficulty to its minimum within 38 days of starting the attack." The proposal blocks this with a rule that the time of the first block of an adjustment period ($N \bmod 2016 = 0$) may not be **earlier** (further in the past) than the time of the previous block by more than 7,200 seconds ($T_N \geq T_{N-1} - 7200$), together with a rule that the time of the last block of a period may not be earlier than the time of the first block of that period.[14] As of 2026-10-25 the status of BIP-54 is "Complete," and it is not marked as activated ("Deployed"). Appendix A updates this. One thing to remember here: the safety of proof of work rests on "hashing is hard" and **also on the details of the adjustment rule**, and those details are still being fixed.

4. Code: a hand miner

[Intermediate] We add one line to the toy chain of Chapter 7: "raise the nonce until the first k bits of the header hash are 0." Running time differs from machine to machine, so it is not shown, and **only the number of attempts** is counted. That number does not depend on the hardware.

```
# NB08-1: a hand miner — how many times must the nonce change before a hash with k leading zero
bits turns up? (for study; toy difficulty)
import hashlib

def mine(header_prefix: bytes, bits: int):
    """Raise the nonce from 0 and find a nonce for which the first `bits` bits of
SHA256(SHA256(prefix||nonce)) are 0. Returns (nonce, number of attempts, hash)."""
    target = 1 << (256 - bits)                # "first `bits` bits are 0" ⇔ hash < 2^(256-
bits)
    nonce = 0
    while True:
        h = hashlib.sha256(hashlib.sha256(header_prefix + nonce.to_bytes(4,
"little")).digest()).digest()
        if int.from_bytes(h, "big") < target:
            return nonce, nonce + 1, h
        nonce += 1

prefix = b"block#1|prev=00..|merkle=ab..|time=1231006505|"
print(f"{'leading 0 bits k':>16} {'nonce found':>11} {'attempts':>10} {'expected 2^k':>12}
{'first 8 hex':>11}")
for bits in [4, 8, 12, 16, 20]:
    nonce, tries, h = mine(prefix, bits)
    print(f"{bits:>16} {nonce:>11,} {tries:>10,} {2**bits:>12,} {h.hex()[:8]:>11}")
```

```
# Output — bcbook (Python 3.13), exactly as run
leading 0 bits k nonce found   attempts expected 2^k first 8 hex
               4           3          4           16    04468f48
               8         329        330          256    004737bc
              12       6,665      6,666        4,096    000c395c
              16      24,541     24,542       65,536    00002bbf
              20     978,082    978,083    1,048,576    00000fc3
```

[Intermediate] The number of attempts swings widely around 2^k. With luck it is a fraction of the expected value; without it, several times the expected value. That is a property of the geometric distribution. We mine many times at the same difficulty and look at the distribution.

Mining 200 times: the distribution of attempts

```
# NB08-2: mine 200 blocks at the same difficulty (k=12) — the distribution of attempts
(geometric distribution)
import math
bits = 12
tries_list = []
for i in range(200):
    _, tries, _ = mine(f"block#{i}|".encode() + prefix, bits)
    tries_list.append(tries)
tries_list.sort()
n = len(tries_list)
mean = sum(tries_list) / n
print(f"k={bits}: expected attempts 2^k = {2**bits:,}")
print(f"200 runs: mean {mean:,.0f}, median {tries_list[n//2]:,}, min {tries_list[0]:,}, max
{tries_list[-1]:,}")
print(f"geometric distribution, theory: mean {2**bits:,}, median ≈ ln2·2^k =
{math.log(2)*2**bits:,.0f}, standard deviation ≈ 2^k = {2**bits:,}")
quick = sum(1 for t in tries_list if t <= 2**bits * 0.1)
slow = sum(1 for t in tries_list if t >= 2**bits * 3)
print(f"blocks done within 1/10 of the expected value: {quick} ({quick/2:.0f}%) | blocks that
took 3 times the expected value or more: {slow} ({slow/2:.0f}%)")
```

```
# Output — bcbook (Python 3.13), exactly as run
k=12: expected attempts 2^k = 4,096
200 runs: mean 4,061, median 3,058, min 27, max 25,448
geometric distribution, theory: mean 4,096, median ≈ ln2·2^k = 2,839, standard deviation ≈ 2^k =
4,096
blocks done within 1/10 of the expected value: 23 (12%) | blocks that took 3 times the expected
value or more: 9 (4%)
```

[Intermediate] The mean is near 2^k, but the spread is as large as the mean. In Bitcoin, "10 minutes" is an average, and actual block intervals are spread just as widely, in the same shape (the exponential distribution in the advanced box). This spread is not a defect; it is the nature of a lottery. And it is exactly this spread that gives the attacker an opening in Chapter 9.

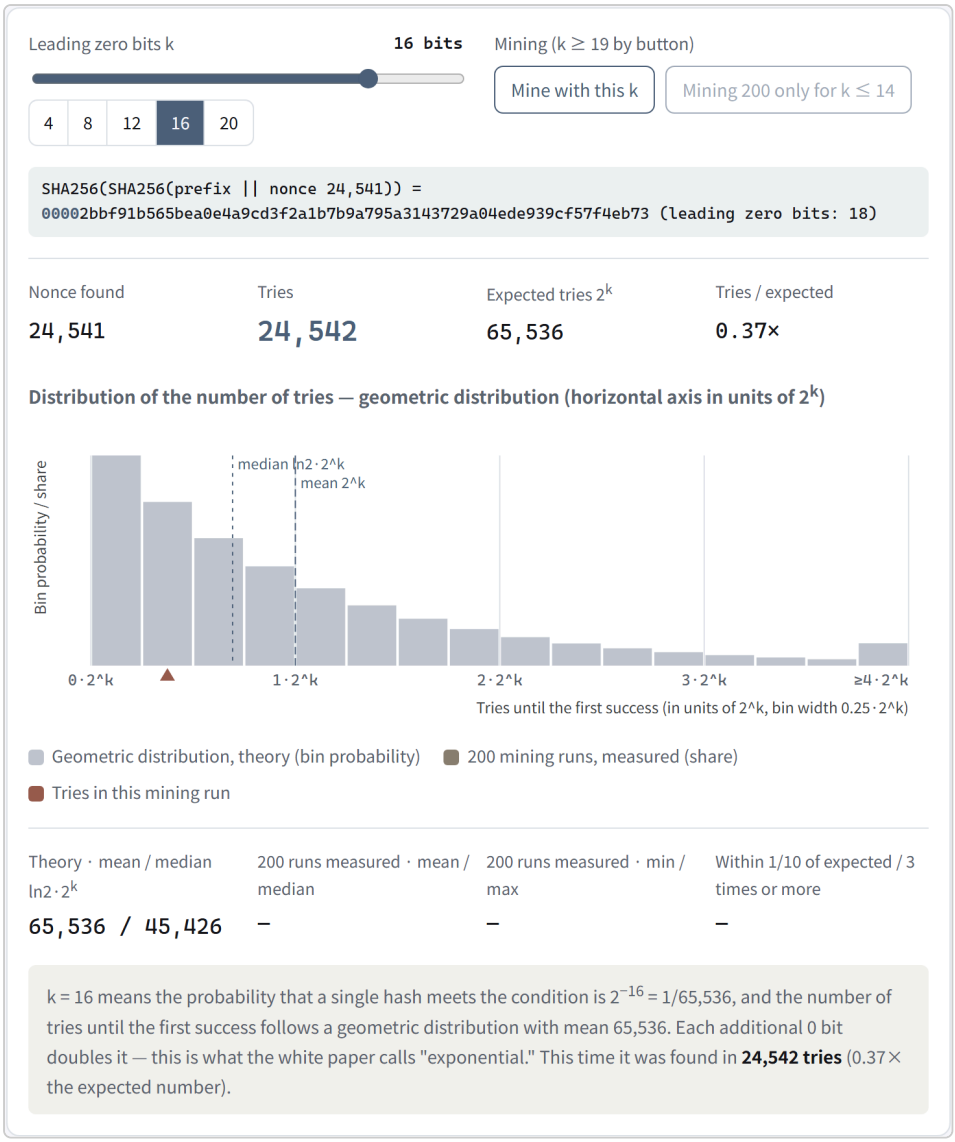


Figure 8-2. Hand miner. Raising the nonce from 0 until the hash begins with 16 zero bits, the search succeeds at 24,541 (24,542 tries; 65,536 expected). The number of tries follows a geometric distribution, and its spread is as large as its mean. Running time differs from machine to machine and is not shown. — In the HTML edition you can change the values here (HTML edition, Chapter 8, Section 4).

From nBits to target and difficulty

```
# NB08-3: nBits → target → difficulty → hash-rate estimate. Genesis (0x1d00ffff) and the chain
tip on 2026-10-25 (0x17020093)
import math
def nbits_to_target(nbits: int) -> int:
    exponent, coefficient = nbits >> 24, nbits & 0x007FFFFF
    return coefficient * 256 ** (exponent - 3)

T_MAX = nbits_to_target(0x1d00ffff)              # the easiest target (genesis nBits; equal to
powLimit reduced to compact form)
print("T_max = 0x%064x" % T_MAX)
print("expected hashes per block at difficulty 1, 2^256/T_max = %.6f × 2^32" % (2**256 / T_MAX /
2**32))

for name, nbits in [("Genesis (2009-01-03)", 0x1d00ffff), ("Height 972,958 (as of 2026-10-25)",
0x17020093)]:
    T = nbits_to_target(nbits)
    D = T_MAX / T
    p = (T + 1) / 2**256
    print(f"\n{name}: nBits = 0x{nbits:08x}")
    print("  target T = 0x%064x" % T)
    print("  probability that one hash succeeds p = %.3e  (leading 0 bits ≈ %.1f)" % (p, 256 -
math.log2(T + 1)))
    print("  difficulty D = T_max/T = %.1f" % D)
    print("  expected hashes per block ≈ D·2^32 = %.3e" % (D * 2**32))
    H = D * 2**32 / 600
    print("  hash-rate estimate assuming 600 s, D·2^32/600 = %.4e H/s" % H + (" = %.0f EH/s" %
(H / 1e18) if H >= 1e15 else " = %.1f MH/s" % (H / 1e6)))
```

```
# Output — bcbook (Python 3.13), exactly as run
T_max = 0x00000000ffff0000000000000000000000000000000000000000000000000000
expected hashes per block at difficulty 1, 2^256/T_max = 1.000015 × 2^32

Genesis (2009-01-03): nBits = 0x1d00ffff
  target T = 0x00000000ffff0000000000000000000000000000000000000000000000000000
  probability that one hash succeeds p = 2.328e-10  (leading 0 bits ≈ 32.0)
  difficulty D = T_max/T = 1.0
  expected hashes per block ≈ D·2^32 = 4.295e+09
  hash-rate estimate assuming 600 s, D·2^32/600 = 7.1583e+06 H/s = 7.2 MH/s

Height 972,958 (as of 2026-10-25): nBits = 0x17020093
  target T = 0x0000000000000000000200930000000000000000000000000000000000000000
  probability that one hash succeeds p = 1.656e-24  (leading 0 bits ≈ 79.0)
  difficulty D = T_max/T = 140577680051919.6
  expected hashes per block ≈ D·2^32 = 6.038e+23
  hash-rate estimate assuming 600 s, D·2^32/600 = 1.0063e+21 H/s = 1006 EH/s
```

[Intermediate] At genesis the probability that one hash succeeds was about 2^{-32}, and in October 2026 it is one 10^{14}-th of that. Converted through difficulty, the network's computing power has grown 10^{14}-fold, yet the target interval was 10 minutes then and is 10 minutes now. The adjustment rule made it so. (This is an estimate under the "600-second assumption," not a measurement.)

The difficulty adjustment rule

```
# NB08-4: simulating the adjustment rule — if the hash rate suddenly becomes 16 times larger,
how many adjustments does difficulty need to catch up? (the Core rule as it is)
TIMESPAN, SPACING, INTERVAL = 1_209_600, 600, 2016      # the values in chainparams.cpp and
params.h
def retarget(T_old: int, actual_timespan: int) -> int:
    """pow.cpp CalculateNextWorkRequired: clamp the time taken to [timespan/4, timespan*4],
adjust in proportion, cap at powLimit."""
    actual = max(TIMESPAN // 4, min(TIMESPAN * 4, actual_timespan))
    T_new = T_old * actual // TIMESPAN
    return min(T_new, T_MAX)

T = T_MAX // 1000                       # starting from some difficulty (1000)
H1 = 2**256 / (T + 1) / 600             # take as 1 the hash rate (hashes/second) that gives
exactly 600 seconds per block at that difficulty
hashrate = 1.0
print(f"{'round':>6} {'difficulty':>10} {'mean block interval (s)':>23} {'next
difficulty':>15}")
for period in range(1, 7):
    if period == 2:
        hashrate = 16.0                 # from the round-2 period on, the hash rate jumps 16-fold
    D = T_MAX / T
    expected_hashes_per_block = 2**256 / (T + 1)
    seconds_per_block = expected_hashes_per_block / (hashrate * H1)
    actual_timespan = int(seconds_per_block * INTERVAL)
    T_next = retarget(T, actual_timespan)
    print(f"{period:>6} {D:>10,.0f} {seconds_per_block:>23,.0f} {T_MAX / T_next:>15,.0f}")
    T = T_next
```

```
# Output — bcbook (Python 3.13), exactly as run
 round difficulty mean block interval (s) next difficulty
     1      1,000                     600           1,000
     2      1,000                      38           4,000
     3      4,000                     150          16,000
     4     16,000                     600          16,000
     5     16,000                     600          16,000
     6     16,000                     600          16,000
```

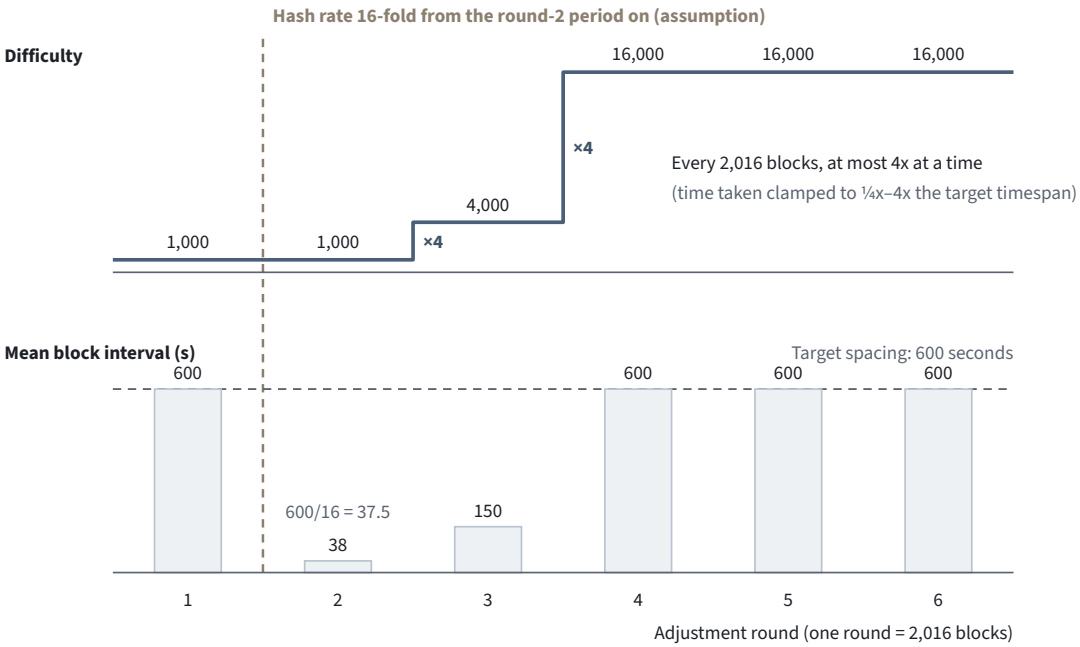


Figure 8-3. Simulation of the Bitcoin Core adjustment rule (output of NB08-4). The 16-fold hash rate is an assumption of the code, not an observation from the actual chain. Because one adjustment can raise difficulty by at most 4 times, difficulty reaches 16 times over two adjustments and the block interval returns to 600 seconds.

[Intermediate] If the hash rate becomes 16 times larger all at once, blocks appear about every 38 seconds (600/16 = 37.5 seconds), but since one adjustment can raise difficulty by at most 4 times, difficulty reaches 16 times only after two adjustments (4 × 4) and the interval returns to 600 seconds. After that it is in equilibrium. The limits are a buffer that keeps the adjustment from overreacting to a single period that happened to run fast or slow, and the price is a slow response to sudden change.

5. The "longest chain" and the share q

Proof of work does two jobs. First, it answers the question left open in Chapter 7: **"which of two chains is the history?"** §4 of the white paper: "The proof-of-work also solves the problem of determining representation in majority decision making. If the majority were based on one-IP-address-one-vote, it could be subverted by anyone able to allocate many IPs. Proof-of-work is essentially **one-CPU-one-vote**. The majority decision is represented by the longest chain, which has the greatest proof-of-work effort invested in it."[15] Second, it makes tampering expensive: "To modify a past block, an attacker would have to redo the proof-of-work of the block and all blocks after it and then catch up with and surpass the work of the honest nodes."[16]

This shows what the security variable is. The condition "catch up with and surpass the work of the honest nodes" turns on **the ratio of the attacker's hash rate to everyone else's**. If the attacker has a share q of the total hash rate and the rest have $p = 1 - q$, the probability that the attacker finds the next block is q. Even if the total hash rate grows 10-fold, this probability stays the same as long as q is the same, and because difficulty adjustment brings the block interval back to 10 minutes, the shape of "the race counted in blocks" also stays the same. The only path by which absolute speed enters is the adjustment rule, and adjustment does not change q. So the proposition "faster computers will break the blockchain" is, taken by itself, wrong. The condition for a break is "does someone have a large share of the total **alone**?" The next chapter works out the mathematics of this race in full.

Miners choose which transactions to put in a block. The fifth step in §5 of the white paper is "5) Nodes accept the block only if all transactions in it are valid and not already spent,"[17] and §6 describes the "incentive" that gives new coins to the block's creator through the block's first transaction, along with transaction fees.[18] Miners carry transactions in exchange for fees, but no rule compels them to include one transaction or to put off another. This fact is the starting point of Chapter 17 and Chapter 21, and the background to the question below. Calculations such as electricity costs and mining profitability are outside the scope of this book (see the notes to the reader in Chapter 3). Appendix A treats the energy debate in its structure only, without figures.

> **? A LAWYER'S QUESTION**
>
> Whose agent is a miner? A miner is not the agent of the person who sent the transaction, nor of the person who receives it, and is bound by no contract. Does a miner who leaves a valid transaction out of a block have any duty? If a group of miners refuses every transaction from a particular address, what act is that, and toward whom? If the majority vote of "one CPU, one vote" has legal meaning, what kind of body is that majority? → Chapter 28

▾ [Advanced] Block arrivals are a Poisson process

[Advanced] When the probability p that one hash succeeds is very small and attempts happen very many times per second, "the probability that a block appears within time t" follows the **exponential distribution**, the continuous limit of the geometric distribution. For the block interval T,

$$P(T > t) = e^{-t / t_0}, \qquad t_0 = 600 \text{ s}$$

Reading the symbols: t_0 is the average block interval, which the adjustment rule holds at 600 seconds. A numerical example: the probability that a block appears within 10 minutes is $1 - e^{-1} \approx 63\%$, and the probability that none appears for more than 30 minutes is $e^{-3} \approx 5\%$. Memorylessness follows directly from the formula: $P(T > t_1 + t_2 \mid T > t_2) = P(T > t_1)$.[19] If the intervals are exponentially distributed, "the number of blocks in a given time" follows the **Poisson distribution**, and "the time it takes for z blocks to appear" follows the **gamma distribution**. §11 of the white paper approximated the attacker's progress as Poisson while simplifying by assuming that the honest chain mined its z blocks "in exactly the average time." Grunspan and Pérez-Marco's exact formula is that simplification worked out with the gamma distribution. Chapter 9 computes the two results side by side.

6. Summary

Proof of work came from spam prevention, and Bitcoin added to it "difficulty that the network adjusts by itself." Mining is a lottery, so every attempt has the same probability and no memory, and the adjustment rule brings the winning interval back to an average of 10 minutes. So the cost of tampering is priced in computation, and safety rests not on absolute speed but on **the share q**. The blank in Chapter 7, "which of two chains is the history?", has been answered: "the one on which more work has piled up." That this answer is probabilistic, that "history" hardens as time passes but never hardens completely, is the subject of the next chapter.

Sources for this chapter

- Adam Back, "Hashcash — A Denial of Service Counter-Measure," technical report, 2002-08-01. http://www.hashcash.org/papers/hashcash.pdf — purpose (p. 1), definition of the cost function and the double-spending database (p. 3), dynamic adjustment (p. 4). First announcement: cypherpunks, 1997-03-28, http://www.hashcash.org/papers/announce.txt.
- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — definition of proof of work, one CPU one vote, difficulty adjustment (§4, p. 3), network steps (§5, p. 3), incentive (§6, p. 4).
- Bitcoin Core v31.1 (tag commit 9be056a8a72b624dae9623b2f7bded92c2a21c91): `src/kernel/chainparams.cpp`, `src/consensus/params.h`, `src/pow.cpp` — adjustment interval, target timespan, limits, powLimit. Checked against an independent

implementation: btcd commit dd169fe0, `chaincfg/params.go`.

- Antoine Poinsot and Matt Corallo, "Consensus Cleanup," BIP 54 (Status: Complete, accessed 2026-10-25) — the timewarp bug and the 7,200-second rule.
- Cyril Grunspan and Ricardo Pérez-Marco, "Double Spend Races," *IJTAF* 21(8): 1850053, 2018; arXiv:1702.02867 — memorylessness of mining time (§2).
- Andreas M. Antonopoulos and David A. Harding, *Mastering Bitcoin*, 3rd ed., O'Reilly, 2023 (CC BY-SA 4.0, GitHub manuscript commit 94c9ba64) — the nBits conversion formula and the 2,015-block off-by-one (chapter 12).
- Bundled data `data/hashrate_2026-10-25.json` (reference date 2026-10-25): headers of blocks 971,712 and 972,958, difficulty, estimated hash rate. Sources, formulas, and limitations are in the README of the data folder.
- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library modules `hashlib` and `math`. Notebook NB08.

Notes

1. Adam Back, "[ANNOUNCE] hash cash postage implementation," cypherpunks mailing list, 1997-03-28 (copy on the author's site, http://www.hashcash.org/papers/announce.txt; the list of papers on the same site reads "Mar 1997 - hashcash package 1st announced"). The text of the 2002 report gives the first proposal as "May 1997," two months off. This book follows the date of the announcement email and records the discrepancy here. Accessed 2026-10-25.

2. Adam Back, "Hashcash — A Denial of Service Counter-Measure," technical report, 2002-08-01, p. 1: "Hashcash was originally proposed as a mechanism to throttle systematic abuse of un-metered internet resources such as email, and anonymous remailers in May 1997." http://www.hashcash.org/papers/hashcash.pdf.

3. Back (2002), p. 3 (§3, The Hashcash cost-function): "Hashcash is a non-interactive, publicly auditable, trapdoor-free cost function with unbounded probabilistic cost." "The hashcash cost-function is based on finding partial hash collisions on the all 0 bits k-bit string 0^k. The fastest algorithm for computing partial collisions is brute force."

4. Back (2002), p. 3: "The server needs to keep a double spending database of spent tokens, to detect and reject attempts to spend the same token again."

5. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), p. 3 (§4, Proof-of-Work).

6. Back (2002), p. 4 (§4.1, Dynamic throttling): "With interactive hashcash it becomes possible to dynamically adjust the work factor required for the client based on server CPU load." For the non-interactive (mail) variant the report describes no adjustment device (p. 3: there is no channel through which the server could send a challenge), and "there is no adjustment device" is this book's reading.

7. Nakamoto (2008), p. 3.

8. Cyril Grunspan and Ricardo Pérez-Marco, "Double Spend Races," *International Journal of Theoretical and Applied Finance* 21(8): 1850053 (2018), §2 (arXiv:1702.02867v3, p. 5): "For each new hash the work is started from scratch, therefore the random variable T measuring the time it takes to mine a block is memoryless ...".

9. Andreas M. Antonopoulos and David A. Harding, *Mastering Bitcoin*, 3rd ed. (O'Reilly, 2023), chapter 12: "target = coefficient × 2^(8 × (exponent – 3))". As an example it decodes `0x1903a30c` from block 277,316. GitHub manuscript, commit 94c9ba64, `ch12_mining.adoc`.

10. This book's bundled data file `data/hashrate_2026-10-25.json` (reference date 2026-10-25, accessed 2026-10-24 18:17 UTC). The raw header bytes of block 971,712 (the last retarget block) and block 972,958 (the tip of the chain at the time of observation) were obtained from two explorers (Blockstream Esplora and mempool.space) and checked to be byte-identical, and nBits `0x17020093` and the difficulty 140,577,680,051,919.6 were recomputed directly from the header. Estimated hash rate

$D \cdot 2^{32} / 600 \approx 1.006 \times 10^{21}$ H/s (about 1,006 EH/s). The comparison value that mempool.space gave at the same time, by a different method, was 1.077×10^{21} H/s, and neither value is the true value. These values change from edition to edition, so the main text gives only their order of magnitude and Appendix A updates them.

11. Bitcoin Core v31.1 (tag commit 9be056a8a72b624dae9623b2f7bded92c2a21c91), `src/kernel/chainparams.cpp`, mainnet: `consensus.nPowTargetTimespan = 14 * 24 * 60 * 60; // two weeks`, `consensus.nPowTargetSpacing = 10 * 60;`, `consensus.powLimit = uint256{"00000000ffff…ffff"}`; `src/consensus/params.h`: `int64_t DifficultyAdjustmentInterval() const { return nPowTargetTimespan / nPowTargetSpacing; }`. Accessed 2026-10-25. The independent implementation btcd (commit dd169fe0) has the same values in `chaincfg/params.go`: TargetTimespan 14 days, TargetTimePerBlock 10 minutes, RetargetAdjustmentFactor 4.

12. Same release, `src/pow.cpp`, function `CalculateNextWorkRequired`: `int64_t nActualTimespan = pindexLast->GetBlockTime() - nFirstBlockTime; if (nActualTimespan < params.nPowTargetTimespan/4) nActualTimespan = params.nPowTargetTimespan/4; if (nActualTimespan > params.nPowTargetTimespan*4) nActualTimespan = params.nPowTargetTimespan*4;` followed by (the `else` branch that mainnet uses; the Testnet4 branch under `if (params.enforce_BIP94)` is omitted) `bnNew.SetCompact(pindexLast->nBits); bnNew *= nActualTimespan; bnNew /= params.nPowTargetTimespan; if (bnNew > bnPowLimit) bnNew = bnPowLimit;`. When an adjustment happens is fixed by `GetNextWorkRequired`, which returns the previous nBits unchanged if `(pindexLast->nHeight+1) % params.DifficultyAdjustmentInterval() != 0`.

13. *Mastering Bitcoin*, 3rd ed., chapter 12: "While the target calibration happens every 2,016 blocks, because of an off-by-one error in the original Bitcoin software, it is based on the total time of the previous 2,015 blocks (not 2,016 as it should be), resulting in a retargeting bias toward higher difficulty by 0.05%." The summary formula in the same place, "New Target = Old Target * (20,160 minutes / Actual Time of Last 2015 Blocks)", has the ratio inverted, the opposite of the Core code (`bnNew *= nActualTimespan; bnNew /= nPowTargetTimespan`). This book treats the code as authoritative.

14. Antoine Poinsot and Matt Corallo, "Consensus Cleanup," BIP 54 (Status: Complete), Motivation and Specification sections. The rule: "if N % 2016 is equal to 0, the timestamp of the block must be set to a value higher than or equal to the value of the timestamp of block at height N-1 minus 7200"; the second rule is that the time of a block with N % 2016 = 2015 must be no earlier than the time of block N−2015 (same Specification section). github.com/bitcoin/bips, file commit 927b6de9, accessed 2026-10-25.

15. Nakamoto (2008), p. 3. Emphasis added.

16. Nakamoto (2008), p. 3.

17. Nakamoto (2008), p. 3 (§5).

18. Nakamoto (2008), p. 4 (§6, Incentive): "By convention, the first transaction in a block is a special transaction that starts a new coin owned by the creator of the block." "The incentive can also be funded with transaction fees."

19. Grunspan and Pérez-Marco (2018), §2 (arXiv v3, p. 5): "P[T > t1 + t2 | T > t2] = P[T > t1]. Therefore we have P[T > t1 + t2] = P[T > t1 + t2|T > t2].P[T > t2] = P[T > t1].P[T > t2]."

PART 1 · TECHNICAL FOUNDATIONS

Chapter 9. The probabilistic nature of finality: what "confirmations" mean

KEY TAKEAWAY

A blockchain has no moment of "done." Once a transaction is in a block and z more blocks have been stacked on top of it, the probability that an attacker succeeds in reversing it shrinks exponentially in z, and that is all. The probability is a function of two things only, the attacker's share of the hash rate q and z. Neither absolute computing speed nor the **time** waited enters it. "6 confirmations" is a convention, not a rule. At $q = 0.1$ the probability that 6 confirmations are overturned is 0.02% by the white paper's approximation and 0.06% by the exact formula. As q approaches 0.5 the number of confirmations needed for the same level of safety grows without bound, and at $q \geq 0.5$ no number of confirmations makes a payment safe. After this chapter you will see that the legal question "when was the payment completed?" translates, technically, into a single probability.

Learning objectives

1. Read the attacker's catch-up probability in §11 of the white paper from a table, without the derivation, and see how it changes with q and z.
2. **[Intermediate]** Derive the gambler's-ruin formula $(q / p)^z$ and locate where the white paper's Poisson approximation comes in.
3. **[Advanced]** Work out how later papers (Rosenfeld 2014; Grunspan and Pérez-Marco 2018) corrected the white paper's approximation, and by how much the correction raises the safe number of confirmations.
4. Confirm in code that "6 confirmations" is a convention rather than a rule, and that the security variable is the share q, not absolute speed.

What a reader of the [Basic] paragraphs alone takes away. A blockchain has no moment of "done." The probability of reversal only gets smaller. This is the basis for misconception ⑦ in Appendix B ("'6 confirmations' means final").

1. What an attacker can and cannot do

Start with the scope. Even an attacker holding a majority of the hash rate **cannot do whatever it likes.** The first paragraph of §11 of the white paper reads: "We consider the scenario of an attacker trying to generate an alternate chain faster than the honest chain. Even if this is accomplished, it does not throw the system open to arbitrary changes, such as creating value out of thin air or taking money that never belonged to the attacker. Nodes are not going to accept an invalid transaction as payment, and honest nodes will never accept a block containing them. An attacker can only try to **change one of his own transactions to take back money he recently spent**."[1] Hash rate does not break the signature rules (Chapter 5). What hash rate can do is **change the order of history**: build another branch that carries, in place of the transaction the attacker sent, a transaction returning the same coins to the attacker, and make that branch the longer one. This is the **double-spending** attack.

Meni Rosenfeld sets out the attack in four steps: "1. Broadcast to the network a transaction in which the attacked merchant is paid. 2. **Secretly** mine a branch which builds on the latest block at the time (before the transaction made it into a block), which includes instead a conflicting transaction which pays the attacker. 3. Wait until the transaction to the merchant receives enough confirmations and the merchant, confident in his payment, sends the product. 4. If necessary, continue extending the secret branch (which contradicts the transaction) until it is longer than the public branch (which includes the transaction), then broadcast it. Because the new branch is longer than the one currently known by the network, it will be considered valid, and the payment to the merchant will be replaced by the payment to the attacker."[2] The merchant loses both the product and the coins, and the attacker ends up with both.

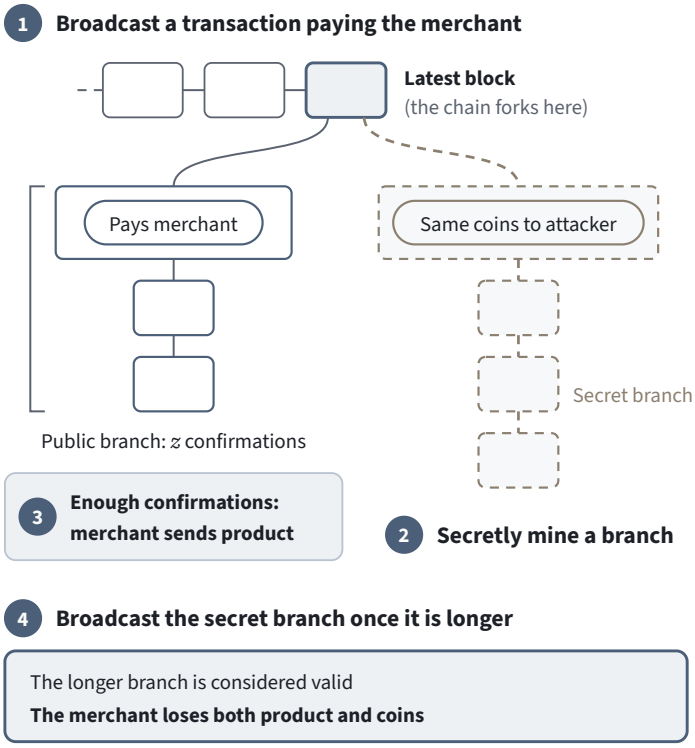


Figure 9-1. The four steps of a double-spending attack, in Rosenfeld's order. The public branch carries the transaction that pays the merchant; the secret branch carries a transaction that sends the same coins to the attacker.

So the merchant's question is not "is this transaction final?" but "**how many blocks must I wait before the probability of reversal is small enough?**" The number of blocks stacked on a transaction, counting the block that contains it, is called its number of **confirmations**.[3] §11 of the white paper computes this probability.

2. [Intermediate] The formula in §11 of the white paper, in two stages

Stage one: the gambler's ruin

[Intermediate] Let p be the probability that the honest miners find the next block and $q = 1 - p$ the probability that the attacker does. Each time a block appears, the difference in length between the two chains (the gap) moves by $+1$ if the honest side found it and by -1 if the attacker did. The white paper calls this a "Binomial Random Walk" and likens the probability that the attacker makes up a gap of z to "a Gambler's Ruin problem."[4] It is the probability that a gambler with unlimited credit, starting at a deficit, ever reaches breakeven.

[Intermediate] Write q_z for the probability of ever reaching a gap of 0 from a gap of z. Then $q_0 = 1$, and conditioning on who finds the next block gives $q_z = p\, q_{z+1} + q\, q_{z-1}$. Substituting a solution of the form $q_z = r^z$ gives $pr^2 - r + q = 0$, whose roots are $r = 1$ and $r = q / p$. When $q < p$, the only solution that goes to 0 as $z \to \infty$ is $(q / p)^z$, which gives[5]

$$q_z = \begin{cases} 1 & p \le q \\ (q / p)^z & p > q \end{cases}$$

Reading the symbols: q / p is the ratio of the attacker's hash rate to the honest hash rate, and z is the number of blocks the attacker has to make up. A numerical example: at $q = 0.1$, $q / p = 1 / 9$, and at $z = 6$, $q_z = 9^{-6} \approx 1.9 \times 10^{-6}$. This is the mathematics of the "51% attack." At $q \geq 1 / 2$, $q_z = 1$, and any gap is eventually closed. In Rosenfeld's words, "If the attacker controls more than half of the total network hashrate, he always succeeds in catching up, from any disadvantage."[6]

Stage two: how much has the attacker already mined?

[Intermediate] The attacker was not idle while the merchant waited for z confirmations. If the attacker's secret branch is already k blocks long at the moment the merchant sends the product, the real gap is $z - k$. The white paper handles k as follows: "The recipient waits until the transaction has been added to a block and z blocks have been linked after it. He doesn't know the exact amount of progress the attacker has made, but assuming the honest blocks took the average expected time per block, the attacker's potential progress will be a Poisson distribution with expected value" $\lambda = z\, q / p$.[7] Multiply the Poisson probability of each amount of progress k by the probability of catching up from that point, add them up, and rearrange to avoid the infinite tail:[8]

$$P = 1 - \sum_{k=0}^{z} \frac{\lambda^k e^{-\lambda}}{k!} \left(1 - (q / p)^{z-k}\right), \qquad \lambda = z\frac{q}{p}$$

Reading the symbols: λ is the expected number of blocks the attacker mined while the honest chain mined z blocks, and each term of the sum is "the probability that the attacker mined k blocks × the probability that it then fails to catch up." The white paper turns this formula into C code, `AttackerSuccessProbability(q, z)`, and prints tables for $q = 0.1$ and $q = 0.3$. A numerical example: at $q = 0.1$ and $z = 6$, $P = 0.0002428$. The code below reproduces the tables to the seventh decimal place.

3. Code: reproducing the white paper's tables, and going beyond them

```
# NB09-1: the formula in §11 of the white paper — a direct port of the C code that reproduces
the paper's tables (q=0.1, 0.3)
import math

def nakamoto_P(q: float, z: int) -> float:
    """P = 1 − Σ_{k=0}^{z} λ^k e^{-λ}/k! · (1 − (q/p)^{z-k}),  λ = z·q/p   (Nakamoto 2008,
p.7)"""
    p = 1.0 - q
    lam = z * (q / p)
    total = 1.0
    for k in range(z + 1):
        poisson = math.exp(-lam)
        for i in range(1, k + 1):
            poisson *= lam / i
        total -= poisson * (1 - (q / p) ** (z - k))
    return total

def gamblers_ruin(q: float, z: int) -> float:
    """First formula of §11, q_z: probability the attacker ever catches up from z blocks behind
(ignores the Poisson progress)"""
    p = 1.0 - q
    return 1.0 if p <= q else (q / p) ** z

for q, zs in [(0.1, range(0, 11)), (0.3, range(0, 51, 5))]:
    print(f"q={q}")
    for z in zs:
        print(f"  z={z:<3} P={nakamoto_P(q, z):.7f}   (q_z, gambler's ruin only =
{gamblers_ruin(q, z):.7f})")
```

```
# Output — bcbook (Python 3.13), exactly as run
q=0.1
  z=0   P=1.0000000   (q_z, gambler's ruin only = 1.0000000)
  z=1   P=0.2045873   (q_z, gambler's ruin only = 0.1111111)
  z=2   P=0.0509779   (q_z, gambler's ruin only = 0.0123457)
  z=3   P=0.0131722   (q_z, gambler's ruin only = 0.0013717)
  z=4   P=0.0034552   (q_z, gambler's ruin only = 0.0001524)
  z=5   P=0.0009137   (q_z, gambler's ruin only = 0.0000169)
  z=6   P=0.0002428   (q_z, gambler's ruin only = 0.0000019)
  z=7   P=0.0000647   (q_z, gambler's ruin only = 0.0000002)
  z=8   P=0.0000173   (q_z, gambler's ruin only = 0.0000000)
  z=9   P=0.0000046   (q_z, gambler's ruin only = 0.0000000)
  z=10  P=0.0000012   (q_z, gambler's ruin only = 0.0000000)
q=0.3
  z=0   P=1.0000000   (q_z, gambler's ruin only = 1.0000000)
  z=5   P=0.1773523   (q_z, gambler's ruin only = 0.0144583)
  z=10  P=0.0416605   (q_z, gambler's ruin only = 0.0002090)
  z=15  P=0.0101008   (q_z, gambler's ruin only = 0.0000030)
  z=20  P=0.0024804   (q_z, gambler's ruin only = 0.0000000)
  z=25  P=0.0006132   (q_z, gambler's ruin only = 0.0000000)
  z=30  P=0.0001522   (q_z, gambler's ruin only = 0.0000000)
  z=35  P=0.0000379   (q_z, gambler's ruin only = 0.0000000)
  z=40  P=0.0000095   (q_z, gambler's ruin only = 0.0000000)
  z=45  P=0.0000024   (q_z, gambler's ruin only = 0.0000000)
  z=50  P=0.0000006   (q_z, gambler's ruin only = 0.0000000)
```

[Intermediate] The values match the table on p. 8 of the white paper to the seventh decimal place. The right-hand column is the gambler's-ruin value alone, ignoring the Poisson progress. The difference between the two columns is the effect of the attacker having mined while the merchant waited, and the effect is not small. At $q = 0.1$ and $z = 6$, 1.9×10^{-6}

becomes 2.4×10^{-4}, more than a hundred times larger.

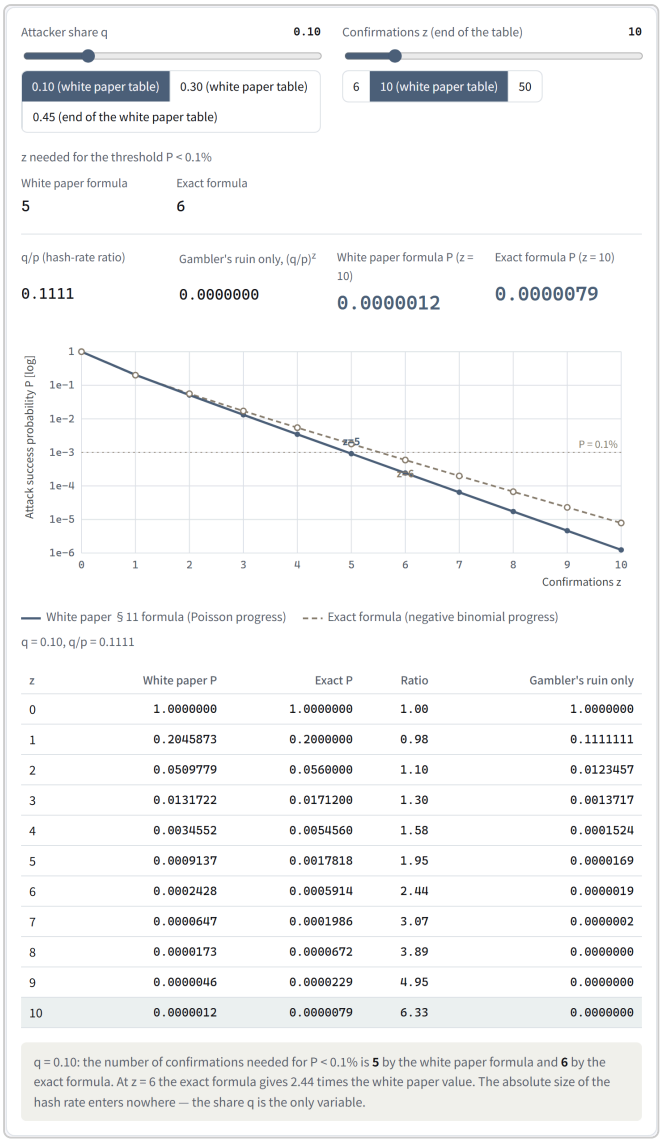
Confirmations needed for "P < 0.1%"

```
# NB09-2: "P < 0.1%" on p. 8 of the white paper — the minimum number of confirmations z for each
attacker share q
def min_z(P_func, q, threshold=0.001):
    z = 0
    while P_func(q, z) > threshold:
        z += 1
    return z

print("q      z (Nakamoto formula)")
for q in (0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45):
    print(f"{q:.2f}   {min_z(nakamoto_P, q):>4}")
```

```
# Output — bcbook (Python 3.13), exactly as run
q      z (Nakamoto formula)
0.10      5
0.15      8
0.20     11
0.25     15
0.30     24
0.35     41
0.40     89
0.45    340
```

[Intermediate] This is the "P < 0.001" table on p. 8 of the white paper (5, 8, 11, 15, 24, 41, 89, 340). As q nears 0.5, $q / p \to 1$, the base of the decay approaches 1, and the number of confirmations needed diverges. The table also shows where "6 confirmations" came from. It is the 5 (white paper formula) or the 6 (exact formula, below) in the $q = 0.1$ row. Rosenfeld's assessment is this: "There is nothing special about the default, often-cited figure of 6 confirmations. It was chosen based on the assumption that an attacker is unlikely to amass more than 10% of the hashrate, and that a negligible risk of less than 0.1% is acceptable. Both these figures are arbitrary, however; 6 confirmations are overkill for casual attackers, and at the same time powerless against more dedicated attackers with much more than 10% hashrate."[9]



z	White paper P	Exact P	Ratio	Gambler's ruin only
0	1.0000000	1.0000000	1.00	1.0000000
1	0.2045873	0.2000000	0.98	0.1111111
2	0.0509779	0.0560000	1.10	0.0123457
3	0.0131722	0.0171200	1.30	0.0013717
4	0.0034552	0.0054560	1.58	0.0001524
5	0.0009137	0.0017818	1.95	0.0000169
6	0.0002428	0.0005914	2.44	0.0000019
7	0.0000647	0.0001986	3.07	0.0000002
8	0.0000173	0.0000672	3.89	0.0000000
9	0.0000046	0.0000229	4.95	0.0000000
10	0.0000012	0.0000079	6.33	0.0000000

q = 0.10: the number of confirmations needed for P < 0.1% is **5** by the white paper formula and **6** by the exact formula. At z = 6 the exact formula gives 2.44 times the white paper value. The absolute size of the hash rate enters nowhere — the share q is the only variable.

Figure 9-2. Double-spending probability. The attacker's success probability by number of confirmations z at attacker share q = 0.1: the formula in §11 of the white paper (solid line) and the exact negative-binomial formula (dashed line). At z = 6 the white paper value is 0.0002428 and the exact formula gives 0.0005914. — In the HTML edition you can change the values here (HTML edition, Chapter 9, Section 3).

[Advanced] A correction: the white paper's approximation is optimistic

[Advanced] Negative binomial, not Poisson: the exact formula and what follows from it

[Advanced] The white paper's simplification is the assumption that the z honest blocks arrived in exactly the average time. In reality that time is dispersed (the advanced box in Chapter 8: the gamma distribution). In his 2012 draft Rosenfeld drops the assumption and models m, the number of blocks the attacker mined while the honest chain mined n blocks, as a **negative binomial** variable: "it is the number of successes (blocks found by the attacker) before n failures (blocks found by the honest network), with a probability q of success," with probability $P(m) = \binom{m+n-1}{m} p^n q^m$.[10] Grunspan and Pérez-Marco gave the same correction in closed form in their 2018 paper: "Let $0 < q < 1/2$, respectively $p = 1-q$, be the relative hash power of the group of the attackers, respectively of honest miners. After z blocks have been validated by the honest miners, the probability of success of the attackers is $P(z) = I_{4pq}(z, 1/2)$ where $I_x(a, b)$ is the regularized incomplete beta function ...." And: "In general, for $z \geq 2$, these probabilities $P(z)$ are larger than those $P_{SN}(z)$ obtained by Nakamoto. From the standpoint of bitcoin security, this shows than [sic] larger confirmation times z are necessary compared to those z_{SN} given by Nakamoto ...."[11]

[Advanced] The exact formula can be computed without special functions. If the number of blocks k mined by the attacker is negative binomial, then

$$P(z) = \sum_{k=0}^{z-1} \binom{k+z-1}{k} p^z q^k \left(\frac{q}{p}\right)^{z-k} + \left(1 - \sum_{k=0}^{z-1} \binom{k+z-1}{k} p^z q^k\right)$$

Reading the symbols: the first sum is "the probability that the attacker, having mined fewer than z blocks, makes up the remaining gap by gambler's ruin," and the second term is "the probability that it has already mined z or more and caught up." A numerical example: at $q = 0.1$ and $z = 6$ the value is 0.000591, which is 2.4 times the white paper's 0.000243. The code below computes this finite sum and checks whether it matches (1) the $q = 10\%$ row of Table 1 in Rosenfeld's paper and (2) the "required z" row of Table 1 in Grunspan and Pérez-Marco.

```
# NB09-3: the exact formula — computed as a finite negative-binomial sum and checked against
Rosenfeld Table 1 and Grunspan-Pérez-Marco Table 1 (no external packages)
from math import lgamma, log, exp

def exact_P(q: float, z: int) -> float:
    """While the honest chain mines z blocks, the attacker's block count k ~ NegBin; if k<z it
catches up with (q/p)^(z-k), if k>=z it already has.
    The binomial coefficients get large, so compute in logs: log C(k+z-1, k) = lgamma(k+z) -
lgamma(k+1) - lgamma(z)."""
    if z == 0: return 1.0
    p = 1.0 - q
    if q >= p: return 1.0
    caught, mass = 0.0, 0.0
    for k in range(z):
        log_pk = lgamma(k + z) - lgamma(k + 1) - lgamma(z) + z * log(p) + k * log(q)
        pk = exp(log_pk)
        mass += pk
        caught += pk * (q / p) ** (z - k)
    return caught + (1.0 - mass)

print("Rosenfeld (2014) Table 1, q=10% row: 20%, 5.600%, 1.712%, 0.546%, 0.178%, 0.059%,
0.020%, 0.007%, 0.002%, 0.001%")
print("This book's finite sum, q=10% row:  ", ", ".join(f"{100*exact_P(0.10, n):.3f}%" for n
in range(1, 11)))
print()
print("Grunspan-Pérez-Marco (2018) Table 1: z needed for P<0.1% —  q: 0.10 0.15 0.20 0.25 0.30
0.35 0.40 0.45")
print("  the paper's z (exact formula) : 6 9 13 20 32 58 133 539")
print("  this book's finite sum        :", " ".join(str(min_z(exact_P, q)) for q in (0.10,
0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45)))
print("  the paper's z_SN (Nakamoto)   : 5 8 11 15 24 41 81 340   ← the 81 at q=0.40 differs
from the 89 on p. 8 of the white paper and in NB09-2 above (see the text below)")
print()
print(f"{'q':>5} {'z':>4} {'Nakamoto':>10} {'exact':>10} {'ratio':>6}")
for q, z in ((0.1, 1), (0.1, 3), (0.1, 6), (0.3, 5), (0.3, 10), (0.3, 24), (0.45, 40), (0.45,
340)):
    a, b = nakamoto_P(q, z), exact_P(q, z)
    print(f"{q:>5.2f} {z:>4} {a:>10.7f} {b:>10.7f} {b/a:>6.2f}")
```

```
# Output — bcbook (Python 3.13), exactly as run
Rosenfeld (2014) Table 1, q=10% row: 20%, 5.600%, 1.712%, 0.546%, 0.178%, 0.059%, 0.020%,
0.007%, 0.002%, 0.001%
This book's finite sum, q=10% row:   20.000%, 5.600%, 1.712%, 0.546%, 0.178%, 0.059%, 0.020%,
0.007%, 0.002%, 0.001%

Grunspan-Pérez-Marco (2018) Table 1: z needed for P<0.1% —  q: 0.10 0.15 0.20 0.25 0.30 0.35
0.40 0.45
  the paper's z (exact formula) : 6 9 13 20 32 58 133 539
  this book's finite sum        : 6 9 13 20 32 58 133 539
  the paper's z_SN (Nakamoto)   : 5 8 11 15 24 41 81 340   ← the 81 at q=0.40 differs from the
89 on p. 8 of the white paper and in NB09-2 above (see the text below)

    q    z   Nakamoto      exact  ratio
 0.10    1  0.2045873  0.2000000   0.98
 0.10    3  0.0131722  0.0171200   1.30
 0.10    6  0.0002428  0.0005914   2.44
 0.30    5  0.1773523  0.1976173   1.11
 0.30   10  0.0416605  0.0651067   1.56
 0.30   24  0.0008106  0.0040007   4.94
 0.45   40  0.3454201  0.3713860   1.08
 0.45  340  0.0009947  0.0089679   9.02
```

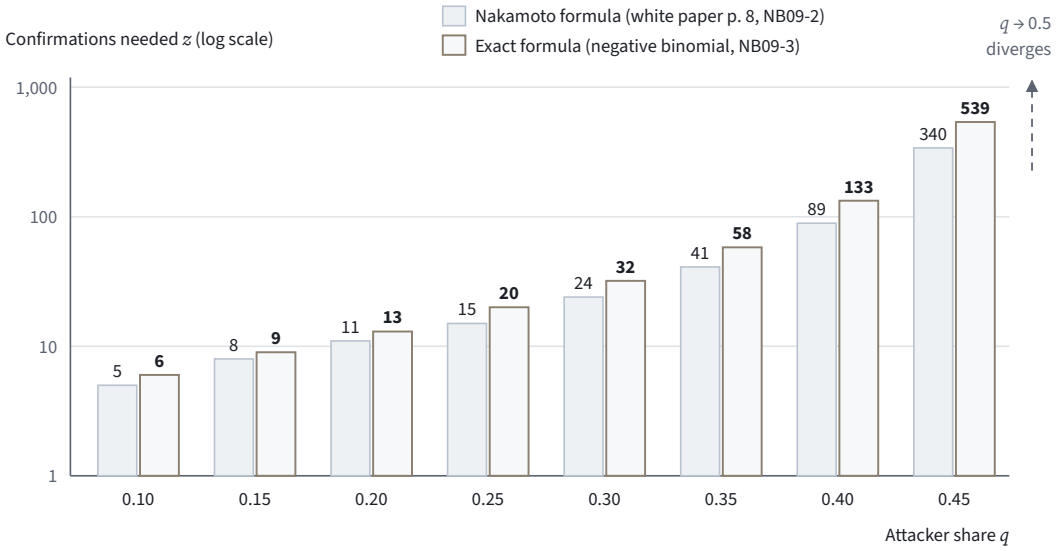


Figure 9-3. Bars comparing the number of confirmations z needed for $P < 0.1\%$ at each attacker share q under the Nakamoto formula (NB09-2) and the exact formula (NB09-3), on a logarithmic vertical scale. The exact formula asks for more confirmations in every column, and the number needed diverges as q approaches 0.5.

[Advanced] The finite sum reproduces both Rosenfeld's table and Grunspan and Pérez-Marco's. The two papers' models look slightly different in their setup (Rosenfeld has the attacker mine one block in advance and counts "getting longer" as success, while Grunspan and Pérez-Marco's formula has no block mined in advance and formally counts a "tie" as success), but they give the same numbers. The ratio column shows that at $z = 1$ the white paper's value is in fact the larger one, and that the exact formula is larger for $z \geq 2$. That is what Grunspan and Pérez-Marco wrote. The ratio widens as z grows and as q grows. At $q = 0.45$ and $z = 340$ it is ninefold.

[Advanced] One discrepancy should be recorded. Table 1 in Grunspan and Pérez-Marco gives the z_{SN} needed under the Nakamoto formula at $q = 0.40$ as **81**, whereas the table on p. 8 of the white paper and this book's recomputation (NB09-2) give **89**. The other seven columns all agree. This book follows the white paper and the recomputation. The z (exact formula) row in Grunspan and Pérez-Marco agrees with this book's finite sum in every column.

```
# NB09-4: Monte Carlo — do not take the formula on trust; run the race yourself (fixed seed)
import random

def simulate(q: float, z: int, trials: int, seed: int = 20261025) -> float:
    """The attacker mines too until the honest chain has mined z blocks (probability q per
block). After that, success if the gap falls to 0 or below; cut off as failure once it exceeds
z+200."""
    rng = random.Random(seed)
    p = 1.0 - q
    wins = 0
    for _ in range(trials):
        honest, attacker = 0, 0
        while honest < z:                        # while the merchant waits for z confirmations
            if rng.random() < q: attacker += 1
            else: honest += 1
        gap = honest - attacker
        while 0 < gap < z + 200:                 # the race after that
            gap += -1 if rng.random() < q else 1
        if gap <= 0: wins += 1
    return wins / trials

for q, z, n in ((0.1, 3, 200_000), (0.1, 6, 200_000), (0.3, 10, 50_000)):
    mc, ex, na = simulate(q, z, n), exact_P(q, z), nakamoto_P(q, z)
    se = (ex * (1 - ex) / n) ** 0.5
    print(f"q={q} z={z:>2} n={n:>7,}: Monte Carlo {mc:.5f} | exact {ex:.5f} | Nakamoto {na:.5f}
| MC standard error ≈ {se:.5f}")
```

```
# Output — bcbook (Python 3.13), exactly as run
q=0.1 z= 3 n=200,000: Monte Carlo 0.01728 | exact 0.01712 | Nakamoto 0.01317 | MC standard error
≈ 0.00029
q=0.1 z= 6 n=200,000: Monte Carlo 0.00061 | exact 0.00059 | Nakamoto 0.00024 | MC standard error
≈ 0.00005
q=0.3 z=10 n= 50,000: Monte Carlo 0.06414 | exact 0.06511 | Nakamoto 0.04166 | MC standard error
≈ 0.00110
```

[Intermediate] The result of the random race agrees with the **exact formula** within the standard error and lies farther from the white paper's value. Even without trusting the formula, the race itself gives the same answer.

```
# NB09-5: invariance to absolute hash rate — only the share q enters the formula
print("attacker hash rate a, honest hash rate h  →  q = a/(a+h),  P(z=6)")
for a, h in ((1, 9), (1e6, 9e6), (1e20, 9e20), (3e20, 7e20)):
    q = a / (a + h)
    print(f"  a={a:.0e} H/s, h={h:.0e} H/s → q={q:.2f}, exact P(6)={exact_P(q, 6):.6f}, Nakamoto
P(6)={nakamoto_P(q, 6):.6f}")
```

```
# Output — bcbook (Python 3.13), exactly as run
attacker hash rate a, honest hash rate h  →  q = a/(a+h),  P(z=6)
  a=1e+00 H/s, h=9e+00 H/s → q=0.10, exact P(6)=0.000591, Nakamoto P(6)=0.000243
  a=1e+06 H/s, h=9e+06 H/s → q=0.10, exact P(6)=0.000591, Nakamoto P(6)=0.000243
  a=1e+20 H/s, h=9e+20 H/s → q=0.10, exact P(6)=0.000591, Nakamoto P(6)=0.000243
  a=3e+20 H/s, h=7e+20 H/s → q=0.30, exact P(6)=0.156450, Nakamoto P(6)=0.132111
```

Scale the total hash rate up by a factor of 10^{20} and the probability stays the same as long as q is the same, because the formula has no place for absolute speed. Three observations by Rosenfeld sum this up. "The probability of success depends on the number of blocks, and not on the time constant T_0." "Waiting for more confirmations exponentially decreases the probability of double-spending success. The decay rate depends on the attacker's relative

hashrate." "The probability of success depends on the number of confirmations and not on the amount of time waited."[12] And one more line: "No amount of confirmations will reduce the success rate to 0."[13]

4. "Done" is a probability

We can now write down the answer that technology can give to "when was the payment completed?" The answer is not a time but **a function**: $P(q, z)$, the probability that an attacker with hash-rate share q reverses the transaction once z blocks have been stacked on it. It shrinks exponentially as z grows but never becomes 0. The closer q is to 1/2, the larger the z needed to bring it down to the same level, without bound, and at $q \geq 1 / 2$ no z brings it down. This is different from the "finality" that payment systems use, a concept under which a rule declares that "from this point on, the transfer cannot be revoked." Which rule of which system fixes that point is something this book could not support with primary sources, so only the concept is noted here (Chapter 28, Section 1(c)). Bitcoin has no such point inside the protocol, and each recipient chooses z according to its own risk preference. Proof of stake in Chapter 10 tries to put a different kind of answer, "economic finality," in this place.

> **? A LAWYER'S QUESTION**
>
> When is a payment to be regarded as completed? When the transaction is propagated to the network, when it enters a block, or when z blocks have been stacked on it? If a contract or terms of service fix "6 confirmations" as the point of completion, does that become the standard? If the chain is later reorganized and the transaction disappears, was there never any performance, or was there performance that then ceased to exist? Where on this probability function should point-in-time concepts such as attachment, consummation of an offense, and delay in performance be pinned? → Chapter 28

▾ **[Advanced] Does the attack pay? Rosenfeld's economic model**

[Advanced] The probabilities above belong to an adversarial setting that assumes the counterparty is an attacker. In §6 Rosenfeld asks a more realistic question. An attacker attacks only when the attack pays. He supposes that the attacker runs the attack against k merchants at once and buys from each a product priced at v, that the real value of the product is αv ($0 < \alpha \leq 1$, a loss on converting it back to money), and that if the attack fails the attacker pays the price kv and loses the reward oB of the o blocks mined on the secret branch. With success probability r, the expected profit is

$$kv(\alpha + r - 1) - (1 - r)\, oB$$

and for this to be positive, $v > \dfrac{(1 - r)\, oB}{k(\alpha + r - 1)}$ must hold.[14] Reading the symbols: B is the block reward, o is the number of blocks thrown away on failure, and $r = P(q, n)$ is the probability above. No numerical example is given here. Rosenfeld himself wrote that

"These values should be taken with a grain of salt, because of the many modeling assumptions made,"[15] and his table presupposes the block reward of the time and arbitrary values of k and α. What this book keeps is the structure. **The larger the transaction amount, the more confirmations the merchant has to require, and the cost of an attack is priced by "the block rewards thrown away when it fails."** This structure leads into the discussion of the security budget in Chapter 20.

5. Summary

Even a majority of the hash rate cannot forge a signature. It can only change the order of history. The probability that the reordering succeeds is a function of q and z alone. It falls exponentially in z but never becomes 0, and at $q \geq 1 / 2$ it does not fall. The white paper's formula is optimistic, and the exact formula asks for more confirmations at the same level of safety (6 rather than 5 at $q = 0.1$, and 32 rather than 24 at $q = 0.3$). "6 confirmations" is a convention, and "done" is a probability. Every number in this chapter was recomputed from the formulas in the original papers and checked against a random race.

Sources for this chapter

- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — §11, Calculations (pp. 6–8): scope of the attack, binomial random walk, gambler's ruin, Poisson approximation, C code, tables.
- Meni Rosenfeld, "Analysis of Hashrate-Based Double Spending," arXiv:1402.2009 [cs.CR], 2014-02-09 (first draft 2012-12-11) — attack steps (pp. 3–4), definition of confirmations (p. 3), negative binomial model and formula (1) (p. 7), observations (pp. 6, 8–9), Table 1 (p. 10), assessment of 6 confirmations (p. 10), economic model (pp. 10–12).
- Cyril Grunspan and Ricardo Pérez-Marco, "Double Spend Races," *International Journal of Theoretical and Applied Finance* 21(8): 1850053, 2018. DOI 10.1142/S021902491850053X; arXiv:1702.02867 (v3, 2020-05-06) — Theorem 1 and Table 1 (p. 2), Theorem 2 (proof of the exponential decay, p. 2).
- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library modules `math` and `random` (finite sums, no special-function package). Notebook NB09. The original reproduction script was taken from the editor's research notes of 2026-09-28 and rewritten.

Notes

1. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), p. 6 (§11, Calculations). Emphasis added.
2. Meni Rosenfeld, "Analysis of Hashrate-Based Double Spending," arXiv:1402.2009 (2014; first draft 2012-12-11), pp. 3–4 (§3, Playing catch-up). Emphasis added.
3. Rosenfeld (2014), p. 3: "A transaction is said to have n confirmations if it is included in a block which is part of the valid chain, and there are n blocks in the path from that block to the leaf of the chain, inclusive."
4. Nakamoto (2008), p. 6: "The race between the honest chain and an attacker chain can be characterized as a

Binomial Random Walk. The success event is the honest chain being extended by one block, increasing its lead by +1, and the failure event is the attacker's chain being extended by one block, reducing the gap by -1. The probability of an attacker catching up from a given deficit is analogous to a Gambler's Ruin problem."

5. Nakamoto (2008), p. 6. The white paper defines q_z as the "probability the attacker will ever catch up from z blocks behind" and states the formula above (for the derivation it points to reference [8], Feller). The derivation above was supplied by this book.

6. Rosenfeld (2014), p. 6.

7. Nakamoto (2008), p. 7. In the original the quoted sentence ends "with expected value:" and the formula λ = z q/p follows on its own line.

8. Nakamoto (2008), p. 7: the formula and the C code that follow "Rearranging to avoid summing the infinite tail of the distribution..." The tables for $q = 0.1$ and $q = 0.3$ and the "P < 0.001" table are on p. 8.

9. Rosenfeld (2014), p. 10.

10. Rosenfeld (2014), p. 7 (§4, Waiting for confirmations): "We will not use this assumption, but rather model m more accurately as a negative binomial variable; it is the number of successes (blocks found by the attacker) before n failures (blocks found by the honest network), with a probability q of success." Rosenfeld's formula (1) assumes that the attacker mined one block in advance of the attack and counts "getting longer" as success. It gives the same values as the formula below, which has no block mined in advance and counts a tie as success (checked against the values in his Table 1).

11. Cyril Grunspan and Ricardo Pérez-Marco, "Double Spend Races," *International Journal of Theoretical and Applied Finance* 21(8): 1850053 (2018), Theorem 1 (arXiv:1702.02867v3, p. 2). The first quotation stops before the displayed definition of $I_x(a, b)$; the second stops mid-sentence, and the original continues "in particular this happens when the share of hashrate q of the attackers is important."

12. Rosenfeld (2014), pp. 6 and 8.

13. Rosenfeld (2014), p. 9.

14. Rosenfeld (2014), pp. 11–12 (§6, Economics of double-spending). Expected profit: "kαv − (1 − r)(kv + oB) = kv(α + r − 1) − (1 − r)oB"; break-even condition: "v > (1 − r)oB / (k(α + r − 1))".

15. Rosenfeld (2014), p. 12.

PART 1 · TECHNICAL FOUNDATIONS

Chapter 10. Proof of stake: collateral instead of energy

KEY TAKEAWAY

Proof of work gives the right to write a block to "whoever burned electricity," and proof of stake (PoS) gives it to "whoever put up a deposit." The source of security changes. In proof of work an attack is carried out by paying for an outside resource (computation); in proof of stake an attack is carried out **at the risk of losing your own deposit** held inside the system. So proof of stake has one device that proof of work lacks: **slashing**, which cuts the deposit of a validator who breaks the rules. Punishment is part of security. After this chapter you will know what validators, stake, attestations, and slashing mean, where the "2/3" rule comes from, and what proof of stake promises in place of the probabilistic finality of Chapter 9.

Learning objectives

1. Explain what validators, stake, attestations, and slashing mean, and say how proof of work and proof of stake differ in "the kind of cost."
2. [Intermediate] Read the two slashing conditions of Casper (Casper FFG) and the "2/3" finality rule, and compute slashing penalties under the rules of the current Ethereum specification.
3. [Advanced] Understand the structure of the attacks peculiar to proof of stake (long-range revision, nothing at stake) and of the defenses against them (withdrawal delay, weak subjectivity, inactivity leak).
4. Explain "staking rewards" not as returns but in terms of mechanism and risk structure.

What a reader of the [Basic] paragraphs alone takes away. Proof of stake is "a deposit instead of electricity," and punishment (slashing) is part of security. Misconception ⑩ in Appendix B ("PoS is a structure in which the rich get richer") can be judged, after reading this chapter, only in part.

1. What failure this technology grew out of

The price of proof of work is electricity and hardware. Bitcoin's safety rests on the fact that "it is hard for an attacker to have a large share of the total hash rate alone" (Chapter 9), and that difficulty comes from the cost of amassing mining equipment and power. That cost leaves the system: miners pay their electricity bills to power companies. What the people who proposed proof of stake put forward was efficiency, that is, doing away with that consumption of resources. And, as this book sums it up, what their design added to proof of work is **punishment**. In proof of work, a miner who breaks the rules loses only "the electricity spent on that block," and the system has no lever inside it for imposing any penalty beyond that.

The 2017 Casper paper by Vitalik Buterin and Virgil Griffith introduces proof of stake this way: "In a PoS system, a blockchain appends and agrees on new blocks through a process where anyone who holds coins inside of the system can participate, and the influence an agent has is proportional to the number of coins (or 'stake') it holds. This is a vastly more efficient alternative to proof of work (PoW) 'mining' and enables blockchains to operate without mining's high hardware and electricity costs."[1] It then states its central claim: "If a validator violates a rule, we can detect the violation and know which validator violated the rule. Accountability allows us to penalize malfeasant validators, solving the 'nothing at stake' problem that plagues chain-based PoS. The penalty for violating a rule is a validator's entire deposit."[2] Being able to punish: that is what proof of stake added to proof of work.

What "nothing at stake" means becomes clear when it is set against proof of work. (The Casper paper only names this problem and does not explain it, so what follows is this book's explanation.) When two branches appear in proof of work (a fork), a miner has to choose one of them, since hash rate can be spent in only one place at a time. Signatures cost nothing, so in a proof of stake system without a punishment device, a validator could sign **both branches** and collect the reward whichever one wins. If everyone does that, the fork never converges. Casper's answer is "if evidence of signing in two places is put on the chain, the deposit is taken away."

2. Intuition: deposits, votes, and punishment

The participants in a proof of stake system are called **validators**. To become a validator, you put coins into the system as **stake** (a deposit). The Casper paper's definition: "Each validator has a deposit; when a validator joins, its deposit is the number of deposited coins. After joining, each validator's deposit rises and falls with rewards and penalties. Proof of stake's security derives from the **size of the deposits**, not the **number of validators**, so for the rest of this paper, when we say '2/3 of validators', we are referring to the deposit-weighted fraction ...."[3] The "one CPU, one vote" of proof of work has become "one coin, one vote."

Validators do two jobs. When their turn comes they **propose** a block, and they **vote** (attest) that a block proposed by another validator is "the head of the correct chain." Casper has validators vote not on every block but on **checkpoints** at fixed intervals, and when validators

with at least 2/3 of the deposits vote for the same pair of checkpoints, that link is called a "supermajority link." A checkpoint reached by a supermajority link is "justified," and when a supermajority link runs from a justified checkpoint to the checkpoint immediately after it, the earlier one is **"finalized."**[4] In place of proof of work's "the more blocks pile up, the more the probability of reversal shrinks, exponentially," Casper sets up a **discrete finality**: "once 2/3 stamp it in two stages, it is done."

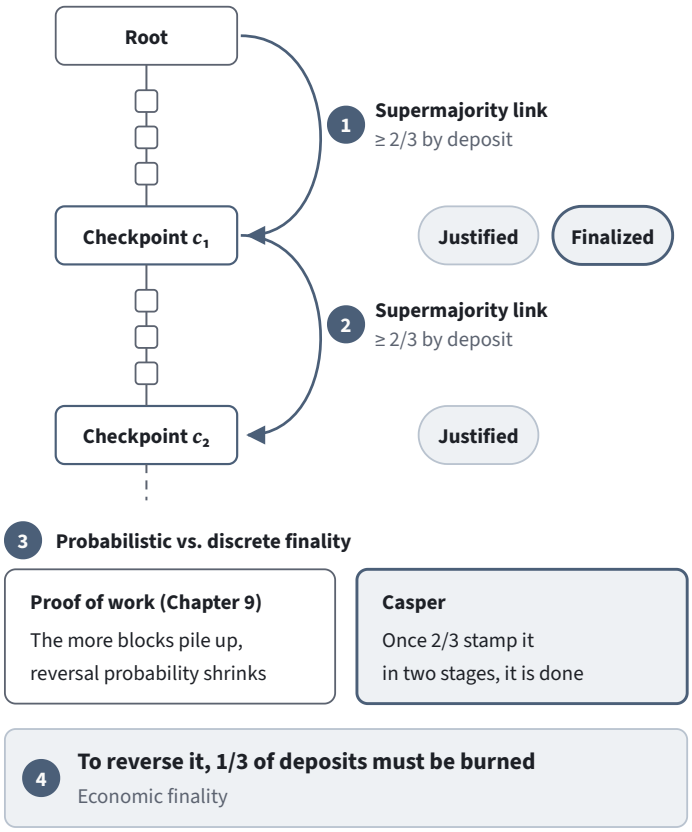


Figure 10-1. Casper's two-stage stamp. A checkpoint is justified when a supermajority link of at least 2/3 by deposit reaches it, and when a supermajority link then runs from it to the checkpoint immediately after it, the earlier one is finalized. The small squares between checkpoints are blocks.

What does that "done" mean? The paper's central result goes like this: "The most notable property of Casper is that it is **impossible for any two conflicting checkpoints to be finalized** unless ≥ 1/3 of the validators violate one of the two Casper Commandments/slashing conditions …."[5] To reverse it, at least 1/3 of all deposits would have to break the rules and leave evidence of doing so, and once that evidence is put on the chain their deposits are gone. It is not "cannot be reversed" but "to reverse it, 1/3 of the deposits must be burned." This is called **economic finality**. It is a different kind of thing from the probabilistic finality of Chapter 9. One is a probability that gets smaller; the other is a price tag.

Two prohibitions: the slashing conditions

Casper's "two commandments" are simple. A validator v must not cast a pair of votes of either of the following kinds. (I) Voting for two different checkpoints at the same height (a double vote). (II) Casting votes such that the source and target of one vote lie entirely between the source and target of another (a surround vote).[6] If evidence that either one was broken is put on the chain as a transaction, "the validator's entire deposit is taken away with a small 'finder's fee' given to the submitter of the evidence transaction."[7] The current Ethereum specification's function `is_slashable_attestation_data` writes these two down in code as they are: a double vote, where the two attestation data differ but have the same target epoch, and a surround vote, where one side's source epoch is smaller than the other's and its target epoch is larger.[8]

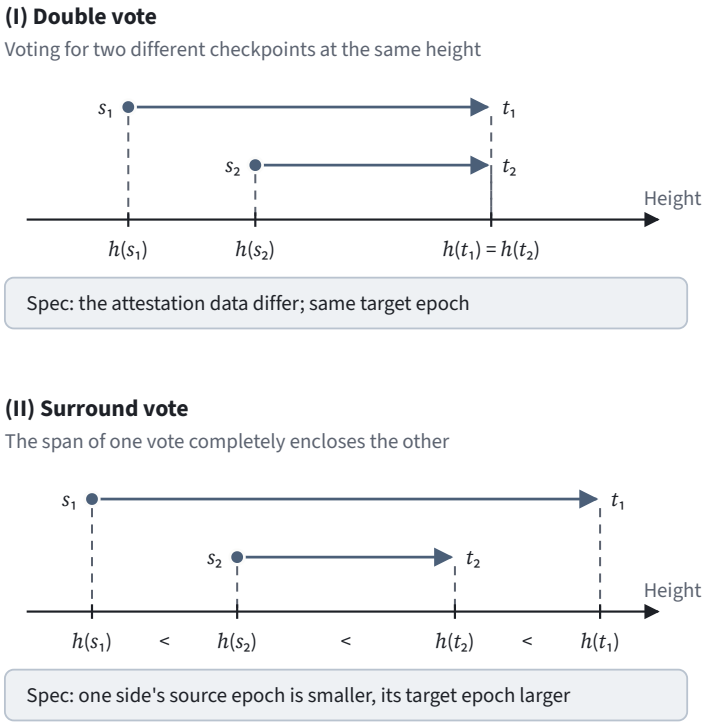


Figure 10-2. Two votes by one validator, drawn as arrows from source s to target t. In (I), a double vote, the target heights are the same; in (II), a surround vote, the span of one vote completely encloses the other. The bottom row gives the conditions in the current specification's function is_slashable_attestation_data.

3. The current rules: numbers from the specification

[Intermediate] Ethereum's proof-of-stake "Beacon Chain" started out with its minimum genesis time set at 12 noon (UTC) on December 1, 2020.[9] It prepared the consensus layer with the Bellatrix fork on September 6, 2022 (epoch 144,896), and later that month, when terminal total difficulty was reached, the execution layer moved from proof of work to proof of stake (the comment in the configuration file gives an estimated date of September 15, 2022). The Capella (2023-04), Deneb (2024-03), Electra (2025-05), and Fulu (2025-12) forks followed.[10] The numbers below were read from the current specification **as of 2026-10-25** (after Fulu was activated). They have changed from fork to fork, so the trend notes in Appendix A update them.[11]

ITEM	CURRENT VALUE (AS OF 2026-10)	MEANING
`MIN_ACTIVATION_BALANCE`	32 ETH	Minimum balance needed for a validator to be activated
`MAX_EFFECTIVE_BALANCE_ELECTRA`	2,048 ETH	Upper limit on the effective balance that one "compounding validator" (0x02 withdrawal credentials; the spec's term) can have (from Electra on)
`MIN_SLASHING_PENALTY_QUOTIENT_ELECTRA`	4,096	Immediate slashing penalty = effective balance ÷ 4,096
`WHISTLEBLOWER_REWARD_QUOTIENT_ELECTRA`	4,096	Whistleblower reward = effective balance ÷ 4,096
`PROPORTIONAL_SLASHING_MULTIPLIER_BELLATRIX`	3	Multiplier for the correlation penalty (below)

[Intermediate] Slashing cuts twice. First comes the **immediate penalty**. The specification's `slash_validator` puts the validator into the exit process, marks it as slashed, and deducts "effective balance ÷ `MIN_SLASHING_PENALTY_QUOTIENT_ELECTRA`."[12] For a 32 ETH validator that is 32/4096 = 0.0078125 ETH. Next comes the **correlation penalty**. About 18 days later (after half of `EPOCHS_PER_SLASHINGS_VECTOR`), `process_slashings` deducts **a further** fraction: the sum of the deposits slashed in the same period, multiplied by 3 (capped at the total active balance), divided by the total active balance.[13] If a validator breaks the rules alone, the second penalty is close to 0, but if 1/3 of the deposits break them together, $3 \times 1 / 3 = 1$, and they lose **everything**. The Casper paper's "entire deposit" has been implemented in the current rules as a proportional penalty that "comes closer to everything the more validators break the rules together." A small mistake (one validator double-signing through an operational accident) is treated lightly, and an organized attack (collusion by 1/3) is fatal. This proportionality is the heart of the design.

What is the staking "reward" paid for?

For proposing blocks and taking part in attestations, validators receive coins newly issued by the protocol (for their share of transaction fees, see Chapter 13 and Chapter 21). This is often called "staking returns," but this book does not use that term. Looked at structurally, it is a **payment by the protocol** for three things: (1) the opportunity cost of keeping a deposit locked up, (2) the labor and equipment of running a node 24 hours a day, and (3) the risk of losing the deposit through an operational mistake or through taking part in an attack. On the receiving side it is a reward; on the system's side it is the cost of maintaining security (Chapter 20). And the risk is not abstract. A single operational accident, running two nodes with the same key at the same time, produces a double signature and gets the validator slashed. This book names no staking service and gives no reward rate.

> **? A LAWYER'S QUESTION**
>
> What is a staking reward paid for? Is it consideration for a **service**, the service of validation? Is it **fruits** borne by the deposited coins? Or is it **original acquisition** of new coins? If a deposit disappears through slashing, is that a fine, a contractual penalty, or the enforcement of a security interest? Is a business that takes in other people's coins and stakes them on their behalf a trustee or a debtor? → Chapter 28

4. Code: stake-weighted selection and slashing arithmetic

[Intermediate] The actual specification runs to thousands of lines, but the two core pieces of arithmetic are short: drawing proposers in proportion to stake, and computing slashing penalties. All of it is a toy, not for real use.

```
# NB10-1: stake-weighted random selection — checking "one coin, one vote" by simulation (for
study, fixed seed)
import random
rng = random.Random(20261025)

stakes = {"A": 32, "B": 32, "C": 64, "D": 128, "E": 256}      # stake per validator (toy values
in ETH)
total = sum(stakes.values())
names, weights = list(stakes), list(stakes.values())

N = 100_000
picks = {n: 0 for n in names}
for _ in range(N):                                             # in each slot, draw one proposer
in proportion to stake
    picks[rng.choices(names, weights=weights)[0]] += 1

print(f"{'validator':>9} {'stake':>6} {'stake share':>11} {'pick share':>10}")
for n in names:
    print(f"{n:>9} {stakes[n]:>6} {stakes[n]/total:>11.3f} {picks[n]/N:>10.3f}")
print('Twice the stake, twice the picks: "one coin, one vote." If the reward is proportional to
the number of picks, it is proportional to stake.')
print('What is preserved, though, is the ratio: if everyone grows at the same rate, the absolute
gap grows at that rate and the ratio stays the same. On this point misconception ⑩ in Appendix
B is judged "Holds in part."')
```

```
# Output — bcbook (Python 3.13), exactly as run
validator  stake stake share pick share
        A     32       0.062      0.062
        B     32       0.062      0.061
        C     64       0.125      0.126
        D    128       0.250      0.253
        E    256       0.500      0.498
Twice the stake, twice the picks: "one coin, one vote." If the reward is proportional to the
number of picks, it is proportional to stake.
What is preserved, though, is the ratio: if everyone grows at the same rate, the absolute gap
grows at that rate and the ratio stays the same. On this point misconception ⑩ in Appendix B is
judged "Holds in part."
```

```
# NB10-2: slashing penalties — the immediate penalty and the correlation penalty under the
current specification's rules (Electra and later), keeping the specification's order of integer
divisions
GWEI = 10**9
MIN_SLASHING_PENALTY_QUOTIENT_ELECTRA = 4096
WHISTLEBLOWER_REWARD_QUOTIENT_ELECTRA = 4096
PROPORTIONAL_SLASHING_MULTIPLIER_BELLATRIX = 3
EFFECTIVE_BALANCE_INCREMENT = 1 * GWEI                          # 1 ETH = 10^9 Gwei
(presets/mainnet/phase0.yaml)

def immediate_penalty(effective_balance_gwei: int) -> int:     # slash_validator
    return effective_balance_gwei // MIN_SLASHING_PENALTY_QUOTIENT_ELECTRA

def correlation_penalty(effective_balance_gwei: int, slashed_sum_gwei: int, total_active_gwei:
int) -> int:   # process_slashings
    increment = EFFECTIVE_BALANCE_INCREMENT
    adjusted = min(slashed_sum_gwei * PROPORTIONAL_SLASHING_MULTIPLIER_BELLATRIX,
total_active_gwei)
    per_increment = adjusted // (total_active_gwei // increment)
    return (effective_balance_gwei // increment) * per_increment

eb = 32 * GWEI                                                  # one 32 ETH validator
total_active = 1_000_000 * 32 * GWEI                            # assumption: 1 million
validators of 32 ETH = 32 million ETH active (a toy assumed value)
print(f"immediate penalty: {immediate_penalty(eb)/GWEI:.7f} ETH (= 32/4096), whistleblower
reward: {eb // WHISTLEBLOWER_REWARD_QUOTIENT_ELECTRA / GWEI:.7f} ETH")
print(f"{'share slashed in same period':>28} {'correlation penalty (ETH)':>25} {'of
balance':>10}")
for frac in [0.0001, 0.01, 0.10, 1/3, 0.5]:
    slashed_sum = int(total_active * frac)
    pen = correlation_penalty(eb, slashed_sum, total_active)
    print(f"{frac:>28.4%} {pen/GWEI:>25.4f} {pen/eb:>10.1%}")
print("Break the rules alone and the second penalty is close to 0; if 1/3 of all stake breaks
them together, 3 × 1/3 = 100% — everything is lost.")
```

```
# Output — bcbook (Python 3.13), exactly as run
immediate penalty: 0.0078125 ETH (= 32/4096), whistleblower reward: 0.0078125 ETH
share slashed in same period correlation penalty (ETH) of balance
                      0.0100%                    0.0096       0.0%
                      1.0000%                    0.9600       3.0%
                     10.0000%                    9.6000      30.0%
                     33.3333%                   32.0000     100.0%
                     50.0000%                   32.0000     100.0%
Break the rules alone and the second penalty is close to 0; if 1/3 of all stake breaks them
together, 3 × 1/3 = 100% — everything is lost.
```

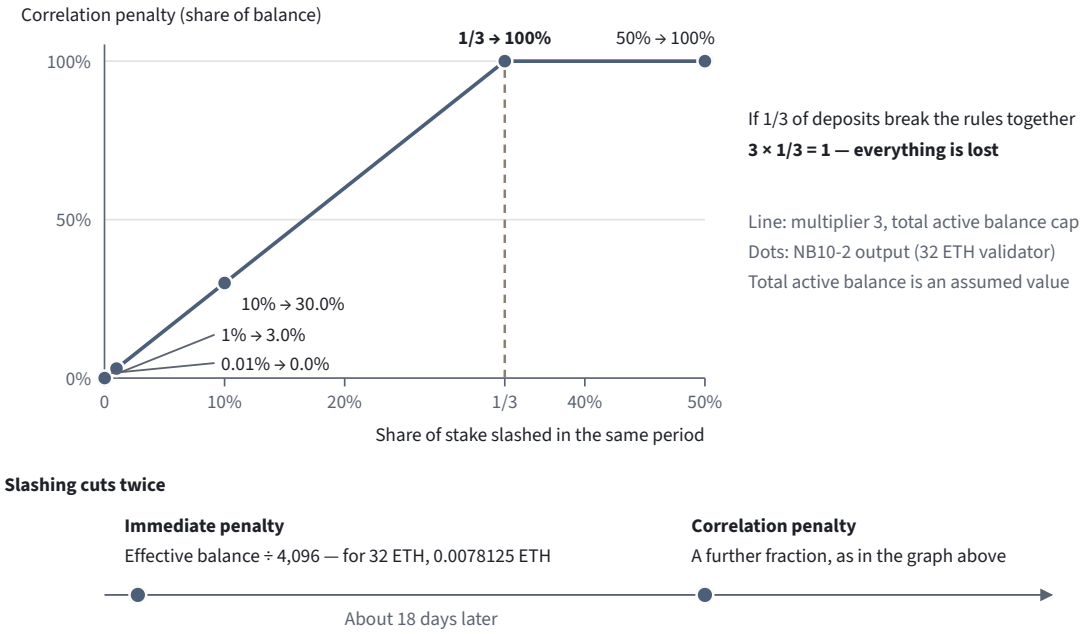


Figure 10-3. The correlation penalty (as a share of balance) against the share of stake slashed in the same period. The dots are the output of NB10-2, and the line is drawn from the multiplier 3 and the cap at the total active balance. The total active balance is NB10-2's assumed value for illustration. The band below shows the order of events: the immediate penalty, then the correlation penalty about 18 days later.

[Intermediate] The shape of the penalty table is the design intent itself. For an attack to succeed, 1/3 of the deposits must collude, and that is exactly where the penalty reaches 100%. The price tag of economic finality, "the cost of reversal = 1/3 of total deposits," is written into the code in this way. The `total_active` above is an assumed value for illustration. The actual total active balance changes every epoch, and this book does not give it.

▾ [Advanced] Two attacks peculiar to proof of stake, and the defenses: long-range revision and catastrophic crash

[Advanced] There are two attacks that proof of work does not have. §4 of the Casper paper deals with them.

Long-range revision. Former validators who have already withdrawn their deposits have nothing more to lose. "Once a coalition of validators has withdrawn their deposits, if that coalition had more than 2/3 of deposits long ago in the past, they can use their historical supermajority to finalize conflicting checkpoints without fear of getting slashed (because they have already withdrawn their money)."[14] In proof of work, rewriting the past means redoing the past work, but signatures cost nothing, so rewriting the past with past keys costs nothing. There are two defenses. One is a **withdrawal delay** before a deposit can be taken out (the paper says approximately "four months worth of blocks"[15]). The other is the assumption that **clients never revert a finalized block and log on at some regular interval (for example, once every 1–2 months) to check the latest chain.**[16] The second assumption commonly goes by the name "weak subjectivity," but that name does not appear in the Casper paper or in the

copies of the specification this book read, so only its meaning is given here. A node that has been away for a long time cannot tell by itself which chain is the real one and has to rely on outside information (social consensus). A proof-of-work node only had to count the work from genesis onward. This difference adds a new layer to the question in Chapter 23, "who decides which chain is the real one?"

Catastrophic crash. If more than 1/3 of the validators drop out at once (a network partition, a failure, or on purpose), supermajority links cannot be formed and **nothing is finalized.** The chain grows, but no stamp is applied. The paper's prescription is the "inactivity leak": slowly draining the deposits of validators who do not vote, and waiting until the validators who remain and vote again make up 2/3.[17] This device can create a situation in which two chains are each finalized by a different group. For that case the paper writes that "each validator should simply favor whatever finalized checkpoint it saw first," then concedes that "The exact algorithm for recovering from these various attacks remains an open problem," and assumes that validators who see obviously malfeasant behavior will manually create a "minority soft fork" and that the market will choose that fork.[18] The finality of proof of stake is discrete and tidy in normal times, but in an emergency it falls back on social coordination.

5. Proof of work and proof of stake: different kinds of cost

Here are the two systems in one table. It is not a table of which is better but of **what is different**.

	PROOF OF WORK	PROOF OF STAKE
Right to write a block	Whoever wins the hash lottery	A validator drawn in proportion to stake
Unit of one vote	Computation ("one CPU, one vote")	Stake ("one coin, one vote")
Nature of the cost of an attack	Resources bought **outside** the system (equipment, power)	Loss of deposits placed **inside** the system
Penalty for breaking the rules	The resources spent on that block are wasted	Slashing (deposit cut, in proportion to the scale of collusion)
Finality	Probabilistic: the probability of reversal falls exponentially as blocks pile up, but is not 0	Economic: settled by a two-stage vote of 2/3; reversing it means at least 1/3 of stake is slashed
A node that has been away for a long time	Knows the real chain by itself, by counting the work	Has to get the most recent finalized point from outside (assumes periodic log-on)
Characteristic weakness	Concentration of 51% of the hash rate	Long-range revision, catastrophic crash, concentration of stake

What should we make of the claim that "PoS is a structure in which the rich get richer"? If validators are drawn in proportion to stake and rewards are proportional too, **the ratios** are preserved (NB10-1 above). If everyone grows at the same rate, the gap stays the same as a ratio. But the compounding effect of large participants putting their rewards back into stake, economies of scale in running nodes, and the threshold of a 32 ETH minimum stake can

work against small participants. In proof of work, too, economies of scale in equipment and power work in the same direction. So Appendix B places this proposition at "Holds in part." Proportionality by itself does not widen the gap; thresholds and economies of scale can. Which system is more concentrated is a matter for measurement, and this book does not give those figures.

6. Summary

Proof of stake is "a deposit instead of electricity." Because there are deposits, punishment is possible, and because there is punishment, it can promise economic finality: "once 2/3 stamp it, it is done." The other side of that promise is the proportional slashing penalty, and where the promise does not reach (long-range revision, catastrophic crash), auxiliary devices come in: the withdrawal delay, weak subjectivity, and the inactivity leak. The current numbers (32 ETH, 1/4096, multiplier 3) are values that have changed from fork to fork, and they are as of 2026-10. The next chapter, Chapter 11, sets consensus aside and looks at "two ways a ledger records balances."

Sources for this chapter

- Vitalik Buterin and Virgil Griffith, "Casper the Friendly Finality Gadget," arXiv:1710.09437v4 [cs.CR], 2019-01-22 (v1 2017-10-25). https://arxiv.org/abs/1710.09437 — introduction to PoS and accountability (p. 1), deposits and 2/3 (p. 2), justification and finalization (p. 3), the two commandments, slashing, and Theorem 1 (p. 4), withdrawal delay (p. 6), long-range revision and weak subjectivity (pp. 6–7), inactivity leak and the open problem (p. 8).
- ethereum/consensus-specs (master commit b834fa8411cb2152563e8835c5b72c28f7503e7a, 2026-10-23): `specs/phase0/beacon-chain.md` (`is_slashable_attestation_data`), `specs/electra/beacon-chain.md` (Gwei values, Rewards and penalties, `slash_validator`, `process_slashings`), `configs/mainnet.yaml` (genesis time, fork epochs), `presets/mainnet/*.yaml` (history of the constants). Checked against the `/eth/v1/config/spec` response of a public beacon node. Accessed 2026-10-25.
- go-ethereum v1.17.10 (commit 1df30b18), `params/chainspecs/mainnet.json` — terminal total difficulty.
- This book's `widgets/data.json`, `ethereum.staking` (as_of 2026-10-25) — the constants above and their permalinks.
- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library module `random`. Notebook NB10.

Notes

1. Vitalik Buterin and Virgil Griffith, "Casper the Friendly Finality Gadget," arXiv:1710.09437v4 [cs.CR] (2019-01-22; v1 2017-10-25), p. 1 (§1). There is no journal version; the arXiv version (v4) is cited.
2. Same paper, p. 1 (§1.1): "Accountability. If a validator violates a rule, we can detect the violation and know which validator violated the rule. Accountability allows us to penalize malfeasant validators, solving the 'nothing at stake' problem that plagues chain-based PoS. The penalty for violating a rule is a validator's entire

deposit." The same paragraph claims: "Because proof of stake security is based on the size of the penalty, which can be set to greatly exceed the gains from the mining reward, proof of stake provides strictly stronger security incentives than proof of work." That is the authors' claim. This book does not repeat their assessment and carries over only the structure. For a comparison of costs, see Chapter 20.

3. Buterin and Griffith (2019), p. 2 (§2). Emphasis added.

4. Same paper, p. 3 (§2): "A supermajority link is an ordered pair of checkpoints (a, b) … such that at least 2/3 of validators (by deposit) have published votes with source a and target b." "A checkpoint c is called justified if (1) it is the root, or (2) there exists a supermajority link c′ → c where checkpoint c′ is justified." "A checkpoint c is called finalized if (1) it is the root or (2) it is justified and there is a supermajority link c → c′ where c′ is a direct child of c." The paper takes blocks whose height is a multiple of 100 as checkpoints (main text, p. 2), and in footnote 2 it expresses the gap between checkpoints as "99 blocks." Current Ethereum uses different parameters, in units of epochs.

5. Same paper, p. 4 (§2). Emphasis added. Theorem 1 (Accountable Safety, p. 4): "Two conflicting checkpoints am and bn cannot both be finalized."

6. Same paper, p. 4 (Figure 2, "The two Casper Commandments"): "AN INDIVIDUAL VALIDATOR ν MUST NOT PUBLISH TWO DISTINCT VOTES … SUCH THAT EITHER: I. h(t1) = h(t2). Equivalently, a validator must not publish two distinct votes for the same target height. OR II. h(s1) < h(s2) < h(t2) < h(t1). Equivalently, a validator must not vote within the span of its other votes."

7. Same paper, p. 4: "If a validator violates either slashing condition, the evidence of the violation can be included into the blockchain as a transaction, at which point the validator's entire deposit is taken away with a small 'finder's fee' given to the submitter of the evidence transaction."

8. ethereum/consensus-specs, `specs/phase0/beacon-chain.md`, function `is_slashable_attestation_data` (master commit b834fa84, 2026-10-23). The comment in the original: "Check if data_1 and data_2 are slashable according to Casper FFG rules." The code: `(data_1 != data_2 and data_1.target.epoch == data_2.target.epoch)` (Double vote) `or (data_1.source.epoch < data_2.source.epoch and data_2.target.epoch < data_1.target.epoch)` (Surround vote). Accessed 2026-10-25.

9. consensus-specs `configs/mainnet.yaml` (commit b834fa84): `MIN_GENESIS_TIME: 1606824000`, with the comment "Dec 1, 2020, 12pm UTC". This value is a lower bound on the genesis time, and the same file has `GENESIS_DELAY: 604800` (7 days). This book did not separately confirm the actual genesis time from its source material.

10. Same file (the time portion of each comment omitted): `ALTAIR_FORK_EPOCH: 74240 # Oct 27, 2021, …`, `BELLATRIX_FORK_EPOCH: 144896 # Sept 6, 2022, …`, `CAPELLA_FORK_EPOCH: 194048 # April 12, 2023, …`, `DENEB_FORK_EPOCH: 269568 # March 13, 2024, …`, `ELECTRA_FORK_EPOCH: 364032 # May 7, 2025, …`, `FULU_FORK_EPOCH: 411392 # December 3, 2025, …`. The execution-layer transition (reaching terminal total difficulty) is recorded only as an estimated date, in the comment "# Estimated on Sept 15, 2022" attached to `TERMINAL_TOTAL_DIFFICULTY: 58750000000000000000000` in the same file. go-ethereum v1.17.10 `params/chainspecs/mainnet.json` gives the same terminal total difficulty.

11. The values come from consensus-specs `specs/electra/beacon-chain.md` (the Gwei values and Rewards and penalties tables, `slash_validator`, `process_slashings`) and from the `/eth/v1/config/spec` response of a public beacon node (the two agree). They are recorded, with permalinks, in this book's `widgets/data.json` under `ethereum.staking`. Accessed 2026-10-25 (2026-10-24 UTC).

12. consensus-specs `specs/electra/beacon-chain.md`, `slash_validator`: `initiate_validator_exit(state, slashed_index)`; `validator.slashed = Boolean(True)`; `validator.withdrawable_epoch = max(validator.withdrawable_epoch, epoch + EPOCHS_PER_SLASHINGS_VECTOR)`; `state.slashings[epoch % EPOCHS_PER_SLASHINGS_VECTOR] += validator.effective_balance`; `slashing_penalty = validator.effective_balance // MIN_SLASHING_PENALTY_QUOTIENT_ELECTRA`; `decrease_balance(state, slashed_index, slashing_penalty)`; the whistleblower reward, `whistleblower_reward = validator.effective_balance // WHISTLEBLOWER_REWARD_QUOTIENT_ELECTRA`.

13. Same file, `process_slashings`: `adjusted_total_slashing_balance = min(Gwei(sum(state.slashings)) * PROPORTIONAL_SLASHING_MULTIPLIER_BELLATRIX, total_balance)`; `penalty_per_effective_balance_increment = adjusted_total_slashing_balance // (total_balance // increment)`; each slashed validator for which `epoch + EPOCHS_PER_SLASHINGS_VECTOR // 2 ==`

`validator.withdrawable_epoch` receives a penalty proportional to its number of effective-balance increments. `EPOCHS_PER_SLASHINGS_VECTOR` is 8,192 epochs (`presets/mainnet/phase0.yaml`). History of the quotients: Phase0 128·512·1 → Altair 64·—·2 → Bellatrix 32·—·3 → Electra 4096·4096·3 (the preset file of each fork).

14. Buterin and Griffith (2019), p. 6 (§4.1): "Once a coalition of validators has withdrawn their deposits, if that coalition had more than 2/3 of deposits long ago in the past, they can use their historical supermajority to finalize conflicting checkpoints without fear of getting slashed (because they have already withdrawn their money). This is called the long-range revision attack ...".

15. Same paper, p. 6 (§3): "At the start of the end dynasty, the validator's deposit is locked for a long period of time, called the withdrawal delay (approximately 'four months worth of blocks'), before the deposit is withdrawn."

16. Same paper, p. 6 (§4.1): "In simple terms, long-range attacks are prevented by a fork choice rule to never revert a finalized block, as well as an expectation that each client will 'log on' and gain a complete up-to-date view of the chain at some regular frequency (e.g., once per 1–2 months)."

17. Same paper, p. 8 (§4.2): "We can recover from this by instituting an 'inactivity leak' which slowly drains the deposit of any validator that does not vote for checkpoints, until eventually its deposit sizes decrease low enough that the validators who are voting are a supermajority."

18. Same paper, p. 8: "If this situation happens, then each validator should simply favor whatever finalized checkpoint it saw first. The exact algorithm for recovering from these various attacks remains an open problem."

PART 1 · TECHNICAL FOUNDATIONS

Chapter 11. UTXOs and accounts: two ways a ledger records "balances"

KEY TAKEAWAY

Bitcoin has no "account balance." What it has is a heap of **unspent transaction outputs (UTXOs)**, and "my balance" is a figure the wallet adds up on the spot from the outputs my keys can open. Ethereum, by contrast, keeps an **account** for each address and records in it a balance and a sequence number (the nonce). The two models do the same job in different ways. Neither is the right one; what differs is the shape of parallel processing, privacy, programmability, and mistaken transfers. After this chapter you will see that "I sent coins by mistake" is a different event in each of the two systems.

Learning objectives

1. Compare Bitcoin's unspent output (UTXO) model with Ethereum's account-and-nonce model, and say what a "balance" is in each.
2. [Intermediate] Imitate the state transition functions of the two models in code, and see where double spending is stopped.
3. Distinguish the shapes that a "mistaken transfer" takes in the two models.
4. Understand what change outputs and address reuse imply for privacy.

What a reader of the [Basic] paragraphs alone takes away. Bitcoin has no "account balance." There is only the sum of the transaction outputs not yet spent. That is why a "mistaken transfer" arises differently in the two systems.

1. What failure this technology grew out of

A bank ledger is kept account by account. Each account has a balance, and a transfer subtracts from one account and adds to another. This method presupposes that **only one party writes the ledger**. The question of what happens when two teller windows withdraw from the same account at the same moment, the bank solves with locks on a central server. A blockchain, though, has no central server. Anyone can create a transaction and broadcast it, and who came first is settled only when it gets into a block (Chapter 9). So instead of recording balances, Nakamoto defined **the coin itself as a chain of successive signatures** (Chapter 5) and made "the output not yet spent" at the end of that chain the thing that carries value. If balances are not kept as state, the very problem of "changing the same state twice" disappears. If two transactions try to spend the same output, only one of them can get into a block, and the other is invalid on the ground that "it refers to an output already spent."

Ethereum took a different road, because for contract code to read and write state, "state" has to exist explicitly. Buterin's 2014 white paper lists several limitations of Bitcoin's UTXO scripts, among them "[v]alue-blindness," the inability to control in fine detail the amount withdrawn from an output, and "[l]ack of state," the fact that an output is either spent or unspent and nothing else, so that multi-stage contracts cannot be built.[1] Ethereum keeps state explicitly, account by account. The two models came out of different purposes.

2. UTXO: a heap of unspent outputs

A Bitcoin transaction is made up of **inputs** and **outputs**. §9 of the white paper: "To allow value to be split and combined, transactions contain multiple inputs and outputs. Normally there will be either a single input from a larger previous transaction or multiple inputs combining smaller amounts, and at most two outputs: one for the payment, and one returning the change, if any, back to the sender."[2] Each output records an amount and "the condition under which it can be spent" (a hash of a public key, or the public key itself; see Chapter 5 and Chapter 12). Each input points to **which output of an earlier transaction** it spends and carries a signature that satisfies that output's condition.

Once an output has been used as an input, it is "spent" forever. The set of outputs not yet spent is the **UTXO set (Unspent Transaction Output set)**, and this is Bitcoin's "state." Buterin summed this up neatly as Bitcoin's state transition: "The 'state' in Bitcoin is the collection of all coins (technically, 'unspent transaction outputs' or UTXO) that have been minted and not yet spent, with each UTXO having a denomination and an owner (defined by a 20-byte address which is essentially a cryptographic public key)". And the state transition function `APPLY(S, TX) → S'` returns an error if a UTXO that an input points to is not in the state, returns an error if the signature does not match the owner of that UTXO, returns an error if the input denominations add up to less than the output denominations, and otherwise returns the state with all input UTXOs removed and all output UTXOs added.[3]

Look for the "balance" in this definition. There is none. A person's balance is "the sum of the denominations of the UTXOs that person's keys can open," and that is not a number the

ledger records but **a number the wallet adds up by scanning the UTXO set**. Transaction fees come out of the same place. If the inputs add up to more than the outputs, the difference is the fee, and, as §6 of the white paper says, the miner who made the block takes it.

Change and privacy

An output can be spent only whole. To send 3 coins from a 10-coin output, you create two outputs: 3 coins to the recipient and 7 coins to **a new address of your own**. The 7-coin output is the change. §10 of the white paper advises using a new key for each transaction but goes on to state the limit: "Some linking is still unavoidable with multi-input transactions, which necessarily reveal that their inputs were owned by the same owner. The risk is that if the owner of a key is revealed, linking could reveal other transactions that belonged to the same owner."[4] In the UTXO model, privacy comes not from "addresses being anonymous" but from "the habit of switching addresses," and that habit is broken by input merging and change patterns. This is the basis for misconception ⑧ in Appendix B ("blockchains are anonymous").

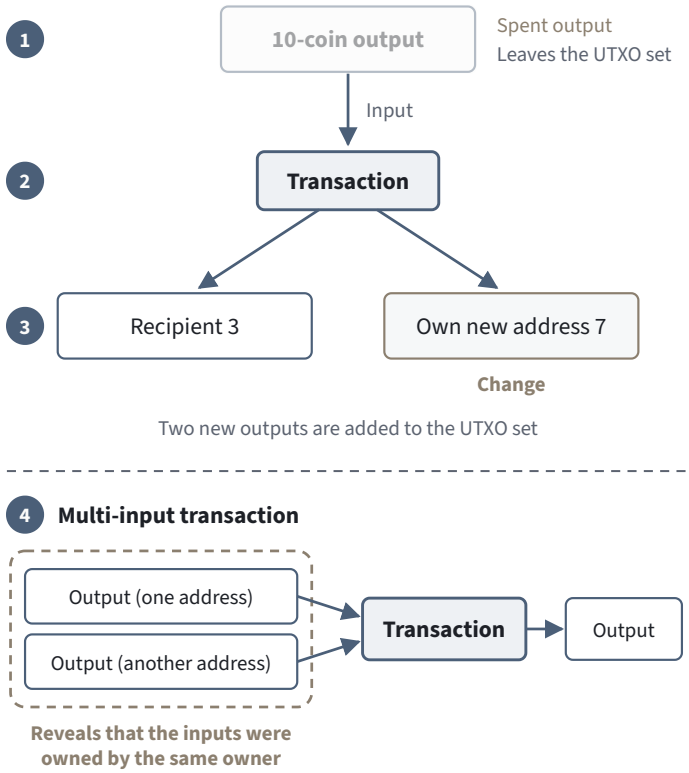


Figure 11-1. One 10-coin output is spent as an input and becomes two outputs, 3 to the recipient and 7 to the sender's own new address (change), and the spent output leaves the UTXO set. Below: how a transaction that merges several inputs reveals that those inputs belonged to the same owner (white paper §10).

3. Accounts: recording state for each address

Ethereum's state is a collection of "accounts." Buterin's white paper: "In Ethereum, the state is made up of objects called 'accounts', with each account having a 20-byte address and state transitions being direct transfers of value and information between accounts. An Ethereum account contains four fields: The **nonce**, a counter used to make sure each transaction can only be processed once; The account's current ether balance; The account's contract code, if present; The account's storage (empty by default)".[5] The Yellow Paper defines the same four fields formally. The world state σ is "a mapping between addresses (160-bit identifiers) and account states," and the account state $\sigma[a]$ consists of nonce ("the number of transactions sent from this address or, in the case of accounts with associated code, the number of contract-creations made by this account"), balance ("the number of Wei owned by this address"), storageRoot (the root hash of the Merkle Patricia tree that holds the storage), and codeHash (the hash of the EVM code).[6] There are two kinds of account: **externally owned accounts**, moved by private keys, and **contract accounts**, moved by code.[7] Contracts are the subject of Chapter 13.

Here the balance is **a number written in the ledger**. A transfer lowers the balance of the sending account and raises the balance of the receiving account. How, then, is the original problem, "subtracting twice from the same balance," prevented? **With the nonce**. An account's nonce is the number of transactions that account has sent, and each transaction carries a nonce saying "this is my nth transaction." A transaction whose nonce does not match is invalid. Even if the same signed transaction is broadcast twice, the second one is rejected because its nonce has already gone by. In the UTXO model, "the output has already been spent" did that job; in the account model, "the nonce has already been used" does it.

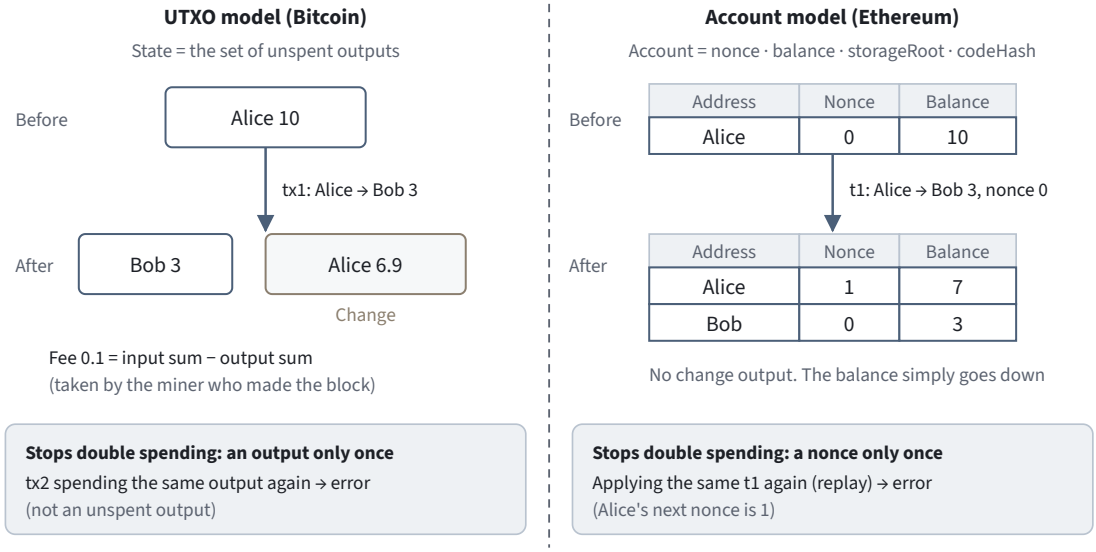


Figure 11-2. The same payment (Alice → Bob 3) placed in the two ledgers. On the left is tx1 of NB11-1, from which a fee of 0.1 is deducted; on the right is t1 of NB11-2, and the account-side example leaves the fee out. What prevents double spending is, respectively, "an output only once" and "a nonce only once."

4. [Intermediate] Code: two state transition functions

[Intermediate] Signature verification was done in Chapter 5, so here names stand in for "who has the key." Both functions are direct transcriptions of Buterin's description of `APPLY(S, TX)`.

```
# NB11-1: UTXO state transition — remove from the set the outputs the inputs point to, and add
the new outputs (for study)
import hashlib, json

def txid(tx: dict) -> str:
    return hashlib.sha256(json.dumps(tx, sort_keys=True).encode()).hexdigest()[:12]

def apply_utxo(S: dict, tx: dict):
    """S: {(txid, index): {"owner":..., "value":...}}.  tx: {"inputs":[(txid, index, signer)],
"outputs":[(owner, value)]}
    The three APPLY steps of Buterin (2014) as they are. On success returns (new state, fee); on
failure, an error string."""
    S = dict(S)
    in_sum = 0
    for (prev_id, idx, signer) in tx["inputs"]:
        key = (prev_id, idx)
        if key not in S:
            return f"error: input {key} is not an unspent output (already spent or nonexistent)"
        if S[key]["owner"] != signer:
            return f"error: the owner of input {key} is {S[key]['owner']} but the signer is
{signer}"
        in_sum += S[key]["value"]
    out_sum = sum(v for _, v in tx["outputs"])
    if in_sum < out_sum:
        return f"error: input sum {in_sum} < output sum {out_sum}"
    for (prev_id, idx, _) in tx["inputs"]:
        del S[(prev_id, idx)]
    tid = txid(tx)
    for i, (owner, value) in enumerate(tx["outputs"]):
        S[(tid, i)] = {"owner": owner, "value": value}
    return S, in_sum - out_sum

def balance(S, who):                        # the "balance" is not in the ledger — it is added up
each time
    return sum(o["value"] for o in S.values() if o["owner"] == who)

coinbase = {"inputs": [], "outputs": [("Alice", 10)]}          # 10 to Alice as a block reward
S = {(txid(coinbase), 0): {"owner": "Alice", "value": 10}}
print("initial UTXO set:", {k: v for k, v in S.items()})

tx1 = {"inputs": [(txid(coinbase), 0, "Alice")], "outputs": [("Bob", 3), ("Alice", 6.9)]}   #
pay 3 + 6.9 change, 0.1 fee
S, fee = apply_utxo(S, tx1)
print("after tx1 — balance Alice:", balance(S, "Alice"), "Bob:", balance(S, "Bob"), "| fee:",
round(fee, 2))
print("UTXO set size:", len(S), "(the one output of 10 is gone, and two outputs, 3 and 6.9, have
appeared)")

tx2 = {"inputs": [(txid(coinbase), 0, "Alice")], "outputs": [("Carol", 10)]}                # a
double spend that tries to spend the same output again
print("tx2 (double spend):", apply_utxo(S, tx2))
tx3 = {"inputs": [(txid(tx1), 0, "Mallory")], "outputs": [("Mallory", 3)]}                 #
someone else's output
print("tx3 (someone else's output):", apply_utxo(S, tx3))
tx4 = {"inputs": [(txid(tx1), 0, "Bob")], "outputs": [("Carol", 3)]}                       # Bob
sends the 3 he received to Carol
S, fee = apply_utxo(S, tx4)
print("after tx4 — Bob:", balance(S, "Bob"), "Carol:", balance(S, "Carol"))
```

```
# Output — bcbook (Python 3.13), exactly as run
initial UTXO set: {('02c83c8a0105', 0): {'owner': 'Alice', 'value': 10}}
after tx1 — balance Alice: 6.9 Bob: 3 | fee: 0.1
UTXO set size: 2 (the one output of 10 is gone, and two outputs, 3 and 6.9, have appeared)
tx2 (double spend): error: input ('02c83c8a0105', 0) is not an unspent output (already spent or
nonexistent)
tx3 (someone else's output): error: the owner of input ('99736848c857', 0) is Bob but the signer
is Mallory
after tx4 — Bob: 0 Carol: 3
```

```
# NB11-2: account state transition — subtract and add balances, but use the nonce to block "the
same transaction twice" (for study)
def apply_account(S: dict, tx: dict):
    """S: {address: {"nonce": n, "balance": b}}.  tx: {"from", "to", "value", "nonce"}"""
    S = {k: dict(v) for k, v in S.items()}
    sender = S.setdefault(tx["from"], {"nonce": 0, "balance": 0})
    if tx["nonce"] != sender["nonce"]:
        return f"error: the next nonce of {tx['from']} is {sender['nonce']} but the transaction
nonce is {tx['nonce']}"
    if sender["balance"] < tx["value"]:
        return f"error: balance {sender['balance']} < amount sent {tx['value']}"
    sender["nonce"] += 1
    sender["balance"] -= tx["value"]
    S.setdefault(tx["to"], {"nonce": 0, "balance": 0})["balance"] += tx["value"]
    return S

S = {"Alice": {"nonce": 0, "balance": 10}}
t1 = {"from": "Alice", "to": "Bob", "value": 3, "nonce": 0}
S = apply_account(S, t1)
print("after t1:", S)
print("applying the same t1 again (replay attack):", apply_account(S, t1))
t2 = {"from": "Alice", "to": "Carol", "value": 3, "nonce": 1}
S = apply_account(S, t2)
print("after t2:", S, "— there is no change output. The balance simply goes down")
t3 = {"from": "Alice", "to": "0x000…dead", "value": 4, "nonce": 2}      # to an address that
does not exist (has no key) — a mistaken transfer
S = apply_account(S, t3)
print("after t3 (mistaken transfer):", S, "— the receiving account is created automatically and
the balance is recorded. Only the key to that address can make a transaction that sends it
back")
```

```
# Output — bcbook (Python 3.13), exactly as run
after t1: {'Alice': {'nonce': 1, 'balance': 7}, 'Bob': {'nonce': 0, 'balance': 3}}
applying the same t1 again (replay attack): error: the next nonce of Alice is 1 but the
transaction nonce is 0
after t2: {'Alice': {'nonce': 2, 'balance': 4}, 'Bob': {'nonce': 0, 'balance': 3}, 'Carol':
{'nonce': 0, 'balance': 3}} — there is no change output. The balance simply goes down
after t3 (mistaken transfer): {'Alice': {'nonce': 3, 'balance': 0}, 'Bob': {'nonce': 0,
'balance': 3}, 'Carol': {'nonce': 0, 'balance': 3}, '0x000…dead': {'nonce': 0, 'balance': 4}} —
the receiving account is created automatically and the balance is recorded. Only the key to that
address can make a transaction that sends it back
```

5. Comparing the two models, and the two shapes of a "mistaken transfer"

Here is a comparison table.

	UTXO MODEL (BITCOIN)	ACCOUNT MODEL (ETHEREUM)
State	The set of unspent outputs	Address → (nonce, balance, storage, code)
"Balance"	Not in the ledger. The wallet computes the sum of the outputs my keys open	A number written in the ledger
Transaction	Inputs (reference to an earlier output + signature) and outputs (amount + locking condition)	Receiving account, amount, nonce, data, gas-related fields, and a signature (the sending account is recovered from the signature); simplified
Preventing double spending	An output is spent only once	A nonce is used only once
Change	A separate output to one's own address is created	None. The balance just goes down
Parallelism	Transactions that spend different outputs can be verified in any order	Transactions from the same account have to be processed in nonce order
Privacy	Addresses can be switched, but input merging and change link them	History piles up at one address
Programmability	Limited to the locking conditions (scripts) of outputs	The code and storage of contract accounts (Chapter 13)

Now let us draw "I sent coins by mistake" in the two systems. **In the UTXO model**, a mistaken transfer is "having created an output with the wrong locking condition." That output stays in the UTXO set as it is, and only someone with a key that satisfies the condition can spend it. If it is a condition for which no one has a key (an address produced by a typo, for example), the output remains in the set forever. No one has "taken" it. **In the account model** it is "the balance of the receiving account having gone up." As with t3 in NB11-2 above, even if there was no account at the receiving address, one comes into being at that moment. Only the key to that address can make a transaction that sends it back. In both cases the ledger does not ask "who is the rightful holder of the right?" If the signature is valid, the transaction is valid. The difference is **what is left, and where**. On one side "an output that has not been opened" is left; on the other, "an increased balance."

> **? A LAWYER'S QUESTION**
>
> To whom do coins sent by mistake belong? In the UTXO model, all that has happened is that "an output no one's key opens" has come into being; in the account model, "the balance figure of some address" has gone up. Which of the two does the phrase "transfer of possession" fit? If the recipient has moved them to an address of their own, what is that act? If the recipient has lost the key and cannot move them, what becomes of the duty to return them? As for the framework the Korean courts have used on this question, Part 5 only points to the cases. → Chapter 28

6. Summary

Bitcoin keeps the set of unspent outputs as its state instead of balances, and prevents double spending with "an output is spent only once." Ethereum keeps a balance and a nonce for each account, and prevents it with "a nonce is used only once." The former has the advantage in parallelism and in switching addresses; the latter in programs that read and write state. Whether or not a "balance" is in the ledger: this one difference changes the shape of mistaken transfers, of attachment, and of privacy, all of them. Chapter 12, the last chapter of Part 1, checks the concepts so far, byte by byte, in one real Bitcoin block.

Sources for this chapter

- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008 — inputs, outputs, and change (§9, p. 5), the linking risk of multiple inputs (§10, p. 6), fees (§6, p. 4).
- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2014 (first draft 2013). https://ethereum.org/whitepaper/ (official PDF, accessed 2026-10-25) — Bitcoin's state transition and the definition of UTXO, the four fields and two kinds of Ethereum accounts, limitations of UTXO scripts.
- Gavin Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper, Shanghai version efc5f9a, 2025-02-04. https://ethereum.github.io/yellowpaper/paper.pdf — the world state and the four fields of the account state (§4.1, p. 4).
- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library modules `hashlib` and `json`. Notebook NB11.

Notes

1. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," official PDF on ethereum.org (2014; first draft 2013; the copy obtained is a revision from April 2014 or later), "Scripting" section. As limitations of UTXO-based scripts it lists Lack of Turing-completeness, Value-blindness, Lack of state, and Blockchain-blindness. Of these, this book carries over only the gist of "Lack of state - UTXO can either be spent or unspent; there is no opportunity for multi-stage contracts ..." and "Value-blindness - there is no way for a UTXO script to provide fine-grained control over the amount that can be withdrawn." The Ethereum white paper does not itself state the causal link "and therefore the account model."

https://ethereum.org/whitepaper/ (accessed 2026-10-25).

2. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), p. 5 (§9, Combining and Splitting Value).

3. Buterin (2014), "Bitcoin As A State Transition System" section (pp. 5–6 of the PDF). The state transition: "1. For each input in TX: i. If the referenced UTXO is not in S, return an error. ii. If the provided signature does not match the owner of the UTXO, return an error. 2. If the sum of the denominations of all input UTXO is less than the sum of the denominations of all output UTXO, return an error. 3. Return S with all input UTXO removed and all output UTXO added."

4. Nakamoto (2008), p. 6 (§10, Privacy).

5. Buterin (2014), "Ethereum Accounts" section. Emphasis added.

6. Gavin Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Shanghai version efc5f9a (2025-02-04), §4.1, World State (p. 4): "The world state (state), is a mapping between addresses (160-bit identifiers) and account states ...". "nonce: A scalar value equal to the number of transactions sent from this address or, in the case of accounts with associated code, the number of contract-creations made by this account." "balance: A scalar value equal to the number of Wei owned by this address." "storageRoot: A 256-bit hash of the root node of a Merkle Patricia tree that encodes the storage contents of the account ...". "codeHash: The hash of the EVM code of this account ...". https://ethereum.github.io/yellowpaper/paper.pdf (accessed 2026-10-25). This version is based on the Shanghai fork and may not reflect changes made in later forks.

7. Buterin (2014), same section: "In general, there are two types of accounts: externally owned accounts, controlled by private keys, and contract accounts, controlled by their contract code."

PART 1 · TECHNICAL FOUNDATIONS

Chapter 12. Anatomy of a real block: unpacking the 80 bytes of the genesis block by hand

KEY TAKEAWAY

Everything learned in Part 1 is checked here against one piece of real data. Bitcoin's first block (the genesis block, January 3, 2009) is 285 bytes long, and its first 80 bytes are the header. In this chapter we cut those 80 bytes into six fields, apply SHA-256 twice to recompute the block hash and match it against the published value `000000000019d668…` , and read the sentence embedded in the only transaction (the coinbase) directly from the bytes. A block is nothing mysterious. It is an 80-byte head and a list of transactions, and anyone can recompute it at home and check that it matches. Only the standard library is used.

Learning objectives

1. Read the 80 bytes of a block header as six fields (version, previous hash, Merkle root, time, bits, nonce), and handle byte order (endianness).
2. Compute the double SHA-256 of the header, match it to the published block hash, and confirm that the hash is below the target encoded in nBits.
3. **[Intermediate]** Unpack the coinbase transaction byte by byte, and confirm that its txid equals the Merkle root and that its output is a P2PK of 50 BTC.
4. Understand what the phrase "anyone can recompute it" means for the verifiability of a blockchain.

What a reader of the [Basic] paragraphs alone takes away. The "block" of a blockchain is an 80-byte head + a list of transactions, and anyone can recompute it at home and check that it matches.

1. Where the data came from

The bytes in this chapter are read from snapshot files bundled with this book: `genesis_block.hex` (the whole 285-byte block) and `genesis_header.hex` (the first 80 bytes). On October 24, 2026 (UTC), the raw bytes were obtained from the public APIs of two different block explorers and checked to be **byte-for-byte identical**, and those values were compared with the genesis constants embedded in the source code of two implementations, Bitcoin Core and btcd.[1] The code in this book does not call the network. It reads only these files (for environments without the files, the same bytes are also written into the code).

The genesis block is written into the Bitcoin Core source **as a constant**. A node does not receive this block from the network; it builds it internally. That code builds the block from time 1231006505, nonce 2083236893, bits 0x1d00ffff, version 1, and a reward of 50 coins, and asserts that its hash is `000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f` and its Merkle root is `4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b`.[2] Below, we perform this assertion again from the outside.

2. Six slots for 80 bytes

[Intermediate] First we have to talk about byte order. Bitcoin writes numbers **little-endian**: the low-order byte comes first. The decimal number 1231006505 is `0x495FAB29` in hexadecimal, but it goes into the header in the order `29 AB 5F 49`. Hash values work the same way. Internally the bytes are stored reversed, and the human-readable `000000000019d668…` is that value displayed reversed. "The hash starts with 0s" is true of the display format; in memory the 0s are **at the end**. Leave out this reversal and none of the calculations below will come out right.

```
# NB12-1: cut the 80-byte genesis block header into six fields (bundled snapshot file; if it is
missing, a constant with the same bytes)
import struct, hashlib, datetime, os

HEADER_HEX =
("01000000000000000000000000000000000000000000000000000000000000000000000003ba3edfd7a7b12b27ac72c
3e67768f617fc81bc3888a51323a9fb8aa4b1e5e4a29ab5f49ffff001d1dac2b7c")   # the same bytes as
data/genesis_header.hex

def load_hex(path, fallback):
    try:
        with open(path) as f: return f.read().strip()
    except OSError:
        return fallback

header = bytes.fromhex(load_hex("data/genesis_header.hex", HEADER_HEX))
print("header length:", len(header), "bytes")
assert header.hex() == HEADER_HEX, "the bundled file and the constant differ"

fields = [("version (nVersion)", 0, 4), ("previous block hash (hashPrevBlock)", 4, 32), ("Merkle
root (hashMerkleRoot)", 36, 32),
          ("time (nTime)", 68, 4), ("target (nBits)", 72, 4), ("nonce (nNonce)", 76, 4)]
for name, off, size in fields:
    raw = header[off:off + size]
    print(f"[{off:>2}:{off+size:>2}] {name:<36} {raw.hex()}")

nVersion = struct.unpack("<I", header[0:4])[0]          # "<" = little-endian
hashPrev = header[4:36][::-1].hex()                     # hashes are displayed reversed
merkle   = header[36:68][::-1].hex()
nTime    = struct.unpack("<I", header[68:72])[0]
nBits    = struct.unpack("<I", header[72:76])[0]
nNonce   = struct.unpack("<I", header[76:80])[0]
print()
print("nVersion =", nVersion)
print("hashPrevBlock =", hashPrev, "(0 because this is the genesis block: there is no parent to
point to)")
print("hashMerkleRoot =", merkle)
print("nTime =", nTime, "=", datetime.datetime.fromtimestamp(nTime,
datetime.timezone.utc).strftime("%Y-%m-%d %H:%M:%S UTC"))
print(f"nBits = 0x{nBits:08x}  ({nBits})")
print("nNonce =", nNonce, f"(0x{nNonce:08x})")
print("matches CreateGenesisBlock(1231006505, 2083236893, 0x1d00ffff, 1, 50*COIN) in the Core
source:",
      (nTime, nNonce, nBits, nVersion) == (1231006505, 2083236893, 0x1d00ffff, 1))
```

```
# Output — bcbook (Python 3.13), exactly as run
header length: 80 bytes
[ 0: 4] version (nVersion)                  01000000
[ 4:36] previous block hash (hashPrevBlock)
0000000000000000000000000000000000000000000000000000000000000000
[36:68] Merkle root (hashMerkleRoot)
3ba3edfd7a7b12b27ac72c3e67768f617fc81bc3888a51323a9fb8aa4b1e5e4a
[68:72] time (nTime)                        29ab5f49
[72:76] target (nBits)                      ffff001d
[76:80] nonce (nNonce)                      1dac2b7c

nVersion = 1
hashPrevBlock = 0000000000000000000000000000000000000000000000000000000000000000 (0 because this
is the genesis block: there is no parent to point to)
hashMerkleRoot = 4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b
nTime = 1231006505 = 2009-01-03 18:15:05 UTC
nBits = 0x1d00ffff  (486604799)
nNonce = 2083236893 (0x7c2bac1d)
matches CreateGenesisBlock(1231006505, 2083236893, 0x1d00ffff, 1, 50*COIN) in the Core source:
True
```

[Intermediate] The six fields agree with the four constants in the Core source. The time

1231006505 is 18:15:05 (UTC) on January 3, 2009. A previous block hash of 0 is a feature of the genesis block alone. The chain of Chapter 7 starts from this 0.

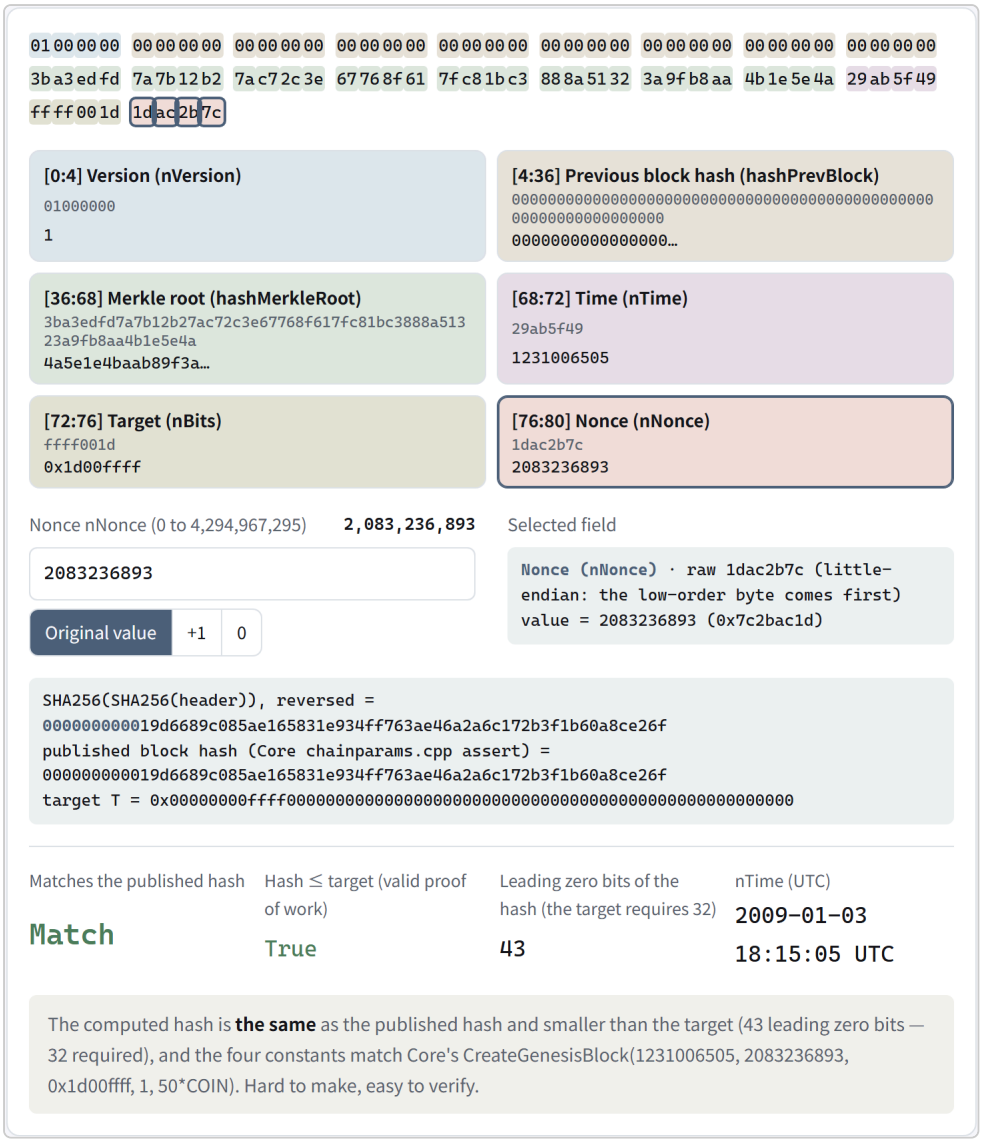


Figure 12-1. Block header anatomy. The 80 bytes of the genesis block header are colored as six fields, and the double SHA-256 is recomputed and compared with the published block hash 000000000019d668…. Change the nonce by 1 and the condition fails. — In the HTML edition you can change the values here (HTML edition, Chapter 12, Section 2).

3. Recomputing the hash and matching it to the published value

[Intermediate] As Chapter 4 and Chapter 7 said, the block hash is the double SHA-256 of the 80-byte header. We compute it, reverse it, and compare it with the published value. Then, following the rule in Chapter 8, we check whether that hash is below the target that nBits points to.

```
# NB12-2: double SHA-256 → block hash. Does it match the published value, is it below the
target, and what happens if the nonce changes?
def sha256d(b: bytes) -> bytes:
    return hashlib.sha256(hashlib.sha256(b).digest()).digest()

KNOWN = "000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f"   # the assert value
in Core's chainparams.cpp
block_hash = sha256d(header)[::-1].hex()
print("computed block hash: ", block_hash)
print("published block hash:", KNOWN)
print("match:", block_hash == KNOWN)

def nbits_to_target(nbits: int) -> int:
    return (nbits & 0x007FFFFF) * 256 ** ((nbits >> 24) - 3)
target = nbits_to_target(nBits)
h_int = int(block_hash, 16)
print("target T = 0x%064x" % target)
print("hash ≤ target (valid proof of work):", h_int <= target)
print("number of leading 0 bits in the hash:", 256 - h_int.bit_length(), "(the minimum the
target requires is 32)")

# change the nonce by just 1
tampered = header[:76] + struct.pack("<I", nNonce + 1)
t_hash = sha256d(tampered)[::-1].hex()
print("hash of the header with the nonce changed to 2083236894:", t_hash, "→ below the target?",
int(t_hash, 16) <= target)
```

```
# Output — bcbook (Python 3.13), exactly as run
computed block hash:  000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
published block hash: 000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
match: True
target T = 0x00000000ffff0000000000000000000000000000000000000000000000000000
hash ≤ target (valid proof of work): True
number of leading 0 bits in the hash: 43 (the minimum the target requires is 32)
hash of the header with the nonce changed to 2083236894:
9b227a4a5daa0cbae6874144bc5d7797d0513e320aceadeb3b06304971a41b1c → below the target? False
```

It comes down to three lines. The computed hash is the same as the published hash. That hash is smaller than the target (it has more than 32 leading 0 bits). Changing the nonce by just 1 breaks the condition. On January 3, 2009, someone computed billions of hashes on average to find this nonce, 2083236893 (in Chapter 8, "about 2^{32} per block at difficulty 1"), and we have just verified that fact with two hashes. Hard to make, easy to verify. That is the asymmetry of proof of work.

4. [Intermediate] Unpacking the transaction: the coinbase and the sentence inside it

[Intermediate] After the header come the number of transactions and the transactions themselves. The genesis block has only one transaction, and it is a **coinbase** transaction, that is, "a special transaction that starts a new coin owned by the creator of the block" (§6 of the white paper, in Chapter 8). The input of a coinbase transaction does not point to a previous output (hash 0, index 0xffffffff), and in the script in that slot the miner can put arbitrary data. In that slot of the genesis block sits a famous sentence. The code below **reads that sentence directly from the bytes.** This book does not write it from memory.

```
# NB12-3: the whole 285-byte block — transaction count, the inputs and outputs of the coinbase
transaction, the sentence inside it, txid == Merkle root
BLOCK_HEX = HEADER_HEX +
("0101000000010000000000000000000000000000000000000000000000000000000000000000ffffffff4d04ffff00
1d010445546865732054696d65732030332f4a616e2f32303039204368616e63656c6c6f72206f6e206272696e6b206f66
207365636f6e64206261696c6f757420666f722062616e6b73ffffffff0100f2052a01000000434104678afdb0fe5548
271967f1a67130b7105cd6a828e03909a67962e0ea1f61deb649f6bc3f4cef38c4f35504e51ec112de5c384df7ba0b8d
578a4c702b6bf11d5fac00000000")
block = bytes.fromhex(load_hex("data/genesis_block.hex", BLOCK_HEX))
assert block.hex() == BLOCK_HEX and block[:80] == header
print("block length:", len(block), "bytes | header 80 + rest", len(block) - 80)

pos = 80
tx_count = block[pos]; pos += 1                    # transaction count (a variable-length
integer; 1 byte here)
print("transaction count:", tx_count)
tx_start = pos
version = struct.unpack("<I", block[pos:pos+4])[0]; pos += 4
in_count = block[pos]; pos += 1
prev_txid = block[pos:pos+32][::-1].hex(); pos += 32
prev_index = struct.unpack("<I", block[pos:pos+4])[0]; pos += 4
script_len = block[pos]; pos += 1
script_sig = block[pos:pos+script_len]; pos += script_len
sequence = struct.unpack("<I", block[pos:pos+4])[0]; pos += 4
print(f"transaction version {version}, input count {in_count}, previous output =
({prev_txid[:8]}…, 0x{prev_index:08x}) ← points to no output = coinbase")
print("input script (scriptSig) length:", script_len, "bytes")

# read the scriptSig in "pushes": if the leading byte is n (1 to 75), put the next n bytes on
the stack
i, pushes = 0, []
while i < len(script_sig):
    n = script_sig[i]; pushes.append(script_sig[i+1:i+1+n]); i += 1 + n
for k, p in enumerate(pushes):
    print(f"  push {k}: length {len(p)}  {p.hex() if len(p) <= 4 else p.hex()[:16] + '…'}")
print("  push 0 as a little-endian integer:", int.from_bytes(pushes[0], "little"), "(= the same
number as nBits 486604799)")
print("  push 2 as ASCII:", repr(pushes[2].decode("ascii")))

out_count = block[pos]; pos += 1
value = struct.unpack("<Q", block[pos:pos+8])[0]; pos += 8
spk_len = block[pos]; pos += 1
spk = block[pos:pos+spk_len]; pos += spk_len
locktime = struct.unpack("<I", block[pos:pos+4])[0]; pos += 4
print(f"output count {out_count}, amount {value:,} satoshis = {value/1e8} BTC, locking script
{spk_len} bytes, locktime {locktime}")
print("  locking script = [push 65 bytes] public key (first byte 0x%02x) … [0x%02x =
OP_CHECKSIG]" % (spk[1], spk[-1]))
print("  public key, 65 bytes =", spk[1:66].hex())

tx_bytes = block[tx_start:pos]
print("transaction length:", len(tx_bytes), "bytes | read to the end of the block:", pos ==
len(block))
txid = sha256d(tx_bytes)[::-1].hex()
print("txid = double SHA-256(transaction) =", txid)
print("Merkle root in the header          =", merkle)
print("with only one transaction, Merkle root == txid :", txid == merkle)
```

```
# Output — bcbook (Python 3.13), exactly as run
block length: 285 bytes | header 80 + rest 205
transaction count: 1
transaction version 1, input count 1, previous output = (00000000…, 0xffffffff) ← points to no
output = coinbase
input script (scriptSig) length: 77 bytes
  push 0: length 4  ffff001d
  push 1: length 1  04
  push 2: length 69  5468652054696d65…
  push 0 as a little-endian integer: 486604799 (= the same number as nBits 486604799)
  push 2 as ASCII: 'The Times 03/Jan/2009 Chancellor on brink of second bailout for banks'
output count 1, amount 5,000,000,000 satoshis = 50.0 BTC, locking script 67 bytes, locktime 0
  locking script = [push 65 bytes] public key (first byte 0x04) … [0xac = OP_CHECKSIG]
  public key, 65 bytes =
04678afdb0fe5548271967f1a67130b7105cd6a828e03909a67962e0ea1f61deb649f6bc3f4cef38c4f35504e51ec112
de5c384df7ba0b8d578a4c702b6bf11d5f
transaction length: 204 bytes | read to the end of the block: True
txid = double SHA-256(transaction) =
4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b
Merkle root in the header          =
4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b
with only one transaction, Merkle root == txid : True
```

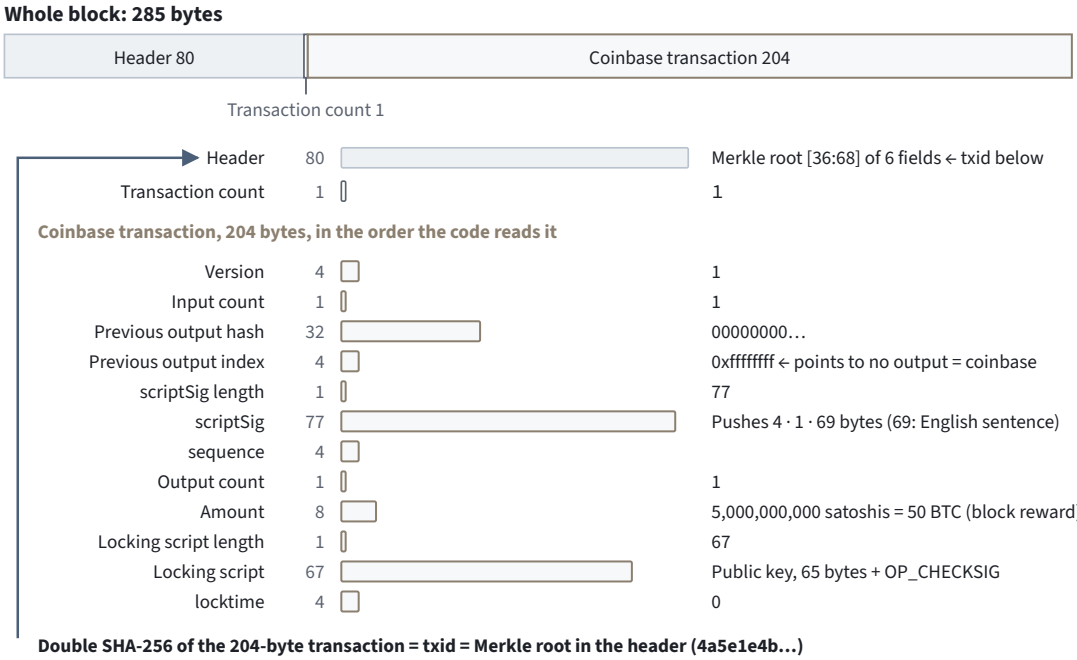


Figure 12-2. A byte map that splits the 285-byte genesis block into the header (80), the transaction count (1), and the coinbase transaction (204), and lays the transaction out field by field in the order NB12-3 reads it. Bar lengths are proportional to byte counts, and the double SHA-256 of the whole transaction equals the Merkle root in the header. 50 BTC is the amount written as the genesis block's reward.

Put together what the bytes said. There is one transaction. Its input points to nothing (coinbase), and the input script holds three pieces: the number 486604799, which is the same as nBits; the number 4; and a 69-byte English sentence. There is one output, the amount is 5,000,000,000 satoshis, that is, 50 BTC, and the locking condition is "sign with this 65-byte public key" (P2PK, Chapter 5). The double SHA-256 of the whole transaction equals the Merkle root in the header ("with one transaction, root = txid" in Chapter 6). And the sentence, exactly as read from the data, is `The Times 03/Jan/2009 Chancellor on brink of second bailout for banks`. It is also identical, character for character, to the constant in the

Bitcoin Core source.[2]

[Intermediate] This book does not speculate about why the sentence is there. Two things can be confirmed technically. First, it is a string in the form of a newspaper headline, and if that headline really ran in the newspaper of that date (this book has not checked it against the newspaper itself), it records an unpredictable event, so it is evidence that **the block cannot have been made before that date** (the method of Haber and Stornetta in Chapter 4: put in the bits of an unpredictable event to show "after"). Second, Bitcoin Core treats the coinbase output of the genesis block as **unspendable**. In the words of the source comment, "Special case for the genesis block, skipping connection of its transactions (its coinbase is unspendable)."[3] The 50 BTC is written in the ledger, but no one can spend it.

5. Compared with a block from 2026

The genesis block's nBits, `0x1d00ffff`, is difficulty 1. On this book's reference date (2026-10-25; observed 2026-10-24 UTC), the block at the tip of the chain (height 972,958) had nBits `0x17020093`, a difficulty of about 1.4×10^{14} (as of 2026-10; Chapter 8 and Appendix A).[4] The shape of the header has not changed in 17 years. The same 80 bytes, the same six slots, the same double SHA-256. What has changed is the target that nBits points to, and the amount of computation the network pours in to meet it. That block's header also unpacks in exactly the same way with this chapter's code. The bundled data contains the hashes and nBits of two blocks (971,712 and 972,958), so you can try the nBits conversion right away. The raw 80-byte headers are not bundled, so you have to fetch them yourself from a public node or explorer.

6. Summary: closing Part 1

The nine chapters of Part 1 all fit inside the 285 bytes of the genesis block. The hash (Chapter 4) gives the block its name. Signatures and public keys (Chapter 5) are the locking conditions on outputs. The Merkle root (Chapter 6) ties the transactions to the header. The previous block hash (Chapter 7) makes the chain. nBits and the nonce (Chapter 8) are the proof of work. The number of blocks stacked on top (Chapter 9) is the degree of "finality," and the set of outputs (Chapter 11) is the state. Proof of stake (Chapter 10) replaces "finding a nonce" in this frame with "a deposit and a vote." The fact that all of this **can be recomputed at home** is what is actually meant when people say a blockchain runs "without trust." You need not believe anyone's word. You need only trust the bytes and the hash function. Part 2 moves to a world in which **code runs** on top of this ledger.

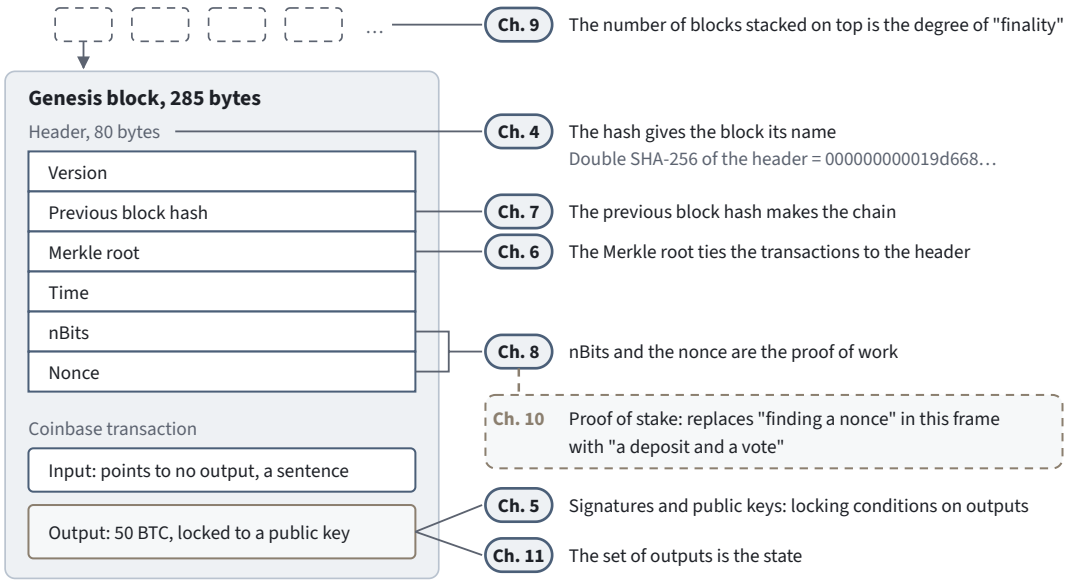


Figure 12-3. A map pinning each chapter of Part 1 to its place in the 285 bytes of the genesis block. Chapter 10's proof of stake is drawn as a side branch that replaces "finding a nonce" in this frame with "a deposit and a vote."

Sources for this chapter

- This book's bundled data `data/genesis_block.hex`, `data/genesis_header.hex`, `data/genesis_block.meta.json`, `data/hashrate_2026-10-25.json` (reference date 2026-10-25). Sources, access times, and comparison results are in each file and in the README of the data folder.
- Bitcoin Core v31.1 (tag commit 9be056a8a72b624dae9623b2f7bded92c2a21c91): `src/kernel/chainparams.cpp` (genesis constants, the sentence, the output script, the asserts), `src/validation.cpp` (special handling of the genesis coinbase). Accessed 2026-10-25.
- btcd (commit dd169fe0c32a56087d28b92fff3b8cf4fb43477a), `chaincfg/genesis.go`: the genesis constants of an independent implementation.
- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008: the coinbase

(§6, p. 4), the 80-byte header (§7, p. 4).

- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library modules `struct`, `hashlib`, and `datetime`. Notebook NB12.

Notes

1. This book's `data/genesis_block.meta.json` (reference date 2026-10-25). Source 1: Blockstream Esplora `/api/block/000000000019d668…/raw` (2026-10-24T18:15:22Z); source 2: mempool.space, same path (18:15:24Z). The two 285-byte responses are byte-for-byte identical, and the SHA-256 of that raw 285-byte block is `5299fac924b5a2fc19a88876a0042c19ac4d11fe69c3f66e47516e26185f9e99`. Source comparison: Bitcoin Core v31.1 `src/kernel/chainparams.cpp` (commit 9be056a8) L134–L137, `CreateGenesisBlock(1231006505, 2083236893, 0x1d00ffff, 1, 50 * COIN)` and the asserts on the hash and the Merkle root; btcd (commit dd169fe0) `chaincfg/genesis.go`, Version 1, Timestamp 0x495fab29, Bits 0x1d00ffff, Nonce 0x7c2bac1d.

2. Bitcoin Core v31.1, `src/kernel/chainparams.cpp`, mainnet: `genesis = CreateGenesisBlock(1231006505, 2083236893, 0x1d00ffff, 1, 50 * COIN); consensus.hashGenesisBlock = genesis.GetHash(); assert(consensus.hashGenesisBlock == uint256{"000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f"}); assert(genesis.hashMerkleRoot == uint256{"4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b"});`. `CreateGenesisBlock` in the same file uses `const char* pszTimestamp = "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks";` and the output script `"04678afdb0fe…b6bf11d5f"_hex << OP_CHECKSIG`. Accessed 2026-10-25.

3. Bitcoin Core v31.1, `src/validation.cpp`: `// Special case for the genesis block, skipping connection of its transactions` `// (its coinbase is unspendable)` `if (block_hash == params.GetConsensus().hashGenesisBlock) {`. Accessed 2026-10-25.

4. This book's `data/hashrate_2026-10-25.json`. Block 972,958: hash `00000000000000000000001857369fd717b186adc5c22960b54b87875029cc65ee`, nBits 0x17020093; the header bytes from the two explorers match, the recomputed double SHA-256 matches, and hash ≤ target was confirmed.

PART 2 · SMART CONTRACTS AND APPLICATIONS

Chapter 13. The Ethereum Virtual Machine and gas: a ledger that runs code

KEY TAKEAWAY

The Bitcoin ledger records "who has how much." The Ethereum ledger also records, on top of that, **programs and their memory (storage)**, and when a transaction comes in, every node runs the same program in the same order and arrives at the same result. This executor is called the Ethereum Virtual Machine (EVM). A machine that can run any program can also run a program that never stops, so Ethereum makes you purchase **gas**, which puts a price on every instruction, in advance, and when the gas runs out it stops execution and reverts the changes. After this chapter you will see that what the word "smart contract" points to is not a contract in the legal sense but **a stored program that executes itself when conditions are met**, and that gas is the safety device that makes the program stop. The question "is it a contract?" is opened by this chapter and taken up by Part 5.

Learning objectives

1. Explain what "code runs on the ledger" means with four words: externally owned account, contract account, transaction, and message.
2. Say why gas is needed (the halting problem, charging for resources), and know what is reverted and what remains when a transaction uses up all its gas.
3. **[Intermediate]** Read the gas-related fields of a transaction (gas limit, maximum fee, priority fee) and the shares of the fee (burned, validator). The formula for the amounts is left to Chapter 21.
4. **[Intermediate]** Read the smallest contract written in Solidity, and imitate state transitions and running out of gas in Python.

What a reader of the [Basic] paragraphs alone takes away. A smart contract is not a "contract" but **a conditional, self-executing program**, and gas is the safety device that makes the program stop.

1. What failure this technology grew out of

Bitcoin has programs too. Every output carries a short script that says "to spend these coins, satisfy this condition" (Chapter 11), and the condition is usually "produce a signature for this public key." Vitalik Buterin's 2014 Ethereum white paper lists four things this script cannot do. It has no loops, so some computations cannot be written at all. It cannot finely control how much is taken out of an output. An output is only "spent" or "unspent," so contracts that proceed in several stages cannot be built. And a script cannot read other information on the chain, such as block headers.[1] Chapter 11 took up the middle two of these to explain the account model. This chapter takes up the first and the fourth.

So what the Ethereum white paper proposed was not a particular application but **a base layer**: "a blockchain with a built-in Turing-complete programming language, allowing anyone to write smart contracts and decentralized applications where they can create their own arbitrary rules for ownership, transaction formats and state transition functions." And it defines smart contracts as "cryptographic 'boxes' that contain value and only unlock it if certain conditions are met."[2] Notice that the words "party" and "agreement" do not appear in this definition. There is only a box, a condition, and value. This chapter's question box comes from here.

Gavin Wood's Yellow Paper states the motivation for this system more bluntly: "one key goal is to facilitate transactions between consenting individuals who would otherwise have no means to trust one another. This may be due to geographical separation, interfacing difficulty, or perhaps the incompatibility, incompetence, unwillingness, expense, uncertainty, inconvenience, or corruption of existing legal systems." And it says it reaches that aim "By specifying a state-change system through a rich and unambiguous language, and furthermore architecting a system such that we can reasonably expect that an agreement will be thus enforced autonomously ...."[3] A lawyer reading this stops short. It is a declaration that the system will stand in for the failings of the legal system. This book does not assess that declaration. In this chapter it shows only **what the declaration means technically**.

2. Intuition: accounts, code, transactions, messages

In Chapter 11 we saw that an Ethereum account has four fields (nonce, balance, code, storage) and that there are two kinds of account, externally owned accounts and contract accounts. An externally owned account is moved by a private key and has no code. A contract account has code and no private key. The only way to move a contract account is **to send it a message**, and when a message arrives, that account's code runs.

The white paper distinguishes "messages" from "transactions": "'Messages' in Ethereum are somewhat similar to 'transactions' in Bitcoin, but with three important differences. First, an Ethereum message can be created either by an external entity or a contract, whereas a Bitcoin transaction can only be created externally. Second, there is an explicit option for Ethereum messages to contain data. Finally, the recipient of an Ethereum message, if it is a

contract account, has the option to return a response; this means that Ethereum messages also encompass the concept of functions." And: "The term 'transaction' is used in Ethereum to refer to the **signed data package** that stores a message to be sent from an externally owned account."[4] Put simply, a transaction is something a person starts, while "message" also covers the inner calls that arise when that transaction touches a contract and the contract in turn calls another contract. That contracts can call contracts is the property the Ethereum white paper calls the "first class citizen" property: "the idea that contracts have equivalent powers to external accounts, including the ability to send message [sic] and create other contracts."[5]

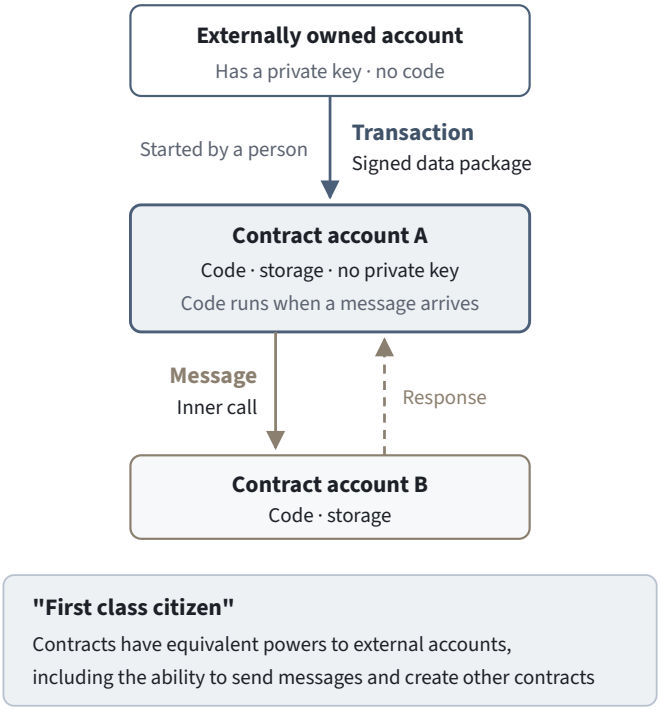


Figure 13-1. The structure in which a transaction signed by an externally owned account runs the code of contract account A, and A sends a message to contract B and receives a response. A transaction is started by a person; "message" also covers the inner calls that contracts make ("first class citizen").

What goes inside a transaction was passed over in Chapter 11's comparison table with only the words "gas-related fields." Here we unpack it. The Ethereum white paper's transaction contains "the recipient of the message, a signature identifying the sender, the amount of ether and the data to send, as well as two values called STARTGAS and GASPRICE."[6] The Yellow Paper (Shanghai version) sets these out as formal fields. Common to all transactions are the type, the nonce, the **gas limit (gasLimit)**, the recipient (to), the value, and the signature values r and s. Type 2 transactions, introduced in the London fork, have the **maximum fee per gas (maxFeePerGas)** and the **maximum priority fee (maxPriorityFeePerGas)**; the earlier type 0 and type 1 transactions have a single **gas price (gasPrice)**. A transaction that creates a new contract carries initialization code (init), and an ordinary transaction carries data (data).[7] The Yellow Paper's description of the gas limit is the core of this chapter: "A scalar value equal to the maximum amount of gas that should be used in executing this transaction. This is **paid up-front**, before any computation is done and may not be increased later ...."

The path of a single transaction

The white paper sets out the state transition function `APPLY(S, TX) → S'` in six steps. Read them side by side with the UTXO version in Chapter 11.[8]

1. "Check if the transaction is well-formed (ie. has the right number of values), the signature is valid, and the nonce matches the nonce in the sender's account. If not, return an error."
2. "Calculate the transaction fee as STARTGAS * GASPRICE, and determine the sending address from the signature. Subtract the fee from the sender's account balance and increment the sender's nonce. If there is not enough balance to spend, return an error."
3. "Initialize GAS = STARTGAS, and take off a certain quantity of gas per byte to pay for the bytes in the transaction."
4. "Transfer the transaction value from the sender's account to the receiving account. If the receiving account does not yet exist, create it. If the receiving account is a contract, **run the contract's code either to completion or until the execution runs out of gas**."
5. "If the value transfer failed because the sender did not have enough money, or the code execution ran out of gas, **revert all state changes except the payment of the fees**, and add the fees to the miner's account."
6. "Otherwise, refund the fees for all remaining gas to the sender, and send the fees paid for gas consumed to the miner."

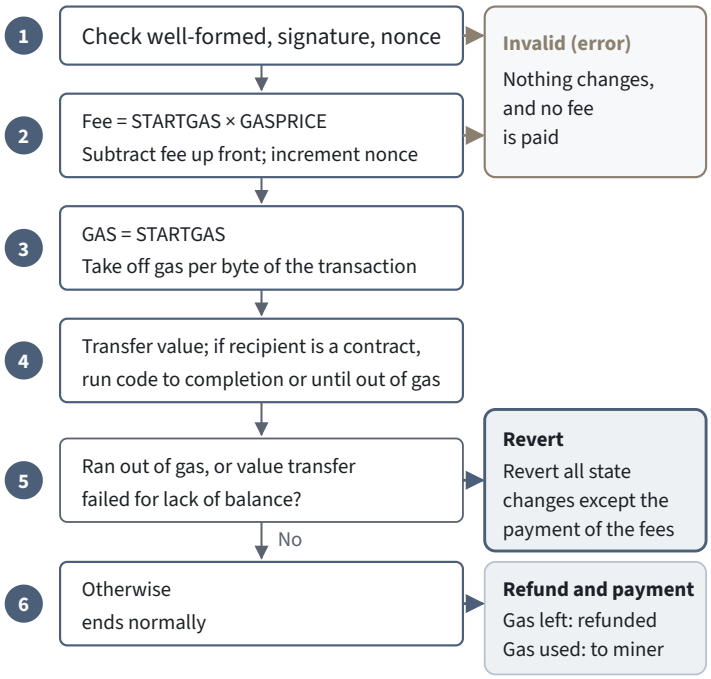


Figure 13-2. The six steps of the Ethereum white paper's state transition function APPLY(S, TX) → S′ and its three outcomes. A transaction caught at step 1 or 2 is invalid and nothing changes; if it runs out of gas, everything is reverted except the payment of the fees; if it ends normally, the remaining gas is refunded. "Miner" is the term of the 2014 white paper.

Steps 5 and 6 are the two faces of gas. If the gas runs out, **execution is reverted but the fee stays paid.** If gas is left over, **what is left comes back.** In one sentence of the Ethereum white paper: "If transaction execution 'runs out of gas', all state changes revert - except for the payment of the fees, and if transaction execution halts with some gas remaining then the

remaining portion of the fees is refunded to the sender."[9] Why these two rules are needed is the subject of the next section.

3. Gas: how to stop a program that does not stop

The EVM allows loops and recursion. The Ethereum white paper does not hide the consequence: "the Ethereum virtual machine is Turing-complete; this means that EVM code can encode any computation that can be conceivably carried out, including infinite loops." That raises the question "can malicious users essentially shut miners and full nodes down by forcing them to enter into an infinite loop?", and its root lies in "a problem in computer science known as the **halting problem**: there is no way to tell, in the general case, whether or not a given program will ever halt."[10] Until a node takes a transaction and runs it, the node cannot know whether it will finish, and running it is itself a cost.

The solution is not to try to know what cannot be known in advance but **to make the transaction purchase an upper limit in advance**: "our solution works by requiring a transaction to set a maximum number of computational steps that it is allowed to take, and if execution takes longer computation is reverted but fees are still paid."[11] The Yellow Paper puts the same thing in a single word. The EVM is "a quasi-Turing-complete machine; the quasi qualification comes from the fact that the computation is intrinsically bounded through a parameter, gas, which limits the total amount of computation done."[12] An attacker who runs an infinite loop can burden the nodes only for as much gas as the attacker bought, and the attacker pays for that gas (since the London fork, the base-fee share of it is burned and only the priority-fee share goes to the validator; see "Where the fee goes" below). One example the Ethereum white paper gives shows the character of this device well: "An attacker sees a contract with code of some form like send(A,contract.storage[A]); contract.storage[A] = 0, and sends a transaction with just enough gas to run the first step but not the second (ie. making a withdrawal but not letting the balance go down). The contract author does not need to worry about protecting against such attacks, because if execution stops halfway through the changes get reverted."[13] Running out of gas does not leave a "failed but half-done" state. Either everything happens, or everything is as if it never happened. Except for the fee.

What gas measures

[Intermediate] Gas is not money but **a unit of computation.** Section 5 of the Yellow Paper: "In order to avoid issues of network abuse and to sidestep the inevitable questions stemming from Turing completeness, all programmable computation in Ethereum is subject to fees. The fee schedule is specified in units of gas (see Appendix G for the fees associated with various computation). Thus any given fragment of programmable computation (this includes creating contracts, making message calls, utilising and accessing account storage and executing operations on the virtual machine) has a **universally agreed cost** in terms of gas."[14] Each instruction has a set price. Taking a few lines from the fee schedule in Appendix G of the Yellow Paper gives the table below. A light instruction such as addition costs 3 gas, the base charge for one transaction is 21,000 gas, and newly writing a nonzero value to storage costs 20,000 gas. Writing to storage is nearly 7,000 times as costly as addition because, as the Yellow Paper puts it, the aim is "to incentivise minimisation of the use of storage (which corresponds directly to a larger state database on all nodes)", and for the same reason §9.2 of the Yellow Paper says that for an instruction that clears a storage slot "the execution fee for an operation that clears an entry in the storage is not only waived, a qualified refund is given".[15] Appendix G of the same version, however, charges G_{sreset} 2,900 even when a slot is cleared to 0 and adds R_{sclear} 4,800 to the refund counter, so the description in §9.2 and the table in Appendix G do not agree. This book's table and code follow Appendix G.

ITEM (SYMBOL IN APPENDIX G OF THE YELLOW PAPER)	GAS	MEANING
$G_{verylow}$	3	One light instruction such as ADD, SUB, PUSH, or DUP
G_{mid} / G_{high}	8 / 10	One JUMP / JUMPI (conditional branch)
$G_{transaction}$	21,000	Base charge paid for every transaction
$G_{txdatazero}$ / $G_{txdatanonzero}$	4 / 16	Per byte of transaction data that is 0 / that is not 0
$G_{coldsload}$ / $G_{warmaccess}$	2,100 / 100	A storage slot touched for the first time in this transaction / access to a slot already touched
G_{sset}	20,000	Changing a storage slot from 0 to a nonzero value
G_{sreset}	2,900	Changing a nonzero value to another value, or clearing it to 0
R_{sclear}	4,800	Amount added to the refund counter when a nonzero value is cleared to 0
G_{create}	32,000	The CREATE instruction, which makes one new contract

The values above are taken from Appendix G of the Shanghai version of the Yellow Paper (2025-02-04).[16] The fee schedule has been revised fork after fork (for example, EIP-3529 in the London fork cut the refund cap from 1/2 to 1/5[17]), so this book does not call these "current values." The code in this chapter uses the values in this table **as constants, unchanged**.

Where the fee goes: a question carried over from Chapter 10

Chapter 10 passed "the share of transaction fees" on to this chapter. The answer splits in two. Section 5 of the Yellow Paper: "Since the introduction of EIP-1559 by Buterin et al. [2019] in the London hard fork, every transaction included in a block must pay a **base fee**, which is specified as wei per unit of gas consumed and is constant for each transaction within a block. The ether that is paid to meet the base fee is **burned (taken out of circulation)**." And: "To incentivize validators to include transactions, there is an additional fee known as a **priority fee**, which is also specified as wei per unit of gas consumed. ... Ether used to satisfy the priority fee is delivered to the beneficiary address, the address of an account typically under the control of the validator."[18] The total the sender pays is "the sum of the base fee per gas and the priority fee per gas multiplied by the total gas consumed."

So the "share of fees" that the validators of Chapter 10 receive is the priority fee only, and the base fee **goes to no one and disappears.** Why it was designed this way (to remove validators' incentive to manipulate fees, among other reasons), how the base fee is adjusted from block to block, and the economics surrounding "burning" are dealt with in Chapter 21. This chapter records only the structure in which a fee is **units of gas × price per gas**, and the fact that the price splits into two pieces.

▾ **[Advanced] The Yellow Paper's formalism: eight validity conditions, up-front payment, refunds**

[Advanced] Section 6 of the Yellow Paper sets out in prose eight "intrinsic validity" checks that a transaction must pass before it goes into execution (the formal definition, equation (69), adds an upper limit on the length of initialization code and a condition on the remaining block gas limit). The transaction is correctly RLP-encoded; the signature is valid; the nonce equals the sender account's current nonce; the sender account has no contract code (EIP-3607); **the gas limit is no smaller than the intrinsic gas g_0**; the account balance is at least the up-front payment v_0; a type 2 transaction's maxFeePerGas (gasPrice for types 0 and 1) is at least the block's base fee H_f; and in a type 2 transaction, maxPriorityFeePerGas is no larger than maxFeePerGas.[19]

[Advanced] **Intrinsic gas g_0** is the gas needed before a single line of code runs. For each byte of transaction data, add $G_{txdatazero}$ if it is 0 and $G_{txdatanonzero}$ otherwise; for a contract-creation transaction, add $G_{txcreate}$ and an amount proportional to the length of the initialization code; for every transaction, add $G_{transaction}$; and for each access-list entry, add a set amount.[20] A numerical example: a simple transfer with empty data has $g_0 = 21{,}000$. If the data has 4 nonzero bytes and 28 zero bytes, it is $21{,}000 + 4 \times 16 + 28 \times 4 = 21{,}176$.

[Advanced] For a type 2 transaction, the **effective gas price** p and the **priority fee** f are $f = \min(T_f, T_m - H_f)$ and $p = f + H_f$. Reading the symbols: T_f is the maximum priority fee written in the transaction, T_m the maximum fee per gas, and H_f the base fee in the block header. A numerical example: with a base fee of 10, a maximum priority fee of 2, and a maximum total of 15 (in wei per gas, toy values), $f = \min(2, 15 - 10) = 2$ and $p = 12$. If the base fee rises to 14, then $f = \min(2, 1) = 1$ and $p = 15$, and the priority fee is cut. The **up-front payment** is $v_0 = T_g T_m + T_v$ (gas

limit × maximum fee per gas + value transferred), but what is actually taken from the balance first is $T_g p$.[21] When execution ends, g^*, which is the remaining gas g' plus the refund counter A_r capped at "1/5 of the gas used," is returned at the original price p (equation (82), in the footnote above).

[Advanced] **Machine state.** The EVM is a stack machine with 256-bit words. "The word size of the machine (and thus size of stack items) is 256-bit. This was chosen to facilitate the Keccak-256 hash scheme and elliptic-curve computations. The memory model is a simple word-addressed byte array. The stack has a maximum size of 1024. The machine also has an independent storage model; … Unlike memory, which is volatile, storage is non volatile and is maintained as part of the system state."[22] The machine state μ during execution is a tuple of six, "the gas available" (g), "the program counter" (pc), and "the memory contents, the active number of words in memory (counting continuously from position 0), the stack contents, and the returndata buffer",[23] and the **exceptional halting** function Z is true "if there is insufficient gas, if the instruction is invalid (and therefore its δ subscript is undefined), if there are insufficient stack items, if a JUMP/JUMPI destination is invalid, the new stack size would be larger than 1024 or state modification is attempted during a static call."[24] Exceptional halts "do not leave state changes intact. Rather, the machine halts immediately and reports the issue to the execution agent" (§9.1). The code below imitates only one of these, "insufficient gas."

4. Solidity for reading: the smallest contract

[Intermediate] Contracts are usually not written directly in EVM bytecode but in a high-level language such as Solidity and then compiled. The official Solidity documentation uses "simple storage (SimpleStorage)" as its first example: a contract with nothing but a slot that stores one number, a function that writes to that slot, and a function that reads it.[25] The code below is **new code written by this book with the same structure, and it is for reading only.** This book does not compile or deploy it (see the notes to the reader in Chapter 3). The `SPDX-License-Identifier` line at the top is the Solidity convention for stating a license, and this code, like the book's other code, is under CC BY-NC 4.0.

```
// SPDX-License-Identifier: CC-BY-NC-4.0
pragma solidity ^0.8.0;

// Example for reading only — new code written by this book. Not compiled or deployed.
contract OneNumber {
    uint256 number;                                  // state variable: one slot that stays on
the blockchain

    function set(uint256 newNumber) public {         // anyone can call this and overwrite the
slot
        number = newNumber;
    }

    function get() public view returns (uint256) {   // only reads the slot (view)
        return number;
    }
}
```

[Intermediate] The documentation's explanation of its own example fits this code as it stands, so we read along with that explanation (just read the variable name as this book's `number`). "A contract in the sense of Solidity is a collection of code (its functions) and data (its state) that resides at a specific address on the Ethereum blockchain." The one-line state variable declaration "declares a state variable called storedData of type uint (unsigned integer of 256 bits). You can think of it as a single slot in a database that you can query and alter by calling functions of the code that manages the database." And what the documentation adds about what this contract does matters to lawyers: "This contract does not do much yet apart from (due to the infrastructure built by Ethereum) **allowing anyone to store a single number that is accessible by anyone in the world without a (feasible) way to prevent you from publishing this number**. Anyone could call set again with a different value and overwrite your number, but the number is still stored in the history of the blockchain."[26] One state variable, one function to write, one function to read. It carries the name "contract," but there is no offer, no acceptance, and no party here. Adding access restrictions to make it "only I can change it" is the job of the documentation's next example, and that leads to the tokens of Chapter 14.

> **? A LAWYER'S QUESTION**
>
> The code above bears the name "contract." By the Ethereum white paper's definition it is a box that releases value only when certain conditions are met; by the Yellow Paper's motivation it is a system "such that we can reasonably expect that an agreement will be thus enforced autonomously." Where, here, are the offer and the acceptance? Is deploying the code the offer, and is sending a transaction that calls a function the acceptance? Or are both factual acts rather than declarations of intention? When the code behaves differently from what its author intended, is the "content of the contract" the code or the intention? If execution was reverted because the gas ran out but the fee remained paid, what was performed and what was not? In a footnote, the Yellow Paper goes so far as to say that the tools that create transactions "could ultimately become so causally removed from their human-based initiation … that there could be a point at which they rightly be [sic] considered autonomous agents."[27] Who, then, is the one declaring intention? → Chapter 28

5. Code: imitating state transitions and running out of gas with a toy EVM

[Intermediate] The real EVM's instruction set (Appendix H of the Yellow Paper) is much larger than this, but eleven instructions are enough to show the point of this chapter. Below is a toy machine that uses the values of the Appendix G fee schedule unchanged as constants. It has only a stack and storage; there is no memory, no calls, and no events. Of the Appendix H rules for the cost of storage writes, it imitates only two branches: "the extra cost for a slot touched for the first time" and "from 0 to a nonzero value / everything else." **All of it is for study and must not be used for real.**

```
# NB13-1: toy EVM — runs bytecode one instruction at a time, deducting gas (for study; not for
real use)
# The fee schedule values are copied unchanged from Appendix G of the Yellow Paper (Shanghai
version). The instruction set and the storage cost rules are greatly simplified.
G_VERYLOW, G_MID, G_HIGH, G_JUMPDEST = 3, 8, 10, 1
G_COLDSLOAD, G_WARMACCESS, G_SSET, G_SRESET = 2100, 100, 20000, 2900
MOD = 2**256

OPS = {0x00: "STOP", 0x01: "ADD", 0x02: "MUL", 0x03: "SUB", 0x54: "SLOAD", 0x55: "SSTORE",
       0x56: "JUMP", 0x57: "JUMPI", 0x5b: "JUMPDEST", 0x60: "PUSH1", 0x80: "DUP1"}

class OutOfGas(Exception):
    def __init__(self, msg, steps=0):
        super().__init__(msg); self.steps = steps      # also carries the number of instructions
run before stopping

def run(code: bytes, storage: dict, gas: int, trace: bool = False):
    """Runs code from address 0. On success: (remaining gas, new storage, number of instructions
run). Out of gas: OutOfGas.
    Storage is changed in a copy, so if an exception is raised, the caller "reverts" simply by
keeping the original storage."""
    st = dict(storage); stack = []; pc = 0; touched = set(); steps = 0
    def charge(c):
        nonlocal gas
        if gas < c:
            raise OutOfGas(f"pc={pc}: needs {c} gas, {gas} gas left", steps)
        gas -= c
    while pc < len(code):
        name = OPS.get(code[pc])
        if name is None:
            raise OutOfGas(f"pc={pc}: unknown instruction 0x{code[pc]:02x}")   # the real EVM
has a separate exception; simplified here
        steps += 1
        if trace:
            print(f"    pc={pc:>2} {name:<8} stack={stack} gas={gas}")
        if name == "STOP":
            break
        elif name == "PUSH1":
            charge(G_VERYLOW); stack.append(code[pc + 1]); pc += 2; continue
        elif name == "DUP1":
            charge(G_VERYLOW); stack.append(stack[-1])
        elif name in ("ADD", "MUL", "SUB"):
            charge(G_VERYLOW); b, a = stack.pop(), stack.pop()
            stack.append({"ADD": (a + b) % MOD, "MUL": (a * b) % MOD, "SUB": (a - b) % MOD}
[name])
        elif name == "SLOAD":
            key = stack.pop(); charge(G_WARMACCESS if key in touched else G_COLDSLOAD);
touched.add(key)
            stack.append(st.get(key, 0))
        elif name == "SSTORE":
            key, val = stack.pop(), stack.pop()
            cost = 0 if key in touched else G_COLDSLOAD
            touched.add(key)
            cost += G_SSET if (st.get(key, 0) == 0 and val != 0) else G_SRESET
            charge(cost)
            st[key] = val
        elif name == "JUMPDEST":
            charge(G_JUMPDEST)
        elif name == "JUMP":
            charge(G_MID); pc = stack.pop(); continue
        elif name == "JUMPI":
            charge(G_HIGH); dest, cond = stack.pop(), stack.pop()
            if cond != 0:
                pc = dest; continue
        pc += 1
    return gas, st, steps

# Bytecode imitating the core storage operation of OneNumber.set(42): PUSH1 42, PUSH1 0, SSTORE,
STOP  (writes 42 to slot 0; ABI decoding and RETURN are left out)
set42 = bytes([0x60, 42, 0x60, 0, 0x55, 0x00])
gas_left, st, n = run(set42, {}, gas=30_000, trace=True)
```

```
print(f"write 42 to empty storage: {n} instructions, gas used {30_000 - gas_left} (= 3 + 3 +
2100 + 20000), storage {st}")

set7 = bytes([0x60, 7, 0x60, 0, 0x55, 0x00])
gas_left2, st2, n2 = run(set7, st, gas=30_000)
print(f"overwrite 42 with 7:       {n2} instructions, gas used {30_000 - gas_left2} (= 3 + 3 +
2100 + 2900), storage {st2}")

# Imitating the core operation of OneNumber.get(): PUSH1 0, SLOAD, STOP  — reading costs far
less than writing
get = bytes([0x60, 0, 0x54, 0x00])
gas_left3, _, _ = run(get, st2, gas=30_000)
print(f"read slot 0:               gas used {30_000 - gas_left3} (= 3 + 2100)")
```

```
# Output — bcbook (Python 3.13), exactly as run
    pc= 0 PUSH1    stack=[] gas=30000
    pc= 2 PUSH1    stack=[42] gas=29997
    pc= 4 SSTORE   stack=[42, 0] gas=29994
    pc= 5 STOP     stack=[] gas=7894
write 42 to empty storage: 4 instructions, gas used 22106 (= 3 + 3 + 2100 + 20000), storage {0:
42}
overwrite 42 with 7:       4 instructions, gas used 5006 (= 3 + 3 + 2100 + 2900), storage {0: 7}
read slot 0:               gas used 2103 (= 3 + 2100)
```

[Intermediate] Writing a value to storage for the first time took 22,106 gas, overwriting it took 5,006 gas, and reading it took 2,103 gas. Even in a program of four instructions, nearly all of the cost comes **from storage access**, because storage is what every node must keep forever. This is the concrete shape of the saying "putting data on the chain is costly," and it is half of the reason why the rollups of Chapter 18 "put the data on chain but do the execution outside."

[Intermediate] Now we put this machine inside the account state transition function of Chapter 11. In the order of Section 6 of the Yellow Paper: validity checks → up-front payment → execution → (revert on failure) → refund → distribution of fees. The fee distribution follows Section 5 of the Yellow Paper: the base fee is burned and the priority fee goes to the beneficiary address.

```
# NB13-2: applying a transaction — validity checks, up-front payment, execution, revert, refund,
burning the base fee and paying the priority fee (for study)
G_TRANSACTION, G_TXDATAZERO, G_TXDATANONZERO = 21000, 4, 16

def intrinsic_gas(data: bytes) -> int:                        # the part of Yellow Paper equation
(64) for an ordinary transaction (no contract creation, no access list)
    return G_TRANSACTION + sum(G_TXDATAZERO if b == 0 else G_TXDATANONZERO for b in data)

def apply_tx(state: dict, tx: dict, base_fee: int, beneficiary: str):
    """state: {addr: {"nonce","balance","code"(bytes or None),"storage"(dict)}}.
    tx: {"from","to","value","data","gas_limit","max_fee","max_priority","nonce"}.  All units
are toy values."""
    S = {a: {**acc, "storage": dict(acc.get("storage", {}))} for a, acc in state.items()}
    sender = S[tx["from"]]
    g0 = intrinsic_gas(tx["data"])
    # --- intrinsic validity ((3)(5)(6)(7)(8) of Yellow Paper §6) ---
    if tx["nonce"] != sender["nonce"]:                  return S, f"invalid: nonce {tx['nonce']}
≠ account nonce {sender['nonce']}"
    if tx["gas_limit"] < g0:                            return S, f"invalid: gas limit
{tx['gas_limit']} < intrinsic gas {g0}"
    if tx["max_fee"] < base_fee:                        return S, f"invalid: max fee
{tx['max_fee']} < base fee {base_fee}"
    if tx["max_priority"] > tx["max_fee"]:              return S, "invalid: priority fee > max
fee"
    v0 = tx["gas_limit"] * tx["max_fee"] + tx["value"]
    if sender["balance"] < v0:                          return S, f"invalid: balance
{sender['balance']} < up-front cost {v0}"
    # --- effective price (equations (66)(67)) and irreversible up-front payment (equations (73)
(74)) ---
    priority = min(tx["max_priority"], tx["max_fee"] - base_fee)
    price = base_fee + priority
    sender["nonce"] += 1
    sender["balance"] -= tx["gas_limit"] * price
    gas = tx["gas_limit"] - g0
    # --- execution ---
    to = S.setdefault(tx["to"], {"nonce": 0, "balance": 0, "code": None, "storage": {}})
    status = "success"
    try:
        if to["code"]:
            gas, new_storage, _ = run(to["code"], to["storage"], gas)
            to["storage"] = new_storage
        sender["balance"] -= tx["value"]; to["balance"] += tx["value"]
    except OutOfGas as e:
        status = f"out of gas ({e}) → execution reverted"
        gas = 0                                         # discard the storage and transfer
changes (original state kept). The up-front payment and the nonce increment remain
    # --- refund and distribution (the refund counter Ar of equation (82) is 0 in this toy) ---
    gas_used = tx["gas_limit"] - gas
    sender["balance"] += gas * price                     # unused gas is returned at the price
it was bought for
    S.setdefault(beneficiary, {"nonce": 0, "balance": 0, "code": None, "storage": {}})
["balance"] += gas_used * priority
    burned = gas_used * base_fee
    return S, f"{status} | gas used {gas_used} | paid by sender {gas_used * price} = burned
{burned} + validator {gas_used * priority}"

state = {"Alice": {"nonce": 0, "balance": 10_000_000, "code": None, "storage": {}},
         "Store": {"nonce": 1, "balance": 0, "code": set42, "storage": {}}}
BASE = 10                                               # toy base fee (per gas)
tx1 = {"from": "Alice", "to": "Store", "value": 0, "data": b"", "gas_limit": 50_000, "max_fee":
15, "max_priority": 2, "nonce": 0}
state, msg = apply_tx(state, tx1, BASE, "Val")
print("tx1 set(42):", msg)
print("   Store.storage =", state["Store"]["storage"], "| Alice balance", state["Alice"]
["balance"], "| Val balance", state["Val"]["balance"])

tx2 = {**tx1, "nonce": 1, "gas_limit": 21_500}           # above the intrinsic gas of 21000,
but not enough to reach SSTORE
state, msg = apply_tx(state, tx2, BASE, "Val")
print("tx2 gas limit 21,500:", msg)
print("   Store.storage =", state["Store"]["storage"], "(unchanged) | Alice balance",
```

```
state["Alice"]["balance"], "(the fee was paid) | nonce", state["Alice"]["nonce"])

tx3 = {**tx1, "nonce": 2, "max_fee": 9}                  # a transaction that writes a max fee
below the base fee of 10
_, msg = apply_tx(state, tx3, BASE, "Val")
print("tx3 max fee 9:", msg)
```

```
# Output — bcbook (Python 3.13), exactly as run
tx1 set(42): success | gas used 43106 | paid by sender 517272 = burned 431060 + validator 86212
   Store.storage = {0: 42} | Alice balance 9482728 | Val balance 86212
tx2 gas limit 21,500: out of gas (pc=4: needs 5000 gas, 494 gas left) → execution reverted | gas
used 21500 | paid by sender 258000 = burned 215000 + validator 43000
   Store.storage = {0: 42} (unchanged) | Alice balance 9224728 (the fee was paid) | nonce 2
tx3 max fee 9: invalid: max fee 9 < base fee 10
```

[Intermediate] In tx1 the fee the sender paid is gas used × (base 10 + priority 2), and the base-fee share of it was added to no account (burned). Only the priority-fee share came into the validator account Val. tx2 is a transaction sent with its gas limit lowered to 21,500. It exceeds the intrinsic gas of 21,000, so it is a valid transaction and execution starts, but the gas runs out before it reaches the storage write (since 42 is already there, 2,100 + 2,900 = 5,000 gas). Storage is unchanged, the nonce went up, and the fee left the balance. This is what "reverted, but the fee stays paid" looks like. tx3 writes a maximum fee lower than the base fee, so it is a transaction that **cannot get into a block at all**. It is caught by the validity checks, nothing changes, and no fee is paid. "Failure during execution" and "an invalid transaction" are different events.

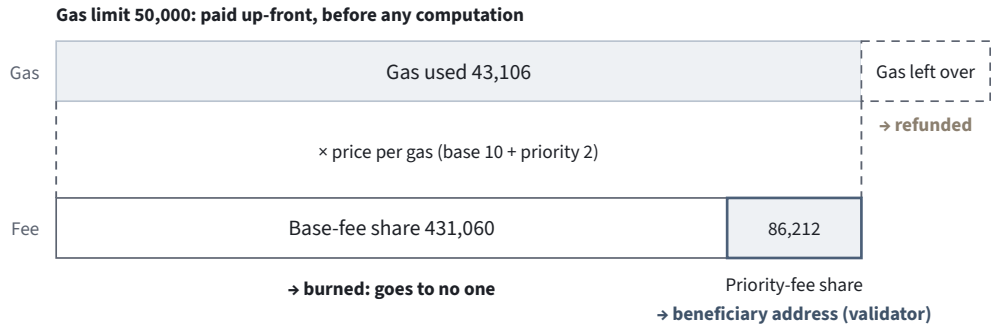


Figure 13-3. A two-tier bar that splits the gas limit into gas used and gas left over (refunded), and then splits the fee for the gas used into the base-fee share (burned) and the priority-fee share (validator). The figures are the toy values of tx1 in NB13-2, Section 5.

[Intermediate] Finally, the halting problem itself. We run a program that never ends.

```
# NB13-3: infinite loop — gas stops code that does not stop (for study)
# JUMPDEST(0) ; PUSH1 0 ; JUMP  → jumps back to address 0 forever. 1 + 3 + 8 = 12 gas per loop.
forever = bytes([0x5b, 0x60, 0x00, 0x56])
for budget in [100, 1_000, 100_000]:
    try:
        run(forever, {}, gas=budget)
        print("finished?!")                                # this line should never run
    except OutOfGas as e:
        print(f"gas {budget:>7}: stopped after {e.steps:>6} instructions (about {e.steps //
3:>5} loops) — {e}")

state2 = {"Alice": {"nonce": 0, "balance": 10_000_000, "code": None, "storage": {}},
          "Loop": {"nonce": 1, "balance": 0, "code": forever, "storage": {}}}
tx = {"from": "Alice", "to": "Loop", "value": 0, "data": b"", "gas_limit": 100_000, "max_fee":
15, "max_priority": 2, "nonce": 0}
state2, msg = apply_tx(state2, tx, BASE, "Val")
print("transaction to the infinite-loop contract:", msg)
print("   Alice balance", state2["Alice"]["balance"], "— the full price of the gas limit
(100,000 × 12 = 1,200,000) was paid. The nodes did only 100,000 gas worth of work")
```

```
# Output — bcbook (Python 3.13), exactly as run
gas     100: stopped after     27 instructions (about     9 loops) — pc=3: needs 8 gas, 0 gas
left
gas    1000: stopped after    252 instructions (about    84 loops) — pc=3: needs 8 gas, 0 gas
left
gas  100000: stopped after  25002 instructions (about  8334 loops) — pc=3: needs 8 gas, 0 gas
left
transaction to the infinite-loop contract: out of gas (pc=3: needs 8 gas, 0 gas left) →
execution reverted | gas used 100000 | paid by sender 1200000 = burned 1000000 + validator
200000
   Alice balance 8800000 — the full price of the gas limit (100,000 × 12 = 1,200,000) was paid.
The nodes did only 100,000 gas worth of work
```

The program does not end, but the machine stops. With a gas budget of 100 it halts in the ninth loop; with 100,000, in the 8,334th loop (12 gas per loop). How much an attacker can burden the nodes is proportional to the gas the attacker bought in advance, and the attacker pays for it. This is just the Ethereum white paper's second example: "the attacker will be required to submit a value for STARTGAS limiting the number of computational steps that execution can take, so the miner will know ahead of time that the computation will take an excessively large number of steps."[28] Gas is a unit for measuring computation and, at the same time, **a device that makes a request for computation write its own upper limit into the transaction.**

6. Summary: a program called a "contract"

Ethereum is a machine that puts programs and their storage on the ledger and, when a transaction comes in, has every node run the same program and arrive at the same state. Because it can run any program, it can also run programs that never stop, so it puts a gas price on every instruction, makes you purchase gas in advance, and when the gas runs out, reverts everything except the fee. The fee splits into the base fee (burned) and the priority fee (validator). The ten-line "contract" read in this chapter had no parties and no agreement. Putting a "balance ledger" on top of it to make what people call tokens is the next chapter, Chapter 14, and the machine that swaps those tokens without people is Chapter 15. This chapter supplied the technical half of misconception ② in Appendix B ("smart contracts are contracts"), and Chapter 28 takes up the legal half.

Sources for this chapter

- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2014 (first draft 2013). https://ethereum.org/whitepaper/ (official PDF, accessed 2026-10-25): the four limitations of Bitcoin script ("Scripting," pp. 11–12 of the PDF), the built-in Turing-complete language and the "cryptographic boxes" (p. 13), messages and transactions, the first class citizen property, running out of gas and refunds (p. 14), the six steps of the state transition function (p. 15), the EVM code execution model (p. 17), the halting problem, the gas solution, and the attack examples (pp. 28–29).
- Gavin Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper, Shanghai version efc5f9a, 2025-02-04. https://ethereum.github.io/yellowpaper/paper.pdf: the motivation (§1.1); transaction fields and footnote 3 (§4.2, pp. 4–5); gas and payment, burning of the base fee, the priority fee (§5, pp. 8–9); the eight validity conditions, intrinsic gas, the effective price, up-front payment, refunds (§6, pp. 9–10); the execution model, machine state, exceptional halting (§9, pp. 13–16); the fee schedule (Appendix G, p. 28). This version is based on the Shanghai fork, and changes in later forks may not be reflected in it.
- The Solidity Authors, *Solidity Documentation*, v0.8.38 (compiler v0.8.38 released 2026-10-22), "Introduction to Smart Contracts." https://docs.soliditylang.org/en/v0.8.38/introduction-to-smart-contracts.html (accessed 2026-10-25; example and explanation compared with the v0.8.37 version): the structure of the SimpleStorage example and the sentences explaining it. The Solidity code in Section 4 was newly written by this book with the same structure; the documentation's example code is not reproduced. Repository https://github.com/argotorg/solidity (GPL-3.0; the old address github.com/ethereum/solidity, which the documentation pages link to, redirects to this address).
- Code execution: Python 3.13 (conda environment `bcbook`), standard library only (no external packages). Notebook NB13. The Solidity code is for reading only and was not compiled or deployed.

Notes

1. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," official PDF on ethereum.org (2014; first draft 2013; the version obtained is a revision from after 2014-04), "Scripting" section (pp. 11–12 of the PDF). The original headings of the four limitations: "Lack of Turing-completeness", "Value-blindness", "Lack of state", "Blockchain-blindness". The original of the first: "while there is a large subset of computation that the Bitcoin scripting language supports, it does not nearly support everything. The main category that is missing is loops." https://ethereum.org/whitepaper/ (accessed 2026-10-25).

2. Buterin (2014), p. 13 of the PDF ("Ethereum" section): "Ethereum does this by building what is essentially the ultimate abstract foundational layer: a blockchain with a built-in Turing-complete programming language, allowing anyone to write smart contracts and decentralized applications where they can create their own arbitrary rules for ownership, transaction formats and state transition functions. ... Smart contracts, cryptographic 'boxes' that contain value and only unlock it if certain conditions are met, can also be built on top of our platform, with vastly more power than that offered by Bitcoin scripting because of the added powers of Turing-completeness, value-awareness, blockchain-awareness and state." The introduction on p. 1 makes the same point: "What Ethereum intends to provide is a blockchain with a built-in fully fledged Turing-complete programming language that can be used to create 'contracts' that can be used to encode arbitrary state transition functions ...".

3. Gavin Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Shanghai version efc5f9a (2025-02-04), §1.1 Driving Factors (p. 1): "one key goal is to facilitate transactions between consenting individuals who would otherwise have no means to trust one another. This may be due to geographical separation, interfacing difficulty, or perhaps the incompatibility, incompetence, unwillingness, expense, uncertainty, inconvenience, or corruption of existing legal systems. By specifying a state-change system through a rich and unambiguous language, and furthermore architecting a system such that we can reasonably expect that an agreement will be thus enforced autonomously, we can provide a means to this end." https://ethereum.github.io/yellowpaper/paper.pdf (accessed 2026-10-25).

4. Buterin (2014), "Messages and Transactions" section (p. 14 of the PDF): "'Messages' in Ethereum are somewhat similar to 'transactions' in Bitcoin, but with three important differences. First, an Ethereum message can be created either by an external entity or a contract, whereas a Bitcoin transaction can only be created externally. Second, there is an explicit option for Ethereum messages to contain data. Finally, the recipient of an Ethereum message, if it is a contract account, has the option to return a response; this means that Ethereum messages also encompass the concept of functions. The term 'transaction' is used in Ethereum to refer to the signed data package that stores a message to be sent from an externally owned account." Emphasis added.

5. Same section, same page: "An important consequence of the message mechanism is the 'first class citizen' property of Ethereum - the idea that contracts have equivalent powers to external accounts, including the ability to send message and create other contracts."

6. Buterin (2014), same section: "Transactions contain the recipient of the message, a signature identifying the sender, the amount of ether and the data to send, as well as two values called STARTGAS and GASPRICE."

7. Wood (2025), §4.2 The Transaction (pp. 4–5). Common fields type, nonce, gasLimit, to, value, r, s; accessList, chainId, and yParity for types 1 and 2; for type 2, maxFeePerGas ("the maximum number of Wei to be paid per unit of gas for all computation costs incurred as a result of the execution of this transaction") and maxPriorityFeePerGas ("the maximum number of Wei to be paid to the block's fee recipient as an incentive to include the transaction"); for types 0 and 1, gasPrice. The text on gasLimit is quoted in the main text; emphasis added. The Yellow Paper states that the transaction types (0, 1, 2) were introduced by EIP-2718 and that there are three as of the London release. This version of the Yellow Paper is based on the Shanghai fork, and later forks (such as type 3 transactions, which carry blobs) may not be reflected in it.

8. Buterin (2014), "Ethereum State Transition Function" section (p. 15 of the PDF). The six steps in the original: "1. Check if the transaction is well-formed (ie. has the right number of values), the signature is valid, and the nonce matches the nonce in the sender's account. If not, return an error. 2. Calculate the transaction fee as STARTGAS * GASPRICE, and determine the sending address from the signature. Subtract the fee from the

sender's account balance and increment the sender's nonce. If there is not enough balance to spend, return an error. 3. Initialize GAS = STARTGAS, and take off a certain quantity of gas per byte to pay for the bytes in the transaction. 4. Transfer the transaction value from the sender's account to the receiving account. If the receiving account does not yet exist, create it. If the receiving account is a contract, run the contract's code either to completion or until the execution runs out of gas. 5. If the value transfer failed because the sender did not have enough money, or the code execution ran out of gas, revert all state changes except the payment of the fees, and add the fees to the miner's account. 6. Otherwise, refund the fees for all remaining gas to the sender, and send the fees paid for gas consumed to the miner." Emphasis in the main text added. "Miner" is the term of the 2014 white paper; since the switch to proof of stake, the fee-recipient address of the validator that proposed the block takes its place (Chapter 10).

9. Buterin (2014), "Messages and Transactions" section (p. 14 of the PDF).

10. Buterin (2014), "Computation And Turing-Completeness" section (p. 28 of the PDF): "An important note is that the Ethereum virtual machine is Turing-complete; this means that EVM code can encode any computation that can be conceivably carried out, including infinite loops. ... This naturally leads to a problem: can malicious users essentially shut miners and full nodes down by forcing them to enter into an infinite loop? The issue arises because of a problem in computer science known as the halting problem: there is no way to tell, in the general case, whether or not a given program will ever halt." Emphasis added.

11. Same section, same page: "our solution works by requiring a transaction to set a maximum number of computational steps that it is allowed to take, and if execution takes longer computation is reverted but fees are still paid. Messages work in the same way."

12. Wood (2025), §9 Execution Model (pp. 13–14): "It is a quasi-Turing-complete machine; the quasi qualification comes from the fact that the computation is intrinsically bounded through a parameter, gas, which limits the total amount of computation done."

13. Buterin (2014), p. 29 of the PDF.

14. Wood (2025), §5 Gas and Payment (p. 8). Emphasis added.

15. Wood (2025), §9.2 Fees Overview (p. 14): "Storage fees have a slightly nuanced behaviour—to incentivise minimisation of the use of storage (which corresponds directly to a larger state database on all nodes), the execution fee for an operation that clears an entry in the storage is not only waived, a qualified refund is given; in fact, this refund is effectively paid up-front since the initial usage of a storage location costs substantially more than normal usage."

16. Wood (2025), Appendix G. Fee Schedule (p. 28). The table in the original: "Gverylow 3 Amount of gas to pay for operations of the set Wverylow. ... Gmid 8 ... Ghigh 10 ... Gwarmaccess 100 Cost of a warm account or storage access. ... Gcoldsload 2100 Cost of a cold storage access. Gsset 20000 Paid for an SSTORE operation when the storage value is set to non-zero from zero. Gsreset 2900 Paid for an SSTORE operation when the storage value's zeroness remains unchanged or is set to zero. Rsclear 4800 Refund given (added into refund counter) when the storage value is set to zero from non-zero. ... Gcreate 32000 Paid for a CREATE operation. ... Gtxdatazero 4 Paid for every zero byte of data or code for a transaction. Gtxdatanonzero 16 Paid for every non-zero byte of data or code for a transaction. Gtransaction 21000 Paid for every transaction." The actual cost rule for storage writes (the SSTORE entry in Appendix H) is further divided by "whether the slot is touched for the first time" and by the combination of "original value, current value, new value," and the code in this chapter imitates only two of those cases.

17. Wood (2025), §6.2 (p. 10), equation (82) and footnote 9: "The total refundable amount is the legitimately remaining gas g′, added to Ar, with the latter component being capped up to a maximum of one fifth (rounded down) of the total amount used Tg − g′." Footnote 9: "The max refundable proportion of gas was reduced from one half to one fifth by EIP-3529 by Buterin and Swende [2021] in the London release".

18. Wood (2025), §5 (pp. 8–9): "Since the introduction of EIP-1559 by Buterin et al. [2019] in the London hard fork, every transaction included in a block must pay a base fee, which is specified as wei per unit of gas consumed and is constant for each transaction within a block. The ether that is paid to meet the base fee is burned (taken out of circulation). ... To incentivize validators to include transactions, there is an additional fee known as a priority fee, which is also specified as wei per unit of gas consumed. The total fee paid by the transactor therefore is the sum of the base fee per gas and the priority fee per gas multiplied by the total gas consumed. Ether used to satisfy the priority fee is delivered to the beneficiary address, the address of an account

typically under the control of the validator." Emphasis added.

19. Wood (2025), §6 Transaction Execution (p. 9): "(1) The transaction is well-formed RLP, with no additional trailing bytes; (2) the transaction signature is valid; (3) the transaction nonce is valid (equivalent to the sender account's current nonce); (4) the sender account has no contract code deployed (see EIP-3607 by Feist et al. [2021]); (5) the gas limit is no smaller than the intrinsic gas, g0, used by the transaction; (6) the sender account balance contains at least the cost, v0, required in up-front payment; (7) the maxFeePerGas, Tm, in the case of type 2 transactions, or gasPrice, Tp, in the case of type 0 and type 1 transactions, is greater than or equal to the block's base fee, Hf; and (8) for type 2 transactions, maxPriorityFeePerGas, Tf, must be no larger than maxFeePerGas, Tm."

20. Wood (2025), §6.2 Execution (pp. 9–10), equation (64): "We define intrinsic gas g0, the amount of gas this transaction requires to be paid prior to execution, as follows: g0 ≡ Σ (Gtxdatazero if i = 0, Gtxdatanonzero otherwise) + (Gtxcreate + R(‖ Ti ‖) if Tt = ∅, 0 otherwise) + Gtransaction + Σ (Gaccesslistaddress + ‖ TA[j]s ‖ Gaccessliststorage) ...".

21. Wood (2025), §6.2 (p. 10), equations (66)–(68): "p ≡ Tp if Tx = 0 ∨ Tx = 1; f + Hf if Tx = 2", "f ≡ Tp − Hf if Tx = 0 ∨ Tx = 1; min(Tf, Tm − Hf) if Tx = 2", "v0 ≡ TgTp + Tv if Tx = 0 ∨ Tx = 1; TgTm + Tv if Tx = 2". The irreversible changes at the start of execution (equations (73)–(75)): "the nonce of the account of the sender, S(T), is incremented by one and the balance is reduced by part of the up-front cost, Tgp."

22. Wood (2025), §9.1 Basics (p. 14): "The EVM is a simple stack-based architecture. The word size of the machine (and thus size of stack items) is 256-bit. This was chosen to facilitate the Keccak-256 hash scheme and elliptic-curve computations. The memory model is a simple word-addressed byte array. The stack has a maximum size of 1024. The machine also has an independent storage model; this is similar in concept to the memory but rather than a byte array, it is a word-addressable word array. Unlike memory, which is volatile, storage is non volatile and is maintained as part of the system state." On exceptional execution: "The machine can have exceptional execution for several reasons, including stack underflows and invalid instructions. Like the out-of-gas exception, they do not leave state changes intact. Rather, the machine halts immediately and reports the issue to the execution agent ...".

23. Wood (2025), §9.4.1 Machine State (p. 15): "The machine state μ is defined as the tuple (g, pc, m, i, s, o) which are the gas available, the program counter pc ∈ N256, the memory contents, the active number of words in memory (counting continuously from position 0), the stack contents, and the returndata buffer."

24. Wood (2025), §9.4.2 Exceptional Halting (p. 15), the explanation below equation (158): "This states that the execution is in an exceptional halting state if there is insufficient gas, if the instruction is invalid (and therefore its δ subscript is undefined), if there are insufficient stack items, if a JUMP/JUMPI destination is invalid, the new stack size would be larger than 1024 or state modification is attempted during a static call."

25. The Solidity Authors, *Solidity Documentation* v0.8.38, "Introduction to Smart Contracts — A Simple Smart Contract — Storage Example." https://docs.soliditylang.org/en/v0.8.38/introduction-to-smart-contracts.html (accessed 2026-10-25; the heading of the document reads "0.8.38-develop"). The same example appears, identical character for character, in the documentation of the immediately preceding fixed version, v0.8.37 (https://docs.soliditylang.org/en/v0.8.37/introduction-to-smart-contracts.html), and the heading of the v0.8.37 documentation also reads "0.8.37-develop," so "-develop" appears to be the way this documentation site generates its titles, not a mark of an unfinished version (the two versions were compared on 2026-10-25). The documentation's example code is distributed under GPL-3.0, so this book does not reproduce it. The original code can be seen at the address above. In the documentation's example, the state variable is named `storedData` and the function argument `x`.

26. Same document, same section: "A contract in the sense of Solidity is a collection of code (its functions) and data (its state) that resides at a specific address on the Ethereum blockchain. The line uint storedData; declares a state variable called storedData of type uint (unsigned integer of 256 bits). You can think of it as a single slot in a database that you can query and alter by calling functions of the code that manages the database." "This contract does not do much yet apart from (due to the infrastructure built by Ethereum) allowing anyone to store a single number that is accessible by anyone in the world without a (feasible) way to prevent you from publishing this number. Anyone could call set again with a different value and overwrite your number, but the number is still stored in the history of the blockchain. Later, you will see how you can impose access restrictions so that only you can alter the number." Emphasis added.

27. Wood (2025), §4.2, footnote 3 (p. 4): "Notably, such 'tools' could ultimately become so causally removed from

their human-based initiation—or humans may become so causally-neutral—that there could be a point at which they rightly be considered autonomous agents. e.g. contracts may offer bounties to humans for being sent transactions to initiate their execution."

28. Buterin (2014), pp. 28–29 of the PDF: "An attacker creates a very long infinite loop with the intent of forcing the miner to keep computing for such a long time that by the time computation finishes a few more blocks will have come out and it will not be possible for the miner to include the transaction to claim the fee. However, the attacker will be required to submit a value for STARTGAS limiting the number of computational steps that execution can take, so the miner will know ahead of time that the computation will take an excessively large number of steps."

PART 2 · SMART CONTRACTS AND APPLICATIONS

Chapter 14. Token standards: what ERC-20 and 721 decide and what they leave undecided

KEY TAKEAWAY

A token is **one balance ledger** inside a contract account from Chapter 13. "Subtract X from A and add X to B, provided that A has X and that A has approved it." That is all, and Buterin called it "a database with one operation." A token **standard** (ERC-20, ERC-721) is an agreement that unifies **the names, inputs, and outputs of the functions** that handle that ledger, so that wallets and exchanges can handle tokens they have never seen before. A standard decides what the functions do, but it does not decide what the token **means** in the world (a voting right, a claim to a share of revenue, a link to a picture file). That is a promise the issuer made outside the standard, and the legal nature of that promise is decided by law, not by code. After this chapter you will be able to separate what the sentence "I bought an NFT" means technically from what that sentence **does not say**.

Learning objectives

1. Distinguish coins (assets built into the protocol) from tokens (ledgers inside a contract), and explain that a token is "one mapping in a contract's storage."
2. **[Intermediate]** Read the six functions and two events of ERC-20 and the ownership, transfer, and approval functions of ERC-721, and separate what the standards fix as "MUST," what they leave "OPTIONAL," and what they are silent on altogether.
3. See how the difference between fungible and non-fungible tokens shows up in the data structure.
4. **[Intermediate]** Read "what a token points to" in two parts: on-chain rights (what the code enforces) and off-chain promises (what the issuer said).

What a reader of the [Basic] paragraphs alone takes away. A token standard is only an agreement on function names. What rights a token carries is decided not by the standard but by **the issuer's promise outside it**.

1. What failure this technology grew out of

The "simple storage" contract in Chapter 13 stored one number. What happens if you store a number **for each address**? Buterin's 2014 white paper says that this is money: "The key point to understand is that all a currency, or token systen [sic], fundamentally is is **a database with one operation**: subtract X units from A and give X units to B, with the proviso that (i) X [sic] had at least X units before the transaction and (2) the transaction is approved by A. All that it takes to implement a token system is to implement this logic into a contract."[1] The white paper writes the logic out in six lines and lists its uses as ranging from "sub-currencies representing assets such as USD or gold to company stocks, individual tokens representing smart property, secure unforgeable coupons, and even token systems with no ties to conventional value at all, used as point systems for incentivization."[2] Keep this list in mind. The same six lines can be a share of stock, a coupon, or a game score. The code does not know the difference.

The problem arose because everyone wrote those six lines **differently**. If one contract calls its sending function `send` and another calls it `transfer`, a wallet program has to write separate code for every token. The motivation section of ERC-20 is a single sentence: "A standard interface allows any tokens on Ethereum to be re-used by other applications: from wallets to decentralized exchanges."[3] 2 years later ERC-721 adapted the same frame to assets that are **distinct from one another**: "EIP-20 is insufficient for tracking NFTs because each asset is distinct (non-fungible) whereas each of a quantity of tokens is identical (fungible)."[4] Both grew out of the inefficiency of having **no agreed function names**, and both answered by unifying the function names. They did nothing more.

2. Intuition: coins, tokens, and a single ledger

First, the words. A **coin** is an asset the protocol itself knows about: Bitcoin's BTC, Ethereum's ETH (ether). Ether is recorded in an account's `balance` field (Chapter 11) and is used to pay for gas. The white paper gives ether two roles: "providing a primary liquidity layer to allow for efficient exchange between various types of digital assets and, more importantly, of providing a mechanism for paying transaction fees."[5] A **token** is an asset the protocol does not know about. Inside the storage of some contract account there is one "address → quantity" mapping, and the functions of that contract change the mapping. To the protocol, a token transfer is just "two storage slots of some contract changed." That is why the cost of writing to storage, seen in Chapter 13, makes up a large share of the gas for a token transfer. The share depends on whether the recipient's slot was empty (20,000 from 0 to non-0), already held a value (2,900), and whether the slot is being touched for the first time in that transaction (+2,100). In an ordinary transfer where both slots already held values, it is smaller than the base transaction cost of 21,000.

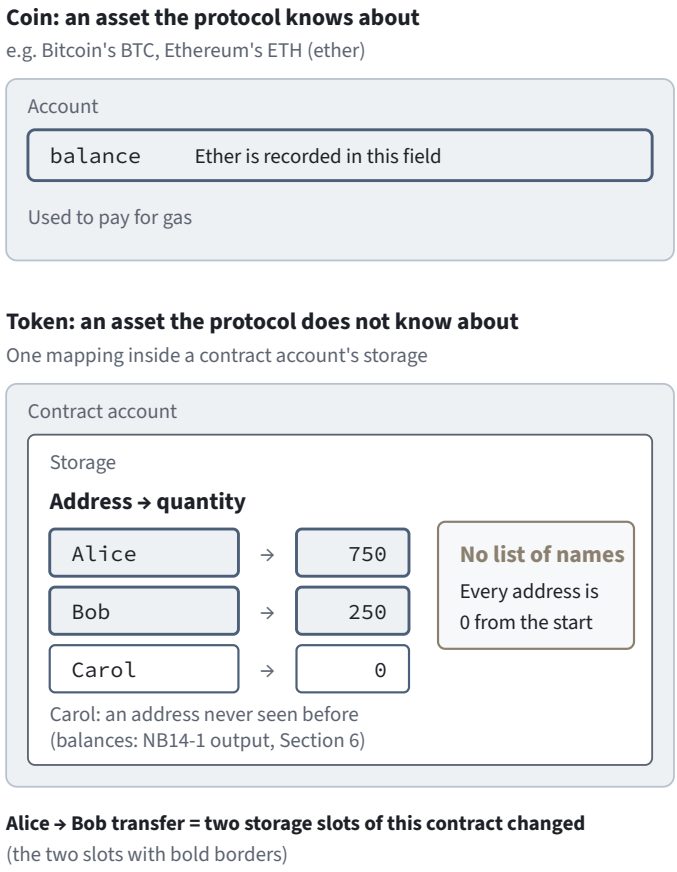


Figure 14-1. Coins and tokens. Ether is recorded in an account's balance field; a token is one "address → quantity" mapping inside a contract account's storage. The mapping has no list of names, and every address reads as 0 from the start (balances in the output of NB14-1 in Section 6).

[Intermediate] The second example in the Solidity documentation quoted in Chapter 13 ("Subcurrency Example") is exactly this ledger. The documentation introduces it like this: "The following contract implements the simplest form of a cryptocurrency. The contract allows only its creator to create new coins (different issuance schemes are possible). Anyone

can send coins to each other without a need for registering with a username and password, all you need is an Ethereum keypair."[6] The code has two essentials. One mapping that pairs each address with a quantity is the ledger, and the two lines inside the send function that subtract from the sender's balance and add to the recipient's balance are the white paper's "one operation" (this book has written new Python code that does the same thing, in Section 6 below). What the documentation adds about mappings shows the nature of a token well: "Mappings can be seen as hash tables which are virtually initialized such that every possible key exists from the start and is mapped to a value whose byte-representation is all zeros. However, it is neither possible to obtain a list of all keys of a mapping, nor a list of all values."[7] So a token contract **does not have a list** of "who has this token." Give it an address and it tells you that address's quantity, and that is all. Every address's balance "exists" as 0 from the start. For a lawyer who pictures a "register of holders," this point is not a small one.

3. ERC-20: the six functions a fungible token promises

[Intermediate] The ERC-20 specification is short. Nine functions and two events make up the whole of it, and three of the functions are optional. The table is this book's summary of the descriptions in the original, and the original wording is in the footnote.[8]

FUNCTION	MEANING	LEVEL REQUIRED BY THE STANDARD
`totalSupply()`	Total quantity of the token	Required
`balanceOf(owner)`	Balance of `owner`	Required
`transfer(to, value)`	Moves `value` from the caller's balance to `to` and **must** emit a `Transfer` event. Should fail if the balance is insufficient (SHOULD)	Required
`transferFrom(from, to, value)`	Moves tokens from the balance of `from` to `to`. For the "withdraw workflow": lets **a contract, on your behalf,** move tokens. Should fail unless `from` has given the caller authority in advance (SHOULD)	Required
`approve(spender, value)`	Allows `spender` to withdraw from the caller's account **several times, up to a total of** `value`. Calling it again overwrites	Required
`allowance(owner, spender)`	The amount `spender` may still withdraw from `owner`	Required
`name()` · `symbol()` · `decimals()`	Name, symbol, number of decimal places	**Optional**: "interfaces and other contracts MUST NOT expect these values to be present"
`Transfer(from, to, value)` event	Must fire whenever tokens are moved, **including transfers of 0**	Required
`Approval(owner, spender, value)` event	Fires on every successful `approve`	Required

[Intermediate] Three points. First, why there is an `approve` / `transferFrom` pair. Chapter 13

said that contract accounts have no private key. So a contract cannot move your tokens **with your signature**. Instead, you first record with `approve` that "this contract may take up to this much of my tokens," and the contract then takes them with `transferFrom`. This is how the exchange contract in Chapter 15 receives tokens. Second, the standard itself attaches a warning to `approve`: "To prevent attack vectors like the one described here and discussed here, clients SHOULD make sure to create user interfaces in such a way that they set the allowance first to `0` before setting it to another value for the same spender. THOUGH The contract itself shouldn't enforce it, to allow backwards compatibility with contracts deployed before"[9]. Which attack this is, the code below reenacts. It is the first case in which the **order** of transactions changes the outcome, and a preview of Chapter 17. Third, the standard does **not set** rules for issuing (minting) or burning. There is only a recommendation (SHOULD) to emit a `Transfer` event with `_from` set to the zero address when new tokens are created. Who may create how many, and on what conditions, is entirely up to the issuer.

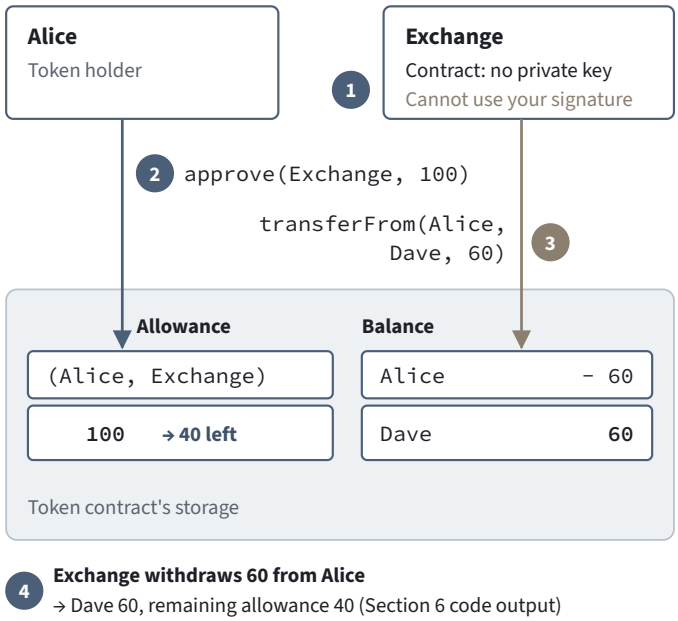


Figure 14-2. The withdraw workflow of approve and transferFrom. A contract, which has no private key, moves tokens only within the allowance the owner recorded in advance. The figures are from the output of NB14-1 in Section 6.

[Intermediate] Where the standard says "SHOULD," implementations diverge. The Uniswap v2 white paper writes that "The ERC-20 standard requires that transferand transferFromreturn a boolean indicating the success or failure of the call [4]. The implementations of one or both of these functions on some tokens—including popular ones ...—instead have no return value," and says that it changed its own code to handle such tokens.[10] "Follows the standard" is closer to "has the same function names," and the detailed behavior has to be read contract by contract. A token using the same names is not the same as a token being the same thing.

4. ERC-721: a ledger of things that are each different

[Intermediate] The ledger of a non-fungible token (NFT) is not "address → quantity" but **"token number → owner address."** ERC-721 says: "Every NFT is identified by a unique `uint256` ID inside the ERC-721 smart contract. This identifying number SHALL NOT change for the life of the contract. The pair `(contract address, uint256 tokenId)` will then be a **globally unique and fully-qualified identifier** for a specific asset on an Ethereum chain."[11] The key functions are `ownerOf(tokenId)` (who owns this number), `balanceOf(owner)` (how many this address has), and transfer and approval. There are two transfer functions. `transferFrom` warns in capital letters that "THE CALLER IS RESPONSIBLE TO CONFIRM THAT `_to` IS CAPABLE OF RECEIVING NFTS OR ELSE THEY **MAY BE PERMANENTLY LOST**," and `safeTransferFrom`, if the recipient is a contract, calls that contract's `onERC721Received` and fails unless a set magic value comes back.[12] It is a device for blocking, at the token level, the problem of "coins sent to an address with no key" seen in Chapter 11. Code can ask whether a contract is ready to receive. It cannot ask a person.

[Intermediate] There are also two kinds of approval. `approve(approved, tokenId)` puts one agent on one token, and `setApprovalForAll(operator, approved)` turns on "approval for a third party ('operator') to manage **all** of `msg.sender`'s assets." The standard explains this operator concept as "a powerful set of tools for wallet, broker and auction applications to quickly use a *large* number of NFTs."[13] There is no ERC-20-style `allowance` because "every NFT is unique, the quantity is none or one."[14] The standard also states what it does **not** decide: "Creation of NFTs ('minting') and destruction of NFTs ('burning') is not included in the specification. Your contract may implement these by other means."[15] And while the documentation of the transfer functions, in the standard's words, "only specify conditions when the transaction MUST throw" and an implementation "MAY also throw in other situations," the standard gives as examples "Disallow transfers if the contract is paused," "Blocklist certain address from receiving NFTs," "Charge a fee to both parties of a transaction," and "Read only NFT registry — always throw from `safeTransferFrom`, `transferFrom`, `approve` and `setApprovalForAll`."[16] That an NFT contract following the standard **can block transfers** is written in the standard itself.

Metadata: where is the "picture"?

The pictures, music, and documents people have in mind when they say "NFT" are not on the chain. ERC-721's **metadata extension** is optional (OPTIONAL), and what it contains is three functions: `name()`, `symbol()`, and `tokenURI(tokenId)`. `tokenURI` returns "A distinct Uniform Resource Identifier (URI) for a given asset," and "The URI may point to a JSON file that conforms to the 'ERC721 Metadata JSON Schema'" (may, in lowercase). That schema has three entries: `name`, `description`, and `image`, "A URI pointing to a resource with mime type image/ *representing the asset to which this NFT represents [sic]."*[17] *So what is written on the chain is one address string*, and the file that address points to holds, in turn, the address of the picture. The standard is candid about why it chose this design. It considered the alternative to "put all metadata for each asset on the blockchain" and dropped it as "too expensive."[18] The storage cost of Chapter 13 made the decision here.

The standard gives two more warnings. "The URI **MAY be mutable** (i.e. it changes from time to time). We considered an NFT representing ownership of a house, in this case metadata about the house (image, occupants, etc.) can naturally change." And: "We also remind everyone that any smart contract can use the same name and symbol as *your* contract. How a client may determine which ERC-721 smart contracts are well-known (canonical) is outside the scope of this standard."[19] The chain guarantees the token number and the owner. It does not guarantee the contents of the file the token points to, whether that file will stay where it is, or whether the contract is the "real" contract of that artist. The authors of the standard wrote so themselves.

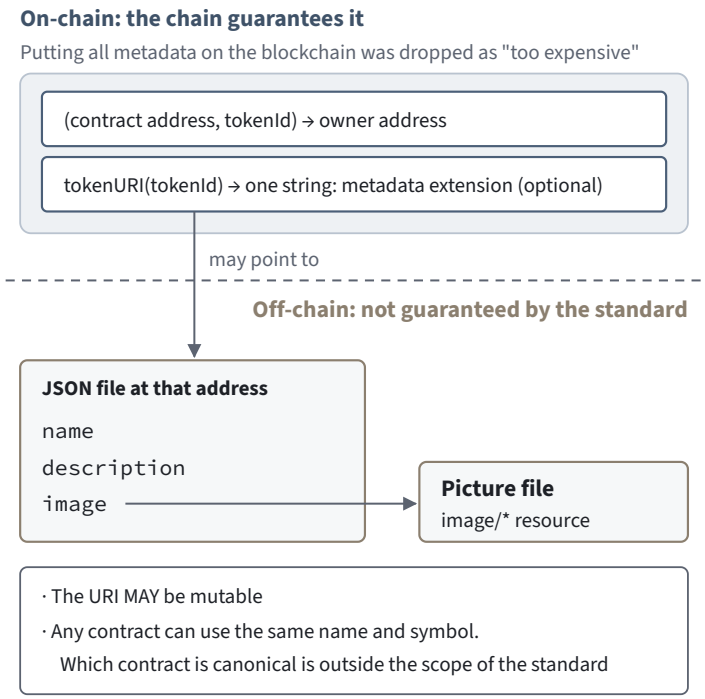


Figure 14-3. What the chain guarantees for an NFT and what it does not. On the chain is the owner of (contract address, tokenId) and, if the contract implements the (optional) metadata extension, a tokenURI string. The JSON and picture file that string may point to, and which contract is the canonical one, lie outside the standard.

> **? A LAWYER'S QUESTION**
>
> What has a person who "bought an NFT" bought? What is written on the chain is the fact that the person's address went into the owner slot of a (contract address, number) pair, and one string returned by `tokenURI`. The picture file sits on an outside server that the string points to, and the standard says the string can change. Copyright (저작권) is not written anywhere. Then is the object of the sale (매매의 목적물) the token, the file the token points to, some right in the file, or the fact of "pointing" itself? If the issuer shuts down the server and the picture disappears, what has perished (멸실)? If another contract is deployed with the same name and symbol, which one is the "genuine article (진품)"? And if the issuer of an ERC-20 token promised "revenue sharing" or "voting rights (의결권)," is that promise in the code or outside it? → Chapter 28 · Chapter 27

5. What a token points to: on-chain rights and off-chain promises

Reduced to one line, this chapter's claim is this. **What the code enforces** and **what the issuer said** sit on different layers. What the code enforces are state transitions such as "move this address's quantity to that address," "the owner of this number is this address," and "block the transfer under this condition." What the issuer said are sentences such as "this token is a share of our company's earnings," "you can vote with this token," and "this token is ownership of this picture," and those sentences are in white papers, on websites, and on message boards. The standards deal only with the former.

The event in which history showed this distinction most clearly is "The DAO" of 2016. The detailed analysis belongs to Chapter 26, and here we look only at the relation between tokens and promises. The 2017 report of investigation by the U.S. Securities and Exchange Commission (SEC) sets out the facts this way: "In exchange for ETH, The DAO created DAO Tokens (proportional to the amount of ETH paid) that were then assigned to the Ethereum Blockchain address of the person or entity remitting the ETH. A DAO Token granted the DAO Token holder **certain voting and ownership rights**." Token holders cast votes on funding proposals "weighted by the number of tokens they controlled," and they were told in promotional materials that they "would receive 'rewards,' which the White Paper defined as, 'any [ETH] received by a DAO [Entity] generated from projects the DAO [Entity] funded.'"[20] What the code enforced was voting in proportion to the number of tokens, and the movement of funds. "Rewards" and "ownership" were words in the white paper and on the website. And the legal nature of those words was decided not by code but by the SEC. The report concluded that "DAO Tokens are securities under the Securities Act of 1933 ('Securities Act') and the Securities Exchange Act of 1934 ('Exchange Act')," and at the outset (p. 2) it wrote: "The automation of certain functions through this technology, 'smart contracts,' or computer code, does not remove conduct from the purview of the U.S. federal securities laws."[21] Whether this conclusion is right, and how another jurisdiction would see the matter, is outside the scope of this book, and Part 5 also treats it only as a survey. What this chapter takes away is the structure. Even when the six lines inside the token contract

are identical, whether the token was a security, a coupon, or a game score was decided by **facts outside the code**.

6. Code: imitating the two ledgers and recomputing the numbers the standards wrote down

[Intermediate] We imitate the two ledgers with Python dictionaries. Signature verification was done in Chapter 5, so "who called" is represented by a name, and events are written to a list. All of it is for study.

```
# NB14-1: imitating the ERC-20 ledger — balance and allowance dictionaries, the six functions,
the two events, and the approve race condition (for study)
class ERC20Toy:
    """Only the required functions of EIP-20. Failures raise exceptions (the standard's SHOULD
throw). Events are written to self.events."""
    def __init__(self, name, symbol):
        self.name, self.symbol = name, symbol             # OPTIONAL items — may or may not be
present
        self.balances = {}                                 # "every address is 0 from the start"
— read as 0 if not in the dictionary
        self.allowances = {}                               # (owner, spender) -> remaining
allowance
        self.events = []
    def totalSupply(self):          return sum(self.balances.values())
    def balanceOf(self, owner):     return self.balances.get(owner, 0)
    def allowance(self, o, s):      return self.allowances.get((o, s), 0)
    def mint(self, to, value):                             # outside the standard — issuance
rules are up to the issuer. Only a Transfer event from 0x0 is recommended (SHOULD)
        self.balances[to] = self.balanceOf(to) + value
        self.events.append(("Transfer", "0x0", to, value))
    def transfer(self, caller, to, value):
        if self.balanceOf(caller) < value:
            raise ValueError(f"insufficient balance: {caller} has only
{self.balanceOf(caller)}")
        self.balances[caller] -= value
        self.balances[to] = self.balanceOf(to) + value
        self.events.append(("Transfer", caller, to, value))   # emit the event even for a
quantity of 0 (MUST)
        return True
    def approve(self, caller, spender, value):
        self.allowances[(caller, spender)] = value            # "calling it again overwrites"
        self.events.append(("Approval", caller, spender, value))
        return True
    def transferFrom(self, caller, frm, to, value):
        if self.allowance(frm, caller) < value:
            raise ValueError(f"insufficient allowance: the amount {frm} allowed {caller} is
{self.allowance(frm, caller)}")
        if self.balanceOf(frm) < value:
            raise ValueError("insufficient balance")
        self.allowances[(frm, caller)] -= value
        self.balances[frm] -= value
        self.balances[to] = self.balanceOf(to) + value
        self.events.append(("Transfer", frm, to, value))
        return True

tok = ERC20Toy("Toy Token", "TOY")
tok.mint("Alice", 1000)
tok.transfer("Alice", "Bob", 250)
print("balances — Alice:", tok.balanceOf("Alice"), "Bob:", tok.balanceOf("Bob"), "Carol (an
address never seen before):", tok.balanceOf("Carol"), "| total supply:", tok.totalSupply())
try:
    tok.transfer("Bob", "Carol", 300)
except ValueError as e:
    print("Bob → Carol 300:", e)

# approve / transferFrom — the withdraw workflow in which a contract (here 'Exchange') takes
tokens on Alice's behalf
tok.approve("Alice", "Exchange", 100)
tok.transferFrom("Exchange", "Alice", "Dave", 60)
print("Exchange withdraws 60 from Alice → Dave:", tok.balanceOf("Dave"), "| remaining
allowance:", tok.allowance("Alice", "Exchange"))

# the approve race condition — reenacting, as a sequence, why the standard recommends "set it to
0 first"
tok2 = ERC20Toy("Toy", "TOY"); tok2.mint("Alice", 1000)
tok2.approve("Alice", "Spender", 100)                      # ① Alice: allows Spender 100
# Alice sends approve(Spender, 50), thinking "100 is too much, cut it to 50." But the order in
which transactions enter a block is not the order in which they were sent.
tok2.transferFrom("Spender", "Alice", "Spender", 100)      # ② Spender takes out all 100 before
Alice's transaction
tok2.approve("Alice", "Spender", 50)                       # ③ then Alice's approve(50) is
```

```
applied — because it "overwrites," a fresh 50 is opened
tok2.transferFrom("Spender", "Alice", "Spender", 50)       # ④ Spender takes out another 50
print("race condition — Alice meant to allow only 50, but the amount Spender took:",
tok2.balanceOf("Spender"), "(= 100 + 50)")
print("number of events:", len(tok2.events), "| last three:", tok2.events[-3:])
```

```
# Output — bcbook (Python 3.13), exactly as run
balances — Alice: 750 Bob: 250 Carol (an address never seen before): 0 | total supply: 1000
Bob → Carol 300: insufficient balance: Bob has only 250
Exchange withdraws 60 from Alice → Dave: 60 | remaining allowance: 40
race condition — Alice meant to allow only 50, but the amount Spender took: 150 (= 100 + 50)
number of events: 5 | last three: [('Transfer', 'Alice', 'Spender', 100), ('Approval', 'Alice',
'Spender', 50), ('Transfer', 'Alice', 'Spender', 50)]
```

[Intermediate] The last reenactment is the first appearance of the "transaction order" problem. Alice sent a transaction to **reduce** the allowance from 100 to 50, and if it is processed after Spender has taken out the 100 first, a fresh 50 is opened. This is because `approve` "overwrites" rather than "adds," and because Alice does not decide the order in which transactions enter the chain. Who decides that order is the subject of Chapter 17.

```
# NB14-2: imitating the ERC-721 ledger — number → owner, approvals and operators, the recipient
check in safeTransferFrom, a mutable URI (for study)
MAGIC = "onERC721Received-magic"                              # in reality
bytes4(keccak256("onERC721Received(address,address,uint256,bytes)")) — computed in NB14-3 below

class Wallet:                                                 # imitates a "contract recipient." A
contract that is ready to receive returns the magic value
    def __init__(self, ready): self.ready = ready
    def onERC721Received(self, operator, frm, tokenId, data): return MAGIC if self.ready else
"something else"

class ERC721Toy:
    def __init__(self):
        self.owner = {}            # tokenId -> owner
        self.approved = {}         # tokenId -> approved address
        self.operators = set()     # (owner, operator)
        self.uri = {}              # tokenId -> URI string (metadata extension, OPTIONAL)
        self.events = []
    def ownerOf(self, tid):
        if tid not in self.owner: raise KeyError(f"invalid NFT {tid}")   # a query about an
invalid NFT (one assigned to the zero address, etc.) fails — the gist of the ownerOf comment in
the original
        return self.owner[tid]
    def balanceOf(self, who):  return sum(1 for o in self.owner.values() if o == who)
    def mint(self, to, tid, uri):                              # outside the standard — creation
rules are up to the issuer
        self.owner[tid] = to; self.uri[tid] = uri; self.events.append(("Transfer", "0x0", to,
tid))
    def tokenURI(self, tid):   self.ownerOf(tid); return self.uri[tid]
    def _authorized(self, caller, tid):
        o = self.ownerOf(tid)
        return caller == o or self.approved.get(tid) == caller or (o, caller) in self.operators
    def approve(self, caller, approved, tid):
        o = self.ownerOf(tid)
        if not (caller == o or (o, caller) in self.operators): raise PermissionError("only the
owner or an operator can approve")
        self.approved[tid] = approved; self.events.append(("Approval", o, approved, tid))
    def setApprovalForAll(self, caller, operator, ok):
        (self.operators.add if ok else self.operators.discard)((caller, operator))
        self.events.append(("ApprovalForAll", caller, operator, ok))
    def transferFrom(self, caller, frm, to, tid):
        if not self._authorized(caller, tid): raise PermissionError(f"{caller} has no authority
to move {tid}")
        if self.owner[tid] != frm: raise ValueError("_from is not the current owner")
        if to == "0x0": raise ValueError("cannot send to the zero address")
        self.owner[tid] = to; self.approved.pop(tid, None)   # "the approval is reset on
transfer"
        self.events.append(("Transfer", frm, to, tid))
    def safeTransferFrom(self, caller, frm, to, tid, data=b"", contracts=None):
        self.transferFrom(caller, frm, to, tid)
        if contracts and to in contracts:                      # if the recipient is a contract,
ask whether it is ready to receive
            if contracts[to].onERC721Received(caller, frm, tid, data) != MAGIC:
                self.owner[tid] = frm; self.events.pop()       # roll back (in reality the whole
transaction reverts)
                raise ValueError(f"recipient {to} did not return the magic value, so the
transfer was rolled back")

nft = ERC721Toy()
nft.mint("Alice", 7, "https://example.invalid/meta/7.json")
print("owner of #7:", nft.ownerOf(7), "| tokenURI:", nft.tokenURI(7))
contracts = {"ReadyContract": Wallet(ready=True), "DumbContract": Wallet(ready=False)}
try:
    nft.safeTransferFrom("Alice", "Alice", "DumbContract", 7, contracts=contracts)
except ValueError as e:
    print("safeTransferFrom → contract not ready:", e, "| owner is still", nft.ownerOf(7))
nft.transferFrom("Alice", "Alice", "DumbContract", 7)       # an "unsafe" transfer sends without
asking — this is where the standard's capital-letter warning applies
print("transferFrom → same contract: owner", nft.ownerOf(7), "(locked forever if the contract
has no code to get it out)")
```

```
nft.mint("Bob", 8, "https://example.invalid/meta/8.json")
nft.setApprovalForAll("Bob", "Marketplace", True)           # Bob makes the marketplace contract
an operator
nft.transferFrom("Marketplace", "Bob", "Carol", 8)           # the operator moves Bob's token to
Carol
print("owner of #8 after the operator's transfer:", nft.ownerOf(8), "| Bob's count:",
nft.balanceOf("Bob"), "| Carol:", nft.balanceOf("Carol"))

nft.uri[8] = "https://example.invalid/meta/8-v2.json"       # the issuer changes the URI — the
owner slot stays as it is
print("after the URI change — owner:", nft.ownerOf(8), "| tokenURI:", nft.tokenURI(8), "(the
chain guarantees the left; the right can change)")
```

```
# Output — bcbook (Python 3.13), exactly as run
owner of #7: Alice | tokenURI: https://example.invalid/meta/7.json
safeTransferFrom → contract not ready: recipient DumbContract did not return the magic value, so
the transfer was rolled back | owner is still Alice
transferFrom → same contract: owner DumbContract (locked forever if the contract has no code to
get it out)
owner of #8 after the operator's transfer: Carol | Bob's count: 0 | Carol: 1
after the URI change — owner: Carol | tokenURI: https://example.invalid/meta/8-v2.json (the
chain guarantees the left; the right can change)
```

[Intermediate] Two scenes carry this chapter's point. `safeTransferFrom` recognized a contract that was not ready to receive and rolled the transfer back, but `transferFrom` sent without asking, and the token was locked in a contract with no code to get it out. And when the issuer changes the URI string, the owner slot does not budge. The chain guarantees "the owner of #8 is Carol" and does not guarantee "what #8 points to."

▾ [Advanced] Recomputing the numbers the standards wrote down: function selectors and interface identifiers

[Advanced] The text of ERC-721 says that "the ERC-165 identifier for this interface is 0x80ac58cd," that the metadata extension's is `0x5b5e139f`, and that the receipt magic value is `bytes4(keccak256("onERC721Received(address,address,uint256,bytes)"))`. Where do these numbers come from? Ethereum calls a function not by its name but by its **selector**, that is, the first 4 bytes of the Keccak-256 hash of the function's signature string. ERC-165 says: "We define the interface identifier as the XOR of all function selectors in the interface."[22] Just as Chapter 12 recomputed the hash of the genesis block and matched it to the public value, here we recompute the numbers the standards wrote down and match them.

[Advanced] There is one snag. Ethereum's Keccak-256 is **not the same** as `sha3_256` in Python's `hashlib`. The two use the same sponge function but different padding bytes (`0x01` in the original Keccak proposal, `0x06` in the NIST standard SHA-3). So the code below writes out the Keccak-f[1600] permutation directly, without a library, **verifies the implementation against itself** by checking that with padding byte `0x06` it gives the same value as `hashlib.sha3_256`, and then changes the padding byte to `0x01` to obtain Ethereum's Keccak-256. It is a study implementation and is not for real use.

```
# NB14-3: Keccak-256 by hand — a single padding byte separates SHA3-256 from Keccak-256. Then
recompute the ERC-165/721 identifiers (for study, not for real use)
import hashlib
from functools import reduce

RC = [0x0000000000000001, 0x0000000000008082, 0x800000000000808A, 0x8000000080008000,
0x000000000000808B, 0x0000000080000001,
      0x8000000080008081, 0x8000000000008009, 0x000000000000008A, 0x0000000000000088,
0x0000000080008009, 0x000000008000000A,
      0x000000008000808B, 0x800000000000008B, 0x8000000000008089, 0x8000000000008003,
0x8000000000008002, 0x8000000000000080,
      0x000000000000800A, 0x800000008000000A, 0x8000000080008081, 0x8000000000008080,
0x0000000080000001, 0x8000000080008008]
ROT = [[0, 36, 3, 41, 18], [1, 44, 10, 45, 2], [62, 6, 43, 15, 61], [28, 55, 25, 21, 56], [27,
20, 39, 8, 14]]   # rotation amounts r[x][y]
M64 = (1 << 64) - 1
def rol(v, n): return ((v << n) | (v >> (64 - n))) & M64 if n else v

def keccak_f(A):                                          # A[x][y]: 5×5 lanes of 64 bits. 24
rounds of θ·ρ·π·χ·ι
    for rc in RC:
        C = [A[x][0] ^ A[x][1] ^ A[x][2] ^ A[x][3] ^ A[x][4] for x in range(5)]
        D = [C[(x - 1) % 5] ^ rol(C[(x + 1) % 5], 1) for x in range(5)]
        A = [[A[x][y] ^ D[x] for y in range(5)] for x in range(5)]                       # θ
        B = [[0] * 5 for _ in range(5)]
        for x in range(5):
            for y in range(5):
                B[y][(2 * x + 3 * y) % 5] = rol(A[x][y], ROT[x][y])                     # ρ,
π
        A = [[B[x][y] ^ ((~B[(x + 1) % 5][y]) & B[(x + 2) % 5][y]) for y in range(5)] for x in
range(5)]   # χ
        A[0][0] ^= rc                                                                   # ι
    return A

def sponge256(msg: bytes, pad_byte: int) -> bytes:        # capacity 512 bits → rate 1088 bits
= 136 bytes, output 256 bits
    rate = 136
    padded = bytearray(msg) + bytearray([pad_byte]) + bytearray((-len(msg) - 2) % rate) +
bytearray([0x80])
    if len(msg) % rate == rate - 1:                       # when only one byte is left: fill
it with the single byte pad_byte | 0x80
        padded = bytearray(msg) + bytearray([pad_byte | 0x80])
    A = [[0] * 5 for _ in range(5)]
    for off in range(0, len(padded), rate):
        block = padded[off:off + rate]
        for i in range(rate // 8):                         # lane i → (x, y) = (i % 5, i //
5), little-endian
            A[i % 5][i // 5] ^= int.from_bytes(block[8 * i:8 * i + 8], "little")
        A = keccak_f(A)
    return b"".join(A[i % 5][i // 5].to_bytes(8, "little") for i in range(4))

def sha3_256_manual(m): return sponge256(m, 0x06)         # NIST SHA-3: padding suffix 0x06
def keccak256(m):       return sponge256(m, 0x01)         # Ethereum Keccak-256: padding
suffix 0x01

for m in [b"", b"abc", b"a" * 200]:
    print(f"{m[:12]!r:16} by-hand SHA3-256 == hashlib.sha3_256 ? {sha3_256_manual(m) ==
hashlib.sha3_256(m).digest()}")
print("keccak256(b'')    =", keccak256(b"").hex())
print("sha3_256(b'')     =", hashlib.sha3_256(b"").hexdigest(), "(only one padding byte
differs, yet the value is entirely different)")

def selector(sig: str) -> int: return int.from_bytes(keccak256(sig.encode())[:4], "big")
print()
print("transfer(address,uint256) selector =",
f"0x{selector('transfer(address,uint256)'):08x}")
print("supportsInterface(bytes4) selector =",
f"0x{selector('supportsInterface(bytes4)'):08x}", "(ERC-165 uses this value as the
identifier)")

erc721 = ["balanceOf(address)", "ownerOf(uint256)",
"safeTransferFrom(address,address,uint256,bytes)",
```

```
          "safeTransferFrom(address,address,uint256)",
"transferFrom(address,address,uint256)", "approve(address,uint256)",
          "setApprovalForAll(address,bool)", "getApproved(uint256)",
"isApprovedForAll(address,address)"]
erc721_meta = ["name()", "symbol()", "tokenURI(uint256)"]
xor = lambda sigs: reduce(lambda a, b: a ^ b, (selector(s) for s in sigs))
print(f"ERC-721 identifier (XOR of 9 functions)    = 0x{xor(erc721):08x}   standard's text:
0x80ac58cd → match {xor(erc721) == 0x80ac58cd}")
print(f"ERC-721 Metadata identifier (3 functions)  = 0x{xor(erc721_meta):08x}   standard's
text: 0x5b5e139f → match {xor(erc721_meta) == 0x5b5e139f}")
print(f"onERC721Received magic value               =
0x{selector('onERC721Received(address,address,uint256,bytes)'):08x}   standard's text:
0x150b7a02 → match {selector('onERC721Received(address,address,uint256,bytes)') ==
0x150b7a02}")
```

```
# Output — bcbook (Python 3.13), exactly as run
b''              by-hand SHA3-256 == hashlib.sha3_256 ? True
b'abc'           by-hand SHA3-256 == hashlib.sha3_256 ? True
b'aaaaaaaaaaaa'  by-hand SHA3-256 == hashlib.sha3_256 ? True
keccak256(b'')    = c5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470
sha3_256(b'')     = a7ffc6f8bf1ed76651c14756a061d662f580ff4de43b49fa82d80a4b80f8434a (only one
padding byte differs, yet the value is entirely different)

transfer(address,uint256) selector = 0xa9059cbb
supportsInterface(bytes4) selector = 0x01ffc9a7 (ERC-165 uses this value as the identifier)
ERC-721 identifier (XOR of 9 functions)    = 0x80ac58cd   standard's text: 0x80ac58cd → match
True
ERC-721 Metadata identifier (3 functions)  = 0x5b5e139f   standard's text: 0x5b5e139f → match
True
onERC721Received magic value               = 0x150b7a02   standard's text: 0x150b7a02 → match
True
```

[Advanced] All three numbers written in the standard documents are recomputed. The third value is the one the original gives as the ERC-165 identifier of the receiver interface `ERC721TokenReceiver`, while the magic value itself is written only as the expression `bytes4(keccak256(…))`. That interface has only one function, so the two values are the same. "Supporting an interface" comes down to knowing these 4-byte numbers: a wallet asks a contract it has never seen `supportsInterface(0x80ac58cd)`, and if true comes back, it believes that "the ERC-721 functions are there." That means the functions **exist**, not that they **behave as expected**. As Section 4 showed, the standard also allows implementations that block transfers.

7. Summary

A token is one mapping in a contract's storage, and a standard is an agreement on the names of the functions that handle that mapping. ERC-20 handles "address → quantity" and ERC-721 handles "number → owner," and neither decides issuance, burning, or meaning. The standard itself says that an NFT's "picture" is off the chain and that its address can change. The same six lines can be a share of stock, a coupon, or a score, and which one it is gets decided outside the code. The next chapter, Chapter 15, looks at contracts that swap these tokens without people. The way those contracts receive tokens is exactly this chapter's `approve` and `transferFrom`. What this chapter adds to misconceptions ② ("Smart contracts are contracts") and ⑫ ("If it is decentralized, no one is responsible") in Appendix B is the fact that even the authors of the standards left "who issues it and what it means" outside the standards.

Sources for this chapter

- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2014 (first draft 2013). https://ethereum.org/whitepaper/ (official PDF, accessed 2026-10-25) — definition and uses of token systems ("Token Systems," PDF pp. 19–20), the role of ether ("Currency And Issuance," p. 30).
- Fabian Vogelsteller & Vitalik Buterin, "ERC-20: Token Standard," EIP-20, Final, created 2015-11-19. https://eips.ethereum.org/EIPS/eip-20 ; permanent link github.com/ethereum/ERCs commit 8358b94b (accessed 2026-10-25) — motivation, specification (functions, events, OPTIONAL markings, the approve warning).
- William Entriken, Dieter Shirley, Jacob Evans, Nastassia Sachs, "ERC-721: Non-Fungible Token Standard," EIP-721, Final, created 2018-01-24, requires 165. https://eips.ethereum.org/EIPS/eip-721 ; permanent link github.com/ethereum/ERCs commit 49e372bc (accessed 2026-10-25) — motivation, specification (interface comments, metadata extension, JSON schema), rationale (identifiers, transfer mechanism, metadata choices).
- Christian Reitwießner et al., "ERC-165: Standard Interface Detection," EIP-165, Final, created 2018-01-23. https://eips.ethereum.org/EIPS/eip-165 ; permanent link github.com/ethereum/ERCs commit 8dd085d1 (accessed 2026-10-25) — definition of the interface identifier.
- The Solidity Authors, *Solidity Documentation* v0.8.38, "Introduction to Smart Contracts — Subcurrency Example." https://docs.soliditylang.org/en/v0.8.38/introduction-to-smart-contracts.html (accessed 2026-10-25; checked against the v0.8.37 version) — the descriptive sentences of the Coin example and the description of mappings. The example code is not reproduced.
- Hayden Adams, Noah Zinsmeister, Dan Robinson, "Uniswap v2 Core," March 2020, §3.3 — the existence of non-standard ERC-20 implementations.
- U.S. Securities and Exchange Commission, *Report of Investigation Pursuant to Section 21(a)*

of the Securities Exchange Act of 1934: The DAO, Release No. 81207, July 25, 2017 — description of the rights attached to DAO tokens (pp. 5–7), the conclusion and the statement that code does not take conduct outside the securities laws (pp. 1–2).

- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library modules `hashlib` and `functools`. Keccak-256 was implemented by hand for study and checked against `hashlib.sha3_256`. Notebook NB14.

Notes

1. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," official ethereum.org PDF (2014; first draft 2013), "Token Systems" section (PDF p. 19). Emphasis added. The original's "(i)" and "(2)," "X had," and the typo "systen" are reproduced as they stand. The Serpent code on p. 20: `from = msg.sender / to = msg.data[0] / value = msg.data[1] / if contract.storage[from] >= value: contract.storage[from] = contract.storage[from] - value / contract.storage[to] = contract.storage[to] + value`. https://ethereum.org/whitepaper/ (accessed 2026-10-25).

2. Same section, p. 19: "On-blockchain token systems have many applications ranging from sub-currencies representing assets such as USD or gold to company stocks, individual tokens representing smart property, secure unforgeable coupons, and even token systems with no ties to conventional value at all, used as point systems for incentivization."

3. Fabian Vogelsteller & Vitalik Buterin, "ERC-20: Token Standard," EIP-20 (Standards Track / ERC, status: Final, created 2015-11-19), "Motivation." https://eips.ethereum.org/EIPS/eip-20 ; permanent link https://github.com/ethereum/ERCs/blob/8358b94b5ed4009473cb4c3040d9bba60e7bb786/ERCS/erc-20.md (accessed 2026-10-25; checked again against the original).

4. William Entriken, Dieter Shirley, Jacob Evans, Nastassia Sachs, "ERC-721: Non-Fungible Token Standard," EIP-721 (Standards Track / ERC, status: Final, created 2018-01-24, requires: 165), "Motivation": "This standard is inspired by the ERC-20 token standard and builds on two years of experience since EIP-20 was created. EIP-20 is insufficient for tracking NFTs because each asset is distinct (non-fungible) whereas each of a quantity of tokens is identical (fungible)." https://eips.ethereum.org/EIPS/eip-721 ; permanent link https://github.com/ethereum/ERCs/blob/49e372bc8e5fab0ba2f7e60da2f0c2895ea5b119/ERCS/erc-721.md (accessed 2026-10-25; checked again against the original).

5. Buterin (2014), "Currency And Issuance" section (PDF p. 30): "The Ethereum network includes its own built-in currency, ether, which serves the dual purpose of providing a primary liquidity layer to allow for efficient exchange between various types of digital assets and, more importantly, of providing a mechanism for paying transaction fees." The issuance plan in the same section (the sale price and so on) was the plan of 2014, and this book does not reproduce it.

6. The Solidity Authors, *Solidity Documentation* v0.8.38, "Introduction to Smart Contracts — Subcurrency Example." The example code (`contract Coin`) is distributed under GPL-3.0, so this book does not reproduce it. The original code can be seen at the address below. https://docs.soliditylang.org/en/v0.8.38/introduction-to-smart-contracts.html (accessed 2026-10-25; checked to be identical to the v0.8.37 version).

7. Same documentation, same section.

8. EIP-20, "Specification." Excerpts from the original: "Callers MUST handle `false` from `returns (bool success)`. Callers MUST NOT assume that `false` is never returned!"; for each of name, symbol, and decimals, "OPTIONAL - This method can be used to improve usability, but interfaces and other contracts MUST NOT expect these values to be present."; transfer: "Transfers `_value` amount of tokens to address `_to`, and MUST fire the `Transfer` event. The function SHOULD `throw` if the message caller's account balance does not have enough tokens to spend. *Note* Transfers of 0 values MUST be treated as normal transfers and fire the `Transfer` event."; transferFrom: "The `transferFrom` method is used for a withdraw workflow, allowing contracts to transfer tokens on your behalf. This can be used for example to allow a contract to transfer tokens on your behalf and/or to charge fees in sub-currencies. The function SHOULD `throw` unless the `_from` account has deliberately authorized the sender of the message via some mechanism."; approve: "Allows `_spender` to withdraw from your account multiple times, up to the `_value` amount. If this function

is called again it overwrites the current allowance with `_value`."; Transfer event: "MUST trigger when tokens are transferred, including zero value transfers. A token contract which creates new tokens SHOULD trigger a Transfer event with the `_from` address set to `0x0` when tokens are created."

9. EIP-20, "approve": "**NOTE**: To prevent attack vectors like the one described here and discussed here, clients SHOULD make sure to create user interfaces in such a way that they set the allowance first to `0` before setting it to another value for the same spender. THOUGH The contract itself shouldn't enforce it, to allow backwards compatibility with contracts deployed before". In the original, "described here" and "discussed here" are links to an outside document and to a GitHub discussion, and this book has not obtained those documents. The structure of the attack is this book's reconstruction in NB14-1 below.

10. Hayden Adams, Noah Zinsmeister, Dan Robinson, "Uniswap v2 Core," March 2020, §3.3, Handling non-standard and unusual tokens (pp. 7–8). The token names the original gives as examples are omitted under this book's rules.

11. EIP-721, "Rationale — NFT Identifiers": "Every NFT is identified by a unique `uint256` ID inside the ERC-721 smart contract. This identifying number SHALL NOT change for the life of the contract. The pair `(contract address, uint256 tokenId)` will then be a globally unique and fully-qualified identifier for a specific asset on an Ethereum chain. ... callers SHALL NOT assume that ID numbers have any specific pattern to them, and MUST treat the ID as a 'black box'. Also note that NFTs MAY become invalid (be destroyed)." Emphasis added.

12. EIP-721, interface comments in "Specification." transferFrom: "Transfer ownership of an NFT -- THE CALLER IS RESPONSIBLE TO CONFIRM THAT `_to` IS CAPABLE OF RECEIVING NFTS OR ELSE THEY MAY BE PERMANENTLY LOST"; safeTransferFrom: "When transfer is complete, this function checks if `_to` is a smart contract (code size > 0). If so, it calls `onERC721Received` on `_to` and throws if the return value is not `bytes4(keccak256("onERC721Received(address,address,uint256,bytes)"))`." Both functions: "Throws unless `msg.sender` is the current owner, an authorized operator, or the approved address for this NFT. Throws if `_from` is not the current owner. Throws if `_to` is the zero address. Throws if `_tokenId` is not a valid NFT." Emphasis added.

13. EIP-721, the setApprovalForAll comment in "Specification," and "Rationale — Transfer Mechanism": "Enable or disable approval for a third party ('operator') to manage all of `msg.sender`'s assets"; "Additionally, an authorized operator may set the approved address for an NFT. This provides a powerful set of tools for wallet, broker and auction applications to quickly use a *large* number of NFTs." Emphasis on "all" added; the italics on "large" are in the original.

14. EIP-721, "Rationale — Transfer Mechanism": "In ERC-721, there is no allowance because every NFT is unique, the quantity is none or one. Therefore we receive the benefits of ERC-20's original design without problems that have been later discovered."

15. Same section.

16. Same section: "The transfer and accept functions' documentation only specify conditions when the transaction MUST throw. Your implementation MAY also throw in other situations. This allows implementations to achieve interesting results: Disallow transfers if the contract is paused ... Blocklist certain address from receiving NFTs ... Disallow unsafe transfers ... Charge a fee to both parties of a transaction ... Read only NFT registry — always throw from `safeTransferFrom`, `transferFrom`, `approve` and `setApprovalForAll`".

17. EIP-721, the ERC721Metadata interface and JSON schema in "Specification": "The **metadata extension** is OPTIONAL for ERC-721 smart contracts (see 'caveats', below). This allows your smart contract to be interrogated for its name and for details about the assets which your NFTs represent."; tokenURI: "A distinct Uniform Resource Identifier (URI) for a given asset. ... The URI may point to a JSON file that conforms to the 'ERC721 Metadata JSON Schema'."; image in the schema: "A URI pointing to a resource with mime type image/* representing the asset to which this NFT represents."

18. EIP-721, "Rationale — Metadata Choices": "*Alternatives considered: put all metadata for each asset on the blockchain (too expensive), use URL templates to query metadata parts (URL templates do not work with all URL schemes, especially P2P URLs), multiaddr network address (not mature enough)*".

19. Same section. Emphasis on "MAY be mutable" added; the italics on "your" are in the original.

20. U.S. Securities and Exchange Commission, *Report of Investigation Pursuant to Section 21(a) of the Securities*

Exchange Act of 1934: The DAO, Exchange Act Release No. 81207 (July 25, 2017), pp. 5–7. The first quotation is on p. 5. The others: "The various promotional materials disseminated by Slock.it's co-founders touted that DAO Token holders would receive 'rewards,' which the White Paper defined as, 'any [ETH] received by a DAO [Entity] generated from projects the DAO [Entity] funded.'" (p. 6); "DAO Token holders were to cast votes, which would be weighted by the number of tokens they controlled, for or against the funding of a specific proposal." (p. 7). Emphasis added. Footnote 2 on p. 2 of the report states: "This Report does not constitute an adjudication of any fact or issue addressed herein, nor does it make any findings of violations by any individual or entity."

21. Same report, pp. 1 and 2. The first quotation comes from: "Based on the investigation, and under the facts presented, the Commission has determined that DAO Tokens are securities under the Securities Act of 1933 ('Securities Act') and the Securities Exchange Act of 1934 ('Exchange Act')." (p. 1). The second quotation is on p. 2. The attack (2016-06-17, movement of about 1/3 of the funds raised) and the hard fork (2016-07-20) are on p. 9, and are treated in Chapter 26.

22. Christian Reitwießner et al., "ERC-165: Standard Interface Detection," EIP-165 (status: Final, created 2018-01-23), "How Interfaces are Identified": "For this standard, an *interface* is a set of function selectors as defined by the Ethereum ABI. … We define the interface identifier as the XOR of all function selectors in the interface." https://eips.ethereum.org/EIPS/eip-165 ; permanent link https://github.com/ethereum/ERCs/blob/8dd085d159cb123f545c272c0d871a5339550e79/ERCS/erc-165.md (accessed 2026-10-25). The identifiers for ERC-721 are given in the interface comments of EIP-721, "Specification": "Note: the ERC-165 identifier for this interface is 0x80ac58cd." / "0x5b5e139f" / "0x150b7a02".

PART 2 · SMART CONTRACTS AND APPLICATIONS

Chapter 15. Decentralized exchanges and AMMs: price formation without an order book

KEY TAKEAWAY

On an exchange, people call out prices. Buyers and sellers post quotes, and an operator matches them. An automated market maker (AMM) has neither quotes nor an operator. One contract keeps two tokens in a pool and accepts every trade by a single rule: "the product of the two quantities does not decrease." The price then becomes **the ratio of the two quantities in the pool**, and a large order pushes the price up by itself (slippage). The people who put tokens into the pool (liquidity providers) receive a fee on every trade, but when the relative price of the two tokens moves, they end up with less than if they had "simply held" the two tokens (impermanent loss). The two effects are the two pans of the same scale. After this chapter you will have checked with formulas that the operator, the counterparty, and the quotes that the word "exchange" presupposes are not present here, and you will see why the question in Part 5, "whom do we regulate?", looks empty.

Learning objectives

1. Contrast price formation on an order-book exchange with price formation in a constant-product AMM, and say what "price" in an AMM is a ratio of.
2. **[Intermediate]** From the constant product $x \cdot y = k$, derive the amount received Δy and the execution price for an order of size Δx, and see where the fee is taken out.
3. **[Intermediate]** Derive the liquidity provider's share calculation, the accrual of fees, and impermanent loss $\frac{2\sqrt{r}}{1+r} - 1$, and read impermanent loss **symmetrically with the fee income**.
4. **[Advanced]** See how concentrated liquidity (supply within a price range) cuts the constant-product curve into pieces and uses one of them, and why an AMM spot price gets manipulated when it is used as an oracle.

What a reader of the [Basic] paragraphs alone takes away. In an AMM the price is not called out by people. It is **the ratio of the two quantities in the pool**. A large order pushes the price up by itself.

1. What failure this technology grew out of

A traditional exchange runs on an **order book**. The 2020 paper by Philip Daian and his coauthors describes it neatly: "Classic exchanges ... generally share an accepted and common exchange design known as a continuous-limit orderbook. Such an orderbook consists of a list of all open offers from buyers and sellers in the system. Prospective buyers place a limit buy order, which specifies a maximum price at which they are willing to buy an asset; sellers correspondingly place limit sell orders. **A centralized counterparty, the exchange operator,** matches buyers and sellers, completing transactions automatically when there is a sell order on the books with a lower price than a buy order on the books."[1] Moving this structure onto a blockchain as it is runs into two problems. First, "on-chain, smart-contract-mediated trades are **slow**." Every step of placing and canceling an order has to wait for a block, and in the meantime orders that have already been filled or canceled look as if they were still live.[2] Second, there is the question of who holds the order book. In the decentralized exchanges the paper examined, "Order books are typically maintained off-chain," and so the paper writes that "'Decentralized' exchange is something of a misnomer, as many such systems have centralized components; ... could more accurately be classified as **non-custodial**: users trade without surrendering control of their funds to a third party in the process."[3]

The other design that followed was to **do away with** the order book. The same paper describes it: "A more radical DEX design, called an automated market maker [3], bypasses order books altogether. The DEX consists of **a smart contract that itself holds a reserve of tokens and/or Ether**. Consider two assets A and B. The contract allows a user to trade between A and B at any time using its reserves as a counterparty, at a set rate. If a user buys A using B, the price of A denominated in B offered by the smart contract for the next trade is increased. If a user instead sells A for B, the price decreases. In this way, single parties can trade without counterparty discovery or matching. Consequently, **constant arbitrage between these and other exchanges is required** to keep the rate offered in lockstep with the market rate for a commodity."[4] Keep this last sentence in mind. An AMM does not "know" the market price. It offers a wrong price until arbitrageurs come from outside and correct it. That is the seed of Chapter 16 and Chapter 17.

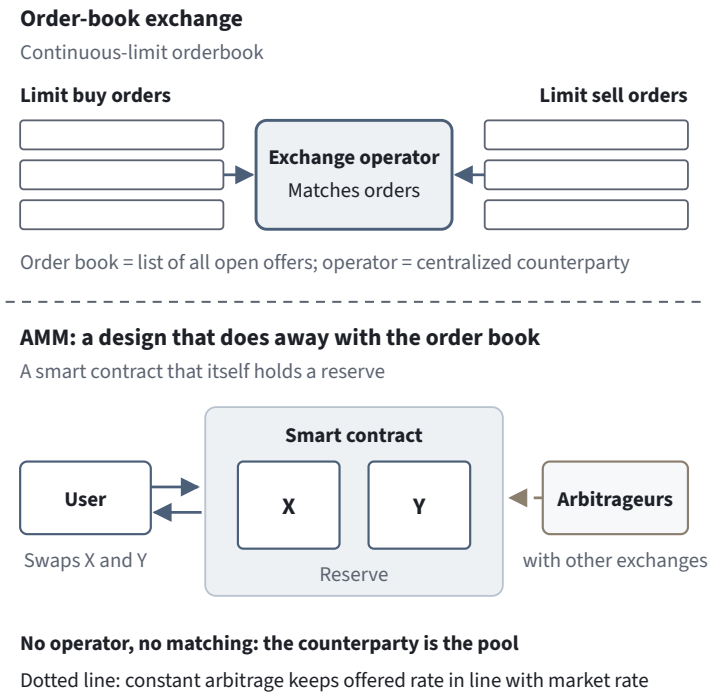


Figure 15-1. The structure of an order-book exchange and of an AMM. An AMM has neither an order book nor an operator, and the job of bringing the offered rate in line with the market rate is done by arbitrageurs outside.

2. Intuition: a pool of two tokens and one product

A pool holds x units of token X and y units of token Y. There is one rule. **After a trade, $x \cdot y$ cannot be smaller than $x \cdot y$ before the trade.** This is the "constant product" rule, the formula used by Uniswap v1 and v2. In the words of the v2 white paper: "Each Uniswap v1 pair stores pooled reserves of two assets, and provides liquidity for those two assets, maintaining **the invariant that the product of the reserves cannot decrease**. Traders pay a 30-basis-point fee on trades, which goes to liquidity providers."[5]

Say you put in Δx of X and want to take Y out. The pool's X becomes $x + \Delta x$, and to keep the product, only k / $(x + \Delta x)$ of Y may remain. So the Y you receive is the difference. As a numerical example, if you put 100 X into a pool containing 1,000 X and 1,000 Y ($k = 1{,}000{,}000$), 1,000,000 / 1,100 ≈ 909.1 Y has to remain, so you receive about 90.9 Y. You put in 100 X and received 90.9 Y, so the execution price is 0.909 Y per X. But the "price" **just before** the trade was y / $x = 1$. The execution price was more than 9% worse than the price just before the trade. This is **slippage**, and it grows the larger the order is relative to the pool. The ratio after the trade is 909.1 / 1,100 ≈ 0.826. The next person starts at 0.826 Y per X. No one "set" the price. Only the ratio changed.

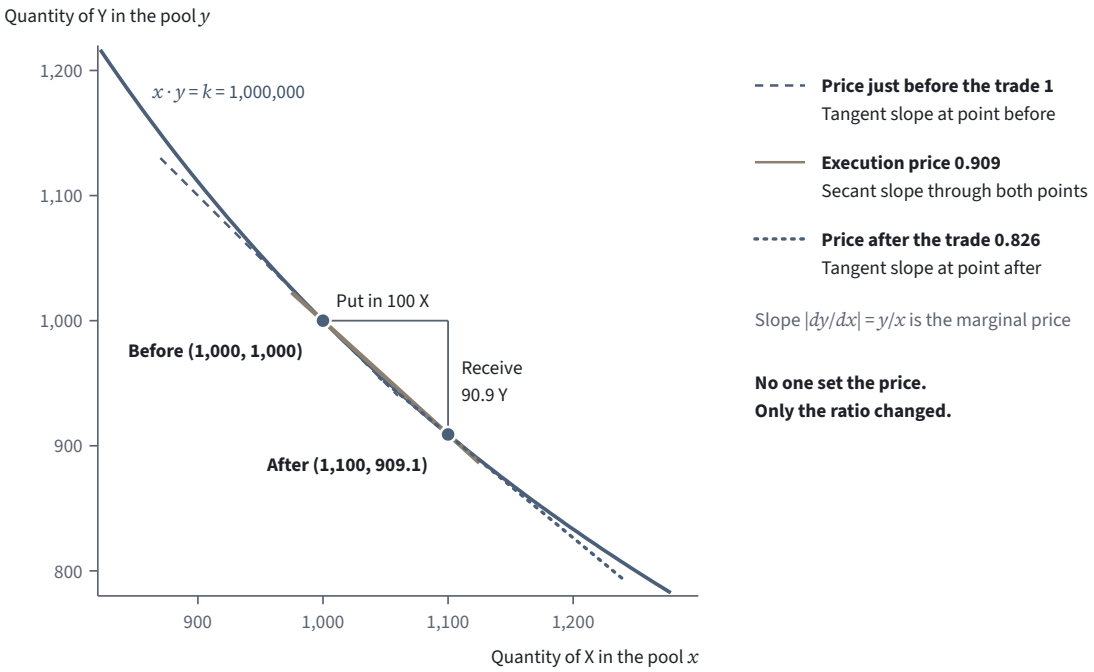


Figure 15-2. A trade that puts 100 X into a pool containing 1,000 X and 1,000 Y. The product stays the same and only the ratio changes. The curve is the graph of the formula $xy = k$ in the text, and the slopes of the three lines are the price just before the trade, 1; the execution price, 0.909; and the price after the trade, 0.826.

Look again at the absence of people here. Your counterparty is the pool. The people who put tokens into the pool (liquidity providers) collectively act as the counterparty, but they neither know about your trade nor consent to it. There is no operator either. The v2 white paper writes that "the contract is not generally upgradeable," yet discloses that "there is a private key that has the ability to update a variable on the factory contract to turn on an on-chain 5-basis-point fee on trades."[6] There is no operator, but there is **a key that controls a switch**. These are the sentences to look for when reading how far the word "decentralized" reaches.

> **? A LAWYER'S QUESTION**
>
> Exchange regulation usually presupposes an "operator": **someone** who registers (등록), conducts listing review (상장 심사), monitors for market manipulation (시세조종), and keeps customer assets segregated (고객 자산 분리 보관). Where is that someone in a constant-product AMM? Is it the person who deployed the code, the thousands of liquidity providers who put tokens into the pool, the address with the key to the fee switch, the majority of governance-token holders, or no one at all? When you traded with the pool, who was the "counterparty (상대방)," and if the price was wrong, who got it wrong? → Chapter 27 · Chapter 28

3. Formulas: constant product, fee, execution price

[Intermediate] Set out the symbols. x, y are the reserves before the trade, $k = xy$, Δx is the amount put in, and Δy is the amount received. Without a fee,

$$(x + \Delta x)(y - \Delta y) = k \quad \Longrightarrow \quad \Delta y = y - \frac{k}{x + \Delta x} = \frac{y\,\Delta x}{x + \Delta x}.$$

Reading the symbols: the numerator is "Y in proportion to what was put in," and the denominator is "X after putting it in." A numerical example: with $x = y = 1{,}000$ and $\Delta x = 100$, $\Delta y = 1{,}000 \times 100 \,/\, 1{,}100 \approx 90.91$. The execution price is $\Delta y \,/\, \Delta x \approx 0.909$, the marginal price just before the trade is $y \,/\, x = 1$, and the marginal price after the trade is $(y - \Delta y) \,/\, (x + \Delta x) \approx 0.826$.

[Intermediate] Marginal price. The v2 white paper writes: "The marginal price offered by Uniswap (not including fees) at time t can be computed by dividing the reserves of asset a by the reserves of asset b." $p_t = r_t^a \,/\, r_t^b$.[7] Mathematically, it is the slope of the curve $y = k \,/\, x$, $|dy \,/\, dx| = y \,/\, x$. It is the price of "a very small trade," and an actual trade gets a worse price than that. The difference is slippage. Written as a fraction of the marginal price, slippage is

$$\text{slippage} = 1 - \frac{\Delta y \,/\, \Delta x}{y \,/\, x} = 1 - \frac{x}{x + \Delta x} = \frac{\Delta x}{x + \Delta x}.$$

It is set only by the ratio of the order size to the pool size. Put 100 into a pool of 1,000 and it is $100 \,/\, 1{,}100 \approx 9.1\%$; put in 10 and it is $10 \,/\, 1{,}010 \approx 1.0\%$; put in 1,000 and it is 50%. The larger the pool, the smaller the slippage on the same order. That is the whole of why liquidity providers are needed.

[Intermediate] Fee. v1 and v2 take 0.3% off the amount put in and keep the product with the rest. The white paper writes, "Uniswap v1's trading fee is applied by reducing the amount paid into the contract by 0.3% before enforcing the constant-product invariant. The contract implicitly enforces the following formula:" $(x_1 - 0.003 \cdot x_{in}) \cdot y_1 \geq x_0 \cdot y_0$, and since in v2 both assets can come in at once, it generalizes this to $(x_1 - 0.003 \cdot x_{in})(y_1 - 0.003 \cdot y_{in}) \geq x_0 y_0$.[8] Reading the symbols: subscript 0 means before the trade, 1 means after it, and x_{in} is the X that came in. With the fee $\gamma = 0.003$ taken out, the amount received is

$$\Delta y = \frac{y\,(1 - \gamma)\,\Delta x}{x + (1 - \gamma)\,\Delta x}$$

A numerical example: put 100 into the same pool and $(1 - \gamma)\Delta x = 99.7$, $\Delta y = 1{,}000 \times 99.7 \,/\, 1{,}099.7 \approx 90.66$. You receive 0.25 fewer than the 90.91 you would receive without a fee. Where did those 0.3 X (= 100 × 0.003) go? **They stay in the pool.** After the trade the pool holds 1,100 X and 909.34 Y, and their product is about 1,000,273, **larger** than before the trade. The fee is not sent off anywhere separately. It accumulates inside the pool in the form of a larger k, and the liquidity providers later take out the enlarged pool in proportion to their shares. This is the key to the next section.

4. Supplying liquidity: fee income and impermanent loss are the two pans of one scale

Why does anyone put tokens into a pool? Because the fee taken on every trade accumulates in the pool. Is it free, then? No. When the relative price of the two tokens moves, the pool rebalances automatically, **selling the token that rose and buying the token that fell**, because arbitrageurs come and correct the ratio. As a result the provider ends up with less than if they had "simply kept the two tokens," and this difference is conventionally called **impermanent loss**. The v2 white paper uses the term too. In v1, ether was part of every pair, so "All liquidity providers have exposure to ETH, and suffer impermanent loss based on changes in the prices of other assets relative to ETH."[9] The name "impermanent" means that the loss disappears if the price goes back to the original ratio, not that the loss is light. This book **does not call supplying liquidity "profit."** It puts fee income and impermanent loss in the same table, and it leaves which of the two is larger as **an empirical question** that depends on trading volume and price movement.

[Intermediate] Shares. A provider receives "liquidity tokens (shares)" in proportion to what it put in. The first depositor receives $s = \sqrt{x_{deposited}\, y_{deposited}}$, and later depositors receive in proportion to the existing reserves: $s_{minted} = \frac{x_{deposited}}{x_{starting}} s_{starting}$.[10] On withdrawal, a provider receives both tokens in proportion to its share, **at the ratio of that moment**. So if trading fees have made k larger, the same share gets more, and if the price has moved, the mix of the two tokens is different. The white paper measures the accrued fees as "the growth in $\sqrt{k}$." The fees accumulated between t_1 and t_2, as a fraction of the pool at t_2, are $f_{1,2} = 1 - \sqrt{k_1} / \sqrt{k_2}$.[11]

[Intermediate] The formula for impermanent loss. Suppose the relative price moves from p to rp (r is the price multiple). Assuming no fee and that arbitrage fully corrects the ratio, the two quantities in the pool become $x' = x / \sqrt{r}$ and $y' = y\sqrt{r}$ (the product is unchanged, and the ratio is $y' / x' = r\,y / x$). Measuring value in units of Y, the supplying side is $V_{pool} = x' \cdot rp + y' = 2y\sqrt{r}$, and the side that simply holds the tokens is $V_{hold} = x \cdot rp + y = y(1 + r)$, so

$$\frac{V_{pool}}{V_{hold}} = \frac{2\sqrt{r}}{1+r}, \qquad \text{impermanent loss} = \frac{2\sqrt{r}}{1+r} - 1 \le 0\,.$$

Reading the symbols: at $r = 1$ it is 0, and the farther r moves from 1 (in either direction), the larger the negative value. A numerical example: at $r = 2$, $2\sqrt{2} / 3 - 1 \approx -5.7\%$; at $r = 4$, $4 / 5 - 1 = -20\%$; and $r = 1 / 4$ also gives -20%. This formula is this book's derivation, and the derivation is in the **[Advanced]** box below and in NB15-2. What matters is the shape. **It is a loss whichever the direction, it is larger the larger the move, and whether the fees make up for it is a separate number.** The code below puts the two numbers in one table.

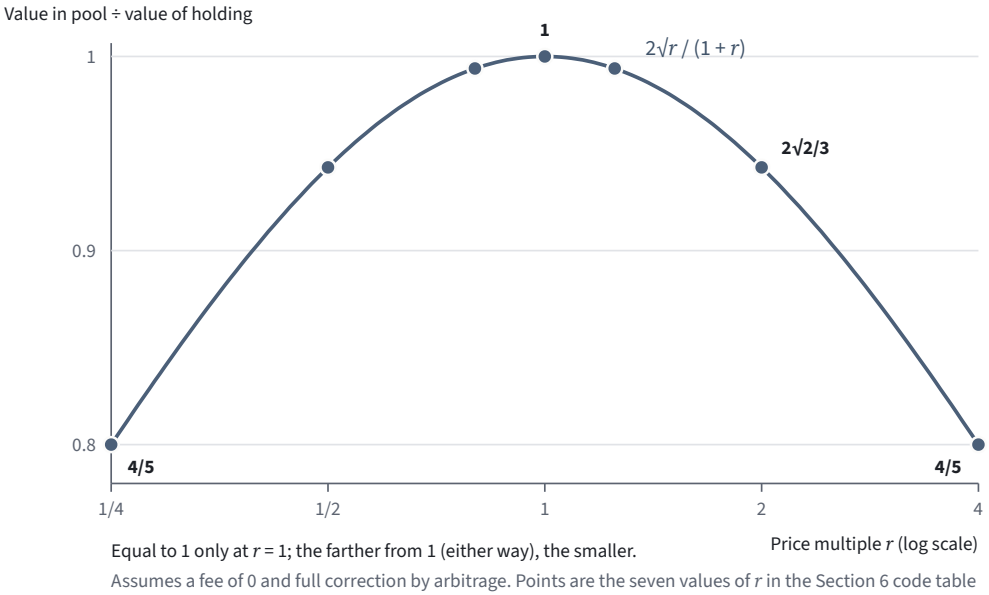


Figure 15-3. For each price multiple r, the value deposited in the pool as a multiple of the value of simply holding the tokens. The curve is the graph of the formula $2\sqrt{r} \,/\, (1+r)$ in the text, and the horizontal axis is the multiple, not time. Assumes a fee of 0 and full correction by arbitrage.

▾ [Advanced] Deriving impermanent loss, and concentrated liquidity: cutting up the curve

[Advanced] Derivation. Reserves (x, y), $k = xy$, and the relative price before the trade $p = y \,/\, x$ (the price of one X in Y). The outside price has become rp. Arbitrageurs trade until the pool's marginal price $y' \,/\, x'$ equals rp. Solving the two conditions $x'y' = k$ and $y' \,/\, x' = rp = ry \,/\, x$ gives $x'^2 = k \,/\, (rp) = x^2 \,/\, r$, that is, $x' = x \,/\, \sqrt{r}$ and $y' = y\sqrt{r}$. The pool's value in Y is $x' \cdot rp + y' = (x \,/\, \sqrt{r})(ry \,/\, x) + y\sqrt{r} = 2y\sqrt{r}$. The value of holding the tokens is $x \cdot rp + y = ry + y$. The ratio is $2\sqrt{r} \,/\, (1+r)$, which is at most 1 by the inequality of arithmetic and geometric means, $\frac{1+r}{2} \geq \sqrt{r}$. Equality holds only at $r = 1$. With fees, k grows a little with every trade, and the pool's value increases by that much. So the provider's final net result is set by the product of "$\sqrt{k}$ grown by fees" and "$2\sqrt{r} \,/\, (1+r)$."

[Advanced] Concentrated liquidity. The constant-product curve spreads liquidity over the whole price range $(0, \infty)$. The v3 white paper writes that this "is simple to implement and allows liquidity to be efficiently aggregated, but means that much of the assets held in a pool are **never touched**."[12] So v3 lets a provider choose a price range $[p_a, p_b]$ and place liquidity only in that range. "A position only needs to maintain enough reserves to support trading within its range, and therefore can act like **a constant product pool with larger reserves (we call these the virtual reserves)** within that range." Measuring the size of liquidity as $L = \sqrt{k}$, the real reserves follow the curve[13]

$$\left(x + \frac{L}{\sqrt{p_b}}\right)\left(y + L\sqrt{p_a}\right) = L^2$$

Reading the symbols: the added terms in the parentheses are the "virtual" part. When

the price reaches p_b, X runs out, and when it reaches p_a, Y runs out. "When the price exits a position's range, the position's liquidity is no longer active, and no longer earns fees. At that point, its liquidity is composed entirely of a single asset …."[14] A numerical example: with a current price of 1, the range $[0.5, 2]$, and $L = 1{,}000$, the virtual reserves are $x_v = L / \sqrt{P} = 1{,}000$ and $y_v = L\sqrt{P} = 1{,}000$, but the amounts that actually have to be put in are $x = L / \sqrt{P} - L / \sqrt{p_b} = 1{,}000 - 707.1 \approx 292.9$ and $y = L\sqrt{P} - L\sqrt{p_a} = 1{,}000 - 707.1 \approx 292.9$. Putting in only 29% of a full-range pool, the position provides the same slippage within the same range. The cost is that once the price leaves the range, the provider is left with only one asset. The statement that this realizes impermanent loss more steeply than a full-range pool is this book's explanation; the original goes only as far as "inactive, no fees, a single asset." The fee was also split into three tiers (0.05%, 0.30%, 1%), so that each pair can have several pools.[15] This book does not say which design is better. One uses the whole curve, and the other cuts it up.

5. A spot price is not an oracle: the bridge to Chapters 16 and 17

The ratio in an AMM looks like a "price" anyone can read. So other contracts are tempted to take it as price information. The v2 white paper shows why that is dangerous with a single attack: "Suppose some other contract uses the current ETH-DAI price to settle a derivative. An attacker who wishes to manipulate the measured price can buy ETH from the ETH-DAI pair, trigger settlement on the derivative contract (causing it to settle based on the inflated price), and then sell ETH back to the pair to trade it back to the true price. This might even be done as **an atomic transaction**, or by a miner who controls the ordering of transactions within a block."[16] As Section 2 showed, a large order pushes the ratio. If some contract reads that pushed ratio as the "price," then push → be read → push back can be bundled into a single transaction. In a footnote, the white paper points to one outside article as "a real-world example of how using Uniswap v1 as an oracle can make a contract vulnerable to such an attack."[17] This book has not obtained that article, so it does not describe the case.

v2's response is **a time-weighted average**. It accumulates the price just before the first trade in each block, and dividing the difference between the accumulated values at two points in time by the elapsed time gives the average price over that interval. "Choosing a longer period makes it more expensive for an attacker to manipulate the TWAP, although it results in a less up-to-date price."[18] It is a trade-off between the cost of manipulation and freshness. Even so, the white paper admits that a miner (or an attacker who uses enough gas to fill an entire block) can manipulate the price at the end of a block, and adds that "unless they mine the next block as well, they may not have a particular advantage in arbitraging the trade back." The larger problem of "bringing facts from off the chain onto the chain" is taken up by Chapter 16, and "the person who decides the order of transactions" by Chapter 17.

6. Code: a constant-product pool, the slippage curve, and the two pans of the scale

Constant-product pool and slippage

[Intermediate] We build one pool class. The sandwich simulation in Chapter 17 reuses this class as it is. All units are **numbers of tokens in the pool**, and there is no currency unit.

```
# NB15-1: constant-product pool — swaps (with fee), adding and removing liquidity, the slippage
curve (for study)
import math

class ConstantProductPool:
    """x·y = k pool. fee is a fraction charged on the amount put in (default 0.003 = 30 bp).
Shares are sqrt(x·y) at first, proportional after that."""
    def __init__(self, x: float, y: float, fee: float = 0.003):
        self.x, self.y, self.fee = float(x), float(y), fee
        self.shares = {"LP0": math.sqrt(x * y)}            # the first provider's share =
geometric mean (v2 equation (13))
    @property
    def k(self): return self.x * self.y
    @property
    def price_x_in_y(self): return self.y / self.x           # marginal price: the price of one
X in Y (v2 equation (1))
    def quote_x_for_y(self, dx: float) -> float:              # put in dx of X: how much Y comes
back (keeps the product after taking the fee)
        eff = dx * (1 - self.fee)
        return self.y * eff / (self.x + eff)
    def swap_x_for_y(self, dx: float) -> float:
        dy = self.quote_x_for_y(dx)
        self.x += dx; self.y -= dy                             # the fee portion stays in the
pool and k grows
        return dy
    def swap_y_for_x(self, dy: float) -> float:
        eff = dy * (1 - self.fee)
        dx = self.x * eff / (self.y + eff)
        self.y += dy; self.x -= dx
        return dx
    def add_liquidity(self, who: str, dx: float) -> float:     # put in X dx and Y dx·(y/x) at
the current ratio (v2 equation (12))
        dy = dx * self.y / self.x
        minted = dx / self.x * sum(self.shares.values())
        self.x += dx; self.y += dy
        self.shares[who] = self.shares.get(who, 0) + minted
        return minted
    def remove_liquidity(self, who: str):                      # returns both tokens in
proportion to the share, at the ratio of that moment
        s_total = sum(self.shares.values())
        frac = self.shares.pop(who) / s_total
        out_x, out_y = self.x * frac, self.y * frac
        self.x -= out_x; self.y -= out_y
        return out_x, out_y

pool = ConstantProductPool(1000, 1000)
print(f"start: x={pool.x:.1f} y={pool.y:.1f} k={pool.k:,.0f} marginal price (price of one X in
Y)={pool.price_x_in_y:.4f}")
print(f"{'order Δx':>8} {'received Δy':>12} {'exec price Δy/Δx':>17} {'slippage':>9} {'marginal
price after':>21}")
for dx in [1, 10, 100, 500, 1000]:
    p = ConstantProductPool(1000, 1000)
    before = p.price_x_in_y
    dy = p.swap_x_for_y(dx)
    exec_price = dy / dx
    print(f"{dx:>8} {dy:>12.3f} {exec_price:>17.4f} {1 - exec_price / before:>9.2%}
{p.price_x_in_y:>21.4f}")

# where did the fee go — look at k after putting in 100
p = ConstantProductPool(1000, 1000); dy = p.swap_x_for_y(100)
print(f"\nafter putting in 100: Δy received={dy:.3f} (90.909 without a fee), pool x={p.x:.1f} y=
{p.y:.3f}, k={p.k:,.1f} (1,000,000 before the trade — the 0.3 X fee stayed in the pool and k
grew)")
```

```
# Output — bcbook (Python 3.13), exactly as run
start: x=1000.0 y=1000.0 k=1,000,000 marginal price (price of one X in Y)=1.0000
order Δx  received Δy  exec price Δy/Δx  slippage  marginal price after
       1        0.996            0.9960     0.40%                0.9980
      10        9.872            0.9872     1.28%                0.9803
     100       90.661            0.9066     9.34%                0.8267
     500      332.666            0.6653    33.47%                0.4449
    1000      499.249            0.4992    50.08%                0.2504

after putting in 100: Δy received=90.661 (90.909 without a fee), pool x=1100.0 y=909.339,
k=1,000,272.8 (1,000,000 before the trade — the 0.3 X fee stayed in the pool and k grew)
```

[Intermediate] The table confirms the arithmetic of Section 2. When the order is 1% of the pool, slippage is about 1% (1.28% including the fee); at 10% it is about 9% (9.34%); put in as much as the pool holds and you lose half. The figures in this table are set **only by the structural variable** "pool of 1,000:1,000," and have nothing to do with what the tokens are. This chapter's interactive figure W7 moves only these two variables (pool size and order size).

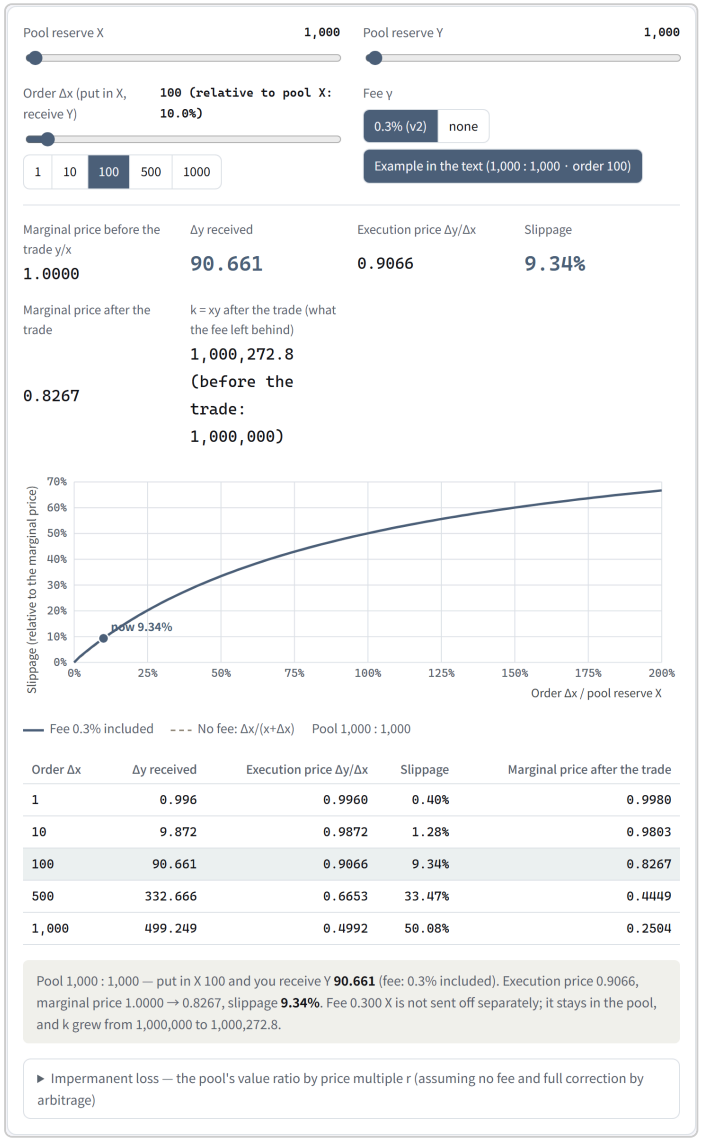


Figure 15-4. AMM slippage. Put X 100 into a 1,000 : 1,000 pool and you receive Y 90.661 (including the 0.3% fee); slippage is 9.34%. The units are token counts in the pool, not currency. — In the HTML edition you can change the values here (HTML edition, Chapter 15, Section 6).

The two pans of the scale: impermanent loss and fees

```
# NB15-2: the two pans of the scale — impermanent loss by price multiple r, and how far sqrt(k)
grown by fees makes up for it (for study)
import math

def impermanent_loss(r: float) -> float:                    # this book's derivation: V_pool /
V_hold − 1 = 2√r/(1+r) − 1
    return 2 * math.sqrt(r) / (1 + r) − 1

print("price multiple r and impermanent loss (assuming fee 0 and full correction of the ratio by
arbitrage):")
print(f"{'r':>6} {'value if kept (Y)':>18} {'value in pool (Y)':>18} {'impermanent loss':>17}")
x0 = y0 = 1000.0
for r in [0.25, 0.5, 0.8, 1.0, 1.25, 2.0, 4.0]:
    hold = x0 * r * (y0 / x0) + y0                              # X valued at the new price
r·(y/x), plus Y
    pool_val = 2 * y0 * math.sqrt(r)
    print(f"{r:>6.2f} {hold:>18.1f} {pool_val:>18.1f} {impermanent_loss(r):>17.2%}")

# the fee pan of the scale — trades go back and forth so the ratio returns to where it was, but
k grows. Count the growth of sqrt(k) by trading volume (as a multiple of the pool).
print("\nfee accrual: swaps of size 10, alternating X→Y and Y→X, N times (the ratio stays almost
in place, k grows)")
print(f"{'round-trip volume/pool':>22} {'sqrt(k) growth (= fee accrual per share)':>40} {'r
where V_pool = V_hold (formula under assumptions)':>51}")
for n_pairs in [10, 100, 1000]:
    p = ConstantProductPool(1000, 1000)
    k0 = p.k
    for _ in range(n_pairs):
        p.swap_x_for_y(10); p.swap_y_for_x(10)
    growth = math.sqrt(p.k / k0) − 1                            # g = √(k2/k1) − 1. Related to f
= 1 − √k1/√k2 in v2 equation (4) by f = g/(1+g) (the reference points differ)
    # the r at which fee growth g exactly offsets impermanent loss: (1+g)·2√r/(1+r) = 1 → the
two roots of a quadratic
    a = 1 + growth
    disc = (2 * a) ** 2 − 4
    s1, s2 = (2 * a − math.sqrt(disc)) / 2, (2 * a + math.sqrt(disc)) / 2   # the two roots for
√r
    print(f"{2 * n_pairs * 10 / 1000:>21.1f}x {growth:>40.4%} {f'{s1**2:.3f} ≤ r ≤
{s2**2:.3f}':>51}")
```

```
# Output — bcbook (Python 3.13), exactly as run
price multiple r and impermanent loss (assuming fee 0 and full correction of the ratio by
arbitrage):
     r  value if kept (Y)  value in pool (Y)  impermanent loss
  0.25             1250.0             1000.0           -20.00%
  0.50             1500.0             1414.2            -5.72%
  0.80             1800.0             1788.9            -0.62%
  1.00             2000.0             2000.0             0.00%
  1.25             2250.0             2236.1            -0.62%
  2.00             3000.0             2828.4            -5.72%
  4.00             5000.0             4000.0           -20.00%

fee accrual: swaps of size 10, alternating X→Y and Y→X, N times (the ratio stays almost in
place, k grows)
round-trip volume/pool sqrt(k) growth (= fee accrual per share) r where V_pool = V_hold (formula
under assumptions)
                0.2x                                     0.0299%
0.952 ≤ r ≤ 1.050
                2.0x                                     0.2985%
0.857 ≤ r ≤ 1.167
               20.0x                                     2.9853%
0.614 ≤ r ≤ 1.628
```

[Intermediate] The tables above are this chapter's scale. The left table shows that if the price

moves in either direction, the provider ends up worse off than by holding the tokens. The right table shows, as trading volume builds up to some multiple of the pool size, up to what range of price movement the fees make up for it. All the figures in both tables are **in pool units** and come with assumptions (0.3% fee, full correction by arbitrage, round-trip trades). Which side is larger in a real pool is an empirical question settled by that pool's trading volume and the price movement over that period, and this book does not give those figures. Neither "supplying liquidity makes a profit" nor "it makes a loss" is the conclusion of this chapter. The conclusion is that **the two terms enter one formula as a product**.

7. Summary

An AMM does away with the order book and the operator and accepts trades by a single rule, "the product does not decrease." The price is the ratio of the two quantities, a large order pushes the ratio and produces slippage, and the fee stays in the pool and makes k larger. Providers share that growth according to their shares, but end up worse off than by holding the tokens to the extent that the price has moved. It is arbitrageurs who bring the ratio in line with the market, and that is why the spot ratio is dangerous to use as an oracle. The next chapter, Chapter 16, looks at the more fundamental problem that "the chain does not know the outside," and Chapter 17 takes this chapter's pool class as it is and looks at "who decides the order of transactions."

Sources for this chapter

- Hayden Adams, Noah Zinsmeister, Dan Robinson, "Uniswap v2 Core," March 2020 (technical white paper, 10 pp.). https://uniswap.org/whitepaper.pdf — the constant-product invariant and the fee (§1, p. 1), the term impermanent loss (§2.1, p. 2), marginal price, oracle manipulation, and the time-weighted average (§2.2, pp. 2–3), fee accrual equation (4) (§2.4, p. 5), fee adjustment equations (9)–(11) (§3.2.1, p. 7), share equations (12) and (13) (§3.4, p. 8).
- Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, Dan Robinson, "Uniswap v3 Core," March 2021 (9 pp.). https://uniswap.org/whitepaper-v3.pdf — concentrated liquidity, virtual reserves, equation (2.2), and leaving the range (§2, p. 2), fee tiers (§1 and §3.1), governance (§4, p. 3).
- Philip Daian et al., "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability," *2020 IEEE Symposium on Security and Privacy*, pp. 910–927, DOI 10.1109/SP40000.2020.00040 — descriptions of order-book exchanges, DEXes, and AMMs (§II-B and C, p. 912), the description of "decentralized" exchange as "something of a misnomer" (footnote 1), the slowness of on-chain trades (§I).
- The impermanent loss formula $2\sqrt{r}\,/\,(1+r)-1$ and the numerical example of concentrated liquidity were derived and computed by this book from the formulas in the original sources above.
- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library module `math`. Notebook NB15. The initial state of interactive figure W7 `amm-slippage` is NB15-1's pool of 1,000:1,000 with an order of 100.

Notes

1. Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, Ari Juels, "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability," *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 910–927, §II-B (p. 912): "Classic exchanges for trading stocks, commodities, and even cryptocurrencies generally share an accepted and common exchange design known as a continuous-limit orderbook. Such an orderbook consists of a list of all open offers from buyers and sellers in the system. Prospective buyers place a limit buy order, which specifies a maximum price at which they are willing to buy an asset; sellers correspondingly place limit sell orders. A centralized counterparty, the exchange operator, matches buyers and sellers, completing transactions automatically when there is a sell order on the books with a lower price than a buy order on the books." Emphasis added.

2. Same paper, §I (p. 910): "Despite their clear benefits, however, many DEXes come with a serious and fundamental weakness: on-chain, smart-contract-mediated trades are slow. Traders thus may attempt to take orders that have already been taken or canceled but appear active due to their views of messages sent on the network." Emphasis added.

3. Same paper, §II-C (p. 912) and footnote 1 (p. 910): "Order books are typically maintained off-chain."; "'Decentralized' exchange is something of a misnomer, as many such systems have centralized components; most systems we call 'decentralized' exchanges could more accurately be classified as non-custodial: users trade without surrendering control of their funds to a third party in the process." Emphasis added.

4. Same paper, §II-C (p. 912). Emphasis added.

5. Hayden Adams, Noah Zinsmeister, Dan Robinson, "Uniswap v2 Core," March 2020, §1, Introduction (p. 1): "Uniswap v1 is an on-chain system of smart contracts on the Ethereum blockchain, implementing an automated liquidity protocol based on a 'constant product formula' [1]. Each Uniswap v1 pair stores pooled reserves of two assets, and provides liquidity for those two assets, maintaining the invariant that the product of the reserves cannot decrease. Traders pay a 30-basis-point fee on trades, which goes to liquidity providers. The contracts are non-upgradeable." Emphasis added. This book cites Uniswap as a **literature source** for the constant-product AMM and does not endorse or compare any trading service (Chapter 3, Notes to the reader).

6. Same paper, §1 (p. 1): "While the contract is not generally upgradeable, there is a private key that has the ability to update a variable on the factory contract to turn on an on-chain 5-basis-point fee on trades. This fee will initially be turned off, but could be turned on in the future, after which liquidity providers would earn 25 basis points on every trade, rather than 30 basis points." §4, Governance, of the v3 white paper (2021) describes this authority: "The factory has an owner, which is initially controlled by UNI tokenholders. The owner does not have the ability to halt the operation of any of the core contracts." (`adams-2021-uniswap-v3-core.pdf`, p. 3).

7. Adams et al. (2020), §2.2, Price oracle (p. 2), equation (1): "The marginal price offered by Uniswap (not including fees) at time t can be computed by dividing the reserves of asset a by the reserves of asset b. pt = r^a_t / r^b_t (1)".

8. Adams et al. (2020), §3.2.1, Adjustment for fee (p. 7), equations (9)–(11): "Uniswap v1's trading fee is applied by reducing the amount paid into the contract by 0.3% before enforcing the constant-product invariant. The contract implicitly enforces the following formula: (x1 − 0.003 · xin)) · y1 >= x0 · y0 (9) With flash swaps, Uniswap v2 introduces the possibility that xin and yin might both be non-zero ... the contract is written to enforce the following invariant: (x1 − 0.003 · xin) · (y1 − 0.003 · yin) >= x0 · y0 (10)". (The unbalanced parenthesis in the original equation (9) is as in the original.) Footnote 5 to §2.3 (flash swaps) on p. 5 adds that because Uniswap charges fees on input amounts, "the fee relative to the withdrawn amount is actually slightly higher," about 0.3009%.

9. Adams et al. (2020), §2.1, ERC-20 pairs (p. 2): "All liquidity providers have exposure to ETH, and suffer impermanent loss based on changes in the prices of other assets relative to ETH. When two assets ABC and XYZ are correlated—for example, if they are both USD stablecoins—liquidity providers on a Uniswap pair ABC/XYZ would generally be subject to less impermanent loss than the ABC/ETH or XYZ/ETH pairs."

10. Adams et al. (2020), §3.4, Initialization of liquidity token supply (p. 8), equations (12) and (13): "s_minted = (x_deposited / x_starting) · s_starting (12)"; "Uniswap v2 initially mints shares equal to the geometric mean of

the amounts deposited: s_minted = √(x_deposited · y_deposited) (13) This formula ensures that the value of a liquidity pool share at any time is essentially independent of the ratio at which liquidity was initially deposited."

11. Same paper, §2.4, Protocol fee (p. 5), equation (4): "The total collected fees can be computed by measuring the growth in √k (that is, √(x · y)) since the last time fees were collected. This formula gives you the accumulated fees between t1 and t2 as a percentage of the liquidity in the pool at t2: f1,2 = 1 − √k1/√k2 (4)".

12. Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, Dan Robinson, "Uniswap v3 Core," March 2021, §2, Concentrated Liquidity (p. 2): "In earlier versions, liquidity was distributed uniformly along the x · y = k reserves curve ... earlier versions were designed to provide liquidity across the entire price range (0, ∞). This is simple to implement and allows liquidity to be efficiently aggregated, but means that much of the assets held in a pool are never touched." Emphasis added.

13. Same paper, §2 (p. 2), equation (2.2): "A position only needs to maintain enough reserves to support trading within its range, and therefore can act like a constant product pool with larger reserves (we call these the virtual reserves) within that range." "The amount of liquidity provided can be measured by the value L, which is equal to √k. The real reserves of a position are described by the curve: $(x + L/\sqrt{p_b})(y + L\sqrt{p_a}) = L^2$ (2.2)". Emphasis added.

14. Same paper, §2 (p. 2): "When the price exits a position's range, the position's liquidity is no longer active, and no longer earns fees. At that point, its liquidity is composed entirely of a single asset, because the reserves of the other asset must have been entirely depleted. If the price ever reenters the range, the liquidity becomes active again."

15. Same paper, §1 and §3.1 (pp. 1 and 3): "The swap fee is no longer locked at 0.30%. Rather, the fee tier for each pool (of which there can be multiple per asset pair) is set on initialization (Section 3.1). The initially supported fee tiers are 0.05%, 0.30%, and 1%."

16. Adams et al. (2020), §2.2 (p. 2): "However, Uniswap v1 is not safe to use as an on-chain price oracle, because it is very easy to manipulate. Suppose some other contract uses the current ETH-DAI price to settle a derivative. An attacker who wishes to manipulate the measured price can buy ETH from the ETH-DAI pair, trigger settlement on the derivative contract (causing it to settle based on the inflated price), and then sell ETH back to the pair to trade it back to the true price. This might even be done as an atomic transaction, or by a miner who controls the ordering of transactions within a block." Emphasis added. "ETH-DAI" is the original's example notation, and this book does not evaluate any particular token.

17. Footnote 1 on the same page: "For a real-world example of how using Uniswap v1 as an oracle can make a contract vulnerable to such an attack, see [3]." Reference [3] is samczsun, "Taking undercollateralized loans for fun and for profit," Sept. 2019 (a blog post). This book has not obtained this post as source material and does not quote its contents.

18. Same paper, §2.2 (p. 3), equations (2) and (3): "Uniswap v2 accumulates this price, by keeping track of the cumulative sum of prices at the beginning of each block in which someone interacts with the contract. Each price is weighted by the amount of time that has passed since the last block in which it was updated ..."; "Users of the oracle can choose when to start and end this period. Choosing a longer period makes it more expensive for an attacker to manipulate the TWAP, although it results in a less up-to-date price."; "A miner (or an attacker who uses enough gas to fill an entire block) could manipulate the price at the end of a block, but unless they mine the next block as well, they may not have a particular advantage in arbitraging the trade back."

PART 2 · SMART CONTRACTS AND APPLICATIONS

Chapter 16. The oracle problem: bringing facts from outside the chain into it

KEY TAKEAWAY

A blockchain knows only what happens inside it. How much sits at which address, and what is written in which storage slot of which contract, every node can compute identically. But "today's temperature in New York," "yesterday's exchange rate" and "whether that flight was delayed" are not in the chain. The device that brings them into the chain is called an **oracle**, and an oracle in the end **is someone writing a value in**. Whether to have one such someone or several, and whether to make them forfeit a deposit if they are wrong, are questions of design, and no design brings "the people you have to trust" down to 0. After this chapter you will understand why the slogan "code is law" holds only inside the chain, and the paradox that when an oracle puts in a wrong value, a contract **can be executed exactly and be wrong at the same time**.

Learning objectives

1. Explain, through determinism (every node performs the same computation), why a blockchain cannot fetch outside facts by itself.
2. Name the three branches of oracle design (a single provider, aggregation over multiple providers, economic collateral) and **what each of them requires you to trust**.
3. **[Intermediate]** Starting from the definition of the "ideal oracle," sort out at which layer each of these failures lies: data-source error, oracle collusion, Sybil attacks and mirroring, and front-running of reports.
4. Explain, connecting it to Chapter 15, why using an AMM's spot ratio as an oracle invites manipulation.

What a reader of the [Basic] paragraphs alone takes away. The slogan "code is law" holds only inside the chain, and facts from outside the chain come in only if you **trust someone**.

1. What failure this technology grew out of

The contracts of Chapter 13 were boxes that release value when a condition is met. But the conditions people want to write are mostly out in the world. Buterin's 2014 white paper says that "Financial derivatives are the most common application of a 'smart contract', and one of the simplest to implement in code," and goes straight on to the obstacle: "The main challenge in implementing financial contracts is that **the majority of them require reference to an external price ticker**; ... The simplest way to do this is through a 'data feed' contract maintained by a specific party (eg. NASDAQ) designed so that that party has the ability to update the contract as needed, and providing an interface that allows other contracts to send a message to that contract and get back a response that provides the price."[1] The first application of a system built to do away with the trusted third party presupposes **a contract maintained by a specific party**. This tension is the whole of this chapter.

The Yellow Paper records the same pattern too, as one of "several patterns of contracts engineering," "two of these that we will briefly discuss" (data feeds and random numbers). "A data feed contract is one which provides a single service: it gives access to information from the external world within Ethereum. **The accuracy and timeliness of this information is not guaranteed** and it is the task of a secondary contract author—the contract that utilises the data feed—to determine how much trust can be placed in any single data feed." As for the rest, "the second part of the pattern would be for an external server to run an Ethereum node, and immediately on discovery of a new block, creates a new valid transaction, sent to the contract, updating said value in storage. The contract's code would accept such updates only from the identity contained on said server."[2] In short: **one server, one key, and one number that key writes**. This is the point where the word "decentralized" wears thinnest.

2. Why the chain does not know the outside

The reason lies in the chain's very strength. Chapter 13 said that every node executes the same transactions in the same order and reaches the same state. That "sameness" is the condition for consensus. If a contract could say mid-execution, "now go and read the temperature from an outside server," a node in Seoul and a node in São Paulo would read different values at different moments and reach **different states**, and block validation would split from node to node. So contract code can read only what is in the chain, and an outside value becomes part of the chain **only after someone writes it in with a transaction**. That is this book's explanation; the 2017 Chainlink white paper states the same thing as a conclusion: "[B]ecause of their underlying consensus protocols, the blockchains on which smart contracts run cannot support native communication with external systems. Today, the solution to this problem is to introduce a new functionality, called an oracle, that provides connectivity to the outside world. **Existing oracles are centralized services.** Any smart contract using such services has a single point of failure, making it no more secure than a traditional, centrally run digital agreement."[3]

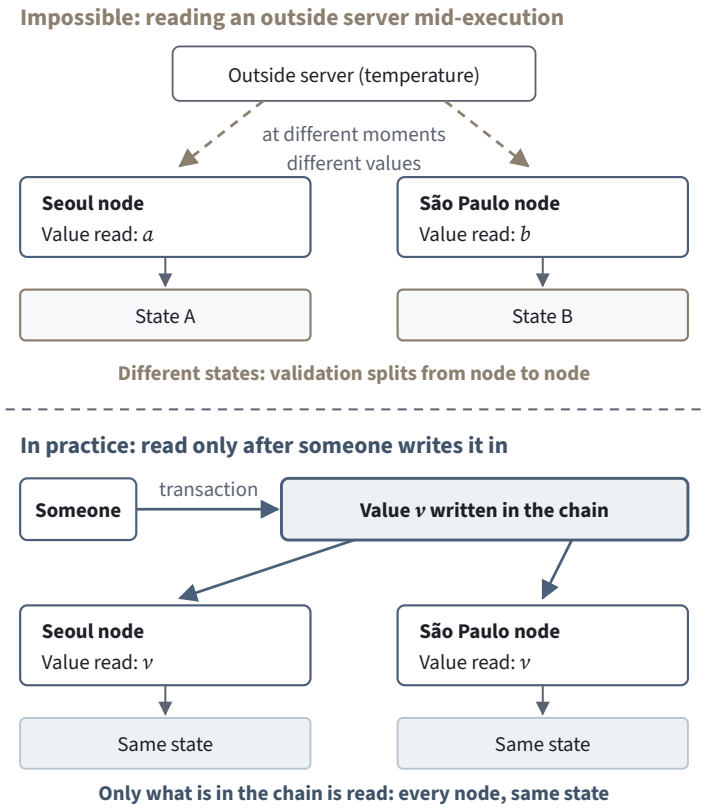


Figure 16-1. Why the chain does not know the outside. If a contract read an outside server during execution, the state would diverge from node to node, so an outside value becomes part of the chain only after someone writes it in with a transaction.

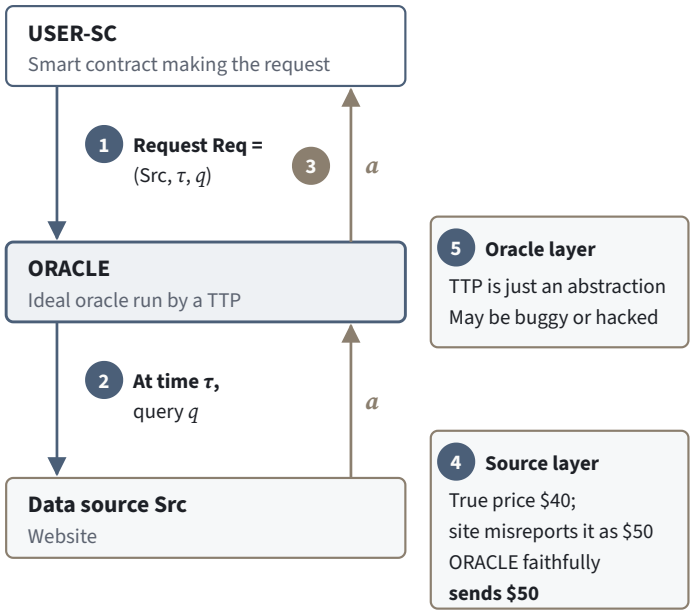
The examples the same white paper gives will be familiar to lawyers: "Securities smart contracts such as bonds, interest rate derivatives, and many others will require access to APIs reporting market prices and market reference data, e.g. interest rates. Insurance smart contracts will need data feeds about IoT data related to the insurable event in question, e.g.: was the warehouse's magnetic door locked at the time of breach, was the company's firewall

online, or did the flight you had insurance for arrive on time. Trade finance smart contracts will need GPS data about shipments, data from supply chain ERP systems, and customs data about the goods being shipped …."[4] Whether an insurance claim is paid, the conditions for paying the price, the moment collateral is disposed of: almost all of the conditions that contract law deals with lie outside the chain. What code enforces automatically goes as far as "if the condition is true, pay," and "is the condition true?" comes in from outside.

3. The ideal oracle, and why it cannot be had

[Intermediate] To define the "security" of an oracle, Section 3 of the Chainlink white paper sets up a thought experiment. "Imagine that a trusted third party (TTP)—an ideal entity or functionality that always carries out instructions faithfully to the letter—were tasked with running an oracle. … suppose that the TTP obtains data from a perfectly trustworthy data source Src." There are only three things to ask of that ideal oracle ORACLE: "1. Accept request: Ingest from a smart contract USER-SC a request Req = (Src, τ, q) that specifies a target data source Src, a time or range of times τ, and a query q; 2. Obtain data: Send query q to Src at time τ; 3. Return data: On receiving answer a, return a to the smart contract." This is **integrity**. "For example, if Src is https://www.FountOfKnowledge.com, τ is 4 p.m., and q = 'price for ticker INTC', the integrity of ORACLE guarantees that it will provide USER-SC with exactly the price of INTC as queried at 4 p.m. at https://www.FountOfKnowledge.com."[5]

[Intermediate] The white paper then sets out, in two layers, why this definition is out of reach in practice. The first layer is **the data source**. "There is, of course, no perfectly trustworthy data source Src. Data may be benignly or maliciously corrupted due to faulty web sites, cheating service providers, or honest mistakes. If Src isn't trustworthy, then even if ORACLE does operate exactly like a TTP as instructed above, it still doesn't completely meet the notion of security we want. … **If the true price of Intel is $40 and https://www.FountOfKnowledge.com misreports it as $50, for example, then ORACLE will send the incorrect value a = $50 to USER-SC.** This problem is unavoidable when using a single source Src." The second layer is **the oracle itself**. "A bigger issue, of course, is the fact that our TTP for ORACLE is just an abstraction. No service provider is unconditionally trustworthy. Even the best-intentioned may be buggy or hacked."[6] That an oracle **can operate exactly as instructed and still be wrong**: this is where this chapter's question box stands.



$40 and $50: a hypothetical illustration, not price data

Figure 16-2. The three steps of the ideal oracle defined in the Chainlink white paper, and the two layers (the data source, the oracle itself) at which that definition is out of reach in practice. The USD 40 and USD 50 are the white paper's hypothetical illustration.

4. Three branches of design: what has to be trusted

Oracle designs differ by where they put "what has to be trusted." The table is this book's arrangement; the basis for each cell is in the text below.

DESIGN	WHAT WRITES THE VALUE	WHAT HAS TO BE TRUSTED	SHAPE OF FAILURE
Single provider (data feed contract)	One server's key	That operator and that source	Operator error, hacking or source error passes straight onto the chain
Aggregation over multiple providers	The answers of n nodes, combined by majority vote, median and the like	"There are enough honest nodes," "the sources are independent of one another"	If the sources copy one another, majority voting means nothing; one party poses as several nodes (Sybil)
Economic collateral	As above, but nodes post a deposit and lose it if wrong	"The penalty is larger than the benefit from manipulation"	In a trade where manipulation would bring in more than the deposit, collateral is no deterrent
Reading an in-chain ratio as it stands (Chapter 15)	The two quantities of an AMM pool	"Nobody pushed the ratio at that moment"	Push, get read, and revert, all within one transaction

[Intermediate] **Distributing sources.** "A simple way to deal with a faulty single source Src is to obtain data from multiple sources, i.e., distribute the data source. ... One, for example, is majority voting. If a majority of sources return the identical value a, the function agg returns a; otherwise it returns an error. In this case, provided that a majority (> k/2) sources are functioning correctly, ORACLE will always return a correct value A. ... For example, agg might discard outliers (e.g., the largest and smallest values ai) and output the mean of the

remaining ones." But the white paper adds a limit at once. "Of course, faults may be **correlated** across data sources in a way that weakens the assurances provided by aggregation. If site Src1 = EchoEcho.com obtains its data from Src2 = TheHorsesMouth.com, an error at Src2 will always imply an error at Src1."[7] Majority voting means something **only when the voters are independent of one another**. It is like ten newspapers that run the same wire-service story printing the same error.

[Intermediate] Distributing oracles. Along with the sources, the oracle nodes too are multiplied, and the aggregation is done "in-contract." "CHAINLINK-SC—which, again, denotes the on-chain part of ChainLink—will itself aggregate oracle responses. ... CHAINLINK-SC will compute A = Agg(A1, A2, . . . , An) for some function Agg (similar to agg, as described above), and send the result A to USER-SC." As benefits it lists "Conceptual simplicity," **trustworthiness**, in that "As CHAINLINK-SC's code can be publicly inspected, its correct behavior can be verified," and flexibility.[8] If the aggregation code sits on the chain, you do not have to trust **the way of aggregating**. What has to be trusted moves to the assumption that "of the n nodes, the honest ones are a majority." The 2021 follow-up white paper restates this in the language of Byzantine fault tolerance (BFT): "Most BFT algorithms require at least $n = 3f + 1$ nodes, where f is the number of potentially faulty nodes; ... We refer to the committee O as honest if it meets this requirement, i.e., has greater than a 2/3-fraction of honest nodes."[9] Chapter 10's "2/3" comes back here. But the 2021 white paper adds that the security of the two kinds of system **cannot be measured with the same yardstick**: "Compromising a PoW or a PoS blockchain via 51% attack requires that an adversary obtain majority resources ephemerally and potentially anonymously ... In contrast, compromising a committee-based system means corrupting a threshold number (typically one-third) of its nodes, where the nodes may be publicly known, well resourced, and trustworthy entities."[10] Which is safer depends on **who the attacker is**.

[Intermediate] Economic collateral and reputation. In the order-matching design of the 2017 white paper, "When an oracle service provider bids on a contract, they commit to it, specifically by **attaching the penalty amount that would be lost due to their misbehavior**, as defined in the SLA."[11] And a reputation system records "Total number of assigned requests," "Total number of completed requests," "Total number of accepted requests" (accepted "when compared with peer responses"), "Average time to respond" and "Amount of penalty payments."[12] This is the same logic as slashing in Chapter 10. Honesty is rational only if there is something to lose by being wrong. Yet the white paper itself records what this device does not stop: **Sybil attacks and mirroring**. "Both our simple and in-contract aggregation protocols seek to prevent freeloading in the sense of dishonest nodes copying honest nodes' answers. But neither protects against Sybil attacks [9]. Such attacks involve an adversary that controls multiple, ostensibly independent oracles. This adversary can attempt to dominate the oracle pool, causing more than f oracles to participate in the aggregation protocol and provide false data at strategic times, e.g., in order to influence large transactions in high-value contracts." And: "Attacks or faults involving $> f$ oracles are especially pernicious in that they are **undetectable from on-chain behavior alone**."[13] Collateral changes the incentive of a single node. That several nodes are in fact one, collateral does not find out.

▾ [Advanced] A fourth road: trusted hardware, and what it shifts

[Advanced] What the 2017 white paper put forward as its "long-term strategy" was not distribution but **hardware**. "A distributed oracle network aims to offer a high degree of protection against faulty oracles. In most deployment scenarios, it seeks to attain a correct response in the face of f Byzantine faults (for $f < n / 2$ in our simple aggregation protocol). Trusted hardware can offer much more ...." Run the oracle in a trusted execution environment (an enclave) such as Intel SGX and distribute its **attestations**, and you get "very strong assurance that the oracle is executing a particular application," and "an oracle running in an enclave that can connect to a data source via HTTPS can provide a strong assurance that the data it retrieves has not been tampered with."[14] Then there is confidentiality. A distributed oracle writes its queries on a public chain, so it runs into the limit that "At some point a server needs to send a query to a target data source server. Thus it must see the query ..."; decrypting and processing the query inside the enclave gets past that limit. In Appendix B the white paper discusses separately "the issue of Intel as a root of trust."[15] Trust has not vanished; it has **moved to the chip maker and its attestation system**. If this chapter's table were given a fourth row, its "what has to be trusted" cell would read "the hardware maker's attestations, and an enclave implementation free of flaws." Another road is written down as well. It is an infrastructure change (TLS-N): if data sources digitally signed the data they serve, "then oracles would not need to be trusted to refrain from tampering with data." But the white paper writes that "Few data sources are likely to in the near future."[16] The cleanest solution is one under which **the outside world has to change**.

5. At which layer does manipulation happen?

The phrase "oracle manipulation" lumps several layers together. This book separates four.

1. **Error or manipulation at the source.** The "site that reports USD 40 as USD 50" of Section 3. The oracle was faithful; the source was wrong.
2. **Collusion or Sybil attacks among oracle nodes.** Section 4's "more than f nodes give false data at strategic times." The aggregation code was faithful; the inputs were wrong.
3. **Reading an in-chain ratio as an oracle.** The attack in Section 5 of Chapter 15. As the Uniswap v2 white paper puts it, "An attacker who wishes to manipulate the measured price can buy ETH from the ETH-DAI pair, trigger settlement on the derivative contract (causing it to settle based on the inflated price), and then sell ETH back to the pair to trade it back to the true price. This might even be done as an atomic transaction, or by a miner who controls the ordering of transactions within a block."[17] Here **there is no separate oracle at all.** A contract simply read two storage slots of another contract (the two reserves) as "the price," and those two slots held values that anyone could push with a transaction. The white paper's answer (a time-weighted average) was a trade-off that raises the cost of manipulation and gives up freshness.
4. **Front-running the report itself.** If the transaction by which an oracle honestly posts a

new value is visible in **the public queue**, someone slips a transaction in ahead of it. The 2021 white paper describes it: "Consider the common design pattern of using an oracle to supply various price feeds to an on-chain exchange: periodically (say every hour), the oracle collects price data for different assets and sends these to an exchange contract. These price-data transactions present obvious arbitrage opportunities: For example, if the newest oracle report lists a much higher price for some asset, an adversary could **front-run** the oracle report to buy up assets and immediately resell them once the oracle's report is processed."[18] The oracle, the source and the aggregation were all correct. Nothing was wrong; only **the order** became money. That is Chapter 17.

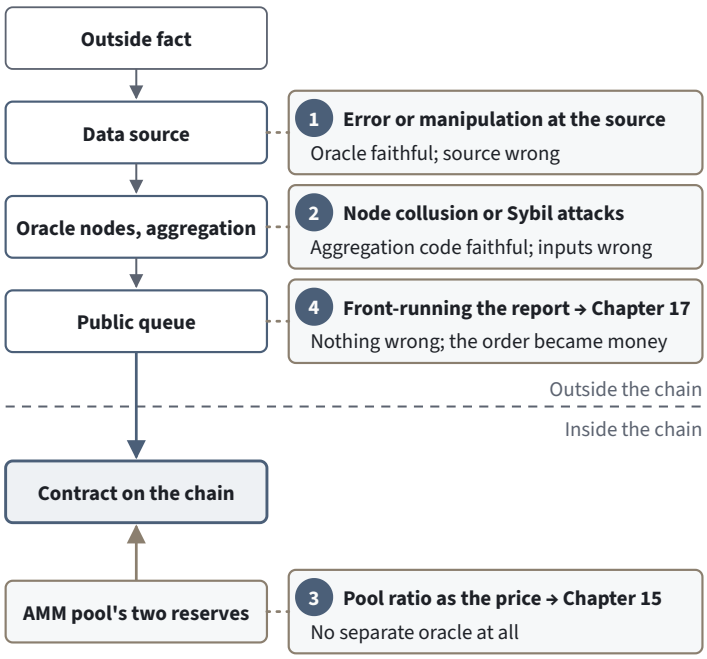


Figure 16-3. Four points of failure on the path by which an outside fact reaches a contract. The numbers are those of the list in the text; the third is a side road that does not pass through an oracle.

One often hears that the third of these four is "a common route for DeFi incidents." This book **did not measure that frequency.** The Uniswap v2 white paper points to a single blog post as a "real-world example" (footnote in Chapter 15), and none of the primary documents this book obtained tallies incidents by type or by size of loss. So this chapter goes only as far as "this is the **structure** of manipulation," and puts off the judgment "it is common" until tallied data are obtained (a task for the Trend notes in Appendix A).

> **? A LAWYER'S QUESTION**
>
> An oracle put in a wrong value, and the contract executed exactly on that value. An insurance claim was not paid, or collateral was disposed of, or a derivative was settled at a wrong price. The code did not deviate by a single line. Is this **performance (이행)**, or is it **mistake (착오)**? If it is mistake, whose mistake is it: the oracle operator who wrote the value in, the contract author who chose to trust that oracle, or the party who chose to use that contract? What if the wrong value came from the data source (a website), and that site had a contract with no one? If more than 1/3 of many nodes colluded, what did they do jointly? And if a contract merely read "two storage slots of another contract" as a price and nobody ran an "oracle," what did the person who pushed that value manipulate? → Chapter 28

6. Summary

Because every node performs the same computation, the chain cannot read the outside, and an outside value comes in only when someone writes it in with a transaction. Put one such someone in charge and you must trust that person; put several in charge and you must trust that they are independent; make them post deposits and you must trust that the deposit is larger than the benefit from manipulation; read an in-chain ratio and you must trust that nobody pushed the ratio at that moment. "Code is law" holds inside the chain, and whether a condition is true comes in from outside it. The next chapter, Chapter 17, looks at the problem raised at the end of this one: "everything was correct, and the order became money." The basis for misconception ⑨ in Appendix B ("code is law") is shared between this chapter and Chapter 26.

Sources for this chapter

- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2014 (first draft 2013). https://ethereum.org/whitepaper/ (accessed 2026-10-25) — the data feed contract pattern ("Financial derivatives and Stable-Value Currencies," p. 20 of the PDF).
- Gavin Wood, Ethereum Yellow Paper, Shanghai version efc5f9a, 2025-02-04 — data feed contracts (§13.1, p. 17).
- Steve Ellis, Ari Juels, Sergey Nazarov, "ChainLink: A Decentralized Oracle Network," 4 September 2017 (v1.0), 38 pp. https://research.chain.link/whitepaper-v1.pdf — the connectivity problem and centralized oracles (Abstract, §1), the ideal oracle and why it cannot be had (§3, pp. 8–10), distributing sources and oracles (§4, pp. 11–12), penalties (§2.1), reputation (§5.2), Sybil attacks and mirroring (§5.3), trusted hardware and TLS-N (§6, pp. 21–25). There is no independent bibliographic record (a DOI or the like) apart from the official host.
- Lorenz Breidenbach and 13 others, "Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks," 15 April 2021 (v1.0), 136 pp.

https://research.chain.link/whitepaper-v2.pdf — BFT committees ($n = 3f + 1$) (§2.3), the incommensurability of the two trust models (§2.4), oracle front-running (§5.1.1).

- Hayden Adams et al., "Uniswap v2 Core," March 2020, §2.2 — manipulation of the AMM spot price, and the time-weighted average.
- This chapter has no code (and no notebook). The frequency claim that "oracle manipulation is a common route for DeFi incidents" was not written, because there are no primary tallies.

Notes

1. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," official PDF on ethereum.org (2014; first draft 2013), "Financial derivatives and Stable-Value Currencies" section (p. 20 of the PDF). Emphasis added. https://ethereum.org/whitepaper/ (accessed 2026-10-25).
2. Gavin Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Shanghai version efc5f9a (2025-02-04), §13.1, Data Feeds (p. 17). Emphasis added. Original: "A data feed contract is one which provides a single service: it gives access to information from the external world within Ethereum. The accuracy and timeliness of this information is not guaranteed and it is the task of a secondary contract author—the contract that utilises the data feed—to determine how much trust can be placed in any single data feed. The general pattern involves a single contract within Ethereum which, when given a message call, replies with some timely information concerning an external phenomenon. ... the second part of the pattern would be for an external server to run an Ethereum node, and immediately on discovery of a new block, creates a new valid transaction, sent to the contract, updating said value in storage. The contract's code would accept such updates only from the identity contained on said server." https://ethereum.github.io/yellowpaper/paper.pdf (accessed 2026-10-25).
3. Steve Ellis, Ari Juels, Sergey Nazarov, "ChainLink: A Decentralized Oracle Network," 4 September 2017 (v1.0), Abstract (p. 1). Emphasis added. Original: "Unfortunately, because of their underlying consensus protocols, the blockchains on which smart contracts run cannot support native communication with external systems. Today, the solution to this problem is to introduce a new functionality, called an oracle, that provides connectivity to the outside world. Existing oracles are centralized services. Any smart contract using such services has a single point of failure, making it no more secure than a traditional, centrally run digital agreement." This white paper is a design document of a particular business; this book does not recommend or compare that design and cites it only as a text that sets out the **definition and classification** of the oracle problem (the notes to the reader in Chapter 3).
4. Same paper, §1, Introduction (pp. 3–4). Original: "Securities smart contracts such as bonds, interest rate derivatives, and many others will require access to APIs reporting market prices and market reference data, e.g. interest rates. Insurance smart contracts will need data feeds about IoT data related to the insurable event in question, e.g.: was the warehouse's magnetic door locked at the time of breach, was the company's firewall online, or did the flight you had insurance for arrive on time. Trade finance smart contracts will need GPS data about shipments, data from supply chain ERP systems, and customs data about the goods being shipped in order to confirm fulfillment of contractual obligations."
5. Ellis et al. (2017), §3, Oracle Security (pp. 8–9).
6. Same paper, §3 (p. 10). Emphasis added. (The "$40" and "$50" in the original are a hypothetical illustration, not price data.)
7. Ellis et al. (2017), §4.1, Distributing sources (p. 11). Emphasis added.
8. Same paper, §4.2, Distributing oracles (pp. 11–13). Original: "Our initial proposed solution in ChainLink will be a simple one called in-contract aggregation. CHAINLINK-SC—which, again, denotes the on-chain part of ChainLink—will itself aggregate oracle responses. ... In other words, CHAINLINK-SC will compute A = Agg(A1, A2, . . . , An) for some function Agg (similar to agg, as described above), and send the result A to USER-SC. This approach is practical for small n, and has several distinct benefits: Conceptual simplicity ... Trustworthiness: As CHAINLINK-SC's code can be publicly inspected, its correct behavior can be verified. ... Flexibility ...".

9. Lorenz Breidenbach and 13 others, "Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks," 15 April 2021 (v1.0), §2.3, Notation (p. 22). Original: "Most BFT algorithms require at least n = 3f + 1 nodes, where f is the number of potentially faulty nodes; remaining nodes are honest, in the sense that they follow the protocol exactly as specified. We refer to the committee O as honest if it meets this requirement, i.e., has greater than a 2/3-fraction of honest nodes." Of the 14 authors, the 7 with academic affiliations each state, in footnotes on the first page, the capacity in which they took part: the 4 authors Cachin, Koushanfar, Miller and Tramèr as advisors, Juels as Chief Scientist at Chainlink Labs, and Moroz and Zhang as researchers at Chainlink Labs.

10. Same paper, §2.4, Note on Trust Models (p. 22). Original: "It is important to recognize that the security of committee-based vs. permissionless decentralized systems is incommensurable. Compromising a PoW or a PoS blockchain via 51% attack requires that an adversary obtain majority resources ephemerally and potentially anonymously, for example by renting hash power in a PoW system. ... In contrast, compromising a committee-based system means corrupting a threshold number (typically one-third) of its nodes, where the nodes may be publicly known, well resourced, and trustworthy entities."

11. Ellis et al. (2017), §2.1, On-Chain Architecture (p. 6). Emphasis added.

12. Same paper, §5.2, Reputation System (p. 18). The five metrics in the original: "Total number of assigned requests ... Total number of completed requests ... Total number of accepted requests ... Average time to respond ... Amount of penalty payments". The description of the last item: "If penalty payments were locked in to assure a node operator's performance, the result would be a financial metric of an oracle provider's commitment not to engage in an 'exit scam' attack, where the provider takes users' money and doesn't provide services."

13. Same paper, §5.3, Certification Service (p. 19). Emphasis added. Original: "Both our simple and in-contract aggregation protocols seek to prevent freeloading in the sense of dishonest nodes copying honest nodes' answers. But neither protects against Sybil attacks [9]. Such attacks involve an adversary that controls multiple, ostensibly independent oracles. This adversary can attempt to dominate the oracle pool, causing more than f oracles to participate in the aggregation protocol and provide false data at strategic times, e.g., in order to influence large transactions in high-value contracts. ... Attacks or faults involving > f oracles are especially pernicious in that they are undetectable from on-chain behavior alone. Additionally, to reduce operational costs, a Sybil attacker can adopt a behavior called mirroring, in which it causes oracles to send individual responses based on data obtained from a single data-source query."

14. Ellis et al. (2017), §6.1, Confidentiality (pp. 21–22). Original: "A distributed oracle network aims to offer a high degree of protection against faulty oracles. In most deployment scenarios, it seeks to attain a correct response in the face of f Byzantine faults (for f < n/2 in our simple aggregation protocol). Trusted hardware can offer much more and is proposed as a better approach to securing the ChainLink network." "Running an oracle in an enclave and distributing attestations can provide very strong assurance that the oracle is executing a particular application ... Additionally, an oracle running in an enclave that can connect to a data source via HTTPS can provide a strong assurance that the data it retrieves has not been tampered with."

15. Same paper, §6.1 (pp. 22–23). Original: "At some point a server needs to send a query to a target data source server. Thus it must see the query, irrespective of whatever confidentiality the query previously enjoyed." "We briefly discuss the issue of Intel as a root of trust in Appendix B."

16. Same paper, §6.2, Infrastructure changes (p. 25). Original: "Many of the challenges in constructing secure oracles arise from the fact that existing data sources don't digitally sign the data they serve. If they did, then oracles would not need to be trusted to refrain from tampering with data." "Unfortunately, until and unless TLS-N becomes a standard, data sources must expressly deploy it for clients to benefit. Few data sources are likely to in the near future."

17. Hayden Adams, Noah Zinsmeister, Dan Robinson, "Uniswap v2 Core," March 2020, §2.2 (p. 2). The original is reproduced in full in a footnote to Chapter 15.

18. Breidenbach et al. (2021), §5.1.1, Oracle Front-Running (pp. 45–46). Emphasis added. As countermeasures the same section gives "[s]peed bumps" (making transactions wait a few blocks, and retroactively adjusting the price when a new report arrives) and "piggybacking" (users attach a signed report to their own transactions), and it records the limits of each.

PART 2 · SMART CONTRACTS AND APPLICATIONS

Chapter 17. MEV and transaction ordering: the power to set the order

KEY TAKEAWAY

Chapter 8 said that the rules do not dictate which transactions a miner puts into a block, or in what order. Chapter 15 said that an AMM's price changes with the order of trades. Put the two together and you get this: **whoever makes the block can set the order of transactions, and can extract money just by setting the order.** That money was named "miner extractable value (MEV)" by a paper that first appeared as a 2019 preprint and was published in 2020; after the move to proof of stake it is read as "maximum extractable value" (the form used on ethereum.org's PBS page). The easiest shape to grasp is **the sandwich**. Put my buy in front of someone else's buy order and my sell behind it, and the formula of Chapter 15 is applied to my two trades, and the difference stays with me. Where traditional markets prohibit front-running by law, here there is no "intermediary" to stop it, and code does not stop it either. The response Ethereum chose is not prohibition but **a division of labor**. It splits the one who assembles the block (the builder) from the one who proposes it (the proposer), so that the power to set the order does not pile up with a few validators. After this chapter you will know that the saying "decentralized markets have no operator" needs a rider: "instead, there is someone who sets the order."

Learning objectives

1. Explain how front-running, back-running and the sandwich follow from two facts: transactions are visible in a public waiting area (the mempool) before they get into a block, and the block producer sets the order.
2. **[Intermediate]** Apply Chapter 15's constant-product formula twice to compute the sandwich attacker's profit and the victim's loss **in pool units**, and see how the victim's slippage tolerance limits the size of the attack.
3. **[Intermediate]** Read the priority gas auction (PGA) as "an auction in which the losers pay too," and see why MEV threatens the stability of the consensus layer (rewinding attacks).
4. Know what proposer–builder separation (PBS) changes and what it does not, and whether, as of October 2026, it sits inside or outside the protocol.

What a reader of the [Basic] paragraphs alone takes away. Blockchains too have "someone who sets the order," and that someone can extract money from the order. The traditional market's prohibition of front-running **is not enforced by code** here.

1. What failure this technology grew out of

Transactions drift around the network before they get into a block. From the 2020 paper by Philip Daian and coauthors: "Ethereum transactions are routed in a peer-to-peer gossip multicast protocol by client node software. This means that **all transaction information is available to all participants in this network**, but earlier to participants with advantageous positions in the gossip topology. Additionally, nodes can simulate the outcome of every transaction given the current or expected system state."[1] If you send a large buy order to an AMM, others can read that order before it gets into a block and calculate how far it will push the price. And those others can act by "observing them and placing their own orders with higher fees to ensure they are mined first."[2] This is **front-running**.

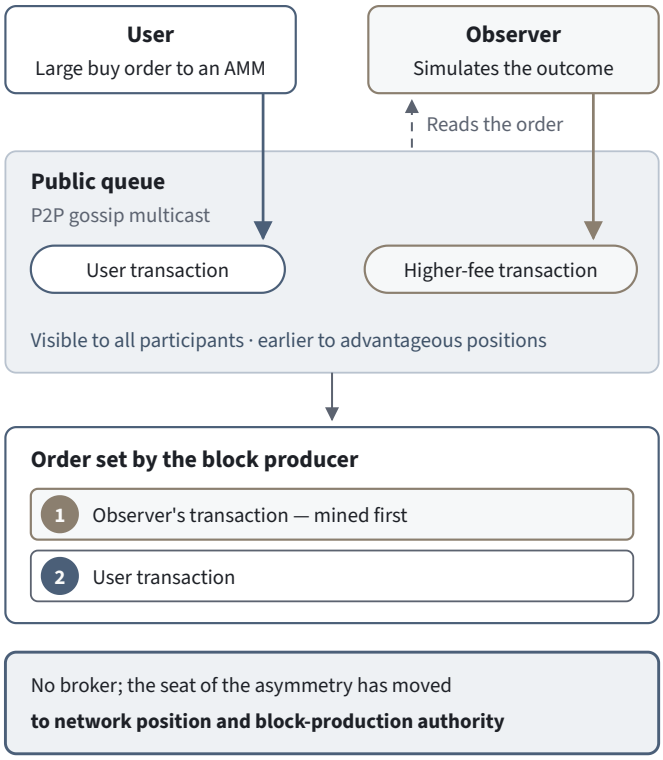


Figure 17-1. While a transaction in the public queue is visible to all participants, an observer simulates its outcome and, with a transaction of its own carrying a higher fee, takes a front place in the order the block producer sets. Where there is no broker, the seat of the asymmetry has moved to network position and block-production authority.

Traditional markets have front-running too, and the paper contrasts the two: "Traditional exchanges experience a classic form of predatory market behavior called frontrunning [4]. In regulated markets, frontrunning is often illegal, and has resulted in prosecutions [26] and tarnished the reputations of financial institutions caught practicing it. Frontrunning generally exploits information asymmetries created by power structures within a financial structure, e.g., brokers having privileged access to user information. **Because there is no single party playing the role of a broker in decentralized systems**, information asymmetries can arise for actors in advantageous positions in underlying infrastructure."[3] The "broker" at which regulation was aimed has gone, but the asymmetry remains. The seat

of the asymmetry has moved **to network position and block-production authority**.

The deeper difference the paper points to is **atomicity**. "Because of the atomic batch-based processing of transactions, and because transactions can themselves be initiated by smart contracts, is possible [sic] to build bots that trade across exchanges through proxy contracts. These proxy contracts can execute batches of orders sequentially within a single transaction, reverting previous trades by throwing an exception if any trade in the batch fails. This means arbitrageurs have the opportunity to compose single transactions that execute multiple trades across multiple exchanges atomically, with an **all-or-nothing** failure model."[4] Chapter 13's "running out of gas leaves no half-finished state" becomes an attack tool here. There is no risk that only one of two trades goes through, so if there is an arbitrage profit it is taken with certainty, and if there is none only the fee is lost.

2. The intuition: slip in front, slip in behind, slip in on both sides

Let us sort out three words. The definition in the 2021 Chainlink 2.0 white paper is concise: "Front-running exemplifies a class of transaction-ordering attacks: There are a number of related attacks such as back-running and **sandwiching (front-running plus back-running)** ...."[5]

- **Front-running.** Knowing that the victim's order will raise the price, you put in an order in the same direction **ahead of it**. Once the victim raises the price, you already have what you bought.
- **Back-running.** You put in an order in the opposite direction **behind** the victim's order. Knowing that the price the victim pushed up will come back down, you take that reversion.
- **Sandwich.** Combine the two. Buy in front; the victim raises the price; sell behind. Chapter 15's formula is applied three times, and two of the three are yours.

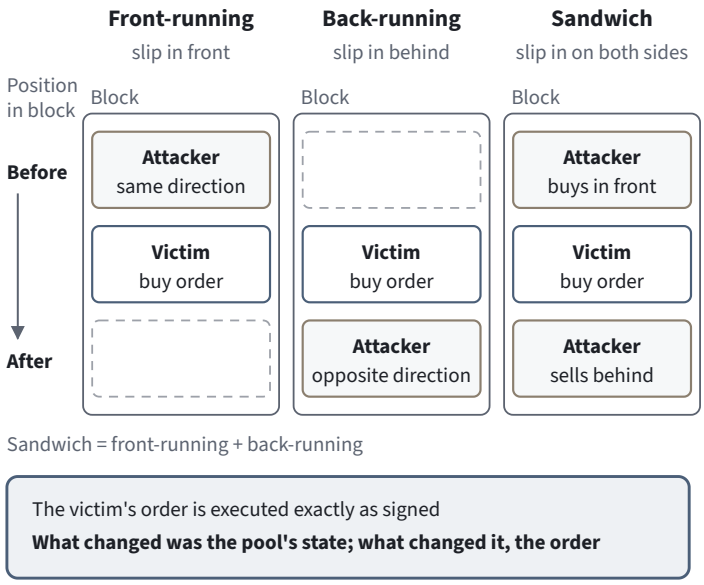


Figure 17-2. Front-running, back-running and the sandwich compared by position in the block. In all three cases the victim's order is exactly as signed; what differs is whether the attacker's order comes before it or after it.

Drawn with Chapter 15's pool, a sandwich looks like this. The pool holds 1,000 each of X and Y, and the victim wants to buy X with 100 Y. Alone, the victim would get about 90.7 X by Chapter 15's formula. If the attacker first buys X with 100 Y, the pool becomes Y 1,100 / X 909.3, and the price of one X has already gone up. The victim now gets only about 75.6 X for the same 100 Y. Then the attacker sells its 90.7 X back and receives more than 100 Y. The attacker's profit is that difference, and the victim's loss is "90.7 − 75.6." The units are all counts of tokens in the pool. The code below runs exactly this arithmetic.

One point matters here. The attacker **did not change** the victim's order. The victim's transaction was executed exactly as the victim signed it. What changed was **the state of the pool at the moment** that transaction was executed, and what changed it was the order. It has the same shape as the `approve` race condition in Chapter 14.

3. Who sets the order: auctions and producers

[Intermediate] The one who sets the order is **whoever makes the block**. But even if the block producer does not build sandwiches itself, bots run auctions to **buy the order** from the producer. The paper calls this a "priority gas auction (PGA)." "In Ethereum, the PGA game we observe consists of transactions issued with the same (account, nonce) pair, expressing a bid to a miner ... To place a bid on an arbitrage opportunity, an arbitrageur simply issues a smart contract transaction with associated gas price g ... If they later wish to increase this bid, ... they simply reissue the transaction with the same nonce and a higher gas price g'. If two arbitrageurs are bidding against each other, what emerges is essentially an auction ...."[6] The nonce of Chapter 11 is used here as a device for "updating a bid in the same slot." And this auction has a peculiar property: **the losers pay too.** The losing transaction "is also mined and included in the final block, as each (account, nonce) pair can include one transaction in the next block to be mined as per the protocol. Miners are incentivized to include even failed transactions, as these transactions pay for attempted execution. ... In game theoretic terms, each auction represents a variant of all pay auction ...."[7] This is where Chapter 13's "when the gas runs out the execution is reverted, but the fee remains" becomes the losers' cost.

[Intermediate] A few of the paper's observations are carried over because they show the structure (the amounts are not carried over). The bots competed to cut their latency, and the size of bid raises converged over time to the minimum raise (12.5%) allowed by the Parity client, which was widely used at the time; the paper reads this as consistent with "a convergence toward elements of cooperative equilibrium."[8] Most of the auctions observed were games among two or three players, and most pure revenue opportunities drew no competition. As one reason for this, the paper offers the hypothesis (Appendix C) that bots that keep losing become discouraged and leave the market, and it adds the arithmetic that, as the number of participants grows, the reward must grow in proportion to the cost of losing for taking part to be profitable. These observations cover the period shown in the paper's figures (horizontal axis 2017-07 to 2019-03) and only the "pure revenue opportunities" on the exchanges the paper supported (which included the top five DEXes by volume at the time), so they are a **lower bound**, and the paper itself writes at length about the limits of its

measurement.[9] What this book takes away is not the numbers but the shape. The order **is sold at auction**, the auction is one in which the losers pay too, and the auction proceeds go to the block producer.

MEV, and why it is a problem for the consensus layer

[Intermediate] Here comes the concept the paper named. "OO fees represent one case of a more general quantifiable value we call **miner-extractable value (MEV)**. MEV refers to the total amount of Ether miners can extract from manipulation of transactions within a given timeframe, which may include multiple blocks' worth of transactions." The money that bots pay in these auctions is the price of **what the producer could have done directly**. "To capture OO fees, a miner can reorder users' transactions and potentially insert their own, reaping profit in Ether directly to their account. For example, a miner could execute the pure revenue transactions described in this work themselves, while still claiming the PGA fees for all the 'losing' arbitrage bots attempting to do the same."[10]

[Intermediate] And the paper's conclusion ties this chapter to Part 1. "In a stable blockchain, block rewards incentivize honest miner behavior. As we show empirically in this section, though, order optimization (OO) fees … **can exceed the block reward** and instead incentivize forking attacks." It gives two attacks. One is "a previously shown undercutting attack [11] that forks a block with significant MEV" (first analyzed by the paper "On the Instability of Bitcoin Without the Block Reward," which Chapter 20 takes up); the other is a newly named attack, the **time-bandit attack**. "Suppose a blockchain has a suffix (subchain) [height0, height1], with current block height height1, in which stealable value exceeds block rewards. An adversary can rewind to height0 and use the resulting MEV to **subsidize** a profitable 51% attack that mines a fork up to or past height1."[11] Chapter 9 said that rewinding grows costlier as blocks pile up. What Chapter 17 adds is that if **something to be had** from rewinding lies within the order of transactions, an incentive arises to bear that cost. In the paper's words, "a key result of our work is that application-layer security poses a current and direct threat to consensus-layer security."[12] The cost side of the calculation goes to Chapter 20, and the strategy side to Chapter 23.

> **? A LAWYER'S QUESTION**
>
> The sandwich attacker did not change a single character of the victim's order. It only placed two orders of its own in front of and behind the victim's, and all three were executed as the code dictates. Is this market manipulation (시세조종)? Is there a "market price" (시세) that was manipulated: can the ratio of two quantities in a pool be called a market price? Who is the "actor" who manipulated it: the bot operator who placed the orders, the producer who took that order and put it in a block, or the relaying party that sold that order at auction? Where are the "market" and the "exchange" that regulation presupposes? Do the grounds for prohibiting front-running in traditional markets (fiduciary duty (신인의무), the abuse of information asymmetry) still stand where there is no intermediary? And if the producer ran the sandwich **itself**, what did it do in its own block? → Chapter 28

4. The response, not prohibition but a division of labor: proposer–builder separation

A law prohibiting front-running cannot be put into code. Code does not know which transaction is the "victim" and which is the "attack," and the power to set the order is of the essence of block production. What Ethereum researchers took aim at was a different problem: **concentration**. Buterin's research post of June 2021: "A major risk threatening the ongoing decentralization of consensus networks is the economics around miner extractable value (MEV), sophisticated tricks to extract profit from the ability to choose the contents of the next block. … While normal PoS rewards are reasonably egalitarian, as single validators earn the same rate of return as powerful pools, there are **significant economies of scale** in finding sophisticated MEV extraction opportunities. A pool that is 10x bigger will have 10x more opportunities to extract MEV but it will also be able to spend much more effort on making proprietary optimizations to extract more out of each opportunity."[13] Chapter 10's question, "is PoS a structure in which the rich get richer?", finds a new ground here. Rewards stay proportional, but **the money extracted from the order** goes more to the larger side.

The proposal is to split the roles. "The best-known solution is **proposer/block-builder separation**. Instead of the block proposer trying to produce a revenue-maximizing block by themselves, they rely on a market where outside actors that we call block-builders produce bundles consisting of complete block contents and a fee for the proposer, and the proposer chooses the bundle with the highest fee. The proposer's choice is reduced to picking the highest-fee bundle, an algorithm so simple that in a decentralized pool it can even be done inside an MPC to prevent cheating."[14] The key is that **the proposer chooses without seeing the block contents**. One of the five desirable properties the post lists is "Bundle un-stealability: proposers should not be able to take bundles proposed by block builders and extract transactions from them to make their own bundles …."[15] The design does not abolish MEV; it **concentrates the work of extracting MEV in specialist builders** and has every proposer receive the payment for it through an auction. Ethereum's official documentation states the same point: "The block builders might have done sophisticated MEV extraction, but the reward for it goes to the block proposer. This means that even if a small pool of specialized block builders dominate MEV extraction, the reward for it could go to any validator on the network, including individual home stakers."[16]

[Intermediate] Before this design went **inside the protocol**, an implementation running outside it appeared first. A post from the Flashbots side in November 2021 sets out the design: "This document outlines the design for a marketplace for block construction (often referred to as block proposer / block builder separation or PBS) compatible with the upcoming Ethereum merge fork. This **trust based** solution closely resembles the current Flashbots auction design with modifications to enable solo staker participation without introducing changes to Ethereum consensus." The structure runs users and searchers → builders → **relays** → proposers, and "Validators receive execution payload **headers** from relays (execution payloads stripped of the transaction content). … The validator selects the most valuable header, signs the payload, and returns it to the relay …." "Since the execution payload header

does not include the transaction content, the risk of validators frontrunning is eliminated. … However, this design also requires validators to **trust the relay** to filter invalid payloads, accurately report payload value, and reveal the transaction content once the header is signed."[17] The software is called MEV-Boost, and the README of its repository calls it "open source middleware run by validators to access a competitive block-building market … an **initial implementation** of proposer-builder separation (PBS) for proof-of-stake (PoS) Ethereum," and writes that "In the future, proposer/builder separation will be enshrined in the Ethereum protocol itself …."[18]

So the picture **as of 2026-10** is this. Inside the protocol there are only proposers; there are no builders or relays. The division of labor is run by middleware and relay operators outside the protocol, and Ethereum's official roadmap page writes: "PBS is in an advanced stage of research, but there are still some important design questions that need to be resolved before it can be prototyped in Ethereum clients. **There is no finalized specification yet.**"[19] This sentence dates quickly, so it is entered in the Trend notes in Appendix A and checked again with each edition. The same page writes that PBS could also be used for censorship resistance (a design in which proposers use "inclusion lists" to force in transactions that builders left out), but it says that this too is "an area of active research."

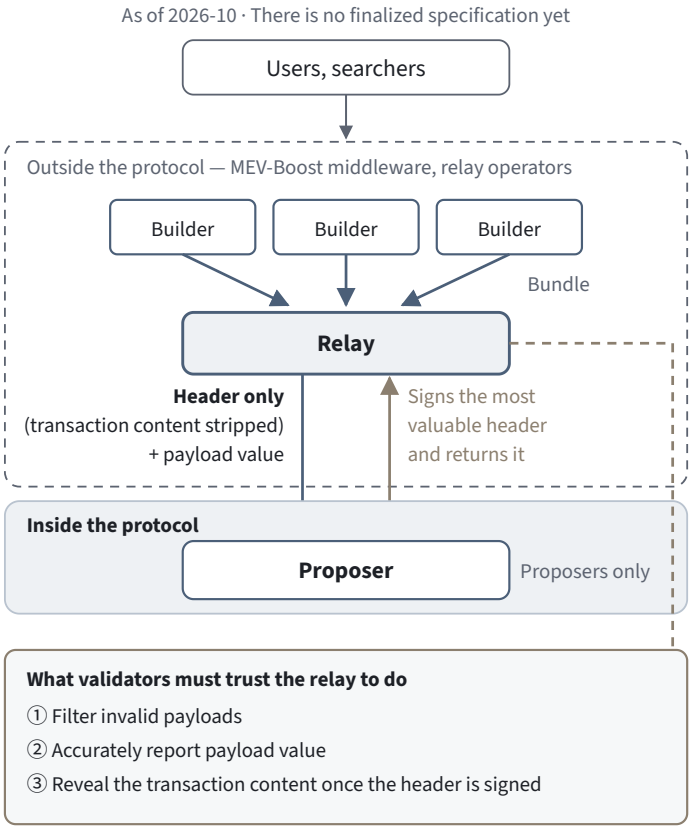


Figure 17-3. The division of labor under proposer–builder separation as of 2026-10. Inside the protocol there are only proposers; builders and relays run outside the protocol (MEV-Boost middleware), and the proposer signs a header stripped of the transaction contents. There is no finalized specification yet.

Let us also record what the division of labor does not change. Sandwiches still happen. What has changed is **who shares the profit from them**. The builder extracts it, the proposer receives it through the auction, and the user still gets a worse price. The other roads for

protecting users (sending transactions straight to a builder instead of to the public queue, queues with encrypted transaction contents, and the "fair sequencing services" proposed in the Chainlink 2.0 white paper[20]) are each designs that trust someone else, and this book evaluates none of them. How the sequencer of a layer-2 rollup handles this problem differently, by keeping its own queue private, is taken up in Chapter 18.

5. Code: running the sandwich in pool units

Three transactions in turn: front, victim, back

[Intermediate] The pool class of Chapter 15 is written out again as it is (so that the notebook runs on its own), and three transactions are put in in order. Units are all counts of tokens in the pool, and "attacker profit" is used **only to show the structure**. These figures are not values from any real market.

```
# NB17-1: the sandwich — take Chapter 15's constant-product pool as it is and put in three
transactions in order, "front, victim, back" (for study, in pool units)
import math

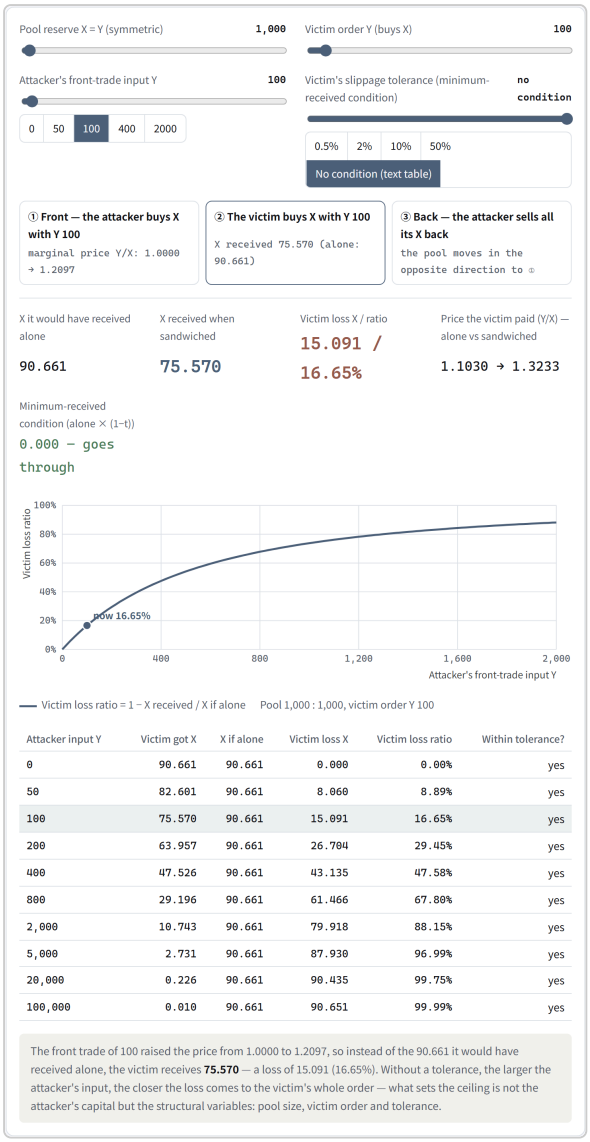
class ConstantProductPool:                                  # same class as NB15-1 in Chapter 15
(swap part only)
    def __init__(self, x, y, fee=0.003):
        self.x, self.y, self.fee = float(x), float(y), fee
    @property
    def price_x_in_y(self): return self.y / self.x
    def swap_x_for_y(self, dx):
        eff = dx * (1 - self.fee); dy = self.y * eff / (self.x + eff)
        self.x += dx; self.y -= dy; return dy
    def swap_y_for_x(self, dy):
        eff = dy * (1 - self.fee); dx = self.x * eff / (self.y + eff)
        self.y += dy; self.x -= dx; return dx

def sandwich(pool_x, pool_y, victim_y, attacker_y):
    """The victim buys X with Y victim_y. The attacker, in front of it, buys X with Y attacker_y
and, behind it, sells all of that X back.
    Returns: (Y the attacker got back - Y put in, X the victim received, X the victim would have
received alone)"""
    alone = ConstantProductPool(pool_x, pool_y).swap_y_for_x(victim_y)
    p = ConstantProductPool(pool_x, pool_y)
    front_x = p.swap_y_for_x(attacker_y) if attacker_y > 0 else 0.0     # ① front: the attacker
buys
    victim_x = p.swap_y_for_x(victim_y)                                 # ② the victim buys, at
the already raised price
    back_y = p.swap_x_for_y(front_x) if attacker_y > 0 else 0.0         # ③ back: the attacker
sells
    return back_y - attacker_y, victim_x, alone

print("Pool X 1,000 / Y 1,000. The victim buys X with Y 100.")
print(f"{'attacker in Y':>14} {'attacker net profit (Y)':>24} {'victim got X':>14} {'X if
alone':>12} {'victim loss X':>14} {'victim loss ratio':>18}")
for a in [0, 50, 100, 200, 400, 800, 2000, 5000, 20000, 100000]:
    profit, vx, alone = sandwich(1000, 1000, 100, a)
    print(f"{a:>14} {profit:>24.3f} {vx:>14.3f} {alone:>12.3f} {alone - vx:>14.3f} {1 - vx /
alone:>18.2%}")
```

```
# Output — bcbook (Python 3.13), exactly as run
Pool X 1,000 / Y 1,000. The victim buys X with Y 100.
 attacker in Y  attacker net profit (Y)   victim got X   X if alone  victim loss X  victim loss
ratio
             0                   0.000         90.661       90.661          0.000
0.00%
            50                   9.353         82.601       90.661          8.060
8.89%
           100                  17.368         75.570       90.661         15.091
16.65%
           200                  30.322         63.957       90.661         26.704
29.45%
           400                  48.047         47.526       90.661         43.135
47.58%
           800                  66.972         29.196       90.661         61.466
67.80%
          2000                  84.967         10.743       90.661         79.918
88.15%
          5000                  92.174          2.731       90.661         87.930
96.99%
         20000                  94.020          0.226       90.661         90.435
99.75%
        100000                  94.020          0.010       90.661         90.651
99.99%
```

[Intermediate] Read the table. The more the attacker puts in, the larger the victim's loss, and the attacker's net profit **approaches the 100 Y the victim put in but does not exceed it.** What the attacker takes is in the end what the victim paid, and putting in capital far larger than the victim's order leaves nothing more to take (this is why the net profit stops near 94 in the lower rows of the table; the rest stays in the pool as fees). So what binds this attack is not the attacker's capital. The **structural variables**, the size of the victim's order and the size of the pool, set the ceiling, and what binds the attacker hardest in practice is the variable in the next cell.



Attacker input Y	Victim got X	X if alone	Victim loss X	Victim loss ratio	Within tolerance?
0	90.661	90.661	0.000	0.00%	yes
50	82.601	90.661	8.060	8.89%	yes
100	75.570	90.661	15.091	16.65%	yes
200	63.957	90.661	26.704	29.45%	yes
400	47.526	90.661	43.135	47.58%	yes
800	29.196	90.661	61.466	67.80%	yes
2,000	10.743	90.661	79.918	88.15%	yes
5,000	2.731	90.661	87.930	96.99%	yes
20,000	0.226	90.661	90.435	99.75%	yes
100,000	0.010	90.661	90.651	99.99%	yes

Figure 17-4. Sandwich attack, the victim's side. In a 1,000 : 1,000 pool, a victim buying X with Y 100 would receive 90.661 alone, but receives 75.570 when an attacker's purchase of 100 is placed in front (a loss of 15.091, or 16.65%). The only output is the victim's loss. — In the HTML edition you can change the values here (HTML edition, Chapter 17, Section 5).

The slippage tolerance binds the attack

```
# NB17-2: the victim's slippage tolerance caps the size of the attack — add a "minimum amount
received" condition and find the attacker's best input (for study, in pool units)
def best_attack(pool_x, pool_y, victim_y, tolerance, fee=0.003, grid=2000):
    """Assume the victim attached to its transaction the condition "I must receive at least (1 -
tolerance) of the X I would have received alone."
    Among the attacker inputs Y that do not break that condition (= the victim's transaction is
not reverted), find by grid search the one with the largest net profit."""
    alone = ConstantProductPool(pool_x, pool_y, fee).swap_y_for_x(victim_y)
    min_out = alone * (1 - tolerance)
    best = (0.0, 0.0)                                          # (net profit, input)
    for i in range(1, grid + 1):
        a = pool_y * 2 * i / grid                              # sweep from 0 up to twice the
pool's Y
        profit, vx, _ = sandwich(pool_x, pool_y, victim_y, a)
        if vx < min_out:                                       # if the victim's trade fails,
the sandwich fails too (only the front trade remains, at the attacker's loss)
            break
        if profit > best[0]:
            best = (profit, a)
    return best, alone, min_out

print("Pool X 1,000 / Y 1,000. Victim order Y 100. The attacker's best input and net profit for
each slippage tolerance the victim attached to the transaction:")
print(f"{'tolerance':>9} {'victim min X':>16} {'attacker best input Y':>21} {'attacker net
profit (Y)':>24}")
for tol in [0.001, 0.005, 0.01, 0.02, 0.05, 0.10, 0.50]:
    (profit, a), alone, min_out = best_attack(1000, 1000, 100, tol)
    print(f"{tol:>9.1%} {min_out:>16.3f} {a:>21.1f} {profit:>24.3f}")
print("\nThe narrower the tolerance, the smaller the attack that can be slipped in (on this 1 Y
grid it is 0 at 0.1%). Net profit is nearly proportional to the tolerance, and the best input
grows faster than that.")
```

```
# Output — bcbook (Python 3.13), exactly as run
Pool X 1,000 / Y 1,000. Victim order Y 100. The attacker's best input and net profit for each
slippage tolerance the victim attached to the transaction:
tolerance     victim min X attacker best input Y  attacker net profit (Y)
     0.1%           90.570                   0.0                   0.000
     0.5%           90.208                   2.0                   0.404
     1.0%           89.754                   5.0                   1.004
     2.0%           88.848                  10.0                   1.992
     5.0%           86.128                  27.0                   5.234
    10.0%           81.595                  56.0                  10.380
    50.0%           45.331                 434.0                  50.317

The narrower the tolerance, the smaller the attack that can be slipped in (on this 1 Y grid it
is 0 at 0.1%). Net profit is nearly proportional to the tolerance, and the best input grows
faster than that.
```

If the victim attaches to its transaction a **narrow** condition, "cancel the trade if I would receive less than this" (a slippage tolerance), the attack the attacker can slip in shrinks accordingly (on the 1 Y grid of the table above it came out as 0 at 0.1%, but on a finer grid a small attack goes through). The narrower the condition, the greater the chance that one's own transaction fails, so the victim trades the risk of failure against the risk of a sandwich. This variable is in the victim's hands. This chapter's interactive figure W10 moves only four structural variables (pool size, victim order, attacker input, tolerance) and uses no currency units.

6. Summary

Transactions are public before they get into a block, the one who makes the block sets the order, and the order changes an AMM's price, so the order itself becomes money. Bots buy that order at auction (an auction in which the losers pay too), the producer can also do it directly, and when that money exceeds the block reward it becomes an incentive for rewinding attacks. Ethereum's response is not prohibition but a division of labor (PBS), and as of 2026-10 that division runs on middleware and relays outside the protocol, with no finalized specification. How the fee market dealt with this problem (the design intent that burning the base fee reduces the producer's incentive to manipulate fees) continues in Chapter 21, the calculation of attack costs in Chapter 20, and the equilibrium of strategies in Chapter 23. The last chapter of Part 2, Chapter 18, looks at designs that move transactions to another layer altogether. There too is someone who sets the order (the sequencer).

Sources for this chapter

- Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, Ari Juels, "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability," *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 910–927, DOI 10.1109/SP40000.2020.00040 — gossip and simulation (§III-C), front-running (§I, §II-D), atomicity (§III-A), the PGA and the all-pay auction (§III-C), observations and limits (§IV, §VI, Appendix C), the definition of MEV and forking attacks (§VII). Aggregate amounts were not carried over, following the principle in the notes to the reader in Chapter 3.
- Lorenz Breidenbach et al., "Chainlink 2.0," 15 April 2021 — the terms front-running, back-running and sandwich (§5.1), fair sequencing services (§5).
- Vitalik Buterin (vbuterin), "Proposer/block builder separation-friendly fee market designs," ethresear.ch, 2021-06-04. https://ethresear.ch/t/proposer-block-builder-separation-friendly-fee-market-designs/9725 ; the same author, "Two-slot proposer/builder separation," 2021-10-10. https://ethresear.ch/t/two-slot-proposer-builder-separation/10980 (accessed 2026-10-25, checked against the Discourse originals).
- Account thegostep, "MEV-Boost: Merge ready Flashbots Architecture," ethresear.ch, 2021-11-04. https://ethresear.ch/t/mev-boost-merge-ready-flashbots-architecture/11177 (accessed 2026-10-25).
- flashbots/mev-boost, README.md, commit 7a60459f (2025-12-05). https://github.com/flashbots/mev-boost/blob/7a60459f1e61ee4d3b256978cd34bef5fe5e95c6/README.md
- ethereum.org, "Proposer-builder separation," Page last update 2026-06-24. https://ethereum.org/en/roadmap/pbs/ (source commit 26bdbe8e; accessed 2026-10-25) — PBS and MEV, current progress (the "as of 2026-10" sentence, indexed in Appendix A).

- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library module `math`. Notebook NB17. The initial state of interactive figure W10

`sandwich` is NB17-1's pool of 1,000:1,000, victim order 100 and attacker input 100.

Notes

1. Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, Ari Juels, "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability," *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 910–927, §III-C (p. 914). Emphasis added. The 2019 arXiv preprint (arXiv:1904.05234) has a different subtitle; this book cites the published version.

2. Same paper, §I (p. 910). Original: "Worse still, adversaries can frontrun orders, observing them and placing their own orders with higher fees to ensure they are mined first."

3. Same paper, §II-D (p. 912). Emphasis added.

4. Same paper, §III-A (p. 913). Emphasis added. ("is possible" is as in the original.)

5. Lorenz Breidenbach et al., "Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks," 15 April 2021 (v1.0), §5.1 (p. 45). Emphasis added. Original: "Front-running exemplifies a class of transaction-ordering attacks: There are a number of related attacks such as back-running and sandwiching (front-running plus back-running) [237] that we don't cover here, but which FSS also helps address."

6. Daian et al. (2020), §III-C (p. 914). Original: "In Ethereum, the PGA game we observe consists of transactions issued with the same (account, nonce) pair, expressing a bid to a miner, where the miner is paid gas fees as described in Section II-A. We call this interaction of issuing repeated bids a priority gas auction, or PGA. To place a bid on an arbitrage opportunity, an arbitrageur simply issues a smart contract transaction with associated gas price g that atomically bundles multiple trades (performing the arbitrage). If they later wish to increase this bid, ... they simply reissue the transaction with the same nonce and a higher gas price g′. If two arbitrageurs are bidding against each other, what emerges is essentially an auction ...".

7. Same paper, same page. Original: "The transaction shown in the red square ... is also mined and included in the final block, as each (account, nonce) pair can include one transaction in the next block to be mined as per the protocol. Miners are incentivized to include even failed transactions, as these transactions pay for attempted execution. ... In game theoretic terms, each auction represents a variant of all pay auction, where instead of paying their full bid, the loser is forced to pay a percentage to the miner."

8. Same paper, §VI-A and §VI-D (pp. 918–920), Figs. 7 and 10. Original: "we have observed empirically that players are consistently and substantially reducing their respective latencies over time (Figure 7)"; "Remarkably as shown in Figure 10, in our observed PGAs, players converge over time to a minimum bid raise of 12.5%, which is the minimum allowed raise in the Parity client." On cooperation: "We do claim, however, that players are converging to a more cooperative state, that is approximated by our model ...".

9. Same paper, §IV-A1, Instrumentation Limitations (p. 915), and Appendix C (pp. 924–925). Original: "Our supported exchanges include the top five DEXes by sustained volume at the time of infrastructure development."; "Because we aim to establish a lower bound (i.e. a potentially too conservative result), we feel this is acceptable."; "of the 2,792 PGAs we observe, 75% (2,096) are played between 2 or less players and 91% are played between 3 or less players." The measurement period was 9 months (§IV-A), and the supported exchanges included the top five DEXes by volume at the time. Aggregate amounts were not carried over, following the principle in the notes to the reader in Chapter 3.

10. Daian et al. (2020), §VII (pp. 920–921). Emphasis added.

11. Same paper, §VII (p. 921) and §VII-C (p. 922). Emphasis added. Original: "In a stable blockchain, block rewards incentivize honest miner behavior. As we show empirically in this section, though, order optimization (OO) fees, or implicit fees a miner is able to reap by leveraging their control of a consensus epoch, can exceed the block reward and instead incentivize forking attacks."; "Suppose a blockchain has a suffix (subchain) [height0, height1], with current block height height1, in which stealable value exceeds block rewards. An adversary can rewind to height0 and use the resulting MEV to subsidize a profitable 51% attack that mines a fork up to or past height1." The original source for the undercutting attack is reference [11] in the paper, Carlsten et al., CCS 2016, which is a primary source for Chapter 20.

12. Same paper, §VII (p. 920). Original: "In other words, a key result of our work is that application-layer security poses a current and direct threat to consensus-layer security."

13. Vitalik Buterin (account vbuterin), "Proposer/block builder separation-friendly fee market designs," ethresear.ch, 2021-06-04. Emphasis added. https://ethresear.ch/t/proposer-block-builder-separation-friendly-fee-market-designs/9725 (accessed 2026-10-25; checked again against the Discourse original).

14. Same post. Emphasis added.

15. Same post, "Desired properties." Original: "Bundle un-stealability: proposers should not be able to take bundles proposed by block builders and extract transactions from them to make their own bundles, preventing the block builder from earning a profit (and possibly harming them even further)". The other four are "untrusted proposer friendliness," "untrusted builder friendliness," "weak proposer friendliness" and "consensus-layer simplicity and safety." The same author's later design, "Two-slot proposer/builder separation" (ethresear.ch, 2021-10-10, https://ethresear.ch/t/two-slot-proposer-builder-separation/10980), makes this concrete as a two-slot structure.

16. ethereum.org, "Proposer-builder separation," roadmap page (Page last update: June 24, 2026). https://ethereum.org/en/roadmap/pbs/ (accessed 2026-10-25; source repository ethereum/ethereum-org-website, last commit to the file 26bdbe8e, 2026-06-24).

17. Account `thegostep` (real name not verified; the Discourse display name is empty), "MEV-Boost: Merge ready Flashbots Architecture," ethresear.ch, 2021-11-04. Emphasis added. Original: "This document outlines the design for a marketplace for block construction (often referred to as block proposer / block builder separation or PBS) compatible with the upcoming Ethereum merge fork. This trust based solution closely resembles the current Flashbots auction design with modifications to enable solo staker participation without introducing changes to Ethereum consensus."; "Validators receive execution payload headers from relays (execution payloads stripped of the transaction content). The relay must attach an indication of the payload value to each header. The validator selects the most valuable header, signs the payload, and returns it to the relay and escrow to be propagated to the network."; "Since the execution payload header does not include the transaction content, the risk of validators frontrunning is eliminated. … However, this design also requires validators to trust the relay to filter invalid payloads, accurately report payload value, and reveal the transaction content once the header is signed." https://ethresear.ch/t/mev-boost-merge-ready-flashbots-architecture/11177 (accessed 2026-10-25).

18. flashbots/mev-boost, README.md (commit 7a60459f, 2025-12-05). Emphasis in the text added. Original: "`mev-boost` is open source middleware run by validators to access a competitive block-building market. MEV-Boost is an initial implementation of proposer-builder separation (PBS) for proof-of-stake (PoS) Ethereum."; "Relays aggregate blocks from **multiple** builders in order to select the block with the highest fees."; "In the future, proposer/builder separation will be enshrined in the Ethereum protocol itself to further harden its trust model." https://github.com/flashbots/mev-boost/blob/7a60459f1e61ee4d3b256978cd34bef5fe5e95c6/README.md (accessed 2026-10-25).

19. ethereum.org, "Proposer-builder separation," "Current progress" (Page last update: June 24, 2026). Emphasis added. On censorship resistance: "inclusion lists can be introduced so that when validators know about transactions but don't see them included in blocks, they can impose them as must-haves in the next block. … This is still an area of active research and the optimal configuration for the inclusion lists has not yet been determined." (Accessed 2026-10-25.)

20. Breidenbach et al. (2021), §5, Fair Sequencing Services (pp. 43–57). Original: "An overarching goal of the FSS in DONs is to empower DeFi creators to realize fair financial systems, that is, systems that don't advantage particular users (or miners) over others on the basis of speed, insider knowledge, or ability to perform technical manipulation. While a crisp, general notion of fairness is elusive, and perfect fairness in any reasonable sense is unachievable, FSS aims to provide developers with a powerful set of tools …". This book records this proposal only as one type of design.

PART 2 · SMART CONTRACTS AND APPLICATIONS

Chapter 18. Scalability: layer 2 and rollups

KEY TAKEAWAY

A blockchain is secure because every node recomputes every transaction in exactly the same way. For the same reason it is slow. Adding nodes does not raise throughput, because everyone is doing the same work. Making blocks bigger raises throughput, but running a node then takes a more powerful machine, and fewer people take part. This tension is often called the "scalability trilemma," and the name is **a design heuristic, not a theorem.** The path Ethereum took is **rollups.** The **execution** of transactions happens on an outside chain (layer 2), and only the transaction **data** and a **summary** of the results are written to the main chain (layer 1). The head office's ledger records only the totals and the branches handle the details, but when a dispute arises it is settled at the head office. There are two ways of "settling." An optimistic rollup "treats a result as correct until someone proves it wrong" and allows a challenge period, and a zero-knowledge rollup "attaches a mathematical proof of correctness every time." After this chapter you will see that layer 2 is not "a faster blockchain" but **a structure that entrusts the data to the main chain and splits off execution**, and that inside that structure there is again someone who "decides the order" (the sequencer).

Learning objectives

1. Explain why throughput is bounded by "capacity per block ÷ block interval," and why making blocks bigger is not free.
2. Distinguish the three things a rollup posts to the main chain (transaction data, a state summary, forced-inclusion messages) and what each one guarantees.
3. [Intermediate] Compare how the challenge period and interactive fraud proof (bisection) of optimistic rollups and the validity proof of zero-knowledge rollups differ on the question "who has to be honest?"
4. Recognize the "trilemma" as a design heuristic, and identify **where rollups moved trust to** rather than eliminating it (the sequencer, upgrade authority, the data availability layer).

What a reader of the [Basic] paragraphs alone takes away. Layer 2 is a structure in which "the head office's ledger records only the totals and the details are handled at the branches," and when a dispute arises it comes back to the head office.

1. What failure this technology grew out of

Chapter 13 said that every node executes the same transactions in the same order. That "everyone does the same work" is the bottleneck. The ZKsync documentation puts it concisely: "Ethereum's decentralized network has limited transaction throughput because its capacity doesn't scale with the number of validators. In essence, each validator performs the same job of validating each processed transaction, creating a bottleneck. In contrast, in Web2 scalability improves with the addition of more servers, as the network's capacity to handle requests increases linearly with each new server."[1] An ordinary server farm **divides** the work; a blockchain **replicates** it. Replication is the source of the security, and replication is the source of the slowness.

Throughput is bounded by simple arithmetic. The upper limit is the amount that fits in one block divided by the time between blocks. Bitcoin divides a block weight of 4,000,000 units by a target interval of 600 seconds. Ethereum divides the block gas limit by a slot interval of 12 seconds.[2] The gas limit is not a protocol constant but a value that validators move a little with each block, and the value observed on this book's reference date was 60,000,000 (**as of 2026-10**; Chapter 21 and Appendix A).[3] That makes 5,000,000 gas per second the upper limit, and if we assume that only the simplest transactions of Chapter 13 (intrinsic gas 21,000) come in, about 238 transactions per second is the theoretical upper limit. This calculation is this book's own, and it is the upper limit "when blocks are filled entirely with the simplest possible transactions," not measured throughput. The easiest way to raise that limit is to enlarge the gas limit (the block size), but then every node has to compute more and store more for every block. That is the constraint the official Ethereum documentation sets out: "While speed and throughput are important, it is essential that scaling solutions enabling these goals **remain decentralized and secure**. Keeping the barrier to entry low for node operators is critical in preventing a progression towards centralized and insecure computing power."[4]

This tension among throughput, decentralization, and security is often called the **"scalability trilemma."** The name needs care. It is **not a proven theorem** that "you can have only two of the three." It is a **heuristic** in which designers sum up their experience that the three goals pull against one another. The three pages of official documentation that this book checked against the source (the "Scaling" page above and the two rollup pages below) list the three goals but do not use the word "trilemma." A claim that some design has obtained all three "at once" depends on how each of the words is measured, and this chapter does no such measuring. Instead it looks **only at the structure** Ethereum chose. The official documentation sums up that choice as follows: "For a long time, sharding the blockchain was expected to scale Ethereum. … However, **scaling by layer-2 rollups has taken over as the primary scaling technique.**"[5]

2. The structure of a rollup: execution outside, data and disputes inside

Here is the one-line definition in the official documentation: "Rollups perform transaction execution outside layer 1 and then the data is posted to layer 1 where consensus is reached. As transaction data is included in layer 1 blocks, this allows rollups to be secured by native Ethereum security."[6] There are two components: "**Onchain contracts**: The optimistic rollup's operation is controlled by smart contracts running on Ethereum. This includes contracts that store rollup blocks, monitor state updates on the rollup, and track user deposits. … **Offchain virtual machine (VM)**: Although contracts managing the optimistic rollup protocol run on Ethereum, the rollup protocol performs computation and state storage on another virtual machine separate from the Ethereum Virtual Machine. The offchain VM is where applications live and state changes are executed …."[7]

The most important distinction in this chapter is **what** a rollup posts to the main chain. A table in the OP Stack documentation, on the Optimism side, divides it into three: "Three different things go to the parent chain, and conflating them is the most common source of confusion about rollup security."[8]

WHAT IS POSTED	WHERE	WHAT IT BUYS
Transaction data	Blobs or calldata	Anyone can **reconstruct** the chain without the sequencer (derivable)
Deposits (forced-inclusion messages)	Deposit contract	The main chain can **force** transactions into layer 2 (censorship resistance)
Output roots (state summary)	Dispute game contract	The main chain can **authenticate** layer 2 state and process withdrawals (settleable)

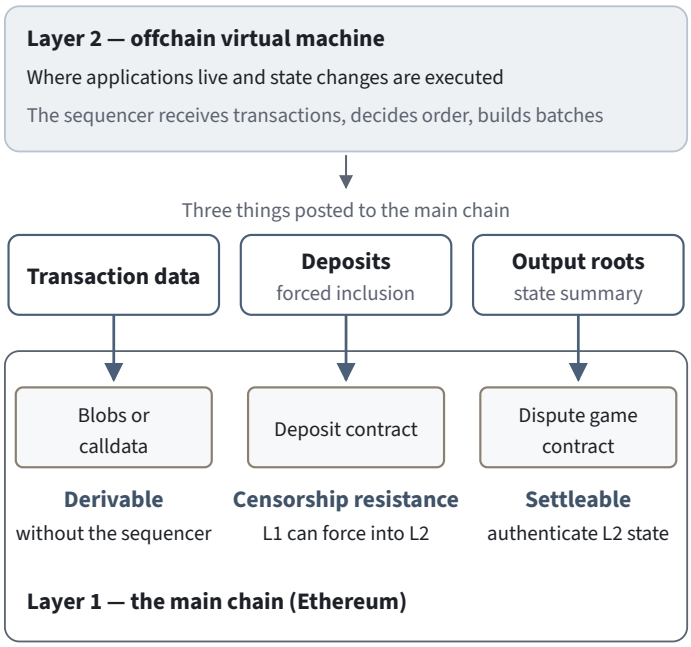


Figure 18-1. The three things a rollup posts to layer 1 and what each one buys. The three rows of the table above are redrawn as paths running down from layer 2 to layer 1.

A sentence the same document adds links back to Part 1: "Transaction data is what makes the chain **derivable**. Output roots are what make it **settleable**. A chain that published data but no output roots would still be a chain anyone could verify, but no one could withdraw from it. A chain that published output roots but no data would be unverifiable, and its proposals uncheckable." (Emphasis added.) This has the same shape as the Merkle root of Chapter 6. A summary (the root) is small, and to check that the summary is true **the original data has to be somewhere.** That is why "data availability" becomes the central term of rollup design. The ZKsync documentation: "If the state of the rollup is unknown to observers of Ethereum, then in scenarios where the validators (centralized or decentralized) stop processing, it becomes impossible to make state transitions without relying on a trusted validator. However, if the data is always available to observers, it's feasible to restore the state and continue processing the network even if the trusted validator ceases its operation."[9]

[Intermediate] There are two places to post data: a transaction's `calldata` (the transaction data field of Chapter 13, which stays in the chain's history permanently) and **blobs**. A blob is a data space, added by the Dencun fork (EIP-4844) in March 2024, that is "not stored by the execution layer." Blobs are priced in a separate fee market, and "the protocol defines only a **minimum window** of roughly 18 days over which the network must serve blob data. That is a serving obligation rather than a deletion deadline: blob data is not permanent archival storage the way execution-layer history is, and longer-term access depends on archival services."[10] That "roughly 18 days" comes from `MIN_EPOCHS_FOR_BLOB_SIDECARS_REQUESTS: 4096` epochs in the consensus-layer configuration (32 slots per epoch × 12 seconds = 6.4 minutes; 4,096 epochs ≈ 18.2 days) (**as of 2026-10**, Appendix A).[11] Chapter 13 showed why writing to storage is costly. Blobs are a design that sets aside a separate place, "one that not every node keeps permanently," so that rollup data can be posted at low cost, and the price of that design is a new question: "who is keeping it after 18 days?"

3. Optimistic rollups: correct until proven wrong

The first approach is "optimistic." "Optimistic rollups are considered 'optimistic' because they **assume** offchain transactions are valid and don't publish proofs of validity for transaction batches posted onchain. … Optimistic rollups instead rely on a **fraud-proving scheme** to detect cases where transactions are not calculated correctly. After a rollup batch is submitted on Ethereum, there's a time window (called a **challenge period**) during which anyone can challenge the results of a rollup transaction by computing a fraud proof." If the challenge succeeds, "the rollup protocol re-executes the transaction(s) and updates the rollup's state accordingly. … the sequencer responsible for including the incorrectly executed transaction in a block receives a penalty." If the period elapses without a challenge, the batch "is deemed valid and accepted on Ethereum."[12] The slashing of Chapter 10 and the "it hardens only as time passes" of Chapter 9 come back into use here. The difference is that in Chapter 9 the hardening was a probability, and here it is **a clock called the challenge period.**

The place where users see that clock is **withdrawal.** "If a user initiates an L2 > L1 transaction to withdraw funds escrowed on L1, they must wait until the challenge period—**lasting roughly seven days**—elapses." (**as of 2026-10**, Appendix A)[13] Inside layer 2 things are fast, but going back to the head office carries a challenge period. To avoid waiting there is a route in which "A liquidity provider assumes ownership of a pending L2 withdrawal and pays the user on L1 (in exchange for a fee)," and that means trusting yet another counterparty.

How does a fraud proof work? Bisection

[Intermediate] How do you prove that something "is wrong"? Rerunning all the transactions on the main chain would cost too much. So an interactive **bisection** is used. The Arbitrum documentation explains it most clearly: "The idea of interactive proving is that Alice and Bob will engage in a back-and-forth protocol, mediated by an L1 contract, to resolve their dispute with minimal work required from the L1 contract. Arbitrum's approach is to **dissect** the dispute. If Alice's claim covers N steps of execution, she posts two claims of size N / 2, which combine to yield her initial N-step claim, then Bob picks one of Alice's N / 2-step claims to challenge. Now the size of the dispute has been cut in half. This process continues, cutting the dispute in half at each stage, until they are disagreeing about **a single step of execution.** Note that so far, the L1 mediator hasn't had to think about execution 'on the merits.' It is only once the dispute is narrowed down to a single step that the L1 mediator needs to resolve the dispute by looking at what the instruction actually does and whether Alice's claim about it is correct."[14] A dispute over N steps shrinks to a single step in $\log_2 N$ round trips. It is the same mathematics by which the Merkle proof of Chapter 6 proved inclusion with $\log_2 n$ hashes.

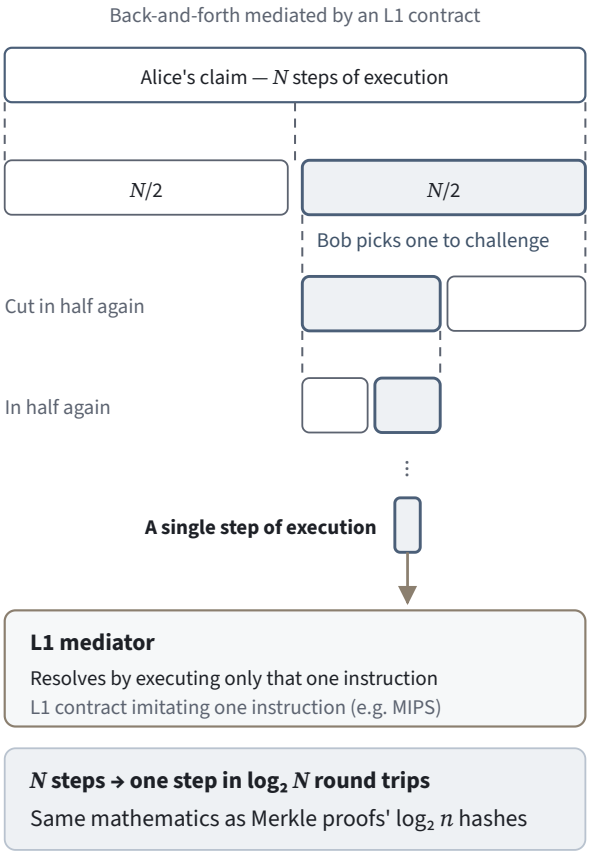


Figure 18-2. Bisection in an interactive fraud proof. The claim is cut in half until it reaches a single step of execution, and then the L1 mediator executes only that one instruction. There are $\log_2 N$ round trips.

[Intermediate] The OP Stack specification describes the same device as three components: "A fault proof ... consists of 3 components: Program: given a commitment to all rollup inputs (L1 data) and the dispute, verify the dispute statelessly. VM: given a stateless program and its inputs, trace any instruction step, and prove it on L1. Interactive Dispute Game: bisect a dispute down to a single instruction, and resolve the base-case using the VM."[15] "[P]rove it on L1" means placing **a contract on layer 1** that imitates one instruction of the layer 2 virtual machine (for example, a single MIPS instruction) and running just that one step on the main chain. In effect, a contract that reproduces one instruction of a different machine sits on top of the EVM of Chapter 13.

[Intermediate] The security claim of this design comes down to a single line: **one honest person is enough.** The Arbitrum documentation: "The key security property of the rollup protocol is that **any one honest validator** can force the correct execution of the chain to be confirmed. This means that executing an Arbitrum chain is as trustless as executing an Ethereum chain. You, and you alone (or someone you hire), can ensure your transactions are processed correctly. And this is true no matter how many malicious people are trying to stop you." It also says that a dispute protocol called BoLD has attached **a time bound** to that guarantee: "a single honest party can force confirmation of the correct outcome **with a known, fixed amount of time**, regardless of how many adversaries participate or how much they place as bond."[16] The official documentation says the same: "the validity of the chain relies on the existence of ***one* honest node**. ... If there are no honest nodes a malicious

operator can steal funds by posting invalid blocks and state commitments."[17] Compare this with the "2/3" of Chapter 10. An optimistic rollup runs not on a majority vote but on **one person's challenge.** The assumption is that this one person is actually awake and has the bond and the skill to raise a challenge.

[Intermediate] One more point. What a fraud proof overturns is **a claim, not the chain.** The OP Stack documentation: "The fault proof system governs only the third row: it decides whether a claim about L2 state is accepted on L1, and it does not reorder the L2 chain. A defeated proposal removes a claim about the chain; it does not roll the chain back."[18] The moment the transaction data is posted to the main chain, the order has hardened ("hard finality" in the Arbitrum documentation), and a fraud proof deals only with whether **the results computed** from that data are correct.

4. Zero-knowledge rollups: a proof of correctness attached every time

The second approach does not wait. "**Zero-knowledge rollups (ZK-rollups)** bundle (or 'roll up') transactions into batches that are executed offchain. ... ZK-rollup operators submit a **summary of the changes** required to represent all the transactions in a batch rather than sending each transaction individually. They also produce **validity proofs** to prove the correctness of their changes. The ZK-rollup's state is maintained by a smart contract deployed on the Ethereum network. To update this state, ZK-rollup nodes must submit a validity proof for verification. ... the validity proof is a cryptographic assurance that the state-change proposed by the rollup is really the result of executing the given batch of transactions."[19] The **verifier contract** on the main chain checks the proof, and if the proof passes, the new state root becomes canonical. Instead of recomputing nearly everything, the main chain checks "a short proof that the computation was correct," and so "There are no delays when moving funds from a ZK-rollup to Ethereum because exit transactions are executed once the ZK-rollup contract verifies the validity proof."[20]

On the name, here is a warning the official documentation sets down. Because of the word "zero-knowledge," it is easy to misunderstand that transactions become **secret.** "ZK-rollups use **succinct** validity proofs to establish that offchain state transitions were computed correctly. Succinctness keeps proofs compact and verification efficient relative to the underlying computation, allowing Ethereum to verify state transitions without re-executing every transaction. A proof system may additionally provide zero knowledge, meaning its proofs reveal no information about private inputs beyond what follows from the public statement. **These are separate properties: using validity proofs for scaling does not, by itself, make a rollup's transactions confidential.**"[21] What rollups use from this technology is the property of being "short"; the property of "hiding" is a bonus, or goes unused. As Section 2 described, the data is made public on the main chain: "the published data is public. ... Publishing this data is what makes permissionless verification possible; it is not a privacy mechanism."

[Intermediate] There are two broad families of proof systems. SNARKs have "small proof sizes and constant-time verification," but they need a **trusted setup**: "creating a Common Reference String (CRS) is necessary," and "if information used to create public parameters

fall [sic] into the possession of malicious actors they may be able to generate false validity proofs." So several participants conduct a ceremony (an MPC ceremony) in which each adds randomness and destroys their own share, and "As long as one honest participant destroys their input, the security of the ZK-SNARK system is guaranteed." STARKs do without that setup and use "publicly verifiable randomness" instead, and they are considered secure against quantum computers, but "they produce larger proof sizes, which are more expensive to verify on Ethereum."[22] The mathematics of the proofs is outside the scope of this book (Appendix E does not cover it either). A lawyer needs to take away one thing: even "a proof of correctness" rests on **premises that have to be trusted.** In some systems, that one of the participants in the setup ceremony was honest; in every system, that the proof circuit translated the actual virtual machine **exactly.** The official documentation states that "Proving general-purpose EVM computation in circuits is more difficult and resource-intensive than proving simple computations …."[23]

[Intermediate] The ZKsync documentation counts four components: "Node Implementation: This component is responsible for receiving transactions from users and processing them. It maintains the off-chain state and handles the aggregation of transactions into batches as well as sends sealed batches onchain. ZK Circuits: These circuits are intricate mathematical constructs that represent verifiable computation logic. They are responsible for determining what can be verified as a valid proof. … Prover: The prover constructs the cryptographic proofs that attest to the correctness of the transactions processed off-chain. … Smart Contracts: These contracts are the on-chain component of the zkRollup. They are responsible for verifying the proofs submitted by the prover and updating the Ethereum blockchain's state accordingly."[24] Of the four, the prover needs heavy machinery. From the table of drawbacks in the official documentation: "Producing validity proofs requires specialized hardware, which may encourage centralized control of the chain by a few parties."[25] Where an optimistic rollup obtains decentralization from "anyone can challenge," a zero-knowledge rollup acquires a different concentration: "few people can produce the proofs."

5. Comparing the two approaches: who has to be honest?

The table rearranges the tables of pros and cons on two pages of the official documentation according to this book's criterion, "what has to be trusted."[26]

	OPTIMISTIC ROLLUP	ZERO-KNOWLEDGE ROLLUP
Basis for correctness	No one submitted a fraud proof within the challenge period	The verifier contract accepted the validity proof
What has to be trusted	**One honest node** is awake and able to challenge	The premises of the proof system (setup ceremony, correctness of the circuit) and its mathematics
Withdrawal to the main chain	Wait out the challenge period (roughly 7 days, as of 2026-10)	As soon as the proof is verified
Data posted to the main chain	All of the transaction data (because a challenger has to re-execute)	The minimal data needed to rebuild the state (original: "the minimal data required to rebuild the latest state")
Who can advance the chain	Any ordinary node (computing a fraud proof needs no special hardware)	Whoever can produce proofs (special hardware)
EVM compatibility	Easy (uses the EVM as it is, or something close to it)	Hard (the EVM has to be translated into circuits)
Common weakness	The sequencer can influence transaction ordering	Same

6. Trust does not disappear; it moves

The last row ties this chapter to Chapter 17. Both approaches have a **sequencer** that receives transactions, decides their order, and builds batches. The OP Stack documentation notes that "The sequencer's mempool is **private**, unlike L1 Ethereum's public one, which limits the MEV opportunities a public pending-transaction pool creates."[27] The front-running of Chapter 17 arose from transactions being visible in a public queue. If a single operator holds the queue privately, front-running by bots goes down, but **that operator decides the order.** The same document draws the boundary of trust in just those terms: "The trust boundary moves as a transaction ages. Before its data is published, a user is trusting the sequencer not to reorder or drop it. After, they are trusting the DA layer."[28] The Arbitrum documentation separates the same thing with two terms. The order the sequencer sends out in real time is **soft finality**, because "it depends on the Sequencer keeping its promises," and once that order has been posted to Ethereum in a batch and has hardened, it is **hard finality**.[29] The "finality" of Chapter 9 turns two-layered again here.

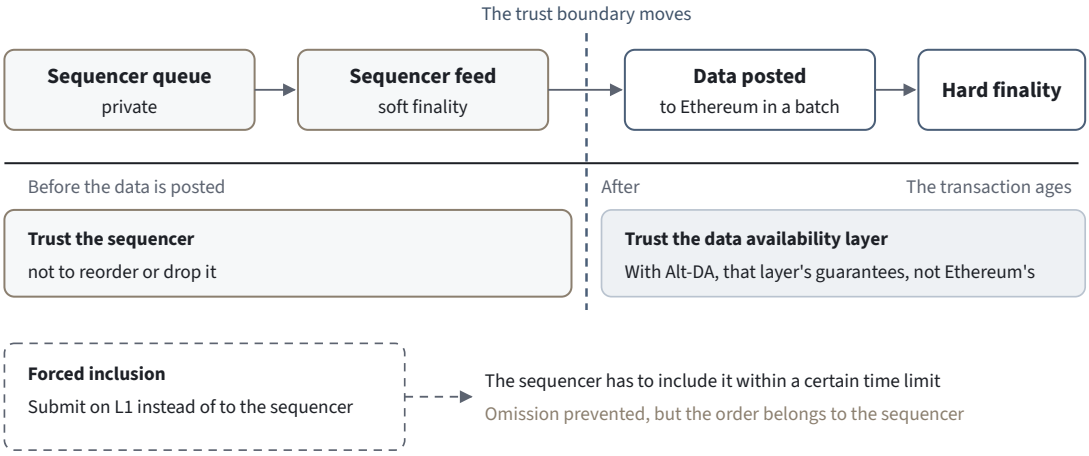


Figure 18-3. The boundary of trust moving as a transaction ages. Before the data is posted to Ethereum you trust the sequencer; after that, the data availability layer. Forced inclusion prevents omission, but the sequencer decides the order.

Against a sequencer **omitting** transactions there is a defense: the second row of the table in Section 2, forced inclusion. "Users can also submit their transactions on L1 instead of to the sequencer, in which case the sequencer has to include the transaction within a certain time limit to continue to produce valid blocks."[30] Omission can be prevented, but **the order** belongs to the sequencer. And if a chain chooses to replace the data availability layer with something other than Ethereum ("Alt-DA"), the documentation says that "the chain's security becomes a function of that layer's availability guarantees rather than Ethereum's," and adds that in that case "the claim that anyone can reconstruct the chain from Ethereum no longer holds."[31] A rollup has not reduced trust to 0. It is a design that **divides what is entrusted to the main chain from what the operator keeps in hand**, and the dividing line differs from rollup to rollup. **The current state** of things, such as how much a given rollup has entrusted to the main chain or who holds the upgrade keys, changes quickly, so it is not written in the text and is left to the Trend notes in Appendix A (with a reference date and sources).

7. Summary: closing Part 2

Rollups move execution outside and post the data and a state summary to the main chain. Optimistic rollups secure correctness with "one honest person's challenge" and a challenge period; zero-knowledge rollups secure it with "a short proof of correctness" and the premises of the proof system. In both, the sequencer decides the order, and in both, security depends on where the data is. The "trilemma" is a name for that tension, not a theorem.

Looking back over Part 2: Chapter 13 covered a ledger that executes code and the gas that stops it; Chapter 14, the ledgers (tokens) on top of it and what those ledgers do not say; Chapter 15, a pool that swaps tokens without people; Chapter 16, the fact that the chain knows nothing of the outside; Chapter 17, the power to decide order; Chapter 18, the structure that moves execution to another layer. One thing recurred across the six chapters. What code enforces and what people have promised sit **on different layers**, and between them remain the places that the word "decentralized" could not erase (operators, oracles, sequencers, keys). Part 3 reads that structure again **through economics.** Chapter 19 asks why

a token that nothing backs can have a positive price, Chapter 20 asks how much it costs, per block, to keep all this secure, and Chapter 21 takes up the fee formula that Chapter 13 put off.

Sources for this chapter

- ethereum.org, "Scaling" (developer documentation). https://ethereum.org/en/developers/docs/scaling/ ; source github.com/ethereum/ethereum-org-website commit 31bb20f8 (2026-10-22). Accessed 2026-10-25 — the goals and constraints of scaling, from sharding to rollups, the definition of a rollup.
- ethereum.org, "Optimistic Rollups." https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/ ; source commit 3e3b0dec. Accessed 2026-10-25 — definition, components, data availability and censorship resistance, the challenge period and withdrawals, interactive proving, one honest node, table of pros and cons.
- ethereum.org, "Zero-knowledge rollups." https://ethereum.org/en/developers/docs/scaling/zk-rollups/ ; source commit 38c32517. Accessed 2026-10-25 — definition, validity proofs and the distinction between succinctness and zero knowledge, SNARKs and STARKs, zkEVM, the blob serving window, table of pros and cons.
- Optimism Docs, "OP Stack architecture." https://docs.optimism.io/op-stack/architecture (accessed 2026-10-25) — the three things posted and what they mean, the private mempool, the moving trust boundary, fault proofs do not roll the chain back, Alt-DA.
- OP Stack Specification, "Fault Proof," github.com/ethereum-optimism/specs commit b733da80. https://github.com/ethereum-optimism/specs/blob/b733da804eeadc597c690f16601091e97af1fc68/specs/fault-proof/index.md — the three components of a fault proof, the contract that verifies a single instruction.
- Arbitrum Docs, "Inside Arbitrum Nitro." https://docs.arbitrum.io/how-arbitrum-works/inside-arbitrum-nitro (accessed 2026-10-25) — interactive proving (bisection), one honest validator and BoLD's time bound, soft and hard finality.
- ZKsync Docs, "ZKsync protocol overview." https://docs.zksync.io/zksync-protocol/rollup (accessed 2026-10-25) — the throughput bottleneck, the data availability problem, the four components.
- ethereum/consensus-specs commit b834fa84: `configs/mainnet.yaml` L74, L118, L194, `presets/mainnet/phase0.yaml` L34 — slot interval, minimum epochs for serving blobs, slots per epoch.
- This book's `widgets/data.json` (as_of 2026-10-25) — Bitcoin block weight and target interval, Ethereum observed gas limit.
- There is no code in this chapter (and no notebook). Figures such as the current state of individual rollups or TVL are not written in the text but sent to the Trend notes in Appendix A. The three documentation sites (Optimism, Arbitrum, ZKsync) can change, so

access dates are attached, and the verbatim quotations were taken from the text retrieved on 2026-10-25.

Notes

1. ZKsync Docs, "ZKsync protocol overview," section "How and why it works?" https://docs.zksync.io/zksync-protocol/rollup (accessed 2026-10-25; a documentation site, so the content may change). This book cites the ZKsync documentation **as a literature source** on zero-knowledge rollup design and does not endorse or compare any service (notes to the reader in Chapter 3).

2. Bitcoin: Bitcoin Core v31.1 `src/consensus/consensus.h` L15 (`MAX_BLOCK_WEIGHT = 4000000`), `src/kernel/chainparams.cpp` L97–L98 (`nPowTargetSpacing = 10*60`); see the `bitcoin` entry in this book's `widgets/data.json` and Chapter 8. Ethereum slot interval: ethereum/consensus-specs `configs/mainnet.yaml` (commit b834fa84, 2026-10-23) L74 `SLOT_DURATION_MS: 12000` . https://github.com/ethereum/consensus-specs/blob/b834fa8411cb2152563e8835c5b72c28f7503e7a/configs/mainnet.yaml (accessed 2026-10-25). L118 of the same file carries a separate value, `SLOT_DURATION_MS_EIP8198: 10000` , which suggests that an amendment changing the slot interval is in preparation. But `EIP8198_FORK_EPOCH` at L69 of the same file is set to the maximum value (2^64−1, the same notation used for other inactive forks), which suggests that no activation epoch had been set on the reference date. This book has not verified the content of that amendment, so the text uses 12 seconds, the value in force on the reference date (Appendix A index).

3. This book's `widgets/data.json` , `ethereum.eip1559.observed_latest_block` : block 26,264,275 (observed 2026-10-24T18:23Z), `gas_limit` 60,000,000; the responses of three public RPC endpoints agreed. The gas limit is "not a protocol constant but a value that validators move with each block within ±1/1024 of the parent" (go-ethereum v1.17.10 `params/protocol_params.go` L26 `GasLimitBoundDivisor = 1024`). The canonical treatment of the constants is in Chapter 21.

4. ethereum.org, "Scaling," section "Scaling overview": "The main goal of scalability is to increase transaction speed (faster finality) and transaction throughput (higher number of transactions per second) without sacrificing decentralization or security."; "While speed and throughput are important, it is essential that scaling solutions enabling these goals remain decentralized and secure. Keeping the barrier to entry low for node operators is critical in preventing a progression towards centralized and insecure computing power." https://ethereum.org/en/developers/docs/scaling/ (source https://github.com/ethereum/ethereum-org-website/blob/31bb20f87637d0d5899bb5c871343c2981210f66/public/content/developers/docs/scaling/index.md , last commit 2026-10-22; accessed 2026-10-25). Emphasis added.

5. Same page, section "Onchain scaling": "For a long time, sharding the blockchain was expected to scale Ethereum. This was going to involve splitting the blockchain into discrete pieces (shards) to be verified by subsets of validators. However, scaling by layer-2 rollups has taken over as the primary scaling technique. This is supported by the addition of a new cheaper form of data attached to Ethereum blocks that is specially designed to make rollups cheap for users." Emphasis added.

6. Same page, section "Rollups."

7. ethereum.org, "Optimistic Rollups," section "How do optimistic rollups interact with Ethereum?": "**Onchain contracts**: The optimistic rollup's operation is controlled by smart contracts running on Ethereum. This includes contracts that store rollup blocks, monitor state updates on the rollup, and track user deposits. In this sense, Ethereum serves as the base layer or 'layer 1' for optimistic rollups. **Offchain virtual machine (VM)**: Although contracts managing the optimistic rollup protocol run on Ethereum, the rollup protocol performs computation and state storage on another virtual machine separate from the Ethereum Virtual Machine. The offchain VM is where applications live and state changes are executed; it serves as the upper layer or 'layer 2' for an optimistic rollup." https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/ (source commit 3e3b0dec; accessed 2026-10-25).

8. Optimism Docs, "OP Stack architecture," section "Where data lives, and what you trust": "Three different things go to the parent chain, and conflating them is the most common source of confusion about rollup security." Table: "Transaction data — Blobs or calldata at a DA address — Anyone can reconstruct the chain without the sequencer; Deposits — OptimismPortal — L1 can force transactions into L2; Output roots — DisputeGameFactory — L1 can authenticate L2 state for withdrawals." It continues: "Transaction data is what

makes the chain derivable. Output roots are what make it settleable. A chain that published data but no output roots would still be a chain anyone could verify, but no one could withdraw from it. A chain that published output roots but no data would be unverifiable, and its proposals uncheckable." https://docs.optimism.io/op-stack/architecture (accessed 2026-10-25; the old address /stack/rollup/overview redirects here; a documentation site, so the content may change). This book cites this document only as a literature source on optimistic rollup design.

9. ZKsync Docs, same page, section "The Data Availability (DA) Problem."

10. ethereum.org, "Zero-knowledge rollups," section "How ZK-rollups publish transaction data on Ethereum": "Since the Dencun upgrade introduced blob-carrying transactions (EIP-4844), rollups can also publish their data in blobs, which are priced in a separate fee market and are usually cheaper than `calldata`. Blobs are not stored by the execution layer, and the protocol defines only a minimum window of roughly 18 days over which the network must serve blob data. That is a serving obligation rather than a deletion deadline: blob data is not permanent archival storage the way execution-layer history is, and longer-term access depends on archival services. `calldata` remains available, and rollups differ in which they use." https://ethereum.org/en/developers/docs/scaling/zk-rollups/ (source commit 38c32517; accessed 2026-10-25). Emphasis added. The Dencun activation date (2024-03-13) is from the fork timeline in Chapter 10 (the comment on `DENEB_FORK_EPOCH` in consensus-specs `configs/mainnet.yaml`).

11. ethereum/consensus-specs `configs/mainnet.yaml` (commit b834fa84) L194 `MIN_EPOCHS_FOR_BLOB_SIDECARS_REQUESTS: 4096`; `presets/mainnet/phase0.yaml` L34 `SLOTS_PER_EPOCH: 32`; L74 `SLOT_DURATION_MS: 12000`. 4,096 × 32 × 12 seconds = 1,572,864 seconds ≈ 18.2 days (this book's calculation). Accessed 2026-10-25.

12. ethereum.org, "Optimistic Rollups," section "What is an optimistic rollup?": "Optimistic rollups are considered 'optimistic' because they assume offchain transactions are valid and don't publish proofs of validity for transaction batches posted onchain. ... Optimistic rollups instead rely on a fraud-proving scheme to detect cases where transactions are not calculated correctly. After a rollup batch is submitted on Ethereum, there's a time window (called a challenge period) during which anyone can challenge the results of a rollup transaction by computing a fraud proof. If the fraud proof succeeds, the rollup protocol re-executes the transaction(s) and updates the rollup's state accordingly. The other effect of a successful fraud proof is that the sequencer responsible for including the incorrectly executed transaction in a block receives a penalty. If the rollup batch remains unchallenged (i.e., all transactions are correctly executed) after the challenge period elapses, it is deemed valid and accepted on Ethereum." Emphasis added.

13. Same page, section "Exiting the rollup": "If a user initiates an L2 > L1 transaction to withdraw funds escrowed on L1, they must wait until the challenge period—lasting roughly seven days—elapses."; "To avoid waiting a week before withdrawing funds to Ethereum, optimistic rollup users can employ a **liquidity provider** (LP). A liquidity provider assumes ownership of a pending L2 withdrawal and pays the user on L1 (in exchange for a fee)." Emphasis in the first quotation added. The "roughly 7 days" is the description as of access on 2026-10-25 and may differ from rollup to rollup and from version to version.

14. Arbitrum Docs, "Inside Arbitrum Nitro," section "Resolving disputes using interactive fraud proofs — Interactive proving." https://docs.arbitrum.io/how-arbitrum-works/inside-arbitrum-nitro (accessed 2026-10-25; a documentation site). This book cites this document only as a literature source on interactive fraud proofs. Emphasis added.

15. OP Stack Specification, "Fault Proof," section "Overview" (commit b733da80): "A fault proof, also known as fraud proof or interactive game, consists of 3 components: Program: given a commitment to all rollup inputs (L1 data) and the dispute, verify the dispute statelessly. VM: given a stateless program and its inputs, trace any instruction step, and prove it on L1. Interactive Dispute Game: bisect a dispute down to a single instruction, and resolve the base-case using the VM." Section "Fault Proof VM": "a smart-contract to verify a single execution-trace step, e.g. a single MIPS instruction." https://github.com/ethereum-optimism/specs/blob/b733da804eeadc597c690f16601091e97af1fc68/specs/fault-proof/index.md (accessed 2026-10-25).

16. Arbitrum Docs, same page, section "Arbitrum Rollup protocol": "The key security property of the rollup protocol is that any one honest validator can force the correct execution of the chain to be confirmed. This means that executing an Arbitrum chain is as trustless as executing an Ethereum chain. You, and you alone (or someone you hire), can ensure your transactions are processed correctly. And this is true no matter how

many malicious people are trying to stop you. Under BoLD, this guarantee comes with a bounded delay: a single honest party can force confirmation of the correct outcome with a known, fixed amount of time, regardless of how many adversaries participate or how much they place as bond." Emphasis added.

17. ethereum.org, "Optimistic Rollups," section "Why fraud proofs matter" and the table of pros and cons: "This also relates to another security property of optimistic rollups: the validity of the chain relies on the existence of *one* honest node."; drawback in the table: "If there are no honest nodes a malicious operator can steal funds by posting invalid blocks and state commitments."; "Security model relies on at least one honest node executing rollup transactions and submitting fraud proofs to challenge invalid state transitions." Emphasis (bold) added; the italics are in the original.

18. Optimism Docs, "OP Stack architecture," section "Where data lives, and what you trust."

19. ethereum.org, "Zero-knowledge rollups," section "What are zero-knowledge rollups?": "**Zero-knowledge rollups (ZK-rollups)** bundle (or 'roll up') transactions into batches that are executed offchain. ... ZK-rollup operators submit a summary of the changes required to represent all the transactions in a batch rather than sending each transaction individually. They also produce validity proofs to prove the correctness of their changes. The ZK-rollup's state is maintained by a smart contract deployed on the Ethereum network. To update this state, ZK-rollup nodes must submit a validity proof for verification. As mentioned, the validity proof is a cryptographic assurance that the state-change proposed by the rollup is really the result of executing the given batch of transactions." Emphasis on "summary of the changes" and "validity proofs" added.

20. Same page: "There are no delays when moving funds from a ZK-rollup to Ethereum because exit transactions are executed once the ZK-rollup contract verifies the validity proof. Conversely, withdrawing funds from optimistic rollups is subject to a delay to allow anyone to challenge the exit transaction with a fraud proof."

21. Same page, section "Validity proofs." Emphasis added. On data publication: "Whichever mechanism a rollup uses, the published data is public. ... Publishing this data is what makes permissionless verification possible; it is not a privacy mechanism."

22. Same page, sections "ZK-SNARKs" and "ZK-STARKs": "For the ZK-SNARK protocol to work, creating a Common Reference String (CRS) is necessary: the CRS provides public parameters for proving and verifying validity proofs. The security of the proving system depends on the CRS setup; if information used to create public parameters fall into the possession of malicious actors they may be able to generate false validity proofs."; "As long as one honest participant destroys their input, the security of the ZK-SNARK system is guaranteed."; "ZK-SNARKs are popular for their small proof sizes and constant-time verification."; "ZK-STARKs are 'transparent', as they can work without the trusted setup of a Common Reference String (CRS). Instead, ZK-STARKs rely on publicly verifiable randomness"; "ZK-STARKs are also secure against quantum computers ... The downside to ZK-STARKs is that they produce larger proof sizes, which are more expensive to verify on Ethereum."

23. Same page, section "ZK-rollups and EVM compatibility": "Unlike optimistic rollups, ZK-rollups are not readily compatible with the Ethereum Virtual Machine (EVM). Proving general-purpose EVM computation in circuits is more difficult and resource-intensive than proving simple computations (like the token transfer described previously)."

24. ZKsync Docs, same page, section "ZKsync components": "Node Implementation: This component is responsible for receiving transactions from users and processing them. It maintains the off-chain state and handles the aggregation of transactions into batches as well as sends sealed batches onchain. ZK Circuits: These circuits are intricate mathematical constructs that represent verifiable computation logic. They are responsible for determining what can be verified as a valid proof. ... Prover: The prover constructs the cryptographic proofs that attest to the correctness of the transactions processed off-chain. These proofs can be verified later on Ethereum, ensuring that only valid transactions are accepted. Smart Contracts: These contracts are the on-chain component of the zkRollup. They are responsible for verifying the proofs submitted by the prover and updating the Ethereum blockchain's state accordingly."

25. ethereum.org, "Zero-knowledge rollups," table of pros and cons: "Producing validity proofs requires specialized hardware, which may encourage centralized control of the chain by a few parties."; "Hardware requirements may reduce the number of participants that can force the chain to make progress, increasing the risk of malicious operators freezing the rollup's state and censoring users."

26. ethereum.org, "Optimistic Rollups," section "Pros and cons of optimistic rollups"; "Zero-knowledge rollups,"

section "Pros and cons of ZK-rollups." The originals of the common weakness: optimistic rollup table, "Centralized rollup operators (sequencers) can influence transaction ordering."; zero-knowledge rollup table, "Centralized operators (sequencers) can influence the ordering of transactions.". Advantage of optimistic rollups: "Computing fraud proofs is open to regular L2 node, unlike validity proofs (used in ZK-rollups) that require special hardware."; advantage of zero-knowledge rollups: "Relies on trustless cryptographic mechanisms for security, not the honesty of incentivized actors as with optimistic rollups."

27. Optimism Docs, "OP Stack architecture," section "Life of a transaction": "The sequencer's mempool is private, unlike L1 Ethereum's public one, which limits the MEV opportunities a public pending-transaction pool creates. Ordering is decided by the execution client, not the rollup node." Emphasis added.

28. Same page, section "Where data lives, and what you trust."

29. Arbitrum Docs, "Inside Arbitrum Nitro," section "How the Sequencer publishes the sequence": "The feed represents the Sequencer's promise to record transactions in a particular order. If the Sequencer is honest and doesn't have a long downtime, this promise will be kept. ... This is 'soft finality' for transactions; it's 'soft' because it depends on the Sequencer keeping its promises."; "This is the final and official record of the transaction sequence. As soon as this Ethereum transaction reaches finality, the Layer 2 Nitro transactions it records will also have finality. ... This is 'hard finality.'"

30. ethereum.org, "Optimistic Rollups," section "Censorship resistance."

31. Optimism Docs, "OP Stack architecture," section "Topology options": "Custom data availability. An Alt-DA layer instead of Ethereum blobs. Cheaper, and the chain's security becomes a function of that layer's availability guarantees rather than Ethereum's. The feature is experimental, and with it the claim that anyone can reconstruct the chain from Ethereum no longer holds: only a commitment reaches L1". The same section describes the standard chain as one with Ethereum data availability, gas paid in ETH, and upgrades by governance.

PART 3 · ECONOMICS

Chapter 19. The functions of money and the "zero intrinsic value" equilibrium

KEY TAKEAWAY

Neither "it has no intrinsic value, so it will end up at zero" nor "the quantity issued is fixed, so it has value" is a conclusion of economics. The functions approach (medium of exchange, unit of account, store of value) is a checklist, not a proof of value, and a positive price for an object that pays no dividend holds as **one among several equilibria** in which the belief that "others will accept it later" fulfills itself. The three models read in this chapter (Garratt–Wallace, Schilling–Uhlig, Biais et al.) all assume or write down an equilibrium with a price of zero (Schilling–Uhlig through the backward self-elimination in their §7), say that positive-price equilibria exist **alongside** it, and say that even within the positive-price equilibria the price path is not pinned down to one. Listing equilibria is not a forecast. After this chapter, when you meet the sentence "virtual assets have no intrinsic value" in a judgment or a paper, you will be able to separate what it says in economic terms from what it does not.

Learning objectives

1. Check crypto-assets against the three functions of money (medium of exchange, unit of account, store of value), while knowing that the check is not a device for deciding whether something has value. Be able to use the distinction between inside money and outside money.
2. **[Intermediate]** Know what the equation of exchange $MV = PT$ says, and why it is hard to use as a tool for explaining prices when the velocity V cannot be observed.
3. **[Advanced]** Read in equations why, in the overlapping generations model (Garratt–Wallace), the zero-price equilibrium, the constant-price equilibrium and the sunspot-collapse equilibrium exist together; why, in the infinitely lived agents model (Schilling–Uhlig), the "fundamental pricing equation" becomes a martingale condition; and why, in the transactional benefits model (Biais et al.), the "fundamental value" depends on the price itself.
4. Distinguish what the three models say in common (multiplicity of equilibria, self-fulfilling beliefs) from what they do not say (which equilibrium is realized, the direction of the price).

What a reader of the [Basic] paragraphs alone takes away. Neither "it has no intrinsic value,

so it is a fraud" nor "it is scarce, so it has value" is a conclusion of economics. What economics says is that **both can be equilibria**, and the model does not settle which one is realized.

1. The question this chapter tries to answer

In Part 1 we looked at what a coin is, byte by byte. It is a chain of outputs linked by signatures (Chapter 5, Chapter 11), and rewriting that chain means redoing the proof of work (Chapter 7, Chapter 8). Nowhere in it is there any "value." Nor is there any promise from an issuer to give anything back. Yet people exchange it for goods. Why can they, and how long can that go on? The two common answers to this question run as follows. One is "there is no dividend and no redemption, so the fundamental value is zero, and the price will end up there." The other is "the cap on issuance is fixed in code, so it is scarce, and being scarce it has value."

Economics takes neither answer as it stands. The first is half right. A price of zero is **one** equilibrium. But there are also positive-price equilibria, and theory does not settle which of them is realized. The second answer is missing a premise. Scarcity supports value "if others accept it," but it does not create that "if they accept it." The world is full of scarce things that nobody accepts. In place of these two answers this chapter puts what economics has actually produced, namely **a listing of equilibria**. That listing is not a forecast, and nowhere does this chapter state a price.

2. The functions approach: a checklist, not a proof

When economics textbooks define money, the framework they use is three functions. A 2023 paper by Gary Gorton and Jeffery Zhang puts it this way: "money has several important properties, with the three most commonly stated ones being **a unit of account, a store of value, and a medium of exchange**." And they add a fourth: "The most obvious (yet not explicitly stated) property is that money must also satisfy **the no-questions-asked (NQA) principle**, which requires that the money be accepted in a transaction without due diligence on its value."[1] Documents on the legal side use the same framework. The 2017 report of the U.S. Securities and Exchange Commission (SEC) on The DAO quotes the definition of the Financial Action Task Force (FATF) in a footnote, defining "virtual currency" as "a digital representation of value that can be digitally traded and functions as: (1) a medium of exchange; and/or (2) a unit of account; and/or (3) a store of value, but does not have legal tender status ... in any jurisdiction."[2] The "and/or" catches the eye. The definition does not require all three functions; doing any one of them is enough.

Run crypto-assets through this framework and the result is no surprise. They are used as a medium of exchange, but narrowly; they are hardly used as a unit of account (their prices are quoted in other currencies); and as a store of value they fluctuate widely. This is a qualitative summary, and the measurements are Makarov–Schoar and Athey et al. in section 3 below. One 2015 paper runs this check, starting from the premise that "In order to serve as money Bitcoin should fulfil the roles of unit of account (Dwyer, 2014), means of payments and store of value," and then writes that the speculative side of Bitcoin overwhelms its other

functions, so that its price is at the mercy of market sentiment, and that this erodes its role as a unit of account.[3] But when the check is done, the original question remains. Doing the functions poorly and having to be priced at zero are different propositions. The functions approach is a checklist for deciding **what to call money**; it is not a device for deciding whether an object can have a positive price. To answer the question of price, another tool is needed.

Inside money and outside money

Before turning to that tool, we bring in one distinction. Rodney Garratt and Neil Wallace's 2018 paper opens its account of how to view bitcoin with this distinction: "Most economists distinguish between inside and outside money. Inside money is inside the economy in the sense that each unit is someone's asset and someone else's liability. ... Examples of inside money are checking accounts and grocery-store coupons. Outside money, in contrast, does not disappear when balance sheets are consolidated. Examples are gold coins under a gold standard and Federal Reserve notes since August 15, 1971 when Nixon closed the gold window. In terms of that dichotomy, bitcoin is best viewed as an outside money. In particular, the issuer of bitcoin makes no promise to redeem it for any other object."[4] This is exactly what we saw in Chapter 11. A UTXO is nobody's liability. So this chapter's question becomes "why does an object with no promise of redemption have a price?", and that is the same question economics has long asked about gold coins and fiat paper money.

The same paper states the premise of its model in one sentence: "people give up other things to get bitcoin only because **they think that others in the future will do the same**. In other words, ownership of bitcoin does not yield utility as might ownership of a Picasso, and is not an input into the production of other things as is farmland, a factory, or a 3D printer. Nor does such ownership entitle the owner to a dividend stream of other valuable objects." And it adds: "When it comes to bitcoin, there is no commodity value to ignore. You cannot even use it as wall paper—as some stories suggest was performed with German marks during the 1922–1923 hyperinflation."[5] If Hume and the founders of the quantity theory analyzed gold and silver money by **ignoring** its commodity value, bitcoin is the pure case in which there is nothing to ignore. On that premise, the question of the equilibrium price is asked.

3. [Intermediate] The equation of exchange and its limits

[Intermediate] The oldest framework for explaining prices by quantities is the equation of exchange. For the money stock M, the velocity V, the price level P and the volume of transactions T,

$$MV = PT$$

Reading the symbols: this is an **identity** saying that the number of times money changes hands in a period (V) times the money stock equals the value of what was traded in that period (PT). A numerical example: if 1,000 coins are each spent twice on average in a period and 4,000 units of goods are traded, the coin price of one unit of goods is $P = MV / T = 0.5$. The appeal of this equation for crypto-assets is plain. M is set by the protocol (Chapter 20) and T is written on the ledger, so it looks as if putting the two in would give the price.

[Intermediate] A 2019 paper by Linda Schilling and Harald Uhlig shows what this equation looks like inside a model. In their model all money is spent once every period, so they write that "the velocity of both currencies equals one. In a world with only one currency, classical quantity theory yields $y_t = D_t / P_t$," and that with two currencies the market-clearing condition becomes[6]

$$y_t = \frac{D_t}{P_t} + \frac{Q_t}{P_t} B_t \qquad (9)$$

Reading the symbols: y_t is the output of goods, D_t the quantity of dollars, P_t the dollar price level, B_t the quantity of bitcoins, and Q_t the dollar price of bitcoin. What stands out here is that there are **two** unknowns (D_t and Q_t). If the central bank moves D_t to target P_t, then as B_t grows on its fixed schedule, what happens to Q_t is not determined by this equation alone. One theorem the paper draws from this equation is of interest: "The block rewards are not a tax on Bitcoin holders but are financed through Dollar taxes imposed by the Dollar central bank." The logic is that when block rewards make B_t larger, satisfying equation (9) at the same P_t and Q_t requires D_t to shrink, and that shows up as a lump-sum tax on the dollar side.[7] The intuition taken up in Chapter 20, that "issuance dilutes holders," does not carry over as such to a model of currency competition. This conclusion, though, is tied to the model's premises (the two currencies are perfect substitutes in trade, and all money is spent every period).

[Intermediate] The limit of the equation of exchange is V. Because it is an identity, defining V as PT / M makes it always true, so its explanatory power comes from the **additional assumption** that V is stable. To test that assumption for crypto-assets you have to measure "economically meaningful transaction volume," and the volume written on the ledger is not that. A 2021 study by Igor Makarov and Antoinette Schoar built a database that groups addresses into entities and reports that "90% of transaction volume on the Bitcoin blockchain is not tied to economically meaningful activities but is the byproduct of the Bitcoin protocol design as well as the preference of many participants for anonymity," and that "Starting from 2015, 75% of real bitcoin volume has been linked to exchanges or exchange-like entities …."[8] When T wobbles that much, V is a residual, not a measurement. A 2016 model by Susan Athey and her coauthors says that "In steady state, exchange rates are determined by the ratio of transaction volume to the supply of Bitcoins," yet after checking it against price data the authors themselves write that it provides "mixed evidence about the ability of the model to explain prices." Their analysis of usage data then records the concentration, that "investors and infrequent users held the majority of Bitcoins," and the share of transaction volume taken by investors, and from this the authors draw the conclusion that "exchange rates are likely to be more sensitive to beliefs about the future of Bitcoin and less sensitive to current transaction volume for its own sake," with the parenthetical caveat "(though current volume is likely a signal of the future)".[9] We went in through the framework of quantities and came out through the framework of beliefs. The next section is the model of those beliefs.

4. The "zero intrinsic value" equilibrium: three models

We read three papers. The depth of the mathematics differs, but the shape of the conclusions is the same. Here is the gist first, without equations.

MODEL	AGENTS	SHAPE OF THE CONCLUSION
Garratt–Wallace (2018)	Generations that live two periods overlap	A zero-price equilibrium always exists. A constant positive-price equilibrium may also exist. There is also an equilibrium that "goes to zero some day, but on a day nobody knows." The day of the collapse cannot be predicted
Schilling–Uhlig (2019)	Agents live forever, alternating between consuming and producing	If both currencies are used and the price is uncorrelated with the pricing kernel, the dollar price of the coin **is a martingale** (Proposition 1); if there is a correlation, its sign sets the direction of the expected change (Corollary 1). Price fluctuations do not invalidate the medium-of-exchange function
Biais et al. (2023)	Generations that live two periods overlap, and the coin provides transactional benefits	The "fundamental value" is the present value of future transactional benefits, but because those benefits **depend on the price**, it has no anchor. Sunspots enter the price, and there are many equilibria

4.1 Overlapping generations: four questions

[Intermediate] Garratt and Wallace put bitcoin B, as an outside money, into Samuelson's two-period overlapping generations model. Each period N people are born. They receive perishable goods w^{young} when young and w^{old} when old; when young they give up goods to acquire coins, and when old they sell the coins to the next generation. If p_t^B is the goods price of the coin, consumption when young is $c_t^{young} = w^{young} - p_t^B b_t$ and when old $c_t^{old} = w^{old} + p_{t+1}^B b_t$ (equations (1) and (2)). Rather than map the whole set of equilibria, the authors ask **four questions**: "(i) Is there an equilibrium in which the value of bitcoin is constant and positive? (ii) Is there an equilibrium in which the value of bitcoin is always zero? (iii) Is there an equilibrium in which the value of bitcoin is zero at some known date T and is positive at some earlier dates? (iv) Is there an equilibrium in which the value of bitcoin is random in the following way: if it has been positive and constant at each date 1, 2, ..., t − 1, then with probability π the value is equal to that constant at t; otherwise, it is zero at t and thereafter."[10]

[Intermediate] The answers are as follows. **(i) is "maybe."** If the utility function and the income stream give an incentive to save when young (scenario (a) of Figure 1), there is a constant positive saving s^{B*}, and the price is set by $s^{B*} = p^B(B / N)$. Otherwise there is none. **(ii) is "yes."** "Suppose at any t, the young at t believe that bitcoin will be worthless at t + 1; that $p_{t+1}^B = 0$. Then, each young person chooses $b_t = 0$ at time t at any $p_t^B > 0$. (Why give up goods which can be consumed when young in order to acquire an asset that will be worthless when it is sold?) Because this conclusion holds for each t, there is an equilibrium in which $p_t^B = 0$ for all t." **(iii) is "no."** If the price is zero at T, nobody acquires the coin at $T - 1$, and by backward induction nobody acquires it before that either. **(iv) is a conditional "yes."** If π is close enough to 1, such an equilibrium exists.[11] Here (ii) and (iii) have to be read together. A price of zero is always an equilibrium, but "it goes to zero on a known date" is not an equilibrium. Of the four questions, the route by which a collapse enters an equilibrium is

(iv), that is, a collapse that comes **with a small probability every period**, and the authors add in their conclusion that generalizing the nature of the randomness makes the set of equilibria larger.

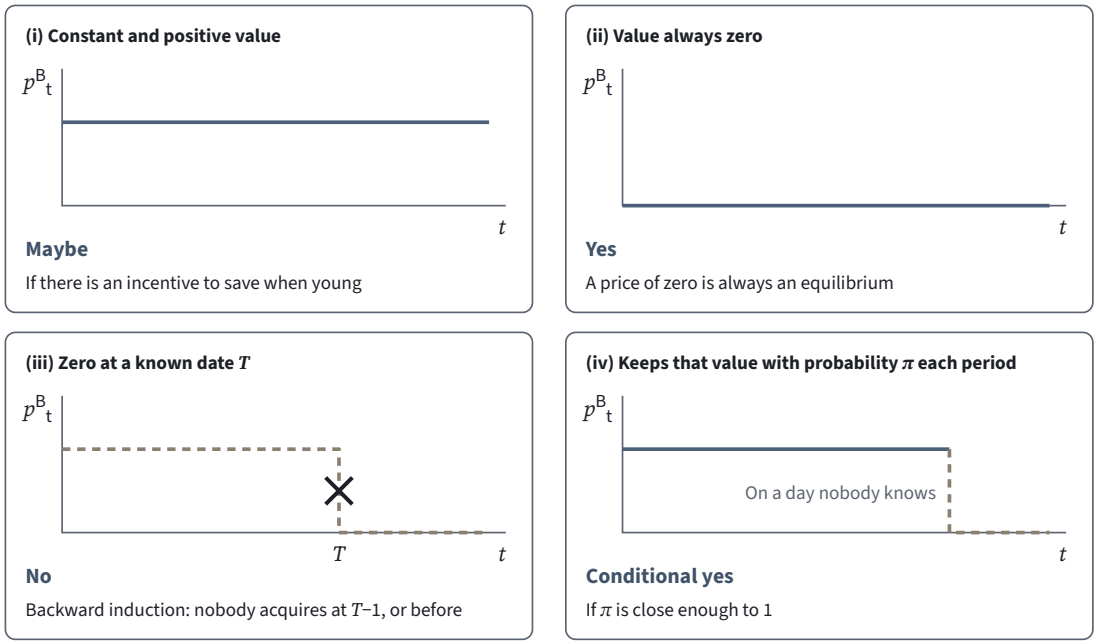


Figure 19-1. Garratt–Wallace's four questions and their answers. The vertical axis is the goods price of the coin in the model, p^B_t, and has no scale. This is a diagram comparing the shapes of equilibria, not price data.

The example the authors attach to (iv) shows the point of this chapter in numbers: "Suppose the answer to question (iv) is positive, that $\pi = 0.99$, that we are living at date 68, that heads has been experienced at all earlier dates, but that tails has occurred at date 68. Someone might be tempted to criticize economists for not having predicted the collapse in the value of money at date 67. At $t = 1$, someone who knows the model and the equilibrium would have said that the probability of a collapse at some time in the first 68 periods is $(1 - .99^{68}) = 0.495$, almost one-half. However, from the vantage point of date 67, a collapse at date 68 happens only with probability 0.01. In other words, ... although a collapse at some time is likely, predicting **when** it will occur is not possible."[12] NB19-1 below runs this arithmetic as it stands. Heads and tails can be read in two ways. They may be a purely extrinsic **sunspot** (Cass–Shell), or **intrinsic** events such as "the appearance of a preferred version of bitcoin, say bitcoin 2," a disruption in which "someone hacks the protocol," or "the passage of a law outlawing bitcoin." In a footnote the authors name the launch of Bitcoin Cash on August 1, 2017 as a candidate for such an event and write only "Time will tell ...."[13]

[Intermediate] With two currencies the problem grows. If there are no storage costs and the two monies are perfect substitutes, a constant-price equilibrium requires only one condition, equation (11) $s^* = p^B B / N + p^M M / N$, so "this allows a wide array of constant money price combinations." Only by assuming strong preferences, of the kind "Suppose a fraction α of each generation only wishes to hold B and a fraction $1 - \alpha$ of each generation only wishes to hold M," is the value pinned down, by equation (13) $p^B = \alpha N s / B$, and "So the price of each money is increasing in the fraction of people that prefer to use it and decreasing in the stock of each money." The authors then write that this analysis "extends

easily to bitcoin 1, bitcoin 2, ..., bitcoin n," and that "absent strong preferences over equivalent types of outside money there would be a high degree of price indeterminacy, even in the simplest, constant price equilibrium."[14] This is where the title's "Bitcoin 1, Bitcoin 2, ..." comes from. The fact from Part 1 that copying code costs almost nothing reads, in economics, as a fact that makes the belief propping up value thinner. Last comes interest. If the coin pays interest $d > 0$**in some other valuable object**, then because "the yield on bitcoin is d / p, which for a given $d > 0$ goes to infinity as p goes to zero," the zero-price equilibrium and the type (iv) equilibrium disappear.[15] Put the other way round, the multiplicity in this chapter comes from "no dividend." If that premise changes, so does the conclusion.

4.2 Infinitely lived agents: the fundamental pricing equation and the martingale

[Intermediate] In Schilling and Uhlig's model, two types of agents live forever and alternate between consuming and producing in odd and even periods. So in each period they split into sellers and buyers, and money is needed. There are two monies: dollars, which the central bank adds and withdraws to meet its price-level target, and bitcoins, whose quantity only grows on a fixed schedule. The condition that must be satisfied in the "fundamental" case, where **both** monies are used in trade, is the paper's Proposition 1, the **fundamental pricing equation**. If bitcoin trades take place in both periods,

$$\mathbb{E}_t\left[u'(c_{t+1})\frac{P_t}{P_{t+1}}\right] = \mathbb{E}_t\left[u'(c_{t+1})\frac{Q_{t+1}/P_{t+1}}{Q_t/P_t}\right] \qquad (1)$$

and "If $u'(c_{t+1}) / P_{t+1}$ is uncorrelated with Q_{t+1}, conditional on time-t information, then the stochastic Bitcoin price process $\{Q_t\}_{t \geq 0}$ is a martingale $Q_t = \mathbb{E}_t[Q_{t+1}]$. (2)"[16] Reading the symbols: $u'(c)$ is the marginal utility of consumption, P the dollar price level, and Q the dollar price of bitcoin. The left side is the real return from keeping dollars and spending them next period; the right side is the real return from keeping bitcoin and spending it next period; and for sellers to accept both, the two must be equal. A numerical example: if there is no correlation and today $Q_t = 100$ (a unitless model value), the conditional expectation of tomorrow's Q_{t+1} is 100. If an increase is likely, there must be a possibility of a correspondingly large decrease. The authors' commentary states exactly what this equation means: "The goods seller will sell against the currency which offers a higher risk-preference adjusted return. Due to the currency competition, sales against both Bitcoins and Dollars implies seller indifference. The risk-adjusted returns, therefore, have to be the same." They add that this is "a version of the celebrated result in Kareken and Wallace (1981)," and that in the case without randomness it reduces to $Q_t = Q_{t+1}$, that is, to "their 'exchange rate indeterminacy result' for time t = 0, that any Q_0 is consistent with some equilibrium".[17]

[Intermediate] Next comes the part that belongs to this paper alone. Agents who live forever can **keep coins without spending them**. Proposition 2 states the "speculative price bound" that must be satisfied in an equilibrium where coins are kept in this way, and the gist is that for coins to be kept, the price must **rise** in expectation. But Proposition 1 said that when both monies are used, the price is a martingale. Put the two side by side and add "mutual impatience" (Assumption 3: today's marginal utility is greater than tomorrow's discounted expected marginal utility), and Theorem 1 follows: "all Bitcoins are spent and (1) holds in

every period t." In the authors' words, "price volatility does not invalidate the medium-of-exchange function of Bitcoin."[18] The circle runs like this. The side keeping the coins wants a rise; for sellers to accept coins, tomorrow's price must not be too low; for that, today's price must be high; and if today's price is that high, the reason to keep the coins disappears. In a non-stochastic economy the conditions are $Q_{t+1} \leq Q_t \leq \beta^2 Q_{t+2} / (\pi_{t+2}\pi_{t+1})$, and since $\beta < 1$ and $\pi \geq 1$, the contradiction is immediate.[19]

[Advanced] The martingale is the special case "if there is no correlation." The general formula, Corollary 1, is $Q_t = \mathbb{E}_t[Q_{t+1}] + \kappa_t \cdot \mathrm{corr}_t(m_{t+1}, Q_{t+1})$ (equation (5)), where $m_t = u'(c_t) / P_t$ is the nominal pricing kernel and $\kappa_t > 0$. "the Dollar denominated Bitcoin price process is a supermartingale (falls in expectation) if and only if in equilibrium the pricing kernel and the Bitcoin price are positively correlated," and "under negative correlation, the Bitcoin price process is a submartingale and increases in expectation."[20] Even inside the model the direction depends **on the sign of the correlation**, and the model does not fix that sign. So the authors do not just say that equilibria "exist"; they give **a recipe for building them**. Following equation (8), $Q_{t+1} = Q_t + \epsilon_{t+1} - \mathrm{cov}_t(\epsilon_{t+1}, m_{t+1}) / \mathbb{E}_t[m_{t+1}]$: "Pick the initial Bitcoin price Q_0 and pick some sequence ϵ_t with $\mathbb{E}_t[\epsilon_{t+1}] = 0$ ... recursively construct the sequence Q_{t+1}, going forward in time."[21] Both Q_0 and ϵ are **picked**. There is not one equilibrium but as many as there are ways to pick. NB19-2 runs this construction with a few choices of ϵ and shows the paths that branch off from the same Q_0. Those paths are **model variables**, not prices. Separately, if there is a cap on issuance and dollar inflation is positive, there is Proposition 3 (Real Bitcoin Disappearance): "the purchasing power of the entire stock of Bitcoin shrinks to zero over time." This too is a conditional proposition, "Only a sufficiently high risk premium can avoid the Bitcoin disappearance," and the authors write that once the free entry of §7 of the paper is added, the result "may no longer obtain".[22]

4.3 Transactional benefits: a "fundamental value" with no anchor

[Intermediate] The 2023 paper by Bruno Biais and coauthors gives the coin **transactional benefits**: the benefit that "citizens of Venezuela or Zimbabwe can use Bitcoin to conduct transactions even when their national currencies and banking systems are in disarray. Cryptocurrencies can be used for cross-border transfers when high costs or government controls hinder transfers via traditional financial institutions," and the costs of "limited convertibility into traditional currencies, transaction costs on exchanges, fees that agents must pay to have their transactions mined, and the risk of a crash of the cryptocurrency."[23] The coin then comes to resemble a stock. "transactional benefits are to cryptocurrencies what dividends are to stocks. There is however a major difference. In perfect markets, dividends do not depend on stock prices and provide a real anchor for valuation. In contrast, the transactional benefits provided by a cryptocurrency depend on its price: the higher the price of the cryptocurrency, the stronger is its purchasing power relative to the standard currency, and consequently the higher are the transactional benefits it delivers."[24] Equation (12) of Proposition 1 writes the price as the discounted sum of future transactional benefits $T_{t+k}p_{t+k}$, and p_{t+k} sits inside it. The fundamental value is a function of the price. Hence, "Because cryptocurrency prices reflect beliefs about future prices rather than real variables independent of prices, equilibrium can reflect exogenous sunspots." The authors construct two kinds of equilibrium: **constant-price equilibria**, one for each probability of a crash, and **volatile-price equilibria**, in which the sunspot moves the price every period. "There are multiple equilibria, one for each possible distribution of the sample path of the sunspot. Within each equilibrium there is extrinsic volatility, since cryptocurrency prices change in response to realizations of the sunspot variable, even when fundamental variables remain constant."[25]

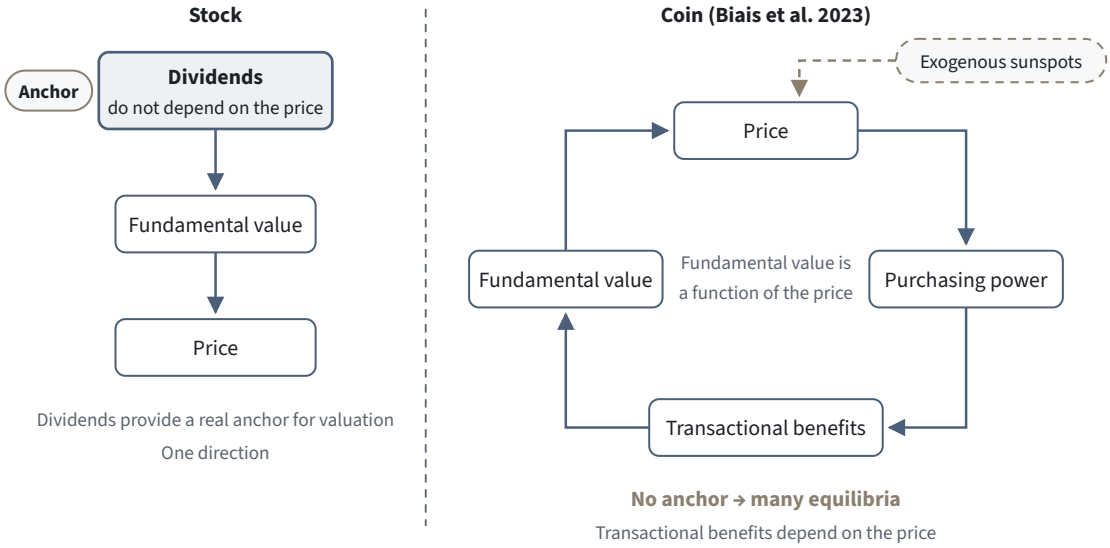


Figure 19-2. Fundamental value compared for a stock and a coin (Biais et al. 2023). Dividends do not depend on the price and so serve as an anchor; transactional benefits depend on the price and form a loop, and sunspots enter through that loop.

[Advanced] The paper also fits the model to data. From mining fees, events such as exchange openings and closures, events such as merchant acceptance, and data on hacks, it builds indices of transaction costs and benefits and calibrates the model under an assumption of

risk neutrality. The authors write in advance that "we do not claim statistical significance or estimation of population parameters because of the relatively short size of our sample and the likely non stationarity of our variables," and summarize the result this way: "changes in fundamentals explain only a small share (around 5%) of the variance of Bitcoin returns. Under the hypothesis that our model is well-specified and our proxies accurate, this implies that most of Bitcoin return variation reflects extrinsic volatility unrelated to fundamentals."[26] When you carry this figure over, you must not cut off the conditional clause. "Around 5%" is a calibrated value **under the assumption** that the model and the proxies are right, and the authors themselves do not call it an estimate. What this chapter takes from the figure is not its size but the admission that even the authors of this model could not explain most of the data with "fundamentals."

▾ **[Advanced] The words "bubble" and "Ponzi": where rational bubbles fit**

[Advanced] The multiplicity in this chapter has the same root as the results of the rational bubbles literature of the 1980s. A 1982 paper by Olivier Blanchard and Mark Watson shows that the solution of the arbitrage condition $\mathbb{E}(p_{t+1} \mid \Omega_t) - p_t + x_t = rp_t$ (equation (1)) is not just the single p_t^*, "the present value of expected dividends": every $p_t = p_t^* + c_t$ (equation (3)) obtained by adding any c_t that satisfies $\mathbb{E}(c_{t+1} \mid \Omega_t) = (1+r)c_t$ is a solution too. "the market price can deviate from its market fundamental value without violating the arbitrage condition." They then give as an example a bubble of which "In each period, the bubble will remain, with probability π, or crash, with probability $1-\pi$" (equation (4)), the same shape as Garratt–Wallace's (iv).[27] Jean Tirole's 1985 paper, which gives necessary and sufficient conditions for an aggregate bubble to exist in an overlapping generations economy, writes: "Since Samuelson [30] developed his consumption loan model it has been well known that there exist economies in which money has a positive value in spite of the fact that it is intrinsically useless (i.e., its market fundamental is zero). In other words there can exist a bubble on money where a bubble is defined as the difference between the market price and the market fundamental."[28] In the language of this literature, the value of fiat money is itself a "bubble." Here the word is not a term of abuse but a defined term. The problem of judging statistically that "it was a bubble" is taken up in Chapter 25; how Maarten van Oordt's 2024 model defines the "bubble equilibrium" and "Ponzi-scheme equivalent payoffs," and how the issuance rule (in particular fee burning) changes that distinction, come up again in Chapters 21 and 22.[29]

5. Code: the listing of equilibria in numbers

[Intermediate] The two cells below **do not deal with prices**. What they deal with are variables and probabilities inside models. The first cell runs Garratt–Wallace's "date 68" arithmetic; the second builds equilibria according to Schilling–Uhlig's equation (8).

```
# NB19-1: Garratt-Wallace question (iv) — a collapse that comes with probability 1-π each
period. The probability of "some time" and the probability of "the very next period" differ (for
study)
pi = 0.99                                              # probability of surviving one more period
(the paper's example)
print(f"π = {pi}: conditional probability of a collapse in the next period = {1 - pi:.2f}")
print(f"{'n':>6} {'P(collapse within n) 1-π^n':>26} {'P(survive through n) π^n':>26}")
for n in [1, 10, 30, 68, 100, 300, 1000]:
    print(f"{n:>6} {1 - pi**n:>26.3f} {pi**n:>26.3f}")
print("\nThe probability of a collapse within 68 periods is almost one-half, but the probability
of a collapse at date 68, seen from date 67, is still 0.01 (the arithmetic on pp. 1891-1892 of
the paper).")
print("The date of the collapse follows a geometric distribution, and its expected value is",
round(1 / (1 - pi)), "periods: there is a mean but no date.")
```

```
# Output — bcbook (Python 3.13), exactly as run
π = 0.99: conditional probability of a collapse in the next period = 0.01
     n P(collapse within n) 1-π^n   P(survive through n) π^n
     1                      0.010                      0.990
    10                      0.096                      0.904
    30                      0.260                      0.740
    68                      0.495                      0.505
   100                      0.634                      0.366
   300                      0.951                      0.049
  1000                      1.000                      0.000

The probability of a collapse within 68 periods is almost one-half, but the probability of a
collapse at date 68, seen from date 67, is still 0.01 (the arithmetic on pp. 1891-1892 of the
paper).
The date of the collapse follows a geometric distribution, and its expected value is 100
periods: there is a mean but no date.
```

```
# NB19-2: Schilling-Uhlig equation (8) — "pick" an initial value Q0 and a mean-zero shock ε, and
one equilibrium path is built. There are as many equilibria as there are ways to pick (for
study)
# Note: Q is a model variable (unitless). It is not the price of any asset, and the paths below
are not forecasts.
import random
random.seed(19)

def construct_path(Q0, steps, eps_draw, cov_term=0.0):
    """Eq. (8): Q_{t+1} = Q_t + ε_{t+1} − cov_t(ε_{t+1}, m_{t+1}) / E_t[m_{t+1}].
    With cov_term=0 this is a martingale (Proposition 1, eq. (2)). With a correlation, cov_term
adds an expected change (Corollary 1)."""
    Q = [Q0]
    for _ in range(steps):
        Q.append(Q[-1] + eps_draw() − cov_term)
    return Q

Q0, steps = 100.0, 12
draws = {
    "ε≡0 (Kareken-Wallace constant price)": lambda: 0.0,
    "ε ~ uniform(-5, 5)":                   lambda: random.uniform(-5, 5),
    "ε ∈ {-20, +20}, each 1/2":             lambda: random.choice([-20.0, 20.0]),
}
print(f"Q0 = {Q0}, {steps} periods. Three equilibrium paths from the same Q0 (martingale, cov
term 0):")
for name, d in draws.items():
    path = construct_path(Q0, steps, d)
    print(f"  {name:<36} → " + " ".join(f"{q:6.1f}" for q in path))

# Numerical check of the martingale property: the mean over many paths stays near Q0 (it neither
rises nor falls in expectation)
N = 20000
end_values = [construct_path(Q0, steps, draws["ε ~ uniform(-5, 5)"])[-1] for _ in range(N)]
print(f"\nMean at the end of period {steps} over {N:,} uniform-shock paths:
{sum(end_values)/N:.2f} (Q0 = {Q0}). The mean stays put; only the spread grows")
print(f"  End of period {steps}: min {min(end_values):.1f}, max {max(end_values):.1f}")

# Corollary 1: positive correlation with the pricing kernel gives a supermartingale (falls in
expectation), negative a submartingale (rises). The model does not fix the sign.
for cov_term, label in [(1.0, "cov > 0 → supermartingale"), (-1.0, "cov < 0 → submartingale")]:
    mean_end = sum(construct_path(Q0, steps, draws["ε ~ uniform(-5, 5)"], cov_term)[-1] for _ in
range(N)) / N
    print(f"  {label}: mean at the end of period {steps} = {mean_end:.2f}")
```

```
# Output — bcbook (Python 3.13), exactly as run
Q0 = 100.0, 12 periods. Three equilibrium paths from the same Q0 (martingale, cov term 0):
  ε≡0 (Kareken-Wallace constant price) →  100.0  100.0  100.0  100.0  100.0  100.0  100.0  100.0
100.0  100.0  100.0  100.0  100.0
  ε ~ uniform(-5, 5)                   →  100.0  101.8  104.6  104.8  104.9  103.9  108.8  106.7
103.2  100.8   98.4   96.7   94.4
  ε ∈ {-20, +20}, each 1/2             →  100.0   80.0  100.0  120.0  100.0   80.0   60.0
40.0   20.0   40.0   60.0   40.0   20.0

Mean at the end of period 12 over 20,000 uniform-shock paths: 99.94 (Q0 = 100.0). The mean stays
put; only the spread grows
  End of period 12: min 64.7, max 138.4
  cov > 0 → supermartingale: mean at the end of period 12 = 87.89
  cov < 0 → submartingale: mean at the end of period 12 = 112.12
```

[Intermediate] The three paths in the second cell are **all equilibria** as long as they satisfy the conditions of equation (8) (a positive price, the upper bound of Corollary 2). The model does not say which one is the "right" path. The first path is Kareken–Wallace's constant price; the second and third start from the same Q_0 and differ only in the size of the shock picked. That the mean of 20,000 paths stays at Q_0 is what a martingale means, and the last two lines are a

numerical version of Corollary 1, in which the sign of the correlation alone changes the direction of the mean. What to take from this cell is not the shape of the paths but the verb "pick."

6. What the models say and what they do not

Laid over one another, the three models have two things in common. First, **a price of zero is always an equilibrium.** Garratt–Wallace and Biais et al. state that this equilibrium exists (Biais et al. write in a footnote that "There always exists an equilibrium such that $p_t = 0 \ \forall$t" and analyze the positive-price equilibria), and Schilling–Uhlig, following Kareken–Wallace in their §7, write down the backward self-elimination: "if tomorrow's price of a currency is zero, its price today is zero as well."[30] Second, **positive-price equilibria are held up by beliefs and are not pinned down to one.** The more clone coins there are (Garratt–Wallace), and the more transactional benefits depend on the price itself (Biais et al.), the thinner the anchor of that belief. And there are things none of them says: which equilibrium is realized, which equilibrium today's price sits on, which direction the next period goes. In their conclusion Garratt and Wallace write: "none of the equilibria described in this paper provide a good description of its value. ... That is why we said at the outset that the number of equilibria is **huge**."[31] That is why this chapter holds on to the phrase "a listing of possibilities."

So look again at the two clichés. "It has no intrinsic value, so it will go to zero" turns the fact that (ii) is an equilibrium into the claim that (iii) holds. The model allows "some day," but it rules out "when." "It is scarce, so it has value" looks only at the fact that B is in the denominator of equation (13) and leaves out the fact that α (the share of people who want to use it) is in the numerator. If $\alpha = 0$, then however small B is, $p^B = 0$. Both clichés are propositions with their conditional clauses deleted, and in a legal document a proposition whose conditions have been deleted is hard to use as material for fact-finding.

Last, before moving on to the box below, look at the place the models give to **legal tender**. Biais et al. note that the price of the standard currency could also go to zero, but "As Starr (1974) shows, however, this can be prevented when the government levies taxes that must be paid in that currency: even if agents anticipate that others will not to accept [sic] the standard currency, as long as they know that the government will accept it for the payment of taxes, demand for the standard currency is strictly positive at prices bounded away from zero," and so they analyze only equilibria in which the price of the standard currency is positive.[32] Garratt and Wallace too, at the opening of their paper, gave as the reason why Kareken and Wallace's conditions for indeterminacy do not fit national currencies that Kareken and Wallace "did not assume that the taxes levied by a country have to be paid in the form of that country's currency or that some prices denominated in the currency of a country are fixed or sticky."[33] Nowhere in the three models is there a mechanism by which value arises because "the law calls it money." What there is, is **a floor of demand called taxation**. That is the line that separates bitcoin from the dollar at the opening of Garratt–Wallace and in Biais et al.; in Schilling–Uhlig the central bank's supply adjustments toward its price-level target play that part, and in §IV of Garratt–Wallace the storage cost of fiat

money does. The line of taxation is drawn by law, not by technology.

> **? A LAWYER'S QUESTION**
>
> Are legal tender and "money" the same thing? In this chapter's models, what separated national currency from bitcoin was not the fact that "the law calls it money" but the floor of demand created by "taxes must be paid in it." Then what changes when the law calls an object "money": the effect of discharging debts (legal tender), its status as a unit of account, its standing as the object of criminal provisions (counterfeiting and currency offenses), the framework of taxation, or something else? Conversely, in which areas of law is an object that people accept, without the law calling it anything, treated as "money," and in which as "property"? Of the three functions in the functions approach, how many does the law require? This book does not answer; in Part 5 it points to each jurisdiction's definitional provisions and the layout of scholarly views. → Chapter 27

7. Summary

The functions approach is a checklist. Crypto-assets do not score well on it, but that does not mean a price of zero. The answer economics has given to the question of price is a listing of equilibria. An outside money with no dividend has both a zero-price equilibrium and positive-price equilibria, and the positive-price equilibria are held up by the belief that "others will accept it" and are not pinned down to one. The overlapping generations model shows that a collapse can come "some day" but that "when" cannot be predicted; the infinitely lived agents model shows that when two currencies are used together the price is a martingale (if it is uncorrelated with the pricing kernel) and that the sign of the correlation sets the direction of the expected change; and the transactional benefits model shows that the "fundamental value" depends on the price itself and so has no anchor. This chapter never once stated a price, and the chapters that follow do not either. Chapter 20 looks at how the issuance rule B_t, which this chapter treated as "given," is the source of funding for security, and Chapter 21 at how fees, the other half of that funding, are set.

Sources for this chapter

- Rodney Garratt & Neil Wallace, "Bitcoin 1, Bitcoin 2, ….: An Experiment in Privately Issued Outside Monies," *Economic Inquiry* 56(3): 1887–1897 (2018), DOI 10.1111/ecin.12569 — inside and outside money (§II, p. 1887), the model's premises (pp. 1887–1888), the four questions and their answers (§III.B, pp. 1890–1891), the date-68 example and the sunspot and intrinsic readings (pp. 1891–1892), two currencies and clone coins (§IV.A, p. 1893), interest (§V, p. 1895), conclusion (§VI, p. 1896). (published version). The working-paper locations noted during checking (§3.1, equations (12)–(14) and (18)) correspond to §IV.A, equations (11)–(13), and §IV.B, equation (18), of the published version.
- Linda Schilling & Harald Uhlig, "Some simple bitcoin economics," *Journal of Monetary Economics* 106: 16–26 (2019), DOI 10.1016/j.jmoneco.2019.07.002, CC BY — Proposition 1, the fundamental pricing equation and the martingale (§3.1, equations (1) and (2), p. 20), Proposition 2, Assumption 3 and Theorem 1 (§3.2, pp. 20–21), Corollary 1 with equation (5) and Proposition 3 (§4, p. 21), the construction of equilibria by equation (8) (p. 22), equation (9) and Theorem 2 (§5, p. 22), free entry (§7, p. 25). (published version). The working-paper numbers noted during checking (equations (40)–(42), Propositions 1 and 2, Theorem 1) correspond in the published version to equations (1)–(3) and the same proposition and theorem numbers.
- Bruno Biais, Christophe Bisière, Matthieu Bouvard, Catherine Casamatta & Albert J. Menkveld, "Equilibrium Bitcoin Pricing," *Journal of Finance* 78(2): 967–1014 (2023), DOI 10.1111/jofi.13206 — the difference between transactional benefits and dividends (p. 968), multiplicity of equilibria and extrinsic volatility (p. 969), the conditions on the calibration result (p. 970), taxes and the standard currency (p. 975), the model (§I), Proposition 1, equations (10)–(12) (§II.A, p. 977), note 18 (p. 979). (published version).
- Susan Athey, Ivo Parashkevov, Vishnu Sarukkai & Jing Xia, "Bitcoin Pricing, Adoption, and Usage: Theory and Evidence," Stanford GSB Working Paper No. 3469 (2016-08; SSRN 2826674) — the steady-state exchange rate and the mixed evidence (pp. 1, 3), concentration of ownership and beliefs (pp. 3–4).
- Igor Makarov & Antoinette Schoar, "Blockchain Analysis of the Bitcoin Market," NBER Working Paper 29396 (2021-10) — the 90% of volume that is not economically meaningful and the 75% linked to exchanges (p. 2).
- Gary B. Gorton & Jeffery Y. Zhang, "Taming Wildcat Stablecoins," *University of Chicago Law Review* 90(3): 909–972 (2023) — the three functions of money and the NQA principle (pp. 911–912).
- SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017-07-25) — the FATF definition of "virtual currency" (p. 3, note 5).
- Eng-Tuck Cheah & John Fry, *Economics Letters* 130: 32–36 (2015), accepted manuscript — the check against the functions (§2).
- Olivier J. Blanchard & Mark W. Watson, NBER Working Paper 945 (1982) — the arbitrage condition and bubble solutions (pp. 2–4). Jean Tirole, *Econometrica* 53(5): 1071–1100 (1985) — the definition of a bubble on money (p. 1071). Maarten R.C. van Oordt, Tinbergen

Institute DP 2024-050/IV (2024) — bubble equilibrium and Ponzi-scheme equivalent payoffs (abstract). Paths are in the notes.

- The equation of exchange $MV = PT$ itself is used as a textbook identity, and no particular original source is cited. Its version inside a model is Schilling & Uhlig (2019) §5, equation (9).
- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard library `random`. Notebook NB19. Neither cell uses price data, and the output consists of model variables.

Notes

1. Gary B. Gorton & Jeffery Y. Zhang, "Taming Wildcat Stablecoins," *University of Chicago Law Review* 90(3): 909–972 (2023), pp. 911–912. Emphasis added. For the source of the three functions the authors cite Desan, *Making Money* (2014) (their note 7).
2. SEC, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Release No. 81207 (2017-07-25), p. 3, note 5. Original (quoting the FATF Report, *Virtual Currencies, Key Definitions and Potential AML/CFT Risks*, 2014-06): "a digital representation of value that can be digitally traded and functions as: (1) a medium of exchange; and/or (2) a unit of account; and/or (3) a store of value, but does not have legal tender status (i.e., when tendered to a creditor, is a valid and legal offer of payment) in any jurisdiction."
3. Eng-Tuck Cheah & John Fry, "Speculative bubbles in Bitcoin markets? An empirical investigation into the fundamental value of Bitcoin," *Economics Letters* 130: 32–36 (2015), accepted manuscript p. 2 (§2). Original: "In order to serve as money Bitcoin should fulfil the roles of unit of account (Dwyer, 2014), means of payments and store of value. In this regard as the speculative aspects of Bitcoin dominates its other functions Bitcoin prices appear to be at the mercy of nothing more that [sic] market sentiments ... Hence, this undermines the role Bitcoin plays as a unit of account." This paper's estimate of a "fundamental value of zero" is taken up in Chapter 25.
4. Rodney Garratt & Neil Wallace, "Bitcoin 1, Bitcoin 2, ...: An Experiment in Privately Issued Outside Monies," *Economic Inquiry* 56(3): 1887–1897 (2018), p. 1887 (§II). (published version).
5. Garratt & Wallace (2018), pp. 1887–1888. Emphasis added.
6. Linda Schilling & Harald Uhlig, "Some simple bitcoin economics," *Journal of Monetary Economics* 106: 16–26 (2019), p. 22 (§5). Original: "Since all Dollars and Bitcoins are spent once in every period, the velocity of both currencies equals one. In a world with only one currency, classical quantity theory yields $y_t = D_t / P_t$. ... In our model instead, equilibrium market clearing implies $y_t = D_t / P_t + (Q_t / P_t)B_t$. (9) There are now two endogenous variables, the Dollar quantity D_t and the Bitcoin price Q_t." (published version, CC BY). The working-paper equation numbers noted during checking (equations (40)–(42)) became (1)–(3) in the published version; this book uses the published numbering.
7. Schilling & Uhlig (2019), p. 22, Theorem 2 (Bitcoin block rewards are financed by Dollar taxes). Original (the explanation following the theorem): "The block rewards to miners ... are financed not by deflating the Bitcoin currency but by the Dollar central bank, which has to accordingly decrease its Dollar supply. It does so per imposing Dollar lump-sum taxes on the population. The block rewards are not a tax on Bitcoin holders but are financed through Dollar taxes imposed by the Dollar central bank."
8. Igor Makarov & Antoinette Schoar, "Blockchain Analysis of the Bitcoin Market," NBER Working Paper 29396 (2021-10), p. 2. Original: "We first document that 90% of transaction volume on the Bitcoin blockchain is not tied to economically meaningful activities but is the byproduct of the Bitcoin protocol design as well as the preference of many participants for anonymity."; "Starting from 2015, 75% of real bitcoin volume has been linked to exchanges or exchange-like entities such as on-line wallets, OTC desks, and large institutional traders." We could not find a journal version and cite the NBER working paper. The figures on concentration of ownership are in Chapter 22.
9. Susan Athey, Ivo Parashkevov, Vishnu Sarukkai & Jing Xia, "Bitcoin Pricing, Adoption, and Usage: Theory and Evidence," Stanford GSB Working Paper No. 3469 (This Draft: August 2016; SSRN 2826674), p. 1 (abstract) and

p. 3. Original: "In steady state, exchange rates are determined by the ratio of transaction volume to the supply of Bitcoins."; "Empirical evidence from Bitcoin prices and utilization provides mixed evidence about the ability of the model to explain prices."; "investors and infrequent users held the majority of Bitcoins"; "this implies that exchange rates are likely to be more sensitive to beliefs about the future of Bitcoin and less sensitive to current transaction volume for its own sake".

10. Garratt & Wallace (2018), p. 1890 (§III.B).

11. Garratt & Wallace (2018), pp. 1890–1891. (iii) original: "Consider what happens at T − 1. As in our argument concerning question (ii), no one is willing to give up goods for money at T − 1. Then, by backward induction, no one gives up goods for money at any t < T. Hence, the answer to question (iii) is no." The condition for (iv): " (b) if ∂f(0;1)/∂s^B > 0, then there exists π < *1 such that if* $\pi \in [\pi, 1]$, then the answer is yes."

12. Garratt & Wallace (2018), pp. 1891–1892. Original: "Suppose the answer to question (iv) is positive, that π = 0.99, that we are living at date 68, that heads has been experienced at all earlier dates, but that tails has occurred at date 68. Someone might be tempted to criticize economists for not having predicted the collapse in the value of money at date 67. At t = 1, someone who knows the model and the equilibrium would have said that the probability of a collapse at some time in the first 68 periods is (1–.99^68) = 0.495, almost one-half. However, from the vantage point of date 67, a collapse at date 68 happens only with probability 0.01. In other words, and consistent with our answer to question (iii), although a collapse at some time is likely, predicting when it will occur is not possible." Emphasis added in the text.

13. Garratt & Wallace (2018), p. 1892 and note 9. Original: "One interpretation is that the uncertainty is purely extrinsic. That is, heads and tails represent the two outcomes of a publicly observed sunspot variable à la Cass and Shell (1983)."; "The appearance of tails could represent the appearance of a preferred version of bitcoin, say bitcoin 2 ... Or, we could imagine that at every date t there is an exogenous probability 1 − π that there is a disruption of platform for transacting bitcoin (someone hacks the protocol) or the passage of a law outlawing bitcoin."; note 9: "The August 1, 2017 launch of Bitcoin Cash, a new version of the bitcoin protocol with a larger block size, could have, at least in principle, led to the abandonment of the original bitcoin. Time will tell how this ultimately plays out."

14. Garratt & Wallace (2018), p. 1893 (§IV.A). Original: "But this allows a wide array of constant money price combinations."; "So the price of each money is increasing in the fraction of people that prefer to use it and decreasing in the stock of each money."; "The above analysis applies equally well to the case of two privately issued outside monies: bitcoin 1 and bitcoin 2. Moreover, it extends easily to bitcoin 1, bitcoin 2, ..., bitcoin n. The key point is that absent strong preferences over equivalent types of outside money there would be a high degree of price indeterminacy, even in the simplest, constant price equilibrium."

15. Garratt & Wallace (2018), p. 1895 (§V). Original: "If d denotes the interest payment (or dividend) and p the constant price, both in units of deposits at the central bank, then the yield on bitcoin is d/p, which for a given d > 0 goes to infinity as p goes to zero. Therefore, such an interest-bearing version of bitcoin cannot have a zero price, which, in turn, rules out the type (ii) and (iv) equilibria in Section III.B ...".

16. Schilling & Uhlig (2019), p. 20 (§3.1), Proposition 1 (Fundamental pricing equation). Original: "Suppose that sales happen both in the Bitcoin-denominated consumption market as well as the Dollar-denominated consumption market at time t as well as at time t + 1, i.e. suppose that $x_t > 0$ and $x_{t+1} > 0$. Then [equation (1)]. If $u'(c_{t+1}) / P_{t+1}$ is uncorrelated with Q_{t+1}, conditional on time-t information, then the stochastic Bitcoin price process $\{Q_t\}_{t \geq 0}$ is a martingale $Q_t = E_t[Q_{t+1}]$. (2)"

17. Schilling & Uhlig (2019), p. 20. Original of the second passage: "The result is a version of the celebrated result in Kareken and Wallace (1981). These authors did not consider stochastic fluctuations. Our martingale result then reduces to a constant Bitcoin price, $Q_t = Q_{t+1}$, and thus their 'exchange rate indeterminacy result' for time t = 0, that any Q_0 is consistent with some equilibrium, provided the Bitcoin price stays constant afterwards ...".

18. Schilling & Uhlig (2019), pp. 20–21, Proposition 2 (Speculative price bound), Assumption 3 (Mutual Impatience) and Theorem 1 (No-Bitcoin-Speculation). Theorem 1, original: "Suppose that $B_t > 0$ and $Q_t > 0$ for all t. Impose Assumption 3. Then all Bitcoins are spent and (1) holds in every period t." It continues: "Consequently, price volatility does not invalidate the medium-of-exchange function of Bitcoin."

19. Schilling & Uhlig (2019), pp. 20–21. Original: "One can see this immediately, if the economy is non-stochastic. The two conditions of the two Propositions 1 and 2 would require $Q_{t+1} \leq Q_t \leq \beta^2 Q_{t+2} / (\pi_{t+2}\pi_{t+1})$, a

contradiction, since β < 1 and $\pi_t \geq 1$ in all t per Assumption 1."

20. Schilling & Uhlig (2019), p. 21 (§4), Corollary 1 (Equilibrium Bitcoin Pricing Formula). Original: "One immediate implication of Corollary 1 is that the Dollar denominated Bitcoin price process is a supermartingale (falls in expectation) if and only if in equilibrium the pricing kernel and the Bitcoin price are positively correlated for all t + 1 conditional on time t-information. Likewise, under negative correlation, the Bitcoin price process is a submartingale and increases in expectation."

21. Schilling & Uhlig (2019), p. 22, equation (8). Original: "Let a sequence of Dollar goods prices P_t and pricing kernels m_t be given. Pick the initial Bitcoin price Q_0 and pick some sequence ϵ_t with $E_t[\epsilon_{t+1}] = 0$ and some chosen conditional covariance with the pricing kernels. Use Eq. (8) to recursively construct the sequence Q_{t+1}, going forward in time."

22. Schilling & Uhlig (2019), p. 21, Proposition 3 (Real Bitcoin Disappearance); p. 22; p. 25 (§7). Original: "In words, the purchasing power of the entire stock of Bitcoin shrinks to zero over time, if inflation is bounded below by a number strictly above one."; "Only a sufficiently high risk premium can avoid the Bitcoin disappearance."; §7: "The result on Bitcoin disappearance may no longer obtain. If over time new currencies enter faster than established currencies devalue, the total purchasing power of all cryptocurrencies may not vanish and can even increase."

23. Bruno Biais, Christophe Bisière, Matthieu Bouvard, Catherine Casamatta & Albert J. Menkveld, "Equilibrium Bitcoin Pricing," *Journal of Finance* 78(2): 967–1014 (2023), p. 968. Original: "For example, citizens of Venezuela or Zimbabwe can use Bitcoin to conduct transactions even when their national currencies and banking systems are in disarray. Cryptocurrencies can be used for cross-border transfers when high costs or government controls hinder transfers via traditional financial institutions. However, in addition to providing these transactional benefits, cryptocurrencies come with costs: limited convertibility into traditional currencies, transaction costs on exchanges, fees that agents must pay to have their transactions mined, and the risk of a crash of the cryptocurrency." (published version).

24. Biais et al. (2023), p. 968. Original: "In this sense, transactional benefits are to cryptocurrencies what dividends are to stocks. There is however a major difference. In perfect markets, dividends do not depend on stock prices and provide a real anchor for valuation. In contrast, the transactional benefits provided by a cryptocurrency depend on its price: the higher the price of the cryptocurrency, the stronger is its purchasing power relative to the standard currency, and consequently the higher are the transactional benefits it delivers. Because cryptocurrency prices reflect beliefs about future prices rather than real variables independent of prices, equilibrium can reflect exogenous sunspots."

25. Biais et al. (2023), p. 969; the model is in §I (pp. 973–975), and Proposition 1 in §II.A (p. 977, equations (10)–(12)). The authors interpret the variable T_{t+1}, defined by $1 + T_{t+1} = (1 + \theta_{t+1}) / (1 + \phi_t)$ in equation (10), as "the net transactional benefit per unit of the cryptocurrency" (p. 977). The location of the "flow of transactional benefits," left blank during checking, is Proposition 1, equations (11)–(12), §II.A of the published version, p. 977.

26. Biais et al. (2023), p. 970. Original: "That said, we do not claim statistical significance or estimation of population parameters because of the relatively short size of our sample and the likely non stationarity of our variables."; "Although our calibration quantifies the effect of fundamentals on required expected returns, it also shows that changes in fundamentals explain only a small share (around 5%) of the variance of Bitcoin returns. Under the hypothesis that our model is well-specified and our proxies accurate, this implies that most of Bitcoin return variation reflects extrinsic volatility unrelated to fundamentals." The data are in §III (the calibration assumes risk neutrality and cites Liu and Tsyvinski (2021) for it, p. 970).

27. Olivier J. Blanchard & Mark W. Watson, "Bubbles, Rational Expectations and Financial Markets," NBER Working Paper 945 (1982-07), pp. 2–4. Original: "Thus the market price can deviate from its market fundamental value without violating the arbitrage condition."; on equation (4): "In each period, the bubble will remain, with probability π, or crash, with probability 1–π. While the bubble lasts, the actual average return is higher than r, so as to compensate for the risk of a crash." (NBER scan).

28. Jean Tirole, "Asset Bubbles and Overlapping Generations," *Econometrica* 53(5): 1071–1100 (1985), p. 1071. (JSTOR scan with no text layer; read directly from the images). Volume, issue and pages follow the printed pages; no DOI is given because the metadata do not match.

29. Maarten R.C. van Oordt, "On Bubbles in Cryptocurrency Prices," Tinbergen Institute Discussion Paper TI

2024-050/IV (2024-08-01), abstract. Original: "Investors in a bubble equilibrium hold the cryptocurrency because they expect its price to appreciate merely due to future investment inflows. Investors who participate in a bubble equilibrium for a cryptocurrency with non-negative money growth experience Ponzi-scheme equivalent payoffs in the aggregate. ... Cryptocurrencies with negative issuance (e.g., that burn transaction fees) may generate positive aggregate cash flows to investors even if their price path follows a bubble trajectory."

30. Biais et al. (2023), p. 979, note 18. Original: "There always exists an equilibrium such that $p_t = 0 \ \forall$t. Here, we investigate equilibria with strictly positive prices."

31. Garratt & Wallace (2018), p. 1896 (§VI). Original: "Bitcoin currently has value and its value moves around. Admittedly, none of the equilibria described in this paper provide a good description of its value. However, the nature of the randomness described above could be generalized to enlarge the set of equilibria ... And we could go on and on. That is why we said at the outset that the number of equilibria is huge." Emphasis added in the text.

32. Biais et al. (2023), p. 975. Original: "In our model, the standard currency price could also go to zero. As Starr (1974) shows, however, this can be prevented when the government levies taxes that must be paid in that currency: even if agents anticipate that others will not to accept the standard currency, as long as they know that the government will accept it for the payment of taxes, demand for the standard currency is strictly positive at prices bounded away from zero. Accordingly, in our analyzes, we focus on equilibria in which the price of the standard currency is strictly positive at all times." (The original's "will not to accept" is reproduced as it stands.)

33. Garratt & Wallace (2018), p. 1887 (§I). Original: "Many would say that their sufficient conditions are not met by the currencies issued by countries. For example, they did not assume that the taxes levied by a country have to be paid in the form of that country's currency or that some prices denominated in the currency of a country are fixed or sticky. What about bitcoin? Bitcoin and its actual and potential rivals ... do seem to satisfy all the assumptions that Kareken and Wallace made to get exchange-rate indeterminacy."

PART 3 · ECONOMICS

Chapter 20. The security budget: from rewards to fees

KEY TAKEAWAY

The security of a blockchain is not free. It is **an upkeep cost paid every block**. The reward that miners (or validators) receive every block draws out the spending of the honest participants, and the size of that spending is the height of the wall an attacker has to get over. Eric Budish writes this as three equations: the zero-profit condition $N^{*}c = p_{block}$, the incentive-compatibility condition $A^{*}N^{*}c \cdot t(A^{*}) > V_{attack}$, and the equilibrium constraint that combines the two, $p_{block} > V_{attack} / (A^{*}\, t(A^{*}))$. In words, **the reward per block must at all times be large relative to the value that can be obtained by attacking.** Bitcoin's reward is made up of the subsidy (new coins) and fees, and the subsidy is designed to halve every 210,000 blocks and go to 0. Fees then have to fill its place, and Miles Carlsten and his coauthors show that in a world where only fees remain, the mining game itself becomes unstable. After this chapter you will know **how to calculate** "the cost of a 51% attack," which variables enter that calculation in place of the coin price, and why a halving is **an event for the security budget**, not a price event.

Learning objectives

1. Confirm from the constants in the Core source that the block reward is made up of the subsidy and fees, and that Bitcoin's subsidy halves every 210,000 blocks so that the total converges to 21 million.
2. **[Intermediate]** Read Budish's three equations (zero profit, incentive compatibility, equilibrium constraint) and say what it means that the cost of a majority attack is **a flow, not a stock**. Calculate the attack cost as "hash-rate share × duration × unit price," knowing that the inputs are hash rate, unit price, and time, not the coin price.
3. **[Intermediate]** Calculate the expected duration $t(A, e, k)$ of a double-spending attack with the formula of Budish's Proposition 3, and reproduce Table II. Place the time-bandit attack of Chapter 17 in the same framework (equation (3)), and know where it differs from the setting of Proposition 3.
4. **[Advanced]** Read why the mining game in a world where the subsidy has reached 0 (Carlsten et al.: the mining gap, undercutting, and selfish mining with the back door open) differs from the block-reward world, and how "the stock loss of specialized capital" (Budish §V) changes the cost calculation.

What a reader of the [Basic] paragraphs alone takes away. The security of a blockchain is not free. It is **an upkeep cost paid every block**, and the source of that upkeep is shifting from new coins to fees.

1. The question this chapter answers

In Chapter 8, proof of work was a device under which "rewriting one block means redoing that much computation," and in Chapter 9 the probability that an attacker catches up with the honest chain shrank with the number of confirmations z as long as the share q did not exceed one half. Both chapters assumed **that "the attacker does not have that much computing power."** Where does that assumption come from? Computing power can be bought and rented. There is no reason an attacker cannot assemble more computing power than the honest participants are using now. What there is, **is only cost.** The question then becomes this. Why are the honest participants spending that much computing power, what decides the size of that computing power, and how much does it cost to exceed it?

Nakamoto's white paper gave this question half a page, §6, "Incentive": "By convention, the first transaction in a block is a special transaction that starts a new coin owned by the creator of the block. This adds an incentive for nodes to support the network, and provides a way to initially distribute coins into circulation, since there is no central authority to issue them. ... The incentive can also be funded with transaction fees. ... Once a predetermined number of coins have entered circulation, the incentive can transition entirely to transaction fees and be completely inflation free." And one sentence on the attacker: "If a greedy attacker is able to assemble more CPU power than all the honest nodes, he would have to choose between using it to defraud people by stealing back his payments, or using it to generate new coins. He ought to find it more profitable to play by the rules, such rules that favour him with more new coins than everyone else combined, than to undermine the system and the validity of his own wealth."[1] This chapter rewrites that half page in economics. To give the conclusion first, the last sentence of the white paper holds **only conditionally**, and the condition is "when what an attack would obtain is small relative to the block reward."

2. The two parts of the reward: subsidy and fees

The person who makes a block receives two things: coins the protocol newly creates (the **subsidy**, block subsidy) and the **fees** paid by the transactions in that block. As we saw in Chapter 11, a fee is the difference between the sum of the inputs and the sum of the outputs, and the subsidy is the output of the coinbase transaction (in the genesis block of Chapter 12 that value was 50 BTC). The subsidy rule is written as constants in the Bitcoin Core source: the halving interval `nSubsidyHalvingInterval = 210000`, the initial subsidy `50 * COIN`, and the `GetBlockSubsidy` function, which shifts 50 BTC right by as many bits as the quotient of the height divided by 210,000 and gives back 0 if that quotient is 64 or more.[2] Because the shift is applied to the integer 5 billion in satoshis, the subsidy falls below 1 satoshi and becomes 0 at the 33rd halving, and the total converges to a value **short of** 21 million BTC. NB20-1 below computes that sum in integer arithmetic.

What this rule means is that **the composition of the reward changes over time.** At first it was almost entirely subsidy, and, as Nakamoto wrote, at the end it becomes entirely fees. The 2016 paper by Carlsten and his coauthors calls this transition a move from the block-reward regime to the transaction-fee regime, and announces that the transition breaks "an unexamined belief that in terms of the security of the block chain (including incentives of the mining game), it is immaterial whether miners receive (say) 25 bitcoins in each block as a block reward or 25 bitcoins in expectation as transaction fees."[3] Section 6 looks at that argument. Before that, we need a framework that decides **how large** the reward has to be.

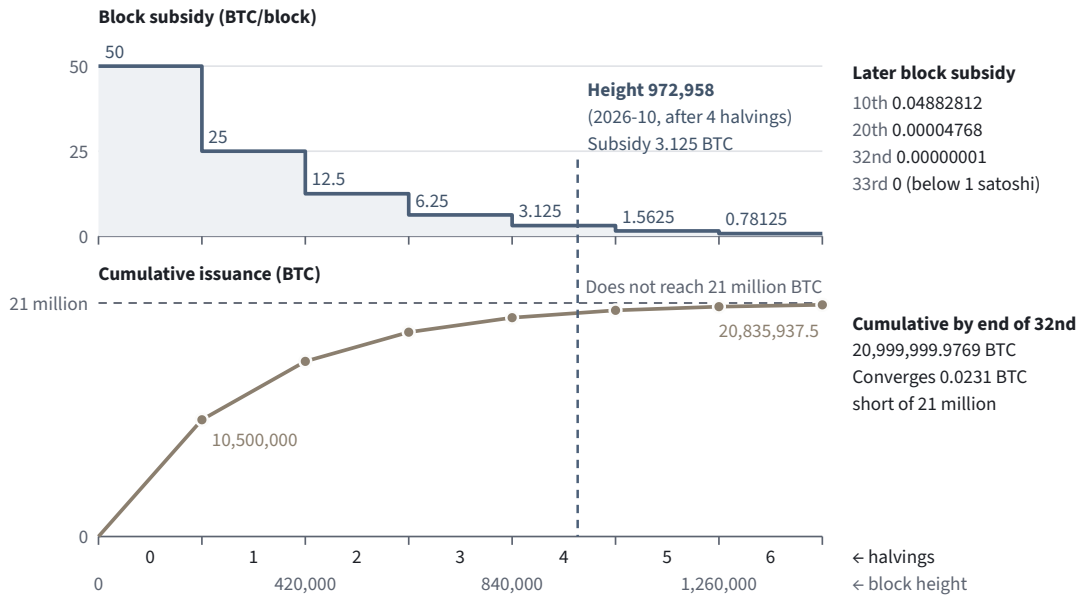
```
# NB20-1: Bitcoin's subsidy schedule, a port of Core's GetBlockSubsidy in the same integer
arithmetic (for study)
COIN = 100_000_000                      # 1 BTC = 1e8 satoshis (consensus/amount.h)
HALVING = 210_000                       # nSubsidyHalvingInterval (chainparams.cpp)

def block_subsidy_sat(height: int) -> int:
    halvings = height // HALVING
    if halvings >= 64:                  # validation.cpp: 0 after 64 or more halvings (guards
the shift width)
        return 0
    return (50 * COIN) >> halvings      # halve 50 BTC `halvings` times; an integer bit shift,
so anything below 1 satoshi is discarded

print(f"{'halvings':>8} {'start height':>12} {'block subsidy (BTC)':>20} {'cumulative issued by
end (BTC)':>30}")
cum = 0
for h in range(0, 36):
    start = h * HALVING
    sub = block_subsidy_sat(start)
    cum += sub * HALVING
    if h <= 6 or sub == 0 or h in (10, 20, 32, 33, 34):
        print(f"{h:>8} {start:>12,} {sub / COIN:>20.8f} {cum / COIN:>30.8f}")
    if sub == 0:
        break
total = sum(block_subsidy_sat(h * HALVING) * HALVING for h in range(0, 64))
print(f"\nSum of the subsidy in integer arithmetic: {total:,} satoshis = {total / COIN:.8f} BTC
({21_000_000 - total / COIN:.8f} BTC short of 21,000,000)")
tip = 972_958                           # chain-tip height observed 2026-10-24 UTC
(data/hashrate_2026-10-25.json)
print(f"Height on the reference date {tip:,}: after {tip // HALVING} halvings, block subsidy
{block_subsidy_sat(tip) / COIN} BTC, next halving at height {(tip // HALVING + 1) * HALVING:,}
(about {((tip // HALVING + 1) * HALVING - tip) * 10 / 60 / 24:.0f} days away, assuming 10
minutes per block)")
```

```
# Output — bcbook (Python 3.13), exactly as run
halvings start height  block subsidy (BTC) cumulative issued by end (BTC)
       0            0          50.00000000              10500000.00000000
       1      210,000          25.00000000              15750000.00000000
       2      420,000          12.50000000              18375000.00000000
       3      630,000           6.25000000              19687500.00000000
       4      840,000           3.12500000              20343750.00000000
       5    1,050,000           1.56250000              20671875.00000000
       6    1,260,000           0.78125000              20835937.50000000
      10    2,100,000           0.04882812              20989746.09270000
      20    4,200,000           0.00004768              20999989.97250000
      32    6,720,000           0.00000001              20999999.97690000
      33    6,930,000           0.00000000              20999999.97690000

Sum of the subsidy in integer arithmetic: 2,099,999,997,690,000 satoshis = 20999999.97690000 BTC
(0.02310000 BTC short of 21,000,000)
Height on the reference date 972,958: after 4 halvings, block subsidy 3.125 BTC, next halving at
height 1,050,000 (about 535 days away, assuming 10 minutes per block)
```



For the same fee revenue, one halving cuts the subsidy portion in half; to keep the wall, fees have to make up that portion

Figure 20-1. The schedule on which the source of the security budget shrinks. The block subsidy halves every 210,000 blocks, and cumulative issuance converges short of 21 million BTC. The values are the output of NB20-1, and there is no price axis.

On the reference date (as of 2026-10), Bitcoin is in the interval after the fourth halving, and the block subsidy is 3.125 BTC.[4] This figure is listed in Appendix A. At the next halving it becomes 1.5625 BTC. This chapter treats a halving **only as an event for the security budget.** For the same fee revenue, one halving cuts the subsidy portion of the reward per block in half, and in an equilibrium where equation (1) of Section 3 holds, it lowers the wall of attack cost by that much. To keep the wall where it is, fees have to make up that portion.

3. Budish's three equations: security is a flow

[Intermediate] Budish's 2025 paper sums itself up: "The core argument is three equations."[5] We read them in turn, using the paper's own symbols. N is the total quantity of "trust support" (hash rate under proof of work, coins locked up as collateral under proof of stake), c is the cost of supplying one unit of it for one unit of time, and p_{block} is the reward a participant receives for adding one block (subsidy and fees together).

First, the zero-profit condition. Anyone can enter and leave, so competition drives participants' profit to 0. The probability that participant i gets the next block is the share x_i / N, and if difficulty D is the expected trust support·time per block, a block appears on average every D / N time. The profit per unit of trust support per unit of time is $\frac{1}{N}\frac{D}{N}p_{block} - c$, and normalizing so that one block appears per unit of time ($D^* = N^*$) gives Proposition 1.

$$N^* c = p_{block} \qquad (1)$$

Reading the symbols: what all the honest participants spend per unit of time equals the block reward paid in that time. A numerical example: if the reward per block is worth 3.2 BTC, the electricity and equipment cost that all honest miners spend every 10 minutes is also worth 3.2 BTC. Budish writes that this is "the standard characterization of a rent-seeking

tournament: the prize in the tournament, p_{block}, is dissipated by expenditures aimed at winning the prize, N^*c," and that "Prat and Walter (2021) provide empirical support that equation (1) describes actual equilibrium behavior in the Bitcoin mining market ...."[6] Look at what the difficulty adjustment of Chapter 8 does here. More hash rate does not mean more blocks (the very property Schilling and Uhlig describe when they write that "the increase in the hash rate has no impact on the total Bitcoin production"[21]), so added spending only splits the reward more ways, and spending therefore rises or falls until profit reaches 0.

Second, the incentive-compatibility condition. Add one **attacker** rather than all the honest participants. The attacker brings AN^* units ($A > 1$) to get a majority of $\frac{A}{A+1}$, and spends AN^*c per unit of time. If $t(A)$ is the expected time until the attack ends, **the gross cost of attack** is $AN^*c \cdot t(A)$, and the attacker picks the A^* that minimizes $A \cdot t(A)$. With V_{attack} the value obtained from the attack,

$$A^*N^*c \cdot t(A^*) > V_{attack} \qquad (2)$$

is the condition under which the system is "incentive compatible (IC) against an outsider attack, on a gross-cost basis."[7]

Third, the equilibrium constraint. N^*c appears in both equations, so substituting (1) into (2) gives Theorem 1.

$$p_{block} > \frac{V_{attack}}{A^* \cdot t(A^*)} \qquad (3)$$

"In words: the equilibrium per block payment to participants for providing trust support on the blockchain must be large relative to the benefits of attacking the blockchain."[8] Reading the symbols: the denominator $A^*t(A^*)$ is a pure number (no unit) that measures "how many honest blocks' worth of cost one attack amounts to," and Table II in Section 4 gives its value. We will give the numerical example after looking at that table. What to take from here is the shape of the equation. **Time** sits on the right-hand side. The attacker pays only while the attack lasts and loses nothing afterward. In Budish's words, "Nakamoto trust is memoryless—it is as secure at a moment in time as the amount of trust support at that moment." Security is **a flow, not a stock.** Hence the analogies that follow: "Imagine if users of the Visa network had to pay fees to Visa, every ten minutes, that were large relative to the value of a successful attack on the Visa network. Imagine a brand was only as trustworthy as its flow investment in advertising. Imagine that a country were only as secure as its flow expenditure on soldiers at the border."[9] The difference from traditional trust (law, reputation, collateral) is **economies of scale.** Courts deter many crimes with the mere threat of punishment, and the police protect many banks without standing in front of any one of them. Nakamoto trust has no such lever. "[I]f attacking the system grows 1,000 times more attractive, then the cost of securing the system must grow 1,000 times as well."[10]

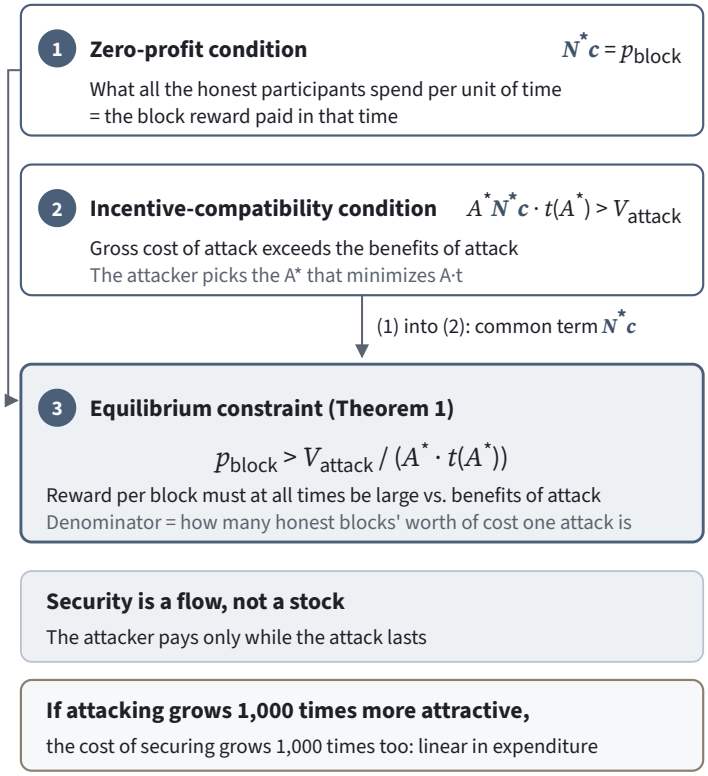


Figure 20-2. How Budish's three equations connect. The zero-profit condition (1) and the incentive-compatibility condition (2) are joined through the common term $N^* c$ to give the equilibrium constraint (3).

[Intermediate] One caveat here connects to Chapter 10. Budish says repeatedly that the three equations are not limited to proof of work: "Trust support can take the form of providing computational work as in Bitcoin ('proof of work'), ... or posting cryptocurrency coins in a proof-of-stake protocol, such as Ethereum." Under proof of stake, c is "the opportunity cost per unit time to supply one unit of stake." Stake does differ from work in one respect: "stakes are not as memoryless as computational work: stakes can be locked up on chain for a period of time, like collateral, ... This opens up the possibility of punishing attackers by confiscating their locked stakes (slashing) ...."[11] That is the "stock loss" of Section 7.

4. How long an attack takes and what it costs: Proposition 3 and Table II

[Intermediate] To fill in the denominator $A^{*}t(A^{*})$ of equation (3), we need to know how long an attack takes. Budish sets out a double-spending attack in five steps: "(i) The attacker sends cryptocurrency in exchange for goods or assets … (ii) The attacker allows those transactions to be added to the blockchain in the usual way … (iii) The attacker works in secret to create an alternative longest chain that does not include the transactions in (i). … (iv) The attacker waits for any escrow periods to elapse … (v) The attacker releases their alternative longest chain." He then turns the probability calculation of Chapter 9 into **an expected time**. Let k be the number of blocks in which the attacker placed transactions and e the escrow (the number of confirmations the receiving side waits for), and suppose the honest side produces one block per unit of time and the attacker A blocks, both as Poisson processes. Proposition 3 then gives[12]

$$t(A,e,k) = (k+e) + \sum_{i=0}^{k+e} \left(\frac{i+1}{A-1}\right) \cdot \frac{(2(k+e)-1-i)!}{(k+e-i)!\,(k+e-1)!} \left(\frac{A}{1+A}\right)^{k+e-i} \left(\frac{1}{1+A}\right)^{k+e} \tag{6}$$

Reading the symbols: the first term $k+e$ is the attacker's best case (already ahead while the honest side produces $k+e$ blocks), and each term of the sum is "the probability that the attacker is i blocks behind at that point × the expected time $(i+1)/(A-1)$ to make up that deficit." As $A \to \infty$, $t \to k+e$, and as $A \to 1^{+}$, $t \to \infty$. Budish notes that "Prior work by Grunspan and Pérez-Marco (2018, 2022) derives closely related mathematical results for the analysis of an attacker with less than 50% of the total hash power." That is the very paper we read in Chapter 9.[13]

```
# NB20-2: Budish's Proposition 3, the expected duration t(A, e, k) of a double-spending attack
and the gross cost A·t in units of honest block rewards (for study)
# The only inputs are the attacker multiple A (share A/(1+A)) and the block count k+e. No coin
price, no fiat unit price.
from math import lgamma, log, exp

def attack_duration(A: float, ke: int) -> float:
    """Equation (6). Factorials via log-gamma, to avoid overflow at large ke. Unit: honest block
time (≈10 minutes)."""
    t = float(ke)
    for i in range(0, ke + 1):
        log_comb = lgamma(2 * ke - i) - lgamma(ke - i + 1) - lgamma(ke)   # (2ke-1-i)! /
((ke-i)! (ke-1)!)
        log_prob = (ke - i) * log(A / (1 + A)) + ke * log(1 / (1 + A))
        t += (i + 1) / (A - 1) * exp(log_comb + log_prob)
    return t

rows = [1.05, 1.10, 1.20, 1.33, 1.50, 2.00, 5.00]
cols = [1, 6, 12, 36, 144]
print("Expected duration t (unit: honest block time). Columns = k+e (double-spent blocks +
escrow)")
print(f"{'A (share)':>14}" + "".join(f"{c:>10}" for c in cols))
for A in rows:
    print(f"{A:>5.2f} ({A/(1+A):4.0%})" + "".join(f"{attack_duration(A, c):>10.2f}" for c in
cols))
print("\nGross cost A·t (unit: honest trust-support cost per block N*c = block reward p_block)")
print(f"{'A (share)':>14}" + "".join(f"{c:>10}" for c in cols))
for A in rows:
    print(f"{A:>5.2f} ({A/(1+A):4.0%})" + "".join(f"{A * attack_duration(A, c):>10.2f}" for c in
cols))

# The attacker picks the A that minimizes A·t. Grid search at k+e = 12 (the paper's base case)
ke = 12
best = min(((A / 100) * attack_duration(A / 100, ke), A / 100) for A in range(101, 600))
print(f"\nk+e = {ke}: the A that minimizes A·t is A* ≈ {best[1]:.2f} (share
{best[1]/(1+best[1]):.0%}), minimum A*·t(A*) ≈ {best[0]:.2f} block rewards")
print(f"→ equation (3): p_block > V_attack / {best[0]:.1f}, i.e., the reward per block must
exceed {1/best[0]:.2%} of the attack value")
```

```
# Output — bcbook (Python 3.13), exactly as run
Expected duration t (unit: honest block time). Columns = k+e (double-spent blocks + escrow)
     A (share)         1         6        12        36       144
 1.05 ( 51%)     25.51     42.43     56.59     97.91    229.61
 1.10 ( 52%)     13.02     22.88     31.73     60.14    167.31
 1.20 ( 55%)      6.79     13.24     19.69     43.10    146.79
 1.33 ( 57%)      4.34      9.60     15.35     38.04    144.18
 1.50 ( 60%)      3.08      7.84     13.43     36.47    144.00
 2.00 ( 67%)      1.89      6.45     12.20     36.01    144.00
 5.00 ( 83%)      1.12      6.00     12.00     36.00    144.00

Gross cost A·t (unit: honest trust-support cost per block N*c = block reward p_block)
     A (share)         1         6        12        36       144
 1.05 ( 51%)     26.78     44.56     59.42    102.81    241.09
 1.10 ( 52%)     14.32     25.16     34.90     66.16    184.04
 1.20 ( 55%)      8.14     15.89     23.63     51.72    176.15
 1.33 ( 57%)      5.78     12.77     20.41     50.59    191.76
 1.50 ( 60%)      4.62     11.76     20.14     54.71    216.01
 2.00 ( 67%)      3.78     12.89     24.39     72.02    288.00
 5.00 ( 83%)      5.59     30.02     60.00    180.00    720.00

k+e = 12: the A that minimizes A·t is A* ≈ 1.43 (share 59%), minimum A*·t(A*) ≈ 20.00 block
rewards
→ equation (3): p_block > V_attack / 20.0, i.e., the reward per block must exceed 5.00% of the
attack value
```

[Intermediate] This output reproduces Budish's Table II to the second decimal place (p. 36 of the

paper). Here is how to read it. An attacker with a 51.2% share ($A = 1.05$) wins in the end, statistically, but at $k + e = 12$ the attack takes 56.6 blocks on average (about 9.5 hours) and costs 59.4 block rewards. Raise the share to 60% and these fall to 13.4 blocks and 20.1 block rewards. As Budish writes, "It is true that a 51% majority is enough to ensure statistically that the attack will eventually succeed, but a cost-minimizing attacker will choose a somewhat larger majority."[14] The last line is the numerical example for equation (3). At $k + e = 12$, $A^* \approx 1.43$ and $A^* t(A^*) \approx 20.0$, so **the reward per block must exceed 5.00% of the attack value.** Budish's Table III starts from this 5.00%: "A cost of 5% per block amounts to 720% of the value secured against attack per day, and about 263,000% of the value secured against attack per year. For example, to secure the system against a $1 billion attack requires $2.63 trillion of annual security expense." Conversely, a longer escrow lowers the cost: if $e + k$ is a week (1,008 blocks), it is 0.09% per block.[15] These dollar figures are **values from a thought experiment** that assumes a V_{attack}; they are not the price of any asset. What this book takes from them is the proportion. If the value to be protected is ten times larger, the security budget must also be ten times larger, and either users pay for it as fees or holders bear it through issuance.

The net cost may be zero

[Intermediate] The calculation above is **the gross cost.** The attacker also makes blocks during the attack, and if the attacker's chain wins, the attacker receives the rewards for those blocks. Budish's Theorem 2 states the extreme case: "If the attacker does not face any cost frictions relative to the costs of honest participants ($\kappa = 0$), the attack concludes without any difficulty adjustment ($D' = D^*$), and the attack does not cause the value of the cryptocurrency to fall ($\Delta_{attack} = 0$), **then the net cost of attack is zero.**" The proof is one line. By equation (1), the attacker's cost $At \cdot N^* c$ equals the reward the attacker receives, $At \cdot p_{block}$. "The intuition behind this result is that the attacker is fully compensated for their trust-support costs for the same reason that honest participants are fully compensated for their trust-support costs under honest play. In effect, permissionless consensus treats the attacker as if they are an honest participant, because the majority determines the truth."[16] The author himself writes that "zero attack frictions seems unrealistic," but the theorem brings out what the probability calculation of Chapter 9 missed. What costs the attacker most is not hashing but **frictions** (less efficient equipment, the cost of starting and stopping) **and the fall in value.**

The time-bandit attack of Chapter 17 is the same calculation

[Intermediate] Chapter 17 described the **time-bandit attack** of Daian and his coauthors and left the cost calculation to this chapter: "Suppose a blockchain has a suffix (subchain) [height0, height1], with current block height height1, in which stealable value exceeds block rewards. An adversary can rewind to height0 and use the resulting MEV to subsidize a profitable 51% attack that mines a fork up to or past height1."[17] Placed in Budish's framework, V_{attack} is the extractable value of that suffix, and the logic of equation (3) (whether the benefit of the attack exceeds the block rewards spent over the attack period) applies as it stands. It is not, however, the same as the setting of Proposition 3. The attacker of Proposition 3 builds the secret chain **side by side** with the honest chain from the moment of the double spend, whereas the time bandit starts **already behind** by the $height_1 - height_0$ blocks that have been settled, so the expected number of blocks needed to catch up is larger than the values in Table II (this book does not derive that formula). The only condition Daian et al. state is that "stealable value exceeds block rewards," and the sentence Chapter 17 quotes, that "application-layer security poses a current and direct threat to consensus-layer security," points to the shape of this comparison. And that threshold falls **as the block reward gets smaller.** Ethereum's burning of the base fee, which Chapter 21 covers, is one of the designs aimed at this point.

5. An attack-cost calculator: the inputs are hash rate, unit price, and time

[Intermediate] Now we build the calculator the design called for. There are three inputs: the network hash rate H on the reference date (data bundled), the multiple A of additional hash rate the attacker brings (or the share $q = A / (1 + A)$), and the cost c per unit of hash rate per unit of time, **which the user enters.** The output is $A \cdot H \cdot c \cdot t(A, e, k)$. The coin price enters nowhere. **The unit** of the cost is decided by what the user enters as c. Enter electricity unit prices and equipment rental and it is fiat currency; if you know none of these, leave it in "block-reward units," as below. Equation (1) allows this. Since $N^* c = p_{block}$, the attack cost is $A \cdot t \cdot p_{block}$, that is, "so many block rewards."

```
# NB20-3: attack-cost calculator. Inputs: hash rate (snapshot), attacker multiple, unit price,
duration. The coin price is not an input (for study)
import json, os

# Hash-rate snapshot for the reference date (data bundled; if the file is missing, a constant
with the same value is used, so it runs on Colab without the file)
try:
    with open(os.path.join("data", "hashrate_2026-10-25.json"), encoding="utf-8") as f:
        snap = json.load(f)
    H_network = float(snap["estimate"]["hashrate_H_per_s"])
except FileNotFoundError:
    H_network = 1.006294e21                 # H/s, estimated from difficulty
140,577,680,051,919.6 as D·2^32/600 (600 seconds assumed)
print(f"Estimated hash rate on the reference date H = {H_network:.3e} H/s =
{H_network/1e18:,.0f} EH/s (2026-10-24 UTC, estimate assuming 600 seconds; about 7% below the
comparison value)")

def attack_cost(A, ke, c_per_hash_second=None, p_block=None):
    """Gross cost. Given c_per_hash_second, in "unit-price units × hash-seconds"; given p_block,
in "block-reward units" via equation (1)."""
    t_blocks = attack_duration(A, ke)
    seconds = t_blocks * 600
    out = {"t_blocks": t_blocks, "hours": seconds / 3600, "attacker_hashrate_hs": A * H_network}
    if c_per_hash_second is not None:
        out["cost_in_price_units"] = A * H_network * c_per_hash_second * seconds
    if p_block is not None:
        out["cost_in_block_rewards"] = A * t_blocks            # equation (1): N*c = p_block
        out["cost_in_coin_of_reward"] = A * t_blocks * p_block
    return out

p_block_subsidy_only = 3.125                 # subsidy on the reference date (BTC). The fee
portion F varies from block to block, so this is a lower bound with it set to 0
print(f"\n(a) In block-reward units: a lower bound with fees set to 0, p_block =
{p_block_subsidy_only} BTC (subsidy only)")
print(f"{'share':>6} {'A':>5} {'k+e':>4} {'expected blocks':>15} {'time (h)':>8} {'block
rewards':>13} {'BTC equiv. (subsidy only)':>25}")
for q in [0.55, 0.60, 0.67]:
    A = q / (1 - q)
    for ke in [6, 12, 144]:
        r = attack_cost(A, ke, p_block=p_block_subsidy_only)
        print(f"{q:>6.0%} {A:>5.2f} {ke:>4} {r['t_blocks']:>15.1f} {r['hours']:>8.1f}
{r['cost_in_block_rewards']:>13.1f} {r['cost_in_coin_of_reward']:>25.1f}")

print("\n(b) In the user's unit-price units: c is not the cost per 1 hash but 'the cost of
running 1 H/s for 1 second'. The value below is a hypothetical unit price that only fills the
slot.")
c_demo = 1e-19                                # hypothetical unit price (no unit). The user
enters actual electricity and equipment unit prices
for q in [0.55, 0.60]:
    A = q / (1 - q)
    r = attack_cost(A, 12, c_per_hash_second=c_demo)
    print(f"  share {q:.0%}, k+e=12: attacker hash rate {r['attacker_hashrate_hs']:.2e} H/s ×
{r['hours']:.1f} hours × unit price → cost {r['cost_in_price_units']:,.0f} (unit-price units)")
print("\nAt the same share and the same duration, the cost is proportional to the unit price c.
Enter half the unit price and the cost halves: what this calculator tells you is a proportion,
not a level.")
```

```
# Output — bcbook (Python 3.13), exactly as run
Estimated hash rate on the reference date H = 1.006e+21 H/s = 1,006 EH/s (2026-10-24 UTC,
estimate assuming 600 seconds; about 7% below the comparison value)

(a) In block-reward units: a lower bound with fees set to 0, p_block = 3.125 BTC (subsidy only)
 share     A  k+e expected blocks time (h) block rewards BTC equiv. (subsidy only)
   55%  1.22    6            12.3      2.0          15.0                       47.0
   55%  1.22   12            18.5      3.1          22.7                       70.8
   55%  1.22  144           145.8     24.3         178.2                      556.7
   60%  1.50    6             7.8      1.3          11.8                       36.7
   60%  1.50   12            13.4      2.2          20.1                       62.9
   60%  1.50  144           144.0     24.0         216.0                      675.0
   67%  2.03    6             6.4      1.1          13.0                       40.7
   67%  2.03   12            12.2      2.0          24.7                       77.3
   67%  2.03  144           144.0     24.0         292.4                      913.6

(b) In the user's unit-price units: c is not the cost per 1 hash but 'the cost of running 1 H/s
for 1 second'. The value below is a hypothetical unit price that only fills the slot.
  share 55%, k+e=12: attacker hash rate 1.23e+21 H/s × 3.1 hours × unit price → cost 1,368,343
(unit-price units)
  share 60%, k+e=12: attacker hash rate 1.51e+21 H/s × 2.2 hours × unit price → cost 1,215,857
(unit-price units)

At the same share and the same duration, the cost is proportional to the unit price c. Enter
half the unit price and the cost halves: what this calculator tells you is a proportion, not a
level.
```

[Intermediate] What to notice in the "block rewards" column of (a) is that the number of block rewards **does not depend on the size of the reward.** At a 60% share and $k + e = 12$ it is 20.1 block rewards. If a halving cuts the subsidy in half, the BTC equivalent of those 20.1 block rewards is cut in half too, in this table, where fees are set to 0 (if fees are positive, it falls by that much less). That is what "a halving is an event for the security budget" means. The unit price in (b) is **a hypothetical value that only fills the slot.** This book does not give actual electricity unit prices or equipment rental. This chapter's interactive figure W6 moves only three variables, share, unit price, and duration; it reads the reference-date hash rate from the bundled snapshot and has no price input field.

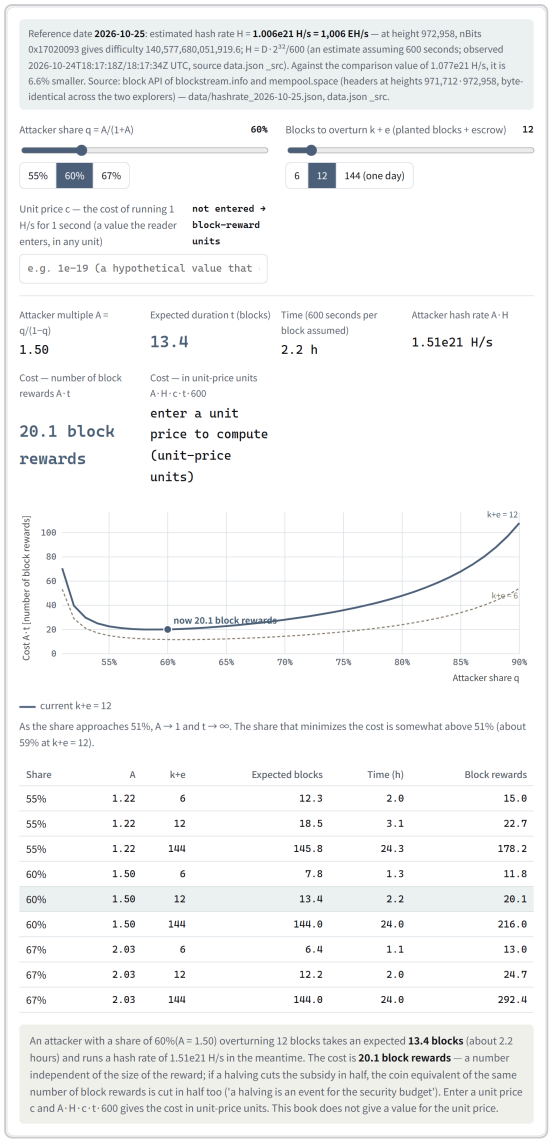


Figure 20-3. 51% attack cost calculator. At the estimated hash rate of 1,006 EH/s on the reference date (2026-10-25), a 60% share with k + e = 12 gives an expected 13.4 blocks (2.2 hours), at a cost equal to 20.1 block rewards. The reader supplies the unit prices of hashing and electricity; this book gives no values. — In the HTML edition you can change the values here (HTML edition, Chapter 20, Section 5).

6. [Advanced] A world where the subsidy has reached 0: Carlsten et al.

▾ [Advanced] How the mining game changes when only fees remain

[Advanced] Carlsten and his coauthors take as their model Bitcoin after the block reward has shrunk to 0. They assume that fees accumulate at a constant rate over time (the fees of the transactions announced in an interval of length t sum to t), that difficulty is set so that one block appears per unit of time, and that there is no latency. The authors write that they chose these simplifications on purpose: "our results are only made stronger by simplifying assumptions, because we are claiming that weird and undesirable consequences arise even if one is willing to grant simplifying assumptions."[18]

[Advanced] **First, the mining gap.** "Without a block reward, immediately after a block is found there is zero expected reward for mining but nonzero electricity cost, making it unprofitable for any miner to mine." If one mining rig accounts for a fraction a of the total effective hash power and its electricity cost per unit of time is p, then "a miner must wait t = p/a time steps after a block is found before mining becomes profitable again." As a result, miners end up "mining for a smaller and smaller fraction of the time between the arrival of blocks (with the difficulty dropping to compensate)," and "the effective hash power in the network would drop, and it would become easier for a malicious miner to fork."[19] When there is a block reward, there is a reward even right after a block, so this gap does not arise.

[Advanced] **Second, undercutting.** When the reward differs from block to block, "which block to build on" becomes a strategy. When there is a fork, the default client extends the block it heard first, but miners "can do strictly better by mining on the block that leaves the most transactions [sic] fees unclaimed" (PettyCompliant). If even some miners behave this way, a strategy opens up of deliberately forking when the head block of the chain is worth more than the unclaimed fees it leaves, and taking slightly fewer fees (undercutting). The authors show (i) that learning, non-atomic miners drift in simulation toward ever more aggressive undercutting, and (ii) that there is nonetheless an equilibrium in which every miner uses the same function-based strategy (Theorem 5.1, with $f(x)$ written in terms of the Lambert W function), and that "in any such equilibrium, the expected number of backlogged transactions after n time steps is $\Theta(\sqrt{n})$," that is, **unprocessed transactions pile up without end.** And (iii) they add the analytical result that "even if 66% of miners remain default compliant, undercutting is profitable." This is because the condition for the expected reward of undercutting to exceed 1, when the fraction using the default strategy is y, is $y \leq 2 / 3$.[20]

[Advanced] **Third, the back door into selfish mining.** Selfish mining (Eyal and Sirer), which we read in Chapter 23, has a share threshold in the block-reward world. In the fee world, blocks can be withheld *selectively*: "Intuitively, imagine you are mining and find yourself solving a new block immediately after a previous block was announced

and before any new transactions have been announced. This block is literally worthless, so instead of publishing, why not use it to try and selfish mine? There is no cost, but a positive probability that you build a lead of two, no matter your hash power. Similarly, imagine instead that just by chance an hour goes by since the last block was found and you just solved a new block including all transactions that arrived during that period. This block is worth roughly six 'normal' blocks, so why risk losing it?" So the Selfish-Mine(β) strategy, which withholds only "small" blocks, for all $\alpha, \gamma < 1$ "strictly outperforms both default mining and 'vanilla' selfish mining in the transaction fees model"; it is profitable "for a miner with an arbitrarily low hash power share, and who is arbitrarily poorly connected within the network"; and "the attack is profitable as soon as it is deployed" (classic selfish mining becomes profitable only after the difficulty adjustment). At $\alpha = 1 / 3$, "both selfish mining and default mining achieve expected reward of ≈1/3, but selfish mining with the optimal cutoff achieves an expected reward of ≈.38, a 13.6% increase!"[22]

[Advanced] What the authors write at the end of their introduction is advice on design: "Nakamoto appears to have viewed the block reward as a necessary but temporary evil to achieve an initial allocation of bitcoins in the absence of a central authority, with the transaction fee regime being the ideal, inflation-free steady state of the system. But our work shows that incentivizing compliant miner behavior in the transaction fee regime is a significantly more daunting task than in the block reward regime. Perhaps instead, designers of new cryptocurrencies must resign themselves to the inevitability of monetary inflation and make the block reward permanent."[23] This is one of the four problems that Ethereum's EIP-1559 lists in its motivation section: "In the long run, blockchains where there is no issuance (including Bitcoin and Zcash) at present intend to switch to rewarding miners entirely through transaction fees. However, there are known issues with this that likely leads to a lot of instability, incentivizing mining 'sister blocks' that steal transaction fees, opening up much stronger selfish mining attack vectors ... There is at present no good mitigation for this."[24] The rest of that design is in Chapter 21.

7. [Advanced] When a stock is at stake: specialized capital and collapse

▾ [Advanced] If the attacker loses capital, by how many orders of magnitude does the cost grow?

[Advanced] Budish's §V adds what the cost model of Section 3 left out: "If the capital used to maintain the blockchain is specialized (as opposed to repurposable), and a majority attack causes the attacker's capital to lose its value or be confiscated, then the attacker cost model needs to be modified. In addition to charging the attacker the flow cost of the attack, we also need to charge the attacker the stock value of their loss of specialized capital." Bitcoin's ASICs (chips "useless for any other application that does not involve performing a large number of SHA-256 hashes") and the stake of proof of stake are capital of this kind. Splitting cost as $c = rC + \eta$ (the rental cost of capital + variable costs), equation (1) becomes $N^*(rC + \eta) = p_{block}$ (7); the approximate condition for an outside attacker who loses all its capital is $N^*C > V_{attack}$ (8); and with the capital share $\mu = rC \,/\, (rC + \eta)$, $N^*C = \mu p_{block} \,/\, r$ (9), so that

$$p_{block} > \frac{r}{\mu} V_{attack} \qquad (10)$$

"This is several orders of magnitude more secure than equation (3) because r is the interest rate per unit time." Budish's illustrative calculation takes $\mu = 0.4$ and an annual discount rate for ASICs of 50%, giving $r \approx 0.0008\%$ per block and $r \,/\, \mu = 0.002\%$; set against the 5.0% of equation (3), this is "a roughly 2,500-fold improvement in the cost of security."[25]

[Advanced] The question then becomes "how does the attacker lose capital?" Budish gives three ways. (1) The attack collapses the value of the currency, and the capital collapses with it: "But vulnerability to collapse is itself a serious problem ... and raises the possibility of an attack motivated by this collapse per se" (sabotage). (2) Legal punishment, which "certainly would work but concedes that Nakamoto trust fails without support from rule of law." (3) The protocol algorithmically confiscates the attacker's capital alone: the companion paper (Budish, Lewis-Pye & Roughgarden 2024) shows that "this approach (i) is impossible for a broad class of permissionless consensus protocols that includes Bitcoin's; (ii) is possible for a class of permissionless consensus protocols that includes proof-of-stake Ethereum's, but only under a strong assumption about network reliability and only if the attacker is less than proportion $\rho = 2 \,/\, 3$ of the total."[26] This is where the slashing of Chapter 10 sits. Against an attacker below 2/3, the attack can be made to put a stock at stake rather than a flow, but against a larger attacker, outside trust (law) is needed. That is Budish's reading.

> **? A LAWYER'S QUESTION**
>
> The rule that the subsidy halves every 210,000 blocks is written in code, and code can be changed by a majority (Chapter 23). This chapter has said that the subsidy is the source of the security budget and that the shift of that source to fees may be unstable. Then suppose that one day a majority of nodes and miners change the rule to "keep the subsidy permanently," in exactly the direction Carlsten et al. advised. Are the rights of existing holders infringed? Is what a holder has some right to the "21 million cap," or only the ability to satisfy the locking conditions of a UTXO? If a minority that did not agree to the rule change stays on the chain with the old rules, which of the two is "the" Bitcoin? This book does not answer, and points to Part 5 and to the discussion of the quantum transition in Part 6. At the quantum transition, "a soft fork that blocks spending of vulnerable coins" poses a question of the same structure, more sharply. → Chapter 27 · Chapter 29

8. Summary

The block reward is made up of the subsidy and fees, and Bitcoin's subsidy halves every 210,000 blocks on its way to 0. That reward draws out the honest participants' spending (zero profit), that spending is the wall an attacker has to get over (incentive compatibility), and so the reward per block must at all times be large relative to the value that can be obtained by attacking (equilibrium constraint). Security is a flow, not a stock; the attack cost is calculated as "attacker hash rate × duration × unit price," and Proposition 3's formula gives the duration. The inputs are hash rate, unit price, and time, not the coin price. A halving is an event that lowers the p_{block} of this equation, and if fees do not fill its place, the wall gets lower. The mining game in a world where only fees remain differs from the block-reward world (the mining gap, undercutting, selfish mining with the back door open), and Ethereum took the path of making the reward permanent and changing the structure of fees (stating that motive is the job of the design document, and the motivation section of EIP-1559 lists this problem as one of four items). Chapter 21 looks at that second path: how fees are set and to whom they go. Chapter 23 asks whether the people this chapter has called "honest participants" really have an incentive to behave honestly.

Sources for this chapter

- Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), §6 Incentive (p. 4).
- Eric Budish, "Trust at Scale: The Economic Limits of Cryptocurrencies and Blockchains," *Quarterly Journal of Economics* 140(1): 1–62 (2025), DOI 10.1093/qje/qjae033, CC BY: the three equations (§III.A–C, pp. 21–25, Proposition 1, Definition 2, Theorem 1), flow versus stock and the analogies (p. 7 and p. 25), linearity (pp. 25–26), Theorem 2 on zero net cost (§III.D, pp. 26–28), proof of stake (p. 19 and p. 22), the double-spending steps, Proposition 3, equation (6) and Table II (§IV.A–B, pp. 33–36), Tables III and IV and the 5.00% (§IV.C, pp.

37–39), specialized capital, equations (7)–(10) and the 2,500-fold figure (§V.A, pp. 41–43), the three ways of losing capital (p. 10 and p. 44). The working-paper version (NBER 24717, 2018) differs in title and content and is not cited.

- Miles Carlsten, Harry Kalodner, S. Matthew Weinberg & Arvind Narayanan, "On the Instability of Bitcoin Without the Block Reward," *CCS '16*, pp. 154–167, DOI 10.1145/2976749.2978408. Read in the extended version posted by the authors, and only section numbers (§1, §3.1–3.2, §5.1–5.5, §6.1–6.2, §7) are cited. Figure and page numbers may differ from the published version and are not used.
- Philip Daian et al., "Flash Boys 2.0," *2020 IEEE S&P*, pp. 910–927: the time-bandit attack (§VII-C, p. 922).
- EIP-1559 (Final), Motivation section: the instability of blockchains without a reward. https://github.com/ethereum/EIPs/blob/e858e912ff55370eb8125873455af531e322d9b2/EIPS/eip-1559.md (accessed 2026-10-25).
- Bitcoin Core v31.1 `chainparams.cpp` L84, `validation.cpp` L1843–L1854, `consensus/amount.h` L15; btcd `chaincfg/params.go` L330, `blockchain/validate.go` L44: the halving interval, the initial subsidy, `GetBlockSubsidy`. Permalinks are in `widgets/data.json` `bitcoin.subsidy._src`.
- `data/hashrate_2026-10-25.json`: height 972,958, difficulty, and estimated hash rate on the reference date (raw responses from Blockstream and mempool.space, 600 seconds assumed). The subsidy of 3.125 BTC "as of 2026-10" is Appendix A, index A-16.
- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard library modules `math`, `json`, and `os`. Notebook NB20. NB20-2 reproduces Budish's Table II, and the unit price in (b) of NB20-3 is a hypothetical placeholder value. The initial state of interactive figure W6 `attack-cost` is the 60% share and $k + e = 12$ of NB20-3 (a) (output: 20.1 block rewards), and it has no asset-price input field (what the user enters is the hash unit price).

Notes

1. Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008), p. 4 (§6, Incentive).
2. Bitcoin Core v31.1 (tag commit 9be056a8). `src/kernel/chainparams.cpp` L84 `consensus.nSubsidyHalvingInterval = 210000;`, `src/validation.cpp` L1843–L1854 `GetBlockSubsidy` (50 * COIN >> halvings, 0 if halvings ≥ 64), `src/consensus/amount.h` L15 `COIN = 100000000`. The `_src` of `bitcoin.subsidy` in `widgets/data.json` has permalinks with line numbers (second independent source: btcd `chaincfg/params.go` L330, `blockchain/validate.go` L44).
3. Miles Carlsten, Harry Kalodner, S. Matthew Weinberg & Arvind Narayanan, "On the Instability of Bitcoin Without the Block Reward," *CCS '16*, pp. 154–167 (2016), extended version posted by the authors, p. 1 (§1). Original: "There has been an unexamined belief that in terms of the security of the block chain (including incentives of the mining game), it is immaterial whether miners receive (say) 25 bitcoins in each block as a block reward or 25 bitcoins in expectation as transaction fees." The stored copy is the extended version, so figures and page numbers are not cited; only section numbers are given.
4. At height 972,958 (observed 2026-10-24T18:17Z; the raw responses from Blockstream and mempool.space agree, `data/hashrate_2026-10-25.json`), 972,958 ÷ 210,000 = 4 (quotient), so the subsidy is $50/2^4 = 3.125$ BTC. Appendix A, index A-16. The coinbase output was not checked directly; the value is computed from the rule in the source code.
5. Eric Budish, "Trust at Scale: The Economic Limits of Cryptocurrencies and Blockchains," *Quarterly Journal of*

Economics 140(1): 1–62 (2025), p. 1 (abstract) and p. 4. Original: "The core argument is three equations. A zero-profit condition on the quantity of honest blockchain 'trust support' (work, stake, etc.) and an incentive-compatibility condition on the system's security against majority attack (the Achilles heel of all forms of permissionless consensus) together imply an equilibrium constraint, which says that the 'flow' cost of blockchain trust has to be large at all times relative to the benefits of attacking the system." (published version, CC BY). The 2018 NBER WP 24717 has a different title ("The Economic Limits of Bitcoin and the Blockchain"); this book cites the title and pages of the published version.

6. Budish (2025), pp. 21–23 (§III.A), Proposition 1. Original: "Let $N^* = \sum x_i^*$. In any zero-profit honest Nakamoto trust equilibrium, $N^* c = p_{block}$ (1) and $D^* = N^*$."; "Proposition 1 is widely known and is the standard characterization of a rent-seeking tournament: the prize in the tournament, p_{block}, is dissipated by expenditures aimed at winning the prize, $N^* c$. Prat and Walter (2021) provide empirical support that equation (1) describes actual equilibrium behavior in the Bitcoin mining market, with some additional nuances related to capital adjustment costs." The decomposition of cost $c = rC + \eta$ (the rental cost of capital + variable costs such as electricity) is on p. 21.

7. Budish (2025), pp. 23–24 (§III.B), Definition 2. Original: "This player can attack by choosing AN^* units of trust support, $A > 1$, for an $\frac{A}{A+1}$ majority at cost $AN^* c$ per unit time. Denote the expected duration of the attack by $t(A)$ … Call $AN^* c \cdot t(A)$ the gross cost of attack. The attacker can choose A to minimize $A \cdot t(A)$."; "Nakamoto trust is incentive compatible (IC) against an outsider attack, on a gross-cost basis, if the gross cost of attack exceeds the benefits of attack: $A^* N^* c \cdot t(A^*) > V_{attack}$. (2)"

8. Budish (2025), p. 25 (§III.C), Theorem 1 (Equilibrium Constraint). Original: "The zero-profit honest Nakamoto trust equilibrium condition (1) and the IC against outsider attack condition (2) together imply the equilibrium constraint: $p_{block} > \frac{V_{attack}}{A^* \cdot t(A^*)}$. (3) In words: the equilibrium per block payment to participants for providing trust support on the blockchain must be large relative to the benefits of attacking the blockchain."

9. Budish (2025), p. 7 and p. 25. Original of the first sentence: "An intuition is that Nakamoto trust is memoryless—it is as secure at a moment in time as the amount of trust support at that moment." The Visa passage is on p. 25.

10. Budish (2025), pp. 25–26. Original: "From a computer security perspective, the key thing to note about constraint (3) is that the security of the blockchain is linear in the amount of expenditure on trust support. For instance, if attacking the system grows 1,000 times more attractive, then the cost of securing the system must grow 1,000 times as well. In contrast, in many other contexts investments in computer security yield convex returns, such as traditional uses of cryptography—analogously to how a lock on a door increases the security of a house by more than the cost of the lock." The economies-of-scale analogy (banks, police, courts) is on pp. 7–8 and in Figure I.

11. Budish (2025), p. 4 and p. 19 (§II.E.3). Original: "Trust support can take the form of providing computational work as in Bitcoin ('proof of work'), providing other kinds of computational resources as in proof-of-storage or proof-of-memory protocols, or posting cryptocurrency coins in a proof-of-stake protocol, such as Ethereum."; "For proof of stake, c is the opportunity cost per unit time to supply one unit of stake." (p. 22); "The more important aspect of proof of stake for the purpose of my argument is that stakes are not as memoryless as computational work: stakes can be locked up on chain for a period of time, like collateral, and observably persist for the time they are locked, like reputation. This opens up the possibility of punishing attackers by confiscating their locked stakes (slashing), which makes attacks more expensive and hence the blockchain more secure."

12. Budish (2025), pp. 33–35 (§IV.A–B), Proposition 3 and equation (6). Original of the five steps: "(i) The attacker sends cryptocurrency in exchange for goods or assets, potentially in many transactions over many blocks. (ii) The attacker allows those transactions to be added to the blockchain in the usual way … (iii) The attacker works in secret to create an alternative longest chain that does not include the transactions in (i). … (iv) The attacker waits for any escrow periods to elapse … (v) The attacker releases their alternative longest chain." Explanation of the formula: "In the attacker's best case, their attack takes k + e time. … Suppose, on the other hand, that the attacker is behind the honest chain by i ⩾ 0 blocks at the time the honest participants produce their k + e block. Given the Poisson arrival processes, it will take the attacker (i+1)/(A−1) of time in expectation to strictly surpass the honest chain. The last part of the expression gives the probability that the attacker's deficit is i blocks." The proof is in Online Appendix B.

13. Budish (2025), p. 34.

14. Budish (2025), pp. 35–36, Table II. Original: "For example, if A = 1.05, which corresponds to a 51.2% attacker majority, the average duration is 56.6 blocks (9.5 hours) and the average gross cost is 59.4 times $N^* c$."

15. Budish (2025), pp. 37–39, Tables III and IV. Original: "To keep the Nakamoto blockchain secure in this base case requires a per block cost that is 5.00% of the value secured against a double-spending attack. This follows directly from equation (3), rewritten as $p_{block} / V_{attack} \geq 1 / (A^* \cdot t(A^*))$, with $t(A^*) = 13.99$ at the attacker-optimal choice of $A^* = 1.43$ (or 59%)."; "the cost declines to 0.09% of V_{attack} if e + k amounts to a week (e + k = 1,008)." The base case is e = 6, k = 6 (main text on p. 37 and footnote 23).

16. Budish (2025), pp. 26–28 (§III.D), Theorem 2 (Zero Net Attack Cost). Emphasis added. Original of the remaining sentence: "To be clear, zero attack frictions seems unrealistic. But zero friction is often useful as a benchmark case ...".

17. Philip Daian et al., "Flash Boys 2.0," *2020 IEEE S&P*, pp. 910–927, §VII-C (p. 922). The same passage as Chapter 17, footnote 11.

18. Carlsten et al. (2016), §3.1. Original: "we assume that for any time interval I of length t, the total sum of transaction fees for transactions announced during I is t"; "We also assume that the difficulty is set so that a hash puzzle is solved by someone in the network every one time unit in expectation ..."; "Put another way, our results are only made stronger by simplifying assumptions, because we are claiming that weird and undesirable consequences arise even if one is willing to grant simplifying assumptions."

19. Carlsten et al. (2016), §3.2. Original: "Without a block reward, immediately after a block is found there is zero expected reward for mining but nonzero electricity cost, making it unprofitable for any miner to mine."; "If a is the fraction of the total (effective) hash power that a single rig generates, then a miner must wait t = p/a time steps after a block is found before mining becomes profitable again."; "it leads to miners mining for a smaller and smaller fraction of the time between the arrival of blocks (with the difficulty dropping to compensate). Clearly, this would have a negative impact for Bitcoin security, as the effective hash power in the network would drop, and it would become easier for a malicious miner to fork."

20. Carlsten et al. (2016), §5.1–5.5. PettyCompliant: "Miners can do strictly better by mining on the block that leaves the most transactions fees unclaimed."; undercutting: "We call this the undercutting attack, where miners will actively fork the head of the chain and leave transactions unclaimed in the hope of incentivizing PettyCompliant miners to build on their block."; Theorem 5.1: "Then it is an equilibrium for every miner to use the strategy Function-fork(f) as long as: Every miner is non-atomic. Miners may only mine on top of chains of length H or H − 1. Furthermore, in any such equilibrium, the expected number of backlogged transactions after n time steps is Θ(√n)."; §5.5: "even if 66% of miners remain default compliant, undercutting is profitable."; "Therefore, whenever y ≤ 2/3, the reward from FunctionFork(x) is at least one, and therefore it is a better choice than PettyCompliant ...".

21. Schilling & Uhlig (2019), p. 24 (§6). Original: "This modeling choice captures the real-life feature, that the increase in the hash rate has no impact on the total Bitcoin production, see Huberman et al. (2017)."

22. Carlsten et al. (2016), §1 (Contribution 3) and §6.1–6.2. Original of the remaining passages: "this strategy works for miners with arbitrarily low hash power and regardless of their connectedness in the Bitcoin network. Moreover, the attack is profitable as soon as it is deployed, whereas classic selfish mining only becomes profitable after a two-week difficulty adjustment period ..."; "when α = 1/3, both selfish mining and default mining achieve expected reward of ≈1/3, but selfish mining with the optimal cutoff achieves an expected reward of ≈.38, a 13.6% increase!"

23. Carlsten et al. (2016), end of §1 (just before "Related Work") and §7 (the sentence to the same effect at the end of §1, and §7: "We see the block reward as integral to the stability of the mining game.").

24. Vitalik Buterin et al., "EIP-1559: Fee market change for ETH 1.0 chain" (Final), Motivation section. Original: "Instability of blockchains with no block reward: In the long run, blockchains where there is no issuance (including Bitcoin and Zcash) at present intend to switch to rewarding miners entirely through transaction fees. However, there are known issues with this that likely leads to a lot of instability, incentivizing mining 'sister blocks' that steal transaction fees, opening up much stronger selfish mining attack vectors, and more. There is at present no good mitigation for this." https://github.com/ethereum/EIPs/blob/e858e912ff55370eb8125873455af531e322d9b2/EIPS/eip-1559.md (accessed 2026-10-25, checked against the raw markdown).

25. Budish (2025), pp. 41–43 (§V, §V.A), equations (7)–(10). The ASIC passage is on p. 42 (checked in the PDF).

Original of the illustrative calculation: "Assume the capital share of mining is μ = 0.4 (De Vries 2018; Digiconomist 2022), and the annual discount rate for ASICs is 50% … which implies that the per unit time discount rate is r ≈ 0.0008%. This means r/μ = 0.002%. … If we use the base case value of $1/(A \cdot t(A)) = 5.0\%$, we have a roughly 2,500-fold improvement in the cost of security."

26. Budish (2025), p. 10 and p. 44 (§V.B). Original: "One possibility is that the value of the cryptocurrency collapses because of the attack, and as a result the value of specialized capital collapses, too. But vulnerability to collapse is itself a serious problem—hardly a reassuring foundation for a novel economic system—and raises the possibility of an attack motivated by this collapse per se. A second possibility is legal punishment, which certainly would work but concedes that Nakamoto trust fails without support from rule of law. A third possibility is that the protocol could algorithmically confiscate or block the attacker's capital in a targeted punishment … we show that this approach (i) is impossible for a broad class of permissionless consensus protocols that includes Bitcoin's; (ii) is possible for a class of permissionless consensus protocols that includes proof-of-stake Ethereum's, but only under a strong assumption about network reliability and only if the attacker is less than proportion ρ = 2/3 of the total." Budish notes that the sabotage attack was first raised by Rosenfeld (2014) (p. 44, footnote 30).

PART 3 · ECONOMICS

Chapter 21. The fee market: from auctions to EIP-1559

KEY TAKEAWAY

Block space is scarce, and the device that decides who gets to use it is **the fee market**. Bitcoin and early Ethereum used a first-price auction (pay-as-bid). Each user bids, the block producer picks the high bids, and those who get in pay their own bids. This design makes bidders guess "what will everyone else pay?", and when the guess is wrong they overpay. EIP-1559, which Ethereum introduced in the London fork on August 5, 2021, changed the fee so that it is set not by an auction but **by a formula**. The protocol computes a **base fee** for every block (raising it if the previous block was fuller than the target and lowering it if emptier, by at most 12.5% at a time). That base fee goes to no one and is **burned**, and the producer receives only the tip the user adds on top. Tim Roughgarden proved that this design always satisfies two of three incentive-compatibility conditions and usually satisfies the third, and he also showed that the reserve price and burning cannot be separated. Burning is a device that removes the producer's incentive to manipulate the fee; it is not a proposition about the price of a coin. After this chapter you will know exactly what it means to call the fee a "congestion toll," and why the question of whom that toll goes to is at the core of the design.

Learning objectives

1. Explain why the first-price auction was a problem on blockchains (overpayment, delays, volatility, the instability of a chain without block rewards), using the Motivation section of EIP-1559 and Roughgarden's four idiosyncrasies.
2. **[Intermediate]** Read the components of EIP-1559 (the base fee, history-dependent adjustment, burning, variable block size with a target and a maximum, the tip and the fee cap), and compute the base fee update formula in integer arithmetic. Distinguish the constants (elasticity multiplier 2, adjustment denominator 8) from an observed value (the gas limit).
3. **[Intermediate]** Read from a table what the three incentive-compatibility notions (DSIC, MMIC, OCA-proof) mean, and where the first-price auction and the 1559 mechanism each fall. Know why burning and the reserve price have to go together.
4. **[Advanced]** See that the blob fee market (EIP-4844) applies the same principle as 1559 to a different resource, and compute its exponential update formula.

What a reader of the [Basic] paragraphs alone takes away. The fee is a "congestion toll," and 1559 changed the toll so that it is set not by an auction but **by a formula**.

1. The question this chapter answers

Chapter 13 stopped at the point where a transaction fee is "gas units × price per gas" and that price splits into two pieces (the base fee and the priority fee). Why two pieces, who sets the base fee and how, and why one of the pieces is burned: that is this chapter. Chapter 17 showed the auctions (PGAs) that bots wage to pay block producers for a place in the ordering, and passed "the design intent that burning the base fee reduces the producer's incentive to manipulate fees" on to this chapter. Chapter 18 passed on to this chapter the fees for a separate resource called the blob. Chapter 20 said that the process by which fees become the source of the security budget could be unstable. All four strands converge on the same question. **To whom, at what price, and as whose revenue is scarce block space to be given?**

2. The first-price auction: what was wrong with it

Roughgarden's 2024 paper sets out the problem in its first lines: "Real estate on a major blockchain is a scarce resource. ... Demand for these blockchains is far larger, necessitating a mechanism that selects a subset of transactions to include 'on-chain' from the pool of all submitted transactions." Then it records the history: "Historically, most blockchain protocols have employed a pay-as-bid transaction fee mechanism. Every transaction is submitted with a bid (in the blockchain's native currency), the miner of a block decides which transactions should be included in it, and upon publication of that block, the bid of each included transaction is transferred from its creator to the miner. We follow blockchain convention and refer to this mechanism as a **first-price auction (FPA)**."[1]

Seen through auction theory, a blockchain is not an ordinary auction house. Here are the four idiosyncrasies Roughgarden lists: "(1) The miner of a block has dictatorial control over its contents and in particular may deviate from the allocation rule intended by the protocol designer. (2) The miner of a block can include fake transactions at no cost that are indistinguishable from real transactions. (3) Payments should be computable from 'on-chain' data, which typically **discloses no information about losing bids**. (4) Miners and users can easily collude off-chain to manipulate a transaction fee mechanism." There are also two things the designer can use: "(5) The choice of mechanism (such as a reserve price) for a given block could be informed by the (publicly visible) outcomes for previous blocks. (6) Revenue from a block need not be transferred directly to the block's miner and could instead be redirected, for example, to a foundation or to the miners of future blocks."[2] The reason the textbook second-price (Vickrey) auction breaks down here at once is (2) and (3). If the winners are made to pay "the next highest bid," the miner can push that bid up with a fake transaction, and since losing bids are not on the chain, no one can check. Roughgarden's example is simple. In a block with three slots, if the top bids are 10, 8, and 3, an honestly run second-price auction earns 3 × 3 = 9, and if the miner inserts a fake transaction with a bid of 8, its net revenue jumps to 2 × 8 = 16 (the gist of Example 3.5 in the original).[3]

The first-price auction holds up well against (1) and (2). It is in the miner's interest to pick the high bids anyway, and fake transactions get it nothing. The trouble is **on the bidders' side**. The Motivation section of EIP-1559 names four problems. First, "Mismatch between volatility of transaction fee levels and social cost of transactions": "It's absurd to suggest that the cost incurred by the network from accepting one more transaction into a block actually is 10x more when the cost per gas is 10 nanoeth compared to when the cost per gas is 1 nanoeth; in both cases, it's a difference between 8 million gas and 8.02 million gas." Second, "Needless delays for users": because the block gas limit is fixed, there is no "slack" mechanism that lets one block be larger and the next smaller. Third, "Inefficiencies of first price auctions": "This is well-known in mechanism design literature to be highly inefficient, and so complex fee estimation algorithms are required. But even these algorithms often end up not working very well, leading to frequent fee overpayment." Fourth, the "Instability of blockchains with no block reward" seen in Chapter 20.[4] The PGA of Chapter 17 is the third problem in the flesh: an auction in which bots raised their bids again and again with the same nonce, and the losers paid for gas too.

3. EIP-1559: a toll set by formula

The Abstract of the EIP puts the design in one paragraph: "There is a base fee per gas in protocol, which can move up or down each block **according to a formula which is a function of gas used in parent block and gas target (block gas limit divided by elasticity multiplier) of parent block**. The algorithm results in the base fee per gas increasing when blocks are above the gas target, and decreasing when blocks are below the gas target. The base fee per gas is burned. Transactions specify the maximum fee per gas they are willing to give to miners to incentivize them to include their transaction (aka: priority fee). Transactions also specify the maximum fee per gas they are willing to pay total (aka: max fee), which covers both the priority fee and the block's network fee per gas (aka: base fee). Senders will always pay the base fee per gas of the block their transaction was included in, and they will pay the priority fee per gas set in the transaction, as long as the combined amount of the two fees doesn't exceed the transaction's maximum fee per gas."[5] Roughgarden breaks the same design down into eight "key ideas."

#	ROUGHGARDEN'S SUMMARY (§4)	THIS BOOK'S READING
(1)	Each block has a reserve price computed by the protocol, the "base fee," and paying it is a prerequisite for inclusion	The **lower bound** of the toll is inside the protocol
(2)	The base fee is a function **of the preceding blocks only** and does not depend on the transactions in the current block	The producer cannot manipulate the toll through its own block
(3)	All revenue from the base fee is burned	The toll is no one's revenue
(4)	Double the maximum block size ($C_{max} = 2C$), with the old maximum C as the **target** size	"Slack": one block can be large and the next small
(5)	Raise the base fee if the latest block is larger than the target, lower it if smaller	Block size is **an on-chain signal of excess demand**
(6)	Users specify not a single bid but **a tip and a fee cap**, and a transaction is included only if its cap is at least the base fee	maxPriorityFeePerGas and maxFeePerGas in Chapter 13
(7)	A transaction of size s with tip δ and cap c, included in a block with base fee r, pays $s \cdot \min\{r + \delta, c\}$	The "effective price" of the Yellow Paper in Chapter 13
(8)	The base fee share ($s \cdot r$) is burned, and the rest ($s \cdot \min\{\delta, c - r\}$) goes to the producer	The "share of fees" in Chapters 10 and 13

6

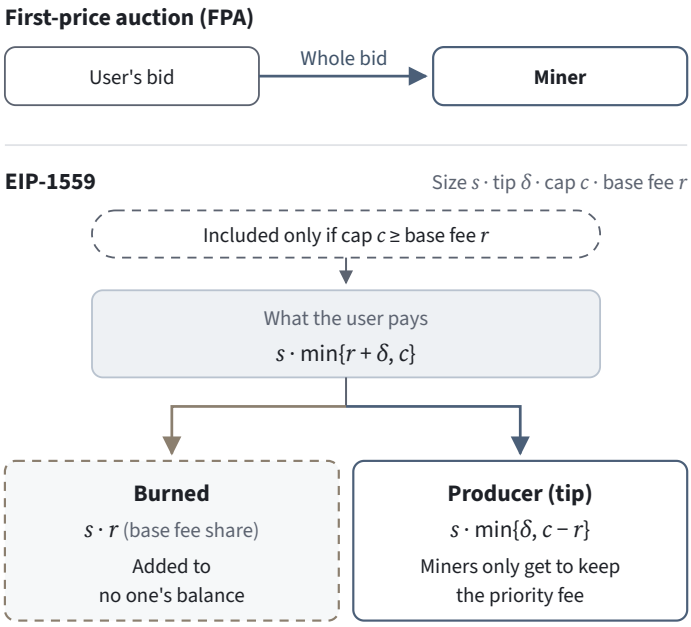


Figure 21-1. Where the fee a user pays goes. In a first-price auction the whole bid goes to the miner; under EIP-1559 the base fee share is burned and only the tip goes to the producer ((6) to (8) in the table above).

[Intermediate] Exactly how much "raise and lower" in (5) means is in the EIP's reference implementation: two constants, `BASE_FEE_MAX_CHANGE_DENOMINATOR = 8` and `ELASTICITY_MULTIPLIER = 2`, and integer arithmetic. The parent block's gas target is its gas limit ÷ 2, and if the parent exceeded the target,

$$b_{t+1} = b_t + \max\left(\left\lfloor \frac{b_t \cdot (g_t - g^*)}{g^*} \right\rfloor / 8,\ 1\right), \qquad \text{below the target: } b_{t+1} = b_t - \left\lfloor \frac{b_t \cdot (g^* - g_t)}{g^*} \right\rfloor / 8$$

with every division an integer division.[7] Reading the symbols: b_t is the parent block's base fee, g_t the parent's gas used, and g^* the parent's gas target. If the parent was full ($g_t = 2g^*$), the base fee goes up by exactly 1/8, that is 12.5%; if it was empty, it goes down by 12.5%; in between, the change is linear. Roughgarden's footnote says the same thing: "Precisely, empty and maximum-size blocks decrease and increase the base fee by 12.5%, respectively, with the effect of other block sizes then determined by linear interpolation. (In particular, blocks matching the target size do not alter the base fee)."[8] The numerical example comes from history. The first block of the London fork, 12,965,000, started with a base fee of `INITIAL_BASE_FEE`, 1 gwei (10^9 wei), and with a gas limit of 30,029,122 and gas used of 30,025,257 it was almost full. The base fee of the next block should then be $10^9 + \lfloor 10^9 \cdot (30{,}025{,}257 - 15{,}014{,}561) \,/\, 15{,}014{,}561 \rfloor \,/\, 8 = 1{,}124{,}967{,}822$ wei, and that is the actual value in block 12,965,001.[9] NB21-1 below runs this check.

```
# NB21-1: EIP-1559 base fee update, with the integer arithmetic of the reference implementation
as is. Checked against the first block of the London fork, then run on a congestion scenario
(for study)
BASE_FEE_MAX_CHANGE_DENOMINATOR = 8      # constant in the EIP-1559 text
ELASTICITY_MULTIPLIER = 2
INITIAL_BASE_FEE = 1_000_000_000          # 1 gwei (in wei)

def next_base_fee(parent_base_fee: int, parent_gas_used: int, parent_gas_limit: int) -> int:
    """A function of the parent block only (Roughgarden's idea (2)). Every division is integer
division."""
    target = parent_gas_limit // ELASTICITY_MULTIPLIER
    if parent_gas_used == target:
        return parent_base_fee
    if parent_gas_used > target:
        delta = parent_base_fee * (parent_gas_used - target) // target //
BASE_FEE_MAX_CHANGE_DENOMINATOR
        return parent_base_fee + max(delta, 1)
    delta = parent_base_fee * (target - parent_gas_used) // target //
BASE_FEE_MAX_CHANGE_DENOMINATOR
    return parent_base_fee - delta

# Check 1: first block of the London fork, 12,965,000 (2021-08-05T12:33:42Z) → 12,965,001.
Header values are the raw responses of two public RPC endpoints (see the footnote)
b_12965000, used, limit = INITIAL_BASE_FEE, 30_025_257, 30_029_122
calc = next_base_fee(b_12965000, used, limit)
print(f"Block 12,965,000: base fee {b_12965000:,} wei, gas {used:,}/{limit:,} (target {limit //
2:,})")
print(f"Computed base fee of 12,965,001: {calc:,} wei; matches the value on the chain,
1,124,967,822: {calc == 1_124_967_822}")

# Check 2: a full block gives exactly +12.5%, an empty block -12.5%
print(f"After a full block: {next_base_fee(10**9, 60_000_000, 60_000_000) / 10**9:.4f} gwei,
after an empty block: {next_base_fee(10**9, 0, 60_000_000) / 10**9:.4f} gwei")

# Scenario: at the gas limit observed on the reference date, 60,000,000 (target 30,000,000),
congestion (20 full blocks) → easing (20 blocks at 1/4 of the target) → normal (at the target)
limit = 60_000_000                        # observed 2026-10-24 (not a constant; Appendix A-10)
scenario = [limit] * 20 + [limit // 8] * 20 + [limit // 2] * 5
b = 10**9
print(f"\n{'Block':>4} {'Gas used':>12} {'Base fee (gwei)':>18} {'Change':>8}")
for i, g in enumerate(scenario, 1):
    nb = next_base_fee(b, g, limit)
    if i <= 3 or i in (10, 20, 21, 22, 23, 30, 40, 41, 45):
        print(f"{i:>4} {g:>12,} {nb / 10**9:>18.4f} {(nb / b - 1):>+8.2%}")
    b = nb
print("After 20 full blocks in a row the base fee is (1.125)^20 ≈ {:.1f} times as high. When
demand falls it comes down (by 12.5% per block if the blocks are empty).".format(1.125 ** 20))
```

```
# Output — bcbook (Python 3.13), exactly as run
Block 12,965,000: base fee 1,000,000,000 wei, gas 30,025,257/30,029,122 (target 15,014,561)
Computed base fee of 12,965,001: 1,124,967,822 wei; matches the value on the chain,
1,124,967,822: True
After a full block: 1.1250 gwei, after an empty block: 0.8750 gwei

Block     Gas used    Base fee (gwei)   Change
   1   60,000,000             1.1250  +12.50%
   2   60,000,000             1.2656  +12.50%
   3   60,000,000             1.4238  +12.50%
  10   60,000,000             3.2473  +12.50%
  20   60,000,000            10.5451  +12.50%
  21    7,500,000             9.5565   -9.37%
  22    7,500,000             8.6606   -9.37%
  23    7,500,000             7.8486   -9.37%
  30    7,500,000             3.9403   -9.37%
  40    7,500,000             1.4723   -9.37%
  41   30,000,000             1.4723   +0.00%
  45   30,000,000             1.4723   +0.00%
After 20 full blocks in a row the base fee is (1.125)^20 ≈ 10.5 times as high. When demand falls
it comes down (by 12.5% per block if the blocks are empty).
```

[Intermediate] On the reference date (as of 2026-10), the observed gas limit on mainnet is 60,000,000, so the target is 30,000,000. This is not a constant. It is a value that producers move block by block within ±1/1024 of the parent's, and it is entered as item A-10 in the index of Appendix A. The two constants (2 and 8) had not changed in any fork up to the reference date.[10] This chapter's interactive figure W9 puts the scenario of this cell in motion, and its only variable is the gas used per block.

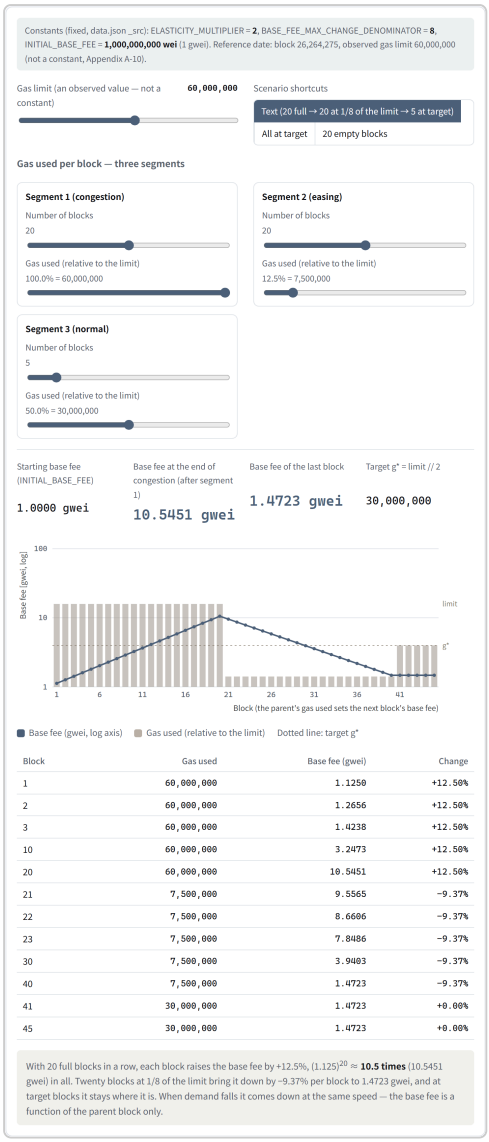


Figure 21-2. EIP-1559 base fee. With a gas limit of 60,000,000 (target 30,000,000), 20 full blocks in a row take the base fee from 1 gwei to 10.5451 gwei (+12.5% per block); 20 blocks at 1/8 of the limit bring it down to 1.4723 gwei; at target blocks it stays where it is. — In the HTML edition you can change the values here (HTML edition, Chapter 21, Section 3).

4. Three kinds of incentive compatibility: who can cheat?

[Intermediate] Roughgarden's contribution was to turn "is this design good?" into "who can deviate, and in what way?" There are three parties who can deviate: users, the producer, and the two in collusion. To each corresponds one condition.

- **DSIC (users).** The condition is that, "assuming that the miner carries out the intended allocation rule, every user (no matter what their valuation) has a dominant strategy—a bid that always maximizes the user's utility (3), no matter what the bids of the other users." FPAs are "of course, not DSIC."[11] My best bid depends on what others will bid.
- **MMIC (the myopic producer).** The condition is that, "for every on-chain history H and mempool M, a myopic miner maximizes its utility … by creating no fake transactions … and following the suggestion of the allocation rule x." First-price auctions are MMIC, and

second-price auctions, as in the example above, are not.[12]

- **OCA-proof (off-chain collusion).** Suppose that the miner and users enter into an agreement (OCA) that specifies "a bid vector b," "an allocation vector x," and "a per-size transfer τ_t from the creator of each transaction $t \in T$ to the miner m," with the bids made on-chain and the transfers off-chain. There must then exist an individually rational bidding strategy such that, "for every on-chain history H, ... there is no OCA under which the utility (3) of **every transaction creator and the utility (5) of the miner** is strictly higher."[13]

[Intermediate] Table 1 of the paper gives the results at a glance.

MECHANISM	MMIC	DSIC	OCA-PROOF
First-price auction (FPA)	yes	no	yes
Second-price auction (SPA)	no	almost	almost
FPA burning a fraction (β) of fees	yes	no	**no**
1559	yes	**usually** (Theorem 5.7)	yes
1559 burning only part of the base fee	yes	usually	**no**
Mechanism without tips (tipless)	yes	yes	usually

[14]

[Intermediate] Let us see what the "usually" of 1559 means. Theorem 5.7 has two conditions: "the base fee should **not be excessively low** for the current demand curve ..., and transaction creators should use individually rational bidding strategies." Then "the 1559 mechanism acts as a posted-price mechanism, with price equal to the reserve price plus the miner's opportunity cost for transaction inclusion," and the user's "obvious optimal bid" is to set the cap at their own valuation and the tip at the marginal cost μ. It breaks down when demand grows quickly and the base fee has not yet risen to catch up. If the transactions willing to pay the base fee do not all fit even in a maximum-size block, Roughgarden writes that, just as the 1559 mechanism "reverts to an FPA in the special case of a zero reserve price," in 1559 too "one might expect bidding wars."[15] The formula did not **replace** the auction. It is a design in which the formula does the work in normal times and the auction can come back when demand spikes. Roughgarden wrote, "We conjecture that there is no TFM that always satisfies all three properties," and added in a footnote that the conjecture was later proved.[16]

```
# NB21-2: first-price auction vs 1559, with the same users and the same block space. How the
guess "what will others bid?" enters what is paid (for study)
# Valuation v is willingness to pay per unit of gas (a unitless model value). Block capacity C =
4 transactions, all of the same size 1. Miner's marginal cost μ = 0.
users = {"A": 10.0, "B": 8.0, "C": 7.5, "D": 7.0, "E": 6.0, "F": 3.0}      # valuations (true)
C = 4

def fpa(bids):
    """First-price auction: include the top C bids; each pays its own bid, all of it to the
miner."""
    order = sorted(bids.items(), key=lambda kv: -kv[1])[:C]
    return {u: b for u, b in order}, sum(b for _, b in order), 0.0

def mech1559(valuations, base_fee, tip=0.01):
    """1559: cap = valuation, tip = a small value near μ (the 'obvious optimal bid' of Theorem
5.7). The base fee is burned; only the tip goes to the miner."""
    eligible = sorted(((u, v) for u, v in valuations.items() if v >= base_fee), key=lambda kv: -
kv[1])[:C]
    paid = {u: min(base_fee + tip, v) for u, v in eligible}
    burned = base_fee * len(eligible)
    return paid, sum(paid.values()) - burned, burned

print("(a) FPA: if users bid their true valuations as they are, they overpay")
paid, miner, burned = fpa(users)
print(f"   Included: {paid}  miner revenue {miner:.2f}, burned 0")
print("   A paid 10 but would have got in by paying just over 6 (highest losing bid E = 6), so
in an FPA bidding your valuation is not a dominant strategy (not DSIC)")

print("\n(a') FPA: imitating something like an equilibrium in which everyone shades down to
'near the highest losing bid'")
shaded = {u: round(v * 0.8, 2) for u, v in users.items()}
paid, miner, burned = fpa(shaded)
print(f"   Included: {paid}  miner revenue {miner:.2f}; how far to shade depends on guesses
about others' bids")

for base_fee in [6.5, 7.2, 3.0]:
    paid, miner, burned = mech1559(users, base_fee)
    label = "adequate" if base_fee == 6.5 else ("high" if base_fee == 7.2 else "excessively
low")
    print(f"\n(b) 1559, base fee {base_fee} ({label}): included {paid}")
    n_elig = sum(1 for v in users.values() if v >= base_fee)
    if n_elig > C:
        print(f"   Burned {burned:.2f}, miner revenue (tips) {miner:.2f}. Eligible users
{n_elig} > capacity {C}: this code does not compute the tip competition (the exception in
Theorem 5.7; see the next line).")
    else:
        print(f"   Burned {burned:.2f}, miner revenue (tips) {miner:.2f}. Users need not guess
others' bids (cap = valuation).")
print("\nWhen the base fee is excessively low, like 3.0, more users are willing to pay it than
there is capacity, and tip competition comes back: this is where 1559 reverts to an FPA (the
conditions of Theorem 5.7).")
```

```
# Output — bcbook (Python 3.13), exactly as run
(a) FPA: if users bid their true valuations as they are, they overpay
   Included: {'A': 10.0, 'B': 8.0, 'C': 7.5, 'D': 7.0}  miner revenue 32.50, burned 0
   A paid 10 but would have got in by paying just over 6 (highest losing bid E = 6), so in an
FPA bidding your valuation is not a dominant strategy (not DSIC)

(a') FPA: imitating something like an equilibrium in which everyone shades down to 'near the
highest losing bid'
   Included: {'A': 8.0, 'B': 6.4, 'C': 6.0, 'D': 5.6}  miner revenue 26.00; how far to shade
depends on guesses about others' bids

(b) 1559, base fee 6.5 (adequate): included {'A': 6.51, 'B': 6.51, 'C': 6.51, 'D': 6.51}
   Burned 26.00, miner revenue (tips) 0.04. Users need not guess others' bids (cap = valuation).

(b) 1559, base fee 7.2 (high): included {'A': 7.21, 'B': 7.21, 'C': 7.21}
   Burned 21.60, miner revenue (tips) 0.03. Users need not guess others' bids (cap = valuation).

(b) 1559, base fee 3.0 (excessively low): included {'A': 3.01, 'B': 3.01, 'C': 3.01, 'D': 3.01}
   Burned 12.00, miner revenue (tips) 0.04. Eligible users 6 > capacity 4: this code does not
compute the tip competition (the exception in Theorem 5.7; see the next line).

When the base fee is excessively low, like 3.0, more users are willing to pay it than there is
capacity, and tip competition comes back: this is where 1559 reverts to an FPA (the conditions
of Theorem 5.7).
```

Burning and the reserve price cannot be separated

[Intermediate] The question Chapter 17 passed on, why the base fee is not given to the producer, is answered by the two "no" entries in Table 1: Roughgarden's Corollaries 5.17 and 5.18. An FPA that burns part of the fees is not OCA-proof: "burning fees in an FPA catalyzes a miner and users to collude off-chain to avoid the burn." Conversely, a 1559 that gives the producer even part of the base fee is not OCA-proof either: "passing revenue from a reserve price to a block's miner opens the door for low-value transactions to pay the reserve price on-chain while receiving a partial refund from the miner off-chain." So "two of the major innovations in the 1559 mechanism, relative to an FPA—a reserve price and burned fees—are **inextricably linked**, as making either change without the other leads to a non-OCA-proof transaction fee mechanism."[17] The EIP itself says something to the same effect: "An important aspect of this fee system is that miners only get to keep the priority fee. The base fee is always burned (i.e. it is destroyed by the protocol). … reducing risks associated with miner extractable value (MEV). … Finally, ensuring the miner of a block does not receive the base fee is important because it removes miner incentive to manipulate the fee in order to extract more fees from users."[18] If the producer received the base fee, it would have an incentive to fill its own blocks on purpose to push up the base fee of the blocks that follow. Burning is the device that cuts off that incentive.

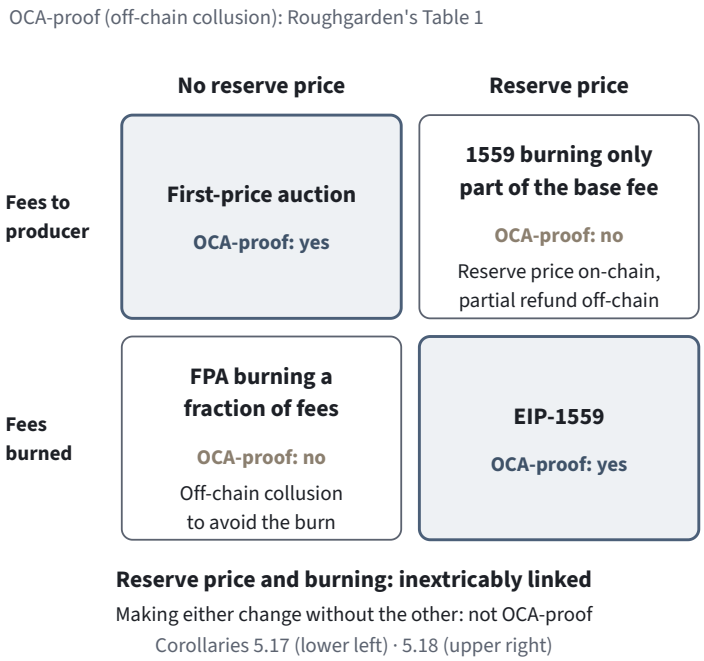


Figure 21-3. The OCA-proof column of Roughgarden's Table 1, rearranged along two axes: whether there is a reserve price, and to whom the fees go. The two cells that change only one of the two are "no."

[Intermediate] Here we cut off one common line of reasoning about burning. "Burning → lower supply → higher price" is not a proposition this chapter deals with, nor is it the ground of this design. The Security Considerations section of the EIP is, on the contrary, cautious: "By burning the base fee, we can no longer guarantee a fixed Ether supply. ... If more is burned on base fee than is generated in mining rewards then ETH will be deflationary and if more is generated in mining rewards than is burned then ETH will be inflationary. Since we cannot control user demand for block space, we cannot assert at the moment whether ETH will end up inflationary or deflationary, so this change causes the core developers to lose some control over Ether's long term quantity."[19] As a game-theoretic alternative to burning, Roughgarden gives a pay-it-forward implementation, which is "to instead split the base fee revenues of a block equally among the miners of the next ℓ blocks," and states the difference between the two **in terms of whom each favors and what becomes variable**: "While burning the base fee revenue favors holders of the native currency (through deflation), this 'pay-it-forward' implementation favors miners (through more transaction fee revenue)." And with burning, "the blockchain's inflation rate would be variable and unpredictable from block to block, but miner revenue (which effectively pays for the blockchain's security [3, 8]) would stay relatively constant .... With the pay-it-forward implementation, the inflation rate would be essentially deterministic but the miner rewards (and, hence, blockchain security) would be unpredictable ...."[20] The security budget of Chapter 20 comes back here. Burning is a choice that settles **where the fees go**, as between holders and producers, and that choice changes how variable the security budget is. Price appears nowhere in this account.

[Intermediate] The gap that remains is manipulation **across several blocks**. Roughgarden's analysis is on the time scale of a single block, and 1559 ties blocks together through its history-dependent base fee: "For example, publishing a smaller-than-target block decreases the base fee for the next block, potentially increasing the revenue of that block's miner. ...

Long-term manipulation by a cartel of miners thus resembles the challenge of sustaining collusion in a repeated multi-player Prisoner's Dilemma game: If all players cooperate … they are all better off, but each player has a myopic incentive to unilaterally deviate from this strategy." He writes that "Persistent and harmful miner collusion has not yet been definitively observed in a major blockchain …," but leaves open that non-myopic strategies may pay off "in a regime in which a single miner or validator might control a significant fraction (30%, say) of the overall hashrate or stake."[21] The game theory of Chapter 23 deals with this gap.

5. [Advanced] The blob fee market: the same principle, a different resource

▾ [Advanced] Blob gas in EIP-4844 and the exponential update

[Advanced] Chapter 18 introduced the "blob" as the place where rollups post their data and passed its pricing on to this chapter. EIP-4844 (Shard Blob Transactions, status Final) gives blobs **a gas of their own**: "We introduce blob gas as a new type of gas. It is independent of normal gas and follows its own targeting rule, similar to EIP-1559. We use the `excess_blob_gas` header field to store persistent data needed to compute the blob gas base fee. For now, only blobs are priced in blob gas."[22] The constants are: one blob is 2^{17} blob gas, the target per block is 393,216 (3 blobs), the maximum is 786,432 (6 blobs), the minimum base fee is 1 wei, and the update fraction is 3,338,477. The update rule is not the linear interpolation of 1559 but **exponential**: "If block N consumes X blob gas, then in block $N + 1$ `excess_blob_gas` increases by X - TARGET_BLOB_GAS_PER_BLOCK, and so the `base_fee_per_blob_gas` of block $N + 1$ increases by a factor of `e**((X - TARGET_BLOB_GAS_PER_BLOCK) / BLOB_BASE_FEE_UPDATE_FRACTION)`. … Hence, it has a similar effect to the existing EIP-1559, but is more 'stable' in the sense that it responds in the same way to the same total usage regardless of how it's distributed. … It [BLOB_BASE_FEE_UPDATE_FRACTION] is chosen to target a maximum change rate of `e**(TARGET_BLOB_GAS_PER_BLOCK / BLOB_BASE_FEE_UPDATE_FRACTION)` ≈ 1.125 per block."[23] That is the same 12.5% as 1559's 1/8. The blob fee too "is deducted from the sender balance before transaction execution and **burned**, and is not refunded in case of transaction failure."[24] The EIP contains a function, `fake_exponential`, that imitates the exponential function in integer arithmetic, and the cell below ports it as it is.

```
# NB21-3: EIP-4844 base fee per blob gas, porting the text's fake_exponential and constants as
they are (for study)
MAX_BLOB_GAS_PER_BLOCK = 786_432          # 6 blobs
TARGET_BLOB_GAS_PER_BLOCK = 393_216       # 3 blobs
MIN_BASE_FEE_PER_BLOB_GAS = 1             # wei
BLOB_BASE_FEE_UPDATE_FRACTION = 3_338_477
GAS_PER_BLOB = 2**17

def fake_exponential(factor: int, numerator: int, denominator: int) -> int:
    """EIP-4844 Helpers: integer approximation of factor * e**(numerator/denominator) by
Taylor expansion (as in the text)."""
    i = 1
    output = 0
    numerator_accum = factor * denominator
    while numerator_accum > 0:
        output += numerator_accum
        numerator_accum = (numerator_accum * numerator) // (denominator * i)
        i += 1
    return output // denominator

def calc_excess_blob_gas(parent_excess: int, parent_blob_gas_used: int) -> int:
    if parent_excess + parent_blob_gas_used < TARGET_BLOB_GAS_PER_BLOCK:
        return 0
    return parent_excess + parent_blob_gas_used - TARGET_BLOB_GAS_PER_BLOCK

def base_fee_per_blob_gas(excess: int) -> int:
    return fake_exponential(MIN_BASE_FEE_PER_BLOB_GAS, excess, BLOB_BASE_FEE_UPDATE_FRACTION)

import math
print(f"Rate of change per block filled to the maximum (6 blobs; the target is 3):
e^(393216/3338477) = {math.exp(TARGET_BLOB_GAS_PER_BLOCK /
BLOB_BASE_FEE_UPDATE_FRACTION):.4f}")
# Scenario: 30 blocks in a row with 6 blobs (the maximum) → 30 blocks with 0 blobs → the
target (3 blobs)
excess = 0
print(f"{'Block':>4} {'Blobs':>6} {'excess_blob_gas':>16} {'Base fee per blob gas
(wei)':>26}")
for i, blobs in enumerate([6] * 30 + [0] * 30 + [3] * 3, 1):
    fee = base_fee_per_blob_gas(excess)
    if i in (1, 2, 3, 10, 20, 30, 31, 32, 40, 50, 60, 61, 63):
        print(f"{i:>4} {blobs:>6} {excess:>16,} {fee:>26,}")
    excess = calc_excess_blob_gas(excess, blobs * GAS_PER_BLOB)
print("After 30 maximum blocks the excess is 30 × 393,216 = {:,} and the fee is e^(that
value/3,338,477) ≈ {:.0f} times as high. After 30 empty blocks the excess returns to 0 and the
fee to the minimum of 1 wei.".format(30 * 393_216, math.exp(30 * 393_216 / 3_338_477)))
```

```
# Output — bcbook (Python 3.13), exactly as run
Rate of change per block filled to the maximum (6 blobs; the target is 3): e^(393216/3338477)
= 1.1250
Block  Blobs  excess_blob_gas Base fee per blob gas (wei)
   1      6               0                          1
   2      6         393,216                          1
   3      6         786,432                          1
  10      6       3,538,944                          2
  20      6       7,471,104                          9
  30      6      11,403,264                         30
  31      0      11,796,480                         34
  32      0      11,403,264                         30
  40      0       8,257,536                         11
  50      0       4,325,376                          3
  60      0         393,216                          1
  61      3               0                          1
  63      3               0                          1
After 30 maximum blocks the excess is 30 × 393,216 = 11,796,480 and the fee is e^(that
value/3,338,477) ≈ 34 times as high. After 30 empty blocks the excess returns to 0 and the fee
to the minimum of 1 wei.
```

[Advanced] The constants in this cell are the values in the text of EIP-4844. In the course of checking, it was noted that the bpo1 and bpo2 forks that followed the fork active on

the reference date (Fusaka) "change only blob parameters." The target and maximum blob counts on mainnet on the reference date may therefore differ from the 3 and 6 of the text, and this book has not verified the current values. They are entered as a task in item A-17 of the index of Appendix A. The principle is the same: a separate resource, a separate target, an exponential update, burning.

> **? A LAWYER'S QUESTION**
>
> The base fee is deducted from the sender's balance and **added to no one's balance**. That is how the code is written, and so Roughgarden describes it as "permanently removed from the circulating supply." To whom does this burned fee belong? Is it consideration the user paid to "the network"? If so, is the network a party that receives payment? Did it go back to all remaining holders "through deflation"? If so, is that a transfer, or did nothing happen at all? Within what framework does the law see "the extinction of value that belongs to no one": abandonment, an ownerless thing (res nullius), consumption, or a new category that is none of these? Under the pay-it-forward alternative, the same value would have gone to future producers, and that choice can be changed by amending the protocol. This book does not answer, and points to the map of scholarly positions in Part 5. → Chapter 27

6. Summary

Block space is scarce, and the fee market allocates it. The first-price auction is strong against the producer but makes users guess "what will everyone else pay?", and the failures of that guessing showed up as overpayment and volatility. EIP-1559 has the protocol compute the lower bound of the toll from the congestion of the previous block (by at most 12.5% at a time), burns that lower bound, and gives the producer only the tip. Within Roughgarden's single-block model, this design always holds up against deviation by the producer and against off-chain collusion, holds up against strategic bidding by users except when demand spikes, and its burning and reserve price are a pair: take away either one and the door to collusion opens. Burning is a choice that sends the fees to the holders' side and a device that cuts off the producer's incentive to manipulate; it is not a proposition about price. The blob fee market applies the same principle to a different resource with an exponential update. Chapter 22 reads the other side, not fees but **issuance and distribution** (who receives how much at the start, and who changes the rules), as design variables.

Sources for this chapter

- Tim Roughgarden, "Transaction Fee Mechanism Design," *Journal of the ACM* 71(4), Article 30 (2024), DOI 10.1145/3674143: the problem setting and the FPA (30:1–30:2), the four idiosyncrasies and the two things the designer can use (30:2), summary of results (30:4–30:5), the burning rule in Definition 2.5 (30:9), the definitions of DSIC, MMIC, and OCA-proof (§3, 30:10–30:13), the second-price auction example (Example 3.5, 30:12), the eight ideas of 1559 with Definition 4.1 and footnote 24 (§4, 30:13–30:15), Table 1 (30:16), Corollaries 5.17 (30:21) and 5.18 (30:22), the discussion in §6 (30:22–30:23). (published version). arXiv 2106.01340v3 (the version under review) and the 2020 report are not cited.
- EIP-1559, "Fee market change for ETH 1.0 chain" (Buterin, Conner, Dudley, Slipper, Norden, Bakhta; Final; created 2019-04-13): Abstract, Motivation (the four problems and the purpose of burning), Specification (the constants and the `validate_block` reference implementation), Security Considerations ("ETH Burn Precludes Fixed Supply"). https://github.com/ethereum/EIPs/blob/e858e912ff55370eb8125873455af531e322d9b2/EIPS/eip-1559.md (accessed 2026-10-25).
- EIP-4844, "Shard Blob Transactions" (Buterin and 6 others; Final; created 2022-02-25): Parameters, Helpers (`fake_exponential`), Gas accounting, Rationale (update rule, throughput). https://github.com/ethereum/EIPs/blob/e858e912ff55370eb8125873455af531e322d9b2/EIPS/eip-4844.md (accessed 2026-10-25).
- go-ethereum v1.17.10 `params/chainspecs/mainnet.json` L17 (`londonBlock` 12965000), `params/protocol_params.go` L157–L159, `params/config.go` L1016–L1023, commit 1df30b18 (`widgets/data.json` `_src`).
- Headers of blocks 12,964,999, 12,965,000, and 12,965,001: raw responses to `eth_getBlockByNumber` from the public RPC endpoints https://eth.drpc.org (all three blocks) and https://rpc.flashbots.net (12,964,999 and 12,965,000; 12,965,001 was pruned) (accessed 2026-10-25; note 9). The gas limit of 60,000,000 on the reference date is from `widgets/data.json` `ethereum.eip1559.observed_latest_block` (Appendix A-10).
- Philip Daian et al., "Flash Boys 2.0" (2020): the PGA observations cited in Chapter 17 (this chapter only points to Chapter 17).
- Code run with Python 3.13 (conda environment `bcbook`), using only the standard library `math` . Notebook NB21. The initial state of interactive figure W9 `fee-market` is the scenario of NB21-1 (gas limit 60,000,000, target 30,000,000; 20 full blocks → 20 blocks at 1/8 → 5 at the target), and its only variable is the gas used per block.

Notes

1. Tim Roughgarden, "Transaction Fee Mechanism Design," *Journal of the ACM* 71(4), Article 30 (2024), 30:1–30:2. Emphasis added. Original: "Real estate on a major blockchain is a scarce resource. ... Demand for these blockchains is far larger, necessitating a mechanism that selects a subset of transactions to include 'on-chain' from the pool of all submitted transactions. Historically, most blockchain protocols have employed a pay-as-bid transaction fee mechanism. Every transaction is submitted with a bid (in the blockchain's native currency), the miner of a block decides which transactions should be included in it, and upon publication of

that block, the bid of each included transaction is transferred from its creator to the miner. We follow blockchain convention and refer to this mechanism as a first-price auction (FPA)." (published version). The 2020 report (arXiv:2012.00854) has a different title ("... for the Ethereum Blockchain: An Economic Analysis of EIP-1559"), and this book cites the JACM version.

2. Roughgarden (2024), 30:2. Emphasis added.

3. Roughgarden (2024), 30:12, Example 3.5 (Second-Price-Type Auctions Are Not MMIC). Original: "Now imagine that the top three bids are 10, 8, and 3. If a miner honestly executes the auction, its revenue will be 3 × 3 = 9. If the miner instead submits a fake transaction with bid 8 and executes the auction (with the top two real transactions included along with the fake transaction), its net revenue jumps to 2 × 8 = 16."

4. Vitalik Buterin, Eric Conner, Rick Dudley, Matthew Slipper, Ian Norden & Abdelhamid Bakhta, "EIP-1559: Fee market change for ETH 1.0 chain" (Standards Track, Core, status Final, created 2019-04-13), Motivation section. Original: "Mismatch between volatility of transaction fee levels and social cost of transactions: ... It's absurd to suggest that the cost incurred by the network from accepting one more transaction into a block actually is 10x more when the cost per gas is 10 nanoeth compared to when the cost per gas is 1 nanoeth; in both cases, it's a difference between 8 million gas and 8.02 million gas."; "Needless delays for users: because of the hard per-block gas limit coupled with natural volatility in transaction volume, transactions often wait for several blocks before getting included ... no one significantly gains from the fact that there is no 'slack' mechanism"; "Inefficiencies of first price auctions: ... This is well-known in mechanism design literature to be highly inefficient, and so complex fee estimation algorithms are required. But even these algorithms often end up not working very well, leading to frequent fee overpayment."; for the fourth item, see Chapter 20, note 24. https://github.com/ethereum/EIPs/blob/e858e912ff55370eb8125873455af531e322d9b2/EIPS/eip-1559.md (accessed 2026-10-25; the raw markdown was downloaded again and compared word for word).

5. EIP-1559, Abstract section. Emphasis added.

6. Roughgarden (2024), 30:13–30:14 (§4). Original (1)–(3): "Each block has a protocol-computed reserve price (per unit size) called the base fee. Paying the base fee is a prerequisite for inclusion in a block. (2) The base fee is a function of the preceding blocks only and does not depend on the transactions included in the current block. (3) All revenues from the base fee are burned—that is, permanently removed from the circulating supply of the native currency." (4)–(5): "Double the maximum block size (i.e., define $C_{max} := 2C$), with the old maximum C serving as the new target block size ($C_{target} := C$). (5) Adjust the base fee upward or downward whenever the size of the latest block is bigger or smaller than the target block size, respectively." (6)–(8): "Rather than a single bid, the creator of a transaction is now responsible for specifying both a tip and a fee cap for it. A transaction will be included in a block only if its fee cap is at least the block's base fee. (7) If a size-s transaction with tip δ and fee cap c is included in a block with base fee r, the transaction creator pays a total of s · min{r + δ, c}. (8) Revenue from the base fee (i.e., s·r) is burned and the remainder (s·min{δ, c − r}) is transferred to the miner of the block." Roughgarden calls (2) "underspecified" ("how, exactly, is the base fee derived from the preceding blocks?") and gives (4) and (5) as the answer: "If miners publish a sequence of maximum-size blocks, how can the protocol distinguish whether the current base fee is too low or exactly right?"

7. EIP-1559, the reference implementation in the Specification section (`validate_block`). Original (relevant part): "parent_gas_target = self.parent(block).gas_limit // ELASTICITY_MULTIPLIER"; "elif parent_gas_used > parent_gas_target: gas_used_delta = parent_gas_used - parent_gas_target; base_fee_per_gas_delta = max(parent_base_fee_per_gas * gas_used_delta // parent_gas_target // BASE_FEE_MAX_CHANGE_DENOMINATOR, 1); expected_base_fee_per_gas = parent_base_fee_per_gas + base_fee_per_gas_delta"; "else: gas_used_delta = parent_gas_target - parent_gas_used; base_fee_per_gas_delta = parent_base_fee_per_gas * gas_used_delta // parent_gas_target // BASE_FEE_MAX_CHANGE_DENOMINATOR; expected_base_fee_per_gas = parent_base_fee_per_gas - base_fee_per_gas_delta". Constants: "INITIAL_BASE_FEE = 1000000000", "BASE_FEE_MAX_CHANGE_DENOMINATOR = 8", "ELASTICITY_MULTIPLIER = 2". The second independent source is go-ethereum v1.17.10 `params/protocol_params.go` L157–L159 (`widgets/data.json` `ethereum.eip1559._src`). The same implementation states that the block gas limit may move only within ±1/1024 of the parent's and must be at least 5,000. The gas limit is not a constant but an observed value that producers signal.

8. Roughgarden (2024), 30:14, footnote 24.

9. The block number at which the London fork activated is given in go-ethereum v1.17.10 `params/chainspecs/mainnet.json` L17 `"londonBlock": 12965000` (commit 1df30b18, accessed 2026-10-25). For the header of block 12,965,000, two public RPC endpoints (`eth_getBlockByNumber` at https://eth.drpc.org and at https://rpc.flashbots.net, accessed 2026-10-25) returned the same values; 12,965,001 was obtained from eth.drpc.org alone (rpc.flashbots.net answered for this block with code 4444 and the message "pruned history unavailable"): 12,965,000: hash `0x9b83c12c…eee71` , timestamp 1628166822 (2021-08-05T12:33:42Z), `baseFeePerGas` 0x3b9aca00 (10^9), `gasLimit` 30,029,122, `gasUsed` 30,025,257; the `gasLimit` of the preceding block 12,964,999 was 15,029,237 (the elasticity multiplier 2 doubled the limit at the fork block); 12,965,001: `baseFeePerGas` 1,124,967,822, `gasLimit` 29,999,798, `gasUsed` 29,985,144. Another commonly used public node (publicnode) had pruned the history at this height and did not respond. Roughgarden's published version, at 30:2, gives the London fork as "block number 12865000 [sic], which was mined on August 5, 2021," and that block number differs by one digit from the client setting 12,965,000 (it appears to be a slip; this book follows the setting). The two sources agree on the date.

10. `widgets/data.json` `ethereum.eip1559.observed_latest_block` (block 26,264,275, 2026-10-24T18:23Z, three RPC endpoints agreeing: `gasLimit` 60,000,000, `gasUsed` 32,652,385) and `_note` (go-ethereum v1.17.10 `params/config.go` L1016–L1023 gives the two constants with no branching by fork).

11. Roughgarden (2024), 30:11 (§3.1). Original: "A TFM is then dominant-strategy incentive compatible (DSIC) if, assuming that the miner carries out the intended allocation rule, every user (no matter what their valuation) has a dominant strategy—a bid that always maximizes the user's utility (3), no matter what the bids of the other users. FPAs are, of course, not DSIC."

12. Roughgarden (2024), 30:12 (§3.2), Definition 3.4. Original: "A TFM (x, p, q) is MMIC if, for every on-chain history H and mempool M, a myopic miner maximizes its utility (5) by creating no fake transactions (i.e., setting F = ∅) and following the suggestion of the allocation rule x (i.e., setting $B_k = x(H, M)$)."; "For example, FPAs are MMIC—the intended allocation rule maximizes miner net revenue, which the miner is happy to do ...". "Myopic" means a miner that looks only at the net revenue of the current block (30:11).

13. Roughgarden (2024), 30:12–30:13 (§3.3), Definitions 3.6 and 3.7. Emphasis added. Original: "A TFM (x, p, q) is OCA-proof if, for every on-chain history H, there exists an individually rational bidding strategy σ_H such that, for every possible set T of outstanding transactions and valuations v, there is no OCA under which the utility (3) of every transaction creator and the utility (5) of the miner is strictly higher than in the outcome $B_k = x(H, M(\sigma_H(v)))$ with on-chain bids $\sigma_H(v)$ and no off-chain transfers."

14. Roughgarden (2024), 30:16, Table 1 ("Which of the Six Listed TFMs are MMIC, DSIC, or OCA-Proof"). The "almost" entries in the table rest on footnote 28 (an SPA that uses the lowest winning bid as a proxy for the highest losing bid is not DSIC in general but is approximately so) and Remark 5.13, and the "usually" entries rest on Theorem 5.7, Remark 5.9, and Corollary 5.15 + Remark 5.16. Summary sentence: "Thus, if we assess these TFMs solely according to these three types of incentive guarantees, FPAs dominate β-burn FPAs with β > 0; the 1559 mechanism dominates these and β-burn 1559 mechanisms with β < 1; and the 1559 mechanism and the tipless mechanism are incomparable."

15. Roughgarden (2024), 30:4 (the summary of results in §1) and 30:22 (§6). Emphasis added. Original: "The 1559 mechanism reverts to an FPA in the special case of a zero reserve price, but Theorem 5.7 identifies two conditions that together are sufficient for the optimality of straightforward bidding: the base fee should not be excessively low for the current demand curve (Definition 5.6), and transaction creators should use individually rational bidding strategies. Under these conditions, the 1559 mechanism acts as a posted-price mechanism, with price equal to the reserve price plus the miner's opportunity cost for transaction inclusion."; "In a block with an excessively low base fee (presumably due to rapidly increasing demand), one might expect bidding wars in the 1559 mechanism (on account of the failure of DSIC)"; "The 'obvious optimal bid' in the 1559 mechanism (setting $c_t = v_t$ and $\delta_t = \mu$) is optimal only in blocks with neither an excessively low base fee nor overbidding."

16. Roughgarden (2024), 30:22 and footnote 33. Original: "We conjecture that there is no TFM that always satisfies all three properties."; footnote 33: "[This conjecture was recently proved by Chung et al. [17]]."

17. Roughgarden (2024), 30:4–30:5 (summary of results in §1) and 30:22 (Corollaries 5.17 and 5.18). Emphasis added. Original: "Finally, OCA-proofness considerations show that two of the major innovations in the 1559 mechanism, relative to an FPA—a reserve price and burned fees—are inextricably linked, as making either change without the other leads to a non-OCA-proof transaction fee mechanism. Intuitively, burning fees in an

FPA catalyzes a miner and users to collude off-chain to avoid the burn (Corollary 5.17) while passing revenue from a reserve price to a block's miner opens the door for low-value transactions to pay the reserve price on-chain while receiving a partial refund from the miner off-chain (Corollary 5.18)."

18. EIP-1559, last paragraph of the Motivation section. Original: "An important aspect of this fee system is that miners only get to keep the priority fee. The base fee is always burned (i.e. it is destroyed by the protocol). This ensures that only ETH can ever be used to pay for transactions on Ethereum, cementing the economic value of ETH within the Ethereum platform and reducing risks associated with miner extractable value (MEV). Additionally, this burn counterbalances Ethereum inflation while still giving the block reward and priority fee to miners. Finally, ensuring the miner of a block does not receive the base fee is important because it removes miner incentive to manipulate the fee in order to extract more fees from users." From this paragraph this book takes only the part about the mechanism (removing the miner's incentive to manipulate), and leaves statements such as "cementing the economic value of ETH" only as a statement of the designers' intent.

19. EIP-1559, Security Considerations section, "ETH Burn Precludes Fixed Supply." Original: "By burning the base fee, we can no longer guarantee a fixed Ether supply. This could result in economic instability as the long term supply of ETH will no longer be constant over time. While a valid concern, it is difficult to quantify how much of an impact this will have. If more is burned on base fee than is generated in mining rewards then ETH will be deflationary and if more is generated in mining rewards than is burned then ETH will be inflationary. Since we cannot control user demand for block space, we cannot assert at the moment whether ETH will end up inflationary or deflationary, so this change causes the core developers to lose some control over Ether's long term quantity."

20. Roughgarden (2024), 30:22–30:23 (§6, "Alternatives to burning"). Original: "One appealing alternative implementation, with the same incentive-compatibility properties, is to instead split the base fee revenues of a block equally among the miners of the next ℓ blocks."; "While burning the base fee revenue favors holders of the native currency (through deflation), this 'pay-it-forward' implementation favors miners (through more transaction fee revenue)."; "Thus, the blockchain's inflation rate would be variable and unpredictable from block to block, but miner revenue (which effectively pays for the blockchain's security [3, 8]) would stay relatively constant (modulo fluctuations in the market price of the native currency). With the pay-it-forward implementation, the inflation rate would be essentially deterministic but the miner rewards (and, hence, blockchain security) would be unpredictable (though never less than that with money burning)." Definition 2.5 (30:9) states that burning "can be equated with a lump-sum refund to holders of the currency through deflation, à la stock buybacks." That is a description of the mechanism, not a proposition about price.

21. Roughgarden (2024), 30:23 (§6, "Multi-block time scales"). Original: "For example, publishing a smaller-than-target block decreases the base fee for the next block, potentially increasing the revenue of that block's miner. ... Long-term manipulation by a cartel of miners thus resembles the challenge of sustaining collusion in a repeated multi-player Prisoner's Dilemma game: If all players cooperate ... they are all better off, but each player has a myopic incentive to unilaterally deviate from this strategy ..."; "Persistent and harmful miner collusion has not yet been definitively observed in a major blockchain such as Bitcoin or Ethereum."; "especially in a regime in which a single miner or validator might control a significant fraction (30%, say) of the overall hashrate or stake and therefore might plausibly benefit from non-myopic strategies."

22. Vitalik Buterin, Dankrad Feist, Diederik Loerakker, George Kadianakis, Matt Garnett, Mofi Taiwo & Ansgar Dietrichs, "EIP-4844: Shard Blob Transactions" (Standards Track, Core, status Final, created 2022-02-25, requires 1559, 2718, 2930, 4895), Gas accounting section. https://github.com/ethereum/EIPs/blob/e858e912ff55370eb8125873455af531e322d9b2/EIPS/eip-4844.md (accessed 2026-10-25, raw markdown). Checked against the raw file of the same commit.

23. EIP-4844, the Parameters table (`MAX_BLOB_GAS_PER_BLOCK` 786432, `TARGET_BLOB_GAS_PER_BLOCK` 393216, `MIN_BASE_FEE_PER_BLOB_GAS` 1, `BLOB_BASE_FEE_UPDATE_FRACTION` 3338477, `GAS_PER_BLOB` 217) and the Rationale sections "Base fee per blob gas update rule" and "Throughput." Original: "If block N consumes X blob gas, then in block N+1 `excess_blob_gas` increases by X - TARGET_BLOB_GAS_PER_BLOCK, and so the `base_fee_per_blob_gas` of block N+1 increases by a factor of e((X - TARGET_BLOB_GAS_PER_BLOCK) / BLOB_BASE_FEE_UPDATE_FRACTION). Hence, it has a similar effect to the existing EIP-1559, but is more 'stable' in the sense that it responds in the same way to the same total usage regardless of how it's distributed."; "The parameter BLOB_BASE_FEE_UPDATE_FRACTION controls the maximum rate of change of the base fee per blob gas. It is chosen to target a maximum change rate of e**

(TARGET_BLOB_GAS_PER_BLOCK / BLOB_BASE_FEE_UPDATE_FRACTION) ≈ 1.125 per block."; "The values for TARGET_BLOB_GAS_PER_BLOCK and MAX_BLOB_GAS_PER_BLOCK are chosen to correspond to a target of 3 blobs (0.375 MB) and maximum of 6 blobs (0.75 MB) per block. These small initial limits are intended to minimize the strain on the network ... and are expected to be increased in future upgrades".

24. EIP-4844, Gas accounting section. Emphasis added. Original: "The actual `blob_fee` as calculated via `calc_blob_fee` is deducted from the sender balance before transaction execution and burned, and is not refunded in case of transaction failure."

PART 3 · ECONOMICS

Chapter 22. The economics of token design: issuance rules, governance, incentives

KEY TAKEAWAY

Chapter 14 said that a token standard is "a promise about function names," and that what rights a token carries is set by the issuer's promise outside the code. This chapter reads that outside promise **as a set of design variables.** There are three. How much is minted, and by what rule (issuance). Who receives it at the start (distribution). Who can change the rules (governance). The three variables resemble a company's articles of incorporation and the design of its share issuance, but one thing is different. **There is no enforcer.** Inside the code there is no court to call an issuer who broke a promise to account, and no general meeting of shareholders at which to contest a breach of the articles; what there is, is majority vote and forks. The term "tokenomics" is often used to dress these three variables up as grounds for value. This chapter instead reads them **as a map of risks.** Issuance rules can be changed, distribution concentrates, and governance belongs to whoever has the majority. After this chapter you will know what to look at first when you are handed a token design document, and why "what can the minority assert when a majority of governance tokens changes the rules?" is a question of law.

Learning objectives

1. State, in the words of the original sources, the distinction between coins and tokens and the three types of issuance rule (fixed or decreasing, perpetual or variable, elastic) with the economic implications of each. Know how burning changes an issuance rule.
2. Read the four channels of distribution (mining and validation rewards, presales, free distribution, team and foundation allocations) **in terms of mechanism and risk only.** Carry over the measured values of concentration together with their sample and period.
3. **[Intermediate]** Compute voting concentration with the Gini coefficient and the top-k share of holdings, and know the measurement limit "address ≠ person."
4. Read the voting structure of governance tokens through the official record of The DAO, and know whom majority vote, the Curators, and the split were each designed to protect, and where there was no enforcer.

What a reader of the [Basic] paragraphs alone takes away. Token design resembles a company's articles of incorporation and share issuance design in structure, but differs in that **there is no enforcer.**

1. The question this chapter tries to answer

Chapter 19 looks at why the price of a coin that promises nothing can be positive, and Chapter 24 at why a token carrying the promise "we will give you back 1 dollar" suffers runs. Between the two lies a wide territory: tokens that promise something, but whose promise is not redemption. The right to a share of a protocol's fees, the right to vote on rule changes, the right to access a service, or just the words "take part in the growth of the ecosystem." The design documents for these tokens usually set out the issuance amount, the distribution schedule, and the voting structure under the heading "tokenomics." This chapter is about how to read those documents. It does not ask about price. It asks **whom the three variables allow to do what.**

First, one term. The 2018 paper by Ke Wu, Spencer Wheatley, and Didier Sornette draws "an essential distinction that some cryptos are 'coins' (which operate on their own independent network) and others 'tokens' (which operate on top of a coin network as a platform)."[1] This book uses that distinction too. A coin (bitcoin, ether) has its issuance rule in the protocol itself, like the gas of Chapter 13 and the rewards of Chapter 20. A token has its issuance rule in **a contract that someone deployed**, like the ERC-20 contracts of Chapter 14. The way the rules are changed also differs. Changing a coin's rules takes a fork of the kind in Chapter 23; a token's rules depend on how the person who wrote the contract wrote it.

2. Issuance: how much, and by what rule

Issuance rules fall into three types.

TYPE	RULE	EXAMPLE SEEN IN THIS BOOK	WHAT THE DESIGN TAKES ON
Fixed or decreasing	A cap on the total and a schedule of decreases are in the code	Bitcoin: halving every 210,000 blocks, converging to just under 21 million (Chapter 20, NB20-1)	The subsidy goes to 0, so fees have to fill the security budget (Chapter 20, Carlsten et al.)
Perpetual or variable	Minting continues under a rule with no cap, but if there is burning, net issuance can be negative (−)	Ethereum: block rewards continue, and the base fee is burned (Chapter 21)	The designers cannot control net supply (the Security Considerations section of EIP-1559)
Elastic	Supply is increased and decreased to meet a price target	Algorithmic stablecoins (Chapter 24)	Supply adjustment relies on demand for a second token, so a reflexive feedback loop arises (Chapter 24, NB24-3)

The third type is treated in Chapter 24, so here we look at the first and the second. Fixed issuance is the simplest and the one most often packaged as "grounds for value." Chapter 19 took that packaging off. In Garratt and Wallace's equation (13), the issuance amount B is in the denominator, but the proportion α of people who want to use the coin is in the numerator, and if $\alpha = 0$ the price is 0 however small B is. Schilling and Uhlig's Theorem 2 showed that even the intuition that issuance dilutes holders does not survive intact in a model of currency competition: block rewards "are not a tax on Bitcoin holders but are

financed through Dollar taxes."[2] And Carlsten et al. in Chapter 20 set out the **design cost** of fixed issuance. As the subsidy goes to 0 the mining game becomes unstable, so "Perhaps instead, designers of new cryptocurrencies must resign themselves to the inevitability of monetary inflation and make the block reward permanent."[3] Fixed issuance is a choice that stakes the funding of the security budget on a future fee market, and the cost of that choice is measured by equation (3) of Chapter 20.

The second type runs into the same problem from the other side. The Ethereum white paper gave ether the "dual purpose of providing a primary liquidity layer to allow for efficient exchange between various types of digital assets and, more importantly, of providing a mechanism for paying transaction fees,"[4] and EIP-1559 in 2021, by burning part of those fees, made the path of net supply something **the designers cannot control.** This is a sentence read in Chapter 21: because demand for block space cannot be controlled, it cannot be asserted at present whether ETH will turn out inflationary or deflationary, and so the change makes the core developers lose some control over the long-term quantity of ether.[5] Here we cut short, once more, a common way of talking about "burning." Burning is one term in the issuance rule, and, as Roughgarden wrote in Chapter 21, it is a choice that places the **accrual** of fees on the holders' side. Van Oordt's 2024 model says that this choice can change the properties of a bubble equilibrium: "Cryptocurrencies with negative issuance (e.g., that burn transaction fees) may generate positive aggregate cash flows to investors even if their price path follows a bubble trajectory."[6] This is not "burning makes the price go up." It is **a proposition inside the model**, "with burning, a certain equilibrium of this model departs from Ponzi equivalence," and it can be read again after Chapter 25 has defined what a "bubble" is.

For a token (something a contract issues), the issuance rule is in the contract code, and as Chapter 14 showed, the ERC-20 standard **does not fix it.** Whether the total is fixed, whether the issuer can mint more, and who can burn are all functions outside the standard. Even if a design document says "fixed total supply," the supply is not fixed if a minting function remains in the contract. So the first question in reading a token's issuance rule concerns not the document but **who retains minting authority in the contract.**

3. Distribution: who receives it at the start

Issued units go to someone. There are four channels.

- **Mining and validation rewards.** As Nakamoto wrote, the reward "provides a way to initially distribute coins into circulation, since there is no central authority to issue them."[7] Those who receive it are those who put in resources (computing power, collateral), and the basis of distribution is the share of resources.
- **Presales.** Tokens are sold before issuance to raise funds. The DAO of Chapters 14 and 26 is an example with an official record. According to the report of the U.S. Securities and Exchange Commission (SEC), "From April 30, 2016 through May 28, 2016, The DAO offered and sold approximately 1.15 billion DAO Tokens in exchange for a total of approximately 12 million Ether ('ETH')," and "The token price fluctuated in a range of approximately 1 to

1.5 ETH per 100 DAO Tokens, depending on when the tokens were purchased during the Offering Period. … There were no limitations placed on the number of DAO Tokens offered for sale, the number of purchasers of DAO Tokens, or the level of sophistication of such purchasers."[8]

- **Free distribution (airdrops).** Tokens are sent to addresses that meet set conditions (past use, holdings, particular actions). There is no official record that this book has confirmed in a primary source, so only the mechanism and the risks are set down here. Because the recipients bore no cost, they have an incentive to dispose of the tokens at once (dumping); if the conditions are automated, it is hard to stop one person from meeting them through many addresses (Sybil); and even when a distribution looks broad, real control can remain with a few (concentration).
- **Team, foundation, and investor allocations.** A set share of the issuance is assigned to the development team, the foundation, and early investors, and is often locked for a period (vesting). The points at which the locks are released become steps in the supply.

None of the four channels is good or bad in itself. What this chapter asks about is the result, that is, **concentration.** Measured values of concentration are few, and even those are hard to obtain. As Chapter 11 showed, an address is not a person, and a single exchange cold wallet contains the balances of many customers. Makarov and Schoar take that problem on directly: "Determining the concentration of ownership is more complicated than just tracking the holdings of the richest addresses, since many of the largest addresses belong to cold wallets of exchanges and online wallets, which hold Bitcoin on behalf of many investors." So they use graph analysis to sort addresses into those of individuals and those of intermediaries.[9] The results are carried over below together with their samples and periods.

MEASURE	VALUE	SAMPLE AND BASIS	SOURCE AND LIMITS
"Rich addresses" with 1,000 BTC or more	2,258 addresses, 7.9 million BTC in total (almost half of circulation)	Balances on 2020-12-31	Address-level; the authors write that this reading ignores cold wallets
Of those: individual / intermediary / ambiguous	1,013 / 1,154 / 47	Same basis	Classification by graph analysis. The three categories add up to 2,214, 44 fewer than the total of 2,258; the numbers on p. 28 of the original are exactly so, and the original does not explain the difference
Held by intermediaries	5.5 million BTC, about 1/3 of circulation	End of 2020	Rising steadily since 2014
Held by individuals (individuals among the rich addresses + clusters of 1–1,000 BTC with no transactions throughout 2020)	About 400,000 clusters, 8.5 million BTC in total	End of 2020	Intermediary wallets are filtered out by the inactivity condition
Concentration of individual holdings	The top 1,000 clusters have about 3 million BTC, and the top 10,000 clusters more than 5 million BTC (about 1/4 of the total issued)	End of 2020	The authors state explicitly that this is **an underestimate**: they did not merge several addresses of the same entity, and did not treat the roughly 20,000 early addresses as one person (Nakamoto)
Concentration of mining	The top 10% of miners have 90% of capacity, and the top 0.1% (about 50) about 50%	The past 5 years (as of 2021), tracing the reward distribution of the top 20 pools	Concentration is countercyclical: it grows after sharp price falls and after halvings

10

What to take from this table is less the numbers than **the lesson about measurement.** Counting addresses overestimates concentration (cold wallets), and counting clusters underestimates it (several clusters belonging to the same entity). Either way, the word "decentralized" does not mean equality of distribution. Concentration on the mining side connects directly to the threshold of Chapter 23. That the top 0.1% control half of capacity is concentration at the level of individual miners behind the pools (the authors write that pools do not necessarily control their miners), while Eyal and Sirer's threshold is a model value at the pool level with $\gamma = 0$, so the two cannot be joined directly. Still, the fact that the deciding parties are few is something to keep in mind when reading the threshold of Chapter 23.

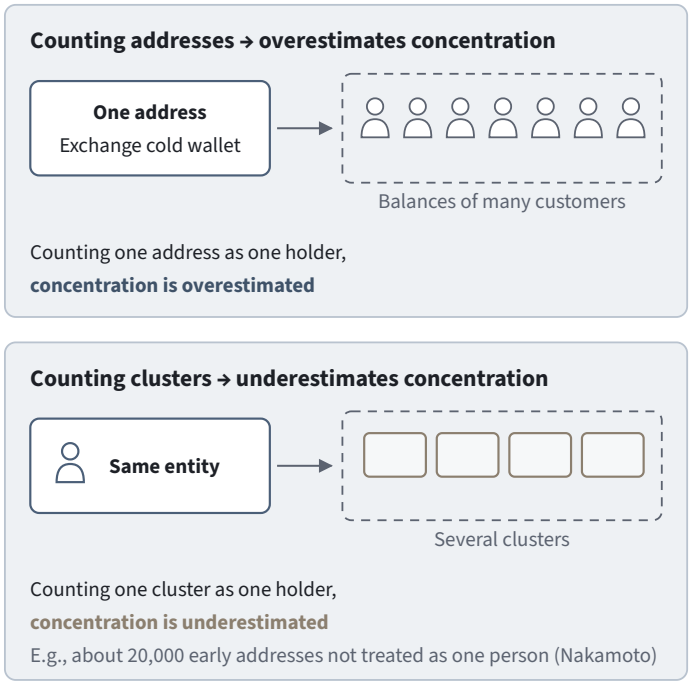


Figure 22-1. The lesson about measurement from the Makarov–Schoar table. Because a single exchange cold-wallet address contains the balances of many customers, counting addresses overestimates concentration, and counting the several clusters of one entity separately underestimates it. No classification figures are included; only the direction of each bias is drawn.

[Intermediate] Two numbers that measure concentration: the Gini coefficient and the top-k share

[Intermediate] If the holdings are sorted in ascending order as $x_1 \le x_2 \le \cdots \le x_n$, the **Gini coefficient** is

$$G = \frac{2\sum_{i=1}^{n} i\,x_i}{n\sum_{i=1}^{n} x_i} - \frac{n+1}{n}$$

Reading the symbols: it is 0 if everyone has the same amount, and $1 - 1/n$ (1 for large n) if one person has everything. The **top-k share of holdings** is the sum of the k largest holdings divided by the total. A numerical example: if five holders have 5, 5, 10, 20, and 60, then $\sum x_i = 100$, $\sum i x_i = 1\cdot 5 + 2\cdot 5 + 3\cdot 10 + 4\cdot 20 + 5\cdot 60 = 425$, and $G = 2\cdot 425 / (5\cdot 100) - 6/5 = 1.7 - 1.2 = 0.5$; the top-1 share is 60% and the top-2 share is 80%. In token-weighted voting, if the top-1 share exceeds 50%, that one address decides every proposal on its own. Both numbers presuppose "address = person," so the lesson of the table above applies directly. If the same entity splits its holdings across addresses, G goes down, and if an exchange votes with its customers' tokens, the top-k share becomes the exchange's. By design, this chapter has no code. The two formulas can be computed by hand; what is hard is not the formula but the denominator.

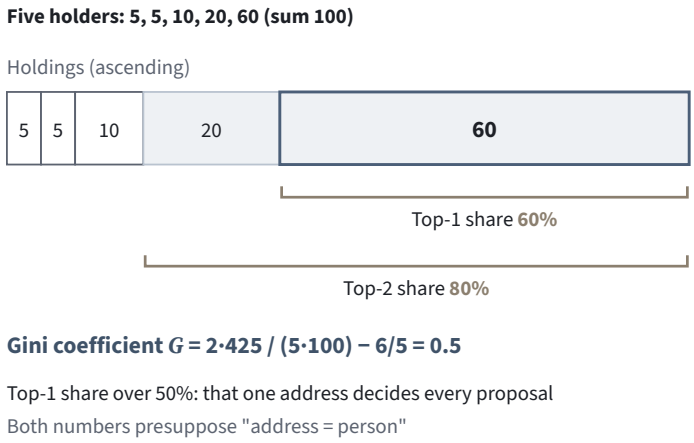


Figure 22-2. The numerical example in the text (holdings of 5, 5, 10, 20, 60) laid out on a single bar. The top-1 and top-2 shares and the Gini coefficient of 0.5 are values obtained from the formula in the text.

4. Governance: who changes the rules

A governance token is a token that carries "the right to vote." What is voted on differs from contract to contract. The governance section of the Uniswap v3 white paper is a concise example: "The factory has an owner, which is initially controlled by UNI tokenholders. The owner does not have the ability to halt the operation of any of the core contracts." There are three things the owner can do. Uniswap v3 "has a protocol fee that can be turned on by UNI governance," and "UNI governance has more flexibility in choosing the fraction of swap fees that go to the protocol, and is able to choose any fraction $1 / N$ where $4 \leq N \leq 10$, or 0. This parameter can be set on a per-pool basis." "UNI governance also has the ability to add additional fee tiers. ... Once a fee tier is added to the factory, it cannot be removed ...." "Finally, UNI governance has the power to transfer ownership to another address."[11] The mathematics of the AMM seen in Chapter 15 stays as it is, and on top of it sits a small set of articles: "who turns on the fee switch." It matters that what cannot be done (halting) and what cannot be undone (removing a tier) are spelled out. Token holders have exactly as much power as the code leaves open.

The most detailed official record is The DAO. Buterin's 2014 white paper described a "decentralized organization" as "a virtual entity that has a certain set of members or shareholders which, perhaps with a 67% majority, have the right to spend the entity's funds and modify its code. ... This essentially replicates the legal trappings of a traditional company or nonprofit but **using only cryptographic blockchain technology for enforcement**."[12] The DAO of 2016 was its first large-scale implementation, and the SEC report records its voting structure: "ETH raised by The DAO was to be distributed to a Contractor to fund a proposal only on a majority vote of DAO Token holders. DAO Token holders were to cast votes, which would be **weighted by the number of tokens they controlled**, for or against the funding of a specific proposal." That vote had a known bias: "The voting process, however, was publicly criticized in that it could incentivize distorted voting behavior and, as a result, would not accurately reflect the consensus of the majority of DAO Token holders. Specifically, as noted in a May 27, 2016 blog post by a group of

computer security researchers, The DAO's structure included a 'strong positive bias to vote YES on proposals and to suppress NO votes as a side effect of the way in which it restricts users' range of options following the casting of a vote.'" A footnote explains the mechanism. Tokens that had been voted were tied up until the end of the voting cycle, while tokens that abstained could be moved at any time, so "DAO Token holders were incentivized either to vote yes or to abstain from voting."[13]

Above the majority vote stood the **Curators.** "Before any proposal was put to a vote by DAO Token holders, it was required to be reviewed by one or more of The DAO's 'Curators.' At the time of the formation of The DAO, the Curators were a group of individuals chosen by Slock.it. According to the White Paper, the Curators of a DAO Entity had 'considerable power.' The Curators performed crucial security functions and maintained ultimate control over which proposals could be submitted to, voted on, and funded by The DAO." One of the Curators said publicly that the Curator had "complete control over the whitelist … [and] clear ability to control the order and frequency of proposals," noting that "curators have tremendous power," and another Curator publicly announced subjective criteria that included his personal ethics. A Curator also had "the power to reduce the voting quorum requirement by 50% every other week."[14] And there was one device for protecting the minority: **the split.** As a footnote in the report puts it, "According to the White Paper, the primary purpose of a split is to protect minority shareholders and prevent what is commonly referred to as a '51% Attack,' whereby an attacker holding 51% of a DAO Entity's Tokens could create a proposal to send all of the DAO Entity's funds to himself or herself." Token holders could turn their tokens back into ETH through "a complicated, multi-week (approximately 46-day) process."[15] Put into the language of corporate law, majority vote (the general meeting of shareholders), a few appointees with the power to put items on the agenda (something like a board of directors, but unelected), and a dissenting shareholder's right of exit (something like the appraisal right) were written in code. But there was **nowhere to contest** them. When an attacker used a defect in the code ("a flaw in The DAO's code," in the words of the SEC report) to move about 1/3 of the funds, the option left was not a court but a hard fork. That incident is read in Chapter 26.

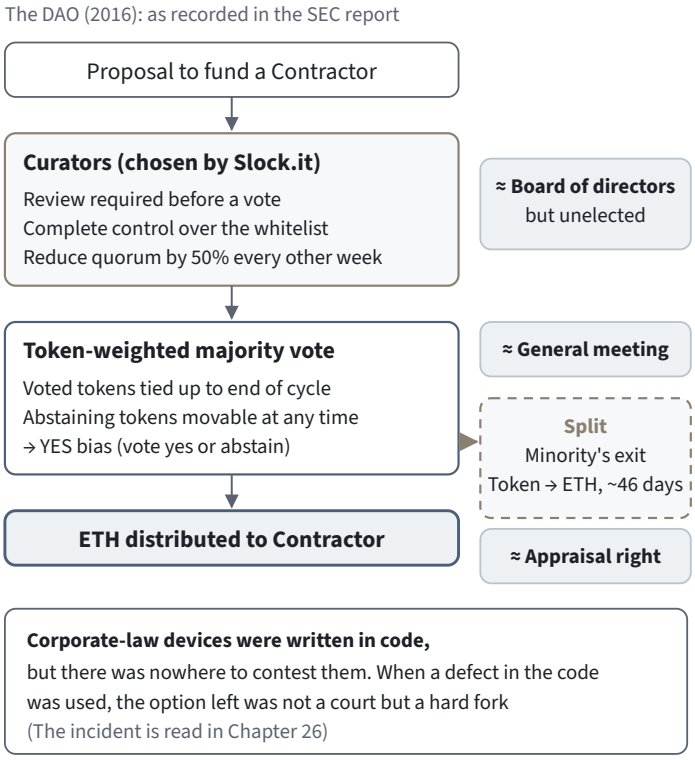


Figure 22-3. The decision path of The DAO as recorded in the SEC report, with its corporate-law counterparts. The Curators decided what was put to a vote, token-weighted voting decided the allocation, and the minority's exit was the split.

Putting the three variables together, here are the questions for reading a governance design document. (1) Is voting power proportional to the number of tokens, and if so, what is the top-k share? (2) Is there a gatekeeper for putting proposals on the agenda, and who chooses and who replaces that gatekeeper? (3) Is there anything the code bars the majority from doing (like Uniswap's "cannot halt")? (4) Is there a way out for the minority, and how long does it take? (5) Does the code execute the result of a vote automatically, or does it need someone's signature (whoever has the multisig keys is the de facto enforcer)? None of these five appears in the issuance-amount chart of a "tokenomics" document.

5. How "tokenomics" is used as a marketing term

Design documents often write the three variables **as a narrative of value**: the issuance cap as "scarcity," burning as "deflation," staking rewards as "returns," airdrops as "giving back to the community." The sentences the SEC report quotes from The DAO's promotional materials are a sample of that tone. The founder explained that participants would send ETH to purchase tokens, "which would permit the participant to vote and entitle the participant to 'rewards,'" and **likened this to "buying shares in a company and getting ... dividends,"** and the website described The DAO's purpose as "To blaze a new path in business for the betterment of its members, existing simultaneously nowhere and everywhere and operating solely with the steadfast iron will of unstoppable code."[16] This chapter does not pass judgment on this narrative. It has only shown that the same variables can **also be read the other way.** Scarcity is not a price without demand (Chapter 19), burning is a choice about where fees accrue (Chapter 21), rewards are a mechanism (Chapters 10 and 20), free distribution is a channel for dumping and Sybil attacks, and voting power belongs to the top k. When you hear the word "tokenomics," there is one translation this chapter suggests: **"Who can change what, and who is there to stop it?"**

> **? A LAWYER'S QUESTION**
>
> A majority of the governance tokens changed the rules. They turned on the fee switch, passed a proposal the minority opposed, or amended the issuance rule. The votes were counted as the code prescribed, and the result was executed automatically. What can the minority assert? In a company there are the principle of shareholder equality (주주평등), the duty of loyalty (충실의무), minority shareholder rights (소수주주권), and the appraisal right (주식매수청구권). Is a DAO a partnership (조합)? If so, do its members bear unlimited liability (무한책임)? Is it a juridical person (법인)? Under which law was it established? Or is it nothing at all? If so, what is the relationship among the token holders, and to whom does a person with the power to put items on the agenda, like the Curators, owe duties? The DAO white paper itself wrote that "the legal status of [DAO Entities] remains the subject of active and vigorous debate and discussion" (SEC report, p. 3, note 10). This book does not answer and points to the issue section in Part 5. → Chapter 28

6. Summary

The variables of token design are three: issuance, distribution, and governance. Issuance rules divide into fixed, perpetual, and elastic, and each pays a price (the security budget, control of net supply, and reflexivity, respectively); for a token, it is the contract, not the standard, that sets the rule. Distribution runs through four channels and results in concentration, and measuring concentration runs into the wall of "address ≠ person." Even so, the measured values point to the top 10,000 clusters having about 1/4 of the total issued and the top 0.1% of miners having about half of capacity. A governance token is voting power to the extent the code leaves open, and the record of The DAO shows majority vote, a gatekeeper, and a right of exit written in code, with nowhere to contest them. It resembles a company's articles but has no enforcer: that is this chapter in one sentence. Chapter 23 takes up how the largest vote that changes the rules, the fork, looks in game theory, and Chapter 26 reads how that vote actually took place in The DAO.

Sources for this chapter

- Ke Wu, Spencer Wheatley & Didier Sornette, "Classification of cryptocurrency coins and tokens by the dynamics of their market capitalizations," *Royal Society Open Science* 5: 180381 (2018), DOI 10.1098/rsos.180381 — the distinction between coins and tokens (p. 2). Only the one-sentence definition was used.
- Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform" (2014; first draft 2013) — "Decentralized Autonomous Organizations" (PDF p. 23), "Currency And Issuance" (PDF p. 30).
- SEC, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Release No. 81207 (2017-07-25) — the sale (pp. 2–3 and 6), promotional language (pp. 4–5), the split and note 21 (p. 6), voting and note 24 (p. 7), the Curators (pp. 7–8).
- Hayden Adams et al., "Uniswap v3 Core," March 2021 — §4 Governance (p. 3).
- Igor Makarov & Antoinette Schoar, NBER Working Paper 29396 (2021) — ownership concentration (pp. 5–6; §5, pp. 26–29), mining concentration (p. 4).
- Susan Athey et al., Stanford GSB WP 3469 (2016) — ownership concentration (p. 3). Satoshi Nakamoto (2008) §6. Linda Schilling & Harald Uhlig (2019) Theorem 2. Miles Carlsten et al. (2016) §7. EIP-1559 Security Considerations. Maarten van Oordt (2024) abstract — each at the same place as in Chapters 19, 20, and 21.
- The numerical example for the Gini coefficient and the top-k share (5, 5, 10, 20, 60 → $G = 0.5$) is this book's hand calculation. By design, there is no code or notebook in this chapter.
- The description of the mechanism and risks of free distribution (airdrops) was written by this book as a structure, without a primary source, and does not refer to any actual distribution.

Notes

1. Ke Wu, Spencer Wheatley & Didier Sornette, "Classification of cryptocurrency coins and tokens by the dynamics of their market capitalizations," *Royal Society Open Science* 5: 180381 (2018), p. 2: "We make an essential distinction that some cryptos are 'coins' (which operate on their own independent network) and others 'tokens' (which operate on top of a coin network as a platform)." The subject of this paper's analysis is the distribution of market capitalizations, so this book takes only the definition of the distinction and does not carry over the results on the distribution (notes to the reader in Chapter 3).

2. Schilling & Uhlig (2019), p. 22, Theorem 2. Chapter 19, note 7.

3. Carlsten et al. (2016), §1 and §7. Chapter 20, note 23.

4. Vitalik Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform" (2014; first draft 2013), section "Currency And Issuance" (PDF p. 30): "The Ethereum network includes its own built-in currency, ether, which serves the dual purpose of providing a primary liquidity layer to allow for efficient exchange between various types of digital assets and, more importantly, of providing a mechanism for paying transaction fees." The same place as Chapter 14, note 5. The 2014 issuance model described in the same section (sale price, initial allocation) was later changed by forks, so this book does not carry it over.

5. EIP-1559, Security Considerations section, "ETH Burn Precludes Fixed Supply." Chapter 21, note 19.

6. van Oordt (2024), abstract. Chapter 19, note 29.

7. Nakamoto (2008), p. 4 (§6). Chapter 20, note 1.

8. SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017-07-25), pp. 2–3 and 6: "From April 30, 2016 through May 28, 2016, The DAO offered and sold approximately 1.15 billion DAO Tokens in exchange for a total of approximately 12 million Ether ('ETH')"; "The token price fluctuated in a range of approximately 1 to 1.5 ETH per 100 DAO Tokens, depending on when the tokens were purchased during the Offering Period. Anyone was eligible to purchase DAO Tokens (as long as they paid ETH). There were no limitations placed on the number of DAO Tokens offered for sale, the number of purchasers of DAO Tokens, or the level of sophistication of such purchasers." Page 3 of the report also gives the amount raised at the close of the offering converted into dollars, but this book does not carry it over.

9. Makarov & Schoar (2021), p. 5 (introduction): "Determining the concentration of ownership is more complicated than just tracking the holdings of the richest addresses, since many of the largest addresses belong to cold wallets of exchanges and online wallets, which hold Bitcoin on behalf of many investors. We develop a suite of algorithms based on graph analysis to classify addresses into those belonging to individual investors or those belonging to intermediaries."

10. Makarov & Schoar (2021), pp. 4, 6, and 26–29 (§5): "There were 2258 such addresses, which controlled 7.9 million bitcoins — almost half of all bitcoins in circulation. … The fact that so few addresses control almost half of the bitcoins in circulation is often taken as prima facie evidence of the high concentration of Bitcoin holdings. This view, however, neglects the fact that some of these addresses belong to cold wallets and therefore, represent holdings of a large number of people." (p. 27); "Overall, out of the total 2258 rich addresses, we classify 1013 as individual, 1154 as linked to intermediaries, and 47 as ambiguous." (p. 28); "By the end of 2020 is was equal to 5.5 million bitcoins, roughly one-third of Bitcoin in circulation at the time." (p. 28); "There are 400,000 of such clusters and they collectively control 8.5 million bitcoins by the end of 2020." (p. 28); "We see that only 1000 clusters control three million bitcoins and the top 10,000 own more than five million bitcoins which is about a quarter of all outstanding bitcoins. It is also important to note that this measurement of concentration most likely is an understatement since we cannot rule out that some of the largest addresses are controlled by the same entity. In particular, in the above calculations, we do not assign the ownership of early bitcoins, which are held in about 20,000 addresses, to one person (Satoshi Nakamoto) but consider them as belonging to 20,000 different individuals." (p. 29); on mining: "The top 10% of miners control 90% and just 0.1% (about 50 miners) control close to 50% of mining capacity. Furthermore, this concentration of mining capacity is counter cyclical and varies with the Bitcoin price." (p. 4). Athey et al. (2016) also write, as of mid-2015, that "Bitcoin ownership is highly concentrated" (p. 3; Chapter 19, note 9).

11. Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer & Dan Robinson, "Uniswap v3 Core," March 2021, §4 Governance (p. 3): "The factory has an owner, which is initially controlled by UNI tokenholders. The owner does not have the ability to halt the operation of any of the core contracts. As in Uniswap v2, Uniswap v3 has a protocol fee that can be turned on by UNI governance. In Uniswap v3, UNI governance has more flexibility in choosing the fraction of swap fees that go to the protocol, and is able to choose any fraction 1/N

where 4 ≤ N ≤ 10, or 0. This parameter can be set on a per-pool basis. UNI governance also has the ability to add additional fee tiers. … Once a fee tier is added to the factory, it cannot be removed (and the tickSpacing cannot be changed). … Finally, UNI governance has the power to transfer ownership to another address." The same source as Chapter 15.

12. Buterin (2014), section "Decentralized Autonomous Organizations" (PDF p. 23): "The general concept of a 'decentralized organization' is that of a virtual entity that has a certain set of members or shareholders which, perhaps with a 67% majority, have the right to spend the entity's funds and modify its code. The members would collectively decide on how the organization should allocate its funds. Methods for allocating a DAO's funds could range from bounties, salaries to even more exotic mechanisms such as an internal currency to reward work. This essentially replicates the legal trappings of a traditional company or nonprofit but using only cryptographic blockchain technology for enforcement." Emphasis added.

13. SEC (2017), p. 7 and note 24: "ETH raised by The DAO was to be distributed to a Contractor to fund a proposal only on a majority vote of DAO Token holders. DAO Token holders were to cast votes, which would be weighted by the number of tokens they controlled, for or against the funding of a specific proposal. The voting process, however, was publicly criticized in that it could incentivize distorted voting behavior and, as a result, would not accurately reflect the consensus of the majority of DAO Token holders. Specifically, as noted in a May 27, 2016 blog post by a group of computer security researchers, The DAO's structure included a 'strong positive bias to vote YES on proposals and to suppress NO votes as a side effect of the way in which it restricts users' range of options following the casting of a vote.'"; note 24: "By voting on a proposal, DAO Token holders would 'tie up' their tokens until the end of the voting cycle. … If, however, a DAO Token holder abstained from voting, the DAO Token holder could avoid these restrictions; any DAO Tokens not submitted for a vote could be withdrawn or transferred at any time. As a result, DAO Token holders were incentivized either to vote yes or to abstain from voting." Emphasis added.

14. SEC (2017), pp. 7–8: "Before any proposal was put to a vote by DAO Token holders, it was required to be reviewed by one or more of The DAO's 'Curators.' At the time of the formation of The DAO, the Curators were a group of individuals chosen by Slock.it. According to the White Paper, the Curators of a DAO Entity had 'considerable power.' The Curators performed crucial security functions and maintained ultimate control over which proposals could be submitted to, voted on, and funded by The DAO."; "One member of the group chosen by Slock.it to serve collectively as the Curator stated publicly that the Curator had 'complete control over the whitelist … the order in which things get whitelisted, the duration for which [proposals] get whitelisted, when things get unwhitelisted … [and] clear ability to control the order and frequency of proposals,' noting that 'curators have tremendous power.' Another Curator publicly announced his subjective criteria for determining whether to whitelist a proposal, which included his personal ethics. Per the White Paper, a Curator also had the power to reduce the voting quorum requirement by 50% every other week."

15. SEC (2017), p. 6 and note 21: "DAO Token holders would also be permitted to redeem their DAO Tokens for ETH through a complicated, multi-week (approximately 46-day) process referred to as a DAO Entity 'split.'"; note 21: "According to the White Paper, the primary purpose of a split is to protect minority shareholders and prevent what is commonly referred to as a '51% Attack,' whereby an attacker holding 51% of a DAO Entity's Tokens could create a proposal to send all of the DAO Entity's funds to himself or herself." The SEC report does not contain the technical account that the flaw lay in a recursive call in the split function (p. 1 says "a flaw in The DAO's code"); Chapter 26 treats it on the basis of the Ethereum Foundation's announcement (2016-06-17).

16. SEC (2017), pp. 4–5: "In November 2015, at an Ethereum Developer Conference in London, Christoph Jentzsch described his proposal for The DAO as a 'for-profit DAO [Entity],' where participants would send ETH (a virtual currency) to The DAO to purchase DAO Tokens, which would permit the participant to vote and entitle the participant to 'rewards.' Christoph Jentzsch likened this to 'buying shares in a company and getting … dividends.'"; "The DAO Website included a description of The DAO's intended purpose: 'To blaze a new path in business for the betterment of its members, existing simultaneously nowhere and everywhere and operating solely with the steadfast iron will of unstoppable code.'" The SEC relied on these materials as one ground for its determination that DAO Tokens were securities (p. 1); that legal determination is taken up in Chapters 27 and 28. Emphasis added.

PART 3 · ECONOMICS

Chapter 23. Game theory: selfish mining and equilibrium

KEY TAKEAWAY

The rules of a blockchain are not rules "one has no choice but to follow." They are rules **"designed so that following them pays,"** and that design holds only under conditions. The white paper wrote that an attacker "ought to find it more profitable to play by the rules," and for a long time the conventional wisdom held that "honest mining is a Nash equilibrium." Ittay Eyal and Emin Gün Sirer showed that the conventional wisdom was wrong. Under the **selfish mining** strategy, in which a pool hides the blocks it finds instead of publishing them at once, the pool earns more than under honest mining once its share α passes a threshold, and that threshold falls from 1/3 to 0 depending on how much of network propagation the pool controls (γ). So the condition is not "safe if a majority is honest" but "more than 2/3 must be honest," and even that is under the most optimistic assumption. In the priority gas auctions of Chapter 17 the bots converged toward a cooperative state (one that the model of Daian et al. approximates), and in the fee-only regime of Chapter 20 undercutting becomes an equilibrium within the model of Carlsten et al. After this chapter you will know exactly which assumptions the statement "a decentralized system runs without the good will of its participants" rests on, and why the question of which of two chains after a hard fork is "the real one" is a question of game theory.

Learning objectives

1. Write the "honesty pays" argument of §6 of the white paper as a 2×2 payoff table and see that it is a claim about a Nash equilibrium. Read how Budish's one-shot game states the "necessary condition for an honest equilibrium."
2. **[Intermediate]** Draw the state machine of the selfish-mining strategy (lead 0, 0′, 1, 2, ...), measure the pool's relative revenue by Monte Carlo, and compare it with the honest share α.
3. **[Advanced]** Read Eyal and Sirer's closed form R_{pool} and the threshold share $\frac{1-\gamma}{3-2\gamma} < \alpha < \frac{1}{2}$, and check whether the simulation reproduces the formula. Understand why γ is a variable of network propagation and why the authors' fix raises the threshold to 1/4.
4. Look at the choice of fork through the frame of a coordination game, and see that the cooperative equilibrium of Chapter 17 and the undercutting equilibrium of Chapter 20 are different places in the same frame.

What a reader of the [Basic] paragraphs alone takes away. The rules of a blockchain are not rules "one has no choice but to follow." They are rules "designed so that following them pays," and that design holds only under conditions.

1. The question this chapter tries to answer

In Chapters 7 and 8 the "immutability" of a blockchain turned out to be a matter of cost, not of mathematics. Who pays that cost, and why, was the question of Chapter 20, and the answer was the block reward. But the ones who receive the reward are people. People act not as the rules tell them but **as it pays them**. So the question is this. Is the behavior the protocol writes down (publish a block as soon as you find it, build on the longest chain, include all transactions) actually the behavior that pays? If it pays, under what conditions? If it does not, what happens? This is a question of game theory, and the answer takes the shape of **an equilibrium**. To say that a combination of actions is an equilibrium means that no one benefits by deviating from it alone. No more and no less.

2. The white paper's argument as a payoff table

The last sentence of §6 of the white paper, read in Chapter 20, is in fact a game-theoretic claim. The attacker has two strategies (follow the rules / undermine the system), and the white paper says that the payoff of the first is larger: "He ought to find it more profitable to play by the rules, such rules that favour him with more new coins than everyone else combined, than to undermine the system and the validity of his own wealth."[1] Written in the simplest frame, this is a 2×2 game with two players ("me" and "everyone else") and two strategies (honest / attack).

ME ＼ THE REST	HONEST	ATTACK
Honest	A reward in proportion to my share; the system survives	If the rest are a majority, the system is theirs
Attack	If I am a majority, V_{attack} − cost; if a minority, the cost only	The system collapses

For "honest, honest" to be an equilibrium in this table, "even if I am a majority, the net payoff of attacking must be smaller than the reward for honesty." That is the content of equation (3) in Chapter 20, and in §III.E Budish restates the same thing **as a Nash equilibrium of a one-shot game**. Each of I players chooses a quantity of trust support x_i and a stance (honest / attack). If someone has $x_i > N / 2$ and attacks, that player takes V_{attack} and the others get 0; otherwise each receives $\frac{x_i}{N}p$. Lemma 1 says that if there is an honest equilibrium then $N^* c \leq p$ (the counterpart of the zero-profit condition), Lemma 2 says that if some player has chosen 0 then $N^* c \geq V_{attack}$ (the counterpart of incentive compatibility), and Proposition 2 reads: "A necessary condition for an honest equilibrium is $p \geq V_{attack} / (1 + 1 / I)$. In the limit as $I \to \infty$ this is $p \geq V_{attack}$ …."[2] This inequality is exactly the condition under which the white paper's "ought to find" holds. And as Budish adds, this simple model looks only at "honest play or majority attack": "Many prior game-theoretic

analyses of blockchains assume that all players are small, eliminating the possibility of majority attack, but allow for a richer set of strategies for such small players than just honestly following the protocol (e.g., Eyal and Sirer 2014 ...)."[3] The rest of this chapter is that richer set of strategies. Can a player deviate **even as a minority**?

3. Conventional wisdom: "honest mining is a Nash equilibrium"

What Eyal and Sirer attacked in 2013 was exactly that conventional wisdom. Here is the opening of the 2018 version published in CACM: "Conventional wisdom asserts that the mining protocol is incentive-compatible and secure against colluding minority groups, that is, it incentivizes miners to follow the protocol as prescribed. We show that the Bitcoin mining protocol is **not** incentive-compatible. We present an attack with which colluding miners' revenue is larger than their fair share."[4] The ground for the conventional wisdom was simple: "Because the protocol was believed to reward miners in proportion to their ratio of the mining power, a miner in a large pool was believed to earn the same revenue as it would in a small pool. Consequently, if we ignore the fixed cost of pool operation and potential economies of scale, there is no advantage for colluding miners to organize into ever-increasing pools." The related-work section quotes the conventional wisdom directly. There is Edward Felten's "there was a folk theorem that the Bitcoin system was stable, in the sense that if everyone acted according to their incentives, the inevitable result would be that everyone followed the rules of Bitcoin as written," and the assumption, "never proven, but taken on intuitive faith," "that a coalition of miners with X% of the network's hash power can make no more than X% of total mining revenues." The 2013 analysis by Kroll, Davey and Felten, "without taking into account block withholding attacks ... argues that the honest strategy constitutes a Nash equilibrium."[5]

4. Selfish mining: the strategy of hiding blocks

The idea of the strategy fits in one paragraph: "The key idea behind this strategy, called Selfish Mining, is for a pool to keep its discovered blocks private, thereby **intentionally forking the chain**. The honest nodes continue to mine on the public chain, while the pool mines on its own private branch. If the pool discovers more blocks, it develops a longer lead on the public chain, and continues to keep these new blocks private. When the public branch approaches the pool's private branch in length, the selfish miners reveal blocks from their private chain to the public. This strategy leads honest miners that follow the Bitcoin protocol to waste resources on mining cryptopuzzles that end up serving no purpose. Our analysis demonstrates that, while both honest and selfish parties waste some resources, **the honest miners waste proportionally more**, and the selfish pool's rewards exceed its share of the network's mining power ...."[6] The "longest chain" rule of Chapter 8 becomes a weapon here. The rule only says "follow the longest one"; it cannot enforce "publish as soon as you find." Only the one who found a block knows when it will be published.

[Intermediate] The strategy is set by the pool's **lead** (how many blocks the private branch is ahead of the public branch). The model is this. The pool's ratio of the mining power is α, the others have $1-\alpha$, and the intervals between block discoveries are exponentially distributed.[7] In state $s = 0, 1, 2, \ldots$, if the pool finds a block (frequency α) the lead grows by one, and in $s \geq 3$, if the others find one (frequency $1-\alpha$), it shrinks by one. If the others find a block when the lead is 2, the pool publishes its entire private branch, goes back to lead 0, and gets both blocks. If the others find a block when the lead is 1, the system enters **state 0′**, in which two public branches of length 1 compete, and from there all three paths lead to state 0: "(1) the pool mines a block on its previously private branch (frequency α), (2) the others mine a block on the previously private branch (frequency $\gamma(1-\alpha)$), and (3) the others mine a block on the public branch (frequency $(1-\gamma)(1-\alpha)$)." Here γ is "the ratio of honest miners that choose to mine on the pool's block."[8] γ is not a strategic variable but **a network variable**. Honest miners extend "the branch they heard first," and how quickly the pool can spread its own block over the network sets γ.

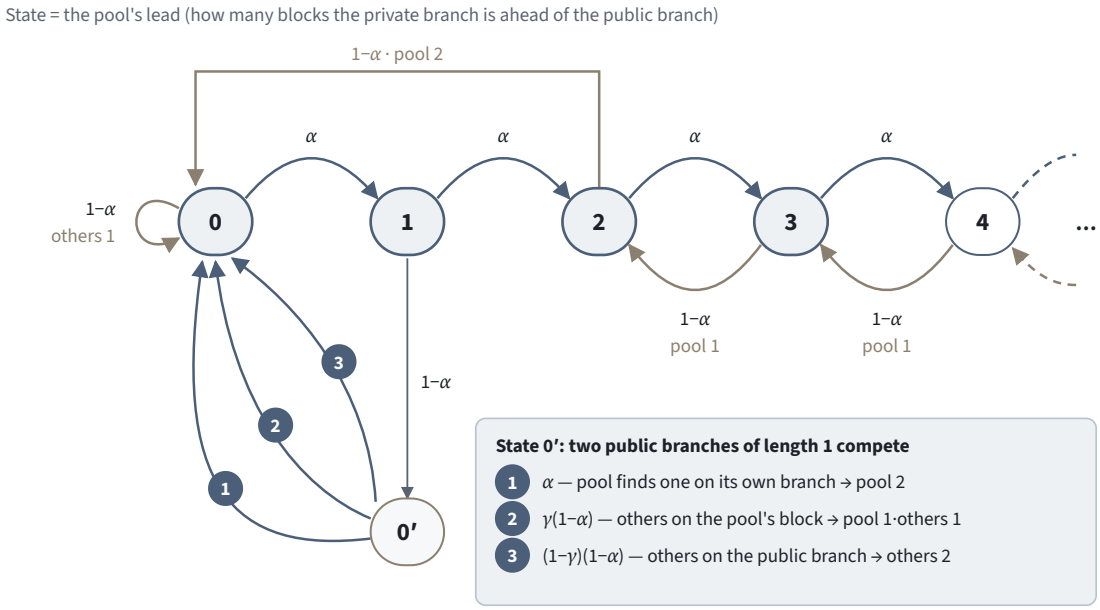


Figure 23-1. The state machine of selfish mining. A state is the pool's private lead; next to each arrow are its frequency and the number of blocks the pool and the others get from that transition. Redrawn from Figure 1 of Eyal & Sirer (2018).

[Intermediate] Rewards attach only to blocks that **remain in the final chain**. The authors list, as (a) to (h), who gets how many blocks in each transition. In short: in state 0′, if the pool finds a block, the pool gets 2 (both of its blocks are settled); if the others find one on the pool's block, the pool gets 1 and the others 1; if the others find one on their own block, the others get 2. If the others find a block at lead 2, the pool gets 2. If the others find a block when the lead is more than 2, "The new block (say with number i) will end outside the chain once the pool publishes its entire branch, therefore the others obtain nothing. However, the pool now reveals its i'th block, and obtains a revenue of one for its i'th block."[9] And one important passage: "As expected, the intentional branching brought on by selfish mining leads the honest miners to work on blocks that end up outside the blockchain. This, in turn, leads to a drop in the total block generation rate with $r_{pool} + r_{others} < 1$. The protocol will adapt the mining difficulty such that the mining rate at the main chain becomes one block per 10min on average. Therefore, the actual revenue rate of each agent is **the revenue rate ratio**; that is, the ratio of its blocks out of the blocks in the main chain."[10] The pool's relative revenue $R_{pool} = r_{pool} \,/\, (r_{pool} + r_{others})$ is the reference quantity of this chapter. Under honest mining this value is α. The cell below runs the state machine as it stands, measures R_{pool}, and compares it with the closed form.

```
# NB23-1: selfish-mining Monte Carlo — run the Eyal-Sirer state machine (lead 0, 0', 1, 2, …)
event by event and measure the pool's relative revenue R_pool (for study)
import random

def selfish_mining_share(alpha: float, gamma: float, n_events: int, seed: int = 23) -> float:
    """Draw n_events block-discovery events (the pool finds the block with probability α).
Returns: the pool's fraction of the blocks left in the main chain."""
    rng = random.Random(seed)
    lead = 0                # the pool's private lead (in blocks)
    tie = False             # state 0': two public branches of length 1 are competing
    pool_blocks = others_blocks = 0
    for _ in range(n_events):
        pool_found = rng.random() < alpha
        if tie:                                  # state 0' — all three paths lead to state 0
            if pool_found:                       # (b) the pool finds a block on its own branch
→ pool 2
                pool_blocks += 2
            elif rng.random() < gamma:           # (c) the others find a block on the pool's
branch → pool 1, others 1
                pool_blocks += 1; others_blocks += 1
            else:                                # (d) the others find a block on their own
branch → others 2
                others_blocks += 2
            tie = False; lead = 0
            continue
        if pool_found:                           # (a) the pool finds a block → lead +1 (reward
comes later)
            lead += 1
            continue
        # the others find a block
        if lead == 0:                            # (e) no private branch → others 1
            others_blocks += 1
        elif lead == 1:                          # (f) lead 1 → the pool publishes its hidden
block, a race begins
            tie = True; lead = 0
        elif lead == 2:                          # (g) lead 2 → the pool publishes two blocks,
the others' block is discarded → pool 2
            pool_blocks += 2; lead = 0
        else:                                    # (h) lead > 2 → the pool publishes one block
(pool 1), the others' block is eventually discarded
            pool_blocks += 1; lead -= 1
    return pool_blocks / (pool_blocks + others_blocks)

def R_pool_closed_form(a: float, g: float) -> float:
    """CACM version, equation (3) = arXiv v5, equation (8): R_pool = [α(1-α)²(4α+γ(1-2α)) - α³]
/ [1 - α(1+(2-α)α)], 0 ≤ α ≤ 1/2"""
    return (a * (1 - a) ** 2 * (4 * a + g * (1 - 2 * a)) - a ** 3) / (1 - a * (1 + (2 - a) * a))

N = 400_000
print(f"{N:,} events. Under honest mining the relative revenue is α itself. Below: the relative
revenue under selfish mining (simulation / closed form).")
print(f"{'α':>5} {'γ':>5} {'sim R_pool':>12} {'formula R_pool':>14} {'honest α':>8} {'selfish
better?':>15}")
for g in [0.0, 0.5, 1.0]:
    for a in [0.10, 0.20, 0.25, 0.30, 0.34, 0.40, 0.45]:
        sim = selfish_mining_share(a, g, N)
        cf = R_pool_closed_form(a, g)
        print(f"{a:>5.2f} {g:>5.1f} {sim:>12.4f} {cf:>14.4f} {a:>8.2f} {'yes' if cf > a else
'no':>15}")
```

```
# Output — bcbook (Python 3.13), exactly as run
400,000 events. Under honest mining the relative revenue is α itself. Below: the relative
revenue under selfish mining (simulation / closed form).
    α     γ   sim R_pool formula R_pool honest α selfish better?
 0.10   0.0       0.0358         0.0356     0.10              no
 0.20   0.0       0.1300         0.1297     0.20              no
 0.25   0.0       0.1948         0.1951     0.25              no
 0.30   0.0       0.2713         0.2731     0.30              no
 0.34   0.0       0.3446         0.3463     0.34             yes
 0.40   0.0       0.4813         0.4837     0.40             yes
 0.45   0.0       0.6488         0.6518     0.45             yes
 0.10   0.5       0.0724         0.0724     0.10              no
 0.20   0.5       0.1830         0.1824     0.20              no
 0.25   0.5       0.2501         0.2500     0.25              no
 0.30   0.5       0.3254         0.3269     0.30             yes
 0.34   0.5       0.3953         0.3970     0.34             yes
 0.40   0.5       0.5235         0.5256     0.40             yes
 0.45   0.5       0.6778         0.6806     0.45             yes
 0.10   1.0       0.1097         0.1092     0.10             yes
 0.20   1.0       0.2363         0.2352     0.20             yes
 0.25   1.0       0.3048         0.3049     0.25             yes
 0.30   1.0       0.3792         0.3806     0.30             yes
 0.34   1.0       0.4456         0.4476     0.34             yes
 0.40   1.0       0.5652         0.5674     0.40             yes
 0.45   1.0       0.7065         0.7094     0.45             yes
```

[Intermediate] Now read the table. With $\gamma = 0$ (in a race, the honest miners always hear their own side's block first), selfish mining does worse than honest mining at $\alpha = 0.30$ and better from 0.34. With $\gamma = 0.5$ the boundary is near 0.25, and with $\gamma = 1$ **even the smallest pool** comes out ahead. The simulation and the closed form agree within the Monte Carlo error (about ±0.005 at 400,000 events). That closed form and the boundary are the subject of the next section.

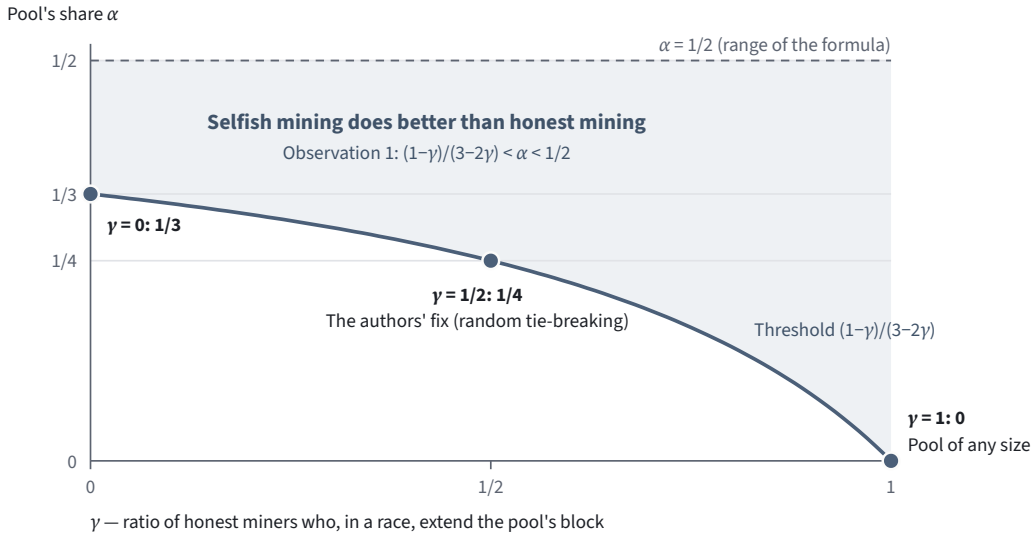


Figure 23-2. The range of shares in which selfish mining does better than honest mining (Observation 1). The curve is the graph of the formula in the text, $\frac{1-\gamma}{3-2\gamma}$, and the three points are the 1/3, 1/4 and 0 of the text.

▾ [Advanced] The closed form, the threshold share, and why 1/3, 1/4 and 0

[Advanced] Solve for the stationary probabilities of the state machine (the CACM

version says "the details are in our full report," and arXiv v5 sets them out as equation (1), the system of equations, and (2)–(5), the solution), and the probabilities of states 0, $0'$, 1 and $k \geq 2$ come out as rational functions of α. Multiply them by the reward for each transition and add them up, and the result is the pool's relative revenue. It is equation (3) of the CACM version and equation (8) of arXiv v5.

$$R_{pool} = \frac{\alpha(1-\alpha)^2(4\alpha + \gamma(1-2\alpha)) - \alpha^3}{1 - \alpha(1 + (2-\alpha)\alpha)}, \qquad 0 \leq \alpha \leq \frac{1}{2}$$

Reading the symbols: α is the pool's ratio of the mining power, and γ is the ratio of honest miners who, in a race, extend the pool's block. A numerical example: at $\alpha = 0.4$ and $\gamma = 0$, $R_{pool} \approx 0.484$, larger than the share of 0.4. Solving for the range in which this formula exceeds α gives **Observation 1**: "For a given γ, a pool of size α obtains a revenue larger than its relative size for α in the range $\frac{1-\gamma}{3-2\gamma} < \alpha < \frac{1}{2}$."[11] At $\gamma = 0$ the lower bound is 1 / 3, at $\gamma = 1 / 2$ it is 1 / 4, and at $\gamma = 1$ it is 0. The authors' explanation says why: "Note that the pool is only at risk when it holds exactly one block secret, and the honest miners might publish a block that would compete with it. For $\gamma = 1$, the pool can quickly propagate its one-block branch if the others find their own branch, so all honest miners would still mine on the pool's block. In this case, the pool takes no risk when following the Selfish-Mine strategy and its revenue is always better than when following the honest algorithm. The threshold is therefore zero, and a pool of any size can benefit by following Selfish-Mine. In the other extreme, $\gamma = 0$, the honest miners always publish and propagate their block first, and the threshold is at 1/3."[12] Then **Observation 2**: "For a pool running the Selfish-Mine strategy, the revenue of each pool member increases with pool size for pools larger than the threshold." This is because revenue rises against share with a slope steeper than 1, and so "rational miners will preferentially join the selfish pool to increase their revenues. ... The selfish pool would therefore increase in size, unopposed by any mechanism, towards a majority."[13] The moment the conventional wisdom's "X% gets X%" breaks, the force that caps the size of a pool disappears.

[Advanced] That the pool **can raise** γ is what makes the attack realistic: "a savvy pool operator can perform a sybil attack on honest miners by adding a significant number of zero-power miners to the Bitcoin miner network. These virtual miners act as advance sensors by participating in data dissemination, but do not mine new blocks. ... once they hear of block X, they ignore it and start propagating block P. ... By adding enough virtual nodes, the pool operator can thus increase γ. The result (see Observation 1), is a threshold close to zero." The authors' fix therefore changes the honest side's **tie-breaking rule**: "when a miner learns of competing branches of the same length, it should propagate all of them, and choose which one to mine on uniformly at random. ... This yields $\gamma = 1 / 2$, which in turn yields a threshold of 1/4." Because it works "independent of the adoption of the change at other miners," "it does not require a hard fork" (backward compatibility and gradual adoption are in other sentences on the same page), and it "does not introduce new vulnerabilities to the protocol," since "Currently, when there are two branches of equal length, the choice of each miner is arbitrary, effectively determined by the network topology and

latency."[14] Even with that fix, though, the authors write that "the system remains vulnerable: there already exist pools whose mining power exceeds the 25% threshold, and at times, even the 33% theoretical hard limit."[15] And the conclusion changes the number in the conventional wisdom: "at least 2/3 of the network needs to be honest to thwart selfish mining; a simple majority is not enough."[16]

Two properties of this attack should be recorded. First, **it is hard to detect**: "It is easy to hide Selfish-Mine behavior, and difficult to ban it. A selfish pool may never reveal its size by using different Bitcoin addresses and IP addresses, and by faking block creation times." There are two signs, an increase in abandoned blocks and a distortion in the distribution of block intervals, but "neither are easy to measure definitively."[17] Second, "collapse" does not mean a value of 0. What the authors call a "collapse" is "a state in which a single entity controls the entire currency system," and "such a collapse does not immediately imply that the value of a Bitcoin drops to 0. The controlling entity will have an incentive to accept most transactions, if only to reap their fees, ... It may also choose to remain covert, and hide the fact that it can control the entire currency."[18] The loss of decentralization may not be visible from outside. The proposition of Part 5, "If it is decentralized, no one is responsible" (misconception ⑫ in Appendix B), is shaken once more here. Having **no** one responsible is different from the one responsible being **invisible**.

5. Other places in the same frame: the coordination game, the cooperative equilibrium, undercutting

Selfish mining came out of one freedom: "the miner can choose when to publish a block." That is not the miner's only freedom. The strategy language of Carlsten et al., seen in Chapter 20, names three: "Which block to extend. How much of the available transactions (and associated fees) to include in the block they are solving. For each unpublished block, whether or not to publish."[19] Each of the three freedoms carries a question of equilibrium.

[Intermediate] Which block to extend: the coordination game. When two branches of the same length compete, what matters to a miner is not "which one is right" but "which one will the others extend." The side that the majority extends wins, and blocks built on the losing side are discarded. This is the textbook coordination game, a game with two equilibria (everyone on A / everyone on B). That is why the protocol supplies a **focal point**, "the one heard first," and Eyal and Sirer's γ and their fix (random tie-breaking) were meant to keep an attacker from manipulating that focal point. A hard fork is this coordination game played for high stakes. When the chain with changed rules and the old chain split, what decides which side is "Bitcoin" or "Ethereum" is not the code but **where the participants coordinate**. As the SEC's report on The DAO, seen in Chapter 14, notes, after the hard fork of July 20, 2016, "A minority group, however, elected not to adopt the new Ethereum Blockchain created by the Hard Fork because to do so would run counter to the concept that a blockchain is immutable. Instead they continued to use the former version of the blockchain, which is now known as 'Ethereum Classic.'"[20] Both chains survived, and which one is "the real one" is the result of coordination, not of proof. Chapter 26 takes up this event.

[Intermediate] How much to include: the undercutting equilibrium. We read this in Section 6 of Chapter 20. When the reward is fees alone, the strategy of "extending the block that leaves the most fees unclaimed" is strictly better, on top of that, undercutting, "deliberately forking and taking less of the fees," becomes an equilibrium, and in that equilibrium unprocessed transactions pile up without end (Theorem 5.1). Translated into the language of this chapter, the conclusion of Carlsten et al. is that **a strategy that was an equilibrium when the reward was constant stops being one when the reward varies**. In the block reward model, "With non-atomic miners, DefaultCompliant is trivially an equilibrium, and this result is robust to general models of latency ... Now, in the transaction-fee model, even when miners are non-atomic, equilibrium behavior is complex and undesirable ...."[21]

[Intermediate] Bots competing over order: the cooperative equilibrium. This is what Chapter 17 handed on. Daian and his coauthors observed bots in priority gas auctions converging, in a race to outbid one another, to a cooperative state in which they **take turns raising by the minimum increment**, and they modeled it with a grim-trigger strategy of a repeated game: "Consider two players who coordinate with the following strategies: at set intervals, players alternate their bids, each time bidding the minimum increase. If any player deviates either by bidding too soon or by raising the bid higher than they are supposed to, the other player will respond by raising their bid such that the game is no longer profitable." Observation 3 says that "In a grim-trigger cooperative PGA, ... the optimal choice for bids is $W[0] = s$ and $W[i] = W[i-1] \times (1+\iota)$," that is, the minimum increment each time, and Theorem 1 says that "For parameters consistent with Ethereum PGAs, there exist a grim-trigger Nash equilibria [sic] for a 2 player PGA where both players follow a D, W cooperative strategy." The authors are careful: "We stress that we don't claim that perfect cooperation is occurring or even that our exact equilibrium is on play. We do claim, however, that players are converging to a more cooperative state, that is approximated by our model, in which they allow opportunities for other players in an effort to maximize their own expected profit over time."[22] The convergence to 12.5% (the minimum increment of the Parity client) recorded in a note to Chapter 17 was the evidence. That an auction designed for "competition" can drift into **tacit collusion** when the participants are few is a long-standing caution of auction design, and a convergence resembling it was observed on chain, even if, as the authors caution, it is not "our exact equilibrium."

Lay the three places over one another and the proposition of this chapter comes into view. The white paper's "honesty pays" is an equilibrium for the case in which participants are small, the reward is constant, publication is immediate, and there is nothing to extract from ordering. Each time one of the four conditions was relaxed (large pools, fee rewards, blocks that can be withheld, MEV), a different equilibrium appeared. The rules are still the same. What changed was **whether following the rules pays**.

```
# NB23-2: the 2×2 payoff table and the "threshold" — fill a publish/withhold game between two
small players with the relative revenue of selfish mining and read off the equilibrium (for
study)
# Two pools P (share a) and Q (share b) each choose "honest (publish at once)" or "withhold
(selfish mining)". The rest (1-a-b) are honest.
# Simplification: if only one pool withholds, its revenue is R_pool(α, γ); if both withhold,
neither knows the other's hidden blocks, so each uses R_pool and the effect of the other also
withholding is ignored (limitation stated).
def payoff(a, b, P_selfish, Q_selfish, gamma):
    rp = R_pool_closed_form(a, gamma) if P_selfish else a
    rq = R_pool_closed_form(b, gamma) if Q_selfish else b
    return rp, rq

for gamma in [0.0, 0.5]:
    print(f"\nγ = {gamma}: threshold (1-γ)/(3-2γ) = {(1-gamma)/(3-2*gamma):.3f}")
    for a, b in [(0.20, 0.20), (0.30, 0.30), (0.40, 0.20)]:
        print(f"  P={a:.2f}, Q={b:.2f}  (rows: P's choice; columns: Q's choice; cells: P's
revenue, Q's revenue)")
        print(f"  {'':>10}{'Q honest':>18}{'Q withhold':>18}")
        for ps, pl in [(False, "P honest"), (True, "P withhold")]:
            cells = []
            for qs in [False, True]:
                rp, rq = payoff(a, b, ps, qs, gamma)
                cells.append(f"({rp:.3f}, {rq:.3f})")
            print(f"  {pl:>10}{cells[0]:>18}{cells[1]:>18}")
        best_P = "withhold" if R_pool_closed_form(a, gamma) > a else "honest"
        best_Q = "withhold" if R_pool_closed_form(b, gamma) > b else "honest"
        print(f"  → P's dominant strategy {best_P}, Q's dominant strategy {best_Q}  (below the
threshold share honest is dominant, above it withhold is)")
print("\nThis table rests on the simplification 'whatever the other does, my revenue is set by
the threshold'. The interaction when both pools withhold at the same time is outside the
model.")
```

```
# Output — bcbook (Python 3.13), exactly as run

γ = 0.0: threshold (1-γ)/(3-2γ) = 0.333
  P=0.20, Q=0.20  (rows: P's choice; columns: Q's choice; cells: P's revenue, Q's revenue)
                       Q honest        Q withhold
    P honest    (0.200, 0.200)    (0.200, 0.130)
  P withhold    (0.130, 0.200)    (0.130, 0.130)
  → P's dominant strategy honest, Q's dominant strategy honest  (below the threshold share
honest is dominant, above it withhold is)
  P=0.30, Q=0.30  (rows: P's choice; columns: Q's choice; cells: P's revenue, Q's revenue)
                       Q honest        Q withhold
    P honest    (0.300, 0.300)    (0.300, 0.273)
  P withhold    (0.273, 0.300)    (0.273, 0.273)
  → P's dominant strategy honest, Q's dominant strategy honest  (below the threshold share
honest is dominant, above it withhold is)
  P=0.40, Q=0.20  (rows: P's choice; columns: Q's choice; cells: P's revenue, Q's revenue)
                       Q honest        Q withhold
    P honest    (0.400, 0.200)    (0.400, 0.130)
  P withhold    (0.484, 0.200)    (0.484, 0.130)
  → P's dominant strategy withhold, Q's dominant strategy honest  (below the threshold share
honest is dominant, above it withhold is)

γ = 0.5: threshold (1-γ)/(3-2γ) = 0.250
  P=0.20, Q=0.20  (rows: P's choice; columns: Q's choice; cells: P's revenue, Q's revenue)
                       Q honest        Q withhold
    P honest    (0.200, 0.200)    (0.200, 0.182)
  P withhold    (0.182, 0.200)    (0.182, 0.182)
  → P's dominant strategy honest, Q's dominant strategy honest  (below the threshold share
honest is dominant, above it withhold is)
  P=0.30, Q=0.30  (rows: P's choice; columns: Q's choice; cells: P's revenue, Q's revenue)
                       Q honest        Q withhold
    P honest    (0.300, 0.300)    (0.300, 0.327)
  P withhold    (0.327, 0.300)    (0.327, 0.327)
  → P's dominant strategy withhold, Q's dominant strategy withhold  (below the threshold share
honest is dominant, above it withhold is)
  P=0.40, Q=0.20  (rows: P's choice; columns: Q's choice; cells: P's revenue, Q's revenue)
                       Q honest        Q withhold
    P honest    (0.400, 0.200)    (0.400, 0.182)
  P withhold    (0.526, 0.200)    (0.526, 0.182)
  → P's dominant strategy withhold, Q's dominant strategy honest  (below the threshold share
honest is dominant, above it withhold is)

This table rests on the simplification 'whatever the other does, my revenue is set by the
threshold'. The interaction when both pools withhold at the same time is outside the model.
```

[Intermediate] The second cell puts this chapter's numbers into the textbook payoff table. For a pool whose share is below the threshold, honest is dominant; for one above it, withholding is. This follows because the simplification **made each pool's revenue independent** of the other's choice; it is not a finding. So within this table, in a world with only small pools "everyone honest" is the equilibrium, and as soon as even one pool passes the threshold, "withhold" is the equilibrium strategy for that pool. As the cell's comments state, the case in which two pools withhold at the same time is outside this simplification. Its analysis is beyond the scope of this book.

> **? A LAWYER'S QUESTION**
>
> After a hard fork both chains survived. They share the same history, the rules diverged from a certain block on, and each side says it is "the" ledger. This chapter wrote that what decides which side wins is the coordination of the participants. Then who decides which ledger is "real": a majority of the hash rate, a majority of node operators, the exchange that attaches the name to one side, the developers, or no one at all? A person who held coins before the fork holds the same balance on both chains after it. Does that person have one thing or two? If a contract said "pay 1 bitcoin," the 1 on which chain? What has a person who deviated within the rules, like a selfish miner, violated: the rules, an expectation, or nothing? This book does not answer, and points to two sections of Part 5. → Chapter 27 · Chapter 28

6. Summary

The white paper's "more profitable to play by the rules" is one claim of equilibrium, and the equilibrium holds when the reward is large compared with the value of an attack (Chapter 20), when participants are small, when the reward is constant, when blocks are published at once, and when there is nothing to extract from ordering. Eyal and Sirer argued that the strategy of hiding blocks gives more than honesty from a share of 1/3 (0 if the pool controls propagation), and that this revenue grows with pool size and pushes the pool toward a majority. So it is not "safe if a majority is honest" but "more than 2/3 must be honest," and even that under optimistic assumptions. If only fees remain, undercutting becomes an equilibrium (Chapter 20); the bots of the ordering auction converge to tacit cooperation (Chapter 17); and the choice of fork is a coordination game, so the "real" chain is decided by coordination, not by proof. The rules stay the same while the equilibrium changes: that is what this chapter meant by "conditional." Chapter 24 looks at the same "multiple equilibria in which belief decides the outcome" in another place, the bank run on a token that carries a promise of redemption.

Sources for this chapter

- Ittay Eyal & Emin Gün Sirer, "Majority Is Not Enough: Bitcoin Mining Is Vulnerable," *Communications of the ACM* 61(7): 95–102 (2018), DOI 10.1145/3212998 — the conventional wisdom and the claim (p. 95), the model (§3.1, p. 97), the strategy, the state machine, the rewards for each transition and equation (3) (§3.2–4.1, pp. 97–99), Observations 1 and 2 and γ (§4.3, pp. 99–100), pool formation (§5, p. 100), the sybil attack and the fix (§6, p. 100), the related work's statements of the conventional wisdom (§7, p. 101), collapse, detection and concealment (§8, pp. 101–102), conclusion (§9, p. 102). arXiv:1311.0243v5 — state probabilities, equations (1)–(5) (p. 8), revenue, equations (6)–(8) (p. 10), Observation 1, equation (9) (p. 11). The FC 2014 conference version (LNCS 8437, pp. 436–454) was not obtained, and its pages are not cited.
- Satoshi Nakamoto (2008), §6, Incentive (p. 4).
- Eric Budish, "Trust at Scale," *QJE* 140(1): 1–62 (2025) — Lemmas 1 and 2 and Proposition 2 of the one-shot game (§III.E, pp. 28–30), the remark on the literature on small players (p. 30).
- Miles Carlsten et al., "On the Instability of Bitcoin Without the Block Reward," *CCS '16* — the three decisions (§3.3), the comparison of equilibria (§5.4). Extended version, section numbers only.
- Philip Daian et al., "Flash Boys 2.0," *2020 IEEE S&P*, pp. 910–927 — the cooperative strategy, Observation 3 and Theorem 1 (§VI-D, pp. 919–920).
- SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017) — Ethereum Classic (p. 9, n. 32).
- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library module `random`. Notebook NB23. NB23-1 runs the Eyal–Sirer state machine event by event and checks it against equation (3) of the CACM version, and NB23-2 is a simplification for study that fills a 2×2 payoff table with that formula (the interaction when two pools withhold at the same time is outside the model).

Notes

1. Nakamoto (2008), p. 4 (§6). The same passage as note 1 in Chapter 20.
2. Eric Budish, "Trust at Scale," *QJE* 140(1): 1–62 (2025), pp. 28–30 (§III.E), Lemmas 1 and 2, Proposition 2. Original: "If there is an honest equilibrium, then $N^*c \leq p$ (analog of equation (1))."; "If there is an honest equilibrium with $x_i^* = 0$ for some player i, then $N^*c \geq V_{attack}$ (analog of equation (2))."; "A necessary condition for an honest equilibrium is $p \geq \frac{V_{attack}}{1+1/I}$. In the limit as I → ∞ this is $p \geq V_{attack}$ (analog of equation (3))."
3. Budish (2025), p. 30. Original: "Note that this simple model purposefully restricts attention to honest play or majority attack. Many prior game-theoretic analyses of blockchains assume that all players are small, eliminating the possibility of majority attack, but allow for a richer set of strategies for such small players than just honestly following the protocol (e.g., Eyal and Sirer 2014; Carlsten et al. 2016; Biais et al. 2019; Saleh 2021)."
4. Ittay Eyal & Emin Gün Sirer, "Majority Is Not Enough: Bitcoin Mining Is Vulnerable," *Communications of the ACM* 61(7): 95–102 (2018), p. 95 (abstract). Emphasis added. (published version). The original paper is FC 2014 (LNCS 8437, pp. 436–454), and the preprint is arXiv:1311.0243 (v5, 2013-11-15). This book uses the page and

equation numbers of the CACM version and gives the arXiv v5 equation numbers in the notes.

5. Eyal & Sirer (2018), p. 95 and p. 101 (§7). Original: "Felten notes that 'there was a folk theorem that the Bitcoin system was stable, in the sense that if everyone acted according to their incentives, the inevitable result would be that everyone followed the rules of Bitcoin as written.' Others have claimed that 'the well-known argument – never proven, but taken on intuitive faith – that a minority of miners can't control the network is a special case of a more general assumption: that a coalition of miners with X% of the network's hash power can make no more than X% of total mining revenues.'"; "Kroll, Davey and Felten's work provides a game-theoretic analysis of Bitcoin, without taking into account block withholding attacks such as selfish mining, and argues that the honest strategy constitutes a Nash equilibrium, implying incentive-compatibility."

6. Eyal & Sirer (2018), p. 95 (§1). Emphasis added.

7. Eyal & Sirer (2018), p. 97 (§3.1). Original: "The system is comprised of a set of miners 1, . . ., n. Each miner i has mining power m_i ... Each miner chooses a chain head to mine, and finds a subsequent block for that head after a time interval that is exponentially distributed ... We assume that miners are rational; that is, they try to maximize their revenue, and may deviate from the protocol to do so."; "The expected relative revenue, or simply the revenue of a pool is the expected fraction of blocks that were mined by that pool out of the total number of blocks in the longest chain."

8. Eyal & Sirer (2018), p. 98 (§4, the state machine in Figure 1). Original: "For state s = 0, 1, 2, . . ., with frequency α, the pool mines a block and the lead increases by one to s + 1. In states s = 3, 4, . . ., with frequency (1 − α), the honest miners mine a block and the lead decreases by one to s − 1. If the others mine a block when the lead is two, the pool publishes its private branch, and the system drops to a lead of 0. If the others mine a block with the lead is 1, we arrive at the aforementioned state 0'. From 0', there are three possible transitions, all leading to state 0 with total frequency 1: (1) the pool mines a block on its previously private branch (frequency α), (2) the others mine a block on the previously private branch (frequency γ(1 − α)), and (3) the others mine a block on the public branch (frequency (1 − γ)(1 − α))."; definition of γ: "We denote by γ the ratio of honest miners that choose to mine on the pool's block, and the other (1−γ) of the non-pool miners mine on the other branch."

9. Eyal & Sirer (2018), pp. 98–99 (§4.1 (a)–(h)). (h) in the original: "Lead was more than 2, others find a block. The others decrease the lead, which remains at least two. The new block (say with number i) will end outside the chain once the pool publishes its entire branch, therefore the others obtain nothing. However, the pool now reveals its i'th block, and obtains a revenue of one for its i'th block."

10. Eyal & Sirer (2018), p. 99. Emphasis added.

11. Eyal & Sirer (2018), p. 99 (§4.3, "The effect of α and γ"), Observation 1 (in the published version the formula for the range itself is typeset as an image and is not in the text layer; arXiv v5, §4.4, equation (9) gives it as $\frac{1-\gamma}{3-2\gamma} < \alpha < \frac{1}{2}$). CACM original: "When the pool's revenue given in Equation 3 is larger than α, the pool will earn more than its relative size by using the Selfish-Mine strategy. ... Observation 1. For a given γ, a pool of size α obtains a revenue larger than its relative size for α in the range [formula]." The closed form of R_{pool} is equation (3) of the CACM version (p. 99) and equation (8) of arXiv v5 (p. 10): "$R_{pool} = \frac{r_{pool}}{r_{pool} + r_{others}} = \cdots = \frac{\alpha(1-\alpha)^2(4\alpha + \gamma(1-2\alpha)) - \alpha^3}{1 - \alpha(1 + (2-\alpha)\alpha)}$." The state probabilities in arXiv v5 are equations (2)–(5) (p. 8). The "Theorem 1" noted during checking is the result that the text of §5 of the published version points to as "(Theorem 1)"; in both versions it is the content of the same section as Observations 1 and 2.

12. Eyal & Sirer (2018), pp. 99–100. Original: "Note that the pool is only at risk when it holds exactly one block secret, and the honest miners might publish a block that would compete with it. For γ = 1, the pool can quickly propagate its one-block branch if the others find their own branch, so all honest miners would still mine on the pool's block. In this case, the pool takes no risk when following the Selfish-Mine strategy and its revenue is always better than when following the honest algorithm. The threshold is therefore zero, and a pool of any size can benefit by following Selfish-Mine. In the other extreme, γ = 0, the honest miners always publish and propagate their block first, and the threshold is at 1/3. With γ = 1/2 the threshold is at 1/4. Figure 3 shows the threshold as a function of γ."

13. Eyal & Sirer (2018), p. 100 (§4.3 and §5). Original: "Observation 2. For a pool running the Selfish-Mine strategy, the revenue of each pool member increases with pool size for pools larger than the threshold."; "At this point, rational miners will preferentially join the selfish pool to increase their revenues. Moreover, the pool's members will want to accept new members, as this would increase their own revenue (Observation 2). The

selfish pool would therefore increase in size, unopposed by any mechanism, towards a majority."

14. Eyal & Sirer (2018), p. 100 (§6.1–6.2). Original: "But a savvy pool operator can perform a sybil attack on honest miners by adding a significant number of zero-power miners to the Bitcoin miner network. These virtual miners act as advance sensors by participating in data dissemination, but do not mine new blocks. … once they hear of block X, they ignore it and start propagating block P. … By adding enough virtual nodes, the pool operator can thus increase γ. The result (see Observation 1), is a threshold close to zero."; "Specifically, when a miner learns of competing branches of the same length, it should propagate all of them, and choose which one to mine on uniformly at random. … This yields γ = 1/2, which in turn yields a threshold of 1/4."; "This improvement is independent of the adoption of the change at other miners, therefore it does not require a hard fork. This change to the protocol does not introduce new vulnerabilities to the protocol: Currently, when there are two branches of equal length, the choice of each miner is arbitrary, effectively determined by the network topology and latency."

15. Eyal & Sirer (2018), p. 102 (§8.4). Original: "Even with our proposed fix that raises the threshold to 25%, the system remains vulnerable: there already exist pools whose mining power exceeds the 25% threshold, and at times, even the 33% theoretical hard limit. Responsible miners should therefore break off from large pools until no pool exceeds the threshold size." The "there already exist" of this sentence refers to the pool statistics of October 2013 (reference 18).

16. Eyal & Sirer (2018), p. 102 (§9) and p. 101 (§7). Original: "We also show that at least 2/3 of the network needs to be honest to thwart selfish mining; a simple majority is not enough."; "Over 2/3 of the participants need to be honest to protect against selfish mining, under the most optimistic of assumptions."

17. Eyal & Sirer (2018), pp. 101–102 (§8.2–8.3). Original of the first passage: "There are two telltale network signatures of selfish mining that can be used to detect when selfish mining is taking place, but neither are easy to measure definitively."

18. Eyal & Sirer (2018), p. 101 (§8.1). Original: "We therefore refer to a state in which a single entity controls the entire currency system as a collapse of Bitcoin. Note that such a collapse does not immediately imply that the value of a Bitcoin drops to 0. The controlling entity will have an incentive to accept most transactions, if only to reap their fees, and because if it mines all Bitcoins, it has strong motivation that they maintain their value. It may also choose to remain covert, and hide the fact that it can control the entire currency."

19. Carlsten et al. (2016), §3.3. Original: "At each instant, every miner makes several distinct decisions: Which block to extend. How much of the available transactions (and associated fees) to include in the block they are solving. For each unpublished block, whether or not to publish." (extended version; cited by section number only).

20. SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017-07-25), p. 9, n. 32. The block at which the fork activated (1,920,000) and the statement that "the fork did not change the protocol" are from EIP-779 (Chapter 26).

21. Carlsten et al. (2016), §5.4. Original: "To understand the impact of Theorem 5.1, first consider the block reward model. With non-atomic miners, DefaultCompliant is trivially an equilibrium, and this result is robust to general models of latency (proof in Appendix D). But as we move to atomic miners, strategies like selfish mining arise and equilibria get messy (if they exist at all). Now, in the transaction-fee model, even when miners are non-atomic, equilibrium behavior is complex and undesirable, as we have just shown."

22. Philip Daian et al., "Flash Boys 2.0," *2020 IEEE S&P*, pp. 910–927, §VI-D (pp. 919–920). Original: "Observation 3. In a grim-trigger cooperative PGA, for all $i \leq imax$, the optimal choice for bids is $W[0] = s$ and $W[i] = W[i-1] \times (1 + \iota)$."; "Theorem 1. For parameters consistent with Ethereum PGAs, there exist a grim-trigger Nash equilibria for a 2 player PGA where both players follow a D, W cooperative strategy." The formal treatment is in Appendix E of the paper.

PART 3 · ECONOMICS

Chapter 24. Stablecoins: design types and the structure of a bank run

KEY TAKEAWAY

The coin of Chapter 19 was an outside money that promises nothing. A stablecoin is the opposite: a token that carries **the promise "1 token will be paid back as 1 dollar,"** and at that moment the problem moves from monetary economics to **the economics of banking.** In Gorton and Zhang's words, the technology is new but "the economics is centuries old," and issuers are "unregulated banks." Depending on what assets back the promise, stablecoins divide into fiat-backed, crypto-overcollateralized, algorithmic and synthetic types, but all four types sit on the same scale. When redemption demands bunch up, reserve assets have to be sold; if they cannot be sold, or cannot be sold at full value, the 1-dollar promise breaks (a depeg); and the belief that it might break itself brings on redemption demands. This is the structure of a bank run exactly as Diamond and Dybvig set it down in 1983: "A demand deposit contract which is not subject to runs provides no liquidity services." After this chapter you will know that the "stability" of a stablecoin is a property not of its price but **of its balance sheet and its redemption procedure,** and you will see how far the run of March 2023 resembled the textbook picture and where it differed (the secondary market). The chapter does not compare the safety of particular coins. It looks only at the structure of the types.

Learning objectives

1. Define a stablecoin as "a token that carries a promise of redemption," and draw the balance sheets of the fiat-backed, crypto-overcollateralized, algorithmic and synthetic types. Know how it differs from the outside money of Chapter 19.
2. **[Intermediate]** Define the collateral ratio and the redeemable ratio, and calculate what the liquidity composition of reserve assets decides when redemption demands bunch up. Know why the "no questions asked" (NQA) principle and information sensitivity are central.
3. **[Advanced]** Read in equations the two equilibria of the Diamond–Dybvig model (the good one and the run), why the choice between them can turn on "almost anything," and what suspension of convertibility and deposit insurance each change. See the reflexive feedback of the algorithmic type in a simple model.
4. Read the runs of May 2022 and March 2023 through the figures of empirical studies (sample, period, estimate), and tell apart which parts match the textbook and which

differ (the existence of a secondary market).

What a reader of the [Basic] paragraphs alone takes away. A stablecoin sits somewhere between a bank deposit and an MMF, and so **it is exposed to the same runs that banks are.**

1. The question this chapter tries to answer

Chapter 19, following Garratt and Wallace, placed bitcoin as an outside money: money of which it is **not** true that "each unit is someone's asset and someone else's liability."[1] A stablecoin sits on the opposite side of that definition. Because the issuer promises "bring me one of these tokens and I will give you 1 dollar," the token is an asset of the holder and a liability of the issuer. It is inside money. And the value of inside money rests not on belief but **on whether the issuer can keep its promise.** That question is one banking has dealt with for hundreds of years. It is why Gary Gorton and Jeffery Zhang titled their 2023 paper "Taming Wildcat Stablecoins": "While the technology underlying cryptocurrencies is new, the economics is centuries old. ... In this case, the new technology has recreated circulating private money in the form of stablecoins, which are similar to the banknotes that circulated in many countries during the nineteenth century. The implication is that stablecoin issuers are **unregulated banks**."[2]

This chapter asks four questions in turn. What backs the promise (the types)? What conditions must the balance sheet meet for the promise to be kept (the collateral ratio and liquidity)? Why can a run happen even when those conditions are met (Diamond–Dybvig)? And how closely did actual runs match that picture (the evidence)? The names of particular coins are used only to point to settled historical events, and the chapter does not ask which coin is safer. Its unit is **the type and the structure.**

2. Four types: what backs the promise

The Bank for International Settlements (BIS) *Annual Economic Report 2022* divides them in two: "The two main types of stablecoin are asset-backed and algorithmic. Asset-backed stablecoins ... are typically managed by a centralised intermediary who invests the underlying collateral and coordinates the coins' redemption and creation. Assets can be held in government bonds, short-term corporate debt or bank deposits, or in other cryptocurrencies. In contrast, algorithmic stablecoins ... rely on complex algorithms that automatically rebalance supply to maintain their value relative to the target currency or asset. To avoid reliance on fiat currency, they often do so by providing users with an arbitrage opportunity relative to another cryptocurrency."[3] Dividing the asset-backed type again by the kind of collateral, and adding a fourth type taken up by a 2026 Federal Reserve Bank of New York blog post, gives the table below. The balance sheet in each row is this book's account of the **structure** of a type, not the actual books of any coin.

TYPE	ASSETS (RESERVES)	LIABILITIES	HOW "1 DOLLAR" BREAKS
Fiat-backed	Cash and bank deposits, short-term government securities, repurchase agreements, (sometimes) commercial paper, etc.	Issued tokens = claims to redemption at par	Reserve assets cannot be sold in time at full value (frozen deposits, discounted commercial paper); doubts about the solvency of the issuer itself
Crypto-overcollateralized	Other crypto-assets (locked in excess of face value because they are volatile), or other stablecoins	Issued tokens	A sharp fall in collateral value uses up the excess; cascading liquidations; contagion from the depeg of another stablecoin held as collateral
Algorithmic	The issuer's **promise to exchange into its own second token** (no real reserves)	Issued tokens	If demand for the second token disappears, there is nothing to exchange for "1 dollar's worth": a reflexive spiral
Synthetic	Long staked crypto-assets + short perpetual futures (delta hedge)	Issued tokens	If the funding rate turns negative or margin costs rise, keeping the hedge in place runs at a loss: feedback between redemptions and liquidations

Box A of the BIS report sets out the mechanism of the third type through the events of May 2022: "An algorithmic stablecoin, it maintained value by adjusting supply in an automated arbitrage trading strategy with another cryptocurrency, Luna, on the Terra blockchain. UST aimed to keep a one-for-one peg to the US dollar by being convertible into one dollar's worth of Luna, and vice versa. For example, should Terra fall to 99 cents, a user could purchase UST on an exchange for 99 cents and then exchange their UST for $1 worth of new units of Luna on the Terra platform. A crucial aspect of this arrangement was that users would only be willing to exchange UST into Luna **if Luna's market capitalisation exceeded that of UST**."[4] The fourth type is explained in a June 2026 post from the New York Fed: "synthetic stablecoins ... target their dollar peg through financial engineering: they combine a long position in cryptocurrency (typically ether staked in a staking pool ...) with an offsetting short position in perpetual futures contracts. ... This structure creates a 'synthetic' dollar position without requiring actual dollar reserves." And: "The key fragility, however, is that

the hedge depends on continuous liquidity in derivatives markets and a consistently positive funding rate. If funding turns negative or margin costs rise, the synthetic dollar becomes expensive to maintain and redemptions begin."[5] The collapse of Terra itself is taken up in Chapter 26. This chapter takes only the mechanism.

Collateral ratio and redeemable ratio

[Intermediate] Whatever the type, the conditions under which the promise is kept can be written with the same two numbers. If S is the number of tokens issued and A the current value of the reserve assets, **the collateral ratio** is

$$\mathrm{CR} = \frac{A}{S}$$

and when redemption demands bunch up to a fraction q of issuance, **the redeemable ratio** is the disposal value of the assets that can be sold **right now** to meet those demands, divided by the amount demanded:

$$\mathrm{RR}(q) = \frac{\sum_i \ell_i A_i \,(1 - h_i)}{qS}$$

Reading the symbols: A_i is the value of asset class i, $\ell_i \in [0, 1]$ is the share of it that can be disposed of immediately (1 for cash, 0 for a frozen deposit), and h_i is the discount when it is sold in a hurry. A numerical example: with issuance 100 and reserves 100 (cash 10, government bonds 60 (discount 0.5%), bank deposits 25, commercial paper 5 (discount 5%)) and no frozen deposits, at $q = 0.3$ the disposable value is $10 + 59.7 + 25 + 4.75 = 99.45$, comfortably above the demand of 30. But **if the deposits of 25 are frozen** and $q = 0.8$, the disposable value is 74.45 against a demand of 80, so RR $= 0.93$ and redemption at par is impossible. And this with the collateral ratio still at 1.0. The collateral ratio is a number about solvency and the redeemable ratio a number about liquidity, and a run starts with the second number. NB24-2 below runs this arithmetic dynamically.

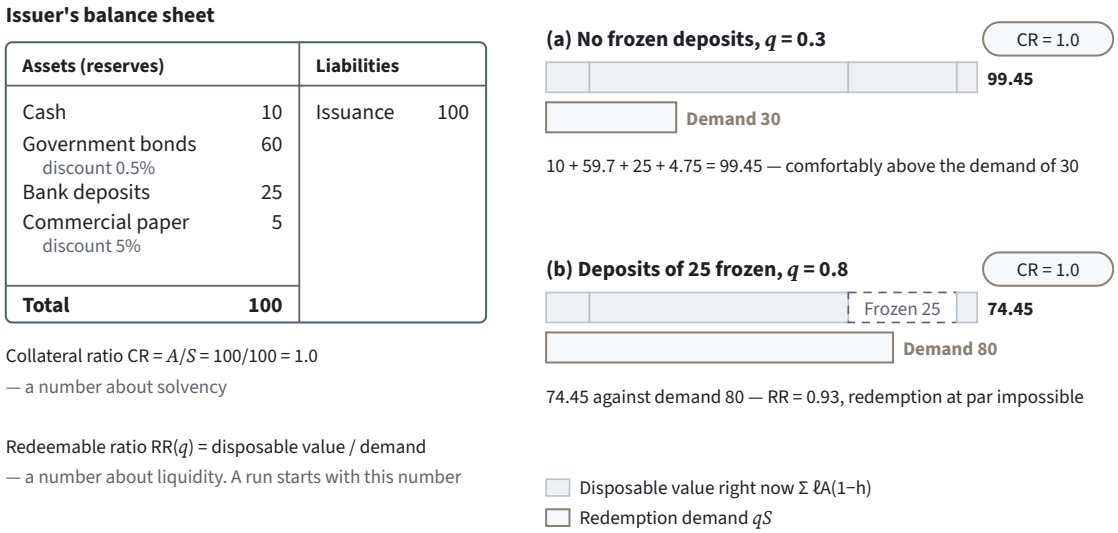


Figure 24-1. The issuer's balance sheet from the numerical example in the text, in two cases. The collateral ratio is 1.0 in both, but whether the value that can be disposed of immediately covers the redemption demand differs.

3. Why runs happen: the economics of "no questions asked"

Gorton and Zhang say that stablecoins are not yet money, and give as the reason the NQA principle we saw in Chapter 19: "it cannot just be assumed that an object will be used as a medium of exchange. For that to happen, the object must satisfy the NQA principle. Imagine attempting to purchase goods and services, or enter into contractual arrangements, when the value of the medium of exchange fluctuates **because parties are differentially informed** about its value. Engaging in such transactions would be incredibly difficult. Private producers of money therefore try to design an instrument that is information insensitive: no party to a transaction would engage in due diligence on its value because doing so would be too expensive. And because all parties to the transaction know this, they accept the instrument at par." Gorton and Zhang write that the best way to get that insensitivity is "backing the debt with debt," "by backing private banknotes with state bonds, by backing demand deposits with portfolios of loans, or by backing sale and repurchase agreements (repos) with specific bonds" (restating the argument of Dang, Gorton, Holmström and Ordoñez).[6] What, then, is a run? "if the price is to stay fixed and not vary (as other prices usually do in response to changes in demand), then it is the **quantity** that must sometimes adjust. During times of heightened economic uncertainty, the quantity can adjust very quickly to zero in a bank run, a situation in which the backing of the money becomes suspect."[7] Because the price is nailed down, a shock escapes through the quantity, and the adjustment of quantity becomes a queue in which those who come first are paid first.

Here stablecoin issuers stand in a different place from banks. "Stablecoin issuers therefore face a trade-off with respect to opacity and transparency. On one hand, it would be best if the backing for their stablecoins were so opaque that nobody would find it profitable to produce information about the backing assets. On the other hand, if the backing were not credible, then the market would want to produce information about the backing. Stablecoin issuers may take the view that transparency is best, because they are not regulated and cannot rely on bank examiners (and, thus, cannot be opaque)."[8] Banks used deposit insurance and examination to make it "unnecessary to ask"; stablecoins, with monthly attestation reports, say "ask and we will answer." The two solutions are not the same. Being able to ask also means that, once there is a reason to ask, everyone asks at once. A 2023 New York Fed report reports what that "asking" finds: "stablecoins exhibit an even wider range of risk profiles than MMFs. Some stablecoins report that they are backed by safe assets, such as cash and US Treasuries, … Others, instead, are reportedly backed by riskier collateral, such as corporate debt or even other crypto assets." And: "Reporting quality, however, varies across stablecoin issuers, because there is no standardized or regulatory reporting requirement."[9] Section 6 looks at this report's figures.

4. [Advanced] Diamond–Dybvig: a run is an equilibrium

The 1983 paper answers with one model why banks exist and why they are vulnerable to runs. The authors' summary: "Our model demonstrates three important points. First, banks issuing demand deposits can improve on a competitive market by providing better risk-sharing among people who need to consume at different random times. Second, the demand deposit contract providing this improvement has an undesirable equilibrium (a bank run) in which all depositors panic and withdraw immediately, including even those who would prefer to leave their deposits in if they were not concerned about the bank failing. Third, bank runs cause real economic problems because even healthy banks can fail, causing the recall of loans and the termination of productive investment."[10] Put another way, **liquidity services and vulnerability to runs are two sides of the same thing.** "It is precisely the transformation of illiquid claims into liquid claims that is responsible both for the liquidity service provided by banks and for their susceptibility to runs."[11]

▾ [Advanced] The skeleton of the model, two equilibria, and two remedies

[Advanced] There are three periods ($T = 0, 1, 2$) and one good. The production technology turns 1 unit put in at period 0 into $R > 1$ at period 2, but if it is interrupted at period 1 it gives back only the original 1. That is, "long-term capital investments are somewhat irreversible." Everyone is alike at period 0 and learns their own type **privately** at period 1. Type 1 wants to consume only at period 1 (a fraction t), and type 2 consumes at period 2.[12] If types were public information, the optimal insurance contract would give type 1 $c_1^* > 1$ and type 2 $c_2^* < R$, sharing risk between the two. But types are private information, so a market cannot use that contract, and the bank imitates it **with demand deposits**. "the demand deposit contract gives each agent withdrawing in period 1 a fixed claim of r_1 per unit deposited in period 0. Withdrawal tenders are served **sequentially** in random order until the bank runs out of assets."[13] The amount V_1 received by someone who takes r_1 at period 1, and the amount V_2 received by someone who waits and withdraws at period 2, are

$$V_1(f_j, r_1) = \begin{cases} r_1 & f_j < r_1^{-1} \\ 0 & f_j \geq r_1^{-1} \end{cases}, \qquad V_2(f, r_1) = \max\left\{\frac{(1 - r_1 f)R}{1 - f},\ 0\right\}$$

(equations (6) and (7)). Reading the symbols: f_j is the fraction of withdrawals served before me, and f is the total fraction withdrawn at period 1. If $r_1 f$ exceeds 1, the bank's assets run out and those further back in line get 0, and those who waited share the remaining $(1 - r_1 f)$ grown by a factor of R.[14]

[Advanced] **Two equilibria.** Set $r_1 = c_1^*$: "If this contract is in place, it is an equilibrium for type 1 agents to withdraw at T = 1 and for type 2 agents to wait, provided this is what is anticipated. This good equilibrium achieves optimal risk-sharing. Another equilibrium (a bank run) has all agents panicking and trying to withdraw their deposits at T = 1: if this is anticipated, all agents will prefer to withdraw at T = 1. This is

because the face value of deposits is larger than the liquidation value of the bank's assets." And: "For all $r_1 > 1$, runs are an equilibrium. ... if $r_1 = 1$, the bank simply mimics direct holding of the assets, ... A demand deposit contract which is not subject to runs provides no liquidity services."[15] Which equilibrium is realized lies outside the model. "This could happen if the selection between the bank run equilibrium and the good equilibrium depended on some commonly observed random variable in the economy. This could be a bad earnings report, a commonly observed run at some other bank, a negative government forecast, or even sunspots. ... The observed variable need not convey anything fundamental about the bank's condition. The problem is that once agents have deposited, **anything** that causes them to anticipate a run will lead to a run."[16] The sunspots of Chapter 19 come back here. There they were a device for selecting a collapse to a value of 0; here, for selecting a run.

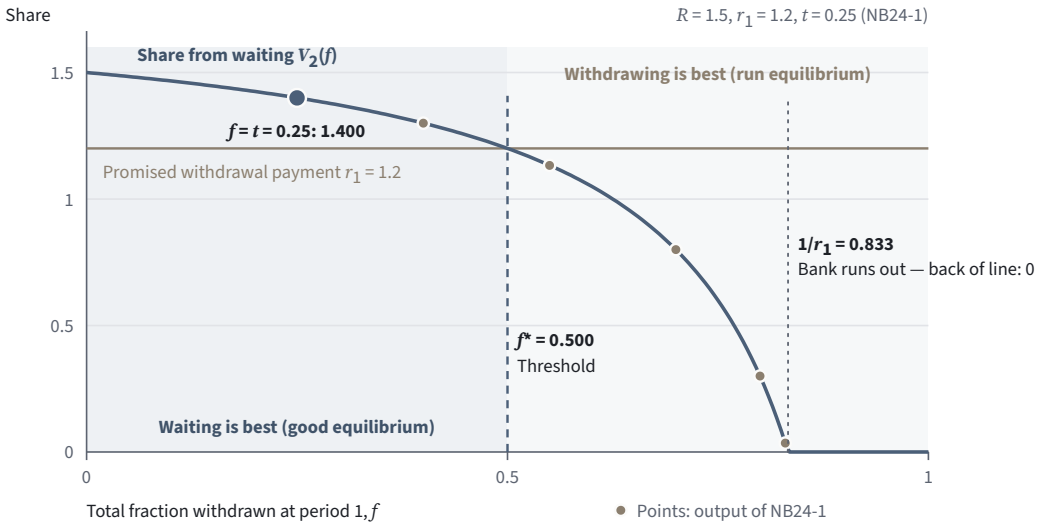


Figure 24-2. The share of those who wait and the promised withdrawal payment r_1 in the Diamond–Dybvig model. Withdrawals are served sequentially, so when $r_1 f$ exceeds 1 the bank's assets run out and those further back in line get 0. The curve is the graph of equation (7) in the text ($R = 1.5, r_1 = 1.2$), and the points and numbers are the output of NB24-1.

[Advanced] **Two remedies.** The first is **suspension of convertibility.** "If banks can suspend convertibility when withdrawals are too numerous at T = 1, anticipation of this policy prevents runs by removing the incentive of type 2 agents to withdraw early." With a suspension point $\hat{f}$, "no type 2 agent will withdraw at T = 1 because no matter what the agent anticipates about others' withdrawals, the agent receives higher proceeds by waiting until T = 2 to withdraw ... Therefore, there is a unique Nash equilibrium which has $f = t$. In fact, this is a dominant strategy equilibrium ...." But "This contract works perfectly only in the case where the normal volume of withdrawals, t, is known and not stochastic."[17] If t is random, Proposition 1 holds, "Bank contracts (which must obey the sequential service constraint) cannot achieve optimal risk-sharing when t is stochastic ...," and suspension of convertibility prevents runs but leaves behind the inefficiency of locking in, after the fact, some type 1 agents. The second is **deposit insurance.** "Demand deposit contracts with government

deposit insurance achieve the unconstrained optimum as a unique Nash equilibrium (in fact, a dominant strategies equilibrium) if the government imposes an optimal tax to finance the deposit insurance." The key is that the government can tax **after the withdrawals are over.** "As the government can impose a tax on an agent who has withdrawn, the government can base its tax on f, the realized total value of T = 1 withdrawals. This is in marked contrast to a bank, which must provide sequential service and cannot reduce the amount of a withdrawal after it has been made." And: "for all possible anticipated withdrawal policies of other agents, participating in a bank run never pays. As a result, no strategic issues of confidence arise."[18] The authors add that the lender of last resort (the discount window) can do the same work as deposit insurance when the technology is riskless.[19]

```
# NB24-1: Diamond-Dybvig's two equilibria — with a demand deposit r1 > 1, for each total
period-1 withdrawal fraction f, is the share from "waiting," V2, larger than "withdrawing,"
r1? (for study)
R, r1, t = 1.5, 1.2, 0.25          # period-2 output R, promised period-1 payment r1 (> 1),
fraction t of type 1 (those who must consume at period 1)

def V2(f):                          # equation (7): the share of those who wait
    return max((1 - r1 * f) * R / (1 - f), 0.0) if f < 1 else 0.0

print(f"R = {R}, r1 = {r1}, type-1 fraction t = {t}. Withdrawal fraction at which the bank
runs out, 1/r1 = {1 / r1:.3f}")
print(f"{'Period-1 f':>12} {'Waiting V2(f)':>16} {'Payout r1':>10} {'Type 2 best':>14}")
for f in [t, 0.40, 0.55, 0.70, 0.80, 0.83, 0.90, 1.00]:
    v = V2(f)
    print(f"{f:>12.2f} {v:>16.3f} {r1:>10.2f} {'wait' if v > r1 else 'withdraw (run)':>14}")
# The threshold where waiting becomes worse than withdrawing: (1 - r1 f)R/(1 - f) = r1  →  f*
= (R - r1)/(r1 (R - 1))
f_star = (R - r1) / (r1 * (R - 1))
print(f"\nThreshold f* = (R - r1)/(r1(R - 1)) = {f_star:.3f}. If you believe others will
withdraw more than this, withdrawing is your best response too: belief decides the outcome.")
print(f"At f = t = {t} (normal), type 2 waits and receives {V2(t):.3f} (> r1). At f = 1 (run),
everyone shares an average of 1 and the back of the line gets 0: everyone is worse off than
with no bank.")
print("\nAdd suspension of convertibility (period-1 payments stop when f reaches the
suspension point) and the share of those who wait cannot fall below r1, so waiting becomes a
dominant strategy for type 2 — when t is known.")
```

```
# Output — bcbook (Python 3.13), exactly as run
R = 1.5, r1 = 1.2, type-1 fraction t = 0.25. Withdrawal fraction at which the bank runs out,
1/r1 = 0.833
  Period-1 f    Waiting V2(f)  Payout r1    Type 2 best
        0.25            1.400       1.20           wait
        0.40            1.300       1.20           wait
        0.55            1.133       1.20 withdraw (run)
        0.70            0.800       1.20 withdraw (run)
        0.80            0.300       1.20 withdraw (run)
        0.83            0.035       1.20 withdraw (run)
        0.90            0.000       1.20 withdraw (run)
        1.00            0.000       1.20 withdraw (run)

Threshold f* = (R - r1)/(r1(R - 1)) = 0.500. If you believe others will withdraw more than
this, withdrawing is your best response too: belief decides the outcome.
At f = t = 0.25 (normal), type 2 waits and receives 1.400 (> r1). At f = 1 (run), everyone
shares an average of 1 and the back of the line gets 0: everyone is worse off than with no
bank.

Add suspension of convertibility (period-1 payments stop when f reaches the suspension point)
and the share of those who wait cannot fall below r1, so waiting becomes a dominant strategy
for type 2 — when t is known.
```

5. Carrying it over to stablecoins: what is the same and what is different

Put the stablecoin counterpart in each place in the model. Bank = issuer. Illiquid assets = the reserve assets that cannot be sold right now, or cannot be sold without a discount (ℓ_i and h_i in Section 2). Demand deposit = claim to redemption at par. Sequential service = the redemption queue in the primary market (the issuer's window). The two equilibria and the sunspot selection carry over unchanged. Two things are different.

First, **there is no deposit insurance.** Table 1 of Gorton and Zhang lays out the options. The status quo and "Implement bank-like regulations on stablecoin issuers, but no insurance" solve neither runs nor NQA, and only three, "Issue stablecoins from within the insured-bank regulatory perimeter," "Require stablecoins to be backed one-for-one by Treasuries or central bank reserves," and "Replace stablecoins with a central bank digital currency," solve both, the authors write.[20] Why one-for-one backing removes runs can be seen directly from Diamond–Dybvig. If $r_1 = 1$ and all the assets can be disposed of immediately, a run is not an equilibrium, and at that price the core of banking, liquidity transformation, disappears as well. This book does not say which option is right. Which one each jurisdiction chose is for the survey in Chapter 27.

Second, **there is a secondary market.** Bank deposits are not transferable, but stablecoins trade on exchanges. A December 2025 note from the Federal Reserve Board points to this difference in the events of March 2023: "Unlike bank deposits, stablecoins are actively traded on exchanges in the secondary markets. This means that the suspension of redemption on the primary market cannot arrest a run, as temporarily closing a bank would."[21] The first remedy of Diamond–Dybvig, suspension of convertibility, works differently here. Close the window in the primary market and the redemption pressure escapes into the price on the secondary market (the Fed note writes of "[p]ent up redemption pressure" while the window was closed). The feedback by which a falling price brings on more redemption demands is not something the note reports; **it is an assumption of this book's model,** and NB24-2 below puts it in as a simple rule.

Reserve composition and redemption demand

```
# NB24-2: reserve composition × redemption demand → depeg feedback — types A, B and C are
illustrations of structure, not the books of any coin (for study)
# Feedback rule (this book's simplification): if the redeemable ratio RR < 1, or the issuer
closes the window, the secondary-market price moves down toward RR,
# and if the price is below 1, more holders come forward to redeem (by the 'sensitivity' below).
Prices and quantities are all model variables.
def run_dynamics(reserves, q0, sensitivity=1.5, rounds=6, window_open=True):
    """reserves: [(name, weight, disposable share ℓ, fire-sale discount h)], weights sum to 1
(collateral ratio 1.0). q0: initial redemption demand fraction."""
    S = 1.0                                  # issuance normalized to 1
    q, price, history = q0, 1.0, []
    redeemed = 0.0
    for r in range(1, rounds + 1):
        liquid = sum(w * l * (1 - h) for _, w, l, h in reserves)        # value that can be
disposed of right now (per issuance of 1)
        demand = min(q, 1.0) - redeemed
        rr = liquid / demand if demand > 0 else float("inf")
        if window_open and rr >= 1:
            redeemed += demand; price = 1.0
            history.append((r, q, round(rr, 2), price, "redeemed at par"))
            break
        price = max(min(rr, 1.0), 0.0) if window_open else max(price - 0.05, 0.0)   # if the
window closes, the price slides (rule)
        history.append((r, round(q, 3), round(rr, 2), round(price, 3), "window open" if
window_open else "window closed"))
        q = min(1.0, q + sensitivity * (1 - price) * (1 - q))          # if the price is below
1, more holders come forward to redeem
    return history

types = {
    "A Mostly cash and T-bills":            [("cash", 0.20, 1.0, 0.0), ("T-bills", 0.75, 1.0,
0.005), ("bank deposits", 0.05, 1.0, 0.0)],
    "B Deposits and CP mixed in":           [("cash", 0.05, 1.0, 0.0), ("T-bills", 0.40, 1.0,
0.005), ("bank deposits", 0.30, 1.0, 0.0), ("commercial paper", 0.25, 1.0, 0.05)],
    "B' B with all bank deposits frozen":   [("cash", 0.05, 1.0, 0.0), ("T-bills", 0.40, 1.0,
0.005), ("bank deposits (frozen)", 0.30, 0.0, 0.0), ("commercial paper", 0.25, 1.0, 0.05)],
    "C Crypto-overcollateralized (initial collateral ratio 1.5 → 0.83 after a 45% sharp fall in
collateral)": [("collateral crypto-asset — value left after a 45% sharp fall, fire-sale discount
15%", 1.5 * 0.55, 1.0, 0.15)],
}
for name, res in types.items():
    cr = sum(w for _, w, _, _ in res)
    print(f"\n[{name}] collateral ratio {cr:.2f}")
    for q0 in [0.10, 0.40, 0.80]:
        hist = run_dynamics(res, q0)
        last = hist[-1]
        print(f"  initial demand {q0:.0%}: {len(hist)} round(s) → demand {last[1]:.0%},
redeemable ratio {last[2]}, price {last[3]} ({last[4]})")

print("\n[B' with the issuer closing the window] — with a secondary market, closing the window
still carries the run into the price")
for row in run_dynamics(types["B' B with all bank deposits frozen"], 0.40, window_open=False):
    print("  round {} demand {} RR {} price {} {}".format(*row))
```

```
# Output — bcbook (Python 3.13), exactly as run

[A Mostly cash and T-bills] collateral ratio 1.00
  initial demand 10%: 1 round(s) → demand 10%, redeemable ratio 9.96, price 1.0 (redeemed at
par)
  initial demand 40%: 1 round(s) → demand 40%, redeemable ratio 2.49, price 1.0 (redeemed at
par)
  initial demand 80%: 1 round(s) → demand 80%, redeemable ratio 1.25, price 1.0 (redeemed at
par)

[B Deposits and CP mixed in] collateral ratio 1.00
  initial demand 10%: 1 round(s) → demand 10%, redeemable ratio 9.86, price 1.0 (redeemed at
par)
  initial demand 40%: 1 round(s) → demand 40%, redeemable ratio 2.46, price 1.0 (redeemed at
par)
  initial demand 80%: 1 round(s) → demand 80%, redeemable ratio 1.23, price 1.0 (redeemed at
par)

[B' B with all bank deposits frozen] collateral ratio 1.00
  initial demand 10%: 1 round(s) → demand 10%, redeemable ratio 6.85, price 1.0 (redeemed at
par)
  initial demand 40%: 1 round(s) → demand 40%, redeemable ratio 1.71, price 1.0 (redeemed at
par)
  initial demand 80%: 6 round(s) → demand 97%, redeemable ratio 0.7, price 0.704 (window open)

[C Crypto-overcollateralized (initial collateral ratio 1.5 → 0.83 after a 45% sharp fall in
collateral)] collateral ratio 0.83
  initial demand 10%: 1 round(s) → demand 10%, redeemable ratio 7.01, price 1.0 (redeemed at
par)
  initial demand 40%: 1 round(s) → demand 40%, redeemable ratio 1.75, price 1.0 (redeemed at
par)
  initial demand 80%: 6 round(s) → demand 96%, redeemable ratio 0.73, price 0.726 (window open)

[B' with the issuer closing the window] — with a secondary market, closing the window still
carries the run into the price
  round 1 demand 0.4 RR 1.71 price 0.95 window closed
  round 2 demand 0.445 RR 1.54 price 0.9 window closed
  round 3 demand 0.528 RR 1.3 price 0.85 window closed
  round 4 demand 0.634 RR 1.08 price 0.8 window closed
  round 5 demand 0.744 RR 0.92 price 0.75 window closed
  round 6 demand 0.84 RR 0.82 price 0.7 window closed
```

[Intermediate] When reading the table, do not get caught on the type names. A, B and C differ only in the **composition** of their reserve assets, and this book does not say which type is in fact which coin. Three things show. Even with the same collateral ratio of 1.0, the disposable share ℓ and the fire-sale discount h decide the outcome (B and B′), and this is a problem of liquidity. Even with an initial collateral ratio of 1.5, a 45% sharp fall in the collateral takes the collateral ratio itself below 1 (0.83) (C), so the break from par in that case is a problem not of liquidity but **of solvency**; in the overcollateralized type the two problems are tied together through a single collateral price. And **if the rule that the price slides when the window closes is put into the cell,** closing the window, unlike at a bank, does not stop the run. This third point is not a result the cell showed but a consequence of the assumption that a secondary market exists, and the basis for that assumption is the Fed note below. The interactive figure W8 for this chapter moves only the three variables of this cell (the redemption demand fraction, the liquidity composition of reserve assets, and the collateral ratio) and uses no coin names.

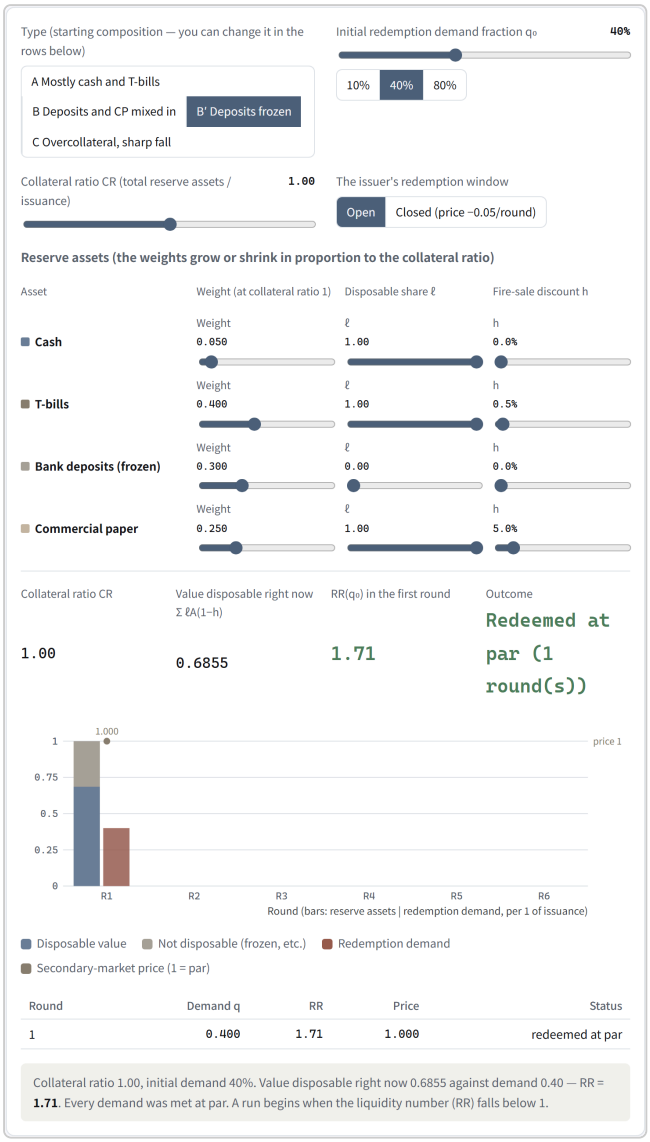


Figure 24-3. Stablecoin bank run. In type B′ (30% of bank deposits frozen), a redemption demand of 40% is met at par with a redeemable ratio of 1.71, but at 80% it slides after six rounds to a demand of 97% and a price of 0.704. The types are illustrations of structure, not the books of any coin. — In the HTML edition you can change the values here (HTML edition, Chapter 24, Section 5).

Algorithmic type: a promise of exchange in place of reserves

[Advanced] Reflexive feedback in the algorithmic type: a simple model

[Advanced] The algorithmic type backs its 1 dollar not with reserve assets but **with a promise of exchange into a second token.** Translated into model variables, the mechanism the BIS describes runs like this. There is a stable token S and a volatile token L, and one S can always be exchanged for "1 dollar's worth of L." If the price of L is p_L, one S is burned and $1 / p_L$ new units of L are minted. For this arrangement to work, there must be demand to take up L, and in the BIS's terms users would only be willing to exchange "if [L's] market capitalisation exceeded that of [S]." When demand wavers and p_L falls, more L has to be minted to hand over the same 1 dollar's worth, the added supply pushes p_L down further, and that brings on more redemptions. This is the mechanism of what the BIS calls "a classic run dynamic,"[22] and what actually happened in May 2022, and in what order, is read in Chapter 26. The cell below imitates this feedback with a single demand function. Every number is a model value and is fitted to no data.

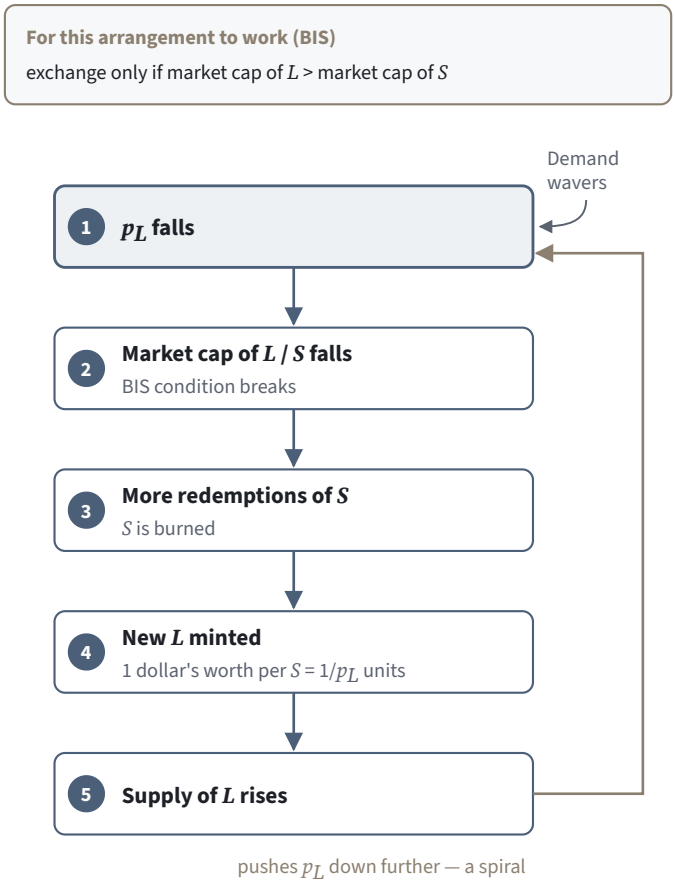


Figure 24-4. Reflexive feedback in an algorithmic stablecoin. The loop draws the order in which the text translated the BIS mechanism into model variables; no numbers are included.

```
# NB24-3: reflexive feedback in the algorithmic type — does a spiral arise from just three
things: the exchange promise "one S = 1 dollar's worth of L," the demand curve for L, and
redemptions that grow once the backing breaks? (for study, model variables)
def algorithmic_spiral(S0, L0, p0, eps=1.0, rounds=10, base_redeem=0.05, panic=0.6,
shock_round=3, shock=0.3):
    """Demand curve for L: p_L = D / L**(1+eps). With eps > 0, the price falls more than
proportionally as supply rises (demand is inelastic; with eps=1, L market cap = D/L).
    Each round some S holders burn S and receive 1/p_L units of L. If the backing (L market
cap / S) is below 1, the redemption fraction grows by panic.
    At shock_round the demand level D shrinks by a factor of shock. All numbers are model
values."""
    S, L = float(S0), float(L0)
    D = p0 * L ** (1 + eps)
    print(f"{'Round':>5} {'S supply':>9} {'L supply':>11} {'p_L':>8} {'L mcap / S':>12}
{'Redeem share':>12}")
    for r in range(1, rounds + 1):
        if r == shock_round:
            D *= shock                                          # demand shock (model variable)
        p_L = D / L ** (1 + eps)
        backing = L * p_L / S                                   # dollars' worth that all of L
can pay back per S
        share = base_redeem + (panic * (1 - backing) if backing < 1 else 0.0)
        share = min(share, 0.9)
        print(f"{r:>5} {S:>9,.0f} {L:>11,.0f} {p_L:>8.4f} {backing:>12.2f} {share:>12.2f}")
        burn = S * share
        S -= burn
        L += burn / p_L                                         # mint new L worth "1 dollar"
for each S burned
    return S, L, D / L ** (1 + eps)

S_end, L_end, p_end = algorithmic_spiral(S0=1_000, L0=2_000, p0=1.0)
print(f"\nEnd: S {S_end:,.0f}, L {L_end:,.0f}, p_L {p_end:.4f}.")
print("After the demand shock, once the market cap of L falls below S (once the BIS condition
breaks), redemptions mint more L, the added supply pushes p_L down further, and that brings on
more redemptions — a spiral.")
print("The numbers in this table are model values fitted to no data. What shows is only the
shape: when the backing is \"demand for another token,\" the backing disappears when that
demand disappears.")
```

```
# Output — bcbook (Python 3.13), exactly as run
Round  S supply    L supply      p_L   L mcap / S Redeem share
    1     1,000       2,000   1.0000         2.00         0.05
    2       950       2,050   0.9518         2.05         0.05
    3       902       2,100   0.2721         0.63         0.27
    4       659       2,996   0.1337         0.61         0.29
    5       471       4,400   0.0620         0.58         0.30
    6       328       6,699   0.0267         0.55         0.32
    7       222      10,663   0.0106         0.51         0.35
    8       145      17,966   0.0037         0.46         0.37
    9        91      32,602   0.0011         0.40         0.41
   10        54      65,409   0.0003         0.34         0.45

End: S 30, L 151,116, p_L 0.0001.
After the demand shock, once the market cap of L falls below S (once the BIS condition
breaks), redemptions mint more L, the added supply pushes p_L down further, and that brings on
more redemptions — a spiral.
The numbers in this table are model values fitted to no data. What shows is only the shape:
when the backing is "demand for another token," the backing disappears when that demand
disappears.
```

6. Evidence: two runs

[Intermediate] Three empirical studies have set down the runs of May 2022 and March 2023 in data. This book takes from them **only the figures about structure,** and attaches a sample and period to each figure.

STUDY	DATA AND PERIOD	FINDINGS	LIMITATIONS
Anadu et al., NY Fed Staff Report 1073 (2023/2024)	Daily stablecoin flows and prices; stress events in crypto markets (identified by days of large declines in the price of Bitcoin)	**Flight to safety**: on stress days, net outflows from coins whose reserve assets are "risky" by the issuer's report into "safe" coins. **Depeg threshold**: at a price of 1 dollar or above, flows are unrelated to price, but once the price drops below 1 dollar the daily outflow rate jumps by 34 bp (compare the "95-cent" threshold for MMFs). The May 2022 run started in algorithmic and risky-collateral coins; the March 2023 run started in a coin previously regarded as "safe" and spread to other coins that held that coin as collateral. Flight tends to happen **within the same blockchain**	The "safe/risky" classification depends on the issuers' self-reports (footnote 6). The threshold estimate is identified from the discontinuity around a price of 1 dollar: an observed response, not a causal one
Du, Sonawane and Watsky, FEDS Note (2025-12-17)	Hourly primary-market net issuance, secondary-market prices and trading volume for 2023-03-09 to 14 (data providers Allium and CryptoCompare)	The issuer disclosed that it held USD 3.3 billion, **about 8% of total reserves,** on deposit at the failed bank → redemption demands spiked → the primary market was effectively halted over the weekend → secondary-market low of 86 cents → recovery after the authorities announced full protection of deposits, and back to the peg after redemptions resumed on Monday. Once the primary market closed, secondary-market trading jumped (about USD 2 billion per hour), and **decentralized exchanges accounted for most of the trading.** Contagion, because another coin held this coin as collateral through a 1:1 exchange facility	A description of a single event, with no counterfactual (stated by the authors). Policy discussion is outside its scope
Anadu et al., Liberty Street Economics (2026-07-31)	The same issuer's reserve composition (weighted average maturity of the reserve MMF, repo share, bank-deposit share), monthly and quarterly, 2023–2026	After the event, the weighted average maturity of the reserve MMF fell below the 5th percentile of the distribution for Treasury-only MMFs and stayed there; the repo share jumped from 0 to over 90% and then came back to 69%; bank deposits fell below their pre-event level and became concentrated in large (GSIB) banks → the **type** of risk changed, "away from interest-rate risk and toward counterparty risk"	Voluntary disclosures of a single issuer

23 24 25

[Intermediate] Put the three studies on top of each other and what resembles the textbook separates from what does not. What resembles it is the redemption queue in which those who come first are paid first, and contagion (to other coins that held one coin as collateral). Two things differ. First, the trigger was not a sunspot but **fundamental information**: the disclosure that about 8% of reserves were tied up, and the Fed note too writes that "Solvency concerns around [the issuer] are likely to have played a part."[27] Second, the secondary market. Over the weekend when the window was closed, trading crowded into the secondary market and the price hit its low. The recovery came in two stages: the price "recovered sharply" after the authorities' announcement on protecting deposits, and "fully recovered" on the Monday the issuer resumed redemptions. Because the note does not deal with counterfactuals, these data cannot say which of the two stopped the run. What can be said is only the order, and **the structure** in which the second remedy of Diamond–Dybvig

(deposit insurance) was applied not to the stablecoin itself but to the bank where its reserve assets sat.

[Advanced] Go one step further and there is feedback on the reserve-asset side as well. BIS Working Paper 1270 "examines the impact of dollar-backed stablecoin flows on short-term US Treasury yields using daily data from January 2021 to March 2026," and reports that "Using local projections and a granular instrumental variable exploiting idiosyncratic variation in stablecoin growth at the token-blockchain pair level, we find a $3.5 billion (2-standard deviation) inflow lowers 3-month Treasury bill yields by 0.71 basis points on impact, and up to 4 basis points within 10 days, with limited spillovers to longer tenors," and that the effects are "increasing under Treasury market stress and with the scale of the stablecoin sector."[26] These estimates are from the June 2026 revision, and because the identification strategy (an instrumental variable) is stated, they can be read, on the premise that the instrument's exclusion restriction holds, as an effect rather than a correlation. The authors describe the estimates as "statistically significant" and present the uncertainty through confidence intervals in figures in the body of the paper, so this book takes only the point estimates. The implication runs **in the opposite direction from a run,** and that is the authors' own word: "our estimates likely constitute a lower bound for outflow effects, as they are identified from a period of secular market growth that affords issuers discretion over the timing of purchases." If redemptions bunch up and reserve Treasuries have to be sold, yields could move through the same channel, and this is the place where the Fed note writes that "had the run unfolded differently, the pressure for [stablecoin] redemptions might have forced [the issuer] to liquidate backing assets (for example, U.S.Treasury securities), with potential knock-on effects on traditional financial markets."[28] The size of the sector and its Treasury holdings as of the reference date are kept in the trend notes in Appendix A (A-18).

> **? A LAWYER'S QUESTION**
>
> What does a stablecoin holder have? Gorton and Zhang ask, "In order for the stablecoins to be considered a deposit—let alone a demand deposit—one must determine whether the underlying contract between the holder of stablecoins and the stablecoin issuer is an equity contract or a debt contract," and write that the question has the same shape as the 1970s dispute over whether MMFs took "deposits."[29] Is the holder a depositor (예금자) (and if so, is the holder within the scope of deposit insurance and bank regulation)? An unsecured creditor (무담보 채권자) (and if so, does the holder rank with general creditors (일반 채권자) if the issuer becomes insolvent)? Or the beneficiary (수익자) of a trust (신탁) over the reserve assets (and if so, are the reserve assets separated from the insolvency estate (도산재단))? In March 2023 the issuer publicly stated that it would "cover any shortfall using corporate resources, involving external capital if necessary" (Fed note §1, footnote 27). What rights does that statement give holders? If the law requires one-for-one backing, what must the legal structure of that "backing" be: segregated custody (분리 보관), priority (우선권), a trust? This book does not answer; it points to the survey by jurisdiction and the issue sections of Part 5. → Chapter 27 · Chapter 28

7. Summary

A stablecoin is a token that carries a promise of redemption, and so it belongs not to the economics of outside money but to the economics of banking. Whether what backs the promise is fiat assets, crypto-assets held as overcollateral, an exchange into a second token, or a hedged derivatives position, the way 1 dollar breaks is the same. When redemptions bunch up, assets cannot be sold in time at full value, and the belief that the promise might break brings on redemptions. The collateral ratio is a number about solvency and the redeemable ratio a number about liquidity, and a run starts with the second. Diamond–Dybvig showed that a run is a second equilibrium that exists alongside the good one, that the choice between them can turn on signals unrelated to fundamentals, and what suspension of convertibility and deposit insurance each change. A stablecoin has no deposit insurance and has a secondary market, so closing the window does not stop a run. The run of March 2023 showed dynamics resembling this picture (the trigger was fundamental information, and the order of recovery only as far as the data tell), and the algorithmic type of May 2022 is a structure without liquidity transformation, so the correspondence differs; Chapter 26 reads that event. This chapter has not said which coin is safe. "Stability" is a property not of the price but of the balance sheet and the procedure. Chapter 25 asks about the opposite end: whether the word "bubble" can be attached by statistics to the price of an asset that promises nothing. The collapse of May 2022 is read as a structure of failure in Chapter 26.

Sources for this chapter

- Gary B. Gorton & Jeffery Y. Zhang, "Taming Wildcat Stablecoins," *University of Chicago Law Review* 90(3): 909–972 (2023): abstract (p. 909), quantity adjustment and runs (p. 912), Table 1 (p. 913) and drawbacks (p. 914), NQA and debt backed by debt (p. 918), the opacity trade-off (p. 919), whether it is a demand deposit (p. 920).
- Douglas W. Diamond & Philip H. Dybvig, "Bank Runs, Deposit Insurance, and Liquidity," *Journal of Political Economy* 91(3): 401–419 (1983). Read in the reprint in *Federal Reserve Bank of Minneapolis Quarterly Review* 24(1): 14–23 (2000), and the pages cited are those of the reprint (the three points p. 15, the model p. 16, the contract and equations (6) and (7) pp. 17–18, the two equilibria and sunspots p. 18, suspension of convertibility p. 19, Proposition 1 p. 20, Proposition 2 p. 21, the discount window p. 22). The page numbers of the JPE original were not checked.
- Bank for International Settlements, "III. The future monetary system," *BIS Annual Economic Report 2022*, pp. 75–115: the two types (p. 81), Box A (p. 82).
- Kenechukwu Anadu and 9 others, "Runs and Flights to Safety: Are Stablecoins the New Money Market Funds?," Federal Reserve Bank of New York Staff Reports No. 1073 (2023-09, rev. 2024-04): abstract and introduction (pp. 1–3).
- Chuan Du, Ria Sonawane & Cy Watsky, "In the Shadow of Bank Runs," *FEDS Notes* (2025-12-17), DOI 10.17016/2380-7172.3958: Summary, §§1–1.2. HTML (accessed 2026-10-25).
- Pablo D. Azar & Jeff Garofano, "Synthetic Stablecoins and Financial Stability," *Liberty*

Street Economics (2026-06-23), DOI 10.59576/lse.20260623: the structure and fragility of the synthetic type. HTML (accessed 2026-10-25).

- Kenechukwu Anadu and 8 others, "Stablecoins and (Non)Crypto Shocks: A 2026 Update," *Liberty Street Economics* (2026-07-31): changes in reserve composition. HTML (accessed 2026-10-25).
- Rashad Ahmed & Iñaki Aldasoro, "Stablecoins and safe asset prices," BIS Working Papers No 1270 (2025-05, rev. 2026-06): abstract.
- Rodney Garratt & Neil Wallace (2018), p. 1887: inside and outside money (the same place as in Chapter 19).
- The equations for the collateral ratio and the redeemable ratio, and the feedback rules in NB24-2 and NB24-3, are this book's simplifications and are fitted to the books of no coin.
- Code run: Python 3.13 (conda environment `bcbook`), standard library only. Notebook NB24. The initial state of interactive figure W8 `stablecoin-run` is type B′ of NB24-2 with an initial demand of 40%; its variables are the three of redemption demand fraction, liquidity composition of reserve assets, and collateral ratio, and it uses no coin names.

Notes

1. Garratt & Wallace (2018), p. 1887. Chapter 19, footnote 4.
2. Gary B. Gorton & Jeffery Y. Zhang, "Taming Wildcat Stablecoins," *University of Chicago Law Review* 90(3): 909–972 (2023), p. 909 (abstract). Emphasis added. Original: "While the technology underlying cryptocurrencies is new, the economics is centuries old. ... In this case, the new technology has recreated circulating private money in the form of stablecoins, which are similar to the banknotes that circulated in many countries during the nineteenth century. The implication is that stablecoin issuers are unregulated banks. Based on lessons learned from economic theory and financial history, we argue that circulating private money is not an effective medium of exchange because it is not always accepted at par and its issuers are vulnerable to destabilizing bank runs."
3. Bank for International Settlements, "III. The future monetary system," *BIS Annual Economic Report 2022*, pp. 75–115 (2022-06-26), p. 81. Original: "The two main types of stablecoin are asset-backed and algorithmic. Asset-backed stablecoins, such as Tether, USD Coin and Binance USD, are typically managed by a centralised intermediary who invests the underlying collateral and coordinates the coins' redemption and creation. Assets can be held in government bonds, short-term corporate debt or bank deposits, or in other cryptocurrencies. In contrast, algorithmic stablecoins, such as TerraUSD before its implosion, rely on complex algorithms that automatically rebalance supply to maintain their value relative to the target currency or asset. To avoid reliance on fiat currency, they often do so by providing users with an arbitrage opportunity relative to another cryptocurrency." The coin names the original gives as examples are omitted from the main text of this book.
4. BIS (2022), p. 82, Box A "The collapse of the TerraUSD stablecoin". Emphasis added. On the course and scale of the collapse, see Chapter 26.
5. Pablo D. Azar & Jeff Garofano, "Synthetic Stablecoins and Financial Stability," Federal Reserve Bank of New York *Liberty Street Economics*, 2026-06-23. Original: "synthetic stablecoins like Ethena (USDe) target their dollar peg through financial engineering: they combine a long position in cryptocurrency (typically ether staked in a staking pool like Lido) with an offsetting short position in perpetual futures contracts. ... This structure creates a 'synthetic' dollar position without requiring actual dollar reserves."; "The key fragility, however, is that the hedge depends on continuous liquidity in derivatives markets and a consistently positive funding rate. If funding turns negative or margin costs rise, the synthetic dollar becomes expensive to maintain and redemptions begin." https://libertystreeteconomics.newyorkfed.org/2026/06/synthetic-stablecoins-and-financial-stability/ (HTML, reopened on 2026-10-25 and checked word for word). The protocol names the original gives are omitted from the main text.

6. Gorton & Zhang (2023), p. 918. Emphasis added. Original: "But it cannot just be assumed that an object will be used as a medium of exchange. For that to happen, the object must satisfy the NQA principle. Imagine attempting to purchase goods and services, or enter into contractual arrangements, when the value of the medium of exchange fluctuates because parties are differentially informed about its value. Engaging in such transactions would be incredibly difficult. Private producers of money therefore try to design an instrument that is information insensitive: no party to a transaction would engage in due diligence on its value because doing so would be too expensive. And because all parties to the transaction know this, they accept the instrument at par."; "And this is best accomplished by backing the debt with debt—for example, by backing private banknotes with state bonds, by backing demand deposits with portfolios of loans, or by backing sale and repurchase agreements (repos) with specific bonds."

7. Gorton & Zhang (2023), p. 912. Emphasis added. Original: "However, if the price is to stay fixed and not vary (as other prices usually do in response to changes in demand), then it is the quantity that must sometimes adjust. During times of heightened economic uncertainty, the quantity can adjust very quickly to zero in a bank run, a situation in which the backing of the money becomes suspect. Stablecoin issuers are vulnerable to these bank runs, which can spill over into the broader economy."

8. Gorton & Zhang (2023), p. 919.

9. Kenechukwu Anadu, Pablo D. Azar, Marco Cipriani, Thomas M. Eisenbach, Catherine Huang, Mattia Landoni, Gabriele La Spada, Marco Macchiavelli, Antoine Malfroy-Camine & J. Christina Wang, "Runs and Flights to Safety: Are Stablecoins the New Money Market Funds?," Federal Reserve Bank of New York Staff Reports No. 1073 (2023-09, revised 2024-04), pp. 1–2 and footnote 6. Original: "In fact, stablecoins exhibit an even wider range of risk profiles than MMFs. Some stablecoins report that they are backed by safe assets, such as cash and US Treasuries, whose value remains stable or even increases during times of stress. Others, instead, are reportedly backed by riskier collateral, such as corporate debt or even other crypto assets ..."; footnote 6: "Our characterization of a stablecoin as 'safe' or 'risky' is based on the composition of its assets (also known as 'reserves') as reported by the stablecoin issuer. Reporting quality, however, varies across stablecoin issuers, because there is no standardized or regulatory reporting requirement." (2024-04 revision).

10. Douglas W. Diamond & Philip H. Dybvig, "Bank Runs, Deposit Insurance, and Liquidity," *Journal of Political Economy* 91(3): 401–419 (1983). This book read it in the reprint in *Federal Reserve Bank of Minneapolis Quarterly Review* 24(1): 14–23 (2000), and page numbers are **those of the reprint** (reprint p. 15). The reprint states in its head footnote that "This article is reprinted, with permission, from the Journal of Political Economy (1983, vol. 91, no. 3, pp. 401-19). ... The article was edited for publication in the Federal Reserve Bank of Minneapolis Quarterly Review."

11. Diamond & Dybvig (1983/2000), reprint p. 18.

12. Diamond & Dybvig (1983/2000), reprint p. 16. Original: "long-term capital investments are somewhat irreversible"; "The model has three periods (T = 0, 1, 2) and a single homogeneous good. The productive technology yields R > 1 units of output in period 2 for each unit of input in period 0. If production is interrupted in period 1, the salvage value is just the initial investment."; "Each faces a privately observed, uninsurable risk of being of type 1 or of type 2. In period 1, each agent's type is determined and revealed privately to the agent. Type 1 agents care only about consumption in period 1, and type 2 agents care only about consumption in period 2."

13. Diamond & Dybvig (1983/2000), reprint p. 17. Emphasis added. Original: "In our model, the demand deposit contract gives each agent withdrawing in period 1 a fixed claim of r_1 per unit deposited in period 0. Withdrawal tenders are served sequentially in random order until the bank runs out of assets."; the sequential service constraint: "a bank's payoff to any agent can depend only on the agent's place in line and not on future information about agents later in line." (p. 18)

14. Diamond & Dybvig (1983/2000), reprint p. 18, equations (6) and (7). Original: "$V_1(f_j, r_1) = r_1$ if $f_j < r_1^{-1}$; 0 if $f_j \geq r_1^{-1}$" and "$V_2(f, r_1) =$ max $\{(1 - r_1 f)R / (1 - f), 0\}$ where f_j is the quantity of withdrawers' deposits serviced before agent j and f is the total quantity of demand deposits withdrawn, both as fractions of total demand deposits."

15. Diamond & Dybvig (1983/2000), reprint p. 18. Original of the second passage: "For all $r_1 > 1$, runs are an equilibrium. ... but if $r_1 = 1$, the bank simply mimics direct holding of the assets, and the bank is therefore no improvement on simple competitive-claims markets. A demand deposit contract which is not subject to runs provides no liquidity services."

16. Diamond & Dybvig (1983/2000), reprint p. 18. Emphasis added.

17. Diamond & Dybvig (1983/2000), reprint p. 19. Original of the second passage: "Given this contract, no type 2 agent will withdraw at T = 1 because no matter what the agent anticipates about others' withdrawals, the agent receives higher proceeds by waiting until T = 2 to withdraw ... Therefore, there is a unique Nash equilibrium which has f = t. In fact, this is a dominant strategy equilibrium ...". Proposition 1 (p. 20): "Bank contracts (which must obey the sequential service constraint) cannot achieve optimal risk-sharing when t is stochastic and has a nondegenerate distribution."

18. Diamond & Dybvig (1983/2000), reprint p. 21, Proposition 2. Original of the third passage: "Furthermore, deposit insurance prevents runs because, for all possible anticipated withdrawal policies of other agents, participating in a bank run never pays. As a result, no strategic issues of confidence arise."

19. Diamond & Dybvig (1983/2000), reprint p. 22. Original: "Through its discount window, the Federal Reserve can, as a lender of last resort, provide a service similar to deposit insurance. ... The identity of deposit insurance and discount window services occurs because the technology is riskless."

20. Gorton & Zhang (2023), p. 913, Table 1 "Options to Address Stablecoins". Columns: Eliminate Runs? / Achieve NQA?; rows: Status quo (No/No), Implement bank-like regulations on stablecoin issuers, but no insurance (No/No), Issue stablecoins from within the insured-bank regulatory perimeter (Yes/Yes), Require stablecoins to be backed one-for-one by Treasuries or central bank reserves (Yes/Yes), Replace stablecoins with a central bank digital currency (Yes/Yes). The authors write that each of the last three also has drawbacks (p. 914, such as the problem of under-issuance under one-for-one Treasury backing).

21. Chuan Du, Ria Sonawane & Cy Watsky, "In the Shadow of Bank Runs: Lessons from the Silicon Valley Bank Failure and Its Impact on Stablecoins," *FEDS Notes*, Board of Governors of the Federal Reserve System, 2025-12-17, §1.2. https://www.federalreserve.gov/econres/notes/feds-notes/in-the-shadow-of-bank-run-lessons-from-the-silicon-valley-bank-failure-and-its-impact-on-stablecoins-20251217.html (HTML, reopened on 2026-10-25 and checked word for word; correction history dated 2026-01-08).

22. BIS (2022), p. 82, Box A. Original: "As UST dropped below its peg, a classic run dynamic took hold as investors sought to redeem their funds." The account of what followed is quoted in Chapter 26.

23. Anadu et al. (2023/2024), pp. 2–3. Original: "we isolate stress events in crypto markets associated with large declines in the price of Bitcoin and study the response of stablecoin investors to such events. We find that, during these episodes, investors run from riskier stablecoins to safer ones ..."; "while a stablecoin priced $1.00 or above experiences flows that are uncorrelated with prices, once its price drops below the $1 threshold, its daily outflow rate jumps by 34 basis points. This suggests that investors consider a stablecoin to have depegged when its price drops below $1, similar to the 95-cent 'break-the-buck' threshold for MMFs."; "The most significant episode so far occurred when Terra, an algorithmic stablecoin and the fourth-largest stablecoin at the time, suffered a run and subsequently collapsed in early May 2022. Following a classic flight-to-safety dynamic, this run had negative spillover effects on other algorithmic stablecoins and stablecoins backed by risky assets, whereas those backed by relatively safer assets experienced net inflows."; "Unlike the May 2022 Terra run, which occurred among the riskier stablecoins, the March 2023 run had at its epicenter USDC, which had previously been considered among the safest stablecoins due to its portfolio of Treasury securities and bank deposits. The run spilled over into Dai and Frax, both of which were partially collateralized by USDC."; abstract: "as flight-to-safety flows occur within MMF families, stablecoin flows tend to happen within blockchains."

24. Du, Sonawane & Watsky (2025), §§1–1.2. Original: "At 10 p.m. on Friday evening, Circle announced that it was unable to withdraw $3.3 billion of USDC reserves from SVB (around 8% of total reserves at the time), sparking a wave of redemption requests for the stablecoin."; "At its trough, USDC traded at 86 cents to the dollar."; "USDC's price on the secondary market recovered sharply after the backstop announcement on Sunday and fully recovered once Circle began processing redemptions on Monday, March 13."; "Hourly trading volume shot up to nearly $2 billion on March 11"; "during the crisis, trading on DEXs rose dramatically and accounted for most of the activity."; contagion: "a crypto-collateralized stablecoin, Dai, was operating one-to-one exchange facilities — called Peg Stability Modules ('PSMs') — against USDC and other stablecoins. Over the weekend, these PSMs were rapidly drained of liquidity. Dai, and other stablecoins with otherwise no exposure to SVB, also lost their peg to the dollar."; scope: "Policy discussions relating to the design of stablecoin regulations are outside of the scope of our study. Future research should explore relevant counterfactuals". The "86 cents" in the table is **a price figure** the original gives as the low (the low during the

weekend of 2023-03-11; footnote 9 of the original notes that on an hourly opening-price chart it appears as about 90 cents). It is set down once, with its source and date, under the exception in the notes to the reader in Chapter 3 (past data used to explain a method by the scale of an event).

25. Kenechukwu Anadu, Pablo D. Azar, Sean Baker, Marco Cipriani, Thomas M. Eisenbach, Mattia Landoni, Gabriele La Spada, Marco Macchiavelli & J. Christina Wang, "Stablecoins and (Non)Crypto Shocks: A 2026 Update," *Liberty Street Economics*, 2026-07-31. Original: "On March 11, 2023, Circle reported that approximately 8 percent of USDC's reserves were held at SVB"; "Following SVB's failure, the CRF's WAM fell below that of the median TO fund and has remained below the 5th percentile of the WAM distribution of TO MMFs."; "the CRF's holdings of repurchase agreements (repos) spiked from zero to over 90 percent of its net assets. This share has since retreated some but still stands at 69 percent"; "USDC's bank deposits rose briefly after SVB's failure but have since remained below the levels that prevailed before SVB's failure. ... after SVB's collapse, USDC's bank deposits became, and have remained, concentrated in GSIBs."; "Overall, the SVB event triggered a change in the type of risk held by one of the largest stablecoin issuers, away from interest-rate risk and toward counterparty risk." https://libertystreeteconomics.newyorkfed.org/2026/07/stablecoins-and-noncrypto-shocks-a-2026-update/ (HTML, accessed and checked word for word on 2026-10-25). Both posts are used (the 2026-06-23 post for the synthetic type, the 2026-07-31 post for reserve composition).

26. Rashad Ahmed & Iñaki Aldasoro, "Stablecoins and safe asset prices," BIS Working Papers No 1270 (2025-05, revised 2026-06), abstract. Original: "This paper examines the impact of dollar-backed stablecoin flows on short-term US Treasury yields using daily data from January 2021 to March 2026. Using local projections and a granular instrumental variable exploiting idiosyncratic variation in stablecoin growth at the token-blockchain pair level, we find a $3.5 billion (2-standard deviation) inflow lowers 3-month Treasury bill yields by 0.71 basis points on impact, and up to 4 basis points within 10 days, with limited spillovers to longer tenors. Effects are state-dependent, increasing under Treasury market stress and with the scale of the stablecoin sector." (2026-06 revision). The uncertainty is presented through the impulse-response figures (confidence intervals) in the body and not as figures in a table, so this book takes only the point estimates in the abstract and states the identification strategy with them. The lower-bound sentence is from the conclusion.

27. Du, Sonawane & Watsky (2025), §1.1. Original: "Solvency concerns around Circle are likely to have played a part."; the two stages of recovery: "USDC's price on the secondary market recovered sharply after the backstop announcement on Sunday and fully recovered once Circle began processing redemptions on Monday, March 13."; redemption pressure: "Pent up redemption pressure"; the issuer's public statement: "Public reassurances from Circle that it would 'stand behind USDC and cover any shortfall using corporate resources, involving external capital if necessary,' may have helped to shore up the price of USDC during the weekend, but were not sufficient to restore the peg."

28. Du, Sonawane & Watsky (2025), Summary. Original: "But had the run unfolded differently, the pressure for USDC redemptions might have forced Circle to liquidate backing assets (for example, U.S.Treasury securities), with potential knock-on effects on traditional financial markets."

29. Gorton & Zhang (2023), p. 920. Original: "In order for the stablecoins to be considered a deposit—let alone a demand deposit—one must determine whether the underlying contract between the holder of stablecoins and the stablecoin issuer is an equity contract or a debt contract. The distinction between equity contracts and debt contracts arose during the 1970s, as critics of money market funds alleged that their business practice of redeeming shares for cash was essentially deposit taking and therefore a violation of the Glass-Steagall Act."

PART 3 · ECONOMICS

Chapter 25. Econometric tests for bubbles: what they measure and what they cannot

KEY TAKEAWAY

The sentence "Bitcoin is a bubble" turns up often, in court and in papers, but before it can become **a testable proposition** two things have to be settled. What is the bubble a difference from (the fundamental value)? And what statistical shape does that difference take (explosive growth)? The standard tools econometrics has produced, the right-tailed unit root tests of PWY and PSY (SADF and GSADF), answer only the second question. They measure whether the price grew **faster than a unit root for a while**, and they put dates on the start and end of that interval. The first question is settled not by a test but by **an assumption**. That is why studies of the price of the same asset split, into a bubble that makes up 48.7% of the price under a stochastic bubble model (Cheah and Fry), rejection at the 1% level for Bitcoin alone under the PWY test (Hafner), and no explosive departure from the fundamental value (the marginal cost of mining) (Podhorsky). (The exchanges and periods also differ.) When volatility changes over time, textbook critical values cannot be trusted. Where variance rises they reject too often, and for some crypto-assets the error runs the other way (Hafner). And depending on how the fundamental value is set, the same price both is and is not a bubble (Podhorsky). This chapter uses no actual price data. It runs the tests on synthetic data with a planted explosive interval and sees, knowing the right answer, **what the tests catch and what they miss**. The conclusion of this chapter is the list of things to ask when a "bubble finding" is offered as evidence: the sample, the period, the fundamental-value assumption, the critical values, and the treatment of volatility.

Learning objectives

1. Know the definition of a rational bubble (price minus fundamental value) and the two assumptions it demands of a test. Distinguish what the word "bubble" can and cannot mean in a legal judgment.
2. [Intermediate] Read the ADF regression and the explosive coefficient ($\beta > 0$, that is, $\rho > 1$), and know what a right-tailed unit root test takes as its null hypothesis. Compute SADF and GSADF statistics on synthetic data and compare them with PSY's critical values.
3. [Advanced] Read the double recursion of the GSADF statistic, the minimum-window rule, and date-stamping (BSADF), and confirm through size distortion under time-varying volatility why Monte Carlo critical values and wild bootstrap critical values differ.

4. Read the bubble findings of 6 studies in the literature (1 retracted study listed separately) in a table, together with sample, period, fundamental-value assumption, and critical values, and point to where the conclusions part. Know how to handle a retracted paper in a line of citations.

What a reader of the [Basic] paragraphs alone takes away. A "bubble finding" is a statistic saying that the price **grew faster than a unit root for a while**, and what the fundamental value is, is an assumption, not a test.

1. The question this chapter tries to answer

Chapter 19 used models to see why the price of an asset that promises nothing can be other than 0, and those models said that the path of the price is not one but many: there are multiple equilibria. Some of those paths are called "bubbles." Blanchard and Watson made the definition clear in 1982: "Rationality of both behavior and of expectations often does not imply that the price of an asset be equal to its fundamental value. In other words, there can be **rational** deviations of the price from this value, rational bubbles." And they attached the second question right away, considering "whether the presence of bubbles in a particular market can be detected statistically. The task is much easier if there are data on both prices and returns. ... In markets in which returns are difficult to observe, possibly because of a nonpecuniary component, such as gold, the task is more difficult. We consider the use of both 'runs tests' and 'tail tests' and conclude that they give **circumstantial evidence** at best."[1] Bitcoin is an asset without returns (dividends), so it sits on Blanchard and Watson's "more difficult" side. This chapter looks at what econometrics, 40 years later, has made it possible to do on that difficult side, and at what still has to be assumed.

For lawyers this question matters for one reason. A finding that "it was a bubble" can be submitted **as evidence** in cases of fraud, market manipulation, or breach of the duty to explain. Without knowing what that finding measured and what it assumed, a court receives rhetoric, not statistics, as evidence.

2. Two assumptions the definition of a bubble requires

In 1985 Tirole defined a bubble as "the difference between the market price and the market fundamental" and wrote: "Since Samuelson [30] developed his consumption loan model it has been well known that there exist economies in which money has a positive value in spite of the fact that it is intrinsically useless (i.e., its market fundamental is zero). In other words there can exist a bubble on money ...."[2] The two assumptions of the test follow from this definition.

ASSUMPTION	CONTENT	THE PROBLEM FOR BITCOIN
Fundamental-value assumption	A bubble is a difference, so there must be something to subtract (the fundamental value)	It pays no dividends. Each study subtracts something different: estimated within the model and found indistinguishable from 0 (Cheah and Fry), the marginal cost of mining (Podhorsky), a power function of the number of active users (Wheatley et al.), or nothing at all, looking at the price itself (most of the PSY line of work)
Statistical-shape assumption	Within a bubble interval the price grows **faster than a unit root** (explosively)	If the fundamental value is itself explosive (like mining difficulty), explosiveness in the price is not evidence of a bubble

Why the second assumption holds follows directly from the formula in Chapter 19. In Blanchard and Watson's equation (1) the bubble term must satisfy $\mathbb{E}(c_{t+1} \mid \Omega_t) = (1+r)\, c_t$: its expected value must grow by a factor of $(1+r)$ each period for holders to have a reason to keep it. A unit root process keeps its expected value where it is ($\mathbb{E}(y_{t+1}) = y_t$); a bubble's expected value **grows**. So a bubble test asks not "is the coefficient 1?" but "is the coefficient **greater than** 1?" It becomes a **right-tailed** unit root test. In PSY's words, "Right-sided unit root tests ... are informative about mildly explosive or submartingale behavior in the data and are therefore useful as a form of market diagnostic or warning alert."[3]

3. [Intermediate] The ADF regression and the explosive coefficient

[Intermediate] The starting point is the ordinary unit root regression. PSY's equation (4) estimates, over a window running from the r_1 fraction to the r_2 fraction of the sample,

$$\Delta y_t = \hat{\alpha}_{r_1,r_2} + \hat{\beta}_{r_1,r_2}\, y_{t-1} + \sum_{i=1}^{k} \hat{\psi}^{i}_{r_1,r_2}\, \Delta y_{t-i} + \hat{\varepsilon}_t$$

and takes the window's ADF statistic $ADF_{r_1}^{r_2}$ to be the t ratio of β.[4] Reading the symbols: y_t is the (log) price, $\Delta y_t = y_t - y_{t-1}$, k is the number of lags for short-run dynamics, and $r_w = r_2 - r_1$ is the width of the window (as a fraction of the sample). The null hypothesis is $\beta = 0$ (a unit root, with a weak drift allowed), and the alternative is $\beta > 0$ (explosive). Written in levels, $y_t = \alpha + \rho\, y_{t-1} + \cdots$ with $\rho = 1 + \beta$, and the t value is the same as for a test of $\rho = 1$ against $\rho > 1$. This is the opposite of the ordinary ADF test, which looks to the **left** ($\beta < 0$, stationarity). A numerical example: if an interval with $\rho = 1.04$ lasts 30 periods, the expected value grows $1.04^{30} \approx 3.24$ times. That is the size of "mild explosiveness."

[Intermediate] The PWY (2011) test **fixes the start of the window at the beginning of the sample and extends the end**, computing the ADF statistic each time and taking the maximum.

$$SADF(r_0) = \sup_{r_2 \in [r_0, 1]} ADF_0^{r_2}$$

r_0 is the width of the first window.[5] The reason for expanding the window is simple. A single regression on the whole sample buries a short explosive interval in a long unit-root stretch, but a window that ends where the explosive interval ends catches it. One problem remains. Because the start of the window is fixed at the beginning of the sample, **a second bubble, after the first has collapsed,** is hard to catch because of the collapse that came before it. A

sample that contains a collapse shows "pseudo stationary behavior." Solving that problem was the contribution of PSY 2015, and it is the subject of the next section.

[Intermediate] The three cells below are **a demonstration of method**: they run the tests on synthetic data with a planted explosive interval and see, knowing the right answer, what the tests catch and what they miss. No actual market price data are used.

```
# NB25-1: PWY/PSY tests on synthetic data — computes SADF and GSADF on a fake price series with
a planted explosive interval (a demonstration of method; no actual market price data are used)
import numpy as np

rng = np.random.default_rng(25)      # fixed seed — the same results are reproduced

def make_series(T=200, bubble=(100, 130), beta=1.04, sigma=1.0, collapse_to=None):
    """A driftless random walk in which only the interval [bubble[0], bubble[1]) grows as y_t =
beta·y_{t-1} + e_t; when the interval ends, the series drops back to near its level before the
interval."""
    y = np.empty(T); y[0] = 30.0
    e = rng.normal(0.0, sigma, T)
    for t in range(1, T):
        if bubble[0] <= t < bubble[1]:
            y[t] = beta * y[t - 1] + e[t]
        elif t == bubble[1]:
            y[t] = (collapse_to if collapse_to is not None else y[bubble[0] - 1]) + e[t]
        else:
            y[t] = y[t - 1] + e[t]
    return y

def adf_grid(y, min_win):
    """Computes at once the lag-0 ADF t statistic for every window [s, e] (Δy_t regressed on a
constant and y_{t-1}; the t ratio of the y_{t-1} coefficient).
    Returns: a (T-1)×(T-1) array A, where A[s, e] is the t of the regression whose window uses
Δy for observations s+1 … e+1. Windows that are too short are NaN."""
    x = y[:-1]; z = np.diff(y)                    # x_t = y_{t-1}, z_t = Δy_t, t = 1 … T-1
    n = len(z)
    def ps(a):                                    # prefix sums with a leading 0 — any range sum
in one subtraction
        return np.concatenate(([0.0], np.cumsum(a)))
    Px, Pz, Pxx, Pxz, Pzz = ps(x), ps(z), ps(x * x), ps(x * z), ps(z * z)
    s = np.arange(n)[:, None]; e = np.arange(n)[None, :]
    cnt = (e - s + 1).astype(float)
    Sx = Px[e + 1] - Px[s]; Sz = Pz[e + 1] - Pz[s]
    Sxx = Pxx[e + 1] - Pxx[s]; Sxz = Pxz[e + 1] - Pxz[s]; Szz = Pzz[e + 1] - Pzz[s]
    with np.errstate(invalid="ignore", divide="ignore"):
        den = cnt * Sxx - Sx * Sx
        beta = (cnt * Sxz - Sx * Sz) / den
        alpha = (Sz - beta * Sx) / cnt
        rss = Szz - alpha * Sz - beta * Sxz
        s2 = rss / (cnt - 2)
        t = beta / np.sqrt(s2 * cnt / den)
    t[(cnt < min_win) | (e < s)] = np.nan
    return t

def sadf_gsadf(y, r0):
    T = len(y); w = int(np.floor(r0 * T))
    A = adf_grid(y, w)
    sadf = np.nanmax(A[0, :])                     # start fixed (0), end extended
    bsadf = np.nanmax(A, axis=0)                  # for each end r2, the maximum over all
starts — for date-stamping
    gsadf = np.nanmax(bsadf)                      # the maximum over all (r1, r2)
    return sadf, gsadf, bsadf, A

T = 200
r0 = 0.01 + 1.8 / np.sqrt(T)                      # PSY's minimum-window rule → 0.137 (T = 200)
y = make_series(T)
sadf, gsadf, bsadf, A = sadf_gsadf(y, r0)
PSY_T200 = {"SADF 95%": 1.40, "SADF 99%": 1.90, "GSADF 95%": 2.08, "GSADF 99%": 2.70}   # PSY
(2015) Table 1, finite sample, T=200, r0=0.137
print(f"Synthetic data: T = {T}, explosive interval t ∈ [100, 130), β = 1.04, then a collapse.
Minimum window r0 = {r0:.3f} → {int(np.floor(r0*T))} observations")
print(f"SADF  = {sadf:.3f}   (PSY Table 1 critical values 95% {PSY_T200['SADF 95%']:.2f}, 99%
{PSY_T200['SADF 99%']:.2f})")
print(f"GSADF = {gsadf:.3f}   (PSY Table 1 critical values 95% {PSY_T200['GSADF 95%']:.2f}, 99%
{PSY_T200['GSADF 99%']:.2f})")
print(f"Single ADF on the full sample (start 0, end T) = {A[0, -1]:.3f}  ← the explosive
interval is buried in the long unit-root stretch and does not show")
peak = int(np.nanargmax(bsadf)) + 1
print(f"Endpoint where BSADF is largest: t = {peak} (just before 130, the end of the planted
```

```
interval, means the test caught it well)")
```

```
# Output — bcbook (Python 3.13), exactly as run
Synthetic data: T = 200, explosive interval t ∈ [100, 130), β = 1.04, then a collapse. Minimum
window r0 = 0.137 → 27 observations
SADF  = 11.918   (PSY Table 1 critical values 95% 1.40, 99% 1.90)
GSADF = 12.361   (PSY Table 1 critical values 95% 2.08, 99% 2.70)
Single ADF on the full sample (start 0, end T) = -2.466  ← the explosive interval is buried in
the long unit-root stretch and does not show
Endpoint where BSADF is largest: t = 129 (just before 130, the end of the planted interval,
means the test caught it well)
```

[Intermediate] The first cell shows three things. The ADF run once on the whole sample does not see the explosiveness. The SADF, which expands the window, and the GSADF, which sweeps every window, exceed PSY's finite-sample critical values (at T = 200 and $r_0 = 0.137$, GSADF 95% 2.08 and 99% 2.70).[6] And the endpoint where the BSADF is largest lies near the end of the planted interval. All of this was seen **knowing the right answer**. Real data come with no right answer, and that is why the next two sections are needed.

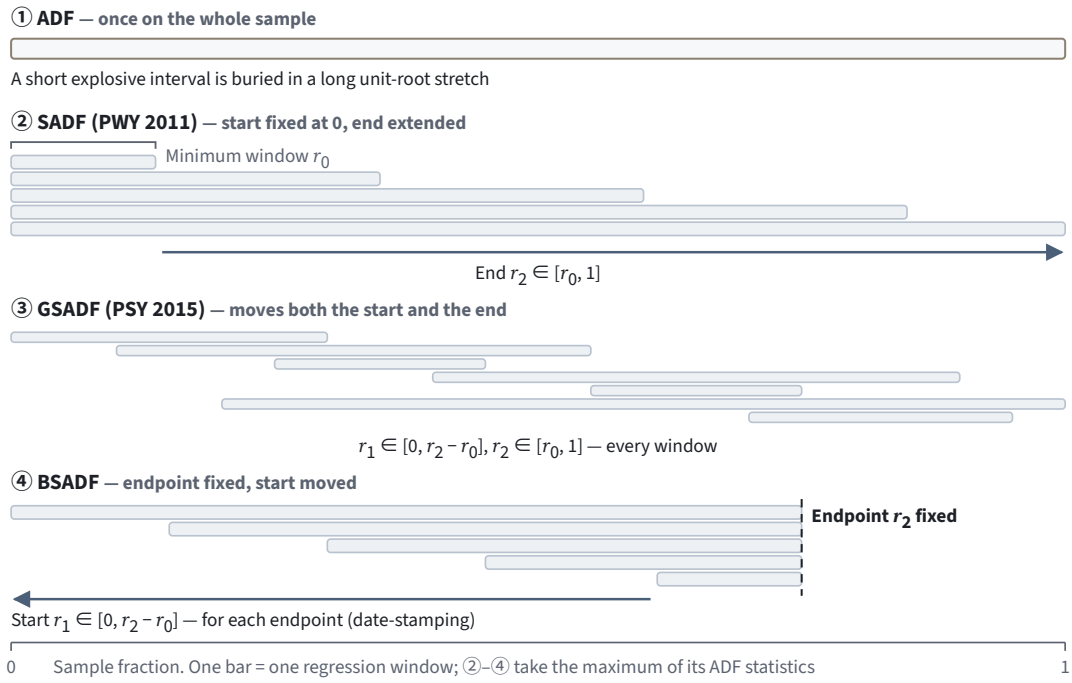


Figure 25-1. How the four statistics place their regression windows. The horizontal axis is the sample fraction, and each bar is one regression window. The positions of the bars are illustrative, to show the arrangement.

4. [Advanced] GSADF, the minimum window, date-stamping, and critical values

▾ [Advanced] Double recursion and date-stamping

[Advanced] PSY's statistic moves **both** the start and the end of the window.

$$GSADF(r_0) = \sup_{\substack{r_2 \in [r_0, 1] \\ r_1 \in [0,\, r_2 - r_0]}} ADF_{r_1}^{r_2}$$

Theorem 1 gives the limit distribution of this statistic under the null hypothesis (a

random walk with asymptotically negligible drift) as a functional of Wiener processes, and that distribution depends on the minimum window r_0.[7] That is why the tables of critical values are given for each r_0, and the smaller r_0, the larger the critical value: "when r_0 decreases from 0.190 to 0.055, the 95% asymptotic critical value of the GSADF statistic rises from 1.89 to 2.30." The more windows one sweeps, the more chances there are for a large value to turn up by accident. For the same reason the critical values of GSADF are larger than those of SADF ("when $r_0 = 0.10$, the 95% asymptotic critical value of the GSADF statistic is 2.19 whereas that of the SADF statistic is 1.49").[8]

[Advanced] **Date-stamping.** Asking in real time "does this observation belong to a bubble interval?" is the regulator's question, and PSY answer it with **the backward SADF**. Fix the endpoint r_2, move the start from 0 to $r_2 - r_0$, and take the maximum of the ADF statistics: that is $BSADF_{r_2}(r_0)$, and PWY's recursive ADF (start fixed) is a special case of it.[9] The origination date is the first observation at which $BSADF_{r_2}$ exceeds the critical value, and the termination date is the first observation, once L_T more have passed, at which it falls below the critical value. A minimum-duration condition such as $L_T = \log(T)$ is there "to exclude short lived blips in the fitted autoregressive coefficient," and it is adjusted to the frequency of the data.[10] The second cell builds a sequence of critical values by Monte Carlo and stamps the dates.

```
# NB25-2: Monte Carlo critical values and date-stamping — generates many null (random walk)
series to get the 95% and 99% quantiles of GSADF and the sequence of BSADF critical values,
then stamps dates on the synthetic data (a demonstration of method)
def mc_critical_values(T, r0, reps=2000, seed=2025):
    g = np.random.default_rng(seed)
    w = int(np.floor(r0 * T))
    gs = np.empty(reps); bs = np.empty((reps, T - 1))
    for i in range(reps):
        yn = 30.0 + np.cumsum(g.normal(0.0, 1.0, T))           # null: driftless random walk
        A = adf_grid(yn, w)
        bs[i] = np.nanmax(A, axis=0); gs[i] = np.nanmax(bs[i])
    return gs, bs

gs_null, bs_null = mc_critical_values(T, r0)
cv95, cv99 = np.quantile(gs_null, [0.95, 0.99])
print(f"Null random walks, {len(gs_null)} replications → GSADF critical values 95% {cv95:.2f},
99% {cv99:.2f}   (PSY Table 1: 2.08, 2.70 — this cell: 2,000 replications, lag 0, no drift;
PSY: weak drift d=η=1)")
print(f"GSADF of the synthetic data {gsadf:.3f} → at 95%: {'reject (explosive)' if gsadf >
cv95 else 'fail to reject'}, at 99%: {'reject' if gsadf > cv99 else 'fail to reject'}")

# date-stamping: for each endpoint t, use the 95% quantile of the null BSADF_t as the critical
value (PSY use the SADF critical value based on ⌊T r2⌋ observations; here a sequence taken
directly from the same Monte Carlo stands in for it)
bs_cv95 = np.nanquantile(bs_null, 0.95, axis=0)
LT = int(np.ceil(np.log(T)))                       # minimum duration log(T) ≈ 5.3 → 6
observations
flag = bsadf > bs_cv95
episodes, start = [], None
for t, f in enumerate(flag):
    if f and start is None: start = t
    if (not f or t == len(flag) - 1) and start is not None:
        end = t if not f else t + 1
        if end - start >= LT: episodes.append((start + 1, end + 1))   # +1: Δy index → time in
the original series
        start = None
print(f"Explosive intervals stamped with the critical-value sequence (endpoint-wise BSADF 95%;
minimum duration {LT} observations): {episodes}")
print(f"Planted interval: [100, 130). The start is usually caught a few observations late and
the end just after the collapse — the test sees 'growing now', not 'has started to grow'.")
print("An interval caught outside the planted one is a false positive — allow 5% at every
endpoint and somewhere among the 170-odd endpoints one crosses by chance (the multiplicity
issue Oldani et al. describe).")
```

```
# Output — bcbook (Python 3.13), exactly as run
Null random walks, 2000 replications → GSADF critical values 95% 2.11, 99% 2.65   (PSY Table
1: 2.08, 2.70 — this cell: 2,000 replications, lag 0, no drift; PSY: weak drift d=η=1)
GSADF of the synthetic data 12.361 → at 95%: reject (explosive), at 99%: reject
Explosive intervals stamped with the critical-value sequence (endpoint-wise BSADF 95%; minimum
duration 6 observations): [(49, 63), (102, 130)]
Planted interval: [100, 130). The start is usually caught a few observations late and the end
just after the collapse — the test sees 'growing now', not 'has started to grow'.
An interval caught outside the planted one is a false positive — allow 5% at every endpoint
and somewhere among the 170-odd endpoints one crosses by chance (the multiplicity issue Oldani
et al. describe).
```

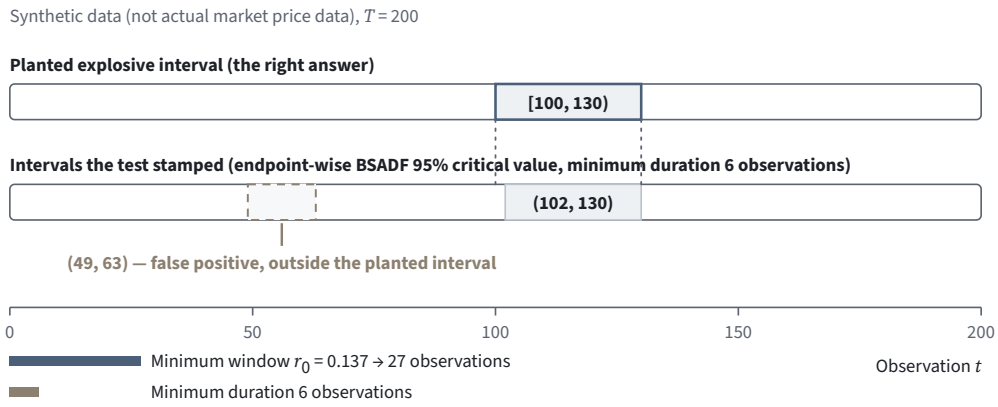


Figure 25-2. Synthetic data (not actual market price data): the planted explosive interval and the intervals the test stamped. The intervals and observation counts are the output of NB25-1 and NB25-2, and no price curve is drawn.

[Advanced] When volatility changes: the wild bootstrap

[Advanced] Textbook critical values come from the assumption that the variance of the errors is constant. Crypto-asset prices do not behave that way. Hafner takes this up directly in his 2020 paper: "As shown by Harvey et al. (2016), the original test of PWY is biased in the presence of non-stationary volatility." Harvey et al. "suggest a wild bootstrap procedure to correct the size of the PWY test and show that it has good power properties," and Hafner extends the approach to stochastic volatility and applies it to 11 crypto-assets and the CRIX index. The result **does not run in one direction**. "Taking into account global heteroskedasticity has the effect of increasing the p-values that were found to be very close to zero ... In other words, the evidence in favor of speculative bubbles is much less pronounced than when this effect is ignored." But "not all p-values increase, in particular the larger ones tend to decrease under the correction." For Bitcoin, with a PWY statistic of 5.0347, the textbook p-value of 0.0001 rises to a wild bootstrap p-value of **0.0032** (still a rejection at 1%), and "at the 1% level one would reject the null hypothesis in favor of a speculative bubble only for the Bitcoin," while at 5% the number grows to 9 of 12 series (11 crypto-assets + CRIX), or 8 of 11 counting the crypto-assets alone.[11] In 2025 Oldani and coauthors wrote that PSY's critical values "are not robust to two potential inference issues: 1) time-varying unconditional heteroscedasticity in the ADF errors and 2) the multiplicity issue (the probability of a false positive increases with the number of hypotheses tested)," and built critical values with the composite wild bootstrap of Phillips and Shi (2020).[12] The third cell shows this size distortion on synthetic data.

```
# NB25-3: size distortion under time-varying volatility, and the wild bootstrap — on null data
with no bubble, where only the variance rises, how often do textbook critical values say
"bubble"? (a demonstration of method)
def null_with_vol_break(T, g, s1=1.0, s2=3.0, brk=None):
    brk = T // 2 if brk is None else brk
    sig = np.where(np.arange(T) < brk, s1, s2)
    return 30.0 + np.cumsum(g.normal(0.0, 1.0, T) * sig)

g = np.random.default_rng(2526); reps = 400; w = int(np.floor(r0 * T))
rej_const = rej_break = 0
for i in range(reps):
    y0 = 30.0 + np.cumsum(g.normal(0.0, 1.0, T))                        # constant variance
    y1 = null_with_vol_break(T, g)                                       # variance 9x
(standard deviation 3x) from the middle
    rej_const += np.nanmax(adf_grid(y0, w)) > cv95
    rej_break += np.nanmax(adf_grid(y1, w)) > cv95
print(f"{reps} null series of each kind, Monte Carlo 95% critical value {cv95:.2f} — nominal
size 5%")
print(f"  constant variance:            rejection rate {rej_const / reps:.1%}")
print(f"  standard deviation 3x midway: rejection rate {rej_break / reps:.1%}   ← with no
bubble at all, it says 'bubble' this often")

# wild bootstrap (the skeleton of the Hafner and Phillips-Shi approach): multiply the
residuals of the observed data by ±1 or normal weights to rebuild null samples, and take the
critical value from them — the change of variance over time is preserved as it is
def wild_bootstrap_cv(y, w, B=199, seed=7):
    gb = np.random.default_rng(seed)
    resid = np.diff(y) - np.mean(np.diff(y))                             # residuals under the
null (unit root): Δy minus its mean
    stats = np.empty(B)
    for b in range(B):
        eb = resid * gb.normal(0.0, 1.0, len(resid))                     # wild: multiply each
residual by an independent N(0,1)
        yb = y[0] + np.concatenate(([0.0], np.cumsum(eb)))
        stats[b] = np.nanmax(adf_grid(yb, w))
    return np.quantile(stats, 0.95)

# pick one null series that textbook critical values would call a "bubble" (trying seeds from
0 in order — reproducible)
for sd in range(200):
    y_break = null_with_vol_break(T, np.random.default_rng(sd))
    stat = np.nanmax(adf_grid(y_break, w))
    if stat > cv95: break
cv_wb = wild_bootstrap_cv(y_break, w)
print(f"\nOne null series with changing variance (seed {sd}): GSADF {stat:.2f}. Textbook
critical value {cv95:.2f} → {'reject' if stat > cv95 else 'fail to reject'}; wild bootstrap
95% critical value {cv_wb:.2f} → {'reject' if stat > cv_wb else 'fail to reject'}")
print("The bootstrap critical value moves with the volatility structure of the data. The same
statistic can be 'bubble' under one critical value and 'not a bubble' under another.")
```

```
# Output — bcbook (Python 3.13), exactly as run
400 null series of each kind, Monte Carlo 95% critical value 2.11 — nominal size 5%
  constant variance:            rejection rate 4.8%
  standard deviation 3x midway: rejection rate 34.0%   ← with no bubble at all, it says
'bubble' this often

One null series with changing variance (seed 0): GSADF 2.32. Textbook critical value 2.11 →
reject; wild bootstrap 95% critical value 3.45 → fail to reject
The bootstrap critical value moves with the volatility structure of the data. The same
statistic can be 'bubble' under one critical value and 'not a bubble' under another.
```

5. What to take as the fundamental value: three answers

So far we have been testing the second assumption (the explosive shape). The first assumption (what to subtract) is a choice, not a test, and the literature has made three choices.

Estimated, and indistinguishable from 0. The 2015 paper by Eng-Tuck Cheah and John Fry estimates the collapse-size parameter v of a stochastic bubble model and tests the null hypothesis "$v = 0$". On data from January 1 to November 30, 2013, it found $v = 0.546$ (standard error 0.090, $t = 6.06$) and a growth rate of the long-run fundamental value of $\tilde{\mu} = 0.00166$ (standard error 0.00195, $p = 0.394$), and the authors wrote that "during the bubble price rises are so dramatic that the estimated long-term fundamental value is not statistically different from zero" and that "Equation (16) suggests that the bubble accounts for around 48.7% of observed prices." The third point of their conclusion is "the fundamental value of Bitcoin is zero."[13] This conclusion is a proposition inside the model. Getting from the estimate that the **growth rate** of the fundamental value is indistinguishable from 0 to the conclusion that its **level** is 0 requires the model's equations (12) and (17), and those equations are assumptions.

The marginal cost of mining. Andrea Podhorsky's 2024 paper derives the fundamental value from the world of Chapter 20. Because the protocol manages supply by adjusting difficulty, "the marginal cost of mining the target supply of bitcoins" is the fundamental value the model implies, and that fundamental value can itself be **explosive**. In the abstract's words, "the bitcoin's volatile and explosive price path is a consequence of the Bitcoin protocol's system of supply management." The empirical work has three steps. Both the price and the fundamental value have unit roots (ADF −2.51, $p = .36$; −1.90, $p = .62$); the two are cointegrated (Engle–Granger −2.74, $p = .071$, at the 10% level); and when the PSY tests are run **on the price**, SADF 3.7 and GSADF 3.9 exceed the 1% critical values (2.2 and 2.9), but when they are run **on the residuals**, the price minus the fundamental value, the values −.08 and 1.52 fall short of even the 10% critical values (1.3 and 2.2). "Since the raw price data indicates the presence of bubbles while the residuals $\hat{v}t$ do not, ... we can attribute the apparent bubbles in the price of the bitcoin to the fundamental value."[14] Same price, different fundamental value, opposite conclusion.

Network size. Spencer Wheatley and coauthors (Sornette among them) in 2019 took a "generalized Metcalfe's Law" (market capitalization as a power function of the number of active users) as the fundamental value, identified four intervals of large departure from it as bubbles, and then tried to predict the window for the time of the collapse with the LPPLS (log-periodic power law singularity) model. The estimated exponent is not 2 but about 1.69 ("a more sparsely connected network in which each user is on average linked to $\approx N^{2/3}$ other users"), and the four bubbles are May–August 2012 (84 days), January–April 2013 (98 days), October–November 2013 (47 days), and June 2015 to December 2017 (924 days). (The market capitalization and growth-factor columns of their table are not carried over.)[15] The authors themselves set out the model uncertainty at length: changing the fitting window produces "4 orders of magnitude difference between the predicted carrying capacities," and "precise inference based on a single model ... is misleading, and long-term predictions effectively meaningless."[16] This book does not carry over their market capitalization figures or their judgment of a "current overvaluation" (the notes to the reader in Chapter 3). What it keeps is the structure. If the fundamental value is set by the number of users, a bubble becomes "an interval in which the price is above what the number of users justifies," and that judgment hangs on a single exponent of a power function.

6. The literature's findings in one table: sample, period, assumptions, and critical values together

[Intermediate] Here the findings of the empirical studies read in this chapter are set out in one table. **The findings are not carried over on their own.** When the sample, period, fundamental-value assumption, and critical values differ, the same "rejection" is a different proposition. All the numbers are **test statistics on past data**, not prices, and this book has not carried over price levels, rates of return, or market capitalizations.

STUDY	DATA, SAMPLE, PERIOD	FUNDAMENTAL-VALUE ASSUMPTION	METHOD AND CRITICAL VALUES	FINDING	THIS BOOK'S NOTE
Cheah and Fry (2015)	CoinDesk index, daily, 2010-07-18 to 2014-07-17; test window 2013-01-01 to 11-30	0 (estimated within the model → indistinguishable from 0)	Stochastic bubble model, $H_0{:}v = 0$	$v = 0.546$ (SE 0.090); the bubble is about 48.7% of the price	"Fundamental value 0" is a conclusion that hangs on the model's equations (12) and (17)
Cheung, Roca and Su (2015)	Mt. Gox USD, daily, 2010-07-17 to 2014-02-18, N = 1,307	None (the log price itself)	PSY GSADF, $r_0 \approx 2\%$ (20 observations), 99% critical value	99% critical value exceeded 33 times; three big bubbles: 2011-04-24 to 07-03 (66 days as reported in the paper; 71 days on the calendar counting both ends; cause of the difference not verified), 2013-01-27 to 04-15 (79 days), 2013-11-05 to 2014-02-18 (106 days)	Prices from a single exchange; the third is cut off at the end of the sample (the closure of the exchange)
Hafner (2020)	Top 11 crypto-assets as of 2017-12-31 + CRIX; BTC from 2013-04-28, 1,709 observations	None	PWY SADF, $r_0 = 0.1$; Monte Carlo p-values vs. wild bootstrap (5,000 replications)	BTC 5.0347: p 0.0001 → WB p 0.0032 (the only crypto-asset rejected at 1%); at 5%, 9 of 12 series (11 crypto-assets + CRIX)	With time-varying volatility the evidence becomes "much less pronounced", though larger p-values sometimes go down instead
Podhorsky (2024)	1,764 daily observations (period in §4 of the paper), minimum window 93	Marginal cost of mining the target supply (cointegrated with the price, EG −2.74, p .071)	PSY SADF and GSADF, 2,000 Monte Carlo replications	Price: 3.7 and 3.9 > 1% (2.2 and 2.9). Residuals: −.08 and 1.52 < 10% (1.3 and 2.2)	The same price becomes "not a bubble" when the fundamental value changes
Oldani, Bruno and Signorelli (2025)	BTC, ETH, XRP daily log prices, 2018-01-01 to 2024-06-14, T = 2,357, $\lfloor r_0 T \rfloor = 110$	None	PSY GSADF, Phillips–Shi composite wild bootstrap (B = 499, T_b = 183 days)	BTC 3.709, ETH 2.289, XRP 3.966, all above the 1% bootstrap critical values (2.086, 2.124, 2.104). Four BTC intervals (2018-11, 2019-05 to 06, 2020-12 to 2021-03, 2024-02 to 03)	XRP exceeds on the statistic, but there is only a "short-lived single bubble" (2020-11-23 to 25); with lag 7 the test "does not support exuberant behavior at all"; on weekly data it reaches only 10%; and the abstract says "No test supports the exuberance of XRP …." This gap among the text, the tables, and the abstract is a point for the reader to check directly

STUDY	DATA, SAMPLE, PERIOD	FUNDAMENTAL-VALUE ASSUMPTION	METHOD AND CRITICAL VALUES	FINDING	THIS BOOK'S NOTE
Wheatley et al. (2019)	Bitstamp USD, hourly, 2012-01-01 to 2018-01-08; number of active users	Generalized Metcalfe's law (exponent ≈ 1.69)	Departure from the fundamental value + LPPLS	Four bubbles (84, 98, 47, and 924 days; growth factors not carried over)	The fundamental value hangs on a single power-law exponent; the authors state the model uncertainty
Corbet, Lucey and Yarovaya (2018)	—	—	—	**Retracted** (retraction notice FRL 91: 109483, 2026-03)	This book does not rely on it. Reason for the retraction not verified

17 18 19

[Intermediate] What to take from the table is not the sum of the findings but **the places where they part**. For the same Bitcoin price, (a) if no fundamental value is subtracted, most studies find explosive intervals; (b) if the marginal cost of mining is subtracted, none are found; (c) if volatility is adjusted for, the evidence weakens, but Bitcoin remains; and (d) if the exchange, frequency, or period changes, the number and length of the intervals change. None of these four is an answer to "was it a bubble?" Each is an answer to "what did we decide to call a bubble?"

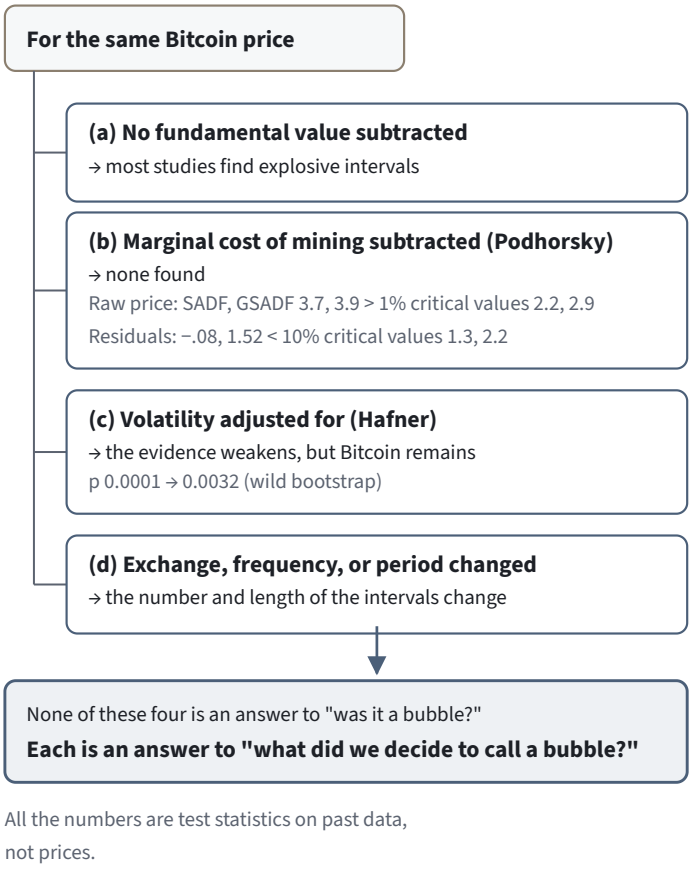


Figure 25-3. Where findings on the same Bitcoin price part when the assumptions about fundamental value, volatility, and data change. The exchanges and periods differ from study to study. Every number in the figure is a test statistic.

7. What the test says and what it does not

What a right-tailed unit root test **says**: in some interval, the autoregressive coefficient of the (log) price was significantly greater than 1. The start and end of that interval can be estimated as dates. Multiple intervals can be caught (PSY). If volatility changes, the critical values must be rebuilt from the data (Hafner; Phillips and Shi).

What it **does not say**: why it happened (a demand shock, a change in the fundamental value, or manipulation). Who did it (the test looks at a single price series). What comes after (PSY themselves describe the approach as "ex ante (or anticipative)," an "early warning alert system," but the content of that alert is "it is growing now," not "it is about to burst"). And **what the fundamental value is**: this is an assumption outside the test. The "circumstantial evidence at best" that Blanchard and Watson wrote in 1982 still holds, in that the structure of the question has not changed even as the tools have become more refined.

One more thing should be recorded. The models of Chapter 19 said that bubbles **can be rational**. Tirole gave the conditions under which bubbles exist, and Blanchard and Watson wrote that "phenomena such as runaway asset prices and market crashes are consistent with rational bubbles." If so, a statistical finding that "it was a bubble" does not in itself imply anyone's irrationality or deception. To imply that, further premises are needed, and those premises are a matter for fact-finding, not for statistics.

> **? A LAWYER'S QUESTION**
>
> An expert witness submits an opinion that "the price over the relevant period was explosive at the 1% level in a GSADF test and therefore was a bubble." After this chapter, the questions to ask are settled. What data (exchange, frequency, period)? What was taken as the fundamental value, or was none taken? Are the critical values from a textbook table or from a bootstrap built on the data? How was the change in volatility handled? What are the minimum window and the minimum-duration condition? And how is the finding, even if it is statistically correct, connected to **which act** of the defendant? Is the existence of an explosive interval evidence of market manipulation, evidence of fraudulent deception, or an observation that one of multiple equilibria was realized? Conversely, does a finding that "it was not a bubble" (cointegration with the marginal cost of mining, for instance) help the defendant? This book does not answer, and points to the issue section in Part 5. → Chapter 28

8. Summary

A bubble is the difference between price and fundamental value, and testing for it rests on two assumptions: what is subtracted, and whether the difference takes an explosive shape. Right-tailed unit root tests (PWY's SADF, PSY's GSADF) deal only with the second. As the synthetic data showed, tests that move the window catch a planted explosive interval and mostly stamp its dates, but under time-varying volatility textbook critical values reject too often, and bootstrap critical values move with the data. The literature has given three answers to the first assumption (0, the marginal cost of mining, network size), and when the answer changes, the same price both is and is not a bubble. So when you receive a "bubble finding," ask about the sample, period, fundamental value, critical values, and treatment of volatility together. Part 3 ends here. It started from "where does value come from?" in Chapter 19, passed through the security budget, fees, token design, game theory, and stablecoins, and has arrived at "can a bubble be measured?", and the common answer **was conditional**. Models enumerate possibilities, tests measure shapes, and neither prophesies the one path that reality will take. Chapter 26 reads four cases in which those conditions actually broke down.

Sources for this chapter

- Peter C. B. Phillips, Shuping Shi and Jun Yu, "Testing for Multiple Bubbles: Historical Episodes of Exuberance and Collapse in the S&P 500," *International Economic Review* 56(4): 1043–1078 (2015), DOI 10.1111/iere.12132 — the meaning of right-tailed tests (p. 1047), the null model, equation (3), and the regression, equation (4) (pp. 1047–1048), SADF (p. 1048), GSADF and Theorem 1 (p. 1049), the minimum-window rule and Table 1 (p. 1050), BSADF (p. 1051), date-stamping (p. 1052). (published version).
- Olivier J. Blanchard and Mark W. Watson, NBER Working Paper 945 (1982) — abstract and introduction. Jean Tirole, *Econometrica* 53(5): 1071–1100 (1985) — p. 1071. Both are the same sources as in Chapter 19.
- Christian M. Hafner, *Journal of Financial Econometrics* 18(2): 233–249 (2020) — introduction (p. 235), data, Table 1 (p. 236), §4, Table 6 (pp. 245–246).
- Andrea Podhorsky, *International Review of Financial Analysis* 93: 103158 (2024), CC BY — abstract, §4 date-stamping and equation (14) (p. 11), §5 Tables 2 and 3 (pp. 11–12).
- Chiara Oldani, Giovanni S. F. Bruno and Marcello Signorelli, *Journal of Economic Asymmetries* 31: e00420 (2025), CC BY — §4 data and bootstrap procedure (pp. 5–6), Tables 1 and 2 and the BTC intervals (p. 6), ETH and XRP (p. 7), abstract (p. 1).
- Eng-Tuck Cheah and John Fry, *Economics Letters* 130: 32–36 (2015) — pp. 4–6 and p. 9 (Table 2) of the author accepted manuscript; the page numbers of the published version were not checked. Adrian Cheung, Eduardo Roca and Jen-Je Su, *Applied Economics* 47(23): 2348–2358 (2015) — pp. 2353 and 2356 of the published version.
- Spencer Wheatley et al., *Royal Society Open Science* 6: 180538 (2019), CC BY — pp. 5–7, Table 1.

- Corbet, Lucey and Yarovaya, *Finance Research Letters* 26: 81–88 (2018) — **retracted** (notice FRL 91: 109483, 2026). The text was not obtained and is not cited.
- Code execution: Python 3.13 (conda environment `bcbook`), `numpy` 2.5.4, the only numerical package used in Part 3. It was used to compute the ADF statistics for all windows at once with prefix sums (with pure-Python double loops the Monte Carlo would take tens of minutes); the reasons for choosing it are in `notebooks/README.md` and `requirements.txt`. Notebook NB25. **Only synthetic data were used**: seeds 25, 2025, 2526, 0 (search), and 7; explosive interval t ∈ [100, 130), β = 1.04. No actual market price data were used.

Notes

1. Olivier J. Blanchard and Mark W. Watson, "Bubbles, Rational Expectations and Financial Markets," NBER Working Paper 945 (July 1982), abstract and introduction. Emphasis added. The second quotation begins mid-sentence; the original of its opening reads: "In the last two sections, we consider whether the presence of bubbles in a particular market can be detected statistically. The task is much easier if there are data on both prices and returns." The same source as footnote 27 in Chapter 19.

2. Jean Tirole, "Asset Bubbles and Overlapping Generations," *Econometrica* 53(5): 1071–1100 (September 1985), p. 1071 (introduction). Original: "Since Samuelson [30] developed his consumption loan model it has been well known that there exist economies in which money has a positive value in spite of the fact that it is intrinsically useless (i.e., its market fundamental is zero). In other words there can exist a bubble on money where a bubble is defined as the difference between the market price and the market fundamental." (JSTOR scan with no text layer, checked against the page images). The same source as footnote 28 in Chapter 19 (bibliographic details confirmed from the cover of the scan the editor obtained).

3. Peter C. B. Phillips, Shuping Shi and Jun Yu, "Testing for Multiple Bubbles: Historical Episodes of Exuberance and Collapse in the S&P 500," *International Economic Review* 56(4): 1043–1078 (November 2015), p. 1047. Original: "Right-sided unit root tests, as discussed in PWY, are informative about mildly explosive or submartingale behavior in the data and are therefore useful as a form of market diagnostic or warning alert." (published version). PWY is Phillips, Wu and Yu, *IER* 52 (2011), 201–226.

4. PSY (2015), pp. 1047–1048, equation (4). Original: "The empirical regression model can then be written as $\Delta y_t = \hat{\alpha}_{r_1,r_2} + \hat{\beta}_{r_1,r_2} y_{t-1} + \sum_{i=1}^{k} \hat{\psi}^{i}_{r_1,r_2} \Delta y_{t-i} + \hat{\varepsilon}_t$, where k is the (transient) lag order. The number of observations in the regression is $T_w = \lfloor T r_w \rfloor$ ... The ADF statistic (t-ratio) based on this regression is denoted by $ADF^{r_2}_{r_1}$." The model under the null is equation (3), $y_t = dT^{-\eta} + \theta y_{t-1} + \varepsilon_t$, $\theta = 1$ (p. 1047).

5. PSY (2015), p. 1048. Original: "The PWY test relies on repeated estimation of the ADF model on a forward expanding sample sequence, and the test is obtained as the sup value of the corresponding ADF statistic sequence. ... The PWY test is then a sup statistic based on the forward recursive regression and is simply defined as $SADF(r_0) = \sup_{r_2 \in [r_0, 1]} ADF^{r_2}_0$."

6. PSY (2015), p. 1050, Table 1 "The asymptotic and finite sample critical values of the SADF and GSADF tests against an explosive alternative". Finite-sample row "T = 200, r0 = 0.137": SADF 90/95/99% = 1.12/1.40/1.90, GSADF 90/95/99% = 1.84/2.08/2.70. Note to the table: "The finite sample critical values are obtained from Monte Carlo simulation with 2000 replications." The minimum-window rule (same page): "we recommend a rule for choosing r0 that is based on a lower bound of 1% of the full sample and has the simple functional form $r_0 = 0.01 + 1.8 / \sqrt{T}$". The ADF in this cell has lag 0, while PSY's finite-sample critical values were obtained under a null model with $d = \eta = 1$ (weak drift), and the lag is not stated. The random numbers, the number of replications, and the drift differ, so there is no reason for the two sets of numbers to be exactly the same. The next cell measures this directly.

7. PSY (2015), p. 1049. Original: "we denote this statistic by GSADF(r0). That is, $GSADF(r_0) = \sup_{r_2 \in [r_0, 1],\, r_1 \in [0, r_2 - r_0]} ADF^{r_2}_{r_1}$."; Theorem 1: "When the regression model includes an intercept and the null hypothesis is a random walk with an asymptotically negligible drift (i.e., $dT^{-\eta}$ with η > 1/2 and constant d) as in (3), the limit distribution of the GSADF test statistic is [a functional of Wiener processes] ...".

The formula for the limit distribution is in the text on p. 1049.

8. PSY (2015), p. 1050. Original: "First, as the minimum window size r0 decreases, critical values of both test statistics increase. For instance, when r0 decreases from 0.190 to 0.055, the 95% asymptotic critical value of the GSADF statistic rises from 1.89 to 2.30. Second, critical values for the GSADF statistic are larger than those of the SADF statistic. For instance, when r0 = 0.10, the 95% asymptotic critical value of the GSADF statistic is 2.19 whereas that of the SADF statistic is 1.49."

9. PSY (2015), p. 1051. Original: "In particular, the backward SADF test performs a sup ADF test on a backward expanding sample sequence where the endpoint of each sample is fixed at r2 ... and the start point varies from 0 to r2 − r0 ... The backward SADF statistic is then defined as the sup value of the ADF statistic sequence over this interval, viz., $BSADF_{r_2}(r_0) = \sup_{r_1 \in [0, r_2 - r_0]} ADF_{r_1}^{r_2}$. The PWY procedure (i.e., the recursive ADF test) for bubble identification is a special case of the backward sup ADF test in which r1 = 0 ...". Why the PWY approach misses a second bubble (pseudo stationarity; the criticism of Evans 1991) is on the same page.

10. PSY (2015), p. 1052. Original: "The origination date of a bubble ⌊Tre⌋ is calculated as the first chronological observation whose ADF statistic exceeds the critical value. ... The estimated termination date of a bubble ⌊Trf⌋ is the first chronological observation after ⌊Tr̂e⌋ + LT whose ADF statistic goes below the critical value. PWY impose a condition that for a bubble to exist its duration must exceed a slowly varying (at infinity) quantity such as LT = log(T). This requirement helps to exclude short lived blips in the fitted autoregressive coefficient and, as discussed below, can be adjusted to take into account the data frequency." Equations (6) and (7) (p. 1052) compare the BSADF with the SADF critical value scv based on $\lfloor Tr_2 \rfloor$ observations. NB25-2 uses the endpoint-wise Monte Carlo quantiles as critical values and discards intervals shorter than L_T, which differs slightly from the treatment in equation (7) (the first observation that falls below after the start plus $\delta \log T$). It is a simplification for demonstration.

11. Christian M. Hafner, "Testing for Bubbles in Cryptocurrencies with Time-Varying Volatility," *Journal of Financial Econometrics* 18(2): 233–249 (2020), p. 235 (introduction) and pp. 245–246 (§4, Table 6). Original: "As shown by Harvey et al. (2016), the original test of PWY is biased in the presence of non-stationary volatility. ... Harvey et al. (2016) suggest a wild bootstrap procedure to correct the size of the PWY test and show that it has good power properties."; abstract: "This paper extends this approach to the case where volatility is time varying ..."; "Taking into account global heteroskedasticity has the effect of increasing the p-values that were found to be very close to zero ... In other words, the evidence in favor of speculative bubbles is much less pronounced than when this effect is ignored."; Table 6, BTC row: PWY 5.0347, p-value 0.0001, WB 5% 2.8349, WB 1% 4.2929, WB p-value 0.0032; "not all p-values increase, in particular the larger ones tend to decrease under the correction, which is the case for BCH, ETH, XMR, and DASH"; "Only one p-value is smaller than 1% (BTC with 0.0032), meaning that at the 1% level one would reject the null hypothesis in favor of a speculative bubble only for the Bitcoin. At the 5% level, this number increases to 9 (all cryptos except IOTA, BCH, and XLM)". The 9 includes CRIX (8 of 11 counting the crypto-assets alone). Data: the top 11 crypto-assets by market capitalization as of 2017-12-31 and the CRIX index; BTC from 2013-04-28, 1,709 observations (Table 1, p. 236); $r_0 = 0.1$, wild bootstrap with 5,000 replications.

12. Chiara Oldani, Giovanni S. F. Bruno and Marcello Signorelli, "Collapsing bubbles in the prices of cryptocurrencies," *Journal of Economic Asymmetries* 31: e00420 (June 2025), pp. 5–6 (§4.1). Original: "The Monte Carlo CVs for SADF and GSADF tabulated in the original PSY paper ... are not robust to two potential inference issues: 1) time-varying unconditional heteroscedasticity in the ADF errors and 2) the multiplicity issue (the probability of a false positive increases with the number of hypotheses tested) ... Phillips and Shi (2020) (PS) suggest a composite wild bootstrap procedure for computing robust CVs ...". The procedure has five steps (pp. 5–6), with a bootstrap window of $T_b = 183$ days and $B = 499$. The text gives the minimum window as "$r_0 = 0.1 + 1.8 / \sqrt{T}$", but $\lfloor r_0 T \rfloor = 110$ in Table 2 matches the value computed with the PSY rule $0.01 + 1.8 / \sqrt{2357}$, so the 0.1 in the text appears to be a typo (this book's judgment; to be checked against the original).

13. Eng-Tuck Cheah and John Fry, "Speculative bubbles in Bitcoin markets? An empirical investigation into the fundamental value of Bitcoin," *Economics Letters* 130: 32–36 (May 2015). This book read the author accepted manuscript (AAM), and page numbers refer to that version. Data: "daily closing prices for the Bitcoin Coindesk Index from July 18th 2010 to July 17th 2014" (p. 5); test window "January 1st 2013-November 30th 2013" (p. 5). Results (p. 6): "We reject the null hypothesis (11) and have significant evidence of a bubble. Further, the parameter μ̃ is not statistically different from zero suggesting that equation (17) applies. Thus, during the bubble price rises are so dramatic that the estimated long-term fundamental value is not

statistically different from zero. … Equation (16) suggests that the bubble accounts for around 48.7% of observed prices."; Table 2 (p. 9): v 0.546 (E.S.E. 0.090, t 6.060, p 0.000), μ̃ 0.00166 (0.00195, 0.852, 0.394); conclusion (p. 6): "Thirdly, the fundamental value of Bitcoin is zero." The null and alternative hypotheses are equation (11), $H_0{:}v = 0$, $H_1{:}v > 0$ (p. 4).

14. Andrea Podhorsky, "Bursting the bitcoin bubble: Do market prices reflect fundamental bitcoin value?," *International Review of Financial Analysis* 93: 103158 (May 2024), abstract and pp. 11–12 (§5). Original: "demonstrates that the bitcoin's volatile and explosive price path is a consequence of the Bitcoin protocol's system of supply management" (abstract); "The ADF test on the price of the bitcoin p_{bt} resulted in a test statistic of −2.51 and a p-value of .36, and the ADF test on the fundamental value … −1.90 and a p-value of .62 …"; "the Engle–Granger cointegration test … resulted in a test statistic of −2.74 and a p-value of .071, indicating that we can reject the null hypothesis of no cointegration at the 10% level of significance."; "For the price data, the SADF and GSADF statistics are 3.7 and 3.9, respectively, which both exceed their 1% right-tailed critical values (3.7 > 2.2 and 3.9 > 2.9)"; "Next, the summary SADF and GSADF tests are applied to the residuals v̂t. The SADF and GSADF statistics are −.08 and 1.52, respectively, which fall well below their 10% right-tailed critical values (−.08 < 1.3) and (1.5 < 2.2)"; "Since the raw price data indicates the presence of bubbles while the residuals v̂t do not, from (14) it follows that we can attribute the apparent bubbles in the price of the bitcoin to the fundamental value …". The sample is 1,764 daily observations, the minimum window 93 ($r_0 = .01 + 1.8 / \sqrt{1764}$), and the critical values come from 2,000 Monte Carlo replications (p. 11). CC BY.

15. Spencer Wheatley, Didier Sornette, Tobias Huber, Max Reppen and Robert N. Gantner, "Are Bitcoin bubbles predictable? Combining a generalized Metcalfe's Law and the Log-Periodic Power Law Singularity model," *Royal Society Open Science* 6: 180538 (2019), pp. 5–7. Exponent: footnote 11, "Our finding of a smaller exponent b ≈ 1 + 2/3 expresses a more sparsely connected network in which each user is on average linked to ≈N^{2/3} other users …"; OLS coefficients (a, b) = (1.51, 1.69) (caption of Figure 2, p. 6). Table 1 (p. 7), "Bubble statistics. Columns: start, end (time of peak value, prior to correction), duration in days, starting and peak market cap, growth factor … and average daily return": 1 — 25 May 2012 / 18 Aug 2012 / 84 days; 2 — 3 Jan 2013 / 11 Apr 2013 / 98 days; 3 — 7 Oct 2013 / 23 Nov 2013 / 47 days; 4 — 8 June 2015 / 18 Dec 2017 / 924 days (the market capitalization and growth-factor columns are not carried over; see the notes to the reader in Chapter 3). Data: "The price data for Bitcoin is from Bitstamp, in USD, hourly from 1 January 2012 to 8 January 2018; the Bitcoin circulating supply comes from blockchain.info." The market capitalization column was not carried over. CC BY.

16. Wheatley et al. (2019), p. 5. Original: "More generally, within the sample, the fitted curves are similar, but, beyond the sample, differences explode such that there are 4 orders of magnitude difference between the predicted carrying capacities. Here, model uncertainty dominates uncertainty of estimated parameters. … Therefore, precise inference based on a single model—notably omitting any limitation imposed by the physical Bitcoin network—is misleading, and long-term predictions effectively meaningless. However, smoothing of past values is not problematic, and short-term projections may be reasonable."

17. Adrian (Wai-Kong) Cheung, Eduardo Roca and Jen-Je Su, "Crypto-currency bubbles: an application of the Phillips–Shi–Yu (2013) methodology on Mt. Gox bitcoin prices," *Applied Economics* 47(23): 2348–2358 (2015), published version. Data (p. 2353): "most actively traded bitcoins (i.e., MT.GOX) over the period 17 July 2010 to 18 February 2014. They are all denominated in the US dollar. There are 1307 daily observations in the sample period."; minimum window (p. 2353): "Following Phillips et al. (2013a), we set r0 to approximately 2% of the data (i.e., 20)"; results (p. 2356): "The GSADF test statistic … exceeds its corresponding 99% critical value … 33 times over the sample period, suggesting that 33 episodes of bubbles are identified. … The duration of the bubbles ranges from 1 day only to 106 days. … there were three big bubbles … The start date and the end date of the first big bubble are 24 April 2011 and 3 July 2011, respectively, covering 66 days in between while the start date and the end date of the second big bubble are 27 January 2013 and 15 April 2013, respectively, with a lifespan of 79 days. The third (and the longest) bubble has a history of 106 days, covering the period from 5 November 2013 to 18 February 2014 (i.e., the end of the sample period)". The "66 days" for the first bubble is exactly as printed on p. 2356 of the published version. But from 2011-04-24 to 07-03 is 71 days counting both ends (the 79 and 106 days of the second and third intervals agree with a count that includes both ends), and the paper does not explain whether 66 is a count of observations that leaves out missing days or an error. This book does not correct the paper's number; it carries it over as printed and records the difference.

18. Oldani et al. (2025), pp. 6–7, Table 2 "GSADF Tests with bootstrap CVs and fixed lag order = 0 (daily data 01/01/2018–14/06/2024)": BTC 3.709 ***(RTBS90/95/99 = 1.246/1.500/2.086), ETH 2.289*** (1.210/1.513/2.124), XRP 3.966*** (1.340/1.769/2.104); "Number of obs. T = 2357; initializing window size ⌊r0T⌋ = 110; RTBS: right-tail

Bootstrap CVs ... based on wild bootstrap with 499 replications and bootstrap window Tb = 183." BTC intervals (p. 6): "The first bubble episode starts on 2018-11-24 and terminates on 2018-12-08 with a break in between. The second starts on 2019-05-11 and terminates on 2019-06-28 ... The third is the longest interval, spanning four months, from 2020-12-19 to 2021-03-13 ... The last starts on 2024-02-28 and terminates on 2024-03-15 ...". XRP (p. 7): "for XRP ln prices the GSADF test with zero lag order rejects unit root at the 1 % level (Table 2), but only a short-lived single bubble is found, from 2020-11-23 to 2020-11-25"; p. 8: "With a fixed lag order of 7, GSADF does not support exuberant behavior at all (Table 4)."; abstract (p. 1): "No test supports the exuberance of XRP that is traded on a centralized ledger ...". The authors' linking of the BTC intervals of 2018 and 2019 to the halving, and of the 2020–21 interval to post-pandemic exuberance, carries over interpretations from earlier studies, so this book does not take it up (Chapter 20: the halving is not used as a price event).

19. Shaen Corbet, Brian Lucey and Larisa Yarovaya, "Datestamping the Bitcoin and Ethereum bubbles," *Finance Research Letters* 26: 81–88 (September 2018), DOI 10.1016/j.frl.2017.12.006; the title in the Crossref record carries "RETRACTED:". Retraction notice: "Retraction notice to 'Datestamping the Bitcoin and Ethereum Bubbles' [Finance Research Letters 26 (2018) 81-88]," *Finance Research Letters* 91: 109483 (March 2026), DOI 10.1016/j.frl.2026.109483. The check recorded that the Retraction Watch entry date (2025-12-18) differs from the DOI registration date of the notice (2026-02-06) and that **the reason for the retraction could not be confirmed**. This book did not obtain the text of the paper and cites none of its numbers. When reading later literature that cites it (Oldani et al. included), keep in mind that those citations predate the retraction.

PART 4 · READING THROUGH CASES

Chapter 26. Four cases: Mt.Gox, The DAO, Terra-Luna, FTX

NOTICE

From here on we read four cases. To say it in advance: this part does not follow "who made and who lost how much." It asks one thing: **what broke?** Was it the technology, the economic design, the governance, or was it fraud from the start? Names and figures are only those written in judgments and official investigation documents.

The full text of the notice is in the colophon (F.4 of Appendix F).

KEY TAKEAWAY

This chapter places the four cases on **the same diagnostic table**. It has four columns: what broke (technology, economic design, governance, fraud); whether the incident happened on the chain or off it; what the code did and what people did; and what followed (insolvency proceedings, a hard fork, criminal proceedings). Facts are taken only from judgments and decisions (whether they are final is stated in the notes), official investigations, and insolvency documents, and each amount is given once, as written in those documents. The layer that broke is different in each of the four cases. Mt.Gox is **a mismatch between the exchange's books and the chain** (the manipulation of the books was found by the criminal judgments at first instance and on appeal, and no court document has decided the cause of the February 2014 loss); The DAO is **a flaw in the contract code and the governance decision that followed**; Terra is **the reflexivity of the design and the issuer's statements**; and FTX is **the misuse of customer assets and the absence of internal controls**. None of the four is a case in which the consensus rules and ledger integrity learned in Part 1 were broken by an outside attack. That sentence, though, can only be read once it is settled what "the blockchain itself" is, and this chapter says it only after defining that scope in three parts in Section 1. What the lawyer takes away is not the story of the cases but **the diagnostic framework**. Only once you sort out which layer broke can you see where each body of law comes in.

Learning objectives

1. Put the four cases on the same diagnostic table (① the layer that broke, ② on or off the chain, ③ what the code did and what people did, ④ what followed) and read them side by side.
2. Divide the phrase "the blockchain itself" into three layers (consensus rules and ledger integrity / properties of the protocol / the books, contracts, and issuers placed on top) and tell, for each case, which layer the story is about.
3. For each case, distinguish **what the primary documents (judgments, official investigations, insolvency documents) decided and what they did not decide**. Exchange announcements and press reports are not primary documents.
4. Note where the lawyer's questions arise in each case, and hand them over to the issue sections of Part 5.

What a reader of the [Basic] paragraphs alone takes away. What broke is different in each of the four cases, and among them **there is no case in which the consensus rules of Bitcoin or Ethereum were broken by an outside attack and the ledger was reversed.** The places that gave way were the books off the chain, the contract code placed on the chain, the issuer's design and statements, and human hands.

1. How this chapter reads the cases: the diagnostic table and the scope of "the blockchain itself"

Write the cases up one by one as stories and "who lost how much" becomes the axis. This book does not use that axis (the notes to the reader in Chapter 3). Instead it puts the four cases on the same table. There are four columns.

COLUMN	WHAT IT ASKS
① The layer that broke	Was it technology (consensus rules, the ledger, cryptography), economic design (the structure of tokens, collateral, fees), governance (who changes the rules), or fraud (the gap between statements and reality)? It may be more than one
② On or off the chain	Did the incident happen as a state transition on the ledger, or in books, bank accounts, or a company off the ledger?
③ Code and people	Separate what the code did as written from what people did by hand. Here you can see that "it was executed as coded" is not a statement that erases responsibility
④ What followed	Which of insolvency proceedings, a hard fork, criminal proceedings, and administrative investigation followed, and what did those documents decide?

To fill in the second column, the scope of the phrase "the blockchain itself" has to be settled first. The phrase is used to point to three different things.

LAYER	MEANING	WHERE THIS BOOK COVERS IT
(a) Consensus rules and ledger integrity	The rules by which blocks are linked, the choice of the longest (heaviest) chain, signature verification, transaction validity rules. What the "cost of tampering" protects	Chapter 7, Chapter 8, Chapter 9, Chapter 23
(b) Properties of the protocol	Behavior the rules allow that the designers did not intend or that troubles the application layer. Transaction malleability (the property that changing a signature changes the transaction identifier), the power to decide ordering (MEV), and so on	Chapter 5, Chapter 17
(c) What is placed on top	An exchange's internal books, the code of a smart contract, the balance sheet and statements of a stablecoin issuer. The chain does not guarantee them	Chapter 13, Chapter 14, Chapter 24

When this chapter writes "the blockchain itself did not break," it means (a). (b) may have been involved, depending on the case (Mt.Gox claimed so), and (c) is the stage for all four cases. Mixing the three layers to say "blockchains are safe" or "blockchains are dangerous" is something this chapter does not do, either way.

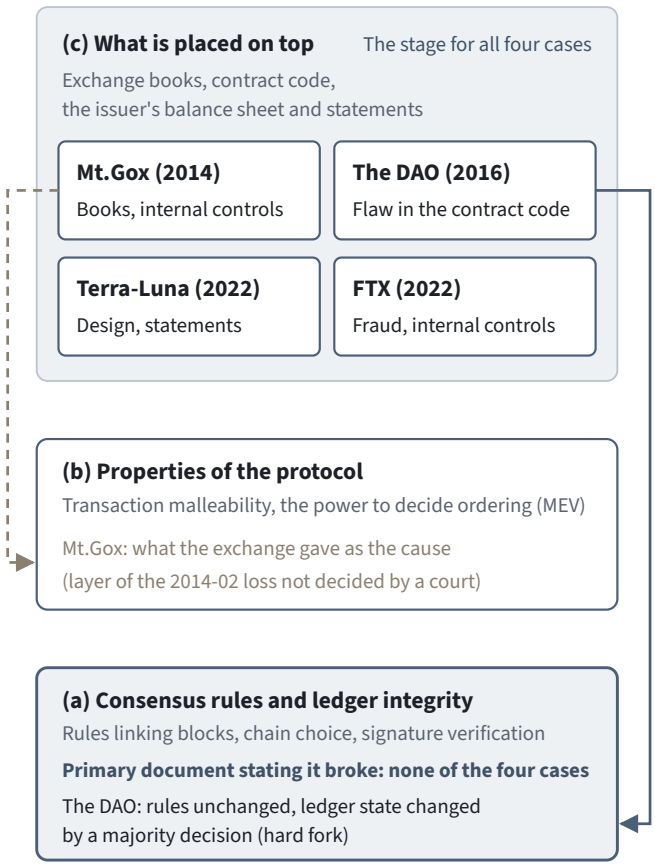


Figure 26-1. The three layers of "the blockchain itself" and where the four cases broke. The sentence "did not break" refers to layer (a); the dotted line is what the Mt.Gox exchange gave as the cause, on which the court did not rule.

The sources of fact are limited to three: **judgments and decisions** that are final or whose finality is stated, **official investigation reports** of regulators, and **documents filed by courts, trustees, and debtors in insolvency proceedings**. An exchange's press release is a source for the fact that "the party said so," not for the truth of what it says, and newspaper reports are

not used. Two additions. For the technical description of The DAO's flaw, which the judgments and investigation documents do not give, this chapter uses Ethereum Foundation announcements and an EIP (what Chapter 22 handed over to this chapter). And the February 7, 2014, Mt.Gox announcement halting withdrawals is quoted from a secondary reproduction, because the original archive could not be opened, and this is stated where it is used.

2. Mt.Gox (2014): a mismatch between the books and the chain

What the documents record

The company that ran the bitcoin exchange Mt.Gox (hereafter "Company M," as the judgment writes it) petitioned the Tokyo District Court (東京地方裁判所) for commencement of civil rehabilitation proceedings (民事再生, minji saisei) on February 28, 2014. The announcement the company issued that day gave its financial position as follows: total assets of JPY 3,841,866,163 and total current liabilities of JPY 6,501,119,371. It also wrote that behind the increase in current liabilities "ビットコインの消失,
預り金の消失という原因が考えられ" (working gloss, not an official translation; the original controls: "the disappearance of bitcoins and the disappearance of deposited funds are thought to be the causes").[1] The same announcement describes the course of events as follows. In early February 2014,
"ビットコインのシステムのバグを悪用した不正アクセスにより" (gloss: "through unauthorized access exploiting a bug in the bitcoin system"), transactions in which a remittance was not completed normally increased; investigation revealed by about February 24 that almost all of roughly 750,000 BTC held by users and roughly 100,000 BTC held by the company itself had disappeared; on the same day a shortfall of at most about JPY 2.8 billion was found between the total of users' deposits and the balance of deposits at financial institutions; and at noon on February 25 all access to the site was suspended. On the cause, it wrote "第三者によるハッキングによる被害を含めて，複数の原因があると見込まれる" (gloss: "multiple causes are expected, including damage from hacking by third parties"). The words "transaction malleability" do not appear here.

Three weeks later, on March 20, the company announced that it had rescanned an old-format wallet it had used before June 2011 and confirmed that about 200,000 BTC (199,999.99 BTC) remained in it, and it lowered its estimate of the loss from about 850,000 BTC to about 650,000 BTC. That announcement, too, says
"ＢＴＣの消失の原因や消失したＢＴＣの数量は現在調査中" (gloss: "the cause of the disappearance of the BTC and the quantity of BTC that disappeared are currently under investigation").[2] On April 16 the Tokyo District Court dismissed the petition for commencement of civil rehabilitation and issued a provisional administration order (保全管理命令, hozen kanri meirei), and on April 24 it made a decision commencing bankruptcy proceedings (破産手続, hasan tetsuzuki). The reason given in the decision is one line: "一件記録によれば，債務者が支払不能の状態にあることが認められる。" (gloss: "According to the entire record, it is found that the debtor is in a state of inability to pay.")[3] In his report for the first creditors' meeting in July 2014, the bankruptcy trustee wrote that he had entrusted the investigation of the loss to outside accounting firms and that "the

completion date is undetermined," and in his May 2016 report he wrote, "Due to the circumstances, such as the limited amount of information that exists, I believe it will be difficult to understand the full picture ...." The March 2018 report records that, when the U.S. Department of Justice indicted the operator of another exchange in July 2017, the allegations included that "he obtained funds by hacking MtGox," and that the trustee had requested information.[4] On a petition by some creditors in November 2017, civil rehabilitation proceedings were commenced on June 22, 2018, and the bankruptcy proceedings were stayed.[5] Among the documents on the trustee's site, this book found none that settles and announces the cause of the loss.

What the exchange gave as the cause, and what the empirical study found

[Intermediate] Before the February 28 announcement, the company had announced on February 7 that it was halting all bitcoin withdrawals. That announcement said that "the increase in the flow of withdrawal requests has hindered our efforts on a technical level" and that "a temporarily [sic] pause on all withdrawal requests" was necessary; it gave no cause.[6] The February 10 announcement was the first to name a cause. Under the heading "Addressing Transaction Malleability" it wrote: "A bug in the bitcoin software makes it possible for someone to use the Bitcoin network to alter transaction details to make it seem like a sending of bitcoins to a bitcoin wallet did not occur when in fact it did occur."[7] This is the **transaction malleability** we saw in Chapter 5. Change the form of a signature and the transaction identifier (txid) changes, and exchange software that tracks the success or failure of transactions only by identifier can read a transaction that was actually sent as "failed" and send it again. In terms of the three layers above, this is the path by which a property of layer (b) deceived the books of layer (c).

[Intermediate] This claim can be tested by observing the network. Christian Decker and Roger Wattenhofer had been running a node on the Bitcoin network since January 2013 and recording the transactions relayed to it (the connected peers were about 20% of reachable nodes), and in that data they counted the traces of malleability attacks: pairs of transactions that spend the same inputs but differ only in the encoding of the signature. Their conclusion: "while MtGox claimed to have lost 850,000 bitcoins due to malleability attacks, we merely observed a total of 302,000 bitcoins ever being involved in malleability attacks. Of these, only 1,811 bitcoins were in attacks before MtGox stopped users from withdrawing bitcoins. Even more, 78.64% of these attacks were ineffective. As such, barely 386 bitcoins could have been stolen using malleability attacks from MtGox or from other businesses."[8] Malleability attacks rose sharply **after** the February 10 announcement (after the announcement had made the method of attack known), and the amount before withdrawals were halted is about 1/470 of the claimed loss. The paper ends with "MtGox needs to explain the whereabouts of 849,600 bitcoins." What the empirical study says goes as far as "the amount explained by malleability attacks is very small"; it does not say where the rest went.

What the criminal judgments found: fictitious balances in the books

The representative director was arrested and indicted in 2015, and on March 15, 2019, the Tokyo District Court sentenced the defendant for the offenses of 私電磁的記録不正作出 and 同供用 (shi-denjiteki kiroku fusei sakushutsu and dō kyōyō, "unauthorized creation of private electromagnetic records" and "use of such records") to 懲役2年6月·執行猶予4年 (imprisonment with work for 2 years and 6 months, suspended for 4 years), and acquitted the defendant on the counts of 業務上横領 (gyōmujō ōryō, "embezzlement in the course of business") and 会社法違反 (kaishahō ihan, "violation of the Companies Act"; aggravated breach of trust).[9] On June 11, 2020, the Tokyo High Court (東京高等裁判所) dismissed the defendant's appeal; the acquittals were not before the appellate court, because the prosecution did not appeal them.[10] This book has not verified whether a final appeal was filed or the date on which the judgment became final (as of 2026-10). What follows are the facts and determinations found in the texts of the two judgments. The "判示事項" (hanji jikō, "matters decided") summaries in case-law databases are editorial products and are not quoted.

The first-instance judgment begins by describing the structure of the exchange. When a user sends money to a bank account designated by the company, or sends bitcoin to an address the company manages, the system confirms this automatically and increases the balance of the user's account. After that, even when trades are concluded between users, "本件取引システム上の口座残高に増減が生じるだけで，…ビットコインも同様に，本件取引システム上の口座残高に変動は生じるが，ブロックチェーン上に変動は生じない" (gloss: "only increases or decreases in the account balances on the Trading System arise, … likewise with bitcoin, the account balances on the Trading System change, but no change arises on the blockchain"). In other words, trading inside the exchange is only the movement of numbers in the books, and the blockchain moves only when a user requests repayment or delivery and the company actually sends it.[11] This is the relationship between the ledger seen in Chapter 11 and Chapter 12 and the layer (c) books placed on top of it. The judgment also reserves its position on the private-law nature of bitcoin: "ビットコインは「有体物」（民法８５条）ではなく，その帰属や利用者間での取引がどのような私法上の性質を有するかも一義的に明らかとはいえないが，本件の判断に影響のない限り，ビットコインの保有・所有や利用者間の売買などと表現し，日常的な意味合いで記載する。" (gloss: "Bitcoin is not a 「有体物」 (yūtaibutsu, "tangible thing") (Civil Code, Article 85), and it cannot be said to be unambiguously clear what private-law nature its attribution and the transactions between users have; but insofar as this does not affect the determination in this case, [this judgment] expresses these as the possession or ownership of bitcoin, sale and purchase between users, and so on, and writes in the everyday sense.")[12] This is the sentence that the map of scholarly positions in 27.2, in Part 5, takes up.

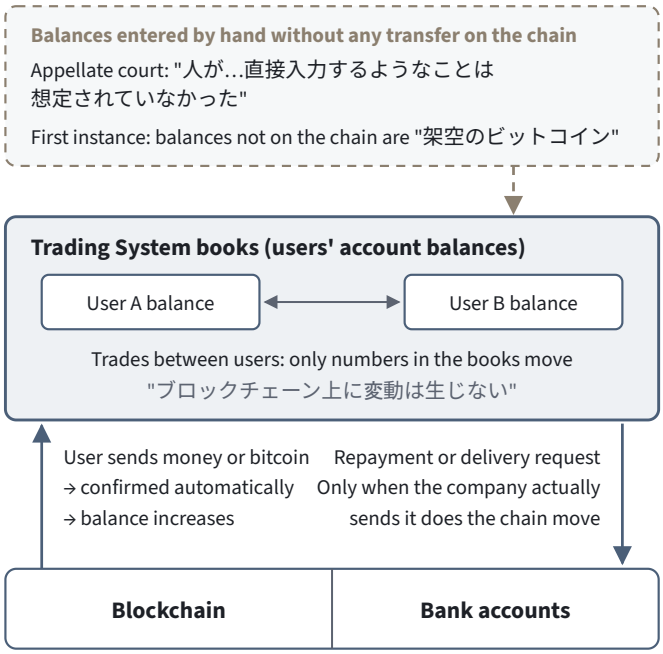


Figure 26-2. The structure of the exchange as described by the first-instance judgment. Trades between users move only numbers in the books, and the judgment called balances entered by hand without any transfer on the chain "架空のビットコイン" ("fictitious bitcoin").

[Intermediate] The skeleton of the facts found is three "balance-increasing acts."[13] First, in August 2011, another exchange in Poland that the defendant's parent company (Ｄ社 in the first-instance judgment, Ｔ社 in the appellate judgment) had acquired lost 17,000 BTC through a system defect and took on "ブロックチェーン上に存在しない１万７０００ＢＴＣの返還債務" (gloss: "an obligation to give back 17,000 BTC that did not exist on the blockchain"); the defendant entered information increasing the U.S. dollar balance of his own account by USD 150,000, used those dollars to purchase about 18,000 BTC from users, and then deleted 17,500 BTC from the bitcoin balance of his account ("１５万米ドル増加行為," gloss: "the USD 150,000 increase act"). Second, in November 2011, when a French bank account the company used was temporarily frozen and about JPY 100 million could not be withdrawn, the defendant, on 8 occasions from November 2011 to April 2012, entered information increasing the bitcoin balance of his own account by 300,000 BTC without sending bitcoin to the company's addresses, and by February 2013 had sold a total of about 300,000 BTC to users and obtained about USD 1.3 million ("３０万ＢＴＣ増加行為," gloss: "the 300,000 BTC increase act"). Third, to recover from the resulting state in which "ブロックチェーン上には存在しないビットコイン３０万ＢＴＣが利用者間で取引されることとなった" (gloss: "300,000 BTC of bitcoin that did not exist on the blockchain came to be traded between users"), on 21 occasions from February to September 2013 he entered information increasing the dollar balance of his account by a total of USD 33.5 million, purchased about 300,000 BTC from users for about USD 29.77 million, and then deleted that bitcoin balance ("本件各行為," gloss: "the acts in this case"). What was charged was the third set of acts, and the court held that, because the third was a means of concealing the second, the two had to be viewed as a single whole.

[Intermediate] The issue was whether these entries created electromagnetic records "不正に" (fusei ni, "improperly"). The defense argued that, since the defendant was the only person with comprehensive authority to conduct the company's business, the defendant's acts were the company's will. The court separated the company's will from the defendant's inner intentions and read it **from the terms of use and the nature of the system**. The key sentence: "ビットコインの存在根拠及びその経済的価値の根拠は，取引されるビットコインが，
ブロックチェーン上に存在することにかかっているのであって，
ブロックチェーン上に存在しないビットコインは，仮に，
本件取引システム上のビットコイン口座に残高が表示されたとしても，
架空のビットコインといわざるを得ない。" (gloss: "The basis for the existence of bitcoin and the basis for its economic value depend on the traded bitcoin existing on the blockchain, and bitcoin that does not exist on the blockchain must be called fictitious bitcoin, even if a balance is displayed in a bitcoin account on the Trading System.") The terms of use stated that the seller "売却オファーの目的となる…ビットコインは，
実際のビットコインに対応することを保証" (gloss: "guarantees that the … bitcoins that are the object of a sale offer correspond to actual bitcoins"), and there was neither a provision nor an on-screen display permitting credit trading. So the company's will was that "B社管理のビットコインアドレスにおけるビットコインの受送付等による裏付けがある場合に限って，本件取引システム上の利用者の口座残高を増加させるべきである" (gloss: "users' account balances on the Trading System should be increased only where there is backing by the receipt or sending, etc., of bitcoin at bitcoin addresses managed by Company B"), and the defendant's authority was also limited to that scope.[14] The appellate court said of this determination that it "概ね同旨をいうものとして是認できる" (gloss: "can be upheld as stating substantially the same"), and added that the evidentiary function of electromagnetic records rests on "それが作出されるべきシステムの設置運営主体の意思に基づき，
正当に作出されることが当然の前提" (gloss: "the natural premise that they are created legitimately, based on the will of the entity that set up and operates the system in which they are to be created").[15]

[Intermediate] On **money**, by contrast, the conclusion was different. Money that users sent to the account designated by the company belonged to the company, and since at the time neither statute nor the terms of use required separate management, it could not be said that money corresponding to the dollar balances in the books had to be in the company's bank accounts.[16] So the transfers from the company's accounts to accounts in the defendant's name and elsewhere were not embezzlement of users' money (the primary count); not embezzlement of the company's money (alternative count 1), because they were accounted for as short-term loans and there was a realistic possibility of repayment; and not aggravated breach of trust (alternative count 2). Set the two conclusions side by side and the structure of the case appears. **On the bitcoin side, the requirement of backing, that it "must be on the chain," was found to be the company's will; on the money side, the requirement that it "must be in the bank" was not.** What made the difference was the property of layer (a) in Section 1, that the "existence" of bitcoin is defined by the blockchain record, and the state of the law, that there was then no statute requiring separate management. Korea's current Act on the Protection of Virtual Asset Users (가상자산 이용자 보호 등에 관한 법률) provides for the separate deposit or trust of users' deposits (Article 6) and for the separate custody of

virtual assets together with the obligation to "동일한 종류와 수량의 가상자산을 실질적으로 보유" (gloss: "actually possess virtual assets of the same kind and quantity") (Article 7(2)); these are obligations placed in the two positions just described. Whether this case was the purpose of that legislation is not stated, because the legislative materials were not checked.[17] What kind of right those provisions presuppose is asked in Chapter 28.

Diagnosis

① The layer that broke is **the exchange's books and internal controls**: layer (c). In the part on which the court found guilt, balances that were not on the chain appeared in the books, without any transfer on the chain, and were traded. Which layer the February 2014 loss belongs to is **something no court document decided.** The exchange cited a property of layer (b) (transaction malleability) and in its later announcement of February 28 also mentioned the possibility of hacking; the empirical study found the amount explained by malleability to be very small; and the trustee said it would be difficult to understand the full picture. ② Off the chain. ③ What the code did was to confirm users' remittances automatically and raise balances, and human hands entered where, as the appellate court wrote, "人が個々の取引やその履歴のデータ等を直接入力するようなことは想定されていなかった" (gloss: "it was not contemplated that a person would directly input individual transactions, their history data, and the like"). ④ What followed were insolvency proceedings (petition for civil rehabilitation → dismissal and provisional administration → bankruptcy → civil rehabilitation on a creditors' petition (2018)) and criminal determinations (guilty of the unauthorized creation of electromagnetic records, not guilty of embezzlement). Whether a final appeal was filed and whether the judgment is final are not verified.

3. The DAO (2016): a flaw in the contract code and the decision that followed

What the documents record

The investigation report of the U.S. Securities and Exchange Commission (SEC) of July 25, 2017 (Release No. 81207) sets out the facts in the most orderly form. The DAO is one example of what is "a term used to describe a 'virtual' organization embodied in computer code and executed on a distributed ledger or blockchain," and it was created by the German company Slock.it and its co-founders. From April 30 to May 28, 2016, approximately 1.15 billion DAO Tokens were sold in exchange for approximately 12 million ETH, and the token holders were to vote on proposals and share in earnings.[18] A proposal went to a vote only after a "Curator" had put it on a whitelist, and the White Paper wrote that the Curators had "considerable power."[19] Then, page 9 of the report: "On June 17, 2016, an unknown individual or group (the 'Attacker') began rapidly diverting ETH from The DAO, causing approximately 3.6 million ETH—1/3 of the total ETH raised by The DAO offering—to move from The DAO's Ethereum Blockchain address to an Ethereum Blockchain address controlled by the Attacker (the 'Attack'). Although the diverted ETH was then held in an address controlled by the Attacker, the Attacker was prevented by The DAO's code from moving the ETH from that address for 27 days." After that, "Slock.it's co-founders and others endorsed a 'Hard Fork'," and "On July 20, 2016, after a majority of the Ethereum network adopted the necessary software updates, the new, forked Ethereum Blockchain became active." Footnote 32 records that a minority did not adopt the fork, on the ground that it would run counter to the concept that "a blockchain is immutable," and kept using the old chain, which is called "Ethereum Classic."[20]

The **technical** description of the flaw was given not by the SEC report but by the Ethereum Foundation blog on the day of the attack (Chapter 22 noted that the SEC wrote only "a flaw in The DAO's code" and handed this description over to this chapter): "The attack is a recursive calling vulnerability, where an attacker called the 'split' function, and then calls the split function recursively inside of the split, thereby collecting ether many times over in a single transaction." And one sentence in the same post states the layer distinction of Section 1 as it is: "This is an issue that affects the DAO specifically; Ethereum itself is perfectly safe."[21] Put into the EVM call model we saw in Chapter 13 (this is this book's reading; the primary document goes only as far as "calls the split function recursively"), the contract sent ether by an external call **before** updating the balance, and the receiving contract, inside that call, called the same function again and repeatedly drew out the balance that had not yet been updated. The code ran as written, without deviating by a single line. What diverged were the author's intention and the code.

The foundation blog first wrote that a soft fork had been proposed (a rule making invalid any transaction that reduces the balance of The DAO and its child contracts, that is, accounts with the same code hash), and in the end it became a hard fork. The post of July 20, 2016, reads: "Block 1920000 contained the execution of an irregular state change which transferred ~12 million ETH from the 'Dark DAO' and 'Whitehat DAO' contracts into the WithdrawDAO recovery contract. The fork itself took place smoothly, with roughly 85% of miners mining on the fork". It also told users who wanted to follow the old chain to update their clients but

run them with the `--oppose-dao-fork` flag.[22] EIP-779, which documented this fork after the event, fixes its character in one sentence: "Unlike other hard forks, the DAO Fork did not change the protocol; all EVM opcodes, transaction format, block structure, and so on remained the same. Rather, the DAO Fork was an 'irregular state change' that transferred ether balances from a list of accounts ('child DAO' contracts) into a specified account (the 'WithdrawDAO' contract)."[23]

Diagnosis

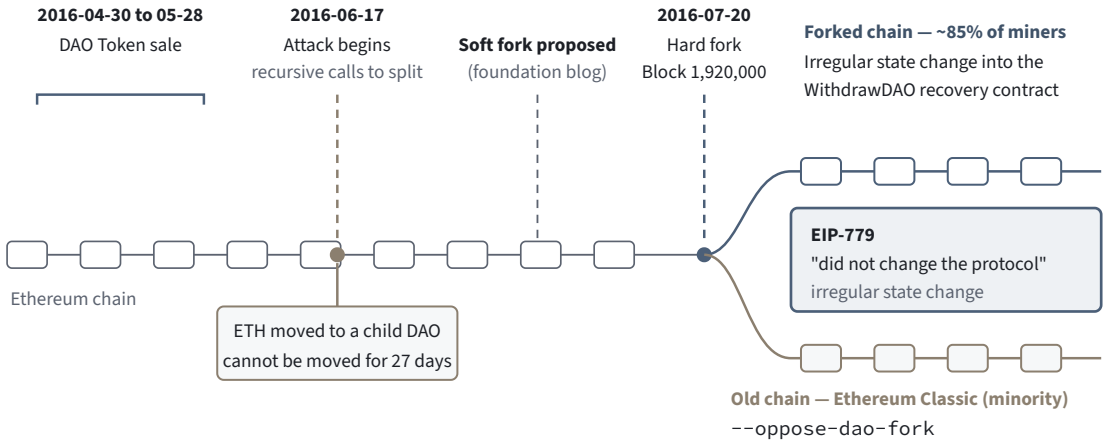


Figure 26-3. The course of The DAO case and the split at block 1,920,000 on 2016-07-20. Intervals between dates are not to scale, and ETH quantities are not shown.

① Two layers broke. One is **a flaw in the contract code of the application layer** (layer (c)), and the other is **governance**. The flaw itself was "an issue that affects the DAO specifically; Ethereum itself is perfectly safe." But the decision that followed, in which a majority of nodes updated their software and accepted an "irregular state change" moving the balances of particular accounts into another account, was a case of **changing the state of the ledger without changing the rules** of layer (a). The sentence of EIP-779 separates exactly these two. The consensus rules were not broken; participants rewrote the state within the rules (through the coordination of a client update). A minority did not follow, and the old chain survived. This is where Chapter 23 looked at "who decides which ledger is the real one" as a coordination game. ② The attack happened **on** the chain, as a state transition set by the contract code. ③ The code did what was written, and people made the decisions (i) to propose a soft fork and (ii) to adopt or reject a hard fork. ④ What followed were the hard fork and the coexistence of two chains, and, a year later, the SEC's investigation report. The SEC determined that "DAO Tokens are securities" while deciding not to pursue an enforcement action, and wrote: "The automation of certain functions through this technology, 'smart contracts,' or computer code, does not remove conduct from the purview of the U.S. federal securities laws."[24] The framework of that determination (Howey) is taken up in Chapter 27, and whether tokens are securities in Chapter 28.

> **? A LAWYER'S QUESTION**
>
> What was The DAO hard fork? Was it a **remedy** that recovered funds drained through the flaw? If so, a remedy on whose authority, by what procedure, and by what standard? Was it a **forfeiture**, in which a majority's software update took back what the attacker had acquired "according to the code"? If so, did the attacker have due process, and what did reversing a result the code permitted, on a platform that said "code is law," violate? Or was it a **change of contract**, in which a majority of the participants changed the rules among themselves? If so, was the dissenting minority's staying on the old chain a rescission of the contract, or a partition? EIP-779 writes that the DAO Fork "did not change the protocol" and was an "irregular state change." Does that distinction have legal meaning? This book does not answer, and points to the issue section of Part 5. → Chapter 28

4. Terra-Luna (2022): the reflexivity of the design and the issuer's statements

What the documents record

The mechanism was set out in Section 2 of Chapter 24, from the 2022 report of the Bank for International Settlements. The algorithmic stablecoin UST sought to keep its peg by being convertible into "one dollar's worth of Luna," and the decisive condition of that arrangement was that "users would only be willing to exchange UST into Luna if Luna's market capitalisation exceeded that of UST."[25] If that condition fails, redemptions mint more Luna, the increased supply lowers Luna's value, and that invites more redemptions: this feedback is the spiral that NB24-3 in Chapter 24 showed with a model. This chapter does not repeat that mechanism and looks only at what court documents wrote after May 2022.

In *SEC v. Terraform Labs Pte. Ltd. and Do Hyeong Kwon* (No. 1:23-cv-01346-JSR) in the United States District Court for the Southern District of New York, the court granted summary judgment on December 28, 2023, on the SEC's claim that the defendants violated Section 5 of the Securities Act through unregistered offers and sales of securities, and on April 5, 2024, the jury returned its verdict on violations of the antifraud provisions. The verdict form reads as follows. On the claim under Section 17 of the Securities Act, Terraform "Liable," with the mental state "Recklessly"; Kwon "Liable," "Intentionally." On the claim for intentional or reckless violation of Rule 10b-5 under the Securities Exchange Act, both "Liable." On control person liability for Terraform's violation, Kwon "Liable."[26] The final judgment of June 12, 2024, records that the defendants "consented to entry of this Final Judgment; waived findings of fact and conclusions of law; and waived any right to appeal," and set monetary remedies together with injunctions. For Terraform: disgorgement of USD 3,586,875,883, prejudgment interest of USD 466,952,423, and a civil penalty of USD 420,000,000 (the total of USD 4,473,828,306 is payable as provided for in the Chapter 11 plan confirmed in the bankruptcy case in the United States Bankruptcy Court for the District of Delaware, and Terraform agreed to seek confirmation of a plan that treats that amount as a general unsecured claim of the SEC; p. 7). For Kwon: disgorgement of USD 110,000,000 (jointly and

severally with Terraform), interest of USD 14,320,196, and a civil penalty of USD 80,000,000.[27] The final judgment also barred the defendants from participating in the purchase or sale of "any crypto asset securities, including but not limited to UST, MIR, LUNA, wLUNA, and LUNA 2.0," and required Terraform to "burn or destroy" the private keys in its possession to the wallets for those tokens.

The criminal case is *United States v. Do Hyeong Kwon* (No. 1:23-cr-00151-PAE) in the same court. This chapter writes from the judgment (Judgment in a Criminal Case, ECF No. 63) only. The defendant "pleaded guilty" to count 1s of the indictment (S1), "18 U.S.C. § 371 Conspiracy to Defraud," and to count 4s, "18 U.S.C. § 1343 and 2 Wire Fraud," and the date the offense ended is given as March 23, 2023, for both counts. The total sentence is 180 months (60 months on count 1s and 180 months on count 4s, to run concurrently; this follows the sentencing transcript, and the judgment's entry by count differs from it[29]). The court also directed credit for the time in custody since arrival in the United States on December 31, 2024, and for 17 months and 8 days of pre-extradition custody in Montenegro. No term of supervised release was imposed; the special assessment is USD 200, and forfeiture is governed by a separate order (Dkt. No. 47). The sentence was imposed on December 11, 2025, by Judge Paul A. Engelmayer.[28] Page 2 of the judgment contains "the Court encourages any court in the Republic of Korea that may sentence the defendant to take this Court's sentence into account," and a statement of the court's support for an application for transfer to Korea after half the custodial term has been served.

Diagnosis

① The layers that broke are **economic design** and **statements**. Chapter 24 showed the reflexivity of the design with a model; what the court documents added is the **determination** that the statements of the issuer and its head fell under the antifraud provisions of the securities laws. The jury found Terraform "Recklessly" and Kwon "Intentionally," and the criminal court accepted guilty pleas to conspiracy to defraud and wire fraud. What was stated, and how, is not written in the primary documents within this chapter's scope (the verdict form and the judgments), and so it is not given here. ② On and off the chain overlap. The swap module ran on the chain as designed, and the statements were made off the chain. ③ The code did as designed; drawing the spiral when the condition failed was part of the design. What people did was choose the design and explain it. ④ What followed were civil proceedings (summary judgment, jury verdict, final judgment by consent), criminal proceedings (guilty plea, 180 months), and insolvency proceedings (United States Bankruptcy Court for the District of Delaware). "What does a holder of a stablecoin have?" is the question Chapter 24 posed, and it is taken up in Chapter 28 and in the stablecoin column of Chapter 27.

5. FTX (2022): misuse of customer assets and the absence of internal controls

What the documents record

When the FTX Group filed for Chapter 11 in the United States Bankruptcy Court for the District of Delaware on November 11, 2022 (West Realm Shires Inc. on November 14), John J. Ray III, who became the new chief executive officer, filed the "Declaration of John J. Ray III in Support of Chapter 11 Petitions and First Day Pleadings" on November 17. He has handled large insolvencies including Enron, and in paragraph 5 he wrote: "Never in my career have I seen such a complete failure of corporate controls and such a complete absence of trustworthy financial information as occurred here. From compromised systems integrity and faulty regulatory oversight abroad, to the concentration of control in the hands of a very small group of inexperienced, unsophisticated and potentially compromised individuals, this situation is unprecedented."[30] The declaration divides the group into four "Silos" (the FTX US group, Alameda, Ventures, and the FTX.com group) and states that "Each of the Silos was controlled by Mr. Bankman-Fried."[31] It then lists the absence of controls item by item. Disbursements were approved on "an on-line 'chat' platform where a disparate group of supervisors approved disbursements by responding with personalized emojis,"[32] and with respect to digital assets there were "the use of an unsecured group email account as the root user to access confidential private keys and critically sensitive data," "the absence of daily reconciliation of positions on the blockchain," "the use of software to conceal the misuse of customer funds," "the secret exemption of Alameda from certain aspects of FTX.com's auto-liquidation protocol," and "the absence of independent governance as between Alameda … and the Dotcom Silo."[33] That at least USD 372 million was transferred without authorization on the petition date itself, and that FTT tokens worth approximately USD 300 million were afterward additionally minted by "an unauthorized source," are also facts the declaration states.[34]

The criminal case is *United States v. Samuel Bankman-Fried* (No. 1:22-cr-00673-LAK) in the United States District Court for the Southern District of New York. The judgment (Judgment in a Criminal Case, ECF No. 424) records that the defendant "was found guilty" on all of counts 1 through 7 of the (S6) indictment, and gives the offenses as wire fraud (counts 1 and 3, 18 U.S.C. §§ 1343 and 2), wire fraud conspiracy (counts 2 and 4, § 1349), securities fraud conspiracy (count 5, § 371), commodities fraud conspiracy (count 6, § 371), and money laundering conspiracy (count 7, § 1956(h)). The sentence was imposed on March 28, 2024, by Judge Lewis A. Kaplan.[35] The United States Court of Appeals for the Second Circuit affirmed the judgment below on June 12, 2026. The appellate opinion summarizes the first-instance sentence as "25 years' imprisonment, to be followed by three years of supervised release" and "a forfeiture of approximately $11 billion," and states the government's theory of the case as follows: "The government's theory at trial was that Bankman-Fried secured investments in FTX by promising customers that their funds would be secure on the platform and used only for cryptocurrency transactions on FTX. Instead, Bankman-Fried freely transferred investor funds from FTX customer accounts to Alameda and elsewhere where he used those funds for purposes investors had not authorized, including for his own

personal benefit." It then describes the evidence as "conservatively stated, robust," and writes that "while Bankman-Fried assured customers, investors, regulators and the public that FTX customer funds were safe, he transferred billions of dollars from FTX customer accounts to Alameda and elsewhere."[36] This book has not verified whether a petition for a writ of certiorari has been filed (as of 2026-10).

Diagnosis

① The layers that broke are **fraud and internal controls**. Not the technology, not the economic design, not the governance of a chain. The items the declaration lists (access to private keys through a shared email account, the absence of daily reconciliation of positions on the chain, software to conceal the misuse of customer funds, the exemption of a particular account from auto-liquidation) all happened **inside the company**, in layer (c). ② Off the chain. The "balances" inside the exchange were book numbers of the same kind as at Mt.Gox, and the chain did not guarantee them, as at Mt.Gox. ③ If the code did anything, it did what people told it to: "the use of software to conceal the misuse of customer funds." What people did is everything. ④ Chapter 11, guilty verdicts on all 7 counts and a 25-year sentence, and affirmance on appeal followed. That the two exchange cases broke in the same place eight years apart shows that the layer distinction of Section 1 is not a question of technology but a question of **who keeps the books, and how**. That this is where Articles 6 and 7 of Korea's Act on the Protection of Virtual Asset Users place their obligations was noted in Section 2.

6. The four cases in one table

The table below gathers the diagnoses of Sections 2–5. The parentheses in "The layer that broke" are the (a), (b), and (c) of Section 1.

	MT.GOX (2014)	THE DAO (2016)	TERRA-LUNA (2022)	FTX (2022)
① The layer that broke	Exchange books and internal controls (c). The layer of the 2014-02 loss not decided by a court: the exchange cited (b) transaction malleability and in its later announcement of February 28 also mentioned the possibility of hacking; the empirical study found the amount explained by malleability very small	Flaw in the contract code (c) + governance (a majority decision that changed the state without changing the rules)	Reflexivity of the economic design (Chapter 24) + statements of the issuer and its head (securities-fraud verdict; guilty pleas to conspiracy to defraud and wire fraud)	Fraud + absence of internal controls (c)
② On or off the chain	Off (manipulation of the books). The layer of the 2014-02 loss not decided by a court	On (state transition of contract code) → the fork, too, was a change of ledger state	On (swap module) and off (statements)	Off (company books, banks, private key management). The unauthorized transfers on the petition date were transfers on the chain, but the declaration says their circumstances are under investigation
③ Code and people	Code: automatic confirmation of remittances → balance update. People: entered unbacked balances by hand ("人が…直接入力するようなことは想定されていなかった")	Code: carried out the recursive calls as written. People: proposed a soft fork; adopted or rejected a hard fork	Code: as designed (a spiral when the condition fails). People: choice of design, and statements	Code: software to conceal the misuse (on people's instructions). People: everything
④ What followed	Petition for civil rehabilitation (2014-02-28) → dismissal and provisional administration (04-16) → commencement of bankruptcy (04-24) → commencement of civil rehabilitation on creditors' petition (2018-06-22). Criminal: guilty at first instance (unauthorized creation of records), not guilty (embezzlement); appeal dismissed. Final appeal and finality not verified	Hard fork (block 1,920,000, 2016-07-20) · two chains coexisting · SEC investigation report (2017; determination that the tokens are securities, no enforcement action)	SEC civil: summary judgment (2023-12-28), jury verdict (2024-04-05), final judgment by consent (2024-06-12). Criminal: guilty plea, 180 months (2025-12-11). Insolvency (Bankr. D. Del.)	Chapter 11 (2022-11-11; WRS 11-14). Criminal: guilty on 7 counts, 25 years (2024-03-28), affirmed on appeal (2026-06-12). Petition for certiorari not verified

	MT.GOX (2014)	THE DAO (2016)	TERRA-LUNA (2022)	FTX (2022)
Primary documents	Judgments of the Tokyo District Court (東京地裁), March 15, 2019, and the Tokyo High Court (東京高裁), June 11, 2020; bankruptcy commencement decision; company announcements; trustee reports	SEC Release 81207, EIP-779, 2 foundation announcements	ECF 229 and 273 (civil), ECF 63 (criminal)	Ray declaration D.I. 24, ECF 424, 2d Cir. opinion

7. The scope of "the blockchain itself did not break"

Now the sentence held back in Section 1 can be written, with its scope set. **None of these four is a case in which (a), the consensus rules and ledger integrity, was broken by an outside attack and the ledger of Bitcoin or Ethereum reversed.** What the documents found at Mt.Gox and FTX was the books off the chain and matters inside a company; what the court decided at Terra was statements; and the attack on The DAO was a matter of an application-layer contract. In none of the four cases is there a primary document stating that the property Part 1 of this book explained as the "cost of tampering" was broken. Terra, though, happened on its own chain, not on Bitcoin or Ethereum, and the primary documents of this chapter do not deal with that chain's consensus layer.

Two things must be added, though. First, for Mt.Gox it has to be put more narrowly. At the time the exchange cited transaction malleability, a property of layer (b), as the cause, but in its later announcement of February 28 it also mentioned the possibility of hacking; the empirical analysis found the amount explained by malleability attacks to be very small; the trustee said the full picture would be difficult to understand; and the criminal judgments at first instance and on appeal did not decide the cause of the loss. So using the sentence "the blockchain did not break" **affirmatively** about the Mt.Gox loss has no backing in any court determination. What this chapter can say goes as far as "there is no document saying that layer (a) broke." Second, The DAO hard fork was not a breaking of layer (a), but it was a case in which **the state of the layer (a) ledger was changed by a majority decision**. EIP-779's sentence that the DAO Fork "did not change the protocol" and the fact that "the state was rewritten" are not a contradiction but a pair, and that pair is exactly what the question box above asks about. What the cost of tampering protects is the ledger against "those who do not follow the rules," not against "a majority that has decided to change the rules." Here once more you can see what Chapter 7 meant when it wrote that immutability is a matter not of mathematics but of cost.

8. Summary

Put the four cases on the same table and the narrative disappears, leaving the structure. The two exchanges broke in books off the chain, and the differences between them were whether the person who falsified the books was a single representative or an organization, and whether the law required separate management. For the token issuer, when the condition of the design failed, the code drew the spiral as designed, and what the court decided was not the design but the statements. The contract code was executed as written and betrayed its author's intention, and what reversed it was not code but the coordination of people. In none of the cases is there a primary document saying that the consensus rules of Part 1 were broken by an outside attack, and that sentence is true only within the scope set in Section 1. What the lawyer takes away is the **columns** of this table. Only after sorting out which layer broke can you see where property law, securities law, insolvency law, or criminal law comes in, and Chapter 27 and Chapter 28 survey those places. And that which law comes in depends on which name these four cases go under, "가상자산" (gasang jasan, "virtual asset"), "crypto-asset," "digital asset," "暗号資産" (angō shisan, crypto-asset), or "cryptoasset," is the subject of Chapter 30.

Sources for this chapter

- Tokyo District Court (東京地方裁判所), judgment of March 15, 2019 (平成31年3月15日 判決), 平成27年(刑わ)第2274号·第2842号, 平成28年(刑わ)第2248号 (私電磁的記録不正作出, 同供用, 業務上横領被告事件): operative part (主文), 「罪となるべき事実」 (facts constituting the offenses), 「事実認定の補足説明」 (supplementary explanation of the fact-finding) 第１–第６, 「量刑の理由」 (reasons for sentencing). (full HTML text from a case-law database). The 「判示事項」 field is the database's editorial product and was not quoted. Whether a final appeal was filed, and the date of finality: not verified.
- Tokyo High Court, 4th Criminal Division (東京高等裁判所第４刑事部), judgment pronounced June 11, 2020 (令和2年6月11日 宣告 判決), 令和元年(う)第948号: operative part, 第１–第４. (courts.go.jp, 14 pages).
- Tokyo District Court, 20th Civil Division (東京地方裁判所民事第２０部), decision commencing bankruptcy proceedings of April 24, 2014 (平成26年4月24日 破産手続開始決定), 平成２６年（フ）第３８３０号. (scan, OCR).
- Announcements of 株式会社ＭＴＧＯＸ (MTGOX Co., Ltd.): 「民事再生手続開始の申立てに関するお知らせ」 (2014-02-28, Japanese and English), 「当社保有ビットコインの残高に関するお知らせ」 (2014-03-20), 「民事再生手続開始申立ての棄却、保全管理命令のお知らせ」 (2014-04-16). Used only as the debtor's statements.
- Reports and announcements of the bankruptcy trustee and civil rehabilitation trustee (破産管財人·再生管財人) 小林信明 (Nobuaki Kobayashi): English parts of the reports for the first (2014-07-23), sixth (2016-05-25), and tenth (2018-03-07) creditors' meetings; "Announcement of Commencement of Civil Rehabilitation Proceedings" (2018-06-22).
- MtGox Team, announcement of 2014-02-07 halting withdrawals: checked against the copy

reproduced in a Business Insider article of 2014-02-07 (**secondary reproduction**); press release of 2014-02-10, "Addressing Transaction Malleability": quoted from the sentences reproduced on p. 6 of Decker and Wattenhofer (2014) (the original URL no longer exists; its location was confirmed through Wayback snapshot 20140210122955).

- Christian Decker and Roger Wattenhofer, "Bitcoin Transaction Malleability and MtGox," arXiv:1403.6676 (2014); *ESORICS 2014*, LNCS, pp. 313–326, DOI 10.1007/978-3-319-11212-1_18: §3 Timeline (pp. 6–7), §4.1 (p. 9), §6 Conclusion (p. 12). (page numbers of the arXiv version).
- SEC, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Exchange Act Release No. 81207 (July 25, 2017): pp. 1–3, 7–9, 11–16, footnotes 2 and 32.
- Vitalik Buterin, "CRITICAL UPDATE Re: DAO Vulnerability" (2016-06-17); "Hard Fork Completed" (2016-07-20), Ethereum Foundation Blog. HTML (reopened and checked word for word on 2026-10-25).
- Casey Detrio, "EIP-779: Hardfork Meta: DAO Fork" (created 2017-11-26, Final). https://eips.ethereum.org/EIPS/eip-779 (HTML, checked word for word on 2026-10-25).
- Bank for International Settlements, *Annual Economic Report 2022*, ch. III, Box A (p. 82), the same place as in Chapter 24.
- *SEC v. Terraform Labs Pte. Ltd. and Do Hyeong Kwon*, No. 1:23-cv-01346-JSR (S.D.N.Y.): Verdict, ECF No. 229 (2024-04-05); Final Judgment, ECF No. 273 (2024-06-12, scan, OCR). `sdny-2024-sec-v-terraform-final-judgment.pdf` (RECAP). SEC press release 2024-73 and the Litigation Release could not be opened because of bot blocking and were not used.
- *United States v. Do Hyeong Kwon*, No. 1:23-cr-00151-PAE (S.D.N.Y.), Judgment in a Criminal Case, ECF No. 63 (imposed 2025-12-11, filed 2025-12-12). (PACER, scan, OCR).
- *United States v. Do Hyeong Kwon*, No. 1:23-cr-00151-PAE (S.D.N.Y.), Transcript of Proceedings re: Sentence held on 12/11/2025, ECF No. 67 (filed 2025-12-30), p. 163. (CourtListener RECAP copy, 167 pages).
- Declaration of John J. Ray III, *In re FTX Trading Ltd., et al.*, Case No. 22-11068 (JTD) (Bankr. D. Del.), D.I. 24 (2022-11-17): ¶¶1, 4–10, 62–71. (RECAP).
- *United States v. Samuel Bankman-Fried*, No. 1:(S6)22-CR-673-001 (LAK) (S.D.N.Y.), Judgment in a Criminal Case, ECF No. 424 (imposed 2024-03-28). (RECAP, scan, OCR).
- *United States v. Bankman-Fried*, No. 24-961-cr (2d Cir. June 12, 2026), DktEntry 73.1: pp. 1–11. (RECAP). Whether a petition for a writ of certiorari was filed: not verified.
- Act on the Protection of Virtual Asset Users (가상자산 이용자 보호 등에 관한 법률) (current version Act No. 21857, in force October 2, 2026), Articles 6 and 7. Korean Law Information Center (Open API, accessed 2026-10-25).
- The Korean names of foreign courts in the Korean edition follow the courts' 「외국 법원 및 사법관련 국제기구 번역 가이드라인」 [Guidelines on the Korean translation of names of foreign courts and judicial international organizations] (review-comment edition,): 도쿄지방재판소 (Tokyo District Court), 도쿄고등재판소 (Tokyo High Court), 미국 연방뉴욕남부지방법원 (United States District Court for the Southern District of New York),

미국 연방제2항소법원 (United States Court of Appeals for the Second Circuit), 미국 연방델라웨어파산법원 (United States Bankruptcy Court for the District of Delaware).

- This chapter has no code or notebook (by design). No price, market capitalization, or rate-of-return data were used, and each amount is given once, as written in judgments, decisions, and debtor announcements.

Notes

1. 株式会社ＭＴＧＯＸ (MTGOX Co., Ltd.), 「民事再生手続開始の申立てに関するお知らせ」 [Notice concerning the petition for commencement of civil rehabilitation proceedings] (平成26年2月28日, February 28, 2014), pp. 1–2, 「１．当社の財務状況と申立てに至った経緯及び理由」. Original: 「資産総額　3,841,866,163 円 / 流動負債総額　6,501,119,371 円」; 「当社の流動負債が増大した背景には，ビットコインの消失, 預り金の消失という原因が考えられ」; 「平成２６年２月初め頃, ビットコインのシステムのバグを悪用した不正アクセスにより，ビットコインの送金（ビットコインの引出）が正常に完了しない取引が増え」; 「ユーザーの取引履歴上のビットコイン保有高であるおよそ７５万ビットコイン，及び, 当社自身の取引履歴上の保有ビットコインである約１０万ビットコインのほぼ全てがなくなっていることが判明しました」; 「金額は調査中であり変動する可能性が高いが最大約２８億円程度」; 「第三者によるハッキングによる被害を含めて，複数の原因があると見込まれるため」. Case number 平成２６年（再）第１２号. (Japanese original), `-en.pdf` (the company's English translation, marked "[Translation from the Japanese]"). This document is the debtor's own statement, not a determination by the court that its contents are true.

2. 株式会社ＭＴＧＯＸ, 「当社保有ビットコインの残高に関するお知らせ」 [Notice concerning the balance of bitcoin held by the company] (平成26年3月20日, March 20, 2014), p. 1. Original: 「平成２６年３月７日（金), 当社が平成２３年６月以前に利用していた旧式ウォレット内に約２０万ＢＴＣ（１９万９９９９．９９ＢＴＣ）が残存していることを確認いたしました」; 「消失したＢＴＣの数量は約６５万ＢＴＣとなるものと見込んでおります（ただし, ＢＴＣの消失の原因や消失したＢＴＣの数量は現在調査中ですので, 調査の結果により数量が変動する可能性があります。)」.

3. Tokyo District Court, 20th Civil Division (東京地方裁判所民事第２０部), decision commencing bankruptcy proceedings (破産手続開始決定) of April 24, 2014 (平成26年4月24日), 平成２６年（フ）第３８３０号, p. 1. Original: 「債務者株式会社MTGOXについて破産手続を開始する。／理由／一件記録によれば, 債務者が支払不能の状態にあることが認められる。」 Bankruptcy trustee (破産管財人): 小林信明 (Nobuaki Kobayashi). The decision is a scan and was read by OCR (the case number follows the OCR of the Japanese page and the text layer at the head of the trustee's 2014 report, 「３８３０」, not the OCR of the English translation page, 「88380」). The dismissal and the provisional administration order are from the company's announcement 「民事再生手続開始申立ての棄却、保全管理命令のお知らせ」 (平成26年4月16日, April 16, 2014). `mtgox-2014-rehabilitation-dismissal-notice.pdf`.

4. Bankruptcy trustee's reports, English part. Report for the first creditors' meeting (2014-07-23), "V. Investigation of disappearance of BTC, etc.": "I began investigation of the existing assets of the bankrupt entity, background behind the disappearance of BTC, etc. by delegating such work to Deloitte Touche Tohmatsu LLC and ReEx Accounting Firm. Such investigation still continues now and the completion date is undetermined as of today." Report for the sixth meeting (2016-05-25), "III. 1. Status of Investigation": "Due to the circumstances, such as the limited amount of information that exists, I believe it will be difficult to understand the full picture; however, I am planning to continue these investigations, as much as possible." Report for the tenth meeting (2018-03-07), "IV.": "On July 26, 2017, the U.S. Department of Justice announced that Alexander Vinnik, manager of a cryptocurrency exchange, was arrested and indicted for involvement in money laundering of more than 4 billion dollars. The allegations in the indictment include the fact that he obtained funds by hacking MtGox. I have contacted the U.S. Department of Justice through a U.S. law firm and have made a request for information." `mtgox-trustee-2016-sixth-creditors-meeting-report.pdf`, `mtgox-trustee-2018-tenth-creditors-meeting-report.pdf`. The original documents of the U.S. criminal proceedings were not checked; this book records only that the trustee reported the fact.

5. Civil Rehabilitation Trustee Nobuaki Kobayashi, "Announcement of Commencement of Civil Rehabilitation

Proceedings" (2018-06-22, updated 2018-10-26), p. 1: "some of the creditors of MTGOX Co., Ltd. ('MTGOX') filed a petition for commencement of civil rehabilitation proceedings against MTGOX with the Tokyo District Court on November 24, 2017."; "at 5:00 p.m. on June 22, 2018, the Tokyo District Court issued an order of the commencement of civil rehabilitation proceedings for MTGOX. As a result, the previously ongoing bankruptcy proceedings were stayed." Case designation "Tokyo District Court 2017 (sai) no. 35".

6. MtGox Team, announcement of 2014-02-07. The original archive (web.archive.org snapshot 20140207054536) could not be opened for this edition, and the text was checked against the copy reproduced in a Business Insider article (Joe Weisenthal, "Bitcoin Plunges As Major Exchange Mt. Gox Halts All Withdrawals," 2014-02-07). **This is a secondary reproduction.** The reproduced original: "In our efforts to resolve the issue being encountered by various bitcoin withdrawals, it was determined that the increase in the flow of withdrawal requests has hindered our efforts on a technical level. To understand the issue thoroughly, the system needs to be in a static state. In order for our team to resolve the withdrawal issue it is necessary for a temporarily pause on all withdrawal requests to obtain a clear technical view of the current processes." (https://www.businessinsider.com/mtgox-halts-withdrawals-2014-2 , accessed 2026-10-25). Decker and Wattenhofer (2014), p. 6, also write that this announcement does not mention transaction malleability ("Specifically [10] does not mention transaction malleability").

7. MtGox Team, press release (2014-02-10), "Addressing Transaction Malleability." The original URL `https://www.mtgox.com/press_release_20140210.html` no longer exists, and its location was confirmed through Wayback snapshot 20140210122955. The sentences quoted in this chapter are as reproduced in Christian Decker and Roger Wattenhofer, "Bitcoin Transaction Malleability and MtGox," arXiv:1403.6676 (2014), p. 6: "Addressing Transaction Malleability: MtGox has detected unusual activity on its Bitcoin wallets and performed investigations during the past weeks. This confirmed the presence of transactions which need to be examined more closely. Non-technical Explanation: A bug in the bitcoin software makes it possible for someone to use the Bitcoin network to alter transaction details to make it seem like a sending of bitcoins to a bitcoin wallet did not occur when in fact it did occur. Since the transaction appears as if it has not proceeded correctly, the bitcoins may be resent. MtGox is working with the Bitcoin core development team and others to mitigate this issue." The same page sums up that the February 10 announcement named malleability as the "sole cause." (arXiv version; published version in *ESORICS 2014*, LNCS, pp. 313–326, DOI 10.1007/978-3-319-11212-1_18).

8. Decker and Wattenhofer (2014), p. 12, §6 Conclusion. The original is as quoted in the text, and the last sentence is: "Even if all of these attacks were targeted against MtGox, MtGox needs to explain the whereabouts of 849,600 bitcoins." The measurement method is in §4 (in the set of conflicting transactions collected by the network-observing node, pairs that differ only in signature encoding were classified as malleability attacks). The data are network observations from January 2013 on, and the authors do not guarantee that the node saw every transaction. This book reads these figures as "an observed upper bound on malleability attacks" and does not use them to positively identify the cause of the loss.

9. Tokyo District Court (東京地方裁判所), judgment of March 15, 2019 (平成31年3月15日 判決), 平成27年(刑わ)第2274号·第2842号, 平成28年(刑わ)第2248号, 6th Criminal Division (刑事第６部), presiding judge (裁判長裁判官) 中山大行, operative part (主文). Original: 「被告人を懲役２年６月に処する。／この裁判が確定した日から４年間その刑の執行を猶予する。／…／本件公訴事実中，各業務上横領，平成２９年３月１０日付け予備的訴因並びに罪名及び罰条追加請求書記載の各業務上横領（予備的訴因１）及び各会社法違反（予備的訴因２）の点については，被告人は無罪。」 The date of preparation at the end of the judgment reads 「平成３１年３月１８日」, and the date of pronouncement is March 15 (the prosecution sought 懲役１０年, 求刑). Original: (full HTML text from a case-law database; the 「判示事項」 field is the database's editorial product and was not used).

10. Tokyo High Court, 4th Criminal Division (東京高等裁判所第４刑事部), judgment pronounced June 11, 2020 (令和2年6月11日 宣告 判決), 令和元年(う)第948号 (私電磁的記録不正作出，同供用被告事件), p. 1. Original: 「本件控訴を棄却する。」；「なお原審では，被告人は，…業務上横領（予備的訴因 会社法違反）の罪にも問われ，原判決はこれらの罪については無罪としたが，この点につき検察官は控訴しなかった。」 The date at the end of the judgment reads 「令和２年６月１８日」. (public PDF from courts.go.jp, 14 pages).

11. Tokyo District Court judgment of March 15, 2019, 「事実認定の補足説明」 (supplementary explanation of the fact-finding), 第１の２(５)ウ. Original: 「本件取引システムにおいては，利用者間でビットコイン取引が行われても，本件取引システム上の口座残高に増減が生じるだけで，本件各銀行口座等で管理されている金銭残高に実際の変動が生じることはなく，また，ビットコインも同様に，

本件取引システム上の口座残高に変動は生じるが，ブロックチェーン上に変動は生じない。そして，利用者が，Ｂ社に対し，金銭の返金やビットコインの送付を請求し，Ｂ社が，金銭を利用者の銀行口座に送金し，あるいは，ビットコインをＢ社が管理するビットコインアドレスから利用者のビットコインアドレスに送付した段階で，実際の変動が生じる。」 Same place, (４): money remitted by users and the company's own money were not managed separately in the bank accounts, and the company's expenses were also paid from those accounts (「利用者が送金した金銭と自社の金銭が分別して管理されておらず，かつ，本件各銀行口座等から，Ｂ社の経費等が支払われていた」).

12. Tokyo District Court judgment of March 15, 2019, 第１の２(３), in parentheses. The original is as quoted in the text. The same section defines and uses, for bitcoin, 「ブロックチェーン上に記録されている状態を「存在」という」 (第１の２(１)). Working gloss: "the state of being recorded on the blockchain is called 'existence' (「存在」)."

13. Tokyo District Court judgment of March 15, 2019, 第２の２(２)–(４). Original (excerpts):
「Ｅ社を買収したＤ社は，利用者に対し，
ブロックチェーン上には存在しない１万７０００ＢＴＣの返還債務を引き受けることとなった」;
「残高約１３米ドルのＣ口座の米ドル口座に係る口座履歴情報に，
残高を１５万米ドル増加させる旨の情報を入力し...合計約１５万米ドルで合計約１万８０００ＢＴＣを購入した。
被告人は，同月２０日，Ｃ口座のビットコイン口座の残高を１万７５００ＢＴＣ削除した」;
「平成２３年１１月１４日から平成２４年４月１３日までの間，合計８回にわたって，
ビットコインをＢ社が管理するビットコインアドレスに送付することなく，...
残高を３０万ＢＴＣ増加させる旨の情報を入力し...利用者との間で，合計約３０万ＢＴＣを売却し，
その対価として，Ｃ口座の米ドル口座に約１３０万米ドルを得た」;「３０万ＢＴＣ増加行為等の結果，
本件取引システム上では，
ブロックチェーン上には存在しないビットコイン３０万ＢＴＣが利用者間で取引されることとなった。そこで，
被告人は，同３０万ＢＴＣを回収する目的で，...平成２５年２月１４日から同年９月２７日までの間，
合計２１回にわたり，...残高を合計３３５０万米ドル増加させる旨の情報を入力し...，利用者から，
代金約２９７７万米ドルで，合計約３０万ＢＴＣを購入した後，
Ｃ口座のビットコイン口座の残高を約３０万ＢＴＣ削除した」. Pages 2–3 of the appellate judgment repeat the same facts as 「原判決が認定した前提事実等の骨子」 (the outline of the premise facts found by the judgment below) (that judgment writes 「Ｔ社」 and 「Ｍ社」 instead of 「Ｄ社」 and 「Ｂ社」, and 「Ａ口座」 instead of 「Ｃ口座」). The amounts and quantities in the text are those written in the judgment, and this chapter gives each of them once.

14. Tokyo District Court judgment of March 15, 2019, 第２の３(３)ア(ア), (イ), (オ). The originals are as quoted in the text; (オ) in full: 「以上を総合すれば，本件取引システム上では，
ブロックチェーン上に存在しないビットコインが取引されることは想定されておらず，
利用規約及び利用者間のビットコインの取引の実態並びにビットコインの性質等からすれば，
本件取引システムの口座残高は，利用者に対し，
ブロックチェーン上に存在するビットコインに裏付けられたものであることを保証しており，
利用者もそのように認識していたと認められる。したがって，Ｂ社の意思は，
本件取引システム上の利用者の口座残高に相当するビットコインがＢ社管理のブロックチェーン上にも存在すべきであり，Ｂ社管理のビットコインアドレスにおけるビットコインの受送付等による裏付けがある場合に限って，
本件取引システム上の利用者の口座残高を増加させるべきであるというものであり，
被告人に与えられた権限もこの範囲に限定されていたと認められる。」 Working gloss: "Taking all of the above together, it is found that the Trading System did not contemplate the trading of bitcoin that does not exist on the blockchain, that, in light of the terms of use, the actual state of bitcoin trading between users, the nature of bitcoin, and so on, the account balances of the Trading System guaranteed to users that they were backed by bitcoin existing on the blockchain, and that users also understood them that way. Accordingly, it is found that Company B's will was that bitcoin corresponding to users' account balances on the Trading System should also exist on the blockchain under Company B's management, and that users' account balances on the Trading System should be increased only where there was backing by the receipt or sending, etc., of bitcoin at bitcoin addresses managed by Company B, and that the authority given to the defendant was also limited to this scope." The English original of the terms-of-use provision: "Seller warrants that the Bitcoins offered to sell or to transfer correspond to actual Bitcoins."

15. Tokyo High Court judgment of June 11, 2020, pp. 8–9 (第３の１) and pp. 10–11 (第３の２(３)). Original: 「...とした原判決の判断は，概ね同旨をいうものとして是認できる」;「本件取引システムは，...
極めて大量の取引を迅速かつ正確に行うための自動プログラムであり，被告人が行ったような，

人が個々の取引やその履歴のデータ等を直接入力するようなことは想定されていなかったと認められる」;
「電磁的記録の証明機能は，それが作出されるべきシステムの設置運営主体の意思に基づき,
正当に作出されることが当然の前提とされているもので,
このように正当に作られたものであることの信頼を確保する必要があるという観点から解釈すべきと解される」.
Working gloss of the second and third: "it is found that the Trading System … is an automatic program for carrying out an extremely large volume of transactions quickly and accurately, and that it was not contemplated that a person would directly input individual transactions, their history data, and the like, as the defendant did"; "the evidentiary function of electromagnetic records is premised, as a matter of course, on their being created legitimately based on the will of the entity that set up and operates the system in which they are to be created, and it should be interpreted from the viewpoint of the need to secure trust that they were so legitimately made."

16. Tokyo District Court judgment of March 15, 2019, 第２の３(２)イ, 第３の２, 第４, 第５. Original:
「利用者がＢ社の指定する銀行口座に送金した金銭は，Ｂ社に帰属すると認められるのであり,
金銭に関しては,
本件取引システム上の利用者の口座残高に相当する金銭がＢ社の本件各銀行口座等にも存在すべきであるとはいえず」;「本件当時頃，前記（ア）の管理方法を禁止する旨の法令はなく，利用規約上も,
分別管理を前提とする規定…はなかった」;「本件各振込送金は,
被告人に対する貸付けとして行われたものであり，返済の現実的可能性があると認めるのが相当である」. The details of the transfers (September to December 2013: JPY 5 million, JPY 5 million, and JPY 10 million to accounts in the defendant's name and elsewhere, JPY 6,009,245 for the purchase of furniture, and JPY 315 million for the acquisition of a business) are in 第３の１. This chapter conveys only the structure of the grounds for acquittal and does not assess whether it was right.

17. Act on the Protection of Virtual Asset Users (가상자산 이용자 보호 등에 관한 법률) (enacted as Act No. 19563; current version Act No. 21857, in force October 2, 2026), Article 6(1) and Article 7(2). Original:
「가상자산사업자는 이용자의 예치금(…)을 고유재산과 분리하여 「은행법」에 따른 은행 등 대통령령으로 정하는 공신력 있는 기관(이하 "관리기관"이라 한다)에 대통령령으로 정하는 방법에 따라 예치 또는 신탁하여 관리하여야 한다.」;「가상자산사업자는 자기의 가상자산과 이용자의 가상자산을 분리하여 보관하여야 하며, 이용자로부터 위탁받은 가상자산과 동일한 종류와 수량의 가상자산을 실질적으로 보유하여야 한다.」 Korean Law Information Center (국가법령정보센터) of the Ministry of Government Legislation (법제처) (Open API, MST 290735, accessed 2026-10-25). This book does not say that these provisions were enacted with Mt.Gox in mind; the legislative materials were not checked.

18. SEC, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Exchange Act Release No. 81207 (July 25, 2017), pp. 1–3. Original: "The DAO is one example of a Decentralized Autonomous Organization, which is a term used to describe a 'virtual' organization embodied in computer code and executed on a distributed ledger or blockchain." (p. 1); "From April 30, 2016 through May 28, 2016, The DAO offered and sold approximately 1.15 billion DAO Tokens in exchange for a total of approximately 12 million Ether ('ETH')" (pp. 2–3). Footnote 2 of the report states that the document "does not constitute an adjudication of any fact or issue": it is an investigation report, not a finding of fact. The dollar equivalents (p. 3) are not reproduced.

19. SEC (2017), pp. 7–8: "Before any proposal was put to a vote by DAO Token holders, it was required to be reviewed by one or more of The DAO's 'Curators.' … According to the White Paper, the Curators of a DAO Entity had 'considerable power.' The Curators performed crucial security functions and maintained ultimate control over which proposals could be submitted to, voted on, and funded by The DAO."

20. SEC (2017), p. 9, "4. Security Concerns, The 'Attack' on The DAO, and The Hard Fork," and footnote 32. The original is as quoted in the text. Footnote 32: "A minority group, however, elected not to adopt the new Ethereum Blockchain created by the Hard Fork because to do so would run counter to the concept that a blockchain is immutable. Instead they continued to use the former version of the blockchain, which is now known as 'Ethereum Classic.'" Same page: "The Hard Fork had the effect of transferring all of the funds raised (including those held by the Attacker) from The DAO to a recovery address, where DAO Token holders could exchange their DAO Tokens for ETH."

21. Vitalik Buterin, "CRITICAL UPDATE Re: DAO Vulnerability," Ethereum Foundation Blog (2016-06-17). The original is as quoted in the text, and it continues: "The leaked ether is in a child DAO …; even if no action is taken, the attacker will not be able to withdraw any ether at least for another ~27 days (the creation window for the child DAO)."; "A software fork has been proposed, (with NO ROLLBACK; no transactions or blocks will

be 'reversed') which will make any transactions that make any calls/callcodes/delegatecalls that reduce the balance of an account with code hash 0x7278d050... lead to the transaction ... being invalid". https://blog.ethereum.org/2016/06/17/critical-update-re-dao-vulnerability (HTML, reopened and checked word for word on 2026-10-25). The address of the child DAO to which the funds were moved is given in this post and also appears in the EIP-779 list, but the hash of the attack transaction was not checked against any primary document and is therefore not given in the text.

22. Vitalik Buterin, "Hard Fork Completed," Ethereum Foundation Blog (2016-07-20). The original is as quoted in the text, and it continues: "The recovery contract is already returning DAO token holders' ether; about 4.5 million ETH has been sent to DAO token holders, and about 463000 ETH is in the control of the curator"; "If any users continue to be interested in following the non-fork chain, they should still update, but run with the --oppose-dao-fork flag enabled, though they should beware of transaction replay attacks". https://blog.ethereum.org/2016/07/20/hard-fork-completed (HTML, checked word for word on 2026-10-25). "~12 million ETH" is a different quantity from the SEC report's "approximately 3.6 million ETH" (the amount the attacker moved): what the fork moved was the entire balance of the contracts of The DAO family, including the attacker's.

23. Casey Detrio, "EIP-779: Hardfork Meta: DAO Fork," Ethereum Improvement Proposals (created 2017-11-26, Status Final), Abstract. The original is as quoted in the text. Specification: "Codename: DAO Fork / Activation: Block == 1,920,000 on Mainnet"; at block 1,880,000 it fixes the list L of The DAO (0xbb9bc244d798123fde783fcc1c72d3bb8c189413), extraBalance, and the child DAOs, and moves their balances to WithdrawDAO (0xbf4ed7b27f1d666546e30d74d50d173d20bca754). https://eips.ethereum.org/EIPS/eip-779 (HTML, checked word for word on 2026-10-25).

24. SEC (2017), pp. 1–2. Original: "The Commission has determined not to pursue an enforcement action in this matter based on the conduct and activities known to the Commission at this time."; "Based on the investigation, and under the facts presented, the Commission has determined that DAO Tokens are securities under the Securities Act of 1933 ('Securities Act') and the Securities Exchange Act of 1934 ('Exchange Act')." (p. 1); "The automation of certain functions through this technology, 'smart contracts,' or computer code, does not remove conduct from the purview of the U.S. federal securities laws." (p. 2). The report's application of Howey (pp. 11–15) rests on the "undeniably significant" efforts of the Curators and Slock.it (p. 12) and on the limits of token holders' voting rights (pp. 13–15).

25. Bank for International Settlements, *Annual Economic Report 2022*, ch. III, p. 82, Box A "The collapse of the TerraUSD stablecoin". The original is reproduced in footnote 4 of Chapter 24. The BIS report is an analytical report of an organization of central banks; among this book's standards for primary documents (judgments, official investigations, insolvency documents), it is not counted as an "official investigation" and is used only as a source for the explanation of the mechanism.

26. *SEC v. Terraform Labs Pte. Ltd. and Do Hyeong Kwon*, No. 1:23-cv-01346-JSR (S.D.N.Y.), Verdict, ECF No. 229 (filed 2024-04-05), pp. 1–2. Original: "1. On the SEC's first claim, under Section 17, we the jury find defendant Terraform Labs Pte. Ltd. ('Terraform'): Liable √"; "2. ... we the jury find that Terraform acted ...: Recklessly √"; "3. ... we the jury find defendant Do Hyeong Kwon ('Kwon'): Liable √"; "4. ... Kwon acted ...: Intentionally √"; "5. On the SEC's second claim, for intentional or reckless violation of Rule 10b-5, we the jury find defendant Terraform: Liable √"; "6. ... Kwon: Liable √"; "7. On the claim for control person liability regarding Terraform's violation of Rule 10b-5, we the jury find defendant Kwon: Liable √". (RECAP copy). The summary judgment of 2023-12-28 (Dkt. No. 149) was confirmed from the account of the proceedings on pp. 1–2 of the final judgment: "On December 28, 2023, the Court granted the SEC's Motion for Summary Judgment and found that Defendants engaged in unregistered offers and sales of securities in violation of Securities Act Sections 5(a) and 5(c) ...".

27. Same case, Final Judgment Against Defendants Terraform Labs Pte Ltd. and Do Hyeong Kwon, ECF No. 273 (filed 2024-06-12), pp. 2 and 5–8 (scanned; read by OCR, with the amounts rechecked digit by digit). Original: "Defendants having entered a general appearance; consented to the Court's jurisdiction over Defendants and the subject matter of this action; consented to entry of this Final Judgment; waived findings of fact and conclusions of law; and waived any right to appeal from this Final Judgment." (p. 2); "Terraform is liable for disgorgement totaling $3,586,875,883, ...together with prejudgment interest thereon in the amount of $466,952,423. Of these amounts, Kwon is jointly and severally liable with Terraform in the amount of $110,000,000 of disgorgement, plus prejudgment interest thereon in the amount of $14,320,196. ... the Court further imposes a civil penalty on Terraform of $420,000,000. The $4,473,828,306 in total monetary remedies

against Terraform shall be payable as provided for in the confirmed Chapter 11 Plan in the Bankruptcy Case" (p. 6); "Kwon is liable for disgorgement totaling $110,000,000, together with prejudgment interest thereon in the amount of $14,320,196. ... the Court further imposes a civil penalty on Kwon of $80,000,000." (p. 8); bankruptcy case designation "In re: Terraform Labs Pte. Ltd., No. 24-10070 (BLS) (Bankr. D. Del.)" (p. 5); "Defendants shall otherwise burn or destroy private keys in Terraform's possession to wallets or blockchain addresses holding Terraform Crypto Assets" (p. 5). This chapter read the amounts from the judgment; the SEC press release (2024-73) could not be opened because of bot blocking and was not used. What is given once here is not "who made how much" but the size of the disgorgement and penalties the court ordered.

28. *United States v. Do Hyeong Kwon*, No. 1:23-cr-00151-PAE (S.D.N.Y.), Judgment in a Criminal Case, ECF No. 63 (Date of Imposition 12/11/2025, filed 12/12/25), pp. 1–5 (5-page scan, OCR). Original: "pleaded guilty to count(s) 1s & 4s of the S1 Indictment"; "18 U.S.C. § 371 Conspiracy to Defraud 3/23/2023 1s / 18 U.S.C. § 1343 and 2 Wire Fraud 3/23/2023 4s" (p. 1); "One hundred eighty (180) months. Sixty (60) months on count 1 and one hundred (120) [sic] months on count 4, these terms to run concurrently. The defendant shall be given credit for time served for both (1) his time in custody since December 31, 2024, when he arrived in the United States; and (2) 17 months and 8 days that the defendant was in pre-extradition custody in Montenegro."; "Separately, the Court encourages any court in the Republic of Korea that may sentence the defendant to take this Court's sentence into account. The Court supports any application supported by the United States Attorney's Office with respect to a transfer of the defendant to the Republic of Korea after completion of half of his custodial sentence." (p. 2); "The Court declines to impose a term of supervised release." (p. 3); "Assessment $ 200.00" (p. 4); "See Order of Forfeiture filed separately at Dkt. No. 47." (p. 5). (PACER copy). The facts underlying the guilty plea are not written in the judgment, so this chapter does not give them. Proceedings in the Republic of Korea may be ongoing, so this chapter does not describe even their facts. The sentence by count as entered in the judgment differs from the sentencing transcript; see the next note.

29. Same case, Transcript of Proceedings re: Sentence held on 12/11/2025, ECF No. 67 (filed 12/30/25), p. 163 (transcript page 169). Original: "it is the judgment of the Court that you are to serve a sentence of 15-years imprisonment, 180 months in the custody of the Bureau of Prisons." / "As to the particular counts, I impose a sentence of five years, 60 months, on Count One, which is the maximum sentence available, and a sentence of 15 years, 180 months, on Count Four, the terms to run concurrently." The entry on p. 2 of the judgment (ECF No. 63), "Sixty (60) months on count 1 and one hundred (120) [sic] months on count 4, these terms to run concurrently," differs from this oral sentence (it is the original as confirmed on the scanned image, not an OCR error). Which prevails when the oral sentence and the written judgment conflict is not stated, because this book has not checked the sources on that point. On the CourtListener RECAP docket (accessed 2026-10-27), there is no entry for an amended judgment after ECF No. 63; RECAP shows only the documents that have been uploaded, so this does not confirm that no such entry exists on the full docket. (CourtListener RECAP copy, 167 pages).

30. Declaration of John J. Ray III in Support of Chapter 11 Petitions and First Day Pleadings, *In re FTX Trading Ltd., et al.*, Case No. 22-11068 (JTD) (Bankr. D. Del.), D.I. 24 (filed 2022-11-17), ¶¶1, 4–5. The original is as quoted in the text; ¶4: "I have over 40 years of legal and restructuring experience. I have been the Chief Restructuring Officer or Chief Executive Officer in several of the largest corporate failures in history. I have supervised situations involving allegations of criminal activity and malfeasance (Enron)." (RECAP copy, 30 pages). The declaration is a sworn statement ("under penalty of perjury"), not a finding of fact by the court. ¶7 states that "all facts set forth in this declaration ... are based on my personal knowledge, my review of relevant materials in the Debtors' files or my opinion."

31. Ray Declaration, ¶¶9–10: "I have identified four groups of businesses, which I refer to as 'Silos.' These Silos include: (a) ... (the 'WRS Silo'), which includes the businesses known as 'FTX US,' ...; (b) a group composed of Debtor Alameda Research LLC and its Debtor subsidiaries (the 'Alameda Silo'); (c) ... (the 'Ventures Silo'); and (d) a group composed of Debtor FTX Trading Ltd. and its Debtor and non-Debtor subsidiaries (the 'Dotcom Silo'), including the exchanges doing business as 'FTX.com' ..."; "Each of the Silos was controlled by Mr. Bankman-Fried." The balance-sheet figures for each Silo (¶¶18–19 and elsewhere) are not reproduced in this chapter, because Ray himself wrote "I do not have confidence in it."

32. Ray Declaration, ¶62: "The Debtors did not have the type of disbursement controls that I believe are appropriate for a business enterprise. For example, employees of the FTX Group submitted payment requests through an on-line 'chat' platform where a disparate group of supervisors approved disbursements by responding with personalized emojis."

33. Ray Declaration, ¶65: "The FTX Group did not keep appropriate books and records, or security controls, with respect to its digital assets. Mr. Bankman-Fried and Mr. Wang controlled access to digital assets of the main businesses in the FTX Group (with the exception of LedgerX, regulated by the CFTC, and certain other regulated and/or licensed subsidiaries). Unacceptable management practices included the use of an unsecured group email account as the root user to access confidential private keys and critically sensitive data for the FTX Group companies around the world, the absence of daily reconciliation of positions on the blockchain, the use of software to conceal the misuse of customer funds, the secret exemption of Alameda from certain aspects of FTX.com's auto-liquidation protocol, and the absence of independent governance as between Alameda (owned 90% by Mr. Bankman-Fried and 10% by Mr. Wang) and the Dotcom Silo (in which third parties had invested)."

34. Ray Declaration, ¶66: "These balances exclude cryptocurrency not currently under the Debtors' control as a result of (a) at least $372 million of unauthorized transfers initiated on the Petition Date, ..., (b) the dilutive 'minting' of approximately $300 million in FTT tokens by an unauthorized source after the Petition Date and (c) the failure of the co-founders and potentially others to identify additional wallets believed to contain Debtor assets." ¶71: "One of the most pervasive failures of the FTX.com business in particular is the absence of lasting records of decision-making."

35. *United States v. Samuel Bankman-Fried*, No. 1:(S6)22-CR-673-001 (LAK) (S.D.N.Y.), Judgment in a Criminal Case, ECF No. 424 (Date of Imposition 3/28/2024), pp. 1–3 (scan, OCR). Original: "was found guilty on count(s) (S6)1 through (S6)7 after a plea of not guilty."; "18 USC §§ 1343 and 2 Wire Fraud ... (S6) 1&3 / 18 U.S.C. § 1349 Wire Fraud Conspiracy ... (S6) 2&4 / 18 U.S.C. § 371 Securities Fraud Conspiracy ... (S6) 5 / 18 U.S.C. § 371 Commodities Fraud Conspiracy ... (S6)6 / 18 USC § 1956(h) Money Laundering Conspiracy ... (S6)7"; "Hon. Lewis A. Kaplan, U.S.D.J." (RECAP copy). For the total term, the appellate opinion's summary (25 years) was used instead of the concurrent and consecutive structure on p. 3 of the judgment. The jury verdict date of 2023-11-02 was confirmed from an entry on the RECAP docket, and this chapter put in the text only what is written in the judgment and the appellate opinion.

36. *United States v. Bankman-Fried*, No. 24-961-cr (2d Cir. June 12, 2026) (Parker, Lee, Kahn, JJ.; opinion by Parker, J.), pp. 2, 6–7, 9. Original: "We AFFIRM the judgment of the district court." (p. 2); "Following a four-week trial, he was convicted on all counts. The district court sentenced him to 25 years' imprisonment, to be followed by three years of supervised release. The district court also imposed a forfeiture of approximately $11 billion." (p. 6); the government's theory and the "robust" assessment are as quoted in the text (pp. 6–7, 9). Page 8 of the opinion states that the defendant's argument that the investments were sound and that in the end there would have been no loss had "essentially been rejected" by the "fraudulent-inducement theory" of *Kousisis v. United States*, 605 U.S. 114 (2025). (DktEntry 73.1, 42 pages). The opinion sets out the facts on the stated standard, "draw the facts from the evidence presented at trial, viewed in the light most favorable to the government" (p. 6), so the factual statements this chapter takes from it are under that standard.

PART 5 · LAW

Chapter 27. The bridge between technology and law (1): why this book does not go deep, the map of scholarly positions, a survey by jurisdiction

KEY TAKEAWAY

Part 5 is **a map of questions**, not a set of answers. It records which legal issue each of the twenty-three "A lawyer's question" boxes raised in Parts 1–4 reaches, and where the primary sources are for finding the judgments and statutes in the original. This chapter has three sections. 27.1 sets out the five reasons this book does not treat the law in depth. 27.2 lays out the scholarly positions on the private-law nature of the question "what is a virtual asset?" **as a map of opposing views**, and records in a correspondence table which **technical facts** from Parts 1–3 (exclusive control through the private key, fungibility, non-replicability, the probabilistic nature of finality, the absence of an issuer, the sharing of the record) each position relies on and which facts counter it. This is the contribution only this book can make, and it does not settle on any position. 27.3 compares five jurisdictions, Korea, the EU, the United States, Japan and the United Kingdom, in a table with a single frame (defined term, competent authority, issuance and circulation, stablecoins, private-law nature), and attaches a reference date to each axis. Statutory provisions are given only where they were checked against the text of the statute, and judgments only where the text of the judgment was checked, together with the court, the date of decision and the case number. Provisions not yet in force (the taxation of virtual assets under Korea's Income Tax Act (소득세법), the transfer of the definition into Japan's Financial Instruments and Exchange Act (金融商品取引法), the full commencement of the UK's 2026 Regulations, and the US GENIUS Act) are marked as not yet in force.

Learning objectives

1. State the five reasons this book treats the law only at the level of a survey, and point to **where** in a legal judgment the technical and economic facts of Parts 1–4 enter.
2. Lay out the positions on the private-law nature of virtual assets as a map of opposing views, and pair each position with the technical facts it relies on and the places where those facts counter it. Which position is right is left unsaid.
3. Read the defined terms, competent authorities, regulation of issuance and circulation, treatment of stablecoins, and private-law nature of five jurisdictions in one table, and understand why each axis needs a reference date.
4. Distinguish in writing between "in force," "promulgated but not yet in force," and "pending."

What a reader of the [Basic] paragraphs alone takes away. This Part is **a map of questions**, not a set of answers. The same technical fact serves as the ground for different conclusions in different jurisdictions and under different positions, and so knowing the technology is the first step toward finding where a legal judgment sits.

27.1 Why this book does not treat the law in depth

This book was written on the assumption that legal scholars would read it, yet it does not treat the law in depth. There are five reasons.

1. **The law in this field changes quickly.** A detailed doctrinal commentary would date sooner than any other Part of this book. The properties of hash functions (Chapter 4) and the structure of a bank run (Chapter 24) will be the same ten years from now, but the table in 27.3 below will change even while this book is being printed. Korea's provision on the taxation of virtual assets has had its commencement postponed three times since promulgation; Japan has promulgated an amendment moving the definition from one statute to another and is waiting for it to take effect; and the United Kingdom has set the full commencement of its new regulatory regime for 2027. So every statement about legal institutions carries a reference date, and values that date easily are kept in the trend notes of Appendix A.
2. **Each jurisdiction has its own system.** Going deep would take a book per jurisdiction. Korea calls the same thing "가상자산" (gasang jasan, "virtual asset"), the EU "crypto-asset," the United States, depending on context, "security," "commodity," "property" or "digital asset," Japan "暗号資産" (angō shisan, "crypto-asset"), and the United Kingdom "cryptoasset," and the jurisdictions differ in which authority's competence and which statute's application each name calls forth (Chapter 30). Putting five jurisdictions in one table is only a starting point for comparison, not a commentary on any of them.
3. **The readers are lawyers.** What this book can offer is not doctrinal commentary but **a bridge that shows where in a legal judgment the technical and economic facts enter**. To answer "what has a person who lost a private key lost?" (Chapter 5) you need to know where the "rightful holder" sits on a ledger that checks only signatures, and to answer

"when was the payment completed?" (Chapter 9) you need to know that finality is a probability. Parts 1–3 supplied those facts, and plugging them into doctrine is the reader's task. The correspondence table in 27.2 is the device that shows where they plug in.

4. **The editor is a sitting judge.** On cases pending in Korea, bills pending before the National Assembly (국회), and issues the Supreme Court of Korea (대법원) has not settled, this book states facts only and does not evaluate them. That is why Korean law is treated more thinly than foreign law. Of Korean judgments, only the three whose text was checked are cited, and the book does not say whether those judgments are right or which scholarly position is sound.
5. **It points to where to read further.** Chapter 28 collects primary sources and treatises for each jurisdiction. Only bibliographic details confirmed in checking the sources are given, and those not confirmed are left blank.

The survey of foreign law covers five jurisdictions: Korea, the EU, the United States, Japan and the United Kingdom. They were chosen because each took a different path: comprehensive codified regulation (EU MiCA), the coexistence of existing law with a special statute (the United States), a division of labor between a payment statute and a financial-instruments statute (Japan), the extension of common-law property categories together with the introduction of a new regulatory regime (the United Kingdom), and Korea, which legislated first on user protection and the regulation of unfair trading. There are six axes of comparison (defined term, competent authority, issuance and circulation, stablecoins, private-law nature, reference date). Each cell gives provision numbers, and the court, date of decision and case number, without commentary.

27.2 The map of scholarly positions on legal nature: positions matched against technical facts

To the private-law question "what is bitcoin?", the scholarship and judgments of each jurisdiction have given different answers. It is a thing. It is not a thing, but it is the object of a right of control akin to a real right. It is not a claim. It is not a property right but a factual state. It is a third category of property. It is an electronic record capable of control. It is money, or it is not. It is a security, or it is not. It is a commodity. It is a pecuniary advantage. This section lays out those positions **by the structure of their grounds, not by whether they are right or wrong**. Which of the technical facts covered in Parts 1–3 does each position rely on, and which counter it? Recording that in a correspondence table is the work of this section, and it does not settle on any position.

The starting point is a 2022 research report of the Judicial Policy Research Institute (사법정책연구원), Lee Hye-jeong (이혜정), *가상자산에 대한 민사집행 연구* [A study of civil execution against virtual assets]. To determine the object of civil execution, the report brings together the debates on legal nature in Korea, the United States, the Commonwealth, Germany and Japan (with an overview of regulation for the EU). It explains that "가상자산을 물건으로 보고 그에 대한 권리를 인정할지에 대해 견해가 통일되어 있지 않아, 가상자산을 취득한 자가 '가상자산을 소유하고 있다'고 단정 짓기 힘들고" (gloss: "views are not unified on whether to regard a virtual asset as a thing and recognize rights in it, so it is hard to state conclusively that a person who has acquired a virtual asset 'owns the virtual asset'"; working gloss, not an official translation; the original controls), and that it therefore uses the word "'소유' 대신 '보유'" (gloss: "'holding' (보유) instead of 'ownership' (소유)").[1] This book uses "holding" for the same reason. The foreign primary sources are the 2023 final report of the Law Commission of England and Wales, the UK's Property (Digital Assets etc) Act 2025, the 2023 UNIDROIT Principles, Article 12 of the US Uniform Commercial Code (2022 amendments), a 2015 judgment of the Court of Justice of the European Union, US decisions of 1946, 2018 and 2023 and the SEC's 2017 report, a 2019 judgment of the Tokyo District Court (東京地方裁判所) in Japan, and three judgments of the Supreme Court of Korea.

The correspondence table

There are four columns: the position (whose, in which jurisdiction), its core claim, the technical facts that claim relies on (and the chapter of this book), and the place among those same technical facts that counters the claim. No row has an empty fourth column. Technical facts do not favor any position one-sidedly.

POSITION	CORE CLAIM	TECHNICAL FACTS RELIED ON (CHAPTER)	TECHNICAL FACTS THAT COUNTER IT (CHAPTER)
① **Thing (movable) theory**: the Korean affirmative view[2]	Only the person holding the private key can order a transfer (exclusive controllability), and an approved transfer cannot be double-spent short of hacking a majority (manageability), so it is a "관리할 수 있는 자연력" ("natural force that can be managed") under Article 98 of the Civil Act (민법)	A signature controls the authority to spend (Ch. 5). A spent UTXO cannot be spent again (Ch. 11). The cost of tampering (Chs. 7, 8)	A private key is information that can be copied (Ch. 5). A transfer is complete only once it is in a block, and that finality is a probability (Ch. 9). No energy can be extracted from bitcoin: the wording "자연력" ("natural force")
② **Not a thing; a right of control akin to a real right**: Korea[3]	The holder exercises sole and exclusive control over its property value and can assert it against anyone, so it is a right without a basis in statute, a "준물권적 구조를 가진 권리" ("right with a quasi-real-right structure")	No one can move it without a signature (Ch. 5)	The record is shared by all participants, so even with the private key it is hard to call this "sole control" (Chs. 6, 7, 12: anyone can download the ledger). The objection from the numerus clausus of real rights (Article 175 of Japan's Civil Code (民法)) raised against Japanese positions ②–④ sits in the same place
③ **Claim theory rejected**: Korea[4]	The performance of putting a transfer order into a block is carried out by the network's program, and each participant only contributes a little of its resources, so there is no "obligor." The network has neither assets available for liability nor a central body, so it cannot be treated as a legal person or an association	Miners and validators are no one's agents (Ch. 8 question box). There is no issuer (Chs. 1, 19)	There is room to regard coins deposited with an exchange as a **claim** against the exchange (a claim for the transfer of virtual assets, 가상자산이전청구권): the fact that the books and the chain are different layers (Ch. 26, Mt.Gox). It can be the **object** of a claim
④ **Factual-state theory**: Japanese positions ⑦ and ⑧[5]	There is no issuer, and the object of ownership is limited to tangible things, so holding carries no property right of any kind, whether real right, claim, membership right or intellectual property right; it is a factual state. (Another opinion supporting this position: as a result, real-right claims are excluded, which secures a money-like treatment based on the "coincidence of ownership and possession.")	The ledger looks only at the validity of signatures; it does not know the "rightful holder" (Ch. 5). Absence of an issuer (Ch. 1)	If it is a factual state, on what basis are coins transferred by mistake or stolen coins given back (Ch. 11; 28.1(a))? It is unclear whether a "normative agreement" of all participants exists as something distinct from the protocol (Ch. 23, coordination game)

POSITION	CORE CLAIM	TECHNICAL FACTS RELIED ON (CHAPTER)	TECHNICAL FACTS THAT COUNTER IT (CHAPTER)
⑤ **Membership-right or partnership-right theory**: Japanese position ⑤, Korean partnership-right view[6]	The holder is a member of an association, the network. Read the "프로그램 시스템" ("program system") as the content of a contract, the "블록체인의 내부적 거래" ("internal transactions of the blockchain") as a common undertaking and "취득" ("acquisition") as a contribution, and it is a partnership under Article 703 of the Civil Act	The rules are maintained and changed by the coordination of participants (Ch. 23). A hard fork is a "vote" (Chs. 22, 26)	There is no statute or declaration of intention for forming, joining or leaving the association. The sets of node operators, miners and holders do not coincide (Chs. 8, 10). The partnership-right view itself says that this partnership right is "이익분배에 대한 기대권이며, 실체적 권리가 아니라 조합원의 지위권" ("an expectancy of a distribution of profits, not a substantive right but a right of status as a partner")
⑥ **Intellectual-property theory**: Japanese position ⑥[5]	Possibly a work protected by copyright	Code is a work (the contract code of Ch. 13)	It is hard to regard mining as a process in which "사상이 창작적으로 표현" ("thoughts are creatively expressed"). A hash value is the output of a function (Ch. 4)
⑦ **"Intangible interest"**: German scholarship in agreement[7]	Not a thing under the Civil Code, but an intangible interest falling under "object," the broader concept above it. Scholarship agrees that it is not a claim (p. 83) and that it lacks corporeality (p. 81)	Absence of an issuer and of an obligor (Ch. 1). Data are not tangible things	"Intangible interest" does not directly supply rules of execution or attribution: by what procedure is it attached (28.1(a))?
⑧ **Third-category property theory**: Law Commission of England and Wales, Property (Digital Assets etc) Act 2025[8][9]	"some digital assets are neither things in possession nor things in action, but that nonetheless the law of England and Wales treats them as capable of being things to which personal property rights can relate." The legislation only confirms "that the fact that a thing is neither a thing in possession nor a thing in action does not prevent it from being a thing to which personal property rights can relate," and the common law draws the boundary	**Rivalrousness**: one person's use excludes another's (Ch. 11, spending a UTXO). It exists independently, outside the system (Chs. 1, 7)	A "balance" on an exchange's books may not be rivalrous (Ch. 26, Mt.Gox's fictitious balances). The report itself dropped "composed of data" from the criteria

POSITION	CORE CLAIM	TECHNICAL FACTS RELIED ON (CHAPTER)	TECHNICAL FACTS THAT COUNTER IT (CHAPTER)
⑨ **Control-based**: UNIDROIT Principles, US UCC Article 12[10][11]	"'Digital asset' means an electronic record which is capable of being subject to control," and a "controllable electronic record" is "a record stored in an electronic medium that can be subjected to control under Section 12-105." **Control** is made the connecting point for the definition and for the rules of transfer and security, while who holds the property right is itself left to "other law" (UNIDROIT Principle 3(3)(a))	Private key = authority to spend (Ch. 5). The exclusivity and transferability of control (Ch. 11)	Control is not a right: a stolen key also gives control (Ch. 5 question box). How is control through multiple signatures (multisig) or through a custodian counted (Ch. 26, FTX's private-key management)?
⑩ **Monetary character**: negative and affirmative views (Korea), EU VAT, US tax law, Supreme Court of Korea[12][13][14][15]	Negative: it cannot serve as a measure of value, and its total issuance is fixed. Affirmative: the possibility of future acceptance. EU: "bitcoin" has "no purpose other than to be a means of payment," and exchange is exempt from VAT. US: for tax purposes it is "property" and "not … currency." Supreme Court of Korea: "형법을 적용하면서 법정화폐와 동일하게 보호해야 하는 것은 아니다" (gloss: "in applying the Criminal Act, it need not be protected in the same way as legal tender")	A check of the functions of medium of exchange, measure of value and store of value (Ch. 19). A floor of demand from taxes is what sets legal tender apart (Ch. 19 model)	What Chapter 19 showed is that "a positive price with an intrinsic value of 0" can be an equilibrium, not that it is money. "What changes when the law calls something money" differs by jurisdiction (Ch. 19 question box)
⑪ **Security character**: Korean view rejecting negotiable-instrument status, US *Howey*, SEC, *Ripple*, GENIUS Act[16][17][18][19][20]	Korea: the prevailing view is that it is hard to treat it as a 유가증권 (negotiable instrument), because there is no issuer and a record on the chain is not an endorsement. US: an "investment contract" is "a contract, transaction or scheme whereby a person invests his money in a common enterprise and is led to expect profits solely from the efforts of the promoter or a third party"; DAO tokens are securities (2017); for the same token, sales to institutions are investment contracts and anonymous trading on exchanges is not (2023). Payment stablecoins were taken out of securities by statute (2025)	A token standard is only a promise about function names and carries no rights (Ch. 14). The "efforts" of curators and issuers (Chs. 22, 26). In the secondary market the buyer does not know who the seller is (Ch. 15 AMM; Ch. 17)	Look at substance rather than form, as in *Howey* ("Form was disregarded for substance and emphasis was placed upon economic reality"), and it is the **promise outside**, not the code, that determines security status (Ch. 14). That is why the same token is judged differently depending on the route of the transaction
⑫ **Commodity character**: US CFTC, *McDonnell*[21]	Virtual currencies are "'goods' exchanged in a market for a uniform quality and value" and fall within the Commodity Exchange Act's "commodity," and the jurisdiction of the Commodity Futures Trading Commission (CFTC) is **concurrent** with that of other agencies	Fungibility (Ch. 11 UTXO; Ch. 14 ERC-20). Underlying asset of futures and derivatives	"Security" and "commodity" overlap on the same thing: naming divides jurisdiction (Ch. 30)

POSITION	CORE CLAIM	TECHNICAL FACTS RELIED ON (CHAPTER)	TECHNICAL FACTS THAT COUNTER IT (CHAPTER)
⑬ **Pecuniary advantage, intangible property**: Supreme Court of Korea[22][23][15]	Bitcoin is "경제적인 가치를 디지털로 표상하여 전자적으로 이전, 저장과 거래가 가능하도록 한 가상자산의 일종으로 사기죄의 객체인 재산상 이익" (gloss: "a kind of virtual asset that represents economic value digitally so that it can be transferred, stored and traded electronically, and a pecuniary advantage that can be the object of fraud") (November 11, 2021); it can be confiscated as "재산적 가치가 있는 무형의 재산" (gloss: "intangible property having property value") (May 30, 2018); and a person who receives a transfer without legal cause may bear an obligation to make restitution of unjust enrichment but is not "타인의 사무를 처리하는 자" (gloss: "a person who administers another's business") (December 16, 2021)	Electronic transfer, storage and trading (Chs. 11, 12). The property that "보관되었던 전자지갑의 주소만을 확인할 수 있을 뿐 그 주소를 사용하는 사람의 인적사항을 알 수 없고, 거래 내역이 분산 기록되어 있어 다른 계좌로 보낼 때 당사자 이외의 다른 사람이 참여해야 하는" (gloss: "only the address of the electronic wallet in which it was kept can be identified, and the personal particulars of the person using that address cannot be known, and the transaction history is recorded in a distributed manner, so that sending it to another account requires the participation of persons other than the parties") (technical facts stated by the judgment)	A criminal-law finding of "pecuniary advantage" does not settle attribution in private law: the judgment itself states that "부당이득반환청구를 할 수 있는 주체가 피해자인지 아니면 거래소인지 명확하지 않다" (gloss: "it is not clear whether the party entitled to claim restitution of unjust enrichment is the victim or the exchange") (Ch. 26, books and chain)
⑭ **Deposit, debt or equity**: stablecoins (Gorton & Zhang)[24]	To decide whether a stablecoin is a demand deposit, one must first determine whether the contract between holder and issuer is an "equity contract or a debt contract": the same shape as the money market fund debate of the 1970s	A token carrying a redemption promise is inside money (Ch. 24). Balance sheet and redemption procedure (Ch. 24)	Because there is a secondary market, closing the window does not stop a run (Ch. 24): a point where it differs from a deposit. The legal effect of the issuer's "public declaration" (Ch. 24 question box)

How to read the table

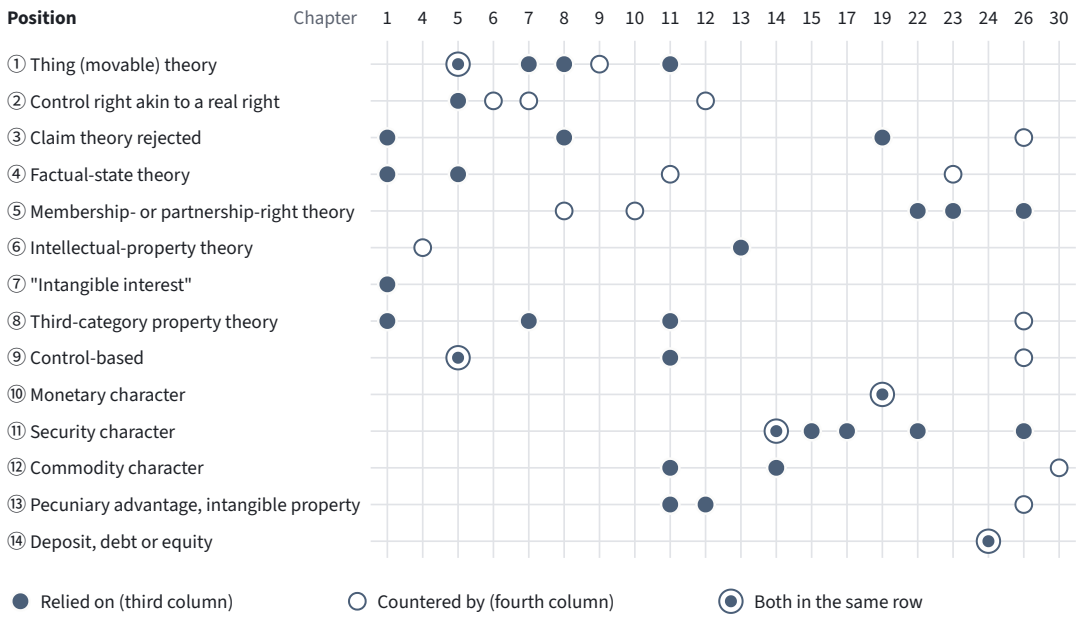


Figure 27-1. The chapters each of the fourteen positions in the 27.2 correspondence table relies on (filled dots, third column) and is countered by (open dots, fourth column). Section references in parentheses (28.1(a) and the like) are not chapters and are not plotted.

The third and fourth columns of the table often point to the same chapter. The fact from Chapter 5 that "a signature controls the authority to spend" is a ground for the thing theory (①), the control-right theory (②) and the control-based approach (⑨), and at the same time a ground for the objections that "a stolen key also gives control" and "a private key is information that can be copied." The fact from Chapter 9 that "finality is a probability" is where the manageability of a thing is shaken, and also where one asks "when was it completed?" (28.1(c)). So one cannot say that any position "fits the technology." Technical facts are not a referee that picks among the positions but a mirror that shows **which facts each position presupposes and which facts it has to explain**.

Two further points. First, courts have avoided this question when they could. The 2019 judgment of the Tokyo District Court stated "ビットコインは「有体物」（民法８５条）ではなく，
その**帰属**や利用者間での取引がどのような私法上の性質を有するかも一義的に明らかとはいえないが，本件の判断に影響のない限り，ビットコインの保有・
所有や利用者間の売買などと表現し，日常的な意味合いで記載する" (gloss: "bitcoin is not a 'tangible thing' (Civil Code Article 85), and it cannot be said to be unambiguously clear what private-law nature its attribution and transactions between users have; but insofar as this does not affect the determination in this case, [the court] will describe it in everyday terms, as the holding and ownership of bitcoin, sales between users, and so on") and decided the case.[25] The 2021 judgment of the Supreme Court of Korea likewise answered with "pecuniary advantage," a category of criminal law, and on attribution in private law it went no further than stating, on the ground that how the bitcoin in that case had been transferred was unclear, that "부당이득반환청구를 할 수 있는 주체가 피해자인지 아니면 거래소인지 명확하지 않다"

(gloss: "it is not clear whether the party entitled to claim restitution of unjust enrichment is the victim or the exchange"). That is a judgment tied to the facts of the case, not an answer about attribution in general. Second, legislation answered **by definition**, not **by nature**. "경제적 가치를 지닌 것으로서 전자적으로 거래 또는 이전될 수 있는 전자적 증표(그에 관한 일체의 권리를 포함한다)" (gloss: "as something having economic value, an electronic token that can be traded or transferred electronically (including all rights relating to it)") in Article 2, subparagraph 1 of Korea's Act on the Protection of Virtual Asset Users (가상자산 이용자 보호 등에 관한 법률; "the Virtual Asset User Protection Act"), and "a digital representation of a value or of a right" in Article 3(1)(5) of the EU's MiCA, are sentences that fix the object of regulation. They are not sentences that decide whether the holder has a real right or a claim.[26] The UK's 2025 Act shows this most clearly: in two sections it confirms only that a thing is not prevented from being "the object of personal property rights," and leaves the boundary to the common law. Chapter 30 sets these definitional provisions side by side.

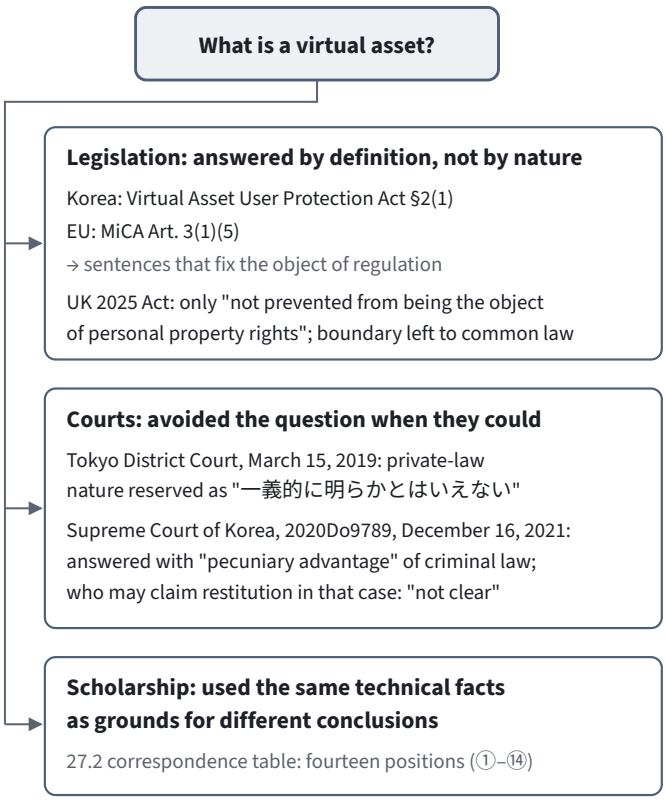


Figure 27-2. To "what is a virtual asset?", legislation answered with definitions, courts avoided the question when they could, and scholarship used the same technical facts as grounds for different conclusions.

Question boxes linked to this section

Of the boxes in Parts 1–3 and Part 6, six point to this section. For each, the table notes which rows of the correspondence table take it up.

BOX	GIST OF THE QUESTION	WHERE THIS SECTION TAKES IT UP
Chapter 5	What has a person who lost the private key lost: possession, ownership, or accessibility? On a ledger that looks only at the validity of signatures, where is the "rightful holder"?	①, ② and ⑨ rely on "private key = exclusive control," and ④ relies on "the ledger does not know the right holder." The UK report's "rivalrousness" (⑧), as a mark of what can be an object of property, and UNIDROIT's "control" (⑨), as the connecting point for the definition and the rules of transfer, are answers that bring in accessibility (neither of them decides attribution itself), and the Supreme Court of Korea (⑬) stated the property that "주소만 확인할 수 있을 뿐 그 주소를 사용하는 사람의 인적사항을 알 수 없" (gloss: "only the address can be identified, and the personal particulars of the person using that address cannot be known")
Chapter 19	Are legal tender and "money" the same thing? What changes when the law calls something "money"?	⑩. EU VAT answered "a means of payment," US tax law "property," and the Supreme Court of Korea "형법을 적용하면서 법정화폐와 동일하게 보호해야 하는 것은 아니" (gloss: "in applying the Criminal Act, it need not be protected in the same way as legal tender") (a judgment limited to the application of the Criminal Act). The taxation row of 27.3
Chapter 20	If a majority changes the issuance rules, are the holders' rights infringed? Is what the holder has a right to the "21 million cap," or only the ability to spend UTXOs?	④ (factual state), ⑤ (partnership right) and ⑨ (control) each imply a different answer. Under the factual-state theory there is no right to infringe; under the partnership-right theory the limits of majority rule are at issue; and on the control-based approach a change of rules changes the very object of control. The restriction on spending vulnerable coins in Chapter 29 asks the same structural question more sharply: 28.1(g)
Chapter 21	To whom does a burned fee belong: abandonment, an ownerless thing, consumption, a new category?	No position directly explains "the extinction of value that belongs to no one." Under the thing theory it is the abandonment of ownership or the destruction of the thing; under the factual-state theory, the end of a state; on the control-based approach, the extinction of the object of control. This book chooses none of them
Chapter 23	After a hard fork, who decides which of the two chains is the "real" ledger? Did a holder before the fork have one or two?	The rivalrousness criterion of ⑧ leaves room to treat the coins on the two chains as separate property, since they are not rivalrous with each other, and under the control criterion of ⑨ the same private key also gives control on both chains. Which chain's coin "the one bitcoin in the contract" refers to remains a question of contract interpretation: 28.1(g)
Chapter 30	When a definitional provision cannot keep up with the technology: analogy, or a legislative gap?	The defined-term row of 27.3 and the concordance of provisions in Chapter 30. Since legislation answered by definition and not by nature, the debate on nature (this table) is called on again where a definition leaves a gap

27.3 A survey by jurisdiction

The table below compares five jurisdictions on six axes. The cells contain provision numbers and the court, date of decision and case number of judgments, and no commentary. **The reference date is 2026-10-25**, and each axis distinguishes "in force," "promulgated, not yet in force," and "not verified." Notes by jurisdiction follow the table.

AXIS	KOREA	EU	UNITED STATES	JAPAN	UNITED KINGDOM
Defined term (provision)	"가상자산": Virtual Asset User Protection Act §2(1) (enacted as Act No. 19563, now Act No. 21857; definition unchanged); Specified Financial Transaction Information Act §2(3) applies it mutatis mutandis	"crypto-asset": MiCA Art. 3(1)(5); "asset-referenced token" (6), "e-money token" (7), "utility token" (9); unique and non-fungible assets excluded by Art. 2(3), financial instruments by Art. 2(4)(a)	No single definition. "security" (Securities Act §2(a)(1), "investment contract": *Howey*), "commodity" (Commodity Exchange Act, cited by *McDonnell* as "Title 7 U.S.C. § 1(a)(9)"), "property" (IRS Notice 2014-21), "digital asset" and "payment stablecoin" (GENIUS Act §2(6), (22)), "controllable electronic record" (UCC §12-102)	"暗号資産": Payment Services Act §2(14) (renamed from "仮想通貨" by 令和元年法律第28号 (Act No. 28 of 2019), in force 2020-05-01; paragraph 14 from 2023-06-01); "電子記録移転権利": Financial Instruments and Exchange Act §2(3); "電子決済手段": Payment Services Act §2(5)	"cryptoasset": FSMA 2000 s. 417(1) (inserted by FSMA 2023 s. 69, in force 2023-08-29); "qualifying cryptoasset" and "qualifying stablecoin": RAO art. 88F, 88G (inserted by SI 2026/102)
Competent authority	Financial Services Commission (Virtual Asset User Protection Act §14 investigation and measures, §13 supervision and inspection); head of the Korea Financial Intelligence Unit (Specified Financial Transaction Information Act §7 reporting)	Member State competent authorities (Art. 93); issuers of significant ARTs and EMTs supervised by the EBA (Art. 117)	Concurrent jurisdiction of the SEC and CFTC (*McDonnell*); for payment stablecoins, the Office of the Comptroller of the Currency (OCC) and other "primary Federal payment stablecoin regulator" (GENIUS §2(25); for non-bank federally licensed issuers (§2(11)), the OCC under §2(25)(D)); FinCEN (MSB registration: SEC DAO report, footnote 20)	Registration with the 内閣総理大臣 (Prime Minister) (Payment Services Act §63-2); Financial Services Agency (金融庁)	FCA (SI 2026/102 reg. 1(3))

AXIS	KOREA	EU	UNITED STATES	JAPAN	UNITED KINGDOM
Issuance and circulation	No provisions on issuance or disclosure (the Act's 22 articles cover protection of users' assets, prohibition of unfair trading, and supervision). Businesses file a report (Specified Financial Transaction Information Act §7); segregation of deposits (Virtual Asset User Protection Act §6); segregated custody and actual holding of virtual assets (§7)	Obligation to draw up and publish a white paper (Title II, Art. 4); authorization of crypto-asset service providers (Art. 59); prohibition of market abuse (Title VI, Arts. 86–92)	If a security, §5 registration or exemption (SEC DAO report); if a commodity, the CEA's anti-fraud provisions (*McDonnell*)	Registration system for 暗号資産交換業 (angō shisan kōkangyō, "crypto-asset exchange business") (§63-2)	The 2026 Regulations make the public offer of a "qualifying cryptoasset" and its admission to trading on an exchange designated activities, and its issuance, custody and the like regulated activities: **full commencement 2027-10-25**
Stablecoins	No separate category (within Virtual Asset User Protection Act §2(1); "전자화폐" (electronic currency), "선불전자지급수단" (prepaid electronic means of payment) and electronic currency issued by the Bank of Korea are excluded: items (c) and (g))	ART (Title III, Art. 16 authorization), EMT (Title IV, Art. 48: credit institutions and electronic money institutions only): **applies from 2024-06-30**	GENIUS Act (enacted 2025-07-18): permitted issuers, not securities or commodities (§17): **effective date under §20 no later than 2027-01-18, possibly earlier depending on the issuance of regulations (not verified)**	電子決済手段 (Payment Services Act §2(5))	"qualifying stablecoin" (RAO art. 88G): 2027-10-25

AXIS	KOREA	EU	UNITED STATES	JAPAN	UNITED KINGDOM
Private-law nature	Supreme Court: pecuniary advantage (November 11, 2021, 2021Do9855); intangible property, confiscable (May 30, 2018, 2018Do3619); a person receiving a transfer without legal cause is not a subject of breach of trust (December 16, 2021, 2020Do9789). Scholarly positions divided under the definition of a thing in Civil Act §98 (27.2). Compulsory tax collection: National Tax Collection Act §55(3), §66(2)(2); Local Tax Collection Act §61(3)	MiCA is a regulatory instrument: within the provisions this book read, private-law nature is not decided (not verified). VAT: exchange is exempt (CJEU, October 22, 2015, C-264/14)	Property for tax purposes (2014-21); UCC Article 12 (2022 model law: state adoption not verified); security and commodity determinations made transaction by transaction	Tokyo District Court (東京地裁), March 15, 2019: not a "有体物"; private-law nature reserved as "一義的に明らかとはいえない"	Property (Digital Assets etc) Act 2025 s. 1 (in force 2025-12-02); Law Com No 412; a possible fiduciary duty of developers is a "serious issue to be tried" (*Tulip Trading v van der Laan* [2023] EWCA Civ 83, 2023-02-03)
Taxation	Income Tax Act §21(1)(27), virtual asset income: **promulgated 2020-12-29, in force 2027-01-01 (postponed three times), not in force as of 2026-10-25**	VAT exemption (*Hedqvist*)	General principles of taxing property transactions (2014-21)	(outside the scope of this book)	(outside the scope of this book)
Reference date	2026-10-25	2026-10-25	2026-10-25	2026-10-25	2026-10-25

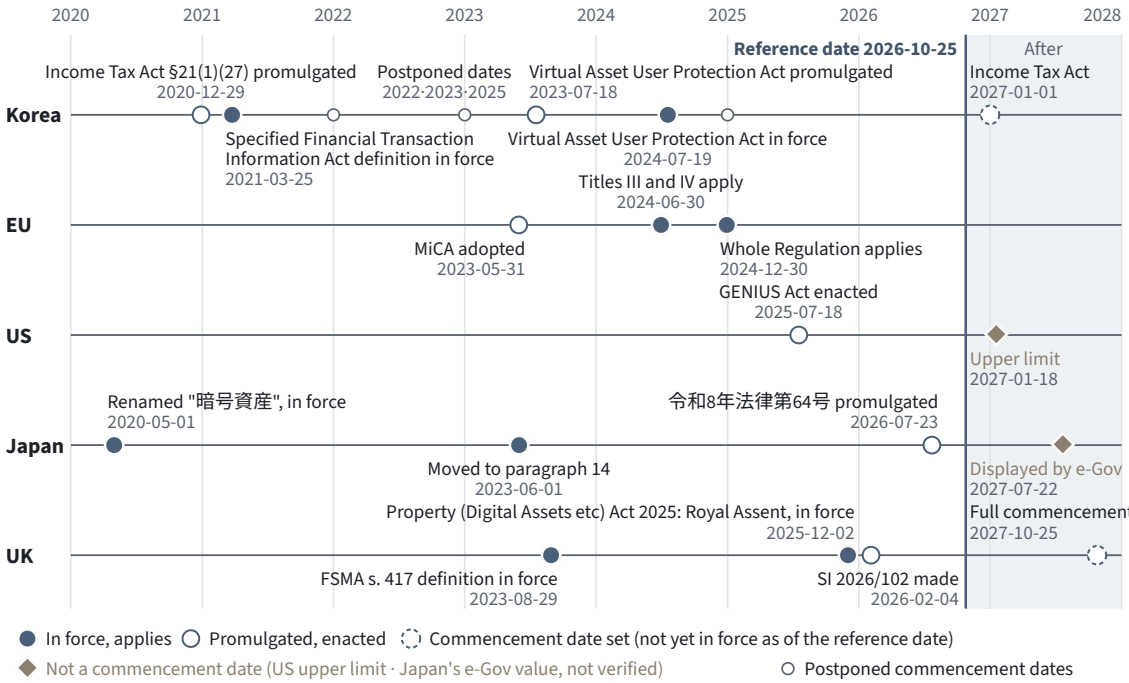


Figure 27-3. The dates on which the definitional and regulatory provisions of the five jurisdictions were promulgated or came into force, and this book's reference date 2026-10-25 (vertical line). Diamonds in the secondary color mark dates that are not commencement dates (the upper limit for the effective date of the US GENIUS Act; the value displayed by Japan's e-Gov, with the cabinet order not verified).

Korea

The Act on the Protection of Virtual Asset Users was enacted on July 18, 2023 as Act No. 19563 and came into force on July 19, 2024. The current version is Act No. 21857, promulgated on August 4, 2026 and in force on October 2, 2026 (a consequential amendment following an amendment of the Criminal Procedure Act (형사소송법)). The definitions in Article 2 have not changed since enactment.[27] In 22 articles, the Act provides for the protection of users' deposits (Article 6), the segregated custody and actual holding of virtual assets (Article 7), the prohibition of unfair trading (Article 10: use of material non-public information, market manipulation, fraudulent trading, and trading by a business in virtual assets it issued itself) and liability for damages (Article 10(6)), supervision and investigation by the Financial Services Commission (금융위원회) (Articles 13 and 14), and penalty surcharges and penal provisions (Articles 17 and 19). It has no provision governing issuance or disclosure. Entry for businesses is by a report to the head of the Korea Financial Intelligence Unit (금융정보분석원장) under Article 7 of the Act on Reporting and Using Specified Financial Transaction Information (특정 금융거래정보의 보고 및 이용 등에 관한 법률; "the Specified Financial Transaction Information Act"), and the definition of "virtual asset" in Article 2, subparagraph 3 of that Act points to Article 2, subparagraph 1 of the Virtual Asset User Protection Act.[28] On taxation, Article 21(1), subparagraph 27 of the Income Tax Act made "가상자산을 양도하거나 대여함으로써 발생하는 소득" (gloss: "income arising from transferring or lending virtual assets") a category of other income. But since this provision was promulgated on December 29, 2020, its commencement has been postponed three times, from 2022 to 2023 to 2025 to January 1, 2027, and **as of the reference date it is not yet in force**. The "current" text on the Korean Law Information Center of the Ministry of

Government Legislation (법제처 국가법령정보센터) displays this subparagraph as if it were in force, so quoting it as it stands would be wrong.[29] In compulsory tax collection, Article 55(3) of the National Tax Collection Act (국세징수법) lets the head of a tax office who is attaching a virtual asset require the delinquent taxpayer, or the business holding the asset in custody, to transfer it: "해당 가상자산의 이전을 문서로 요구할 수 있" (gloss: "may request in writing the transfer of the virtual asset concerned"). Article 66(2), subparagraph 2 permits the direct sale of "가상자산사업자를 통해 거래되는 가상자산" (gloss: "virtual assets traded through a virtual asset service provider"), and Article 61(3) of the Local Tax Collection Act (지방세징수법) also provides for a similar request for transfer (though for local taxes it is a duty, "요구하여야 한다" (gloss: "shall request"), and the requirement of writing attaches only when the request is made to a third party).[30] The Civil Execution Act (민사집행법) has no provision on virtual assets, and the Judicial Policy Research Institute's report proposed, as a legislative draft, procedures for attachment, transfer orders and sale orders, treating virtual assets as "그 밖의 재산권" (gloss: "other property rights").[31] This book did not verify the legislative course after that (as of 2026-10, Appendix A).

EU

The Regulation on markets in crypto-assets (Regulation (EU) 2023/1114, MiCA) was adopted on May 31, 2023 and published in Official Journal L 150 (June 9, 2023). Under Article 149 it **applies from December 30, 2024**, while asset-referenced tokens (Title III) and e-money tokens (Title IV) apply **from June 30, 2024**.[32] The definitions are in Article 3(1). A "crypto-asset" is "a digital representation of a value or of a right that is able to be transferred and stored electronically using distributed ledger technology or similar technology" (point 5), and beneath it come asset-referenced tokens (point 6), e-money tokens (point 7) and utility tokens (point 9). The dates of application have further exceptions in Article 149(4) (some provisions apply from entry into force). Crypto-assets that are "unique and not fungible" (Article 2(3)) and financial instruments and the like (Article 2(4)) are outside the scope.[33] Circulation is built on drawing up and publishing a white paper (Article 4), authorization to issue asset-referenced tokens (Article 16, or a credit institution complying with Article 17), issuance of e-money tokens only by credit institutions and electronic money institutions (Article 48), and authorization of crypto-asset service providers (Article 59), and Title VI (Articles 86–92) prohibits insider dealing and market manipulation both on and off trading platforms.[34] Supervision is carried out by the competent authorities designated by the Member States (Article 93), and issuers of asset-referenced tokens and e-money tokens classified as "significant" are under the supervision of the European Banking Authority (EBA) (Article 117). Within the provisions this book read, the Regulation does not decide private-law nature. Since the whole Regulation was not read through, this is recorded as "not verified." On VAT, the Court of Justice of the European Union held on October 22, 2015 in *Hedqvist* that the exchange of bitcoin for traditional currencies is an exempt transaction under Article 135(1)(e) of the VAT Directive.[13]

United States

The United States has no single definition. Depending on the shape of the transaction, the same token becomes a "security" under the securities laws, a "commodity" under the Commodity Exchange Act, and "property" under the tax laws. The test for a security is the definition of an investment contract laid down by the Supreme Court of the United States in *Howey* in 1946, and in its 2017 report of investigation on The DAO the SEC determined by that test that DAO tokens were securities (Chapter 26). On July 13, 2023, in a summary judgment decision in *SEC v. Ripple Labs*, the United States District Court for the Southern District of New York stated that "XRP, as a digital token, is not in and of itself a 'contract, transaction[,] or scheme' ..." and distinguished sales to institutional investors under written contracts, which were investment contracts, from sales to anonymous buyers on exchanges, which were not. The unit of judgment is the **transaction**, not the token.[19] On commodity status, on March 6, 2018, in a preliminary-injunction decision in *CFTC v. McDonnell*, the United States District Court for the Eastern District of New York determined that virtual currency falls within the "commodity" of the Commodity Exchange Act (in the decision's notation, "Title 7 U.S.C. § 1(a)(9)") and that the CFTC's jurisdiction is concurrent with that of other agencies.[21] In tax law, the Internal Revenue Service's 2014 notice (Notice 2014-21) stated that "virtual currency is treated as property," and its 2023 notice (Notice 2023-34) corrected only the background statement that virtual currency "does not have legal tender status in any jurisdiction," leaving the conclusions of the questions and answers as they were.[14] On payment stablecoins, the GENIUS Act (Pub. L. 119-27), enacted on July 18, 2025, defines "payment stablecoin" (§2(22)) and states expressly that those issued by permitted issuers are neither securities nor commodities (§17). Its effective date is the earlier of 18 months after enactment and 120 days after the primary Federal payment stablecoin regulators issue final regulations (§20). This book did not verify the state of rulemaking as of the reference date.[20] On attribution in private law, the Uniform Law Commission and the American Law Institute approved Article 12 of the Uniform Commercial Code, "Controllable Electronic Records," in 2022. It is a model law, and whether each state has adopted it was not verified.[11]

Japan

In Japan, the payments statute and the financial-instruments statute divide the work. The Payment Services Act (資金決済に関する法律, 平成21年法律第59号 (Act No. 59 of 2009)) replaced the defined term "仮想通貨" (kasō tsūka, "virtual currency") with "暗号資産" through 令和元年法律第28号 (Act No. 28 of 2019; promulgated 2019-06-07, in force 2020-05-01), and the current definition is in Article 2(14) (it moved from paragraph 5 to paragraph 14 with the version in force from 2023-06-01).[35] Article 63-2 of the same Act establishes a registration system, providing that "暗号資産交換業は、内閣総理大臣の登録を受けた者でなければ、行ってはならない" (gloss: "no person other than one registered by the Prime Minister may carry on crypto-asset exchange business"), and Article 2(15) sets the scope of exchange business (buying and selling or exchanging; intermediation, brokerage or agency for these; management of users' money in connection with them; management of crypto-assets on behalf of others).[36] Stablecoins that are assets denominated in legal tender are separately defined as "電子決済手段" (denshi kessai shudan, "electronic payment instruments") (Article 2(5)). Tokens with an investment character are governed by the financial-instruments statute as "電子記録移転権利" (denshi kiroku iten kenri, "electronically recorded transferable rights") in the parenthesis of Article 2(3) of the Financial Instruments and Exchange Act. This pairs with the proviso to the crypto-asset definition, which excludes "金融商品取引法第二十九条の二第一項第八号に規定する権利を表示するもの" (gloss: "those that represent the rights specified in Article 29-2(1)(viii) of the Financial Instruments and Exchange Act").[37] 令和8年法律第64号 (Act No. 64 of 2026), promulgated on July 23, 2026, is an amendment that moves the definition of crypto-asset from the Payment Services Act to Article 2(49) of the Financial Instruments and Exchange Act. **It has been promulgated and partly brought into force, but the part transferring the definition is not yet in force as of the reference date.** The e-Gov statute search displays that version as scheduled to take effect on 2027-07-22, but the cabinet order that actually sets the commencement date could not be verified.[38] On private-law nature, there is the Tokyo District Court judgment of March 15, 2019, which stated that bitcoin is not a "有体物" (yūtaibutsu, "tangible thing") and reserved its private-law nature as "一義的に明らかとはいえない" (gloss: "cannot be said to be unambiguously clear") (Chapter 26), and there are eight strands of scholarly opinion (27.2 table, ④–⑥).

United Kingdom

The United Kingdom has legislated in both property law and regulatory law. On the regulatory side, the defined term "cryptoasset" was inserted into section 417(1) of FSMA 2000 by section 69 of the Financial Services and Markets Act 2023: "any cryptographically secured digital representation of value or contractual rights that— (a) can be transferred, stored or traded electronically, and (b) that uses technology supporting the recording or storage of data (which may include distributed ledger technology)," in force from August 29, 2023.[39] On top of that, The Financial Services and Markets Act 2000 (Cryptoassets) Regulations 2026 (SI 2026/102), made on February 4, 2026, made the "qualifying cryptoasset" (fungible, transferable, not solely a record of value or contractual rights; electronic money, fiat currency and tokens within a limited network excluded) and the "qualifying stablecoin" (one that aims at a stable value in relation to a particular fiat currency and for which fiat currency or other assets are held to maintain it) objects of regulated activities. **The full commencement day of these Regulations is October 25, 2027.** As of the reference date only the parts for preparatory purposes, such as the FCA's making of rules and guidance and the receipt and determination of applications for authorization, are in force (reg. 1(3)).[40] On the property side, the Law Commission in its June 2023 final report (Law Com No 412) recommended legislation confirming the "third category," and the Property (Digital Assets etc) Act 2025 received Royal Assent on December 2, 2025 and came into force the same day. It has two sections. It states only that the fact that a thing is neither "a thing in possession" nor "a thing in action" does not by itself prevent it from being the object of personal property rights, and it does not decide what is property (27.2 ⑧).[89] In case law there is *Tulip Trading Ltd v van der Laan and others* [2023] EWCA Civ 83 (decided February 3, 2023, Lewison, Popplewell and Birss LJJ) in the Court of Appeal (Civil Division) of England and Wales. In a case in which a company claiming to be the holder, which alleged it had lost its private keys to a hack, asserted fiduciary duties against the developers of the bitcoin network, the Court of Appeal said, "The conclusion is not that there is a fiduciary duty in law in the circumstances alleged by Tulip, only that the case advanced raises a serious issue to be tried," and overturned the first-instance judgment (High Court, Chancery Division, Falk J, [2022] EWHC 667 (Ch)), which had set aside permission to serve out of the jurisdiction. Birss LJ added: "If the decentralised governance of bitcoin really is a myth, then in my judgment there is much to be said for the submission that bitcoin developers, while acting as developers, owe fiduciary duties to the true owners of that property."[41] This is the form in which Chapter 8's "whose agent is a miner?" and Chapter 23's "who decides the real ledger?" entered the courtroom, and 28.1(g) takes it up.

Question boxes linked to this section

Three boxes point to this section. Chapter 14's "what did the person who bought an NFT buy?" is taken up by the defined-term row. The EU excluded "unique and not fungible" crypto-assets from the scope of MiCA (Art. 2(3)); the UK's 2026 Regulations made "fungible" a requirement of a "qualifying cryptoasset," which leaves out non-fungible tokens; and in Korea, Article 2, subparagraph 4 of the Enforcement Decree of the Act on the Protection of Virtual Asset Users (가상자산 이용자 보호 등에 관한 법률 시행령) (in force October 2, 2026), made under the delegation in item (h) of Article 2, subparagraph 1 of the Act, excludes "단일하게 존재하여 다른 전자적 증표로 대체할 수 없는 전자적 증표" (gloss: "an electronic token that exists singly and cannot be replaced by another electronic token"), but with a proviso that brings back in "특정 재화나 서비스의 지급수단으로 사용될 수 있는 전자적 증표 등 금융위원회가 정하여 고시하는 전자적 증표" (gloss: "electronic tokens determined and publicly notified by the Financial Services Commission, such as electronic tokens that can be used as a means of payment for particular goods or services") (the originals are in Chapter 30). The same box's "is the issuer's promise to share revenue in the code or outside it?" finds where its answer sits in the fact that the US determination of security status (Howey, SEC, Ripple) looks at the promise outside and at the shape of the transaction. Chapter 15's "does an AMM have an operator?" is taken up by the competent-authority and issuance-and-circulation rows. The regulatory provisions of four jurisdictions presuppose as their addressee "a person" (MiCA Art. 59), "者" (mono, "person") (Payment Services Act Article 63-2), "영업으로 하는 자" (gloss: "a person who does so as a business") (Virtual Asset User Protection Act Article 2, subparagraph 2) or "person" (GENIUS §2(7)) (the addressee provisions of the UK Regulations were not checked in this edition), and in a contract that runs a single constant-product formula that addressee is missing. That Article 2(7)(B)(v) of the GENIUS Act expressly excludes "participating in a liquidity pool or other similar mechanism for the provisioning of liquidity for peer-to-peer transactions" from "digital asset service provider" is a fact showing that the legislature recognized this gap; whether that was the right choice, this book does not say.[42] Chapter 24's "is a stablecoin holder a depositor, a creditor or a beneficiary?" is taken up by the stablecoin row. The EU allowed only credit institutions and electronic money institutions to issue e-money tokens (Art. 48), the United States took payment stablecoins out of securities and commodities and set up a system of permitted issuers (GENIUS §17), Japan defined them separately as "電子決済手段," the United Kingdom made the issuance of a "qualifying stablecoin" a regulated activity, and Korea did not create a separate category. No jurisdiction decided the holder's private-law position (depositor, creditor, beneficiary) directly in those definitional provisions. That question is for Chapter 28.

What the table leaves out

Two things should be stated. First, Singapore and Switzerland are not included in this edition (the original plan was "a line each if needed," and this book's source materials contained no primary sources for those two jurisdictions). Second, **bills pending** in each jurisdiction are not recorded. Under this book's principles they are matters on which facts alone may be stated, but this book's source materials did not contain the texts of the bills, so even the facts could not be verified. Japan's transfer to the Financial Instruments and Exchange Act is not "pending" but "promulgated, not yet in force," so it is in the table. The US GENIUS Act has also been enacted, so it is in the table, with only its effective date recorded as not settled.

Summary

This chapter has done three things. It set out the five reasons for not treating the law in depth, paired fourteen positions on private-law nature with the technical facts of Parts 1–3, and compared five jurisdictions on six axes. Lay the two tables over each other and one thing comes into view. **Legislation answered "what is regulated?" with definitions, courts avoided the question of nature, "what is it?", when they could, and scholarship used the same technical facts as grounds for different conclusions.** Knowing the technical facts, then, is not a matter of choosing a position but of knowing **what each position has to explain**. What a person who lost a private key has lost (Chapter 5), when a payment is completed in a world where finality is a probability (Chapter 9), whether an asset with no issuer has an obligor (Chapter 19), what the minority can assert when the majority changes the rules (Chapters 22, 23, 26): these questions are taken up by the seven issue sections of Chapter 28. And what turns on five jurisdictions calling the same thing by five names is examined in Chapter 30, which sets the definitional provisions side by side.

Sources for this chapter

- Lee Hye-jeong (이혜정), *가상자산에 대한 민사집행 연구* [A study of civil execution against virtual assets] (사법정책연구원 연구총서 2022-12, 2022): Ch. 2, Sec. 3, "가상자산의 성질" [The nature of virtual assets] (pp. 74–103: United States 74–75, Germany 80–84, Japan 84–85 (A), Korean scholarship pp. 88–98, IV p. 103), Ch. 3 (design of civil execution; summary xviii–xix, Sec. 4 pp. 210–212), the legislative background of the National Tax Collection Act amendment (pp. 166–173). (422 pages; cited pages are printed pages). The scholarly positions are reproduced as the report organizes them, and the individual articles were not checked in the original.
- Supreme Court of Korea, 2018Do3619, May 30, 2018; 2021Do9855, November 11, 2021 (공2022상, 56); 2020Do9789, December 16, 2021: full texts from the Korean Law Information Center (Open API, accessed 2026-10-25). The texts of the judgments were read, and their headnotes and summaries were not cited.
- Act on the Protection of Virtual Asset Users (enacted as Act No. 19563, current Act No. 21857), Articles 2, 6, 7, 10, 13, 14 and table of contents; Specified Financial Transaction Information Act, Article 2, subparagraph 3 and Article 7; Income Tax Act, Article 21(1), subparagraph 27 and the history of postponement in the addenda (Act Nos. 17757, 18578, 19196, 20615); National Tax Collection Act, Article 55(3) and Article 66(2); Local Tax Collection Act, Article 61(3); Civil Act, Articles 98 and 703: Korean Law Information Center (Open API, accessed 2026-10-25). The Income Tax Act provision is not in force, contrary to the "current" display (checked against the original addenda).
- Regulation (EU) 2023/1114 (MiCA), OJ L 150, 9.6.2023, p. 40: Art. 2(3)–(4), 3(1)(5)–(9), 4, 16, 48, 59, 86–92, 93, 117, 149.
- CJEU, *Skatteverket v David Hedqvist*, C-264/14, ECLI:EU:C:2015:718 (22 October 2015): ¶¶24, 42, 52, operative part. Cellar XHTML (accessed 2026-10-25).
- *SEC v. W. J. Howey Co.*, 328 U.S. 293 (1946): pp. 298–300.
- SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017-07-25): pp. 1, 11–15.
- *SEC v. Ripple Labs, Inc.*, No. 1:20-cv-10832 (AT) (S.D.N.Y. July 13, 2023), ECF No. 874, 682 F. Supp. 3d 308: pp. 15–23, 34. The SEC announcement on the course of the appeal (Litigation Release) could not be opened in the original because of bot blocking and was not used.
- *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. Mar. 6, 2018): pp. 3, 24.
- IRS Notice 2014-21, 2014-16 I.R.B. 938; Notice 2023-34, 2023-19 I.R.B. 837. `irs-2023-notice-2023-34-modification-2014-21.pdf`.
- GENIUS Act, Pub. L. No. 119-27, 139 Stat. 419 (July 18, 2025): §2(6), (11), (22), (25), §17, §20.
- *Uniform Commercial Code Amendments (2022)*, Article 12, §12-102: annotated edition p. 235.
- 資金決済に関する法律, Article 2(5), (14), (15), Article 63-2; 金融商品取引法, Article 2(3); Article 2(49) of the 金商法 in the not-yet-in-force version reflecting 令和八年法律第六十四号: e-Gov statute API v2 (accessed 2026-10-25).
- 東京地方裁判所 平成31年3月15日 判決 (平成27年(刑わ)第2274号 etc.) [Tokyo District Court,

judgment of March 15, 2019]: "事実認定の補足説明" 第１の２(３).

- Financial Services and Markets Act 2000, s. 417(1); Financial Services and Markets Act 2023, s. 69: legislation.gov.uk (accessed 2026-10-25). The Financial Services and Markets Act 2000 (Cryptoassets) Regulations 2026, SI 2026/102: reg. 1, RAO art. 88F, 88G.
- Law Commission, *Digital Assets: Final Report* (Law Com No 412, 2023): ¶¶2.45–2.61. Property (Digital Assets etc) Act 2025 (c. 29).
- *Tulip Trading Ltd v van der Laan and others* [2023] EWCA Civ 83 (3 February 2023): ¶¶1–12, 91–93.
- UNIDROIT, *Principles on Digital Assets and Private Law* (2023): Principle 2, Introduction 0.6.
- Gorton & Zhang (2023), p. 920: the same place as Chapter 24.
- The Korean edition's Korean names for foreign courts and institutions followed the Korean judiciary's "외국 법원 및 사법관련 국제기구 번역 가이드라인" [Guidelines on the Korean translation of the names of foreign courts and international judicial bodies] (review-draft edition,): 미국 연방대법원 (Supreme Court of the United States), 미국 연방뉴욕남부지방법원 (United States District Court for the Southern District of New York), 미국 연방뉴욕동부지방법원 (United States District Court for the Eastern District of New York), 도쿄지방재판소 (Tokyo District Court), 영국 항소법원 (Court of Appeal), 유럽연합 사법재판소 (Court of Justice of the European Union) (*Hedqvist* was decided by the Court of Justice (Fifth Chamber); the guideline records "유럽연합 사법재판소" as the usage for the CJEU and notes "유럽연합 법원" as an option under consideration, and "유럽연합 대법원·최고법원" as options under consideration for the ECJ). "High Court" was left in the original language because the guideline lists three options without settling on one.
- This chapter has no code or notebook (by design). It has no price or market-capitalization data. Pending bills were not recorded because their texts could not be verified.

Notes

1. Lee Hye-jeong (이혜정), *가상자산에 대한 민사집행 연구* [A study of civil execution against virtual assets] (사법정책연구원 연구총서 2022-12 [JPRI research series 2022-12], published 2022-08-05), p. 103 (Ch. 2, Sec. 3, IV). Original: "한편, 가상자산을 물건으로 보고 그에 대한 권리를 인정할지에 대해 견해가 통일되어 있지 않아, 가상자산을 취득한 자가 '가상자산을 소유하고 있다'고 단정 짓기 힘들고, 이러한 고민은 가상자산 관련 다수의 논문과 판결에서 '소유' 대신 '보유'라는 단어를 사용하고 있는 것에서도 확인할 수 있다." (PDF p. 127). Page numbers in the footnotes below are printed page numbers. Working gloss of the latter half: "this concern can also be seen in the fact that many papers and judgments relating to virtual assets use the word 'holding' instead of 'ownership.'"

2. Lee (2022), pp. 88–89 (Ch. 2, Sec. 3, III.1.A). Grounds of the affirmative view: "비트코인은 '비트코인 주소'에서 관리되고, 개인키를 가진 자에 의하여 기술적으로 통제받으므로 민법 제98조의 자연력에 해당한다는 견해" (footnote 417, Kim I-su (김이수)), "당해 가상자산지갑의 개인키를 가진 자만이 송금지시를 내릴 수 있고(배타적 지배가능성), 그러한 송금지시 내역이 분산원장에 빠짐없이 기록되며, 한 번 송금이 완료된…비트코인은 전 세계 비트코인 네트워크 참여자의 과반수를 동시에 해킹하지 않는 이상 현실적으로 이중지불 될 수 없으므로(관리가능성), 비트코인은 민법상 물건 중 동산이라는 견해" (footnote 418, Jeon Seung-jae (전승재) and Kwon Heon-young (권헌영)). Grounds of the negative view (p. 89): "개인키는 단순한 정보에 불과하고 복제가 가능할 뿐만 아니라, 정보에 대한 '점유'나 '보관'이라는 개념도 명확치 않아 엄격한 의미의 '배타성'이 있다고 보기 어렵다" (footnote 419, Ki No-seong (기노성)), "네트워크 참여자에 의한 승인이 없는 한 비트코인의 이전이 완결되지 않으므로 배타적 지배가능성이 다소 불완전하다" (footnote 420, Jeong Yeong-su (정영수)), "비트코인에서 에너지를 추출할 수 없으므로 전통적인 의미의 '자연력'으로 보기 어렵다" (footnote 421). Original of Article 98 of the Civil Act: "본법에서 물건이라 함은 유체물 및 전기 기타 관리할 수 있는 자연력을 말한다." (Korean Law Information Center (국가법령정보센터), accessed 2026-10-25). This book reproduces the scholarly positions as the report organizes them and did not check the individual articles in the original. Working gloss: footnote 417: "the view that bitcoin is managed at a 'bitcoin address' and is

technically controlled by the person who has the private key, and so falls within the natural forces of Article 98 of the Civil Act"; footnote 419: "a private key is mere information and can be copied, and the concepts of 'possession' or 'custody' of information are also unclear, so it is hard to say that there is 'exclusivity' in the strict sense"; footnote 420: "unless approved by network participants, a transfer of bitcoin is not completed, so exclusive controllability is somewhat incomplete"; Article 98: "In this Act, 'things' means tangible things and electricity and other natural forces that can be managed."

3. Lee (2022), pp. 89–90 (III.1.B). Original: "비트코인 보유자는 비트코인의 재산적 가치에 대해 독점적이고 배타적인 지배권을 행사하고, 거래의 상대방뿐만 아니라 누구에게나 비트코인에 대한 권리를 주장할 수 있으므로 물권과 유사한 권리를 갖는다고 보아야 하나, 이는 법률에 따라 부여된 권리가 아니므로 준물권과 유사한 성격의 '준물권적 구조를 가진 권리'로 본다" (footnote 422, Lee Na-rae (이나래)); contra: "블록체인의 기록은 비트코인 네트워크의 모든 참가자에 의해 공유되고 있으므로 설령 개인키를 가지고 있어도 비트코인을 독점적으로 지배하고 있다고 말하기는 쉽지 않다" (footnote 423, Jeong Yeong-su). The objection from the numerus clausus of real rights against Japanese positions ②–④ is at p. 85 (footnote 402).

4. Lee (2022), p. 90 (III.1.C and D). Original: "① 송금지시의 목적이 되는 '당해 송금지시를 포함한 블록의 생성'이라는 급부는 블록체인 네트워크상에서 실행되는 컴퓨터 프로그램에 의해 이행되고 참여자 개개인은 여기에 필요한 컴퓨팅 자원을 각자 조금씩 제공할 뿐이므로 참여자 개개인을 '채무자'로 보기 힘들고, ② 비트코인 네트워크는 책임재산도 없고 사람에 의해 운영되는 중앙 통제기관도 없으므로 법인·단체로 취급하기도 곤란하며, ③ 비트코인 네트워크를 움직이는 컴퓨터 프로그램 자체를 채무자의 지위에 놓고 법을 따르라고 할 방법도 없다" (footnotes 424–425, Jeon Seung-jae and Kwon Heon-young); the view that it can be the object of a claim (footnote 426). A "가상자산이전청구권" (claim for the transfer of virtual assets) against an exchange is what the enforcement design in Chapter 3 of the report presupposes (pp. xviii–xix). Working gloss of ③: "③ there is also no way to put the computer program that runs the bitcoin network itself in the position of obligor and tell it to obey the law."

5. Lee (2022), pp. 84–85 (II.5.A). This book reproduces the gist of positions ①–⑧ and the comments on each. Original (position ⑦): "비트코인에는 발행자가 존재하지 않는 점이나 소유권의 객체는 유체물에 한정되는 점 등을 근거로 비트코인의 보유는 물권·채권·사원권·지적재산권이라는 재산권을 수반하는 것이 아니라 사실상태에 불과하다는 견해" (footnote 400, 片岡義広 and 武内斉史); assessment of position ⑦: "이 견해에 따르면 물권적 청구의 우려가 배제되어 '소유와 점유의 일치'를 원칙으로 하는 금전과 유사한 취급이 확보되기 때문에 비트코인의 지불수단으로서의 유용성 확보라는 점에서 타당하다는 의견" (footnote 405); criticism of position ⑧: "합의의 법적 성질이나 법적 효과가 명확하지 않고, 프로토콜과는 구별되는 참가자 전원의 어떤 규범적 합의가 있다고 보기 어렵다" (footnote 406); assessment of position ⑥: "비트코인의 채굴에 의해 사상이 창작적으로 표현되어 있다고 평가하기 곤란하다" (footnote 404). The Japanese literature was not checked in the original. Working gloss: footnote 405: "the opinion that this view is sound in securing bitcoin's usefulness as a means of payment, because under it the risk of real-right claims is excluded and a treatment similar to money, governed by the principle of the 'coincidence of ownership and possession,' is secured"; footnote 406: "the legal nature and legal effect of the agreement are not clear, and it is hard to say that there is some normative agreement of all participants distinct from the protocol."

6. Lee (2022), p. 85 (position ⑤ and its criticism: "단체의 성립·가입·탈퇴를 위한 법령이나 의사표시가 없는 이상 법적인 의미에서의 단체가 존재하지 않는다고 볼 수밖에 없으므로", footnote 403), p. 93 (III.1.G and H: the view denying a membership right (footnote 439); the partnership-right view: "'프로그램 시스템'을 계약의 내용으로, '블록체인의 내부적 거래'를 공동사업으로, '비트코인의 취득'을 상호 출자로 이해하여, 블록체인이 특수한 조합이고, 비트코인은 조합권이 될 수 있다는 견해가 있는데, 단, 이 견해도 비트코인이라는 조합권은 향후 예상되는 이익분배에 대한 기대권이며"). Original of Article 703(1) of the Civil Act: "조합은 2인 이상이 상호출자하여 공동사업을 경영할 것을 약정함으로써 그 효력이 생긴다." (accessed 2026-10-25). Working gloss of Article 703(1): "A partnership takes effect when two or more persons agree to make mutual contributions and to carry on a common undertaking."

7. Lee (2022), pp. 81, 83–84 (II.4). Original: "유체성을 가지고 있지 못하다는 점에 대해서는 학설상 다툼이 없으므로" (p. 81, footnote 372); "가상자산이 채권이 아니라는 점에 대해서 학설이 일치한다" (footnote 388); "결국 가상자산은 민법상 물건은 아니지만 그 상위개념인 목적물로서 무형의 이익에 속한다는 점에 관해 학설이 일치하고 있다" (footnote 393: Bachert 2021, Badstuber 2019, Boehm/Bruns 2017, Kütük/Sorge 2014, Spindler/Bille 2014). The German literature was not checked in the original.

8. Law Commission, *Digital Assets: Final Report* (Law Com No 412, HC 1486, ordered to be printed 2023-06-27, published 2023-06-28), p. 19 ¶2.45, p. 22 ¶2.53, p. 23 ¶¶2.57–2.58, 2.60. Original: "we conclude that some digital assets are neither things in possession nor things in action, but that nonetheless the law of England and Wales treats them as capable of being things to which personal property rights can relate."(¶2.45); "We

recommend legislation confirming that the fact that a thing is neither a thing in possession nor a thing in action does not prevent it from being a thing to which personal property rights can relate. Such things include crypto-tokens like bitcoin or ether which are not rights or claims in themselves and which can be used and enjoyed independently of whether any rights or claims in relation to them are enforceable by action."(¶2.53); the criteria (now "indicia"): "(1) it is composed of data ...; (2) it exists independently of persons and exists independently of the legal system; and (3) it is rivalrous."(¶2.57), "we conclude that 'composed of data' need not be a criterion in itself"(¶2.58); the definition of rivalrousness (footnote 82): "A thing is rivalrous if the use or consumption of the thing by one person, or a specific group of persons, necessarily prejudices the use or consumption of that thing by one or more other persons." (304 pages; cited pages are printed pages). Footnote 62 of the report states, "Our report does not consider regulatory matters ...."

9. Property (Digital Assets etc) Act 2025 (c. 29), Royal Assent 2 December 2025, s. 1, s. 2. Original: "1 Objects of personal property rights / A thing (including a thing that is digital or electronic in nature) is not prevented from being the object of personal property rights merely because it is neither— (a) a thing in possession, nor (b) a thing in action."; "2 ... (1) This Act extends to England and Wales and Northern Ireland. (2) This Act comes into force on the day on which it is passed." (legislation.gov.uk print version). The Act has only two sections.

10. UNIDROIT, *Principles on Digital Assets and Private Law* (Rome, 2023), Principle 2(1) and (2), p. 11. Original: "(1) 'Electronic record' means information which is (i) stored in an electronic medium and (ii) capable of being retrieved. (2) 'Digital asset' means an electronic record which is capable of being subject to control." Introduction 0.6 states that "control" is not the same as 'control' in common law jurisdictions or 'possession' in civil law: "The concept of control used in these Principles, for instance, is not intended to be understood as 'control' as used in certain common law jurisdictions. Also, while being akin to the concept of 'possession' as used in certain civil law jurisdictions, control as used in these Principles must not be understood to be identical to such possession" (p. 2). Adopted by the Governing Council at its 102nd session (2023-05).

11. *Uniform Commercial Code Amendments (2022)*, Article 12 "Controllable Electronic Records," § 12-102(a)(1), annotated edition p. 235. Original: "'Controllable electronic record' means a record stored in an electronic medium that can be subjected to control under Section 12-105. The term does not include a controllable account, a controllable payment intangible, a deposit account, an electronic copy of a record evidencing chattel paper, an electronic document of title, electronic money, investment property, or a transferable record." Approved at the annual meetings of the ALI (May 2022) and the ULC (July 2022) (preface). A uniform act is a model law, and this book did not verify whether or when each state adopted it.

12. Lee (2022), pp. 91–92 (III.1.E). Original (negative view): "가치척도의 기준이 되기 위해서는 그 가치가 일정해야 하고 발행 수량을 유동적으로 조절하여 상황에 따라 가치기준을 유지할 수 있어야 하는데, 비트코인은 시장가격이 급등락하고, 발행총량이 2100만 BTC로 한정되어 있기 때문에 화폐로서의 기능을 할 수 없다는 것이다" (footnote 431); (affirmative view): "현재보다는 미래에 화폐로 수용될 가능성을 긍정적으로 전망하고 있다" (footnotes 432–434). This book reproduces both sides but does not join in their view of the future (the notes to the reader in Chapter 3). Working gloss: negative view: "to serve as a standard measure of value, its value must be stable and the amount issued must be adjustable so that the standard of value can be maintained as circumstances require, but bitcoin's market price moves up and down sharply and its total issuance is limited to 21 million BTC, so it cannot function as money"; affirmative view: "they view favorably the possibility that it will be accepted as money in the future rather than at present."

13. Judgment of the Court (Fifth Chamber) of 22 October 2015, *Skatteverket v David Hedqvist*, Case C-264/14, ECLI:EU:C:2015:718 (confirmed from the Cellar metadata), ¶¶24, 42, 52, 57, operative parts 1 and 2. Original: "the 'bitcoin' virtual currency with bidirectional flow, which will be exchanged for traditional currencies in the context of exchange transactions, cannot be characterised as 'tangible property' within the meaning of Article 14 of the VAT Directive, given that, as the Advocate General has observed in point 17 of her Opinion, that virtual currency has no purpose other than to be a means of payment."(¶24); "The 'bitcoin' virtual currency, being a contractual means of payment, cannot be regarded as a current account or a deposit account, a payment or a transfer."(¶42); "In the case in the main proceedings, it is common ground that the 'bitcoin' virtual currency has no other purpose than to be a means of payment and that it is accepted for that purpose by certain operators."(¶52); operative part 2: "Article 135(1)(e) of Directive 2006/112 must be interpreted as meaning that the supply of services such as those at issue in the main proceedings, which consist of the exchange of traditional currencies for units of the 'bitcoin' virtual currency and vice versa, ..., are transactions exempt from VAT, within the meaning of that provision". Cellar XHTML (`publications.europa.eu/resource/cellar/bb73ad87-78a7-11e5-86db-01aa75ed71a1.0006.01/DOC_2`,

accessed 2026-10-25). The judgment interprets the VAT Directive and is not a ruling on private-law nature.

14. IRS Notice 2014-21, 2014-16 I.R.B. 938, §4 Q&A-1 and Q&A-2, p. 2. Original: "Q-1: How is virtual currency treated for federal tax purposes? A-1: For federal tax purposes, virtual currency is treated as property. General tax principles applicable to property transactions apply to transactions using virtual currency."; "Q-2: … A-2: No. Under currently applicable law, virtual currency is not treated as currency that could generate foreign currency gain or loss for U.S. federal tax purposes." IRS Notice 2023-34, 2023-19 I.R.B. 837, §3: "The Department of the Treasury and the Internal Revenue Service are aware that certain foreign jurisdictions have enacted laws that characterize Bitcoin as legal tender. Thus, the sentence in the Background section of Notice 2014-21 stating that virtual currency does not have legal tender status in any jurisdiction is no longer accurate as to Bitcoin."; end of §3: "This change to the Background section of Notice 2014-21 does not affect the answers to the FAQs set forth in section 4 of Notice 2014-21 …". `irs-2023-notice-2023-34-modification-2014-21.pdf`.

15. Supreme Court of Korea, 2020Do9789, December 16, 2021 (below: Suwon High Court (수원고등법원), 2020No171, July 2, 2020; reversed and remanded), Reasons 3.C. Original: "가상자산은 국가에 의해 통제받지 않고 블록체인 등 암호화된 분산원장에 의하여 부여된 경제적인 가치가 디지털로 표상된 정보로서 재산상 이익에 해당한다(대법원 2021. 11. 11. 선고 2021도9855 판결 참조). 가상자산은 보관되었던 전자지갑의 주소만을 확인할 수 있을 뿐 그 주소를 사용하는 사람의 인적사항을 알 수 없고, 거래 내역이 분산 기록되어 있어 다른 계좌로 보낼 때 당사자 이외의 다른 사람이 참여해야 하는 등 일반적인 자산과는 구별되는 특징이 있다. 이와 같은 가상자산에 대해서는 현재까지 관련 법률에 따라 법정화폐에 준하는 규제가 이루어지지 않는 등 법정화폐와 동일하게 취급되고 있지 않고 그 거래에 위험이 수반되므로, 형법을 적용하면서 법정화폐와 동일하게 보호해야 하는 것은 아니다."; Reasons 3.A: "가상자산을 이체받은 자는 가상자산의 권리자 등에 대한 부당이득반환의무를 부담하게 될 수 있다. 그러나 이는 당사자 사이의 민사상 채무에 지나지 않고 … 피고인은 어떠한 경위로 이 사건 비트코인을 이체받은 것인지 불분명하여 부당이득반환청구를 할 수 있는 주체가 피해자인지 아니면 거래소인지 명확하지 않다." Full text of the judgment from the Korean Law Information Center (Open API, judgment serial number 219775, accessed 2026-10-25). The text of the judgment was read; its headnotes and summary were not cited. Working gloss: Reasons 3.C, first sentence: "A virtual asset is information, not controlled by the state, in which economic value conferred by an encrypted distributed ledger such as a blockchain is represented digitally, and it constitutes a pecuniary advantage (see Supreme Court Decision 2021Do9855 decided November 11, 2021)"; third sentence, up to "therefore": "such virtual assets have not so far been subjected under the relevant statutes to regulation equivalent to that of legal tender and are thus not treated the same as legal tender, and their trading involves risk"; Reasons 3.A: "a person who has received a transfer of virtual assets may come to bear an obligation to make restitution of unjust enrichment to the right holder of the virtual assets or others. But this is no more than a civil obligation between the parties … it is unclear by what process the defendant came to receive the bitcoin in this case." (대법원 2021. 12. 16. 선고 2020도9789 판결)

16. Lee (2022), pp. 92–93 (III.1.F). Original: "비트코인을 '유가증권'으로 보기 힘들다는 견해가 대체적인바, 그 논거로 ① 비트코인은 발행인이 없고, 블록체인 상에 트랜잭션이 기재되는 것을 유가증권의 배서로 보기 힘들므로, 발행행위가 필요하고 배서·교부에 의해 이전하는 '유가증권'에 비트코인이 포함된다고 보기 힘든 점, ② … 비트코인 그 자체는 교환의 매개물일 뿐 별도의 권리나 청구권이 화체되어 있지 않다는 점 등이 제시된다" (footnotes 436–438). The discussion of whether it is a financial investment product under the Financial Investment Services and Capital Markets Act (자본시장과 금융투자업에 관한 법률) is at pp. 95–97 of the same report. Working gloss of ②: "bitcoin itself is only a medium of exchange, and no separate right or claim is embodied in it."

17. *SEC v. W. J. Howey Co.*, 328 U.S. 293 (1946) (decided May 27, 1946), pp. 298–299. Original: "an investment contract for purposes of the Securities Act means a contract, transaction or scheme whereby a person invests his money in a common enterprise and is led to expect profits solely from the efforts of the promoter or a third party, it being immaterial whether the shares in the enterprise are evidenced by formal certificates or by nominal interests in the physical assets employed in the enterprise."; "It embodies a flexible rather than a static principle, one that is capable of adaptation to meet the countless and variable schemes devised by those who seek the use of the money of others on the promise of profits."(p. 299); "the arrangements whereby the investors' interests are made manifest involve investment contracts, regardless of the legal terminology in which such contracts are clothed."(p. 300). (the original U.S. Reports, Library of Congress).

18. SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017-07-25), pp. 1, 11–15. Original: "the Commission has determined that DAO Tokens are securities under the Securities Act of 1933 … and the Securities Exchange Act of 1934"(p. 1); "In analyzing whether something is a security, 'form should be disregarded for substance,' …, 'and the emphasis should be on economic realities underlying a transaction, and not on the name appended thereto.'"(p. 11); "Their efforts, not those of DAO Token holders, were the 'undeniably

significant' ones, essential to the overall success and profitability of any investment into The DAO."(p. 15). The same place as Chapter 26, section 3.

19. *SEC v. Ripple Labs, Inc.*, No. 1:20-cv-10832 (AT) (S.D.N.Y. July 13, 2023) (Torres, J.), Order, ECF No. 874, 682 F. Supp. 3d 308, pp. 15, 23, 34. Original: "XRP, as a digital token, is not in and of itself a 'contract, transaction[,] or scheme' that embodies the Howey requirements of an investment contract."(p. 15); institutional sales: the three Howey requirements are met (pp. 16–22); programmatic sales: "Ripple's Programmatic Sales were blind bid/ask transactions, and Programmatic Buyers could not have known if their payments of money went to Ripple, or any other seller of XRP."(p. 23); "the SEC's motion for summary judgment is GRANTED as to the Institutional Sales, and otherwise DENIED. Defendants' motion for summary judgment is GRANTED as to the Programmatic Sales, the Other Distributions, and Larsen's and Garlinghouse's sales"(p. 34). (RECAP). The reporter citation 682 F. Supp. 3d 308 was confirmed from its citation in the same court's order of 2024-08-07 (ECF 973). The SEC announcement reporting the later course of the appeal could not be opened in the original because of bot blocking, so this book does not state it.

20. Guiding and Establishing National Innovation for U.S. Stablecoins Act (GENIUS Act), Pub. L. No. 119-27, 139 Stat. 419 (July 18, 2025), §2(22), §17, §20. Original: "(22) PAYMENT STABLECOIN.—The term 'payment stablecoin'— (A) means a digital asset— (i) that is, or is designed to be, used as a means of payment or settlement; and (ii) the issuer of which— (I) is obligated to convert, redeem, or repurchase for a fixed amount of monetary value, not including a digital asset denominated in a fixed amount of monetary value; and (II) represents that such issuer will maintain, or create the reasonable expectation that it will maintain, a stable value relative to the value of a fixed amount of monetary value; and (B) does not include a digital asset that— (i) is a national currency; (ii) is a deposit ...; or (iii) is a security, ..., except that, for the avoidance of doubt, no bond, note, evidence of indebtedness, or investment contract that was issued by a permitted payment stablecoin issuer shall qualify as a security solely by virtue of its satisfying the conditions described in subparagraph (A), consistent with section 17 of this Act."(139 Stat. 421–422); title of §17: "AMENDMENTS TO CLARIFY THAT PAYMENT STABLECOINS ARE NOT SECURITIES OR COMMODITIES AND PERMITTED PAYMENT STABLECOIN ISSUERS ARE NOT INVESTMENT COMPANIES"; §20: "This Act, and the amendments made by this Act, shall take effect on the earlier of (1) the date that is 18 months after the date of enactment of this Act; or (2) the date that is 120 days after the date on which the primary Federal payment stablecoin regulators issue any final regulations implementing this Act." (govinfo). Counting from the enactment date 2025-07-18, the upper limit is 2027-01-18, and an earlier effective date depends on the issuance of final regulations. Whether the regulations had been issued as of 2026-10-25 this book did not verify (Appendix A). The sentence in §17(a), "The term 'security' does not include a payment stablecoin ...", is added to §202(a)(18) of the Investment Advisers Act; the Securities Act and Securities Exchange Act side is in other subsections of the same section.

21. *CFTC v. McDonnell*, No. 18-CV-361 (JBW), 287 F. Supp. 3d 213 (E.D.N.Y. Mar. 6, 2018) (Weinstein, J.), Memorandum & Order, pp. 3, 24. Original: "Until Congress clarifies the matter, the CFTC has concurrent authority, along with other state and federal administrative agencies, and civil and criminal courts, over dealings in virtual currency."(p. 3); "Virtual currencies can be regulated by CFTC as a commodity. Virtual currencies are 'goods' exchanged in a market for a uniform quality and value. ... They fall well-within the common definition of 'commodity' as well as the CEA's definition of 'commodities' as 'all other goods and articles . . . in which contracts for future delivery are presently or in the future dealt in.' Title 7 U.S.C. § 1(a)(9)."(p. 24); "The jurisdictional authority of CFTC to regulate virtual currencies as commodities does not preclude other agencies from exercising their regulatory power when virtual currencies function differently than derivative commodities."(p. 24). (govinfo). It is a decision at the preliminary-injunction stage.

22. Supreme Court of Korea, 2021Do9855, November 11, 2021 (below: Seoul High Court (서울고등법원), 2020No357, July 9, 2021; appeal dismissed), Reasons 2.A. Original: "비트코인은 경제적인 가치를 디지털로 표상하여 전자적으로 이전, 저장과 거래가 가능하도록 한 가상자산의 일종으로 사기죄의 객체인 재산상 이익에 해당한다." Referenced provision: Article 347 of the Criminal Act (형법). Full text of the judgment from the Korean Law Information Center (Open API, judgment serial number 222659) (accessed 2026-10-25). Court Gazette: 공2022상, 56 (from the reference-precedent indication in the 2020Do9789 judgment). (대법원 2021. 11. 11. 선고 2021도9855 판결)

23. Supreme Court of Korea, 2018Do3619, May 30, 2018 (below: Suwon District Court (수원지방법원), 2017No7120, January 30, 2018; appeal dismissed), Reasons 1.A.3) and 5). Original: "범죄수익은닉규제법에 정한 중대범죄에 해당하는 범죄행위에 의하여 취득한 것으로 재산적 가치가 인정되는 무형재산도 몰수할 수 있다."; "피고인이 범죄수익은닉규제법에 정한 중대범죄에 해당하는 정보통신망법 위반(음란물유포)죄와 도박개장방조죄에 의하여 취득한 비트코인은 재산적 가치가 있는 무형의 재산이라고 보아야 한다. 그 이유는 다음과 같다. ① 비트코인은 경제적인

가치를 디지털로 표상하여 전자적으로 이전, 저장 및 거래가 가능하도록 한, 이른바 '가상화폐'의 일종이다. ② 피고인은 …비트코인을 대가로 지급받아 재산적 가치가 있는 것으로 취급하였다." Full text of the judgment from the Korean Law Information Center (Open API, judgment serial number 197020) (accessed 2026-10-25). The 2018 judgment uses the word "가상화폐" (gasang hwapye, "virtual currency"), and the two 2021 judgments use "가상자산": Chapter 30. Working gloss: "Intangible property recognized as having property value that was acquired through criminal conduct constituting a serious crime under the Criminal Proceeds Concealment Act [Act on Regulation and Punishment of Criminal Proceeds Concealment (범죄수익은닉의 규제 및 처벌 등에 관한 법률)] may also be confiscated."; "The bitcoin that the defendant acquired through the crimes of violating the Information and Communications Network Act [Act on Promotion of Information and Communications Network Utilization and Information Protection (정보통신망 이용촉진 및 정보보호 등에 관한 법률)] (distribution of obscene material) and of aiding the opening of a gambling place, which constitute serious crimes under the Criminal Proceeds Concealment Act, should be regarded as intangible property having property value. The reasons are as follows. ① Bitcoin is a kind of so-called 'virtual currency' that represents economic value digitally so that it can be transferred, stored and traded electronically. ② The defendant … received bitcoin as consideration and treated it as something having property value." (대법원 2018. 5. 30. 선고 2018도3619 판결)

24. Gary B. Gorton & Jeffery Y. Zhang, "Taming Wildcat Stablecoins," *University of Chicago Law Review* 90(3): 909–972 (2023), p. 920. The original is reproduced in Chapter 24, footnote 29: "In order for the stablecoins to be considered a deposit—let alone a demand deposit—one must determine whether the underlying contract between the holder of stablecoins and the stablecoin issuer is an equity contract or a debt contract."

25. 東京地方裁判所 平成31年3月15日 判決 (平成27年(刑わ)第2274号 etc.) [Tokyo District Court, judgment of March 15, 2019], "事実認定の補足説明" [supplementary explanation of the findings of fact] 第１の２(３). The same place as Chapter 26, footnote 12.

26. Act on the Protection of Virtual Asset Users (가상자산 이용자 보호 등에 관한 법률), Article 2, subparagraph 1 (wording as enacted by Act No. 19563, unchanged in the current Act No. 21857); Regulation (EU) 2023/1114 Art. 3(1)(5). The originals are reproduced in Chapter 30.

27. Ministry of Government Legislation, Korean Law Information Center, history of the 가상자산 이용자 보호 등에 관한 법률 (Open API, MST 290735, statute ID 014474, article history, accessed 2026-10-25). Enacted as Act No. 19563 (promulgated July 18, 2023; in force under Article 1 of the Addenda on "공포 후 1년이 경과한 날" (gloss: "the day on which one year has elapsed after promulgation") → July 19, 2024); current Act No. 21857 (promulgated August 4, 2026, in force October 2, 2026). Table of contents: Article 1 (Purpose) through Article 22 (Administrative fines) (22 articles). The original of Article 2, subparagraph 1 is reproduced in Chapter 30.

28. 특정 금융거래정보의 보고 및 이용 등에 관한 법률 (current Act, promulgated August 4, 2026, in force October 2, 2026), Article 2, subparagraph 3 and Article 7(1). Original: "3. '가상자산'이란 「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산을 말한다."; "① 가상자산사업자(이를 운영하려는 자를 포함한다. 이하 이 조에서 같다)는 대통령령으로 정하는 바에 따라 다음 각 호의 사항을 금융정보분석원장에게 신고하여야 한다." (Korean Law Information Center Open API (MST 290725, Articles 2 and 7), accessed 2026-10-25). The definition of virtual asset first entered this Act through Act No. 17113 (promulgated March 24, 2020, in force March 25, 2021), and on July 19, 2024 it took its present wording, which applies the Virtual Asset User Protection Act, through the addenda of that Act (the originals of both versions are in Chapter 30). Working gloss of Article 7(1): "(1) A virtual asset service provider (including a person who intends to operate one; the same applies hereinafter in this Article) shall report the matters in the following subparagraphs to the head of the Korea Financial Intelligence Unit, as prescribed by Presidential Decree."

29. Income Tax Act, Article 21(1), subparagraph 27 (inserted by Act No. 17757, promulgated December 29, 2020). Original: "27. 「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산(이하 '가상자산'이라 한다)을 양도하거나 대여함으로써 발생하는 소득(이하 '가상자산소득'이라 한다)". History of postponement of commencement (original text of the amendments to the addenda in each amending Act): Act No. 17757, Addenda Article 1, subparagraph 2, "2022년 1월 1일" → Act No. 18578 (December 8, 2021) "2023년 1월 1일" → Act No. 19196 (December 31, 2022) "2025년 1월 1일" → Act No. 20615 (December 31, 2024) "2027년 1월 1일". The "current" text on the Korean Law Information Center (MST 280405) shows this subparagraph as if it were in force, so this book follows its check of the original addenda and records it as "not in force" (accessed 2026-10-25).

30. National Tax Collection Act (current, in force October 2, 2026), Article 55(3) and Article 66(2), subparagraph 2; Local Tax Collection Act (current, in force October 2, 2026), Article 61(3). Original: "③ 관할 세무서장은 제2항에 따라 「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산(…)을 압류하려는 경우 체납자[같은 법

제2조제2호에 따른 가상자산사업자(...) 등 제3자가 체납자의 가상자산을 보관하고 있을 때에는 그 제3자를 말한다]에게 대통령령으로 정하는 바에 따라 해당 가상자산의 이전을 문서로 요구할 수 있고, 요구받은 체납자 또는 그 제3자는 이에 따라야 한다. <신설 2021.12.21, 2023.7.18>"; "② ... 2. 가상자산사업자를 통해 거래되는 가상자산: 가상자산사업자를 통한 매각 <개정 2021.12.21>"; Local Tax Collection Act Article 61: "③ ... 해당 가상자산의 이전을 요구하여야 한다. 이 경우 제3자에게 이전을 요구하는 경우에는 문서로 하여야 한다. <신설 2022.1.28, 2023.7.18>" (Open API, accessed 2026-10-25). The legislative background of these provisions is in Lee (2022), pp. 166–173. Working gloss of Article 55(3) of the National Tax Collection Act: "(3) Where the competent head of a tax office intends under paragraph (2) to attach a virtual asset under Article 2, subparagraph 1 of the Act on the Protection of Virtual Asset Users (...), the head may, as prescribed by Presidential Decree, request in writing that the delinquent taxpayer [meaning, where a third party such as a virtual asset service provider under Article 2, subparagraph 2 of the same Act (...) holds the delinquent taxpayer's virtual asset in custody, that third party] transfer the virtual asset concerned, and the delinquent taxpayer or third party so requested shall comply."

31. Lee (2022), pp. xviii–xix (summary) and 210–212 (Ch. 3, Sec. 4, IV.2). Original: "이때 민사집행법상 가상자산의 성질을 '그 밖의 재산권'으로 보고, 이를 기초로 채권집행, 동산집행 규정 등을 적절히 조화하여 가상자산에 대한 집행 규정안을 제시하였다"; "① 채무자가 그의 가상자산을 관리할 경우에는 '그 밖의 재산권'인 가상자산 자체를 압류하고, '가상자산 이전명령'에 의해 집행관이 채무자의 가상자산을 이전받으면, 매각명령에 의해 환가된 금전을 집행법원이 배당하는 절차로 집행안을 구상하였고, ② 제3자가 가상자산을 관리하고 채무자가 제3자에 대해 가상자산이전청구권을 가지는 경우에는 '가상자산이전청구권' 자체를 '그 밖의 재산권'으로서 압류한 후". The difficulties of treating it as tangible movable property (there is no technology for marking attachment on a blockchain, and even if a wallet is marked, the object of attachment is not identified) are at pp. 211–212. Working gloss: first passage: "here, the nature of virtual assets under the Civil Execution Act was taken to be 'other property rights,' and on that basis draft provisions on execution against virtual assets were presented that suitably harmonize the provisions on execution against claims, execution against movables and so on"; second passage: "① where the debtor manages the debtor's own virtual assets, the execution scheme devised is a procedure in which the virtual asset itself, as an 'other property right,' is attached, and, once the execution officer has received the debtor's virtual assets by a 'virtual asset transfer order,' the execution court distributes the money realized by a sale order; ② where a third party manages the virtual assets and the debtor has a claim for the transfer of virtual assets against the third party, the 'claim for the transfer of virtual assets' itself is attached as an 'other property right,' and then."

32. Regulation (EU) 2023/1114 of the European Parliament and of the Council of 31 May 2023 on markets in crypto-assets, OJ L 150, 9.6.2023, p. 40, Art. 149. Original: "2. This Regulation shall apply from 30 December 2024. 3. By way of derogation from paragraph 2, Titles III and IV shall apply from 30 June 2024."(L 150/183). (Cellar PDF).

33. MiCA Art. 3(1)(5)–(7) and (9), Art. 2(3)–(4), L 150/62–63. Original: "(5) 'crypto-asset' means a digital representation of a value or of a right that is able to be transferred and stored electronically using distributed ledger technology or similar technology; (6) 'asset-referenced token' means a type of crypto-asset that is not an electronic money token and that purports to maintain a stable value by referencing another value or right or a combination thereof, including one or more official currencies; (7) 'electronic money token' or 'e-money token' means a type of crypto-asset that purports to maintain a stable value by referencing the value of one official currency; ... (9) 'utility token' means a type of crypto-asset that is only intended to provide access to a good or a service supplied by its issuer"; "3. This Regulation does not apply to crypto-assets that are unique and not fungible with other crypto-assets. 4. This Regulation does not apply to crypto-assets that qualify as one or more of the following: (a) financial instruments; (b) deposits, including structured deposits; (c) funds, except if they qualify as e-money tokens; ...".

34. MiCA Art. 4(1) (L 150/67), Art. 16(1) (L 150/76), Art. 48(1) (L 150/107), Art. 59(1), Title VI Art. 86, 89, 91 (L 150/140–143), Art. 93(1) (L 150/144), Art. 117(1) (L 150/161). Original (excerpts): "A person shall not make an offer to the public of a crypto-asset other than an asset-referenced token or e-money token in the Union unless that person: (a) is a legal person; (b) has drawn up a crypto-asset white paper in respect of that crypto-asset in accordance with Article 6"(Art. 4); "A person shall not make an offer to the public, or seek the admission to trading, of an asset-referenced token, within the Union, unless that person is the issuer of that asset-referenced token and is: (a) a legal person or other undertaking that is established in the Union and has been authorised in accordance with Article 21 by the competent authority of its home Member State; or (b) a credit institution that complies with Article 17."(Art. 16); "A person shall not make an offer to the public or seek the admission to trading of an e-money token, within the Union, unless that person is the issuer of such e-money token and: (a) is authorised as a credit institution or as an electronic money institution"(Art. 48); "A

person shall not provide crypto-asset services, within the Union, unless that person is: (a) a legal person or other undertaking that has been authorised as crypto-asset service provider in accordance with Article 63" (Art. 59); "This Title shall also apply to any transaction, order or behaviour concerning crypto-assets as referred to in paragraph 1, irrespective of whether such transaction, order or behaviour takes place on a trading platform."(Art. 86(2)); "No person shall engage in or attempt to engage in market manipulation."(Art. 91(1)); "Member States shall designate the competent authorities responsible for carrying out the functions and duties provided for in this Regulation."(Art. 93(1)); "Where an asset-referenced token has been classified as significant in accordance with Article 43 or 44, the issuer of such asset-referenced token shall carry out its activities under the supervision of EBA."(Art. 117(1)).

35. 資金決済に関する法律 (平成二十一年法律第五十九号), Article 2(14) (e-Gov statute API v2, current version in force `421AC0000000059_20260812_508AC0000000064`, accessed 2026-10-25). The original is reproduced in Chapter 30. Renaming amendment: 令和元年法律第二十八号 "情報通信技術の進展に伴う金融取引の多様化に対応するための資金決済に関する法律等の一部を改正する法律", promulgated 2019-06-07, in force 2020-05-01 (version of 2020-04-30, Article 2(5) "仮想通貨" → version of 2020-05-01 "暗号資産" → from the version of 2023-06-01, paragraph 14). The e-Gov version identifier of the version in force on 2023-06-01, which moved it from paragraph 5 to paragraph 14, is `421AC0000000059_20230601_504AC0000000061` (令和四年法律第六十一号); the amending text itself was not checked.

36. 資金決済法, Article 63-2 and Article 2(15) (same version). Original: "（暗号資産交換業者の登録）
第六十三条の二　暗号資産交換業は、内閣総理大臣の登録を受けた者でなければ、行ってはならない。";
"この法律において「暗号資産交換業」とは、次に掲げる行為のいずれかを業として行うことをいい、... 一　暗号資産の売買又は他の暗号資産との交換　二　前号に掲げる行為の媒介、取次ぎ又は代理　三　その行う前二号に掲げる行為に関して、利用者の金銭の管理をすること。 四　他人のために暗号資産の管理をすること（...）。" The definition of "電子決済手段" in Article 2(5) was also checked in the same version.

37. 金融商品取引法 (昭和二十三年法律第二十五号), Article 2(3) (e-Gov current version `323AC0000000025_20260812_508AC0000000064`, accessed 2026-10-25). Original of the parenthesis: "（電子情報処理組織を用いて移転することができる財産的価値（電子機器その他の物に電子的方法により記録されるものに限る。）に表示される場合（流通性その他の事情を勘案して内閣府令で定める場合を除く。）に限る。以下「電子記録移転権利」という。）". Proviso to Article 2(14) of the 資金決済法: "ただし、金融商品取引法第二十九条の二第一項第八号に規定する権利を表示するものを除く。" Working gloss of the parenthesis: "(limited to cases where [the rights] are represented in property value that can be transferred using an electronic data processing system (limited to property value recorded by electronic means on electronic devices or other things) (excluding cases specified by Cabinet Office Order, taking into account transferability and other circumstances); hereinafter referred to as 'electronically recorded transferable rights')".

38. 令和八年法律第六十四号 "金融商品取引法及び資金決済に関する法律の一部を改正する法律", promulgated 2026-07-23. From the amendment history and the not-yet-in-force version in the e-Gov statute API v2 (`323AC0000000025_20270722_508AC0000000064`, `current_revision_status: UnEnforced`, accessed 2026-10-25): Article 2(49) of the 金融商品取引法, "この法律において「暗号資産」とは、次に掲げるものをいう。ただし、第二十九条の二第一項第八号に規定する権利を表示するものを除く。"; in the not-yet-in-force version of Article 2 of the 資金決済法 of the same date, the definition of 暗号資産 is removed and paragraph 14 becomes the definition of "電子決済手段サービス仲介業" (denshi kessai shudan sābisu chūkaigyō, "electronic payment instrument service intermediary business"). Commencement stages: 2026-07-23 and 08-12 (in force); 2027-03-31, 04-01, 07-22, 10-01, 2028-01-01, 06-13 (not yet in force). The commencement note of the 2027-07-22 version reads "公布の日から起算して一年を超えない範囲内において政令で定める日". The cabinet order is not verified. This is not "pending" but "promulgated, with the definition part not yet in force": Appendix A. Working gloss: Article 2(49): "In this Act, 'crypto-asset' means the following; provided that those representing the rights specified in Article 29-2(1)(viii) are excluded."; commencement note: "a date specified by Cabinet Order within a period not exceeding one year from the date of promulgation."

39. Financial Services and Markets Act 2000, s. 417(1), definition of "cryptoasset" (inserted by Financial Services and Markets Act 2023 (c. 29), s. 69(4)(a)). The original is as quoted in the text. Commencement: "I2 S. 69 in force at 29.8.2023 by S.I. 2023/779, reg. 4(uu)". legislation.gov.uk `ukpga/2000/8/section/417` and `ukpga/2023/29/section/69` (HTML, accessed 2026-10-25, checked verbatim). s. 69(4)(b) allows the Treasury

to amend the definition by regulations ("(5) The Treasury may by regulations amend the definition of 'cryptoasset' in subsection (1).").

40. The Financial Services and Markets Act 2000 (Cryptoassets) Regulations 2026, SI 2026/102, made 4th February 2026, reg. 1(2)–(3), and art. 88F and 88G, which these Regulations insert into the Regulated Activities Order (SI 2001/544) (pp. 36–37). Original: "(2) Subject to paragraph (3) these Regulations come into force on 25th October 2027 (the 'full commencement day'). (3) These Regulations come into force at the end of the period of 21 days beginning with the day after the day on which they are made, for the following purposes— (a) enabling the FCA—"; "88F.—(1) A qualifying cryptoasset. (2) A 'qualifying cryptoasset' means a cryptoasset which is— (a) fungible, (b) transferable, (c) not solely a record of value or contractual rights, including rights in another cryptoasset, and (d) not excluded by paragraph (4)."; "(4) A cryptoasset does not fall within paragraph (1) if it is— (a) a specified investment cryptoasset, ...; (b) electronic money; (c) currency of the United Kingdom or any other country or territory, including a central bank digital currency; (d) a cryptoasset that— (i) cannot be transferred or sold in exchange for money or other cryptoassets, except by way of redemption with the issuer, and (ii) can only be used by the holder— (aa) to acquire goods or services from the issuer, or (bb) to acquire goods or services within a limited network"; "88G.—(1) A qualifying stablecoin. (2) A 'qualifying stablecoin' means a qualifying cryptoasset where— (a) that cryptoasset seeks or purports to maintain a stable value in relation to a particular fiat currency ('the referenced fiat currency'), and (b) fiat currency ... or other assets are held for the purpose of maintaining a stable value". (legislation.gov.uk). For the FCA's final rules PS26/9–13 (published 2026-06-30), only their place in the list was checked, and this book does not cite them.

41. *Tulip Trading Ltd (a Seychelles company) v van der Laan and others* [2023] EWCA Civ 83 (CA-2022-001050 etc.), Court of Appeal (Civil Division), judgment handed down 3 February 2023, Lewison, Popplewell and Birss LJJ, ¶¶1–2, 5, 91–93. Original: "The question in this appeal is whether the developers who look after bitcoin may arguably owe fiduciary duties or duties in tort to an owner of that cryptocurrency."(¶1); "The conclusion was that Tulip had not established a serious issue to be tried because there was no realistic prospect of establishing that the facts pleaded amount to a breach of fiduciary or tortious duty owed by the defendants to Tulip."(¶5, summarizing the first instance); "I would allow this appeal. The conclusion is not that there is a fiduciary duty in law in the circumstances alleged by Tulip, only that the case advanced raises a serious issue to be tried. The time to decide on the duty in this case is once the facts are established. ... If the decentralised governance of bitcoin really is a myth, then in my judgment there is much to be said for the submission that bitcoin developers, while acting as developers, owe fiduciary duties to the true owners of that property."(¶91, Birss LJ; Popplewell LJ and Lewison LJ agreeing, ¶¶92–93). (judiciary.uk). The case name follows the cover of the appeal judgment, "v van der Laan" (the first defendant, Bitcoin Association for BSV, is not in the list of representation for the respondents on appeal). This judgment is a decision at the jurisdictional stage (whether there is an issue to be tried), not a decision on the merits. The Korean edition writes "High Court" in the original language because the court translation guideline lists three options for its Korean rendering without settling on one.

42. GENIUS Act §2(7), 139 Stat. 419–420. Original: "(7) DIGITAL ASSET SERVICE PROVIDER.—The term 'digital asset service provider'— (A) means a person that, for compensation or profit, engages in the business in the United States ... of— (i) exchanging digital assets for monetary value; (ii) exchanging digital assets for other digital assets; (iii) transferring digital assets to a third party; (iv) acting as a digital asset custodian; or (v) participating in financial services relating to digital asset issuance; and (B) does not include— (i) a distributed ledger protocol; (ii) developing, operating, or engaging in the business of developing distributed ledger protocols or self-custodial software interfaces; (iii) an immutable and self-custodial software interface; (iv) developing, operating, or engaging in the business of validating transactions or operating a distributed ledger; or (v) participating in a liquidity pool or other similar mechanism for the provisioning of liquidity for peer-to-peer transactions."

PART 5 · LAW

Chapter 28. The bridge between technology and law (2): an overview by issue, and where to read further

KEY TAKEAWAY

This chapter takes up the sixteen "lawyer's question" boxes in Parts 1 to 4 that point to it under seven issues: (a) mistaken transfers and criminal liability, (b) the nature of tokens, NFTs, and staking, (c) smart contracts as contracts and the time of performance, (d) the right to deletion of personal information and immutability, (e) market regulation, (f) evidence, and (g) DAOs, hard forks, and governance. Each issue gets one or two pages. The chapter does three things. It takes up the question each box asked, as it was asked. It points to the chapter in Parts 1 to 3 that holds the **technical facts** the question rests on. And it records **at what point** in the question the primary sources whose originals were checked (for judgments, the court, the date of decision, and the case number; for provisions, the statute number and the effective date) touch it. It does not write the answers. The statement of the Supreme Court of Korea (대법원) that punishing "원인불명으로 재산상 이익인 가상자산을 이체받은 자" (gloss: "a person who received by transfer, for an unidentified cause, virtual assets that are a pecuniary advantage"; working gloss, not an official translation; the original controls) for breach of trust violates the principle of legality, the SEC's statement that automation through "smart contracts," or computer code, "does not remove conduct from the purview of the U.S. federal securities laws," and the description by the Court of Appeal of England and Wales (영국 항소법원) of developers' fiduciary duties as "a serious issue to be tried" are all examples that show such a point, not conclusions. Section 28.2 lists only the primary sources and introductory works whose bibliographic details were settled in checking the source materials, and the places where something is missing are left empty.

Learning objectives

1. Have a map of where the sixteen question boxes land among the seven issues.
2. For each issue, tell apart "the point where technical facts enter," "the point the primary sources have already answered," and "the point that is still empty."
3. Be able to find judgments of Korean and foreign courts by the court, the date of decision, and the case number, and provisions by the statute number and the effective date.
4. Use the distinction that a "statistical verdict" (Chapter 25), "executed as coded" (Chapters 13 and 16), and "the decision of a majority" (Chapters 22, 23, and 26) can each be **evidence of** some legal requirement but are not that requirement **itself**.

What a reader of the [Basic] paragraphs alone takes away. In none of the seven issues does technology settle the answer. What technology settles is **where the question arises**, and knowing that place is all this chapter is about.

28.1 An overview by issue

(a) Mistaken transfers and criminal liability, and compulsory execution

The box. Chapter 11 asked: to whom do coins sent by mistake belong? In the UTXO model, all that has happened is that "an output no one's key opens" has come into being; in the account model, "the balance figure of some address" has gone up. Where does the phrase "transfer of possession" fit? If the recipient has moved them to an address of their own, what is that act? If the recipient has lost the key and cannot move them, what becomes of the duty to return them?

Where the technical facts enter. As Chapter 11 showed, Bitcoin has no "balance," only the sum of unspent outputs, while Ethereum does have balances. Either way, the ledger does not record the **cause** of a transfer. Whether it is the price in a sale or a mistake, the ledger does not know (Chapter 5: the ledger looks only at the validity of signatures). And there is no "undo." The sender cannot cancel the transaction (Chapter 9: finality); there is **only a new transaction** by which the recipient sends the coins back. What differs from a mistaken bank remittance is that there is no institution in the middle to carry out a refund. What is the same is that the recipient holds property that is "법률상 원인 없는" (beobryulsang wonin eomneun, "without legal cause").

Where the primary sources touch. The Supreme Court of Korea dealt with exactly this point in its judgment of December 16, 2021 (Supreme Court of Korea, 2020Do9789). The defendant had "알 수 없는 경위로 피해자의 … 거래소 가상지갑에 들어 있던 199.999비트코인 … 을 자신의 계정으로 이체받" (gloss: "received by transfer into his own account, by a course of events that cannot be known, 199.999 bitcoin … that had been in the victim's … virtual wallet at an exchange") and then moved most of it to another account of his own. The court below (원심), applying by analogy the case law on embezzlement (횡령) in a mistaken remittance (착오송금), found breach of trust (배임). The Supreme Court reversed and remanded: "가상자산 권리자의 착오나 가상자산 운영 시스템의 오류 등으로 법률상 원인관계 없이 다른 사람의

가상자산 전자지갑에 가상자산이 이체된 경우, 가상자산을 이체받은 자는 가상자산의 권리자 등에 대한 부당이득반환의무를 부담하게 될 수 있다. 그러나 이는 당사자 사이의 민사상 채무에 지나지 않고 이러한 사정만으로 가상자산을 이체받은 사람이 신임관계에 기초하여 가상자산을 보존하거나 관리하는 지위에 있다고 볼 수 없다." (gloss: "Where virtual assets have been transferred into another person's virtual-asset electronic wallet without a legal causal relationship, owing to a mistake by the holder of the rights in the virtual assets, an error in the virtual-asset operating system, or the like, the person who received the virtual assets may come to bear a duty to return unjust enrichment to the holder of the rights in the virtual assets or others. But this is no more than a civil obligation between the parties, and on this circumstance alone the person who received the virtual assets cannot be regarded as being in a position to preserve or manage the virtual assets on the basis of a relationship of trust.") And: "원인불명으로 재산상 이익인 가상자산을 이체받은 자가 가상자산을 사용·처분한 경우 이를 형사처벌하는 명문의 규정이 없는 현재의 상황에서 착오송금 시 횡령죄 성립을 긍정한 판례 … 를 유추하여 신의칙을 근거로 피고인을 배임죄로 처벌하는 것은 죄형법정주의에 반한다." (gloss: "In the present situation, in which there is no express provision imposing criminal punishment where a person who received by transfer, for an unidentified cause, virtual assets that are a pecuniary advantage uses or disposes of them, punishing the defendant for breach of trust on the basis of the principle of good faith (신의칙), by analogy to the case law … that affirmed the crime of embezzlement upon a mistaken remittance, violates the principle of legality (죄형법정주의).")[1] The judgment touches the question in Chapter 11 in three ways. First, the judgment called virtual assets "재산상 이익" (jaesansang iik, "pecuniary advantage") and did not decide whether they are "재물" (jaemul, "property in the sense of a thing"). The embezzlement count (the primary charge, 주위적 공소사실) was reversed together with the breach-of-trust part only on the ground that it stood in "동일체의 관계" (gloss: "a relationship of identity") with it, and whether virtual assets are 재물 remains an issue on which scholarly views divide. Second, "부당이득반환청구를 할 수 있는 주체가 피해자인지 아니면 거래소인지 명확하지 않다" (gloss: "it is not clear whether the party that can claim the return of unjust enrichment is the victim or the exchange"). The two layers of the exchange's books and the chain that Chapter 26 looked at appear here again. Third, the judgment wrote that virtual assets are "법정화폐와 동일하게 취급되고 있지 않고 그 거래에 위험이 수반되므로, 형법을 적용하면서 법정화폐와 동일하게 보호해야 하는 것은 아니다" (gloss: "not treated in the same way as legal tender and their trading carries risk, so in applying the Criminal Act they need not be protected in the same way as legal tender"). **What** the duty to return that this judgment called a "민사상 채무" (gloss: "civil obligation") is a duty to return, the same quantity of coins or their value, lies outside the scope of what the judgment decided, and the discussion of legal nature in 27.2 has to fill that place.

In Supreme Court of Korea, 2021Do9855, November 11, 2021, the same panel stated that "비트코인은 … 사기죄의 객체인 재산상 이익에 해당한다" (gloss: "Bitcoin … is a pecuniary advantage that can be the object of fraud"), and in Supreme Court of Korea, 2018Do3619, May 30, 2018, the Court, stating that under the Act on Regulation and Punishment of Criminal Proceeds Concealment (범죄수익은닉의 규제 및 처벌 등에 관한 법률) "재산적 가치가 인정되는 무형재산도 몰수할 수 있다" (gloss: "intangible property recognized as having property value may also be confiscated"), upheld the judgment below, which had identified, among the seized bitcoin, the quantity that constituted criminal proceeds and confiscated (몰수) it.[2]

Lay the three judgments over one another, and criminal law has treated bitcoin as property on the side of **the one who obtains it** (the object of fraud, the object of confiscation), while on the side of **the one who received it without cause** (is that person the subject of breach of trust?) it has denied punishment, citing the absence of a relationship of trust. Whether this asymmetry is right, this book does not say.

Compulsory execution. The Civil Execution Act (민사집행법) has no provision on virtual assets. A report of the Judicial Policy Research Institute (사법정책연구원) treats virtual assets as "그 밖의 재산권" (geu bakkui jaesangwon, "other property rights") and, for reasons including that if they were treated as corporeal movables (유체동산) the requirements for an attachment (압류) to take effect could not be met, because "블록체인에 채무자의 가상자산이 압류되었다는 표시를 할 기술은 현재 그 구현이 불가능" (gloss: "the technology to place on a blockchain an indication that the debtor's virtual assets have been attached cannot at present be implemented"), proposed legislation under which, where the debtor manages the virtual assets personally, the virtual assets themselves are attached, and where an exchange manages them, the "가상자산이전청구권" (gloss: "claim for the transfer of virtual assets") is attached, and the assets are transferred to a court enforcement officer (집행관) and sold.[3] For compulsory collection (강제징수) of taxes, legislation already exists. Article 55(3) of the National Tax Collection Act (국세징수법) provides that when the head of a tax office attaches virtual assets, the head "해당 가상자산의 이전을 문서로 요구할 수 있" (gloss: "may request in writing the transfer of the virtual assets concerned") of the delinquent taxpayer or of the business that keeps them in custody, and that the person so requested "이에 따라야 한다" (gloss: "shall comply with it") (inserted December 21, 2021); Article 66(2), subparagraph 2 allows "가상자산사업자를 통해 거래되는 가상자산" (gloss: "virtual assets traded through a virtual asset service provider") to be sold directly through the provider.[4] The two provisions presuppose one technical fact: **coins on a chain cannot be moved without the private key, so enforcement has no choice but to take the form of ordering the person who has the key to "move them"** (Chapter 5). The last question in the Chapter 11 box, "if the recipient lost the key and cannot move them," is exactly the place this request does not reach, and no provision governs that case.

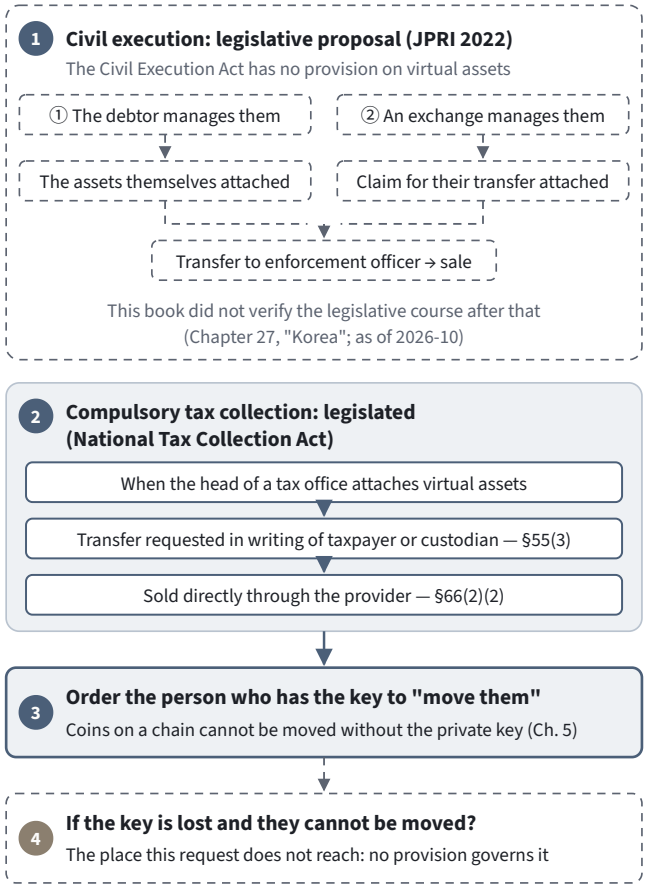


Figure 28-1. The three layers of compulsory execution against virtual assets. Civil execution is the legislative proposal in the Judicial Policy Research Institute report (dashed line), and compulsory tax collection has already been legislated in Article 55(3) and Article 66(2), subparagraph 2 of the National Tax Collection Act. The provisions in force take the form of ordering the person who has the key to move the assets, and no provision governs the case where the key has been lost

(b) The nature of tokens, NFTs, and staking, with taxation in brief

The boxes. Chapter 10 asked: is a staking reward consideration for a service, fruits (과실) borne by the deposited coins, or original acquisition (원시취득) of new coins? Is a deposit that disappears through slashing a fine, a contractual penalty, or the enforcement of a security interest? Is a business that takes in other people's coins and stakes them on their behalf a trustee or a debtor? Chapter 14 asked: did the person who bought an NFT buy the token, the file, a right, or the fact of "pointing to"? If the issuer shuts down its server, what has perished (멸실)? If the issuer promised revenue sharing, is that promise in the code or outside it? Chapter 24 asked: is a holder a depositor, an unsecured creditor, or the beneficiary of a trust? What right does a public pledge to "cover any shortfall using corporate resources" give?

Where the technical facts enter. The fact common to the three boxes is the one from Chapter 14: **a token standard is only a promise about function names, and what a token carries is fixed by the issuer's promise outside the chain.** The balance of an ERC-20 token, the (contract address, number) pair and `tokenURI` string of an ERC-721 token, the redemption promise of a stablecoin, and the issuance rules for staking rewards are all states on the chain, but the chain does not decide whether they are "rights." The fact from Chapter 10 is that staking rewards come from the protocol's issuance rules and that slashing is part of the same rules, and that the deposit is a design in which "punishment is part of security."

Where the primary sources touch. There are three strands.

First, **whether something is a security** turns on the promise outside the chain and on the shape of the transaction. In 2017 the SEC, finding that DAO Tokens were investment contracts, quoted the Supreme Court of the United States' precedent that "form should be disregarded for substance ... and the emphasis should be on economic realities underlying a transaction, and not on the name appended thereto," and said that although token holders had voting rights, "DAO Token holders were substantially reliant on the managerial efforts of Slock.it, its co-founders, and the Curators."[5] The 2023 summary judgment in *Ripple* distinguished the **institutional sales** of the same token, which were investment contracts, from **anonymous trading** on exchanges, which were not: "Programmatic Buyers could not have known if their payments of money went to Ripple, or any other seller of XRP."[6] The answer of U.S. law to the last question in the Chapter 14 box is: "The promise is outside, and to whom and by what route that promise was conveyed decides whether there is a security." In 2025 the U.S. Congress, through the GENIUS Act, took payment stablecoins of permitted issuers out of securities and commodities (§17).[7] Korea's Act on the Protection of Virtual Asset Users (가상자산 이용자 보호 등에 관한 법률; below, the Virtual Asset User Protection Act) excludes from virtual assets the electronically registered stocks, etc. under the Act on Electronic Registration of Stocks, Bonds, etc. (주식·사채 등의 전자등록에 관한 법률) (Article 2, subparagraph 1(d)), and whether something is a security under the Financial Investment Services and Capital Markets Act (자본시장과 금융투자업에 관한 법률) is outside the scope of this book.

Second, on **NFTs**, the definitions of three jurisdictions part ways. EU MiCA does not apply to "crypto-assets that are unique and not fungible with other crypto-assets" (Art. 2(3)). The UK's

Financial Services and Markets Act 2000 (Cryptoassets) Regulations 2026 make "fungible" a requirement of a "qualifying cryptoasset," taking non-fungible tokens out of the scope of regulated activities. In Korea, Article 2, subparagraph 4 of the Enforcement Decree (시행령), under the delegation in Article 2, subparagraph 1(h) of the Virtual Asset User Protection Act, excludes "단일하게 존재하여 다른 전자적 증표로 대체할 수 없는 전자적 증표" (gloss: "electronic tokens that exist singly and cannot be replaced by other electronic tokens"), with the proviso that "특정 재화나 서비스의 지급수단으로 사용될 수 있는 전자적 증표 등 금융위원회가 정하여 고시하는 전자적 증표는 제외" (gloss: "electronic tokens determined and publicly notified by the Financial Services Commission (금융위원회), such as electronic tokens that can be used as a means of payment for particular goods or services, are excluded").[8] On the Chapter 14 box's "what did the person buy?", what the definitional provisions say goes as far as "does regulation come in," and no provision decides whether the object of the sale is a token, a file, or a right. Position ⑨ in 27.2 (the control-based view) implies an answer that takes "the ability to change the owner field of a (contract address, number) pair" as the unit of attribution, and under that answer the "object of control" remains even if the server shuts down and the image disappears. That is one possible reply to the box's question "what has perished?", not this book's reply.

Third, **the position of stablecoin holders** is what Chapter 24 asked in the words of Gorton and Zhang: "equity contract or a debt contract." The legislation of four jurisdictions (Korea has not set up a separate category; see 27.3) has fixed the **qualifications** of issuers and their **reserve assets**. The EU allows e-money tokens to be issued only by credit institutions and electronic money institutions (MiCA Art. 48). The U.S. GENIUS Act wrote into its definition that a permitted issuer is "obligated to convert, redeem, or repurchase for a fixed amount of monetary value" (§2(22)(A)(ii)(I)). Japan defined the first type (item 1) of "電子決済手段" (denshi kessai shudan, "electronic payment instruments") as limited to "通貨建資産" (tsūka-date shisan, "currency-denominated assets"). And the UK made "fiat currency … or other assets are held for the purpose of maintaining a stable value" a requirement of a "qualifying stablecoin."[9] None of these provisions decides, on the spot, whether a holder is a depositor, a creditor, or a beneficiary. Priority in the issuer's insolvency, the segregation of reserve assets, and the effect of a "public pledge" pass to each jurisdiction's insolvency law and trust law, and that point is where the Chapter 24 box pointed.

Taxation, in brief. The Chapter 10 box's "what is a staking reward paid for?" is the question taxation is first required to answer, yet this book's source materials contain no primary source that records that answer. Two things can be written down. The U.S. Internal Revenue Service said in a 2014 notice that it treats virtual currency as "property" and applies the general principles of property transactions, and a 2023 notice left that conclusion as it was.[10] Article 21(1), subparagraph 27 of Korea's Income Tax Act (소득세법) makes "가상자산을 양도하거나 대여함으로써 발생하는 소득" (gloss: "income arising from transferring or lending virtual assets") other income (기타소득), but **as of 2026-10-25 it is not yet in force (effective January 1, 2027)**, and because its wording names only "양도·대여" (gloss: "transfer or lending"), where a staking reward falls cannot be read directly from the provision.[11] This book does not go into that interpretation.

(c) Smart contracts as contracts and the time of performance

The boxes. Chapter 13 asked: in a ten-line "contract," where are the offer (청약) and the acceptance (승낙)? Is deploying the code the offer and calling a function the acceptance, or are both factual acts (사실행위)? When the code behaves differently from what its author intended, is the "content of the contract" the code or the intention? If execution was reverted because the gas ran out but the fee remained paid, what was performed and what was not? If it becomes the "autonomous agent" the Yellow Paper spoke of, who is the one making the declaration of intention (의사표시)? Chapter 16 asked: the oracle put in a wrong value, and the code did not deviate by a single line. Is that performance (이행) or mistake (착오), whose mistake is it, and if no one ran the oracle, what did the person who pushed the value manipulate? Chapter 9 asked: when was the payment completed: when it was propagated, when it entered a block, or when *z* blocks were stacked on it? If the terms of service fix "6 confirmations," is that the standard? If the chain is reorganized and the transaction disappears, does the performance become something that never happened?

Where the technical facts enter. The three boxes rest on three facts. First, Chapter 13: a smart contract is "a box that releases value only when conditions are met," a state transition takes place when a transaction enters a block, and when gas runs out the state is reverted but the fee remains. Second, Chapter 16: the chain cannot know outside facts by itself, and outside facts come in only by trusting someone. Third, Chapter 9: there is no moment of completion, only a probability of reversal that keeps getting smaller, and "6 confirmations" is a convention, not a rule. When the law uses concepts that **have a time and an actor**, such as "declaration of intention," "performance," and "completion," technology gives, for those three, the time as a probability and the actor as a blank.

Where the primary sources touch. On whether smart contracts are contracts, the SEC's 2017 report is the only primary source whose original this book has checked. The report says "The automation of certain functions through this technology, 'smart contracts,' or computer code, does not remove conduct from the purview of the U.S. federal securities laws," and in a footnote introduces "smart contract" with Nick Szabo's definition, "a computerized transaction protocol that executes terms of a contract."[12] The point this sentence touches is that the fact that "the code executed" does not erase the **actor**. It does not say whether code is a contract. The DAO in Chapter 26 is the example: the code ran as written, and the SEC looked at the "efforts" of the people who deployed and promoted it. The Chapter 16 box's "what did the person who pushed the value manipulate?" may be reached by provisions of market regulation, and that is taken up in (e).

The empty point. On the Chapter 9 box's "when is a payment completed?", this book could not cite a primary source. Chapter 9 described the finality of payment systems only through the concept of "a point of irrevocability fixed by rule," without identifying which rule of which system, and said that Part 5 would support it with primary sources on the law of settlement finality. But this book's source materials do not contain the originals of that law (statutes and directives on settlement finality), so this edition could not support it. It is left reserved and reported as such. What can be written is the structure. If the finality of a payment system is **a point fixed by rule**, the finality of a blockchain is **a process in which a probability shrinks**, so terms of service that fix "6 confirmations" **lay** the point of a rule **over**

that process. Even if that overlay becomes the standard between the parties, the transaction itself can disappear if the chain is reorganized (Chapter 9, Section 4). That is the last question of the box, and which jurisdiction's primary sources sit at that point is a task for the next edition.

(d) The right to deletion of personal information and immutability

The box. Chapter 7 asked: someone says that a transaction containing their personal information has been recorded on the chain and asks for it to be deleted. The chain is designed so that it cannot be altered, and altering it would require a great many participants to remake every block that follows. What does the right to deletion of personal information (개인정보 삭제권) demand on a ledger that cannot be erased? Against whom is deletion to be demanded, and what act, technically, is "deleting"? Does a design that keeps personal information off the chain from the start and puts only a hash on it solve the problem, or postpone it?

Where the technical facts enter. What Chapter 7 showed is that immutability is not a mathematical impossibility but **a cost of tampering**. Change one character in one block and the hashes of every block after it no longer match, and removing the mismatch means re-mining everything after it (Chapters 7 and 8). So "deleting" is, technically, one of three things. (i) Remaking everything after that block. This needs the coordination of many, and this book's source materials contain no case in which past records were erased that way on a public mainnet. The DAO fork in Chapter 26 did not re-mine past blocks; it moved balances by putting an exceptional state change into the consensus rules at block 1,920,000 (EIP-779), so it is not of this type. (ii) Each node deleting the data from its own copy, in which case the data remain in other nodes' copies. (iii) Where the chain holds only a hash and the original is outside it, deleting the original outside. In (iii), the hash left on the chain is, as Chapter 4 showed, a fingerprint from which the original cannot be recovered, but if the original survives somewhere else, the hash comes back to life as proof that "that original existed then." That is why the box asked "solve, or postpone."

Where the primary sources touch. Article 36 of Korea's Personal Information Protection Act (개인정보 보호법) provides that a data subject "개인정보처리자에게 그 개인정보의 정정 또는 삭제를 요구할 수 있" (gloss: "may request the personal information controller to correct or delete the personal information") (paragraph (1); there is a proviso for personal information that other statutes specify as subject to collection), that the controller must take "지체 없이 그 개인정보를 조사하여 정보주체의 요구에 따라 정정·삭제 등 필요한 조치" (gloss: "without delay, after examining the personal information, necessary measures such as correction or deletion in accordance with the data subject's request") (paragraph (2); except where other statutes lay down a special procedure), and that "삭제할 때에는 복구 또는 재생되지 아니하도록 조치하여야 한다" (gloss: "when deleting, it shall take measures so that the information cannot be restored or regenerated") (paragraph (3)).[13] Article 17 of the EU General Data Protection Regulation (GDPR) provides, where one of the grounds listed in paragraph 1 applies (and none of the exceptions in paragraph 3 applies), for the data subject's right to obtain "the erasure of personal data concerning him or her without undue delay" and the controller's "obligation to erase personal data without undue delay," and provides that where the controller has made the information public and is obliged to erase it under paragraph 1, the

controller, "taking account of available technology and the cost of implementation, shall take reasonable steps, including technical measures, to inform controllers which are processing the personal data."[14] The two provisions touch the Chapter 7 box at three phrases. "개인정보처리자" (gaeinjeongbo cheorija, "personal information controller") and "controller": who is that on a public blockchain? (Chapter 8: a miner is nobody's agent.) "복구 또는 재생되지 아니하도록" (gloss: "so that it cannot be restored or regenerated"): is that possible in a structure where other nodes' copies and the hash remain? "available technology and the cost of implementation": is the cost of tampering exactly that "cost"? No court judgment from any jurisdiction is in this book's source materials, and this book does not judge.

(e) Market regulation: exchanges, market manipulation, MEV

The boxes. Chapter 8 asked: whose agent is a miner? Does a miner who leaves a valid transaction out of a block have any duty? If a group of miners refuses every transaction from a particular address, what act is that, and toward whom? Chapter 15 asked: where in a constant-product AMM is the "operator" that exchange regulation presupposes, someone who registers, reviews listings, monitors for market manipulation (시세조종), and keeps customer assets segregated? Chapter 17 asked: the sandwich attacker did not change a single character of the victim's order. Is this market manipulation? Is there a "market price" (시세) that was manipulated, who is the "actor" who manipulated it, and where is the "market" that regulation presupposes? If the producer ran the sandwich itself, what did it do in its own block? Chapter 25 asked: is an expert opinion that the price "was explosive at the 1% level in a GSADF test" evidence of market manipulation, evidence of deception (기망), or an observation that one of multiple equilibria was realized?

Where the technical facts enter. The four boxes rest on different facts. Chapter 8: a block producer is bound by no contract, and the majority of "one CPU, one vote" is not a body. Chapter 15: the price in an AMM is the ratio of the two quantities in the pool, and there is no quote, no operator, and no counterparty. Chapter 17: there is a power to decide the order, that power is not blocked by code, and after proposer–builder separation the order is sold at auction. Chapter 25: the verdict of an explosive unit-root test flips with the assumption about fundamental value, and "it was a bubble" is a statistical proposition, not a proposition about conduct.

Where the primary sources touch. For market regulation, the Korean and EU provisions and the U.S. judgment were checked in the original. Article 10 of Korea's Virtual Asset User Protection Act prohibits the use of material nonpublic information (paragraph (1)), matched trades and wash trades (paragraph (2)), "가상자산의 매매를 유인할 목적으로 가상자산의 매매가 성황을 이루고 있는 듯이 잘못 알게 하거나 그 시세를 변동 또는 고정시키는 매매 또는 그 위탁이나 수탁" (gloss: "trading, or the entrustment or acceptance of trading, that for the purpose of inducing the trading of virtual assets misleads others into believing that the trading of virtual assets is brisk, or that changes or fixes their market price") (paragraph (3)), and, among others, "부정한 수단, 계획 또는 기교를 사용하는 행위" (gloss: "the act of using a fraudulent means, scheme, or artifice") (paragraph (4)), as well as trading by a provider in virtual assets it issued itself (paragraph (5)), and it makes violators liable for damages (paragraph (6)). Paragraphs (2) to (4), which begin with "누구든지" (nugudeunji, "any

person"), do not limit their addressees to providers.[15] Title VI of EU MiCA applies, in respect of "crypto-assets that are admitted to trading or in respect of which a request for admission to trading has been made," to "any transaction, order or behaviour ..., irrespective of whether such transaction, order or behaviour takes place on a trading platform," and prohibits insider dealing (Art. 89) and market manipulation (Art. 91).[16] In the United States, the *McDonnell* decision recognized the CFTC's authority to apply the fraud prohibition of the Commodity Exchange Act to spot transactions in virtual currency.[17] Entry for exchanges is by reporting in Korea (Article 7 of the Act on Reporting and Using Specified Financial Transaction Information (특정 금융거래정보의 보고 및 이용 등에 관한 법률)), authorization in the EU (MiCA Art. 59), registration in Japan (資金決済法 第63条の2, the Payment Services Act), and, in the UK, the regulated activities under SI 2026/102, scheduled to come into force in 2027 (27.3).

Where the provisions reach the boxes and where they do not. The "시세를 변동 또는 고정시키는 매매" (gloss: "trading that changes or fixes the market price") of Korea's Article 10(3) and the "market manipulation" of MiCA Article 91 give a **place in the wording** into which the sandwich of Chapter 17 could go. They say "누구든지" and "any person," and nothing in the word "시세" (sise, "market price") limits it to a particular exchange. (How far that reaches is a matter of interpretation, and this book does not go into it.) But the two provisions stop at different places. MiCA presupposes crypto-assets that are "admitted to trading" or for which admission has been requested, and Korea's Article 10(3) requires "매매를 유인할 목적" (gloss: "the purpose of inducing trading"). The "operator" of the Chapter 15 box and the "agent" of the Chapter 8 box are filled by neither provision. The provisions only prohibit conduct. They do not say which of an AMM's deployer, its liquidity providers, and its governance majority is the "operator," or whether a block producer's refusal of transactions is an "act." That the GENIUS Act expressly excluded participation in liquidity pools from "digital asset service provider" (27.3) is one example of legislation knowingly leaving that gap empty. To the Chapter 25 box, an answer can be given from structure rather than from provisions. No provision has "an explosive interval" as a requirement. Korea's Article 10 makes "목적" (gloss: "purpose") and "매매" (gloss: "trading") its requirements, MiCA Article 91 "activities," and the Commodity Exchange Act "fraud." A statistical verdict can be **evidence of** such a requirement, but it is not the requirement itself, and as Chapter 25 showed, that verdict flips with the assumption about fundamental value and the choice of critical values. A verdict of "it was not a bubble" does not, for the same reason, directly help the defendant either.

(f) Evidence: hashes, time-stamps, and originals

The boxes. Chapter 4 asked: only the hash value of a document was recorded on a public ledger, and years later it is submitted together with the original file. What does this hash value prove, and what does it not prove? "That bit string existed at that time," "that document was made at that time," and "the content of that document is true" are three different propositions. Which of the three do the evidence-law concepts of the original, the copy, and authenticity (진정성립) reach? Chapter 1 asked: if there are many ledgers, which copy is the original?

Where the technical facts enter. As Chapter 4 showed, a hash is a fingerprint that cannot be

reversed. The same input gives the same output, and if one character differs, the whole changes. So if the hash is on the chain and the original is submitted, recomputing the hash and matching it proves the proposition "this bit string existed before that block's time." **But the meaning of the block's time has to be read precisely.** As Chapter 7 set out from the original text of Haber and Stornetta, what a time-stamp chain guarantees is a **boundary** in two directions, "it cannot be stamped with a later time / it cannot be back-dated to an earlier time." The time in a Bitcoin header is "a rough time" that the miner writes, and the protocol only restricts it so that it does not stray far from the blocks before and after it (Chapter 8). And a hash says nothing about "making" or "truth." A false document also has a fingerprint. The decoding of the newspaper headline in the genesis block in Chapter 12 is a good example. That the block contains that phrase says only that the block was made **after** the phrase was published, and the block does not prove that the phrase was a real newspaper headline (which is why Chapter 12 wrote conditionally). For Chapter 1's "which copy is the original?", Chapters 6 and 7 give the place of the answer: since all the copies have the same root hash, none of them is "the original," and each of them is the same as the original.

Where the primary sources touch. Article 357 of Korea's Civil Procedure Act (민사소송법) provides "사문서는 그것이 진정한 것임을 증명하여야 한다" (gloss: "a private document shall be proved to be authentic"), Article 358 provides "사문서는 본인 또는 대리인의 서명이나 날인 또는 무인(拇印)이 있는 때에는 진정한 것으로 추정한다" (gloss: "a private document is presumed authentic when it bears the signature, seal, or thumbprint of the person or of an agent"), and Article 3 of the Digital Signature Act (전자서명법) provides that an electronic signature "전자적 형태라는 이유만으로 서명, 서명날인 또는 기명날인으로서의 효력이 부인되지 아니한다" (gloss: "shall not be denied effect as a signature, a signature and seal, or a name and seal solely because it is in electronic form").[18] The point at which the three provisions touch the Chapter 4 box is that "authenticity" is a question about the **author**. A signature or seal raises a presumption about "who wrote it," and a hash proves "what existed." They are different propositions. The digital signature that Chapter 5 showed proves that the holder of the private key approved that bit string, and whose private key it is, is a fact outside the chain. A 2020 judgment of the Tokyo High Court (東京高等裁判所) wrote this about the evidentiary function of electromagnetic records (電磁的記**録**, denjiteki kiroku):
"電磁的記**録**の証明機能は，それが作出されるべきシステムの設置運**営**主体の意思に基づき，正当に作出されることが当然の前提とされているもので" (gloss: "the evidentiary function of an electromagnetic record is premised as a matter of course on its being properly created in accordance with the will of the entity that set up and operates the system in which it is to be created"), and an electromagnetic record
"それを使用することが予定されている一定のシステムやプログラムのもとで用いられて初めて本来の証明機能を果たし得る" (gloss: "can perform its proper evidentiary function only when it is used under the particular system or program in which its use is intended").[19] It is an interpretation of a criminal provision, but it is a judgment that read what the figures in an exchange's books "prove" **from what that system was supposed to do**, and the answer to the Chapter 4 box's "what does a hash prove?" starts the same way, from what that chain was supposed to do (Chapter 7: a time boundary on existence).

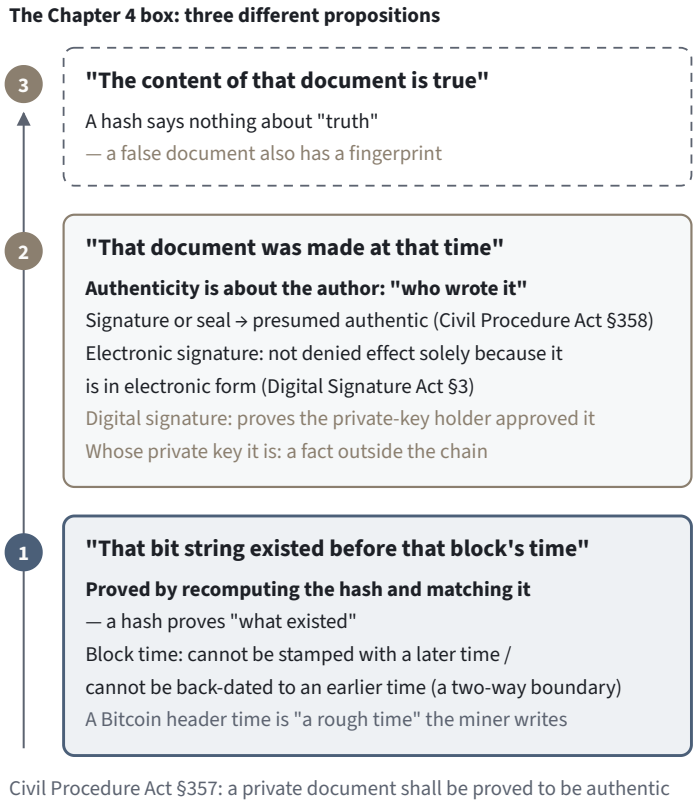


Figure 28-2. The three propositions in the Chapter 4 box and the means that reach each. A hash proves that the bit string existed before the block's time, a signature or seal raises a presumption about who wrote it, and about the truth of the content the hash says nothing

(g) The legal status of DAOs, hard forks, and governance

The boxes. There are four. Chapter 22: a majority of the governance tokens changed the rules. What can the minority assert? Is a DAO a partnership (조합) (do its members bear unlimited liability), a juridical person (법인) (under which law was it established), or nothing at all (to whom are the Curators' duties owed)? Chapter 23: after a hard fork both chains survived. Who decides which ledger is "real": a majority of the hash rate, a majority of the nodes, the exchanges, the developers, or no one? Did a person who had coins before the fork have one thing or two? A contract to "pay 1 bitcoin": 1 on which chain? What did the selfish miner violate? Chapter 26: was The DAO's hard fork a remedy, a forfeiture, or a change of contract by majority vote? Does the distinction that the DAO Fork "did not change the protocol" and was an "irregular state change" have legal meaning? And Chapter 29 in Part 6 asks: is blocking the spending of vulnerable coins by soft fork expropriation (수용), a measure for maintaining order, or the disposal of ownerless property (무주물), and whose agent is the party that decides?

Where the technical facts enter. As Chapter 23 showed, rules are maintained by the coordination of participants, and deciding which chain "wins" is also coordination. As Chapter 22 showed, votes with governance tokens are counted by code and the results are executed automatically. As Chapter 26 showed, a hard fork can be "changing the state without changing the rules," and the minority can stay on the old chain. As Chapter 8 showed, the sets of miners, nodes, and holders do not coincide. When the law uses concepts such as "body" (단체), "member" (구성원), "representative" (대표), and "resolution" (결의),

technology gives several sets that do not overlap and votes counted by code.

Where the primary sources touch. In three places. First, the SEC's 2017 report calls The DAO "an unincorporated organization" and, citing the provision under which "person" in the definition of "issuer" in the securities laws includes "any unincorporated organization," said that The DAO was an issuer of securities.[20] The report also reproduced in a footnote that The DAO white paper itself had written that "the legal status of [DAO Entities] remains the subject of active and vigorous debate and discussion."[21] The answer of U.S. securities law to the Chapter 22 box's "or nothing at all?" is "something sufficient to be an issuer." Second, in its 2023 judgment in *Tulip Trading*, the Court of Appeal of England and Wales said that whether the developers of the Bitcoin network owe fiduciary duties (신인의무) to holders was "a serious issue to be tried," and wrote: "If the decentralised governance of bitcoin really is a myth, then ... there is much to be said for the submission that bitcoin developers, while acting as developers, owe fiduciary duties to the true owners of that property."[22] On the Chapter 23 box's "the developers?", the English court went as far as the point "that may be so, once the facts are established." It is not a conclusion on the merits. Third, EIP-779 documented The DAO fork in these words: "did not change the protocol ... Rather, the DAO Fork was an 'irregular state change'," and the Foundation directed users who wanted to follow the old chain to the "--oppose-dao-fork" flag.[23] On the Chapter 26 box's "does that distinction have legal meaning?", what the technical document gives is the distinction itself, and the meaning is for the law. In Korean law, Article 703(1) of the Civil Act (민법) provides "조합은 2인 이상이 상호출자하여 공동사업을 경영할 것을 약정함으로써 그 효력이 생긴다" (gloss: "a partnership takes effect when two or more persons agree to make mutual contributions and carry on a joint undertaking"), the partnership-right view (조합권설) in 27.2 ⑤ tried to read the network into that provision, and there is a counterargument that "단체의 성립·가입·탈퇴를 위한 법령이나 의사표시가 없다" (gloss: "there is no statute or declaration of intention for the formation of a body, or for joining or leaving it").[24] There are no primary sources touching the Chapter 29 question in this book's source materials. Section 5 of Chapter 29 only introduces three strands of the discussion on the Bitcoin side (leave them as they are, burn them, limit their speed) and the policy options listed in one white paper, and this section records the fact that there is no statute or judgment touching that question.

The empty points. On the Chapter 22 box's "is a DAO a juridical person, and under which law?", this book did not check through primary sources whether any jurisdiction has legislation granting legal personality to DAOs. On the Chapter 23 box's "the 1 bitcoin in a contract: 1 on which chain?", it did not look for judgments or provisions either. Both places are left as tasks for the next edition and reported as reserved.

Table matching boxes to issues

This table brings together the twenty-three boxes taken up by this chapter and Chapter 27. The boxes of Parts 0 and 6 (Chapters 1, 29, and 30) are also listed by the wording of each chapter's box.

CHAPTER	GIST OF THE QUESTION	SECTION THAT TAKES IT UP
1	Which of the many copies of the ledger is the "original"?	28.1(f)

CHAPTER	GIST OF THE QUESTION	SECTION THAT TAKES IT UP
4	What does recording a hash value prove?	28.1(f)
5	What has a person who lost the private key lost?	27.2
7	What does the right to deletion demand on a ledger that cannot be erased?	28.1(d)
8	Whose agent is a miner?	28.1(e)
9	When is a payment completed?	28.1(c), primary sources reserved
10	What is a staking reward paid for?	28.1(b), no primary source on taxation
11	To whom do coins sent by mistake belong?	28.1(a)
13	Is a smart contract a contract?	28.1(c)
14	What has the buyer of an NFT bought?	28.1(b), 27.3
15	Is there an "operator" in an AMM?	27.3, 28.1(e)
16	Is a contract executed on an oracle error a case of mistake?	28.1(c), (e)
17	Is a sandwich market manipulation?	28.1(e)
19	Are legal tender and "money" the same thing?	27.2
20	Is a change in the issuance rule an infringement of rights?	27.2, Chapter 29
21	To whom does a burned fee belong?	27.2
22	What can the minority assert against a governance majority?	28.1(g)
23	After a hard fork, who decides which ledger is "real"?	27.2, 28.1(g)
24	Is a stablecoin holder a depositor or a creditor?	27.3, 28.1(b)
25	Is a bubble verdict evidence of fraud?	28.1(e)
26	Was The DAO fork a remedy or a forfeiture?	28.1(g)
29	Is restricting the spending of vulnerable coins expropriation?	27.2, 28.1(g), no primary sources
30	When a definitional provision cannot keep up with the technology	27.2

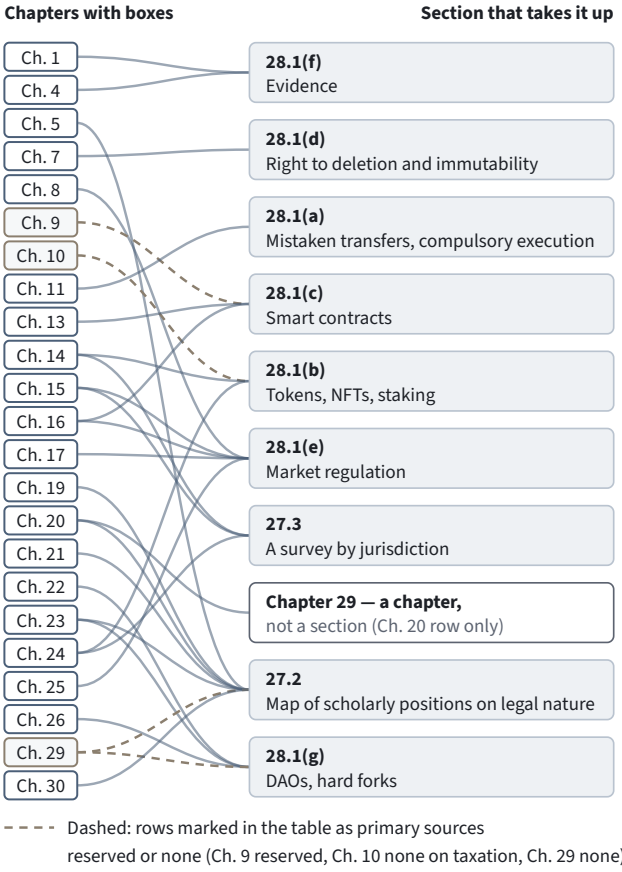


Figure 28-3. The table above drawn as lines. On the left are the 23 chapters that have question boxes, and on the right are the sections that take those boxes up. Dashed lines are the rows marked in the table as primary sources reserved or none (Chapters 9, 10, and 29), and the boxes on the right are placed in an order other than section-number order so that the lines cross less

28.2 Where to read further

Below are the primary sources and introductory works whose bibliographic details were settled in checking the source materials and whose originals this book opened. They are where a reader who wants to do what Part 5 did not do, legal exposition, should go first. Bibliographic details that were not settled are not given, and jurisdictions for which something is missing are left empty.

International and comparative - UNIDROIT, *Principles on Digital Assets and Private Law* (Rome, 2023). 19 principles with commentary. A jurisdiction-neutral design that sets up "control" as the criterion of attribution. - FATF, *Updated Guidance for a Risk-Based Approach to Virtual Assets and Virtual Asset Service Providers* (Paris, 28 October 2021). The international definitions of "virtual asset" and "VASP" and the anti-money-laundering framework. Its publication page warns that it does not reflect the 2025 amendment to Recommendation 1 and other changes. - Gary B. Gorton & Jeffery Y. Zhang, "Taming Wildcat Stablecoins," *University of Chicago Law Review* 90(3): 909–972 (2023). A law article that places stablecoins in the context of the history of banking and of money.

Korea - Lee Hye-jeong (이혜정), *가상자산에 대한 민사집행 연구* [A study of civil execution

against virtual assets] (Judicial Policy Research Institute research series 2022-12, 2022). An overview of the domestic and foreign discussion of legal nature (Chapter 2, Section 3) and a legislative proposal for civil execution (Chapter 3). - Supreme Court of Korea, 2018Do3619, May 30, 2018; Supreme Court of Korea, 2021Do9855, November 11, 2021 (공2022상, 56 [Court Gazette 2022, first half, p. 56]); Supreme Court of Korea, 2020Do9789, December 16, 2021: full texts of the judgments on the Korean Law Information Center. - Act on the Protection of Virtual Asset Users (enacted as Act No. 19563; current version Act No. 21857); Act on Reporting and Using Specified Financial Transaction Information, Article 2, subparagraph 3 and Article 7; Income Tax Act, Article 21(1), subparagraph 27 (effective January 1, 2027); National Tax Collection Act, Articles 55 and 66; Local Tax Collection Act, Article 61: Korean Law Information Center.

EU - Regulation (EU) 2023/1114 (MiCA), OJ L 150, 9.6.2023, p. 40. - CJEU, *Skatteverket v David Hedqvist*, C-264/14, ECLI:EU:C:2015:718 (22 October 2015). - Regulation (EU) 2016/679 (GDPR), OJ L 119, 4.5.2016, p. 1, Art. 17.

United States - *SEC v. W. J. Howey Co.*, 328 U.S. 293 (1946). - SEC, *Report of Investigation Pursuant to Section 21(a) of the Securities Exchange Act of 1934: The DAO*, Release No. 81207 (July 25, 2017). - *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. 2018). - *SEC v. Ripple Labs, Inc.*, 682 F. Supp. 3d 308 (S.D.N.Y. 2023). - IRS Notice 2014-21, 2014-16 I.R.B. 938; Notice 2023-34, 2023-19 I.R.B. 837. - GENIUS Act, Pub. L. No. 119-27, 139 Stat. 419 (2025). - *Uniform Commercial Code Amendments (2022)*, Article 12 (ULC and ALI, edition with comments). - One U.S. law school syllabus: Pepe, *Reading Group: Blockchain Law and Policy* (GW Law, Fall 2022), an example of the range covered when securities law, commodities law, tax law, FinCEN, and FATF are arranged across one semester.

Japan - 資金決済に関する法律(平成21年法律第59号) 第2条·第63条の2 [Payment Services Act (Act No. 59 of 2009), Articles 2 and 63-2]; 金融商品取引法(昭和23年法律第25号) 第2条第3項 [Financial Instruments and Exchange Act (Act No. 25 of 1948), Article 2(3)]; 令和8年法律第64号 [Act No. 64 of 2026]: e-Gov 法令検索. - 東京地方裁判所 平成31年3月15日 判決 (平成27年(刑わ)第2274号, etc.) [Tokyo District Court, judgment of March 15, 2019]; 東京高等裁判所 令和2年6月11日 判決 (令和元年(う)第948号) [Tokyo High Court, judgment of June 11, 2020]: courts.go.jp (appeal), case-law database (first instance).

United Kingdom - Law Commission, *Digital Assets: Final Report* (Law Com No 412, HC 1486, 2023). - Property (Digital Assets etc) Act 2025 (c. 29). - The Financial Services and Markets Act 2000 (Cryptoassets) Regulations 2026, SI 2026/102. - *Tulip Trading Ltd v van der Laan and others* [2023] EWCA Civ 83.

Korean monographs whose bibliographic details only were checked against National Assembly Library collection records. This book did not open the text of the books below, and their bibliographic details are given according to the title, author, publisher, and year in the national academic information records of the National Assembly Library (국회도서관) (retrieved 2026-10-25). Their contents are not summarized or evaluated. - An Hyeon-su (안현수), *가상자산법* [Virtual asset law] (박영사, 2024). National Assembly Library control number MONO12024000082483. - Kim Hong-gi (김홍기), *자본시장법 = 금융사지배구조법 | 금융소비자보호법 | 가상자산법* [Capital markets law = Financial company governance law | Financial consumer protection law | Virtual asset law] (박영사, 2024): a textbook whose title

names the virtual asset law alongside the others. Control number MONO12024000019710. - Lee Jeong-yeop (이정엽), Lee Seok-jun (이석준), and Kim Seong-in (김성인), *가상자산 판례백선. 민사·신청편* [One hundred leading cases on virtual assets: civil and applications] (박영사, 2023). Control number MONO12023000016475. - Lee Jeong-yeop (이정엽), Lee Seok-jun (이석준), Kim Seong-in (김성인), Jang Min-seok (장민석), and Han Ung-hui (한웅희), *가상자산 판례백선. 형사·행정편* [One hundred leading cases on virtual assets: criminal and administrative] (박영사, 2024). Control number MONO12024000019412.

Places left empty. Introductory works (textbooks and commentaries) for Japan, the UK, and the EU are not listed, because their bibliographic details could not be settled through OpenAlex or National Assembly Library records (one edited volume from the UK was identified only by title and editors through a book-review record, so its bibliographic details are incomplete). There are also no primary sources for the law of settlement finality (28.1(c)), the taxation of staking (28.1(b)), or legislation on the legal personality of DAOs (28.1(g)). These are places to fill in the next edition.

Summary

Through the seven issues the same shape kept recurring. The questions the boxes asked arose from technical facts (the ledger does not know the cause, a token does not carry rights, there is no moment of completion, erasing has a cost, there is a power to decide the order, a hash proves only existence, votes are counted by code), and the primary sources touched **one point** in each question. The Supreme Court of Korea did not regard the recipient of a mistaken transfer as the subject of breach of trust, the SEC said that automatic execution by code does not erase the actor, the Court of Appeal of England and Wales left developers' fiduciary duties as an issue to be tried, and the legislation of five jurisdictions defined providers, and four of them stablecoin issuers. No source answered every question in a box, and the empty points (settlement finality, the taxation of staking, the legal personality of DAOs, the object of a contract after a fork) were recorded as empty. What a lawyer takes from this chapter is not answers but this table, which records which chapter of Parts 1 to 3 holds the **premise facts** of which issue. The next chapter looks, through the original text of definitional provisions, at one thing that runs through all of those issues: that five jurisdictions call the same thing by five names, and that those names divide jurisdiction, taxation, and the application of criminal law (Chapter 30).

Sources for this chapter

- Supreme Court of Korea, 2020Do9789, December 16, 2021; Supreme Court of Korea, 2021Do9855, November 11, 2021; Supreme Court of Korea, 2018Do3619, May 30, 2018: full texts of the judgments on the Korean Law Information Center (Open API, retrieved 2026-10-25). The text was read; the headnotes and summaries of holdings are not quoted. The 2010Do891 judgment only as cited in the 2020Do9789 judgment.
- Act on the Protection of Virtual Asset Users, Articles 2, 10, and 14; Act on Reporting and Using Specified Financial Transaction Information, Article 7; Income Tax Act, Article 21(1), subparagraph 27 (not in force); National Tax Collection Act, Article 55(3) and Article 66(2); Local Tax Collection Act, Article 61(3); Personal Information Protection Act, Article 36; Civil Procedure Act, Articles 357 and 358; Digital Signature Act, Article 3; Civil Act, Article 703: Korean Law Information Center (Open API, retrieved 2026-10-25).
- Lee Hye-jeong (이혜정), *가상자산에 대한 민사집행 연구* (Judicial Policy Research Institute, 2022): pp. xviii–xix, 85, 93, 210–212.
- SEC, *Report of Investigation … The DAO*, Release No. 81207 (2017-07-25): pp. 1–4, 11, 13, 15–16.
- *SEC v. Ripple Labs, Inc.*, ECF No. 874 (S.D.N.Y. July 13, 2023): pp. 15, 23, 34.
- *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. Mar. 6, 2018): pp. 3, 24.
- GENIUS Act, Pub. L. No. 119-27 (2025): §2(7), (22), §17, §20.
- IRS Notice 2014-21; Notice 2023-34. `irs-2023-notice-2023-34-modification-2014-21.pdf`.
- Regulation (EU) 2023/1114 (MiCA): Art. 2(3), 48, 59, 86, 89, 91.
- Regulation (EU) 2016/679 (GDPR), OJ L 119, 4.5.2016, p. 1: Art. 17(1)–(2). Cellar XHTML (accessed and checked word for word on 2026-10-25).
- 資金決済に関する法律 第2条第5項, 第63条の2 [Payment Services Act, Article 2(5) and Article 63-2]: e-Gov 法令 API v2 (retrieved 2026-10-25).
- SI 2026/102: RAO art. 88F, 88G.
- *Tulip Trading Ltd v van der Laan and others* [2023] EWCA Civ 83 (3 February 2023): ¶¶1, 91.
- 東京高等裁判所 令和2年6月11日 判決 (令和元年(う)第948号) [Tokyo High Court, judgment of June 11, 2020]: pp. 10–11.
- EIP-779; Ethereum Foundation Blog, 2016-07-20: the same places as in Chapter 26.
- The bibliography in 28.2 consists of the items whose bibliographic details were settled in checking the source materials and of four Korean monographs whose bibliographic details only were checked against records of the National Assembly Library national academic information API (retrieved 2026-10-25, control numbers given). The text of the monographs was not opened. Introductory works for Japan, the UK, and the EU, the law of settlement finality, the taxation of staking, and legislation on the legal personality of DAOs were left empty because there were no settled bibliographic details.
- This chapter has no code or notebook (by design). There are no price or market-capitalization data. Pending bills are not recorded.

Notes

1. Supreme Court of Korea, 2020Do9789, December 16, 2021 (charge: violation of the Act on the Aggravated Punishment, etc. of Specific Economic Crimes (특정경제범죄 가중처벌 등에 관한 법률) (embezzlement) [offense found: violation of the same Act (breach of trust)]; judgment below: Suwon High Court (수원고등법원), 2020No171, July 2, 2020, reversed and remanded; Justices Noh Jeong-hee (노정희) (presiding), Kim Jae-hyung (김재형) (authoring Justice), Ahn Chul-sang (안철상), and Lee Heung-gu (이흥구)). Reasons 1 (summary of the secondary charge (예비적 공소사실)), 3.A, 3.C, and 3.D. The original is as quoted in the text. Cases cited in the judgment: Supreme Court of Korea, 2010Do891, December 9, 2010 (embezzlement upon a mistaken remittance); Supreme Court of Korea, 2019Do9756, February 20, 2020 (en banc) ("타인의 사무를 처리하는 자," "a person who administers another's business"); Supreme Court of Korea, 2021Do9855, November 11, 2021. Full text of the judgment on the Korean Law Information Center (국가법령정보센터) (Open API, precedent serial no. 219775, retrieved 2026-10-25). Because this book has not checked the text of the 2010Do891 judgment, that judgment is described only as "a judgment cited by the 2020Do9789 judgment." What happened after the remand has not been checked. (대법원 2021. 12. 16. 선고 2020도9789 판결, 특정경제범죄가중처벌등에관한법률위반(횡령)[인정된죄명: 특정경제범죄가중처벌등에관한법률위반(배임)]; 원심 수원고등법원 2020. 7. 2. 선고 2020노171 판결; 대법원 2010. 12. 9. 선고 2010도891 판결; 대법원 2020. 2. 20. 선고 2019도9756 전원합의체 판결)

2. Supreme Court of Korea, 2021Do9855, November 11, 2021, Reasons 2.A; Supreme Court of Korea, 2018Do3619, May 30, 2018, Reasons 1.A.3), 5), and 6), and B.2). The original (2018Do3619, B.2)): "피고인으로부터 임의제출받아 압수한 216.1249474비트코인(BTC) 중 191.32333418비트코인(BTC)은 범죄수익은닉규제법에 정한 중대범죄인 이 사건 음란사이트의 운영과 관련한 범죄행위에 의하여 취득한 것이므로, 범죄수익에 해당하여 이를 몰수한다." Working gloss: "Of the 216.1249474 bitcoin (BTC) voluntarily submitted by the defendant and seized, 191.32333418 bitcoin (BTC) were acquired through criminal conduct relating to the operation of the obscene website in this case, which is a serious crime as provided in the Criminal Proceeds Concealment Act; they therefore constitute criminal proceeds and are confiscated." The same place as footnotes 22 and 23 in Chapter 27. (대법원 2021. 11. 11. 선고 2021도9855 판결; 대법원 2018. 5. 30. 선고 2018도3619 판결)

3. Lee Hye-jeong (이혜정), *가상자산에 대한 민사집행 연구* [A study of civil execution against virtual assets] (Judicial Policy Research Institute, 2022), pp. xviii–xix, 210–212. The original: "블록체인에 채무자의 가상자산이 압류되었다는 표시를 할 기술은 현재 그 구현이 불가능하고, 가상자산지갑은 가상자산 관련 정보를 관리하는 것으로서 ... 가상자산지갑 자체에 압류표시를 하는 것이 압류 대상인 가상자산이 무엇인지를 명확히 한다고 보기 어려우므로" (p. 211); "'가상자산을 유체동산'으로 보면 가상자산에 대한 압류 자체가 불가능 또는 무효인 결과로 연결될 수 있는 점" (p. 212). Working gloss: "the technology to place on a blockchain an indication that the debtor's virtual assets have been attached cannot at present be implemented, and since a virtual-asset wallet manages information relating to virtual assets ... it is hard to say that placing an indication of attachment on the virtual-asset wallet itself makes clear what the virtual assets subject to attachment are" (p. 211); "the point that treating 'virtual assets as corporeal movables' may lead to the result that attachment of virtual assets is itself impossible or void" (p. 212). For the gist of the legislative proposal, see footnote 31 in Chapter 27. Whether the Civil Execution Act has since been amended is not verified (Appendix A).

4. National Tax Collection Act (current version, effective October 2, 2026), Article 55(3) and Article 66(2), subparagraph 2; Local Tax Collection Act (지방세징수법) (current version), Article 61(3). For the originals, see footnote 30 in Chapter 27.

5. SEC, Release No. 81207 (2017-07-25), pp. 11, 13. The original: "In analyzing whether something is a security, 'form should be disregarded for substance,' Tcherepnin v. Knight, 389 U.S. 332, 336 (1967), 'and the emphasis should be on economic realities underlying a transaction, and not on the name appended thereto.' United Housing Found., 421 U.S. at 849." (p. 11); "Although DAO Token holders were afforded voting rights, these voting rights were limited. DAO Token holders were substantially reliant on the managerial efforts of Slock.it, its co-founders, and the Curators." (p. 13). The two Supreme Court precedents in the quotation are cited by the SEC report; this book has not checked their originals.

6. *SEC v. Ripple Labs, Inc.*, No. 1:20-cv-10832 (AT) (S.D.N.Y. July 13, 2023), ECF No. 874, 682 F. Supp. 3d 308, pp. 15, 23, 34. For the original, see footnote 19 in Chapter 27.

7. GENIUS Act, Pub. L. No. 119-27 (July 18, 2025), §2(22) and §17; see footnote 20 in Chapter 27. §17(a) (a sentence added to §202(a)(18) of the Investment Advisers Act): "The term 'security' does not include a payment

stablecoin issued by a permitted payment stablecoin issuer, as such terms are defined in section 2 of the GENIUS Act." The effective date (§20) is not yet fixed.

8. Regulation (EU) 2023/1114 Art. 2(3): "This Regulation does not apply to crypto-assets that are unique and not fungible with other crypto-assets."; RAO art. 88F(2)(a) "fungible," as inserted by SI 2026/102; Article 2, subparagraph 1(h) of the Act on the Protection of Virtual Asset Users and Article 2, subparagraph 4 of the Enforcement Decree of the same Act (Presidential Decree, promulgated September 29, 2026, effective October 2, 2026). The original: "4. 수집을 주된 목적으로 하는 전자적 증표, 거래 당사자 간의 거래 확인만을 목적으로 하는 전자적 증표 등 단일하게 존재하여 다른 전자적 증표로 대체할 수 없는 전자적 증표. 다만, 특정 재화나 서비스의 지급수단으로 사용될 수 있는 전자적 증표 등 금융위원회가 정하여 고시하는 전자적 증표는 제외한다." (Korean Law Information Center Open API (MST 290171, Article 2), retrieved 2026-10-25). Working gloss: "4. Electronic tokens that exist singly and cannot be replaced by other electronic tokens, such as electronic tokens whose principal purpose is collection and electronic tokens whose sole purpose is the confirmation of a transaction between its parties; provided that electronic tokens determined and publicly notified by the Financial Services Commission, such as electronic tokens that can be used as a means of payment for particular goods or services, are excluded." The public notice has not been checked. The original is in Chapter 30. Footnotes 33 and 40 in Chapter 27.

9. MiCA Art. 48(1)(a); GENIUS Act §2(22)(A)(ii)(I); 資金決済法 第2条第5項 (Payment Services Act (資金決済に関する法律), Article 2(5)) "電子機器その他の物に電子的方法により記録されている通貨建資産に限り" (Working gloss: "limited to currency-denominated assets recorded by electronic means on electronic devices or other things"); RAO art. 88G(2)(b). For the originals, see footnotes 34, 20, 36, and 40 in Chapter 27.

10. IRS Notice 2014-21, §4, Q&A-1; Notice 2023-34, §3; see footnote 14 in Chapter 27.

11. Income Tax Act, Article 21(1), subparagraph 27 (inserted by Act No. 17757; effective date deferred to January 1, 2027); see footnote 29 in Chapter 27. Not in force "as of 2026-10."

12. SEC, Release No. 81207 (2017-07-25), p. 2, text and n. 3. Footnote 3: "Computer scientist Nick Szabo described a 'smart contract' as: a computerized transaction protocol that executes terms of a contract. The general objectives of smart contract design are to satisfy common contractual conditions (such as payment terms, liens, confidentiality, and even enforcement), minimize exceptions both malicious and accidental, and minimize the need for trusted intermediaries." This book has not checked the original of Szabo's 1994 text.

13. Personal Information Protection Act (current version, promulgated March 10, 2026, effective September 11, 2026), Article 36(1)–(3). The original: "① 제35조에 따라 자신의 개인정보를 열람한 정보주체는 개인정보처리자에게 그 개인정보의 정정 또는 삭제를 요구할 수 있다. 다만, 다른 법령에서 그 개인정보가 수집 대상으로 명시되어 있는 경우에는 그 삭제를 요구할 수 없다. ② 개인정보처리자는 제1항에 따른 정보주체의 요구를 받았을 때에는 개인정보의 정정 또는 삭제에 관하여 다른 법령에 특별한 절차가 규정되어 있는 경우를 제외하고는 지체 없이 그 개인정보를 조사하여 정보주체의 요구에 따라 정정·삭제 등 필요한 조치를 한 후 그 결과를 정보주체에게 알려야 한다. ③ 개인정보처리자가 제2항에 따라 개인정보를 삭제할 때에는 복구 또는 재생되지 아니하도록 조치하여야 한다." (Korean Law Information Center Open API (MST 283839, Article 36), retrieved 2026-10-25). Working gloss: "(1) A data subject who has inspected his or her personal information under Article 35 may request the personal information controller to correct or delete the personal information; provided that deletion may not be requested where other statutes specify the personal information as subject to collection. (2) Upon receiving a data subject's request under paragraph (1), the personal information controller shall, except where other statutes lay down a special procedure for the correction or deletion of personal information, without delay examine the personal information, take necessary measures such as correction or deletion in accordance with the data subject's request, and then notify the data subject of the result. (3) When deleting personal information under paragraph (2), the personal information controller shall take measures so that it cannot be restored or regenerated." Article 36 is unchanged in the versions of two amendments to the same Act scheduled to take effect in 2027 (Act No. 21910, effective March 9, 2027; Act No. 21445, effective July 1, 2027) (Korean Law Information Center, versions scheduled to take effect, retrieved 2026-10-26).

14. Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 (General Data Protection Regulation), OJ L 119, 4.5.2016, p. 1, Art. 17(1)–(2). The original: "1. The data subject shall have the right to obtain from the controller the erasure of personal data concerning him or her without undue delay and the controller shall have the obligation to erase personal data without undue delay where one of the following grounds applies: (a) the personal data are no longer necessary …; (b) the data subject withdraws consent …; (c) the data subject objects …; (d) the personal data have been unlawfully processed; …"; "2. Where the controller has made the personal data public and is obliged pursuant to paragraph 1 to erase the personal

data, the controller, taking account of available technology and the cost of implementation, shall take reasonable steps, including technical measures, to inform controllers which are processing the personal data that the data subject has requested the erasure by such controllers of any links to, or copy or replication of, those personal data." Cellar XHTML (`publications.europa.eu/resource/celex/32016R0679` , accessed and checked word for word on 2026-10-25). The exceptions in paragraph 3 (freedom of expression, legal obligations, the public interest, and so on) are not reproduced.

15. Act on the Protection of Virtual Asset Users (current version, Act No. 21857, effective October 2, 2026), Article 10(1)–(6). The original (paragraph (3), paragraph (4), subparagraph 1, and paragraph (6)): "③ 누구든지 가상자산의 매매를 유인할 목적으로 가상자산의 매매가 성황을 이루고 있는 듯이 잘못 알게 하거나 그 시세를 변동 또는 고정시키는 매매 또는 그 위탁이나 수탁을 하는 행위를 하여서는 아니 된다."; "④ 누구든지 가상자산의 매매, 그 밖의 거래와 관련하여 다음 각 호의 행위를 하여서는 아니 된다. 1. 부정한 수단, 계획 또는 기교를 사용하는 행위"; "⑥ 제1항부터 제5항까지를 위반한 자는 그 위반행위로 인하여 이용자가 그 가상자산의 매매, 그 밖의 거래와 관련하여 입은 손해를 배상할 책임이 있다." (Korean Law Information Center Open API (MST 290735, Article 10), retrieved 2026-10-25). Working gloss of the lead-in of paragraph (4) and of paragraph (6): "(4) No person shall engage in any of the following acts in connection with the trading or other transactions of virtual assets"; "(6) A person who has violated any of paragraphs (1) through (5) shall be liable to compensate for the loss that users have suffered in connection with the trading or other transactions of the virtual assets as a result of the violation." Investigation and measures are in Article 14 (Financial Services Commission), penalty surcharges in Article 17, and penal provisions in Article 19.

16. Regulation (EU) 2023/1114, Title VI "PREVENTION AND PROHIBITION OF MARKET ABUSE INVOLVING CRYPTO-ASSETS", Art. 86(1)–(2), Art. 89(1), Art. 91(1), L 150/140–143. The original: "1. This Title shall apply to acts carried out by any person concerning crypto-assets that are admitted to trading or in respect of which a request for admission to trading has been made. 2. This Title shall also apply to any transaction, order or behaviour concerning crypto-assets as referred to in paragraph 1, irrespective of whether such transaction, order or behaviour takes place on a trading platform." (Art. 86); "For the purposes of this Regulation, insider dealing shall be deemed to arise where a person possesses inside information and uses that information by acquiring or disposing of, for its own account or for the account of a third party, directly or indirectly, crypto-assets to which that information relates." (Art. 89(1)); "No person shall engage in or attempt to engage in market manipulation." (Art. 91(1)).

17. *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. Mar. 6, 2018), pp. 3, 24; see footnote 21 in Chapter 27. "(2) whether the amendments to the CEA under the Dodd-Frank Act permit the CFTC to exercise its jurisdiction over fraud that does not directly involve the sale of futures or derivative contracts. Both questions are answered in the affirmative." (p. 3).

18. Civil Procedure Act (current version, effective October 2, 2026), Articles 357 and 358; Digital Signature Act (current version, effective October 20, 2022), Article 3. The original: "제357조(사문서의 진정의 증명) 사문서는 그것이 진정한 것임을 증명하여야 한다."; "제358조(사문서의 진정의 추정) 사문서는 본인 또는 대리인의 서명이나 날인 또는 무인(拇印)이 있는 때에는 진정한 것으로 추정한다."; "제3조(전자서명의 효력) ① 전자서명은 전자적 형태라는 이유만으로 서명, 서명날인 또는 기명날인으로서의 효력이 부인되지 아니한다. ② 법령의 규정 또는 당사자 간의 약정에 따라 서명, 서명날인 또는 기명날인의 방식으로 전자서명을 선택한 경우 그 전자서명은 서명, 서명날인 또는 기명날인으로서의 효력을 가진다." (Open API, retrieved 2026-10-25). Working gloss of the article headings and of Article 3(2): Article 357 (Proof of Authenticity of Private Documents); Article 358 (Presumption of Authenticity of Private Documents); Article 3 (Effect of Electronic Signatures) "(2) Where an electronic signature has been chosen as the method of signature, signature and seal, or name and seal under statutory provisions or an agreement between the parties, the electronic signature shall have effect as a signature, a signature and seal, or a name and seal." This book has not checked the other provisions on the admissibility of electronic documents as evidence and the method of examining them.

19. Tokyo High Court, 4th Criminal Division (東京高等裁判所第４刑事部), judgment of June 11, 2020 (令和2年6月11日 宣告 判決; 令和元年(う)第948号), pp. 10–11 (第３の２(３)). The original: "電磁的記録には，以下のような文書偽造罪等における文書とは異なる特質がある。すなわち，作成過程については，可視性，可読性がないだけでなく，入力されたデータが既存のデータとともに加工，処理され新たに作り出されることが少なくなく，その作出に複数の者の意思や行為が関わることがある。また，利用過程については，それを使用することが予定されている一定のシステムやプログラムのもとで用いられて初めて本来の証明機能を果たし得るものである。そうであると，電磁的記録の証明機能は，それが作出されるべきシステムの設置運営主体の意思に基づき，

正当に作出されることが当然の前提とされているもので,
このように正当に作られたものであることの信頼を確保する必要があるという観点から解釈すべきと解される。"
Working gloss: "Electromagnetic records have the following characteristics that differ from documents in the crime of forgery of documents and the like. As to the process of creation, they are neither visible nor readable, and in addition they are often newly created by entered data being processed and handled together with existing data, and the wills and acts of several persons may be involved in their creation. As to the process of use, they can perform their proper evidentiary function only when used under the particular system or program in which their use is intended. That being so, the evidentiary function of an electromagnetic record is premised as a matter of course on its being properly created in accordance with the will of the entity that set up and operates the system in which it is to be created, and the provision is to be interpreted from the viewpoint of the need to secure trust that the record has been properly made in this way." The same place as footnote 15 in Chapter 26.

20. SEC, Release No. 81207 (2017-07-25), pp. 1, 15–16. The original: "The United States Securities and Exchange Commission's ... Division of Enforcement ... has investigated whether The DAO, an unincorporated organization; Slock.it UG ('Slock.it'), a German corporation; Slock.it's co-founders; and intermediaries may have violated the federal securities laws." (p. 1); "The definition of 'issuer' is broadly defined to include 'every person who issues or proposes to issue any security' and 'person' includes 'any unincorporated organization.' 15 U.S.C. § 77b(a)(4)." (p. 15); "The DAO, an unincorporated organization, was an issuer of securities, and information about The DAO was 'crucial' to the DAO Token holders' investment decision." (p. 16).

21. SEC (2017), pp. 3–4, n. 10. The original: "The White Paper contained the following statement: A word of caution, at the outset: the legal status of [DAO Entities] remains the subject of active and vigorous debate and discussion. Not everyone shares the same definition. Some have said that [DAO Entities] are autonomous code and can operate independently of legal systems; others have said that [DAO Entities] must be owned or operate[d] by humans or human created entities. ... This paper does not speculate about the legal status of [DAO Entities] worldwide." The same place as quoted in the Chapter 22 question box.

22. *Tulip Trading Ltd v van der Laan and others* [2023] EWCA Civ 83 (3 February 2023), ¶¶1, 91; see footnote 41 in Chapter 27.

23. EIP-779, Abstract; Vitalik Buterin, "Hard Fork Completed" (2016-07-20); see footnotes 23 and 22 in Chapter 26.

24. Civil Act, Article 703(1) (Open API, retrieved 2026-10-25); Lee Hye-jeong (2022), pp. 85, 93; see footnote 6 in Chapter 27.

PART 6 · OPEN PROBLEMS

Chapter 29. The quantum transition: signatures break, hashes stand, and the vulnerable coins

KEY TAKEAWAY

What a quantum computer threatens is not "Bitcoin" but **particular signature schemes**. Shor's algorithm solves the discrete logarithm on an elliptic curve in polynomial time and extracts the private key from the public key; Grover's algorithm speeds up the preimage search of a hash function only by a square root. So the signatures of Chapter 5 break, while the hashes and proof of work of Chapters 4 and 8 stand. The size of the machine needed for an attack has come down from 2,330 logical qubits in 2017 to around 1,200 logical qubits in 2026. Estimates of physical qubits range, depending on the assumptions, from hundreds of thousands to the billions, and even within a single paper the target time alone moves them from 13 million to 1.9 billion. Estimates are functions of assumptions, not observations. What is at risk is **coins whose public keys have already been revealed**, and a secondary tally from the reference-date snapshot puts that range at about one third of issuance. The Bitcoin-side transition proposals (BIP-360 and BIP-361) are Drafts as of 2026-10, no post-quantum element has been activated, and an official Ethereum page has published a roadmap that gives 2029 as a planning target that may shift. What lawyers take away is not the technology but **the last question**: is blocking the spending of vulnerable coins by soft fork an expropriation, a measure for maintaining order, or the disposal of ownerless property, and whose agent is the majority that decides it? This chapter asks that question and does not answer it.

Learning objectives

1. Explain how Shor's algorithm threatens signatures (ECDLP) and how Grover's algorithm threatens hashes, and the asymmetry between the two (signatures break, hashes stand).
2. **[Intermediate]** Distinguish logical from physical qubits, and read from a table how resource estimates move by hundreds of times depending on the assumptions (error rate, code cycle, reaction time, connectivity, target time).
3. Identify the three places where a public key is revealed on the chain (script type, reuse, the mempool), and read the amounts by type in the reference-date snapshot together with the caveat that they are a secondary tally.
4. Know what the transition proposals (BIP-360 and BIP-361, NIST FIPS 203 to 205, Ethereum's four areas) change and what they do not change.
5. Set out as a structure the three paths of the debate over "restricting the spending of

vulnerable coins" (leave them, burn them, limit the speed), and hand the legal questions they lead to over to their place in Part 5.

What a reader of the [Basic] paragraphs alone takes away. What a quantum computer breaks is not "Bitcoin" but **a particular signature scheme**, and the truly hard problem is not the technology but **who decides on the transition, with what authority, and what is to be done with the coins left behind**. The basis for misconception ⑤ in Appendix B ("When quantum computers arrive, Bitcoin is finished") is in this chapter.

1. Two algorithms, two targets: an asymmetry

Part 1 used two cryptographic tools: the hash function of Chapter 4 and the public-key signature of Chapter 5. A quantum computer affects the two **differently**. A 2016 report of the National Institute of Standards and Technology (NIST) set out the difference in one table: symmetric-key cryptography (AES), "Larger key sizes needed"; hash functions (SHA-2, SHA-3), "Larger output needed"; and RSA, elliptic curve signatures (ECDSA), elliptic curve key exchange (ECDH) and finite field signatures (DSA), "No longer secure."[1] Bitcoin's signature is ECDSA over the secp256k1 curve (Chapter 5), and since 2021 Bitcoin has also used Schnorr signatures over the same curve. Both belong to the third line. Bitcoin's proof of work runs SHA-256 twice (Chapter 8), and that belongs to the second line.

Why the difference? The report names two algorithms. In 1994 Peter Shor of Bell Laboratories showed that a quantum computer can solve integer factorization and the discrete logarithm problem **efficiently**, thereby "rendering all public key cryptosystems based on such assumptions impotent."[2] What Lov Grover's search algorithm offers, by contrast, is "a quadratic speedup on unstructured search problems," and the report says: "While such a speedup does not render cryptographic technologies obsolete, it can have the effect of requiring larger key sizes, even in the symmetric key case."[3] In Chapter 5 the security of a signature rested on the fact that no fast way is known to find k from $[k]G$. That is true on a classical computer. On a quantum computer, Shor's algorithm finds exactly that k. In Chapter 4 the security of a hash rested on the fact that finding a preimage takes an expected 2^{256} attempts, and Grover's algorithm cuts that to around 2^{128} operations. That is still an astronomical number, and, as the table above has it, it is the kind of speedup that can be undone by a "[l]arger output."

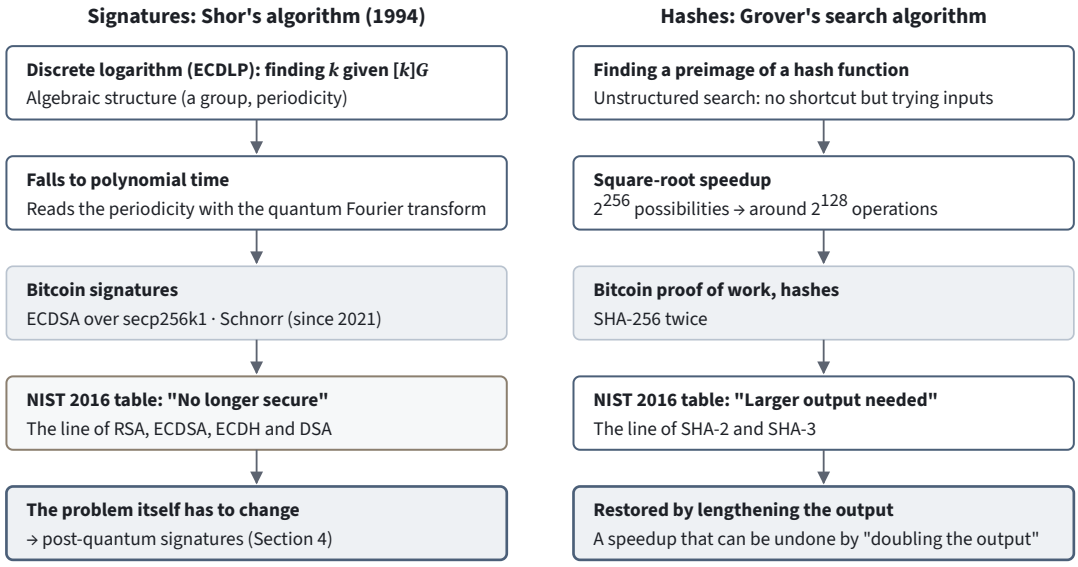


Figure 29-1. The two paths by which Shor's algorithm and Grover's algorithm affect Bitcoin's signatures and hashes. On the left, the structured discrete logarithm problem falls to polynomial time, so the signature problem itself has to be changed; on the right, unstructured search is sped up only by a square root, and security is restored through output length. The quoted phrases are from the table in the NIST 2016 report

Among the literature this chapter opened, the earliest peer-reviewed paper to take on the quantum threat to Bitcoin directly put this asymmetry in numbers. It is a 2018 paper by Divesh Aggarwal and four co-authors. The conclusion of its abstract is two sentences: "We find that the proof-of-work used by Bitcoin is relatively resistant to substantial speedup by quantum computers in the next 10 years, mainly because specialized ASIC miners are extremely fast compared to the estimated clock speed of near-term quantum computers. On the other hand, the elliptic curve signature scheme used by Bitcoin is much more at risk, and could be completely broken by a quantum computer as early as 2027, by the most optimistic estimates."[4] The year in the second sentence is a "most optimistic" extrapolation made in 2017, and this book takes no part in that extrapolation (Section 2). The structure of the first sentence has been confirmed again and again in the later literature. A 2026 white paper by the Google Quantum AI team and others says that Grover-based attacks on mining are "not something to worry about in the next several decades," and gives two reasons: the square-root speedup is almost entirely consumed by the cost of quantum error correction, and Grover's algorithm does not parallelize well, whereas mining is a world of hardware acceleration and massive parallelization.[5]

[Intermediate] In numbers it looks like this. Aggarwal et al. calculate that the number of logical qubits a Grover miner needs is fixed at 2,402 regardless of difficulty. Assuming the then-maximum gate speed of superconducting circuits, 66.7 MHz, a physical gate error rate of 5×10^{-4} and a difficulty of 10^{12}, they put the effective hash rate of one quantum miner at 13.8 GH/s (with about 4.4 million physical qubits): "more than one thousand times slower than off the shelf ASIC devices which achieve hash rates of 14TH/s."[6] The 2026 white paper reaches the same conclusion with a more extreme assumption. Even "Under the fantastical assumption that a CRQC could compute SHA-256 in a single 1 microsecond error correction

cycle," it says, a quantum miner's hash rate is 0.25 TH/s, two orders of magnitude below the 110 TH/s of the common ASIC miner S19 Pro.[7] As Chapter 8 showed, the security of proof of work rests not on absolute speed but on **share**, and nowhere in the literature this chapter opened is there an estimate that quantum miners would be fast enough to overturn the shares.

▾ **[Advanced] Why the asymmetry: a problem with structure and a problem without**

[Advanced] The advanced box in Chapter 5 showed the problem of finding k given the points G and $Q = [k]G$ on an elliptic curve (ECDLP). This problem has **algebraic structure**. The points form a group, and scalar multiplication is a periodic function on that group. Shor's algorithm reads out exactly that periodicity with the quantum Fourier transform. A 2017 paper by Martin Roetteler and colleagues at Microsoft sums it up like this: Shor "presented two polynomial-time quantum algorithms, one for integer factorization and another one for computing discrete logarithms in a finite field of prime order. Shor notes that the latter algorithm can be generalized to other fields. It also generalizes to the case of elliptic curves."[8] Finding a preimage of a hash function, on the other hand, has no such structure. It is an "unstructured search" with no shortcut other than trying inputs one at a time. All that Grover's algorithm achieves there is to reduce N to $\sqrt{N}$, and, by the 1997 result the NIST report cites, an exponential speedup for search problems is **impossible** (reference [3] of the report, Bennett, Bernstein, Brassard and Vazirani 1997). So under the same word "quantum," one problem falls to polynomial time and the other shrinks only by a square root. The three properties of Chapter 4 (preimage, second-preimage and collision resistance) can be restored by lengthening the output, but the signatures of Chapter 5 **have to change the problem itself**. That is the "post-quantum signature" of Section 4.

[Advanced] One more point. Aggarwal et al. show that a variant of hash-based proof of work that requires **finding collisions** (Momentum) allows even less quantum advantage than Grover's search does. Using the result that the quantum lower bound for finding collisions is $|S|^{2/3}$ (Aaronson and Shi 2004), they argue that for an optimal classical time T the quantum time is at least $T^{2/3}$: "unlike for the current proof of work in Bitcoin, with this proposal a quantum computer would not be able to achieve a quadratic advantage as soon as S is more than constant size."[9] This book does not evaluate that proposal and records only the fact that "on the hash side, there is room to make designs that last longer."

2. Resource estimates: two kinds of qubit, five assumptions

The proposition "quantum computers break signatures" is true. "When will such a quantum computer be built?" is a different question, and this book does not answer it (the notes to the reader in Chapter 3: the book makes no projections). Instead it shows **how that question is counted**. Two units have to be separated first. **Logical qubits** are the error-corrected, abstract qubits that an algorithm actually uses; **physical qubits** are the actual devices in the hardware. Making one logical qubit takes hundreds to thousands of physical qubits, and the multiple changes with the error rate of the devices, the error-correcting code and the target running time. So "1,200 logical" and "500,000 physical" describe the same machine in two units. The logical count moves with improvements in algorithms; the physical count moves with those and, in addition, with hardware assumptions.

[Intermediate] Estimates of the resources needed to solve ECDLP on a 256-bit prime-field elliptic curve with Shor's algorithm have come down as shown below. Only the four papers whose originals this book opened are listed.

YEAR	PAPER	TARGET CURVE	LOGICAL QUBITS	NON-CLIFFORD GATES (TOFFOLI)	PHYSICAL QUBITS · TIME	ASSUMPTIONS
2017	Roettel er et al.	NIST P-256	2,330	about 1.26×10^{11}	(logical level only)	general formula: $9n + 2\lceil \log_2 n \rceil + 10$ qubits, $448n^3 \log_2 n + 4090n^3$ gates
2018	Aggarw al et al.	Bitcoin signatures (Roetteler as input)	2,334 (incl. 4 ancilla)	1.28×10^{11}	1.7×10^6 · 6.49 days / 485,550 · 30 min	surface code; error 5×10^{-4} · 66.6 MHz / error 10^{-5} · 10 GHz
2022	Webber et al.	"the 256 bit elliptic curve encryption of keys in the Bitcoin network" (a prior circuit as input)	2,871	5.76×10^9 (T)	13 million · 1 day / 317 million · 1 hour / 1.9 billion · 10 min	surface code; error 10^{-3} · code cycle 1 µs · reaction 10 µs
2026	Babbus h et al.	secp256k1	≤1,200 / ≤1,450	≤90 million / ≤70 million	under 500,000 · 18–23 min (9–12 min from a primed state)	surface code (yoke storage); error 10^{-3} · planar connectivity · reaction 10 µs

[Intermediate] The general formula in the first row is an upper bound that Roetteler et al. obtained by **actually simulating** the circuit: "elliptic curve discrete logarithms on an elliptic curve defined over an n-bit prime field can be computed on a quantum computer with at most $9n + 2\lceil \log_2(n) \rceil + 10$ qubits using a quantum circuit of at most $448n^3 \log_2(n) + 4090n^3$ Toffoli gates," and for P-256, "2330 logical qubits" and "about $1.26 \cdot 10^{11}$ Toffoli gates."[10] Aggarwal et al. took this circuit as input, applied it to Bitcoin signatures, and produced physical qubits and times under two hardware assumptions: "Using a surface code with a physical gate error rate of $p_g = 5 \times 10^{-4}$, ... the time to solve the problem at 66.6 MHz clock

speed is 6.49 days using 1.7×10^6 physical qubits. Looking forward to performance improvements, for 10GHz clock speed and error rate of 10^{-5}, the signature is cracked in 30 minutes using 485550 qubits."[11] Mark Webber and colleagues took a different prior circuit (2,871 logical qubits, 5.76×10^9 T gates) as input and showed how far the number of physical qubits moves when, within the same hardware assumptions, **only the target time is changed**: 13 million for a day, 317 million for an hour, 1.9 billion for 10 minutes.[12] Ryan Babbush and colleagues did not publish their circuits; they published only the resource counts, made verifiable with a zero-knowledge proof. The counts are "≤1200 logical qubits and ≤90 million Toffoli gates or ≤1450 logical qubits and ≤70 million Toffoli gates," and on superconducting architectures the circuits run "in minutes using fewer than half a million physical qubits."[13]

[Intermediate] The code below plugs $n = 256$ into the general formula of the first row and puts the logical resources of the four papers in one table. It does not compute physical qubits, because they are not a number that comes out of one formula but a function of five assumptions.

```
# NB29-1: ECDLP resources for Shor's algorithm — n=256 plugged into the general formula of
Roetteler et al. (2017), and the logical resources of four papers compared (for study)
import math

def roetteler_bound(n):
    """Upper-bound formula in the abstract of Roetteler et al. (2017): logical qubits and
Toffoli gates."""
    qubits = 9 * n + 2 * math.ceil(math.log2(n)) + 10
    toffoli = 448 * n**3 * math.log2(n) + 4090 * n**3
    return qubits, toffoli

for n in (192, 224, 256, 384, 521):
    q, t = roetteler_bound(n)
    print(f"n={n:>3}-bit: logical qubit bound {q:>5},  Toffoli bound {t:.3e}")

q256, t256 = roetteler_bound(256)
print()
print("n=256 — Toffoli bound from the formula", f"{t256:.4e}", "| value the paper reports from
simulation: about 1.26e11 (the formula is an upper bound, so slightly larger)")
print("The 1.28e11 that Aggarwal et al. (2018) plugged in is the formula's bound as is →", f"
{t256/1.28e11:.3f}", "x")

papers = [  # (year, paper, logical qubits, Toffoli or T gate count)
    (2017, "Roetteler et al. (P-256)",           2330, 1.26e11),
    (2018, "Aggarwal et al. (incl. 4 ancilla)",  2334, 1.28e11),
    (2022, "Webber et al. circuit (T gates)",    2871, 5.76e9),
    (2026, "Babbush et al. — low-qubit circuit", 1200, 9.0e7),
    (2026, "Babbush et al. — low-gate circuit",  1450, 7.0e7),
]
print()
print(f"{'Year':<6}{'Paper':<34}{'Logical qubits':>15}{'Gates':>12}{'Gates vs 2017':>20}")
for y, name, lq, gates in papers:
    ratio = "different unit (T)" if "T gates" in name else f"{1.26e11/gates:.0f}x"
    print(f"{y:<6}{name:<34}{lq:>15,}{gates:>12.2e}{ratio:>20}")
print()
print("Physical qubits are not computed here: they are a function of error rate, code cycle,
reaction time, connectivity and target time (Section 2, advanced box).")
```

```
# Output — bcbook (Python 3.13), exactly as run
n=192-bit: logical qubit bound  1754,  Toffoli bound 5.300e+10
n=224-bit: logical qubit bound  2042,  Toffoli bound 8.528e+10
n=256-bit: logical qubit bound  2330,  Toffoli bound 1.287e+11
n=384-bit: logical qubit bound  3484,  Toffoli bound 4.494e+11
n=521-bit: logical qubit bound  4719,  Toffoli bound 1.150e+12

n=256 — Toffoli bound from the formula 1.2875e+11 | value the paper reports from simulation:
about 1.26e11 (the formula is an upper bound, so slightly larger)
The 1.28e11 that Aggarwal et al. (2018) plugged in is the formula's bound as is → 1.006 x

Year  Paper                              Logical qubits       Gates       Gates vs 2017
2017  Roetteler et al. (P-256)                    2,330    1.26e+11                  1x
2018  Aggarwal et al. (incl. 4 ancilla)           2,334    1.28e+11                  1x
2022  Webber et al. circuit (T gates)             2,871    5.76e+09  different unit (T)
2026  Babbush et al. — low-qubit circuit          1,200    9.00e+07              1400x
2026  Babbush et al. — low-gate circuit           1,450    7.00e+07              1800x

Physical qubits are not computed here: they are a function of error rate, code cycle, reaction
time, connectivity and target time (Section 2, advanced box).
```

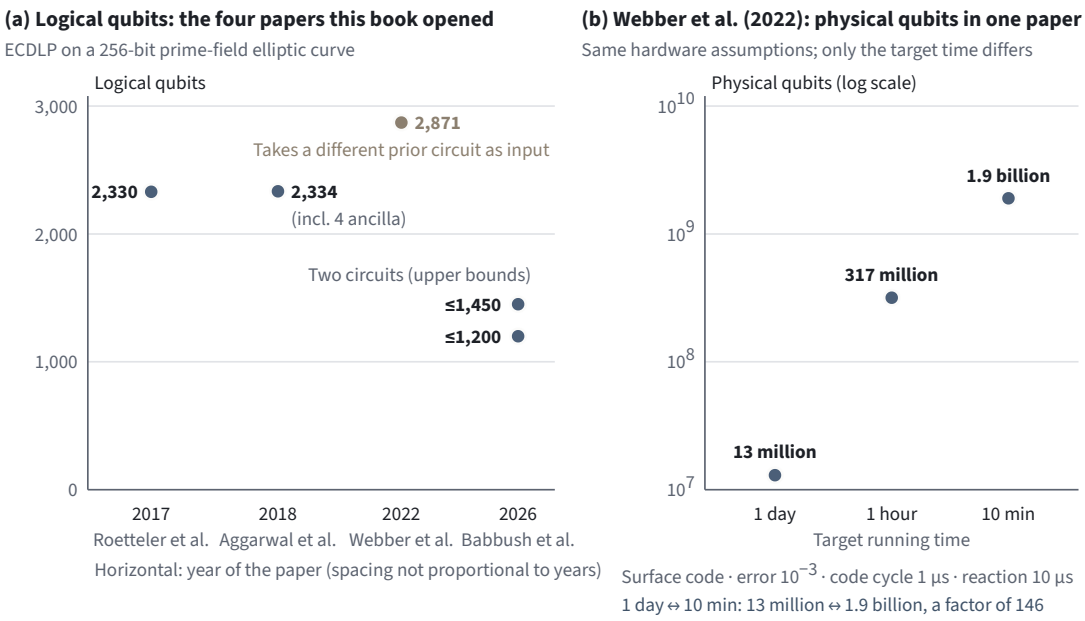


Figure 29-2. The resource estimates of the table above, carried over to two panels as points only. Panel (a) shows the logical qubits of the four papers; the 2022 value takes a different prior circuit as input. Panel (b) shows the physical qubits (log scale) within the single paper by Webber et al., with the hardware assumptions fixed and only the target time changed. Because the assumptions differ, the points are not joined, and no trend line or extension is drawn

What to read from the table is **the direction**, not the figures. In nine years the logical qubits fell to almost half (2,330 → around 1,200), and the gate count fell more than a thousandfold. That is the result of movement in **algorithms and error-correction theory**, not in hardware, and that is why Babbush et al. say that "simple models, such as counting physical qubits, fail to adequately capture technological progress" and propose a threshold model instead: progress comes not continuously but in "discrete jumps corresponding to development of new internal capabilities and overcoming scaling challenges."[14] Here, too, is a reason why this book does not name a date. The "as early as 2027" of the 2017 paper was an extrapolation of the hardware trends of the time, and the 2026 white paper records skepticism about exactly that way of extrapolating. The figures from an expert survey (published March 2026,

26 respondents) are put in Appendix A, not in the text, because opinions are not observations.

▾ [Advanced] Five assumptions: estimates can be compared only under the same assumptions

[Advanced] Here the reasons physical-qubit estimates diverge by hundreds of times are taken one assumption at a time. (i) **Physical error rate.** Aggarwal et al.'s two scenarios differ 50-fold in error rate, 5×10^{-4} against 10^{-5}, and also 150-fold in clock, 66.6 MHz against 10 GHz. Changing the error rate and the clock together moves the physical qubits from 1.7 million to 480,000 and the time from 6.49 days to 30 minutes, and most of the difference in time comes from the clock. (ii) **Code cycle and reaction time.** Webber et al.'s results assume a cycle of 1 μs and a reaction time of 10 μs. Slow platforms (ion traps and the like) have cycles of hundreds of μs, so running the same logical circuit takes far longer. That is why Babbush et al. separate "fast-clock" architectures (superconducting, photonic) from "slow-clock" architectures (neutral atoms, ion traps) and set out two scenarios: if the former arrives first, "on-spend" attacks, which can reach even transactions that are being spent, come with it; if the latter arrives first, "at-rest" attacks, which can reach only public keys exposed for a long time, come first.[15] (iii) **Connectivity.** Babbush et al. assume degree-four connectivity on a planar grid. They note separately that other codes assuming more aggressive connectivity give numbers "closer to one hundred thousand," but that degree-seven connectivity "has yet to be demonstrated in actual superconducting qubit devices," and that the degree-ten Pinnacle architecture (for RSA-2048), with its more demanding device specifications, "would require nearly a month of runtime."[16] (iv) **Target running time.** Within the single paper by Webber et al., a day, an hour and 10 minutes give 13 million, 317 million and 1.9 billion: a factor of 146. Why the target time matters for Bitcoin is shown in the next section, because the public key of a transaction being spent is exposed only in the **short window** before it enters a block. (v) **The logical circuit itself.** This is the drop from 2,330 to 1,200. So "Webber's 317 million ↔ Babbush's under 500,000" is a 600-fold difference, but the two differ in input circuit, in the way error-corrected qubits are stored and in target time. Estimates can be compared only under the same assumptions, and none of the figures is the true value.

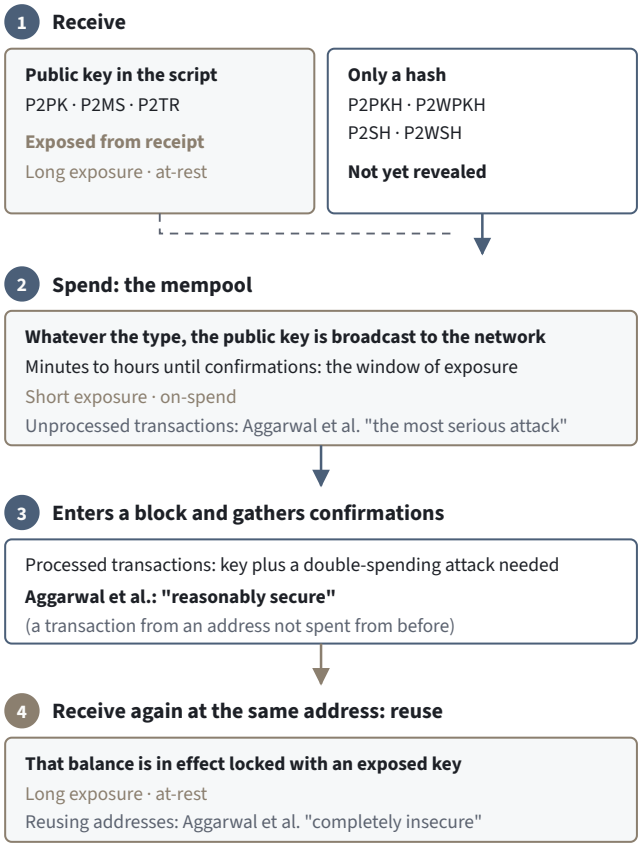
3. What is exposed: three places where a public key appears

The input that Shor's algorithm needs is **the public key**. If the public key cannot be seen, there is no private key to extract. In Chapter 5 we saw that a Bitcoin "address" is not the public key itself but its hash; that an address that has only received money through hash-type outputs (P2PKH, P2WPKH) has therefore not yet revealed its public key; and that the genesis output (P2PK) of Chapter 12 and the Taproot output (P2TR) carry the public key as it is. That distinction is the center of this chapter. A public key is revealed in three places.

1. **Outputs with the key in the script itself.** P2PK (early mining rewards, including the genesis output), bare multisig (P2MS), and the P2TR of 2021. These are exposed from the

moment of receipt.

2. **Reused hash-type addresses.** Spending even once from a P2PKH or P2WPKH address publishes the public key in that input, and if money is received at the same address again, that balance is in effect locked with an exposed key. For P2SH and P2WSH, the script is published.
3. **The mempool.** Whatever the type, the moment an output is spent its public key is broadcast to the network, and the minutes to hours until the transaction enters a block and gathers confirmations are the window of exposure.



Terms: BIP-360 "long/short exposure"; Babbush et al. "at-rest/on-spend"

Figure 29-3. Where a public key is revealed in the life of one output. An output with the key in the script is exposed from the moment of receipt; a hash-type output is exposed in the mempool at the moment of spending; for a reused address, the balance received again is locked with an exposed key. The quoted phrases are Aggarwal et al.'s division into three situations

BIP-360 calls the first two a "long exposure attack" and the third a "short exposure attack," and sets out the vulnerability of each type in a table.[17] Babbush et al. call the same distinction "at-rest" and "on-spend," and also count exposure off the chain: spending on another chain, sharing with third-party apps, and sharing extended public keys (xpubs).[18]

OUTPUT TYPE	VULNERABLE TO LONG EXPOSURE? (BIP-360 TABLE)	WHY
P2PK	Yes	The public key sits in the locking script as it is
P2PKH	No*	Only a hash. Vulnerable if reused after spending
P2MS	Yes	The public keys sit in the locking script as they are
P2SH	No*	Only a script hash. Vulnerable if reused
P2WPKH	No*	Same as above (SegWit)
P2WSH	No*	Same as above (SegWit)
P2TR	Yes	The tweaked public key sits in the script as it is
P2MR (proposed)	No*	Only the Merkle root of the script tree (Section 4)

Take note that the Taproot of 2021 is in the "Yes" column. As Chapter 5 said, Taproot puts a single public key directly in the output for key-path spending. Babbush et al. say that this, "from the standpoint of quantum security, constitutes a security regression."[19] A design of 2021 brought back, for a different reason, the exposure that the P2PK of 2009 already had, and what BIP-360 in Section 4 sets out to do is precisely to remove that key path.

[Intermediate] How much, then, is exposed? This book bundles one **secondary tally** as of the reference date (2026-10-25). It is a tally by a third party that has made its SQL public (Project Eleven), covering the public Bitcoin dataset in BigQuery up to block 972,132 (produced 2026-10-19); this book did not rerun that SQL. The credibility of the source is 🟡, and a primary reproduction is a task for the next edition.[20] The unit is BTC, and the denominator is the scheduled issuance up to that height, 20,100,415.625 BTC (an upper bound that includes the unspendable genesis output and unclaimed subsidies).

CATEGORY	BTC	SHARE OF SCHEDULED ISSUANCE
P2PK (key in the script)	1,715,077.0	8.53%
P2TR (key in the script)	217,063.2	1.08%
Bare P2MS (keys in the script)	416.6	0.002%
P2PKH reused (with a spending history)	1,185,672.2	5.90%
P2WPKH reused	2,098,570.4	10.44%
P2SH reused (script published)	1,199,814.7	5.97%
P2WSH reused	705,381.4	3.51%
Total	**7,121,995.4**	**35.43%**

[Intermediate] Cross-checks agree **only in order of magnitude**, because their dates and methods differ. A May 2025 report from mempool research divided the UTXO set at block 892,385 (2025-04-14) by script type alone and reports that P2PK is 0.03% of UTXOs **by count**

but 8.70% (1,720,270 BTC) **by value**, while P2TR is 34.22% by count but 0.75% (147,912 BTC) by value.[21] In the one category where the two tallies overlap (P2PK) they agree, at 1.71 million and 1.72 million, and that is the only ground for trusting these figures. Industry estimates gather in the same place; this book uses them only as secondary sources. A May 2025 report from Chaincode Labs, citing a 2023 query, says that P2PK "represents only ~0.025% of current UTXOs but locks a disproportionate ~8.68% of Bitcoin's value (~1,720,747 BTC)"; it gives the range including reuse as "Approximately 20-50% of all Bitcoin in circulation (4-10 million BTC)"; and it carries an earlier value from the same organization as this chapter's primary source (Project Eleven), about 6.26 million BTC. (The original says "as of mid January 2015 [sic]," which from the context appears to be a slip for 2025. It is not an independent cross-check.)[22] An analysis by Deloitte (no date in the text; the date of the figures is unknown) gives "over 4 million BTC (about 25% of all Bitcoins)" for P2PK and reused P2PKH alone, and a February 2026 post by CoinShares gives, for P2PK alone, "approximately 1.6 million btc—about 8% of the total supply."[23] The Motivation section of BIP-361 says that "As of March 1, 2026, over 34% of all bitcoin have revealed a public key on-chain ...," but does not state how this was computed.[24] Babbush et al., from their own tally, give "A little over 1.7 million bitcoin (nearly 9% of all bitcoin)" locked by P2PK and "~6.9M total bitcoin across all protocols."[25] The spread has three causes: whether reuse is counted, whether P2TR and P2SH are included, and when the snapshot was taken. For **P2PK at about 1.7 million BTC**, the four sources that tallied directly (the snapshot, mempool, Chaincode, Babbush et al.) converge in the 1.7 millions and the two industry sources give 1.6 to 2 million, so it is, together with the asymmetry of Section 1, the firmest number in this chapter.

```
# NB29-2: exposed amounts by type in the reference-date snapshot (a secondary tally) — reads the
bundled file and recomputes subtotals and shares (for study)
import json, os
from decimal import Decimal

FALLBACK = {  # the same values as data/exposed_output_types_2026-10-25.json (for environments
without the file)
    "height": 972132, "created": "2026-10-19T10:28:55Z", "issued": "20100415.625",
    "rows": [("p2pk_script_type", "1715076.99695260"), ("taproot_script_type",
"217063.18329201"),
             ("multisig_script_type", "416.60645555"), ("pubkeyhash_reuse", "1185672.15392868"),
             ("witness_v0_keyhash_reuse", "2098570.39816002"), ("scripthash_reuse",
"1199814.71613438"),
             ("witness_v0_scripthash_reuse", "705381.38634311"), ("witness_unknown_reuse",
"0.00000000")]}

try:
    with open(os.path.join("data", "exposed_output_types_2026-10-25.json"), encoding="utf-8") as
f:
        snap = json.load(f)
    prim = snap["primary"]
    height, created = prim["snapshot_block_height"], prim["snapshot_created_at_utc"][:19]
    issued = Decimal(snap["denominator"]["issued_supply_btc"])
    rows = [(r["category"], r["btc"]) for r in prim["by_category"]]
    src = "bundled file"
except FileNotFoundError:
    height, created, issued = FALLBACK["height"], FALLBACK["created"],
Decimal(FALLBACK["issued"])
    rows, src = FALLBACK["rows"], "constants (no file)"

print(f"Source: {src} | snapshot block {height:,} ({created}) | 2nd-hand tally (3rd-party SQL,
not rerun) | denominator = scheduled issuance {issued:,} BTC (upper bound)")
print(f"{'Category':<30}{'BTC':>18}{'of issuance':>12}")
script_type = Decimal(0); reuse = Decimal(0)
for cat, btc in rows:
    v = Decimal(btc)
    if cat.endswith("script_type"): script_type += v
    else: reuse += v
    print(f"{cat:<30}{v:>18,.3f}{v / issued * 100:>11.3f}%")
total = script_type + reuse
print("-" * 60)
print(f"{'Subtotal: key in script':<30}{script_type:>18,.3f}{script_type / issued *
100:>11.3f}%")
print(f"{'Subtotal: revealed by reuse':<30}{reuse:>18,.3f}{reuse / issued * 100:>11.3f}%")
print(f"{'Total':<30}{total:>18,.3f}{total / issued * 100:>11.3f}%")
print()
print("How to read: the denominator is an upper bound on the circulating supply, so the shares
lean low. The reuse categories also count scripts without keys as exposed, so they lean high.")
print("Off-chain exposure (xpubs, signed messages) and exposure while in the mempool are not
counted. No conversion to prices is made.")
```

```
# Output — bcbook (Python 3.13), exactly as run
Source: bundled file | snapshot block 972,132 (2026-10-19T10:28:55) | 2nd-hand tally (3rd-party
SQL, not rerun) | denominator = scheduled issuance 20,100,415.625 BTC (upper bound)
Category                                    BTC of issuance
p2pk_script_type                  1,715,076.997      8.533%
taproot_script_type                 217,063.183      1.080%
multisig_script_type                    416.606      0.002%
pubkeyhash_reuse                  1,185,672.154      5.899%
witness_v0_keyhash_reuse          2,098,570.398     10.440%
scripthash_reuse                  1,199,814.716      5.969%
witness_v0_scripthash_reuse         705,381.386      3.509%
witness_unknown_reuse                     0.000      0.000%
------------------------------------------------------------
Subtotal: key in script           1,932,556.787      9.615%
Subtotal: revealed by reuse       5,189,438.655     25.818%
Total                             7,121,995.441     35.432%

How to read: the denominator is an upper bound on the circulating supply, so the shares lean
low. The reuse categories also count scripts without keys as exposed, so they lean high.
Off-chain exposure (xpubs, signed messages) and exposure while in the mempool are not counted.
No conversion to prices is made.
```

[Intermediate] Classifying the type takes a few bytes. The head of the locking script (scriptPubKey) determines which type it is, and so **whether the key is revealed at the moment of receipt**. The code below classifies the genesis output script of Chapter 12 (67 bytes, P2PK) from its real bytes, and shows the classification rules for the other types with fake bytes shaped to match. The fake bytes are not addresses and cannot send money anywhere (not for real use: this is a classifier for study).

```
# NB29-3: tells the output type, and "whether the public key is exposed at the moment of
receipt," from the head of the locking script (for study — fake bytes, not for real use)
GENESIS_SPK = bytes.fromhex(  # scriptPubKey of the genesis coinbase output in Chapter 12 (67
bytes): PUSH65 <public key> OP_CHECKSIG

"4104678afdb0fe5548271967f1a67130b7105cd6a828e03909a67962e0ea1f61deb649f6bc3f4cef38c4f35504e51ec
112de5c384df7ba0b8d578a4c702b6bf11d5fac")

def classify(spk: bytes):
    """Standard locking-script type → (type, key exposed on receipt?, note). Same judgment as
the 'long exposure' column of the BIP-360 table."""
    n = len(spk)
    if n in (67, 35) and spk[0] in (0x41, 0x21) and spk[-1] == 0xAC:
        return "P2PK", True, "public key (65 or 33 bytes) sits in the script as is"
    if n == 25 and spk[:3] == b"\x76\xa9\x14" and spk[-2:] == b"\x88\xac":
        return "P2PKH", False, "only the HASH160 of the public key (20 bytes) — key revealed
when spent, vulnerable if reused"
    if n == 23 and spk[:2] == b"\xa9\x14" and spk[-1] == 0x87:
        return "P2SH", False, "only the HASH160 of the script — script revealed when spent"
    if n == 22 and spk[:2] == b"\x00\x14":
        return "P2WPKH", False, "SegWit v0, key hash of 20 bytes"
    if n == 34 and spk[:2] == b"\x00\x20":
        return "P2WSH", False, "SegWit v0, script hash of 32 bytes"
    if n == 34 and spk[:2] == b"\x51\x20":
        return "P2TR", True, "SegWit v1 (Taproot), tweaked x-only public key of 32 bytes sits
there as is"
    if n == 34 and spk[:2] == b"\x52\x20":
        return "P2MR (proposed, BIP-360 Draft)", False, "proposed SegWit v2 — only the Merkle
root of the script tree, 32 bytes, no key path"
    if n >= 3 and 0x51 <= spk[0] <= 0x60 and spk[-1] == 0xAE:
        return "P2MS (bare multisig)", True, "OP_m <keys> OP_n OP_CHECKMULTISIG — the public
keys sit there as is"
    return "nonstandard/unclassified", None, "this classifier knows only the standard types"

samples = [
    ("genesis output (real bytes)", GENESIS_SPK),
    ("P2PKH shape (fake)",  b"\x76\xa9\x14" + bytes(20) + b"\x88\xac"),
    ("P2SH shape (fake)",   b"\xa9\x14" + bytes(20) + b"\x87"),
    ("P2WPKH shape (fake)", b"\x00\x14" + bytes(20)),
    ("P2WSH shape (fake)",  b"\x00\x20" + bytes(32)),
    ("P2TR shape (fake)",   b"\x51\x20" + bytes(32)),
    ("P2MR shape (fake)",   b"\x52\x20" + bytes(32)),
    ("P2MS 1-of-2 shape (fake)", b"\x51" + b"\x21" + bytes(33) + b"\x21" + bytes(33) +
b"\x52\xae"),
]
print(f"{'Sample':<28}{'Bytes':>6}  {'Type':<32}{'Key exposed on receipt':<24}Note")
for name, spk in samples:
    kind, exposed, why = classify(spk)
    flag = {True: "yes", False: "no*", None: "?"}[exposed]
    print(f"{name:<28}{len(spk):>6}  {kind:<32}{flag:<24}{why}")
print()
print("* no = safe as long as it only receives. Spending reveals the key or script, so reusing
the same address makes it vulnerable (the No* note in the BIP-360 table).")
print("Leading byte of the public key in the genesis output:", GENESIS_SPK[1:2].hex(), "(04 =
uncompressed, 65 bytes) — written on the chain as is since 2009.")
```

```
# Output — bcbook (Python 3.13), exactly as run
Sample                        Bytes  Type                              Key exposed on receipt  Note
genesis output (real bytes)      67  P2PK                              yes
public key (65 or 33 bytes) sits in the script as is
P2PKH shape (fake)               25  P2PKH                             no*                     only
the HASH160 of the public key (20 bytes) — key revealed when spent, vulnerable if reused
P2SH shape (fake)                23  P2SH                              no*                     only
the HASH160 of the script — script revealed when spent
P2WPKH shape (fake)              22  P2WPKH                            no*
SegWit v0, key hash of 20 bytes
P2WSH shape (fake)               34  P2WSH                             no*
SegWit v0, script hash of 32 bytes
P2TR shape (fake)                34  P2TR                              yes
SegWit v1 (Taproot), tweaked x-only public key of 32 bytes sits there as is
P2MR shape (fake)                34  P2MR (proposed, BIP-360 Draft)   no*
proposed SegWit v2 — only the Merkle root of the script tree, 32 bytes, no key path
P2MS 1-of-2 shape (fake)         71  P2MS (bare multisig)              yes                     OP_m
<keys> OP_n OP_CHECKMULTISIG — the public keys sit there as is

* no = safe as long as it only receives. Spending reveals the key or script, so reusing the same
address makes it vulnerable (the No* note in the BIP-360 table).
Leading byte of the public key in the genesis output: 04 (04 = uncompressed, 65 bytes) — written
on the chain as is since 2009.
```

"Exposed" and "stolen" are not the same. Aggarwal et al. separated three situations: address reuse (an exposed key locks a balance: "completely insecure"), processed transactions (a transaction spent with blocks stacked after it; even if the private key is extracted, the double-spending attack of Chapter 9 would also have to be carried out, so it is "reasonably secure against quantum attacks"), and unprocessed transactions (seeing the public key in the mempool, extracting the private key before the transaction enters a block, and getting a different transaction in first: "We view item (3) as the most serious attack").[26] For the third to work, the "target time" of Section 2 has to fit within the block interval, and Babbush et al., applying their own estimate (about 9 minutes) to Bitcoin's 10-minute average interval, put the probability of success at "slightly less than 41%" (under "idealized assumptions"), writing that the probability drops sharply on chains with shorter block intervals.[27] That figure is the output of a model, not an observation. What this book takes from it is the shape: **coins whose public keys are revealed open when the machine arrives, and coins whose keys are not revealed are at risk during the few minutes of spending.** So the transition proposals go two ways: "outputs that do not reveal" and "signatures that are safe even when revealed."

4. Transition proposals: output types, signatures, sunset

4.1 BIP-360: Taproot with the key path removed

BIP-360 proposes adding a new output type, "Pay-to-Merkle-Root (P2MR)," by soft fork. As of 2026-10 its status is **Draft** (version 0.12.1), and it has not been activated.[28] It does one thing: "P2MR outputs operate with nearly the same functionality as P2TR (Pay-to-Taproot) outputs, but with the key path spend removed." Where a Taproot output carried a tweaked public key, a P2MR output carries only **the Merkle root of a script tree** (Chapter 6). A public key may be inside a script, but that script stays hidden behind the tree until it is spent, and so exposure at the moment of receipt disappears.[29] It uses SegWit version 2, and its addresses begin with `bc1z`. The proposers state clearly what this BIP **does not do**: "proposed P2MR outputs are only resistant to 'long exposure attacks' on elliptic curve cryptography"; protection against short exposure in the mempool "may require the introduction of post-quantum signatures in Bitcoin"; and "this proposal does not include the introduction of post-quantum signature schemes."[30] The cost is small, too. A P2MR witness for a depth-1 tree is 135 bytes, larger than the 66 bytes of a Taproot key-path spend but 32 bytes smaller than the equivalent Taproot script-path spend.[31] And in its conclusion it says that "This BIP does not take a position on any specific quantum computing timeline ...."[32]

4.2 Post-quantum signatures: three standards and the problem of size

The exit the asymmetry of Section 1 points to is "a signature that changes the problem." On August 13, 2024, NIST published three standards: FIPS 203 (ML-KEM, lattice-based key encapsulation), FIPS 204 (ML-DSA, lattice-based signatures) and FIPS 205 (SLH-DSA, hash-based signatures).[33] The last two are the signatures. FIPS 204 says that "ML-DSA is believed to be secure, even against adversaries in possession of a large-scale quantum computer," and FIPS 205 that SLH-DSA "is designed to provide resistance against attacks from a large-scale quantum computer." The problem in putting these into Bitcoin and Ethereum is not security but **size**. As Aggarwal et al. already wrote in 2017, "In the blockchain context the most important parameters of a signature scheme are the signature and public key lengths, as these must be stored in some capacity to fully verify transactions ...."[34] In a table:

SCHEME	PUBLIC KEY	SIGNATURE	SOURCE
Schnorr (secp256k1, current Bitcoin)	32 bytes	64 bytes	BIP-340 (Chapter 5); "signature (1 + 64 bytes)" in the witness calculation of BIP-360
ML-DSA-44	1,312 bytes	2,420 bytes	FIPS 204 Table 2
ML-DSA-65	1,952 bytes	3,309 bytes	FIPS 204 Table 2
ML-DSA-87	2,592 bytes	4,627 bytes	FIPS 204 Table 2
SLH-DSA-128s (SHA2, SHAKE)	32 bytes	7,856 bytes	FIPS 205 Table 2
SLH-DSA-128f	32 bytes	17,088 bytes	FIPS 205 Table 2
SLH-DSA-256s	64 bytes	29,792 bytes	FIPS 205 Table 2

Lattice-based signatures are **tens of times** larger than the current signature, and hash-based signatures **more than a hundred times** larger. Chapter 18 showed that block space is a finite resource, and Chapter 21 that a price is put on it. If signatures grow 40-fold, the number of transactions that fit in a block falls by as much, and who bears that cost, and how, is a design question. Aggarwal et al. wrote that hash-based schemes have "the advantage of having provable security, at least assuming the chosen hash function behaves like a random oracle," but that because of Grover their quantum security level is half the classical level, and that at the same level of quantum security lattice-based schemes "have some advantage in signature plus public key length." The choice is still open, and BIP-360, too, says only that "Quantum-resistant signature algorithms (e.g. ML-DSA or SLH-DSA) … should be scrutinized before use."

4.3 BIP-361: the sunset of the old signatures

If BIP-360 is a proposal to **add** a new output type, BIP-361 is a proposal to **withdraw** the old signatures. As of 2026-10 it is a Draft, its type is Informational, and it carries "Requires: TBD Post Quantum Signature BIP": it presupposes a post-quantum signature BIP that does not yet exist.[35] There are two phases: "Phase A: Disallows sending of any funds to quantum-vulnerable addresses, hastening the adoption of PQ address types. Phase B: Restricts ECDSA/Schnorr spends by encumbering them with a quantum-safe rescue protocol, preventing theft of funds in quantum-vulnerable UTXOs. This is triggered by a well-publicized flag-day five years after activation." The table in the specification gives Phase A as "160,000 blocks (~3 years) after BIP-361 activation" and Phase B as "2 years after Phase A activation."[36] A "rescue protocol" is a device that has the spender prove knowledge that a quantum attacker cannot have and that only the true owner has, such as the parent secret key of a BIP-32 wallet, and the proposers write that "If one or more rescue protocols can be designed to cover the majority of the Bitcoin supply, then restricting ECDSA/Schnorr verification will be at most **mildly confiscatory** …."[37] They take the view that P2PK has no such asymmetry of knowledge, so that no rescue protocol can be built for it, and state that for those coins they support a separate proposal (Hourglass, Section 5).

To put the character of this proposal in one line, **it is a timetable that presupposes a signature BIP that does not yet exist**. For that timetable to be adopted by "consensus," a majority of nodes and miners has to run the new rules, as Chapters 7 and 23 showed, and what that majority does in Phase B is to change **after the fact** the conditions for spending particular UTXOs. The question box in Chapter 20 asked whether the rights of holders are infringed if a majority changes the issuance rule. What changes here is not the issuance rule but **spendability itself**. Section 5 goes on to that question.

4.4 Ethereum: four areas and account abstraction

Ethereum's exposure is structured differently from Bitcoin's. In the account model of Chapter 11, any account that has sent even one transaction has revealed its public key, and continuing to use an account is the basic pattern of use. On top of that, the validator signatures (BLS) used by the proof of stake of Chapter 10 and the KZG commitments used for the data availability of Chapter 18 both rely on elliptic curve pairings. The roadmap security page of the official Ethereum site (source commit 290d753c, 2026-07-12) lists four areas: validator consensus signatures (BLS), the commitments for data availability (KZG), account signatures (ECDSA), and the zero-knowledge proof systems of rollups.[38] The sentence in bold on the same page is "**No quantum computer can break Ethereum's cryptography today.**" The same page cites the March 2026 estimate of 1,200 logical qubits (Section 2) as "about 20 times fewer than earlier estimates" (the "20-fold" in Babbush et al. is on the side of physical qubits, under 500,000, while logical qubits fell by about 1.9 times, as the table in Section 2 shows). A subpage (quantum-resistance, commit 643a7a45, 2026-10-23) says that "Current quantum hardware is far from this scale, operating with a few thousand noisy physical qubits" (as of 2026-10, Appendix A).[39]

The shape of the response is **the reverse** of Bitcoin's. Where the Bitcoin proposals are protocol changes that "add one output type and withdraw the old signatures," Ethereum intends to go for **per-account choice** in account signatures (a plan, as of 2026-10): "Rather than a single protocol-wide migration, Ethereum plans to use account abstraction (specifically EIP-8141, being considered for Hegotá in second half of 2026) to give users **signature agility**. Individual accounts could switch to a post-quantum signature scheme without waiting for the entire protocol to change."[40] The higher-level roadmap page on the same site (2026-07-12 source) gives Hegotá a target of 2027, so the years on the two official pages differ. This book does not pick one but records both as they are (Appendix A-29). For validator signatures, the design on the official page is to switch to hash-based signatures (leanXMSS) and to solve, by **aggregating** them with zero-knowledge proofs, the problem that those signatures grow to "roughly 3,000 bytes" against the 96 bytes of BLS (a plan, as of 2026-10).[41] The Foundation's post-quantum page (updated 2026-06-27) says that "We do not believe a cryptographically relevant quantum computer is imminent," while also saying that "Based on our team's current assessment, L1 protocol upgrades could be completed by 2029 ..." (a planning target as of 2026-10, Appendix A).[42] And on dormant assets, the problem of the next section, it says that "Unlike Bitcoin, where roughly 5% of the supply is associated with early address formats widely believed to be abandoned ..., Ethereum's equivalent long-dormant exposure is closer to 0.1% (an EF estimate)"; that two scenarios exist, "do nothing, or freeze vulnerable coins"; and that the matter is "ultimately a community governance question."[43]

5. Vulnerable coins: three paths and one question

However well the transition goes, **coins that cannot move** remain: coins whose keys were lost, coins whose owners have died, and, among the roughly 1.7 million BTC in P2PK of Section 3, those that have not moved once since 2009. Babbush et al. call this the problem of "dormant digital assets" and record two facts: that "a majority of the 1.7 millions [sic] bitcoin have not been transacted since 2009," and that "accounts believed to have been owned by Satoshi Nakamoto hold about 1 million bitcoin ... behind these P2PK scripts." The latter is an **attribution estimate**, and this book takes no part in that attribution.[44] These coins cannot be moved by a software update, because no one has the keys. So when the machine arrives, one of two things happens: someone takes them, or the rules are changed so that no one can take them.

The discussion on the Bitcoin side has been sorted into three paths. Here is Babbush et al.'s summary as it stands: "**Do Nothing**: the protocol is kept unchanged allowing quantum attackers to acquire the dormant assets. **Burn**: the protocol is modified to render dormant assets unspendable after a certain date, effectively confiscating all assets locked by P2PK scripts. **Hourglass** [303]: the protocol is modified to limit the rate at which dormant assets may be spent."[45] The authors of BIP-361 write the same three paths as "Allow anyone to steal vulnerable coins," "Allow throttled theft of coins" and "Allow no one to steal vulnerable coins," and classify the first two as proposals that create a "redistribution dilemma."[46] Babbush et al. record that in an informal poll at a 2025 meeting the three paths received "roughly equal support," and write: "Consensus remains elusive."[47] Burn and Hourglass are technically possible as soft forks, because forbidding or limiting the spending of particular UTXOs **narrows** the rules (Chapters 7 and 23). Doing nothing involves no change of rules. What is hard is not the technology.

It is worth writing down what each of the three paths **presupposes**. "Do nothing" keeps Bitcoin's model of ownership, under which whoever knows the key is the owner, even after the machine arrives (Babbush et al. write that "a CRQC can quickly derive a private key from any public key, thus breaking the foundation of Bitcoin's model of ownership"[48]); "burn" **suspends** by majority vote the model under which whoever knows the key is the owner; and "Hourglass" regulates the speed in between. If the choice is to change the rules, the deciding body is **a majority of nodes and miners**, and whose agent that majority is, the box below asks. The Babbush et al. white paper goes further and lists options on the government side: "digital salvage" of abandoned property, the poor fit of the US Revised Uniform Unclaimed Property Act (RUUPA 2016) and its "holder" requirement, the doctrine of laches (sleeping on one's rights), national security responses, and dialogue with the community. It also argues that "Regulated Destruction of Dormant Assets is Not Viable Policy" (to erase blocks, the whole chain would have to be mined again, and forcing miners to censor transactions would send mining abroad or lead to a hard fork).[49] This book only **presents** that argument. There are two reasons for not evaluating it: the statutes and judgments that deal with this problem are not among this book's source materials (28.1(g)), and the white paper itself is a document from institutions with an interest in the matter.

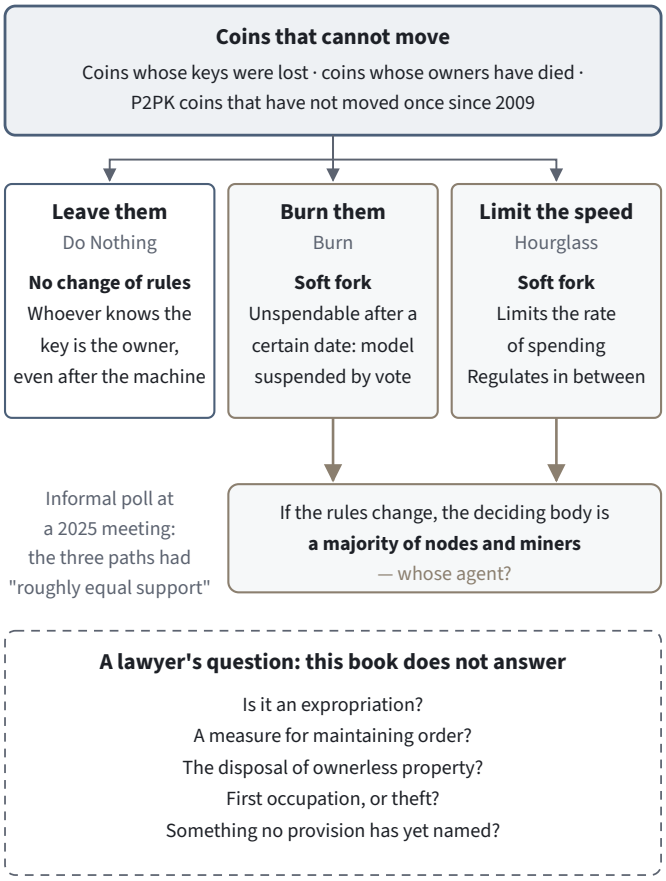


Figure 29-4. The three paths discussed on the Bitcoin side for coins that cannot move, and what each presupposes. For the two paths that change the rules, the deciding body is a majority of nodes and miners. The questions at the bottom are the wording of the question box and are not paired with any path

> **? A LAWYER'S QUESTION**
>
> Suppose that one day a majority of nodes and miners introduces by soft fork the rule "old outputs whose public keys have been revealed can no longer be spent" (Phase B of 4.3, or "Burn"). At that moment the coins of the minority of holders who never lost their keys are locked as well. Is this **an expropriation**? If so, who is the expropriating party, and who pays compensation? Is it **a measure for maintaining order**, since it stops anyone from taking property that someone would take anyway once the machine arrives? Or is it **the disposal of ownerless property**, property whose key no one knows any longer? If so, what becomes of the minority who still know their keys? Does the character change if "Hourglass" only limits the speed? Conversely, if things are "left as they are" and whoever has a quantum computer takes first, is that acquisition first occupation, theft, or something that no provision of any jurisdiction has yet named? And is **the majority that decides** all this, the "participants who follow the rules" of Chapter 7, the agent of the coin holders, of the network, or of itself? What the box in Chapter 20 asked about the issuance rule, this chapter asks again about spendability. This book does not answer, and hands the question to Part 5. The answer to this question differs depending on which of the fourteen positions in 27.2 (factual-state theory, partnership-right theory, control-based, and others) one adopts, and 28.1(g) records that primary materials touching this question are not among this book's source materials. → Chapter 27 · Chapter 28

6. "As of 2026-10": what in this chapter will date

Following the update rule in the notes to the reader in Chapter 3, this section collects the sentences in this chapter that describe the current state of affairs. All of them are entered in Appendix A, and when the edition changes, only that appendix is corrected.

- BIP-360 is a Draft (version 0.12.1), BIP-361 is a Draft, and BIP-54 is Complete but has no deployment section and so is not described as activated: as of 2026-10 (Appendix A-27).
- The post-quantum signature transition proposals this chapter examined (BIP-360 and 361, EIP-8141 and leanXMSS) have not been activated on the mainnet of either Bitcoin or Ethereum (they are at the proposal or planning stage): as of 2026-10 (Appendices A-27 and A-29). This does not mean that properties already in place, such as hash-based protection, are absent.
- Hardware: the official Ethereum page says "a few thousand noisy physical qubits," and the estimate in the 2026 white paper is "fewer than half a million": as of 2026-10 (Appendix A-28). The figures from the survey of 26 experts (published March 2026) are likewise kept only in Appendix A.
- Ethereum roadmap: EIP-8141 is "being considered" for the Hegotá upgrade, the target for the core post-quantum milestones is "approximately 2029," and "These are planning targets and may shift": as of 2026-10 (Appendix A-29).
- Exposed amounts: the table in Section 3 is a secondary tally of block 972,132 of 2026-10-

19, and there is no primary reproduction: as of 2026-10 (Appendix A-30).

- The schedule for retiring ECDSA: the Ethereum page says, without naming the document, that NIST expects deprecation in 2030 and a ban in 2035, and BIP-360 gives 2030 as the CNSA 2.0 schedule and 2035 as that of NIST IR 8547. This book did not open the text of NIST IR 8547: a task (Appendix A-31).

7. Summary

A quantum computer affects the two tools of Part 1 differently. Shor's algorithm brings the discrete logarithm problem underlying signatures down to polynomial time, and Grover's algorithm speeds up the preimage search of hashes only by a square root. So the threat lies not in "Bitcoin" but in **signatures whose public keys are revealed**, and proof of work stands up to the speed and parallelism of ASICs. The size of that machine has come down from 2,330 to around 1,200 logical qubits through advances in algorithms and error-correction theory, and estimates of physical qubits range from hundreds of thousands to the billions depending on five assumptions (error rate, code cycle, reaction time, connectivity, target time), so they can be compared only under the same assumptions. Revealed public keys are in three places (script type, reuse, the mempool), and the secondary tally on the reference date counts about one third of issuance as exposed (P2PK at about 1.7 million BTC is the firmest number, the one on which the sources that tallied directly converge). The transition proposals divide into outputs that do not reveal (BIP-360), signatures that are safe even when revealed (FIPS 204 and 205, with signatures tens to hundreds of times larger), the sunset of the old signatures (BIP-361) and per-account choice (Ethereum), and as of 2026-10 none of them has been activated. And coins that cannot move remain. Leave them, burn them, or limit their speed: all three paths are technically possible, and technology decides none of them. That fact is what lawyers take from this chapter. On the day the model under which whoever knows the key is the owner breaks, which law is to name **the character of the decision that suspends that model** and **the status of the majority that decides**, and how, is an empty place. The next chapter, Chapter 30, looks through the text of the provisions at that "naming" itself: five jurisdictions calling the same bit string by five names.

Sources for this chapter

- NIST, *Report on Post-Quantum Cryptography*, NISTIR 8105 (April 2016), DOI 10.6028/NIST.IR.8105 — Table 1 (p. 2), statements on Shor and Grover (pp. 1, 3), hash-based signatures (pp. 4–5).
- Divesh Aggarwal, Gavin K. Brennen, Troy Lee, Miklos Santha & Marco Tomamichel, "Quantum Attacks on Bitcoin, and How to Protect Against Them," *Ledger* 3 (2018), DOI 10.5195/ledger.2018.127 — stored copy arXiv:1710.10377v1: abstract (p. 1), mining attack (pp. 4–8), three situations of signature attack and resources (pp. 8–9), Momentum (pp. 11–12), comparison of signature schemes, Table II (p. 13). Page numbers follow the arXiv version.
- Martin Roetteler, Michael Naehrig, Krysta M. Svore & Kristin Lauter, "Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms," *ASIACRYPT 2017*, LNCS, pp.

241–270, DOI 10.1007/978-3-319-70697-9_9 — stored copy arXiv:1706.06752v3: general formula (p. 1), statement on Shor (p. 2), P-256 values (p. 4).

- Mark Webber, Vincent Elfving, Sebastian Weidt & Winfried K. Hensinger, "The Impact of Hardware Specifications on Reaching Quantum Advantage in the Fault Tolerant Regime," *AVS Quantum Science* 4(1): 013801 (2022), DOI 10.1116/5.0073075 — stored copy arXiv:2108.12371v3: abstract (p. 1), section on Bitcoin (p. 13).
- Ryan Babbush et al., "Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations," arXiv:2603.28846v2 (2026-04-15), preprint — abstract, pp. 4 and 8 (resources and time), pp. 10–11 (connectivity, scenarios, threshold model), pp. 12–15 (exposure, attack mechanisms), pp. 19–20 (xpubs, proof of work), pp. 36–42 (dormant assets, policy options). Includes authors affiliated with Google Quantum AI and the Ethereum Foundation.
- NIST FIPS 203, 204 and 205 (Published August 13, 2024) — abstracts; FIPS 204 Table 2 (p. 16); FIPS 205 §11, Table 2. `nist-2024-fips-204-module-lattice-signature.pdf`, `nist-2024-fips-205-stateless-hash-signature.pdf`.
- BIP-360 "Pay-to-Merkle-Root (P2MR)" (Draft, v0.12.1), BIP-361 "Post Quantum Migration and Legacy Signature Sunset" (Draft) — github.com/bitcoin/bips commit 100ba872c808946155b1eeb8c4568058f2eea0a7 (`bip-0360.mediawiki`, `bip-0361.mediawiki`, `README.mediawiki` L1286–1298), accessed 2026-10-25 and checked word for word. The status of BIP-54 is from commit 927b6de9915c9262615a6399de51b200f81e5aa4, `bip-0054.md` (Appendix A-3).
- ethereum.org, "A more secure Ethereum" (source commit 290d753c, 2026-07-12) and "Post-quantum cryptography on Ethereum" (source commit 643a7a45, 2026-10-23) — accessed 2026-10-25. Ethereum Foundation Post-Quantum team, https://pq.ethereum.org/ (Last updated June 27, 2026) — accessed 2026-10-25.
- The data bundled with this book, `data/exposed_output_types_2026-10-25.json` (reference date 2026-10-25; primary source the Project Eleven tally at block 972,132, comparison source the mempool research UTXO Set Report at block 892,385) — a secondary tally (🟡); sources and limits in `data/README.md`.
- Secondary sources (industry, 🟡): Milton & Shikhelman, *Bitcoin and Quantum Computing* (Chaincode Labs, May 2025); Deloitte, "Quantum computers and the Bitcoin blockchain" (date not stated); CoinShares, "Quantum Vulnerability in Bitcoin: A Manageable Risk" (2026-02-06). Accessed 2026-10-25. Dollar conversions and statements about markets were not carried over.
- Code execution: Python 3.13 (conda environment `bcbook`), using only the standard-library modules `math`, `json` and `decimal`. NB29-2 reads the bundled `data/exposed_output_types_2026-10-25.json` and, if the file is missing, uses constants with the same values. Notebook NB29.
- This chapter used no price, market-capitalization or rate-of-return data and contains no statement of the kind "because of the quantum threat, the price ...". It gives no date for the appearance of quantum computers. This book did not open Shor's (1994/1997) or Grover's (1996) original papers, Häner et al. (2020), NIST IR 8547, or the text of the US Revised Uniform Unclaimed Property Act.

Notes

1. Lily Chen, Stephen Jordan, Yi-Kai Liu, Dustin Moody, Rene Peralta, Ray Perlner & Daniel Smith-Tone, *Report on Post-Quantum Cryptography*, NISTIR 8105 (April 2016), p. 2, Table 1, "Impact of Quantum Computing on Common Cryptographic Algorithms." The rows in the original: "AES — Symmetric key — Encryption — Larger key sizes needed / SHA-2, SHA-3 — Hash functions — Larger output needed / RSA — Public key — Signatures, key establishment — No longer secure / ECDSA, ECDH (Elliptic Curve Cryptography) — Public key — Signatures, key exchange — No longer secure / DSA (Finite Field Cryptography) — Public key — Signatures, key exchange — No longer secure".

2. NISTIR 8105, p. 1: "In 1994, Peter Shor of Bell Laboratories showed that quantum computers, a new technology leveraging the physical properties of matter and energy to perform calculations, can efficiently solve each of these problems, thereby rendering all public key cryptosystems based on such assumptions impotent [1]." Reference [1] of the report is P. Shor, "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," *SIAM J. Comput.* 26(5): 1484–1509 (1997). This book did not open Shor's original and uses it only through the citations in NIST and in Aggarwal et al. (below).

3. NISTIR 8105, p. 1: "In particular, Grover's search algorithm proffers a quadratic speedup on unstructured search problems. While such a speedup does not render cryptographic technologies obsolete, it can have the effect of requiring larger key sizes, even in the symmetric key case." P. 3: "We don't know that Grover's algorithm will ever be practically relevant, but if it is, doubling the key size will be sufficient to preserve security. Furthermore, it has been shown that an exponential speed up for search algorithms is impossible, suggesting that symmetric algorithms and hash functions should be usable in a quantum era [3]."

4. Divesh Aggarwal, Gavin K. Brennen, Troy Lee, Miklos Santha & Marco Tomamichel, "Quantum Attacks on Bitcoin, and How to Protect Against Them," *Ledger* 3 (2018), DOI 10.5195/ledger.2018.127; the copy stored for this book is arXiv:1710.10377v1 (2017-10-28), p. 1, abstract. Page numbers follow the arXiv version.

5. Ryan Babbush, Adam Zalcman, Craig Gidney, Michael Broughton, Tanuj Khattar, Hartmut Neven, Thiago Bergamaschi, Justin Drake & Dan Boneh, "Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations," arXiv:2603.28846v2 (2026-04-15), p. 20, §III.D: "However, we maintain that this is not something to worry about in the next several decades. Grover-based attacks on Bitcoin mining are not practically relevant for two reasons. First, the quadratic quantum speedup from Grover's algorithm is all but consumed by the overheads of quantum error correction [154]. Second, Grover's algorithm does not parallelize well [155]. In Bitcoin mining, hardware acceleration and massive parallelization are a much greater advantage than the modest speedup from Grover's algorithm." The authors' affiliations include Google Quantum AI and the Ethereum Foundation (Drake); this is noted because the document comes from institutions with an interest in the matter. It is a preprint, with no publication details.

6. Aggarwal et al. (2018), pp. 6–7, §III.A: "The number of logical qubits needed in the Grover algorithm is fixed at 2402, independent of the difficulty." / "Assuming maximum gate speeds attainable on current devices of s = 66.7MHz [KKL+17] and assuming an experimentally challenging, but not implausible, physical gate error rate of pg = 5×10−4, and close to current difficulty D = 10^12, the overheads are cτ = 538.6 and cnQ = 1810.7, implying an effective hash rate of hQC = 13.8GH/s using nQ = 4.4 × 10^6 physical qubits. This is more than one thousand times slower than off the shelf ASIC devices which achieve hash rates of 14TH/s ...". These are 2017 assumptions about difficulty and differ from the difficulty on the reference date (about 1.4×10^{14}, Chapter 8 and Appendix A-4).

7. Babbush et al. (2026), p. 20: "Under the fantastical assumption that a CRQC could compute SHA-256 in a single 1 microsecond error correction cycle, a quantum miner could achieve a hashrate of 0.25 TH/s which is more than two orders of magnitude below 110 ± 3% TH/s of the popular ASIC miner S19 Pro [156]. Under realistic assumptions, rather than fantastical ones, quantum miner's hashrate plummets by over 10 orders of magnitude [157]." The same page: "But for now, quantum mining remains science fiction more than a concrete threat."

8. Martin Roetteler, Michael Naehrig, Krysta M. Svore & Kristin Lauter, "Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms," *ASIACRYPT 2017*, LNCS, pp. 241–270, DOI 10.1007/978-3-319-70697-9_9; the copy stored for this book is arXiv:1706.06752v3 (2017-10-31), p. 2: "In his famous paper [44], Peter Shor presented two polynomial-time quantum algorithms, one for integer factorization and another

one for computing discrete logarithms in a finite field of prime order. Shor notes that the latter algorithm can be generalized to other fields. It also generalizes to the case of elliptic curves. Hence, given the prerequisite that a large enough general purpose quantum computer can be built, the algorithms in Shor's paper completely break all current crypto systems based on the difficulty of factoring or computing discrete logarithms." Page numbers follow the arXiv version.

9. Aggarwal et al. (2018), pp. 11–12, §IV.A: "As the classical running time is m|S|, we see that unlike for the current proof of work in Bitcoin, with this proposal a quantum computer would not be able to achieve a quadratic advantage as soon as S is more than constant size. In particular, since √m|S|^{2/3} is minimized also when S = √(2^{n+ℓ}/t), the running time of even the fastest quantum algorithm is at least T^{2/3}, which is substantially larger than T^{1/2}."

10. Roetteler et al. (2017), p. 1 (abstract) and p. 4: "We conclude that elliptic curve discrete logarithms on an elliptic curve defined over an n-bit prime field can be computed on a quantum computer with at most 9n + 2⌈log2(n)⌉+ 10 qubits using a quantum circuit of at most 448n^3 log2(n) + 4090n^3 Toffoli gates." / "For example, our simulation of the point addition quantum circuit for the NIST standardized curve P-256 needs 2330 logical qubits and the full Shor algorithm would need about 1.26 · 10^11 Toffoli gates." The curve simulated is the NIST curve; secp256k1 is mentioned in the introduction only as a place of use ("the Bitcoin curve secp256k1 from the SEC2 [8] standard," p. 1). It is the same kind of 256-bit prime-field curve, so the scale is the same.

11. Aggarwal et al. (2018), pp. 8–9, §III.B: "Bitcoin uses n = 256 bit signatures so the number of Toffoli gates is 1.28 × 10^11, which can be slightly parallelized to depth 1.16 × 10^11." / "nQ = 2334 × cnQ(pg), where the first factor is the number of logical qubits including 4 logical ancilla qubits" / "Using a surface code with a physical gate error rate of pg = 5 × 10−4, the overhead factors are cτ = 291.7 and cnQ = 735.3, and the time to solve the problem at 66.6 MHz clock speed is 6.49 days using 1.7 × 10^6 physical qubits. Looking forward to performance improvements, for 10GHz clock speed and error rate of 10−5, the signature is cracked in 30 minutes using 485550 qubits."

12. Mark Webber, Vincent Elfving, Sebastian Weidt & Winfried K. Hensinger, "The Impact of Hardware Specifications on Reaching Quantum Advantage in the Fault Tolerant Regime," *AVS Quantum Science* 4(1): 013801 (2022), DOI 10.1116/5.0073075; the copy stored for this book is arXiv:2108.12371v3 (2021-11-11), p. 1 (abstract) and p. 13: "It would require approximately 317 million physical qubits to break the encryption within one hour using the surface code, a code cycle time of 1 μs, a reaction time of 10 μs, and physical gate error of 10−3. To break the encryption instead within one day it would require 13 million physical qubits." / "To break it within 10 minutes with the same code cycle time it would require 1.9 billion physical qubits whereas to break it in 1 day would require only 13 million physical qubits." The input circuit is "the depth optimized approach provided by Haner et al[48] for 256 bit encryption, which corresponds to 5.76 · 10^9 T gates, 2871 logical qubits" (p. 13); the original of that earlier paper is not among this book's source materials.

13. Babbush et al. (2026), p. 1 (abstract), p. 4 and p. 8: "We demonstrate that Shor's algorithm for this problem can execute with either ≤1200 logical qubits and ≤90 million Toffoli gates or ≤1450 logical qubits and ≤70 million Toffoli gates. In the interest of responsible disclosure, we use a zero-knowledge proof to validate these results without disclosing attack vectors. On superconducting architectures with 10−3 physical error rates and planar connectivity, those circuits can execute in minutes using fewer than half a million physical qubits." / P. 4: "(nearly a 20 fold reduction over prior estimates [44])" / P. 8: "If we assume a control system reaction time of 10 microseconds (a standard figure for superconducting qubits [100]), and a 50% overhead per Toffoli gate, then 70 or 90 million Toffoli gates can be resolved in 18 or 23 minutes on a superconducting CRQC — close to the 10 minute average Bitcoin block time." / "we should estimate the time required to launch an on-spend attack starting from this primed state at the moment the public key is learned to be roughly either 9 minutes or 12 minutes."

14. Babbush et al. (2026), p. 11, §II.C: "By contrast, quantum computing, with its broad variety of hardware platforms, is still in the "era of ferment" where simple models, such as counting physical qubits, fail to adequately capture technological progress. Instead, progress comes in discrete jumps corresponding to development of new internal capabilities and overcoming scaling challenges, e.g., by getting device error rates below the threshold for an error-correcting code [125] or implementing coherent interconnects for modular architectures [126–128]. Therefore, progress in quantum computing is better understood using a threshold model rather than in terms of the number of physical qubits."

15. Babbush et al. (2026), pp. 10–11, §II.C: "Scenario 1: On-spend attacks become viable about the same time as at-

rest attacks. This is a possibility if the first CRQCs are fast-clock devices. / Scenario 2: At-rest attacks become viable meaningfully sooner than on-spend attacks. This is a possibility if slow-clock quantum platforms advance more rapidly than fast-clock ones." P. 4: "We delineate how superconducting, silicon, and photonic platforms, with their fast gates and short error correction cycle times, pose a distinct threat to active transactions that slower architectures like neutral atoms and ion traps likely cannot match."

16. Babbush et al. (2026), p. 10: "More aggressive hardware assumptions — such as the "bicycle" architecture used for 2-gross qLDPC codes [115, 117] — could drop qubit counts closer to one hundred thousand physical qubits, but this approach requires non-local degree-seven connectivity that has yet to be demonstrated in actual superconducting qubit devices." / "Consequently, while a Pinnacle-style device would theoretically require even fewer than one-hundred thousand superconducting qubits to break 256-bit ECDLP, a planar degree-four device with half a million qubits remains the better understood and also likely more feasible engineering path."

17. BIP-360 "Pay-to-Merkle-Root (P2MR)" (Hunter Beast, Ethan Heilman, Isabel Foxen Duke; Status: Draft; Version 0.12.1), github.com/bitcoin/bips commit 100ba872c808946155b1eeb8c4568058f2eea0a7, `bip-0360.mediawiki` (accessed 2026-10-25), section "Long Exposure vs Short Exposure Attacks": "A long exposure attack is an attack performed on exposed blockchain data, such as exposed public keys, or the scripts of spent outputs. These are likely to be the earliest quantum attacks made possible on Bitcoin, because attackers will have ample time — as much time as vulnerable keys are exposed — to carry out quantum key recovery. / Short exposure attacks, however, require faster quantum computers, because they must occur within the relatively short time that a transaction is unconfirmed in the mempool." The note to the "No*" entries in the table: "Funds in P2PKH, P2SH, P2WPKH, P2WSH, and P2MR outputs can become vulnerable to long exposure quantum attacks anytime their script reveals a public key." And: "Note: Extended public keys, commonly known as "xpubs," and wallet descriptors also reveal quantum vulnerable public key information."

18. Babbush et al. (2026), p. 12, §III.A; p. 17, §III.B(4), Offchain Exposure; p. 19: "Since the public key is disclosed directly in the locking script, it remains visible to everyone from the moment the bitcoin is received. Therefore, bitcoin locked by a P2PK script is vulnerable to both at-rest and on-spend attacks." / "However, spending such bitcoin does entail disclosure of the public key in the unlocking script, so it remains vulnerable to on-spend attacks." / "In the absence of CRQCs, it is safe to share extended public keys and use normal derivation to obtain keys for cryptocurrency transactions. ... Therefore, in the presence of CRQCs, reuse or disclosure of an extended public key constitutes a pernicious form of Offchain Exposure vulnerability that enables an attacker with a CRQC to compromise multiple private keys at the cost of one."

19. Babbush et al. (2026), p. 13: "However, P2TR stores the tweaked public key directly in the locking script without hiding it behind a hash value which brings back the quantum vulnerability of P2PK and P2MS addresses and therefore, from the standpoint of quantum security, constitutes a security regression [30, 137]."

20. This book's `data/exposed_output_types_2026-10-25.json` (reference date 2026-10-25, accessed 2026-10-24T18:19:27Z). Primary source: Project Eleven Bitcoin Risq List metrics endpoint (snapshot block 972132, produced 2026-10-19T10:28:55Z); the method is public SQL (github.com/p-11/bitcoin-risq-list-query commit bb3177f8, `btc_at_quantum_risk_query.sql`): "(1) outputs whose script type puts a key in the locking script (P2PK, bare multisig, P2TR); (2) unspent balances of P2PKH/P2WPKH/P2SH/P2WSH addresses that have previously been spent from (key or script revealed)". The denominator is the scheduled issuance of 20,100,415.625 BTC, computed with the `GetBlockSubsidy` rule of Bitcoin Core. The exchange-rate field in the response was not carried over (the notes to the reader in Chapter 3). Limits (`data/README.md`): it is a third-party tally and was not recomputed from a node; the P2SH and P2WSH reuse categories count an output as exposed even when the published script contains no key; off-chain exposure and exposure while in the mempool are not counted; the denominator is an upper bound on the circulating supply.

21. mempool research, "UTXO Set Report" (orangesurf, 2025-05-18), https://research.mempool.space/utxo-set-report/ (accessed 2026-10-24); the chainstate at block 892385 (2025-04-14) classified with `bitcoin-utxo-dump`. Values carried into the `comparison` entry of this book's `data/exposed_output_types_2026-10-25.json`: p2pk count 45,257 (0.03%), value 1,720,270 BTC (8.70%); p2tr count 59,262,426 (34.22%), value 147,912 BTC (0.75%). The report itself says that it "will be updated to fix false identification of <37 byte scripts as P2MS outputs".

22. Anthony Milton & Clara Shikhelman, *Bitcoin and Quantum Computing: Current Status and Future Directions* (Chaincode Labs, May 2025), https://chaincode.com/bitcoin-post-quantum.pdf (accessed 2026-10-25), p. 55: "P2PK, one of Bitcoin's original output types used for early mining rewards [BSE18], represents only ~0.025%

of current UTXOs but locks a disproportionate ~8.68% of Bitcoin's value (~1,720,747 BTC) [Erh23]." / "Approximately 20-50% of all Bitcoin in circulation (4-10 million BTC) is potentially vulnerable to CRQC attacks." / "Project 11 reported that, as of mid January 2015, there were ~11.1 million BTC addresses with a non-zero balance and exposed public key for ~6.26 million BTC [PE25]." An industry report (🟡), used in this book only as a secondary source. The dollar conversions were not carried over.

23. Deloitte (Itan Barmes, Bram Bosch, Olaf Haalstra), "Quantum computers and the Bitcoin blockchain," https://www.deloitte.com/nl/en/services/risk-advisory/perspectives/quantum-computers-and-the-bitcoin-blockchain.html (accessed 2026-10-25; the text carries no date, and the datePublished in the page metadata is 2024-10-26, so the date of the figures is unknown): "Nevertheless, there are still over 4 million BTC (about 25% of all Bitcoins) which are potentially vulnerable to a quantum attack." P2PK "circa 2M," reused P2PKH "2.5M." CoinShares, "Quantum Vulnerability in Bitcoin: A Manageable Risk" (Published on Feb 6th, 2026), https://coinshares.com/insights/research-data/quantum-vulnerability-in-bitcoin-a-manageable-risk/ (accessed 2026-10-25): "The exposure is limited to addresses where public keys are visible, primarily legacy Pay-to-Public-Key (P2PK) outputs, which hold approximately 1.6 million btc—about 8% of the total supply." Both are industry materials (🟡; the latter is from an asset manager) and are used only as secondary sources. Neither post's dollar conversions nor its statements about market effects were carried over.

24. BIP-361 "Post Quantum Migration and Legacy Signature Sunset" (Jameson Lopp and five others; Status: Draft; Type: Informational; Assigned 2026-02-11), same commit, `bip-0361.mediawiki`, Motivation section: "As of March 1, 2026, over 34% of all bitcoin have revealed a public key on-chain; those UTXOs could be stolen by an attacker with a sufficiently powerful quantum computer." The text does not give the method of computation.

25. Babbush et al. (2026), p. 12 and p. 14 (caption of Figure 5): "A little over 1.7 million bitcoin (nearly 9% of all bitcoin) is secured by P2PK locking scripts." / "At the time of writing ~6.9M total bitcoin across all protocols are vulnerable. Plot generated using data from bigquery-public-data.crypto bitcoin [139]." The criterion for reuse: "(Addresses that have appeared in an Input) AND (Currently hold a balance in Unspent Outputs)".

26. Aggarwal et al. (2018), p. 8, §III.B: "1. (Reusing addresses) ... Any address that has bitcoin and for which the public key has been revealed is completely insecure. / 2. (Processed transactions) If a transaction is made from an address which has not been spent from before, and this transaction is placed on the blockchain with several blocks following it, then this transaction is reasonably secure against quantum attacks. ... / 3. (Unprocessed transactions) After a transaction has been broadcast to the network, but before it is placed on the blockchain it is at risk from a quantum attack. ..." / "We view item (3) as the most serious attack."

27. Babbush et al. (2026), p. 15: "The timing and success probability of on-spend attacks are depicted in Figure 6. The probability of success is slightly less than 41% under the idealized assumptions stated in the caption." / "Reducing block interval makes on-spend attacks significantly harder. For example, we estimate the success probability of an on-spend attack using a superconducting qubit CRQC against Litecoin, a Bitcoin derivative which targets 2.5 minutes between blocks, to be less than 3%." The same page also describes a fee-bidding contest (RBF) between the attacker and the owner and a "scorched earth" scenario, but this book does not carry that account over.

28. BIP-360 header (same commit): "BIP: 360 / Layer: Consensus (soft fork) / Title: Pay-to-Merkle-Root (P2MR) / Authors: Hunter Beast, Ethan Heilman, Isabel Foxen Duke / Status: Draft / Type: Specification / Assigned: 2024-12-18 / License: BSD-3-Clause / Version: 0.12.1 / Requires: 340, 341, 342". The repository index (README.mediawiki at the same commit, L1286–1291) also lists it as Draft.

29. BIP-360, Abstract and Design sections: "This document proposes a new output type: Pay-to-Merkle-Root (P2MR), via a soft fork. P2MR outputs operate with nearly the same functionality as P2TR (Pay-to-Taproot) outputs, but with the key path spend removed." / "In other words, P2MR outputs commit to the Merkle root of a script tree without committing to an internal key. The script(s) being committed to, however, may contain a key or key-hash." / Address Format: "P2MR outputs use SegWit version 2, resulting in mainnet addresses that start with bc1z, following BIP 173."

30. BIP-360, Abstract and Security Considerations sections: "It is worth noting that proposed P2MR outputs are only resistant to "long exposure attacks" on elliptic curve cryptography; that is, attacks on keys exposed for time periods longer than needed to confirm a spending transaction. / Protection against more sophisticated quantum attacks, including protection against private key recovery from public keys exposed in the mempool while a transaction is waiting to be confirmed (a.k.a. "short exposure attacks"), may require the introduction of post-quantum signatures in Bitcoin." / "While this proposal does not include the introduction of post-

quantum signature schemes, we think it's worth commenting on security considerations related to this possibility. Quantum-resistant signature algorithms (e.g. ML-DSA or SLH-DSA) offer different levels of protection and should be scrutinized before use."

31. BIP-360, section "Transaction Size and Fees": "P2MR witness for a depth-1 tree (135 bytes)" / "P2TR key path spend witness (66 bytes)" / "Thus, the P2MR witness would be 135 - 66 = 69 bytes larger than a P2TR key path spend witness." / "For this reason, a P2MR witness will be 32 bytes smaller than an equivalent P2TR script path spend witness, except at m = 7, where the two control blocks fall on opposite sides of the compact size boundary and the saving is 34 bytes."

32. BIP-360, Conclusion section: "In this proposal, we adopt a "prepared not scared" approach to the possible advancement of quantum computing and offer Bitcoin users an option for increased protection if they so choose. This BIP does not take a position on any specific quantum computing timeline, but rather proposes a flexible and unobtrusive option for users that wish to mitigate this risk according to their own estimate of the timeline."

33. NIST, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS 203 (Published August 13, 2024), DOI 10.6028/NIST.FIPS.203, abstract: "This standard specifies a key-encapsulation mechanism called ML-KEM. The security of ML-KEM is related to the computational difficulty of the Module Learning with Errors problem. At present, ML-KEM is believed to be secure, even against adversaries who possess a quantum computer."; NIST, *Module-Lattice-Based Digital Signature Standard*, FIPS 204 (August 13, 2024), DOI 10.6028/NIST.FIPS.204, abstract: "This standard specifies ML-DSA, a set of algorithms that can be used to generate and verify digital signatures. ML-DSA is believed to be secure, even against adversaries in possession of a large-scale quantum computer."; NIST, *Stateless Hash-Based Digital Signature Standard*, FIPS 205 (August 13, 2024), DOI 10.6028/NIST.FIPS.205, abstract: "SLH-DSA is based on SPHINCS+, which was selected for standardization as part of the NIST Post-Quantum Cryptography Standardization process." Sizes: FIPS 204 Table 2, "Sizes (in bytes) of keys and signatures of ML-DSA" (p. 16): ML-DSA-44 private 2560 / public 1312 / signature 2420; ML-DSA-65 4032 / 1952 / 3309; ML-DSA-87 4896 / 2592 / 4627. FIPS 205 §11, Table 2, "SLH-DSA parameter sets": SLH-DSA-SHA2-128s and SHAKE-128s pk 32 / sig 7 856; 128f pk 32 / sig 17 088; 192s pk 48 / sig 16 224; 192f pk 48 / sig 35 664; 256s pk 64 / sig 29 792. `nist-2024-fips-204-module-lattice-signature.pdf`, `nist-2024-fips-205-stateless-hash-signature.pdf`.

34. Aggarwal et al. (2018), p. 13, §IV.B: "In the blockchain context the most important parameters of a signature scheme are the signature and public key lengths, as these must be stored in some capacity to fully verify transactions, and the time to verify the signature. Looking at Table II, with respect the sum of signature and public key lengths, the only reasonable options are hash and lattice based schemes." The same page: "Hash based schemes like XMSS have the advantage of having provable security, at least assuming the chosen hash function behaves like a random oracle. The generic quantum attack against these schemes is to use Grover's algorithm which means that their quantum security level is half of the classical security level."

35. BIP-361 header (same commit): "BIP: 361 / Layer: Consensus (soft fork) / Title: Post Quantum Migration and Legacy Signature Sunset / Authors: Jameson Lopp, Christian Papathanasiou, Ian Smith, Joe Ross, Steve Vaile, Pierre-Luc Dallaire-Demers / Status: Draft / Type: Informational / Assigned: 2026-02-11 / License: BSD-3-Clause / Requires: TBD Post Quantum Signature BIP". The index, README.mediawiki L1293–1298, also lists it as Draft.

36. BIP-361, Abstract and Specification sections: "Phase A: Disallows sending of any funds to quantum-vulnerable addresses, hastening the adoption of PQ address types. / Phase B: Restricts ECDSA/Schnorr spends by encumbering them with a quantum-safe rescue protocol, preventing theft of funds in quantum-vulnerable UTXOs. This is triggered by a well-publicized flag-day five years after activation." The table: "A — Permitted sends are from legacy scripts to PQ scripts. — Everyone holding or accepting BTC. — 160,000 blocks (~3 years) after BIP-361 activation. / B — At a predetermined block height, nodes tighten requirements on verification of ECDSA/Schnorr. — Everyone holding or accepting BTC. — 2 years after Phase A activation."

37. BIP-361, Rationale, section "Minimizes loss of access to funds," and Backward Compatibility section: "Such rescue protocols rely on asymmetry of knowledge between a quantum attacker and the authentic coin-holder. Any context where an asymmetry exists in favor of the authentic holder can theoretically be turned into a rescue protocol." / "It remains to be seen how much of the legacy Bitcoin supply can be theoretically covered by such rescue protocols. If one or more rescue protocols can be designed to cover the majority of the Bitcoin supply, then restricting ECDSA/Schnorr verification will be at most mildly confiscatory, and will come at little expense to the overall integrity of the Bitcoin network." / "BIP-361 authors support Hourglass for

P2PK because it's not currently believed possible to construct a rescue protocol for P2PK UTXOs, as no knowledge asymmetry is known." Emphasis added.

38. ethereum.org, "A more secure Ethereum" (roadmap/security), source `public/content/roadmap/security/index.md` at commit 290d753cb2ffa47ce0cfc5e65c1539f367b82cde (2026-07-12), section "Quantum resistance" (accessed 2026-10-25): "Four areas have been identified as requiring post-quantum upgrades: validator consensus signatures (BLS), the commitment schemes used for data availability (KZG), account signatures (ECDSA), and the ZK-proof systems used by rollups." The subpage "Post-quantum cryptography on Ethereum," source `public/content/roadmap/security/quantum-resistance/index.md` at commit 643a7a456984eacc8f07105e752a67256ac6bb96 (2026-10-23).

39. ethereum.org roadmap/security (commit 290d753c): "**No quantum computer can break Ethereum's cryptography today.** The hardware required does not yet exist at scale. But recent research suggests the gap is closing faster than previously expected. In March 2026, Google Quantum AI published a paper estimating that breaking 256-bit elliptic curve cryptography (the type Ethereum uses for account signatures) could require roughly 1,200 logical qubits, about 20 times fewer than earlier estimates." The quantum-resistance page (commit 643a7a45): "Current quantum hardware is far from this scale, operating with a few thousand noisy physical qubits. Logical qubits (which correct for errors and perform reliable computation) require many physical qubits each."

40. ethereum.org quantum-resistance page (commit 643a7a45), section "3. Account signatures: ECDSA." The roadmap/security page (commit 290d753c): "EIP-8141 is being considered for the Hegotá upgrade. Core post-quantum infrastructure milestones target completion by approximately 2029. These are planning targets and may shift."

41. ethereum.org quantum-resistance page (commit 643a7a45), section "1. Consensus-layer BLS signatures": "leanXMSS: Ethereum will replace BLS signatures with leanXMSS, a hash-based signature scheme for validators. Hash-based signatures are considered quantum-safe because they rely only on the security of hash functions, which quantum computers weaken but do not break." / "Because hash-based signatures are significantly larger (roughly 3,000 bytes compared to 96 bytes for BLS), switching to leanXMSS would produce significantly more data per slot. To solve this, leanVM acts as an aggregation engine, compressing the data by 250x."

42. Ethereum Foundation Post-Quantum team, "Post-Quantum Ethereum," https://pq.ethereum.org/ (Last updated: June 27, 2026; accessed 2026-10-25): "We do not believe a cryptographically relevant quantum computer is imminent. But migrating a decentralized, global protocol takes years of coordination, engineering, and formal verification. The work must begin well before the threat arrives." / FAQ: "No single fixed date. Based on our team's current assessment, L1 protocol upgrades could be completed by 2029, with full execution-layer migration taking additional years beyond that." The disclaimer at the head of the FAQ: "The Foundation does not set protocol direction … Protocol direction and upgrades are determined through open community governance processes (e.g., All Core Devs), not by any single team or organization."

43. pq.ethereum.org, FAQ "What about long-dormant funds and abandoned wallets?": "Accounts that have never revealed a public key are not directly exposed. However, the question of what should be done about funds that cannot or will not migrate is ultimately a community governance question. Two natural scenarios exist: do nothing, or freeze vulnerable coins. Unlike Bitcoin, where roughly 5% of the supply is associated with early address formats widely believed to be abandoned (including Satoshi's ~1M BTC), Ethereum's equivalent long-dormant exposure is closer to 0.1% (an EF estimate)."

44. Babbush et al. (2026), p. 36, §VII and §VIII.A: "The P2PK locking scripts have seen little use since 2009 and a majority of the 1.7 millions bitcoin have not been transacted since 2009 despite the fact that such coins are known to be vulnerable to at-rest quantum attacks. This leads to the widespread assumption that (in most cases), the private keys of these Bitcoin UTXOs have been lost. For example, accounts believed to have been owned by Satoshi Nakamoto hold about 1 million bitcoin [293, 294] behind these P2PK scripts …". The same page: "Thus, there is no obvious way to update Bitcoin in a fashion that would keep lost P2PK coins and other dormant assets spendable by their owners and also protected against quantum computers."

45. Babbush et al. (2026), p. 37, §VIII.B.1: "The Bitcoin community is considering three general categories of approaches to dealing with dormant assets [26]: Do Nothing: the protocol is kept unchanged allowing quantum attackers to acquire the dormant assets. Burn: the protocol is modified to render dormant assets unspendable after a certain date, effectively confiscating all assets locked by P2PK scripts. Hourglass [303]:

the protocol is modified to limit the rate at which dormant assets may be spent." Emphasis added. The same section adds a fourth option ("Bad Sidechain": a dedicated sidechain that accepts off-chain proofs of ownership) as the authors' own proposal (p. 39). This book does not evaluate that proposal.

46. BIP-361, Rationale section: "Any proposal that allows for the quantum theft of "lost" bitcoin is creating a redistribution dilemma. There are 3 types of proposals: 1. Allow anyone to steal vulnerable coins, benefiting those who reach quantum capability earliest. 2. Allow throttled theft of coins, which leads to RBF battles and ultimately miners subsidizing their revenue from quantum recovered coins. 3. Allow no one to steal vulnerable coins." The "Other Methods" section of BIP-360 describes burning as "burn vulnerable coins as proposed by Jameson Lopp and others in QBIP" and the rate limit as "members of our team have separately proposed Hourglass".

47. Babbush et al. (2026), pp. 38–39: "Consensus remains elusive. Indeed, different members of the Bitcoin community have different philosophical beliefs and financial incentives." / "Indeed, an informal poll at 2025 Presidio Bitcoin Quantum Summit in San Francisco saw roughly equal support for each of the three categories of solutions [26]." The same page: "From a technical perspective, both Burn and Hourglass proposals could be implemented as a soft fork since banning or limiting transactions that spend dormant coins is a narrowing of the protocol rules. However, this requires broad support in the community."

48. Babbush et al. (2026), p. 41, §VIII.C.2: "Bitcoin's ownership model (as well as that of many other cryptocurrencies) is based on the principle that the owner of any given asset is anyone possessing the knowledge of the private key corresponding to the public key to which the asset is locked on the ledger. This principle rests on the assumption that the private key cannot be computed by anyone else. However, a CRQC can quickly derive a private key from any public key, thus breaking the foundation of Bitcoin's model of ownership."

49. Babbush et al. (2026), pp. 39–42, §VIII.C. "Digital salvage": "One public policy option treats dormant assets as effectively abandoned property subject to regulated digital salvage (akin to "sunken treasure"). ... Furthermore, principles such as escheatment [320], treasure trove, adverse possession, and maritime salvage offer general precedent for regulated recovery of abandoned property." RUUPA: "In particular, RUUPA deals with assets in possession and control of a business entity called a "holder" and distinct from the assets' owner. However, no party involved in the operation of the Bitcoin blockchain clearly meets the legal requirements to be the "holder" of the dormant coins." Laches: "Another general legal principle that may apply is the doctrine of laches [321]; if an owner of dormant coins has known for years that their assets are at risk and has failed to transact them to a post-quantum address, then they may be deemed to have failed to assert their rights through inaction." "Destruction not viable": "Due to the cryptographic relationships between the blocks on the chain, rewriting the contents of a block requires changing all the blocks that follow it. ... The computational cost of doing so is equivalent to the computational cost of re-mining more than a decade of Bitcoin's history." / "If a government banned mining transactions spending dormant assets, then foreign miners outside cooperating jurisdictions could continue to include them in their blocks." This book only carries these arguments over and does not evaluate them. This book did not open the text of the US Revised Uniform Unclaimed Property Act or the sources of each doctrine.

PART 6 · OPEN PROBLEMS

Chapter 30. The naming problem: 가상자산, crypto-asset, digital asset, 暗号資産, cryptoasset

KEY TAKEAWAY

The same string of bits is called "가상자산" (gasang jasan, "virtual asset") in Korea, "crypto-asset" in the EU, "security," "commodity," "property" or "digital asset" in the United States depending on the context, 「暗号資産」 (angō shisan, "crypto-asset") in Japan, and "cryptoasset" in the United Kingdom. This chapter sets the definitional provisions of five jurisdictions side by side **in the original wording**, and gives each provision its statute number, its date of entry into force and the reference date. It then looks at what the name decides under three heads: **jurisdiction** (which agency oversees it), **taxation** (as what it is taxed) and **criminal law** (of which offense it can be the object). Technology does not choose the name. What Part 1 showed was an 80-byte header, signatures and state transitions, and whether to call that a "currency," an "asset" or a "security" is in every case a choice made by law. That choice has changed even within a single jurisdiction. Japan changed 「仮想通貨」 (kasō tsūka, "virtual currency") to 「暗号資産」, the Supreme Court of Korea wrote "가상화폐" (gasang hwapye, "virtual currency") in 2018 and "가상자산" in 2021, and the US Internal Revenue Service deleted in 2023 the statement that virtual currency "does not have legal tender status in any jurisdiction." This chapter is the prose version of the concordance in Appendix C, and it was written so that its figures and provisions agree with the table in 27.3.

Learning objectives

1. Read the definitional provisions of five jurisdictions in the original, and compare **what each puts in and what each leaves out** (the exclusion list).
2. Sort out which agency's jurisdiction, which tax framework and which offense the five category words "currency/money," "asset," "security," "commodity" and "property" call in, in each jurisdiction.
3. Confirm, from the original provisions and judgments, three cases in which the name changed within one jurisdiction (Japan 2020, the Supreme Court of Korea 2018→2021, the US IRS 2023).
4. Find, in the wording of the provisions, the places where the definitions fail to keep up with technology: non-fungible tokens, automated markets, assets without an issuer.

What a reader of the [Basic] paragraphs alone takes away. Whether the same thing is called a

"currency" or an "asset" changes its legal fate. **Technology does not choose the name.**

1. Why the name matters

In Chapter 12 we took the 80 bytes of the genesis block apart by hand. They held a version, the previous hash, the Merkle root, a time, the bits and a nonce. The word "currency" was not there, and neither was the word "asset." The UTXO of Chapter 11 is an "unspent transaction output," and the account balance of Chapter 13 is a number in the state tree. Everything this book showed in Parts 1 to 3 is of that kind (bit strings, signatures, state transitions, issuance rules, equilibria), and it was law that put names on them. But each body of law put a different name on them, and the name decides which agency's jurisdiction, which tax law and which offense apply. The four cases of Chapter 26 showed this. The same token was found to be a "security" in the United States, and securities law came in (The DAO, Terra); it was found to be a "commodity," and the Commodity Exchange Act came in. In Korean criminal law it is a "재산상 이익" (jaesansang iik, "pecuniary advantage") and so can be the object of fraud, but whether it is a "재물" (jaemul, "property in the sense of a thing"), the object of embezzlement, is a question the Supreme Court of Korea did not decide. This chapter sets out the **original text** of those names side by side. What the concordance in Appendix C compressed into rows and columns is read here as sentences.

There are three rules for reading the provisions. First, statutory text is reproduced **as enacted.** English-language provisions stand as they are. Korean and Japanese provisions are followed by a working gloss, marked as such, for reading only: a gloss is not an official translation, the original controls, and no statement in this book rests on a gloss (the book's principle: do not settle what a provision means from a translation). Second, each provision is given its statute number and date of entry into force, and the text states whether it is **in force** as of the reference date (2026-10-25). Third, the **exclusion list** of a definitional provision is read with the same weight as the main text. What a jurisdiction leaves out tells you where it places this thing.

2. The definitional provisions of five jurisdictions, in the original

2.1 Korea: "가상자산"

Article 2, subparagraph 1 of the Act on the Protection of Virtual Asset Users (가상자산 이용자 보호 등에 관한 법률; below, the Virtual Asset User Protection Act). This is the wording enacted by Act No. 19563 (promulgated July 18, 2023, in force July 19, 2024), and it is unchanged in the current Act No. 21857 (in force October 2, 2026). **In force as of 2026-10-25.**[1]

> 1. "가상자산"이란 경제적 가치를 지닌 것으로서 전자적으로 거래 또는 이전될 수 있는 전자적 증표(그에 관한 일체의 권리를 포함한다)를 말한다. 다만, 다음 각 목의 어느 하나에 해당하는 것은 제외한다. 가. 화폐 · 재화 · 용역 등으로 교환될 수 없는 전자적 증표 또는 그 증표에 관한 정보로서 발행인이 사용처와 그 용도를 제한한 것 나. 「게임산업진흥에 관한 법률」 제32조제1항제7호에 따른 게임물의 이용을 통하여 획득한 유 · 무형의 결과물 다. 「전자금융거래법」 제2조제14호에 따른 선불전자지급수단 및 같은 조 제15호에 따른 전자화폐 라. 「주식 · 사채 등의 전자등록에 관한 법률」 제2조제4호에 따른 전자등록주식등 마. 「전자어음의 발행 및 유통에 관한 법률」 제2조제2호에 따른 전자어음 바. 「상법」 제862조에 따른 전자선하증권 사. 「한국은행법」에 따른 한국은행(이하 "한국은행"이라 한다)이 발행하는 전자적 형태의 화폐 및 그와 관련된 서비스 아. 거래의 형태와 특성을 고려하여 대통령령으로 정하는 것
>
> *Working gloss (not an official translation; the original controls):* 1. "Virtual asset" means, as something having economic value, an electronic token that can be traded or transferred electronically (including all rights relating to it); provided, however, that anything falling under any of the following items is excluded. (a) [가] an electronic token that cannot be exchanged for money, goods, services, etc., or information on such a token, being one whose places of use and purposes have been restricted by the issuer (b) [나] tangible or intangible results obtained through the use of game products under Article 32(1), subparagraph 7 of the Game Industry Promotion Act (c) [다] prepaid electronic means of payment under Article 2, subparagraph 14 of the Electronic Financial Transactions Act, and electronic currency under subparagraph 15 of the same Article (d) [라] electronically registered stocks, etc. under Article 2, subparagraph 4 of the Act on Electronic Registration of Stocks, Bonds, etc. (e) [마] electronic bills under Article 2, subparagraph 2 of the Issuance and Distribution of Electronic Bills Act (f) [바] electronic bills of lading under Article 862 of the Commercial Act (g) [사] money in electronic form issued by the Bank of Korea under the Bank of Korea Act (hereinafter "the Bank of Korea"), and services related to it (h) [아] anything prescribed by Presidential Decree in consideration of the form and characteristics of transactions

Article 2 of the Enforcement Decree of the Act on the Protection of Virtual Asset Users (가상자산 이용자 보호 등에 관한 법률 시행령), to which item (h) delegates, has an exclusion list of five subparagraphs, and its subparagraph 4 reaches non-fungible tokens (Section 3 and footnote 15).

The same wording first appeared in Article 2, subparagraph 3 of the Act on Reporting and Using Specified Financial Transaction Information (특정 금융거래정보의 보고 및 이용 등에 관한 법률; below, the Specified Financial Transaction Information Act). Act No. 17113 (promulgated March 24, 2020, in force March 25, 2021) put the text of the definition directly in that Act, and on July 19, 2024 the Addenda of the Virtual Asset User Protection Act turned it into the present cross-referencing wording.[2]

> (As inserted by Act No. 17113; in force March 25, 2021 to July 18, 2024) 3. "가상자산"이란 경제적 가치를 지닌 것으로서 전자적으로 거래 또는 이전될 수 있는 전자적 증표(그에 관한 일체의 권리를 포함한다)를 말한다. 다만, 다음 각 목의 어느 하나에 해당하는 것은 제외한다. 가. ~ 바. (same as above) 사. 거래의 형태와 특성을 고려하여 대통령령으로 정하는 것
>
> *Working gloss (not an official translation; the original controls):* (As inserted by Act No. 17113; in force March 25, 2021 to July 18, 2024) 3. "Virtual asset" means, as something having economic value, an electronic token that can be traded or transferred electronically (including all rights relating to it); provided, however, that anything falling under any of the following items is excluded. (a)–(f) [가 ~ 바] (same as above) (g) [사] anything prescribed by Presidential Decree in consideration of the form and characteristics of transactions
>
> (Current) 3. "가상자산"이란 「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산을 말한다.
>
> *Working gloss (not an official translation; the original controls):* (Current) 3. "Virtual asset" means a virtual asset under Article 2, subparagraph 1 of the Act on the Protection of Virtual Asset Users.

The tax provision is Article 21(1), subparagraph 27 of the Income Tax Act (소득세법). It was inserted by Act No. 17757 (promulgated December 29, 2020), its date of entry into force was postponed three times, and it **enters into force on January 1, 2027; as of 2026-10-25 it is not yet in force.**[3]

> 27. 「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산(이하 "가상자산"이라 한다)을 양도하거나 대여함으로써 발생하는 소득(이하 "가상자산소득"이라 한다)
>
> *Working gloss (not an official translation; the original controls):* 27. Income arising from transferring or lending virtual assets under Article 2, subparagraph 1 of the Act on the Protection of Virtual Asset Users (hereinafter "virtual assets") (hereinafter "virtual asset income")

2.2 EU: "crypto-asset"

Regulation (EU) 2023/1114 (MiCA), Art. 3(1), points (5)–(9). Official Journal L 150 (June 9, 2023), p. 63. Under Article 149 it **applies from 2024-12-30** (Titles III and IV from 2024-06-30). **Applies as of 2026-10-25.**[4]

> (5) 'crypto-asset' means a digital representation of a value or of a right that is able to be transferred and stored electronically using distributed ledger technology or similar technology; (6) 'asset-referenced token' means a type of crypto-asset that is not an electronic money token and that purports to maintain a stable value by referencing another value or right or a combination thereof, including one or more official currencies; (7) 'electronic money token' or 'e-money token' means a type of crypto-asset that purports to maintain a stable value by referencing the value of one official currency; (8) 'official currency' means an official currency of a country that is issued by a central bank or other monetary authority; (9) 'utility token' means a type of crypto-asset that is only intended to provide access to a good or a service supplied by its issuer;

The exclusion list is in Article 2.

> 3. This Regulation does not apply to crypto-assets that are unique and not fungible with other crypto-assets.
> 4. This Regulation does not apply to crypto-assets that qualify as one or more of the following: (a) financial instruments; (b) deposits, including structured deposits; (c) funds, except if they qualify as e-money tokens; (d) securitisation positions …; (e) non-life or life insurance products …; (f) pension products …; (g) officially recognised occupational pension schemes …; (h) individual pension products …; (i) a pan-European Personal Pension Product …; (j) social security schemes ….

2.3 United States: no single definition

The United States has no single definitional provision covering this thing as a whole. Courts and agencies decide, according to the shape of the transaction, which category word of which statute the same thing falls under. Five category words have been confirmed in the original texts.

(a) "investment contract": securities. The Securities Act of 1933 includes "investment contract" in "security," and in 1946 the Supreme Court of the United States fixed its meaning as follows. **Case law, good law as of 2026-10-25.**[5]

> an investment contract for purposes of the Securities Act means a contract, transaction or scheme whereby a person invests his money in a common enterprise and is led to expect profits solely from the efforts of the promoter or a third party, it being immaterial whether the shares in the enterprise are evidenced by formal certificates or by nominal interests in the physical assets employed in the enterprise.

(b) "commodity": commodities. In 2018 the United States District Court for the Eastern District of New York quoted the definition in the Commodity Exchange Act and applied it as follows. **District court decision, as of 2026-10-25; subsequent history in higher courts not verified.**[6]

> They fall well-within the common definition of "commodity" as well as the CEA's definition of "commodities" as "all other goods and articles . . . in which contracts for future delivery are presently or in the future dealt in." Title 7 U.S.C. § 1(a)(9).

(c) "property": property for tax purposes. IRS Notice 2014-21 (2014-16 I.R.B. 938). **Administrative interpretation; only a background sentence modified by Notice 2023-34; in effect as of 2026-10-25.**[7]

> Virtual currency is a digital representation of value that functions as a medium of exchange, a unit of account, and/or a store of value. In some environments, it operates like "real" currency -- i.e., the coin and paper money of the United States or of any other country that is designated as legal tender, circulates, and is customarily used and accepted as a medium of exchange in the country of issuance -- but it does not have legal tender status in any jurisdiction. (2014-21, Background, original text; 2023-34 revised the third sentence to read as below)
>
> In certain contexts, virtual currency may serve one or more of the functions of "real" currency -- i.e., the coin and paper money of the United States or of any other country that is designated as legal tender, circulates, and is customarily used and accepted as a medium of exchange in the country of issuance -- but the use of virtual currency to perform "real" currency functions is limited. (2023-34 §3)
>
> Q-1: How is virtual currency treated for federal tax purposes? A-1: For federal tax purposes, virtual currency is treated as property. General tax principles applicable to property transactions apply to transactions using virtual currency. (2014-21 §4)

(d) "digital asset" and "payment stablecoin": federal statute. GENIUS Act, Pub. L. 119-27, 139 Stat. 419 (July 18, 2025), §2(6) and (22). **Enacted. Under §20 the effective date is 2027-01-18 at the latest and may be earlier depending on the publication of final regulations; whether those regulations had been published as of 2026-10-25 is not verified.**[8]

> (6) DIGITAL ASSET.—The term "digital asset" means any digital representation of value that is recorded on a cryptographically secured distributed ledger. (22) PAYMENT STABLECOIN.—The term "payment stablecoin"— (A) means a digital asset— (i) that is, or is designed to be, used as a means of payment or settlement; and (ii) the issuer of which— (I) is obligated to convert, redeem, or repurchase for a fixed amount of monetary value, not including a digital asset denominated in a fixed amount of monetary value; and (II) represents that such issuer will maintain, or create the reasonable expectation that it will maintain, a stable value relative to the value of a fixed amount of monetary value; and (B) does not include a digital asset that— (i) is a national currency; (ii) is a deposit (as defined in section 3 of the Federal Deposit Insurance Act (12 U.S.C. 1813)), including a deposit recorded using distributed ledger technology; or (iii) is a security, as defined in section 2 of the Securities Act of 1933 (15 U.S.C. 77b), section 3 of the Securities Exchange Act of 1934 (15 U.S.C. 78c), or section 2 of the Investment Company Act of 1940 (15 U.S.C. 80a–2), except that, for the avoidance of doubt, no bond, note, evidence of indebtedness, or investment contract that was issued by a permitted payment stablecoin issuer shall qualify as a security solely by virtue of its satisfying the conditions described in subparagraph (A), consistent with section 17 of this Act.

(e) "controllable electronic record": a private-law model act. Uniform Commercial Code Amendments (2022), §12-102(a)(1). **Model law; adoption by individual states not verified.**[9]

> "Controllable electronic record" means a record stored in an electronic medium that can be subjected to control under Section 12-105. The term does not include a controllable account, a controllable payment intangible, a deposit account, an electronic copy of a record evidencing chattel paper, an electronic document of title, electronic money, investment property, or a transferable record.

2.4 Japan: 「暗号資産」

Payment Services Act (資金決済に関する法律, 平成21年法律第59号, Act No. 59 of 2009; 資金決済法 for short), 第2条第14項 (Article 2(14)). The defined term, formerly 「仮想通貨」, was changed to 「暗号資産」 by 令和元年法律第28号 (Act No. 28 of 2019; promulgated 2019-06-07, in force 2020-05-01), and from the version in force on 2023-06-01 it moved from 第5項 to 第14項. **In force as of 2026-10-25.**[10]

> この法律において「暗号資産」とは、次に掲げるものをいう。ただし、
> 金融商品取引法第二十九条の二第一項第八号に規定する権利を表示するものを除く。 一
> 物品等を購入し、若しくは借り受け、又は役務の提供を受ける場合に、
> これらの代価の弁済のために不特定の者に対して使用することができ、かつ、
> 不特定の者を相手方として購入及び売却を行うことができる財産的価値
> （電子機器その他の物に電子的方法により記録されているものに限り、
> 本邦通貨及び外国通貨、通貨建資産並びに電子決済手段
> （通貨建資産に該当するものを除く。）を除く。次号において同じ。）であって、
> 電子情報処理組織を用いて移転することができるもの 二
> 不特定の者を相手方として前号に掲げるものと相互に交換を行うことができる財産的価値であって、電子情報処理組織を用いて移転することができるもの
>
> *Working gloss (not an official translation; the original controls):* In this Act, "crypto-asset (angō shisan)" means the following; provided, however, that anything representing the rights provided for in Article 29-2, paragraph (1), item (viii) of the Financial Instruments and Exchange Act is excluded. (i) [一] proprietary value (財産的価値) that can be used with unspecified persons (不特定の者) to pay the price when purchasing or borrowing goods, etc. or receiving the provision of services, and that can be bought and sold with unspecified persons as counterparties (limited to that which is recorded by electronic means on an electronic device or other object, and excluding Japanese currency and foreign currency, currency-denominated assets (通貨建資産) and electronic payment instruments (電子決済手段) (excluding those that fall under currency-denominated assets); the same applies in the following item), which can be transferred using an electronic data processing system (電子情報処理組織) (ii) [二] proprietary value that can be exchanged reciprocally for what is listed in the preceding item with unspecified persons as counterparties, which can be transferred using an electronic data processing system

The first sentences before and after the change of name:

> (Version of 2020-04-30, 第2条第5項) この法律において「仮想通貨」とは、
> 次に掲げるものをいう。 (Version of 2020-05-01, 第2条第5項) この法律において「暗号資産」とは、次に掲げるものをいう。ただし、金融商品取引法（昭和二十三年法律第二十五号）第二条第三項に規定する電子記録移転権利を表示するものを除く。
>
> *Working gloss (not an official translation; the original controls):* (Version of 2020-04-30, 第2条第5項) In this Act, "virtual currency (kasō tsūka)" means the following. (Version of 2020-05-01, 第2条第5項) In this Act, "crypto-asset (angō shisan)" means the following; provided, however, that anything representing electronically recorded transferable rights (電子記録移転権利) provided for in Article 2, paragraph (3) of the Financial Instruments and Exchange Act (Act No. 25 of 1948) is excluded.

The 「電子記録移転権利」 (denshi kiroku iten kenri, "electronically recorded transferable rights") that the proviso excludes are defined inside the parentheses of 第2条第3項 (Article 2(3)) of the Financial Instruments and Exchange Act (金融商品取引法; 金商法 for short).

> …電子情報処理組織を用いて移転することができる財産的価値（電子機器その他の物に電子的方法により記録されるものに限る。）に表示される場合（流通性その他の事情を勘案して内閣府令で定める場合を除く。）に限る。以下「電子記録移転権利」という。）…
>
> *Working gloss (not an official translation; the original controls):* … limited to cases where represented on proprietary value that can be transferred using an electronic data processing system (limited to that which is recorded by electronic means on an electronic device or other object) (excluding cases specified by Cabinet Office Order (内閣府令) in consideration of transferability (流通性) and other circumstances); hereinafter referred to as "electronically recorded transferable rights".) …

And 令和8年法律第64号 (Act No. 64 of 2026; promulgated 2026-07-23) is an amendment that moves this definition to 金商法 第2条第49項. **It has been promulgated and is partly in force, but the part that transfers the definition is not yet in force as of 2026-10-25.** The first sentence of the version not yet in force:

> (金融商品取引法 第2条第49項, version not yet in force) この法律において「暗号資産」とは、次に掲げるものをいう。ただし、
> 第二十九条の二第一項第八号に規定する権利を表示するものを除く。
>
> *Working gloss (not an official translation; the original controls):* (金融商品取引法 第2条第49項, version not yet in force) In this Act, "crypto-asset (angō shisan)" means the following; provided, however, that anything representing the rights provided for in Article 29-2, paragraph (1), item (viii) is excluded.

2.5 United Kingdom: "cryptoasset"

Financial Services and Markets Act 2000, s. 417(1). Inserted by the Financial Services and Markets Act 2023 (c. 29), s. 69(4)(a), **in force 2023-08-29 (SI 2023/779, reg. 4(uu)). In force as of 2026-10-25.**[11]

> "cryptoasset" means any cryptographically secured digital representation of value or contractual rights that— (a) can be transferred, stored or traded electronically, and (b) that uses technology supporting the recording or storage of data (which may include distributed ledger technology).

What falls within regulated activities is fixed by two articles that 2026 regulations inserted into the Regulated Activities Order: The Financial Services and Markets Act 2000 (Cryptoassets) Regulations 2026, SI 2026/102, made 2026-02-04. **Full commencement day 2027-10-25; as of 2026-10-25 only the parts for the FCA's rule-making are in force.**[12]

> 88F.—(1) A qualifying cryptoasset. (2) A "qualifying cryptoasset" means a cryptoasset which is— (a) fungible, (b) transferable, (c) not solely a record of value or contractual rights, including rights in another cryptoasset, and (d) not excluded by paragraph (4). (3) For the purposes of paragraph (2)(b), the circumstances in which a cryptoasset is to be treated as "transferable" include where it confers transferable rights. (4) A cryptoasset does not fall within paragraph (1) if it is— (a) a specified investment cryptoasset, other than one specified by— (i) article 74A (electronic money); and (ii) article 88F (qualifying cryptoassets); (b) electronic money; (c) currency of the United Kingdom or any other country or territory, including a central bank digital currency; (d) a cryptoasset that— (i) cannot be transferred or sold in exchange for money or other cryptoassets, except by way of redemption with the issuer, and (ii) can only be used by the holder— (aa) to acquire goods or services from the issuer, or (bb) to acquire goods or services within a limited network of service providers which have direct commercial agreements with the issuer. 88G.—(1) A qualifying stablecoin. (2) A "qualifying stablecoin" means a qualifying cryptoasset where— (a) that cryptoasset seeks or purports to maintain a stable value in relation to a particular fiat currency ("the referenced fiat currency"), and (b) fiat currency (which may be referenced fiat currency referred to in sub-paragraph (a)) or other assets are held for the purpose of maintaining a stable value, in accordance with sub-paragraph (a). (3) Where a cryptoasset seeks or purports to maintain a stable value in relation to an asset other than a fiat currency, the cryptoasset is not to be regarded as falling within paragraph (2)(a) even if the asset is expressed in terms of a fiat currency. [(4) and (5) omitted]

The provision on the property-law side is not a definition but a **confirmation**. Property (Digital Assets etc) Act 2025 (c. 29), s. 1. **Royal Assent on 2025-12-02, in force the same day (England and Wales, Northern Ireland).**[13]

> A thing (including a thing that is digital or electronic in nature) is not prevented from being the object of personal property rights merely because it is neither— (a) a thing in possession, nor (b) a thing in action.

2.6 International: "virtual asset"

As a point of reference outside the five jurisdictions, this chapter gives the definition of the Financial Action Task Force (FATF), from the glossary of its updated guidance of October 28, 2021. **A recommendation, with no legally binding force.**[14]

> A virtual asset is a digital representation of value that can be digitally traded, or transferred, and can be used for payment or investment purposes. Virtual assets do not include digital representations of fiat currencies, securities and other financial assets that are already covered elsewhere in the FATF Recommendations.

3. Reading the provisions on top of one another: what goes in and what is left out

Laid on top of one another, the six definitions show one common skeleton: **"a digital representation of a value (or a right)" + "electronic transferability."** Korea has "경제적 가치를 지닌 것으로서 전자적으로 거래 또는 이전될 수 있는 전자적 증표" (gloss: "as something having economic value, an electronic token that can be traded or transferred electronically"); the EU, "a digital representation of a value or of a right that is able to be transferred and stored electronically"; the US GENIUS Act, "any digital representation of value that is recorded on a cryptographically secured distributed ledger"; Japan, 「財産的価値…であって、電子情報処理組織を用いて移転することができるもの」 (gloss: "proprietary value (財産的価値) … which can be transferred using an electronic data processing system (電子情報処理組織)"); the United Kingdom, "digital representation of value or contractual rights that … can be transferred, stored or traded electronically"; and the FATF, "digital representation of value that can be digitally traded, or transferred." Of the technology shown in Part 1, the provisions picked up **transferability** alone. Hashes, signatures, consensus rules and finality appear in none of the definitions. Only the EU, the United Kingdom and the US GENIUS Act put in technical words, "distributed ledger technology" and "cryptographically secured"; Korea and Japan are technology-neutral.

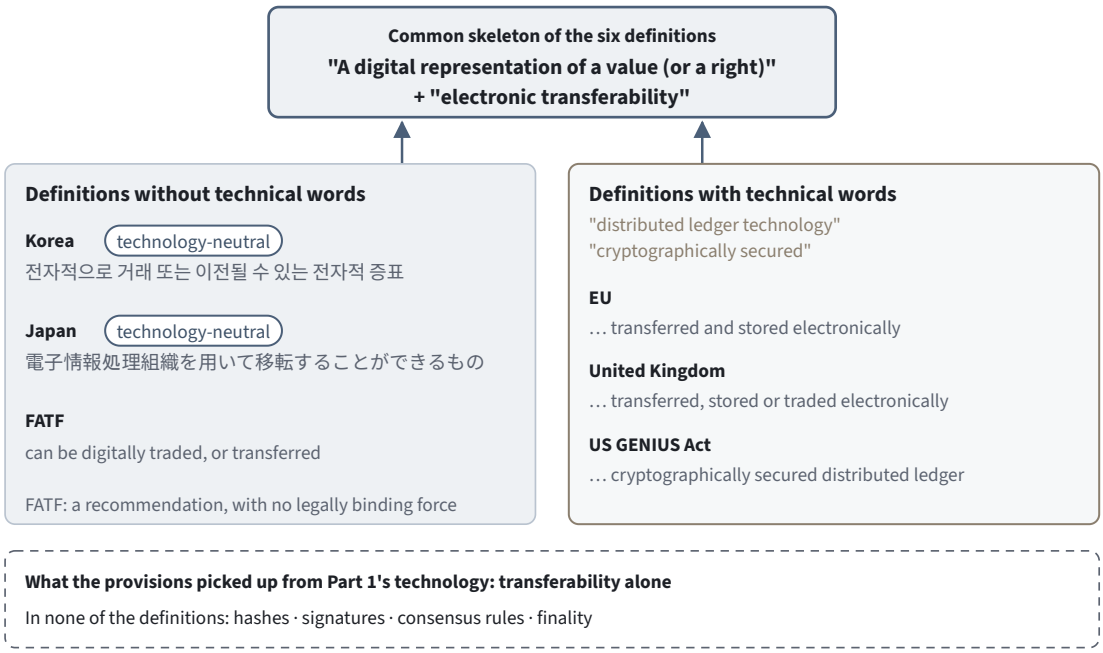


Figure 30-1. The common skeleton of the six definitions laid on top of one another, and whether each uses technical words. Only the EU, the United Kingdom and the US GENIUS Act put in technical words, and of the technology of Part 1 the provisions pick up transferability alone. The phrase under each jurisdiction is part of the provision quoted in the text

Where they diverge is the **exclusion list**.

LEFT OUT	KOREA	EU	UNITED STATES (GENIUS)	JAPAN	UNITED KINGDOM (88F)
Legal tender, central bank digital currency	item (g) (money in electronic form issued by the Bank of Korea)	(8) "official currency" is a separate concept (no exclusion clause in the definition)	(B)(i) national currency, in the payment stablecoin definition only; no exclusion in (6) digital asset	本邦通貨及び外国通貨, 通貨建資産	(4)(c) currency … including a central bank digital currency
Electronic money, prepaid instruments	item (c)	(4)(c) funds (but EMTs are included)	— (deposits are left out separately by (B)(ii))	電子決済手段 (defined separately in 第5項)	(4)(b) electronic money
Securities, financial instruments	item (d) (electronically registered stocks, etc.), items (e) and (f)	(4)(a) financial instruments	(B)(iii) security	proviso: rights under 金商法 第29条の2第1項第8号 (電子記録移転権利)	(4)(a) specified investment cryptoasset
Use-restricted tokens	item (a) (the issuer restricted the places and purposes of use)	— (utility tokens are included, (9))	—	—	(4)(d) redeemable only with the issuer, limited network
Game outputs	item (b)	—	—	—	—
Non-fungible (NFT)	item (h) → Enforcement Decree §2(4) (what the Financial Services Commission (금융위원회) designates by public notice, e.g. tokens that can be used as means of payment, is brought back in by a proviso)	Art. 2(3) unique and not fungible	—	—	(2)(a) fungible requirement
Delegation	item (h) (Presidential Decree)	—	—	—	—

Three things can be read from this table. First, **"currency" is outside everywhere.** Korea, Japan and the United Kingdom expressly leave legal tender and its digital forms out of the definition, the United States leaves "national currency" out of the definition of "payment stablecoin," and the EU keeps "official currency" as a separate concept. Japan changed the name itself from 「仮想通貨」 to 「暗号資産」. This is a legislative choice that pairs with the proposition Chapter 19 showed with a model, that "what sets legal tender apart is the floor of tax demand." Law has kept the name "currency" only for what it issues itself. Second, **all of them draw a boundary with securities, but in different ways.** Korea, the EU, the United Kingdom and the GENIUS Act say "if it is a security, it is outside this law," while Japan leaves

it out by bringing a concept of financial-instruments law, 「電子記録移転権利」, into a proviso. US securities law itself goes the other way: **if it is an "investment contract," it is inside this law**, so the same token becomes a security or not depending on the shape of the transaction (*SEC v. Ripple Labs, Inc.*, No. 1:20-cv-10832 (AT) (S.D.N.Y. July 13, 2023), ECF No. 874, pp. 15, 23, 34; footnote 19 of Chapter 27). Third, **three jurisdictions left out non-fungible tokens, but at different levels.** The EU left them out in the body of the Regulation (Art. 2(3)), and the United Kingdom in the definition for regulated activities (art. 88F(2)(a), "fungible"). In Korea, Article 2, subparagraph 4 of the Enforcement Decree, under the delegation in item (h) of Article 2, subparagraph 1 of the Act, leaves out "단일하게 존재하여 다른 전자적 증표로 대체할 수 없는 전자적 증표" (gloss: "an electronic token that exists singly and cannot be replaced by another electronic token"), but adds a proviso that brings back in "특정 재화나 서비스의 지급수단으로 사용될 수 있는 전자적 증표 등 금융위원회가 정하여 고시하는 전자적 증표" (gloss: "electronic tokens determined and publicly notified by the Financial Services Commission, such as an electronic token that can be used as a means of payment for specific goods or services").[15] Whether the (contract address, number) pair of ERC-721 shown in Chapter 14 comes within the Financial Services Commission's notice as something "지급수단으로 사용될 수 있는" (gloss: "that can be used as a means of payment") depends on what the token points to and on the notice. It is one example of the place that the design for this chapter marked as "when the definitional provision cannot keep up with the technology."

4. What the name decides: jurisdiction, taxation, criminal law

Jurisdiction

The name first decides **who oversees it**. In Korea, "가상자산" calls in supervision and investigation by the Financial Services Commission (Articles 13 and 14 of the Virtual Asset User Protection Act) and reporting to the head of the Korea Financial Intelligence Unit (금융정보분석원장) (Article 7 of the Specified Financial Transaction Information Act), while "전자등록주식등" ("electronically registered stocks, etc.") takes the thing outside that Act and over to the capital markets side. In the EU, "crypto-asset" calls in the competent authorities of the Member States (MiCA Art. 93) and, for significant asset-referenced tokens and e-money tokens, the European Banking Authority (EBA) (Art. 117), while a "financial instrument" is outside MiCA. In Japan, 「暗号資産」 calls in registration with the Prime Minister (内閣総理大臣) (in practice the Financial Services Agency, 金融庁) under 資金決済法第63条の2, and 「電子記録移転権利」 calls in 金商法. In the United Kingdom, "qualifying cryptoasset" calls in the FCA's regulated activities from 2027. In the United States the **agencies choose the name**: a "security" means the SEC, a "commodity" means the CFTC, and the 2018 decision called the two "concurrent."[16] In Chapter 26, Terra is a case in which the SEC brought charges on the basis that it was a "security" and obtained a jury verdict, and The DAO is a case in which the SEC found a "security" and took no enforcement action. The "market without an operator" that the AMM of Chapter 15 and the MEV of Chapter 17 showed is a gap in jurisdiction, in that none of these names has an addressee there (28.1(e)), and the express exclusion of participation in liquidity pools from "digital asset service provider" in §2(7) of the GENIUS Act shows that the legislature knew about that gap.

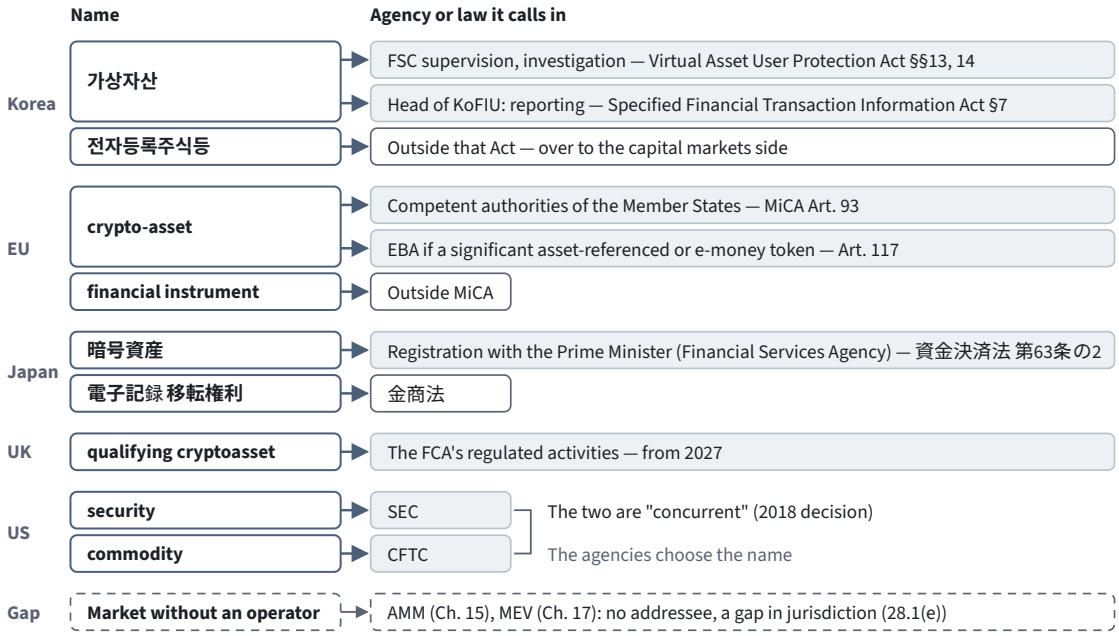


Figure 30-2. The agencies and laws that each name calls in, by jurisdiction. The name first decides who oversees it, and in the United States the agencies choose the name. Select a name and only the places it calls in remain. A market without an operator is a gap in which none of the names has an addressee

Taxation

Next, the name decides **as what the thing is taxed**. In 2014 the IRS decided to treat "virtual currency" as "property," so the general principles for property transactions (the difference between the acquisition price and the transfer price) apply, and because it is not "currency," the framework of foreign currency gain or loss does not come in (Q&A-2).[17] Had the name been "currency," other provisions would have come in. EU value added tax read it the other way, toward "currency." In 2015 the Court of Justice of the European Union said that bitcoin has "no purpose other than to be a means of payment" and is not "tangible property," and it placed the exchange of bitcoin for traditional currencies under the exemption for transactions concerning "currency, bank notes and coins used as legal tender" (Art. 135(1)(e)). The judgment treated the exchange as a supply of services (operative part 1) and decided whether that supply was exempt.[18] Korea made "가상자산소득" (gasang jasan sodeuk, "virtual asset income") one subparagraph of "기타소득" (gita sodeuk, "other income") (Income Tax Act, Article 21(1), subparagraph 27), and that provision is not yet in force. The same bit string is property in the United States, a means of payment for EU value added tax, and (once the provision is in force) a source of other income in Korea. Technology chose none of the three.

Criminal law

Finally, the name decides **of which offense the thing can be the object**. The Supreme Court of Korea called bitcoin a "재산상 이익" (Supreme Court of Korea, 2021Do9855, November 11, 2021), so it can be the object of fraud, but the Court did not decide whether it is a "재물." The Court also held that punishing a person who received a transfer without legal cause for breach of trust, by analogy to the case law on mistaken remittances and on the basis of the principle of good faith, would, "명문의 규정이 없는 현재의 상황에서" (working gloss, not an official translation; the original controls: "in the present situation, in which there is no express provision"), violate the principle of legality in criminal law (Supreme Court of Korea, 2020Do9789, December 16, 2021).[19] In confiscation, under the name "재산적 가치가 있는 무형의 재산" (gloss: "intangible property having proprietary value"), it became subject to confiscation under the Act on Regulation and Punishment of Criminal Proceeds Concealment (범죄수익은닉의 규제 및 처벌 등에 관한 법률) (Supreme Court of Korea, 2018Do3619, May 30, 2018). The same thing fills the places of fraud and confiscation under two names, "재산상 이익" and "무형재산" (muhyeong jaesan, "intangible property"). In the place of breach of trust, without any decision on whether it is a "재물," the conviction was reversed on the ground that the defendant was not "타인의 사무를 처리하는 자" (gloss: "a person who administers another's business"), and the embezzlement part was reversed and remanded along with it on the ground that the two were one and the same. The criminal judgments in Japan's Mt.Gox case show a different shape. The first-instance judgment (2019) came before the statute was renamed to 「暗号資産」 and so wrote 「仮想通貨」, while the appellate judgment (2020) came after the renaming and wrote 「暗号資産（仮想通貨）」.[20] And the offenses the two judgments found proven were not offenses with bitcoin as their object but 私電磁的記録不正作出·同供用 (gloss: "unauthorized creation of private electromagnetic records, and use of such records"), whose object is **the electronic records of the exchange's books**. On the embezzlement of users' money the defendant was acquitted, on the ground that 「利用者が…送金した金銭は，Ｂ社に帰属する」 (gloss: "the money that users … remitted belongs to Company B") (Chapter 26). What the court fixed there was not the name of bitcoin but a **criterion of existence**,
「ブロックチェーン上に存在しないビットコインは…架空のビットコイン」 (gloss: "bitcoins that do not exist on the blockchain … fictitious bitcoins"), and that criterion came from the technical fact of Part 1: bitcoin exists only as a state recorded on a blockchain. The two US criminal judgments (Chapter 26) had "wire fraud," "securities fraud conspiracy" and "commodities fraud conspiracy" as counts, and "commodities fraud" and "securities fraud" standing side by side in one indictment is the criminal-law face of the fact that in the United States the same thing carries two names at once.[21]

5. Three cases in which the name changed within one jurisdiction

Names differ between jurisdictions, and they have also changed within a single jurisdiction. Three cases have been confirmed in the original texts.

JURISDICTION	BEFORE	AFTER	DOCUMENT THAT MADE THE CHANGE	WHAT CHANGED
Japan	「仮想通貨」(資金決済法第2条第5項, version of 2020-04-30)	「暗号資産」(version of 2020-05-01; now 第14項)	令和元年法律第28号 (promulgated 2019-06-07, in force 2020-05-01)	The proviso excluding 電子記録移転権利 came in together with the defined term. The word "currency" (通貨) dropped out of the name
Korea (Supreme Court)	"이른바 '가상화폐'의 일종" (Supreme Court of Korea, 2018Do3619, May 30, 2018)	"가상자산의 일종" (2021Do9855, November 11, 2021); "가상자산은 … 재산상 이익" (2020Do9789, December 16, 2021)	In between, Act No. 17113 amending the Specified Financial Transaction Information Act (in force March 25, 2021) brought "가상자산" in as a statutory term	The judgments' word changed. The judgments do not say that the framework of decision (intangible property having proprietary value / pecuniary advantage) changed because of it
United States (IRS)	"it does not have legal tender status in any jurisdiction" (Notice 2014-21, 2014)	"the use of virtual currency to perform 'real' currency functions is limited" (Notice 2023-34, 2023)	Notice 2023-34 §3	Only the background statement was revised; the conclusion "property" (Q&A-1 and 2) was left as it was

None of the three happened **because the technology changed**. Bitcoin's protocol underwent no change between 2019, 2021 and 2023 that would call for a new name. What changed was where law decided to place it, and that decision was written into the name. This is what the one line the design set down for this chapter, "technology does not choose the name," means.

? A LAWYER'S QUESTION

What does a lawyer do when a definitional provision cannot keep up with the technology? Does the (contract address, number) pair of ERC-721 fall within the "전자적 증표" ("electronic token") of Article 2, subparagraph 1 of Korea's Virtual Asset User Protection Act? The EU and the United Kingdom expressly left out what is "non-fungible," and in Korea Article 2, subparagraph 4 of the Enforcement Decree leaves it out but has a proviso that brings back in what the Financial Services Commission designates by notice (e.g., tokens "지급수단으로 사용될 수 있는," "that can be used as a means of payment"). How does that proviso apply to ERC-721? Under "발행인이 사용처와 그 용도를 제한한 것" (gloss: "one whose places of use and purposes have been restricted by the issuer") (item (a)), what happens to a token that has no "발행인" ("issuer")? Under "전자적으로 거래 또는 이전될 수 있는" (gloss: "that can be traded or transferred electronically"), is a balance on an exchange's books (the 300,000 BTC that were not on the chain, Chapter 26) an "이전될 수 있는 전자적 증표" (gloss: "an electronic token that can be transferred"), or a claim against the issuer (the exchange)? How is the Japanese provision's 「不特定の者を相手方として購入及び売却を行うことができる」 (gloss: "can be bought and sold with unspecified persons as counterparties") to be read for an AMM (Chapter 15), where there is no counterparty? Is this a gap to be filled by analogy, a void that legislation must fill, or a signal to accept the fact that the definitional provision **does not determine the nature of the thing**, and to go back to the discussion of its nature in 27.2? This book does not answer; it points to the map of scholarly positions in Part 5. → Chapter 27

6. Summary

Set the definitional provisions of the five jurisdictions side by side and you see a common skeleton (a digital representation of value and electronic transferability) and exclusion lists that diverge. All of them keep currency outside, but at different levels; all of them draw a boundary with securities, but in different ways; and three of them leave out non-fungible tokens, but at different levels. The name decides jurisdiction (who oversees it), taxation (as what it is taxed) and criminal law (of which offense it is the object), and it has changed even within a single jurisdiction. None of those names is contained in the bit strings, signatures, state transitions and equilibria that this book showed in Parts 1 to 3. What a lawyer takes from this chapter is the habit of looking at two things together when reading a name. One is in **which provision of which law** the name is written and **from when it has been in force**; the other is **what that provision left out**. The concordance in Appendix C compresses this chapter into rows and columns, and the survey table in 27.3 records the agencies and procedures behind the names. Read the three on top of one another and you can see which questions are left empty in which jurisdiction.

Sources for this chapter

- Act on the Protection of Virtual Asset Users, Article 2, subparagraph 1 (wording as enacted by Act No. 19563; current Act No. 21857); Act on Reporting and Using Specified Financial Transaction Information, Article 2, subparagraph 3 (the version in force on March 25, 2021, MST 215971, and the current version, MST 290725); Income Tax Act, Article 21(1), subparagraph 27 (in force January 1, 2027; not yet in force). Korean Law Information Center (Open API, accessed 2026-10-25). The provisions are reproduced as they are, with line breaks added only between items.
- Regulation (EU) 2023/1114 (MiCA), OJ L 150, 9.6.2023, p. 40: Art. 2(3)–(4), 3(1)(5)–(9), 149.
- *SEC v. W. J. Howey Co.*, 328 U.S. 293, 298–299 (1946).
- *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. 2018), pp. 3, 24.
- IRS Notice 2014-21, 2014-16 I.R.B. 938; Notice 2023-34, 2023-19 I.R.B. 837. `irs-2023-notice-2023-34-modification-2014-21.pdf`.
- GENIUS Act, Pub. L. No. 119-27, 139 Stat. 419 (2025): §2(6), (7) and (22), §17, §20.
- *Uniform Commercial Code Amendments (2022)*, §12-102.
- 資金決済に関する法律 第2条第5項 and 第14項, 第63条の2; 金融商品取引法 第2条第3項; 金商法 第2条第49項 in the version reflecting 令和八年法律第六十四号, not yet in force. e-Gov Law API v2 (accessed 2026-10-25). The first sentences of 第2条第5項 in the versions of 2020-04-30 and 2020-05-01 are from the record of `asof=` queries.
- Financial Services and Markets Act 2000, s. 417(1); Financial Services and Markets Act 2023, s. 69. legislation.gov.uk (accessed 2026-10-25). SI 2026/102: reg. 1, RAO arts. 88F and 88G. Property (Digital Assets etc) Act 2025 (c. 29), ss. 1–2.
- FATF, *Updated Guidance for a Risk-Based Approach to Virtual Assets and VASPs* (2021-10-28), Glossary. SEC Release 81207, footnote 5 (reproducing the 2014 FATF definition).

- CJEU, *Skatteverket v David Hedqvist*, C-264/14 (22 October 2015), ¶¶24, 49, 52, operative part.
- Supreme Court of Korea, 2018Do3619, May 30, 2018; Supreme Court of Korea, 2021Do9855, November 11, 2021; Supreme Court of Korea, 2020Do9789, December 16, 2021. Full texts from the Korean Law Information Center case law database.
- Tokyo District Court (東京地方裁判所), judgment of 平成31年3月15日 (平成27年(刑わ)第2274号 etc.); Tokyo High Court (東京高等裁判所), judgment of 令和2年6月11日 (令和元年(う)第948号). The same files as in Chapter 26.
- *United States v. Bankman-Fried*, ECF No. 424 (S.D.N.Y. 2024-03-28); *United States v. Kwon*, ECF No. 63 (S.D.N.Y. 2025-12-11). The same files as in Chapter 26.
- This chapter has no code or notebook (by design). No official translation of any provision is given; the Korean and Japanese provisions carry working glosses (Section 1). This book has not checked the legislative reasons for renaming 「仮想通貨」 as 「暗号資産」, the original text of the definitional provisions of the US securities and commodities statutes, the adoption of UCC Article 12 by each state, or whether the implementing regulations of the GENIUS Act have been published.

Notes

1. Act on the Protection of Virtual Asset Users (Act No. 19563, promulgated July 18, 2023, in force July 19, 2024; current Act No. 21857, promulgated August 4, 2026, in force October 2, 2026), Article 2, subparagraph 1. Korean Law Information Center (국가법령정보센터) (Open API, MST 290735, Article 2, accessed 2026-10-25). The amendment history of Article 2 has one entry only, the enactment (article history for law ID 014474). The defined term is "가상자산," and the Act does not call virtual assets "통화" (tonghwa, "currency") or "화폐" (hwapye, "money"). Items (c) and (g) **exclude** "전자화폐" ("electronic currency") and "전자적 형태의 화폐" ("money in electronic form").

2. Act on Reporting and Using Specified Financial Transaction Information, Article 2, subparagraph 3. The version in force on March 25, 2021 (Act No. 17113, MST 215971, `get_law_text(mst=215971, jo=제2조, efYd=20210325)`) and the current version (MST 290725, in force October 2, 2026) were each opened at the Korean Law Information Center and compared (2026-10-25). The two versions differ in two respects. Into Article 2, subparagraph 1 of the current Virtual Asset User Protection Act, "사. 「한국은행법」에 따른 한국은행(...)이 발행하는 전자적 형태의 화폐 및 그와 관련된 서비스" (working gloss, not an official translation; the original controls: item (g) of the gloss above) was inserted, so that the old item 사 [(g)] became item 아 [(h)]; and the Specified Financial Transaction Information Act handed the definition over to the Virtual Asset User Protection Act and kept only the cross-referencing wording. Item 하 of Article 2, subparagraph 1 of the 2021 version defined "가상자산사업자" (gasang jasan saeopja, "virtual asset service provider") as "가상자산과 관련하여 다음 1)부터 6)까지의 어느 하나에 해당하는 행위를 영업으로 하는 자" (gloss: "a person who engages, as a business, in any of the following acts 1) to 6) in relation to virtual assets").

3. Income Tax Act, Article 21(1), subparagraph 27 (inserted by Act No. 17757; the date of entry into force in Article 1, subparagraph 2 of the Addenda was amended by Act Nos. 18578, 19196 and 20615 from "2022년 1월 1일" to "2023년 1월 1일" to "2025년 1월 1일" to "2027년 1월 1일," that is, from January 1, 2022 to January 1, 2023 to January 1, 2025 to January 1, 2027). The "현행" ("current") text at the Korean Law Information Center (MST 280405) shows this subparagraph as if it were in force, but the date of entry into force confirmed from the original text of the Addenda is January 1, 2027 (footnote 29 of Chapter 27). The provision itself mentions only "양도하거나 대여" ("transferring or lending").

4. Regulation (EU) 2023/1114 of the European Parliament and of the Council of 31 May 2023 on markets in crypto-assets, OJ L 150, 9.6.2023, p. 40, Art. 3(1)(5)–(9) (L 150/63), Art. 2(3)–(4) (L 150/62–63), Art. 149(2)–(3) (L 150/183). (Cellar PDF). The defined term is "crypto-asset," and "currency" is used only in "official currency" (point (8)). A crypto-asset is a "digital representation" of a "value or right."

5. *SEC v. W. J. Howey Co.*, 328 U.S. 293, 298–299 (1946) (decided May 27, 1946). This book did not open the text of

the definition of "security" in Section 2(a)(1) of the Securities Act of 1933 (15 U.S.C. § 77b(a)(1)) itself. It confirmed only the location, from the SEC report (2017, p. 11), which states: "Under Section 2(a)(1) of the Securities Act and Section 3(a)(10) of the Exchange Act, a security includes 'an investment contract.'"

6. *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. Mar. 6, 2018), p. 24. This book did not open the text of 7 U.S.C. § 1a(9) itself; it reproduces the wording quoted in the decision (the decision writes "§ 1(a)(9)").

7. IRS Notice 2014-21, 2014-16 I.R.B. 938, Section 2 Background (p. 1) and Section 4 Q&A-1 (p. 2); IRS Notice 2023-34, 2023-19 I.R.B. 837, Section 3 (pp. 2–3). `irs-2023-notice-2023-34-modification-2014-21.pdf`. The reason given in §3 of Notice 2023-34: "The Department of the Treasury and the Internal Revenue Service are aware that certain foreign jurisdictions have enacted laws that characterize Bitcoin as legal tender." The Notice does not say which jurisdictions, and neither does this book.

8. GENIUS Act, Pub. L. No. 119-27, §2(6) (139 Stat. 419), §2(22) (139 Stat. 421–422), §20 (effective date). (govinfo). §17 contains amendments that exclude the payment stablecoins of permitted issuers from "security" and "commodity" (footnote 20 of Chapter 27).

9. *Uniform Commercial Code Amendments (2022)*, Article 12, §12-102(a)(1), annotated edition, p. 235. Footnote 11 of Chapter 27.

10. 資金決済に関する法律 第二条第十四項 (e-Gov Law API v2, current version `421AC0000000059_20260812_508AC0000000064`); 第二条第五項 in the versions of 2020-04-30 and 2020-05-01 (wording confirmed by `asof=` queries); 金融商品取引法 第二条第三項 (current version `323AC0000000025_20260812_508AC0000000064`); 金商法 第二条第四十九項 in the version reflecting 令和八年法律第六十四号, not yet in force (`323AC0000000025_20270722_508AC0000000064`, `current_revision_status: UnEnforced`). All accessed 2026-10-25. Assets denominated in legal tender are defined separately, in 第二条第五項 of the same Act, as 「電子決済手段」 (denshi kessai shudan, "electronic payment instruments") (「通貨建資産に限り」, gloss: "limited to currency-denominated assets"). This book has not confirmed from primary materials the legislative reasons for renaming 「仮想通貨」 as 「暗号資産」.

11. Financial Services and Markets Act 2000, s. 417(1) "cryptoasset"; Financial Services and Markets Act 2023, s. 69(4) and Commencement Information "I2 S. 69 in force at 29.8.2023 by S.I. 2023/779, reg. 4(uu)". legislation.gov.uk (HTML, accessed 2026-10-25, compared word for word). Footnote 39 of Chapter 27.

12. SI 2026/102, reg. 1(2)–(3); arts. 88F and 88G, which these Regulations insert into the Regulated Activities Order (SI 2001/544) (pp. 36–37). Footnote 40 of Chapter 27.

13. Property (Digital Assets etc) Act 2025 (c. 29), s. 1 and s. 2(1)–(2). Footnote 9 of Chapter 27.

14. FATF, *Updated Guidance for a Risk-Based Approach to Virtual Assets and Virtual Asset Service Providers* (Paris, 28 October 2021), Glossary (the glossary at the end of the document). The publication page warns that this guidance does not reflect the 2025 amendment of Recommendation 1 and other changes. The 2014 FATF report's definition of "virtual currency" (reproduced in footnote 5 of the 2017 SEC report) was written in terms of functions, "(1) a medium of exchange; and/or (2) a unit of account; and/or (3) a store of value", and the 2021 glossary changed this to "payment or investment purposes." It is a move from functions to uses, and this book does not assess what it means.

15. Enforcement Decree of the Act on the Protection of Virtual Asset Users (Presidential Decree, promulgated September 29, 2026, in force October 2, 2026), Article 2 (Scope excluded from virtual assets), subparagraph 4. Original: "4. 수집을 주된 목적으로 하는 전자적 증표, 거래 당사자 간의 거래 확인만을 목적으로 하는 전자적 증표 등 단일하게 존재하여 다른 전자적 증표로 대체할 수 없는 전자적 증표. 다만, 특정 재화나 서비스의 지급수단으로 사용될 수 있는 전자적 증표 등 금융위원회가 정하여 고시하는 전자적 증표는 제외한다." Working gloss: "4. an electronic token that exists singly and cannot be replaced by another electronic token, such as an electronic token whose main purpose is collection or an electronic token whose sole purpose is to confirm a transaction between the parties to it; provided, however, that electronic tokens determined and publicly notified by the Financial Services Commission, such as an electronic token that can be used as a means of payment for specific goods or services, are excluded." Subparagraphs 1 (electronic claims), 2 (mobile gift certificates), 3 (deposits etc. on the Bank of Korea's network) and 5 (Financial Services Commission notice) of the same Article are also on the exclusion list. Korean Law Information Center Open API (MST 290171, Article 2) (accessed 2026-10-25). This book did not check the Financial Services Commission's notice.

16. *CFTC v. McDonnell*, 287 F. Supp. 3d 213 (E.D.N.Y. 2018), pp. 3, 24; footnote 21 of Chapter 27. For where the jurisdictional provisions are, see the table and footnotes of 27.3.

17. IRS Notice 2014-21, §4 Q&A-1 and Q&A-2; footnote 14 of Chapter 27.

18. CJEU, *Skatteverket v David Hedqvist*, C-264/14 (22 October 2015), ¶¶24, 49, 52, operative part 2. ¶49: "Transactions involving non-traditional currencies, that is to say, currencies other than those that are legal tender in one or more countries, in so far as those currencies have been accepted by the parties to a transaction as an alternative to legal tender and have no purpose other than to be a means of payment, are financial transactions." Footnote 13 of Chapter 27.

19. Supreme Court of Korea, 2021Do9855, November 11, 2021, Reasons 2.가; Supreme Court of Korea, 2020Do9789, December 16, 2021, Reasons 3.라; Supreme Court of Korea, 2018Do3619, May 30, 2018, Reasons 1.가.5); footnotes 22, 15 and 23 of Chapter 27. The 2018 judgment writes "이른바 '가상화폐'의 일종" (gloss: "a kind of so-called 'virtual currency'"), and the two 2021 judgments write "가상자산의 일종" (gloss: "a kind of virtual asset") and "가상자산은 … 재산상 이익에 해당한다" (gloss: "virtual assets … constitute a pecuniary advantage"). These are judgments given after Act No. 17113 amending the Specified Financial Transaction Information Act (in force March 25, 2021) brought "가상자산" in as a statutory term. This book does not say that the choice of word affected the decision; the judgments give no reason for it. (대법원 2021. 11. 11. 선고 2021도9855 판결; 대법원 2021. 12. 16. 선고 2020도9789 판결; 대법원 2018. 5. 30. 선고 2018도3619 판결)

20. Tokyo District Court (東京地方裁判所), judgment of 平成31年3月15日 (March 15, 2019) (平成27年(刑わ)第2274号 etc.), 「事実認定の補足説明」 ("supplementary explanation of the findings of fact") 第１の２(１): 「ビットコインは，インターネット上で流通する，いわゆる仮装通貨である。」 (as written in the original judgment; gloss: "Bitcoin is a so-called 仮装通貨 that circulates on the Internet."); Tokyo High Court (東京高等裁判所), judgment of 令和2年6月11日 (June 11, 2020) (令和元年(う)第948号), p. 1: 「本件は，暗号資産（仮想通貨）であるビットコインの取引所を運営していた会社の代表取締役であった被告人が」 (gloss: "This case is one in which the defendant, who was the representative director of a company that operated an exchange for bitcoin, which is a crypto-asset (angō shisan) (virtual currency), …"). Footnotes 9 and 10 of Chapter 26. The sentence on 「架空のビットコイン」 is in 第２の３(３)ア(ア) of the first-instance judgment (footnote 14 of Chapter 26).

21. *United States v. Samuel Bankman-Fried*, No. 1:(S6)22-CR-673-001 (LAK) (S.D.N.Y.), Judgment, ECF No. 424 (2024-03-28), pp. 1–2: "18 U.S.C. § 371 Securities Fraud Conspiracy … (S6) 5 / 18 U.S.C. § 371 Commodities Fraud Conspiracy … (S6)6"; *United States v. Do Hyeong Kwon*, No. 1:23-cr-00151-PAE (S.D.N.Y.), Judgment, ECF No. 63 (2025-12-11): "18 U.S.C. § 371 Conspiracy to Defraud … 1s / 18 U.S.C. § 1343 and 2 Wire Fraud … 4s". Footnotes 35 and 28 of Chapter 26.

APPENDICES

Appendix A. Trend notes (as of 2026-10): index (entries from Parts 1–6 and Part 0)

> Any sentence in the text that says "as of 2026-10" is entered here as soon as it is written (the updating rule in the notes to the reader in Chapter 3). When the edition changes, only the items in this table are updated. Reference date 2026-10-25. There are no items on prices or market capitalization.

#	CHAPTER AND SECTION	STATEMENT OF PRINCIPLE IN THE TEXT	CURRENT STATE (AS OF 2026-10)	PRIMARY SOURCE	TO DO IN THE NEXT EDITION
A-1	Chapter 4, Section 5	SHA-256 is the algorithm specified in FIPS 180-4	The current version is the 2015-08 edition. NIST decided on a revision in a Planning Note of 2023-03-07	https://csrc.nist.gov/pubs/fips/180-4/upd1/final	When the revised edition is published, update the edition and year
A-2	Chapter 5, Section 3	Bitcoin uses ECDSA and Schnorr signatures side by side	BIP-340 and 341 Status Deployed	github.com/bitcoin/bips (bip-0340 commit 2885f13d, bip-0341 commit 24e96e87)	Check only
A-3	Chapter 6, Section 4; Chapter 8, Section 3	BIP-54 seeks to fix the 64-byte transaction weakness of Merkle trees and the timewarp	BIP-54 "Consensus Cleanup" Status **Complete** (no deployment section; not activated)	github.com/bitcoin/bips bip-0054.md commit 927b6de9	If it changes to Deployed, revise the sentences in Chapters 6 and 8
A-4	Chapter 8, Section 3; Chapter 12, Section 5	Difficulty is adjusted every 2,016 blocks	Block 972,958 (observed 2026-10-24 UTC): nBits 0x17020093, difficulty 140,577,680,051,919.6, hash rate estimated under the 600-second assumption ≈1.006×10^21 H/s (comparison value 1.077×10^21)	`data/hashrate_2026-10-25.json` (raw responses from Blockstream and mempool.space)	Replace the snapshot every half year; in the text, give only the order of magnitude

#	CHAPTER AND SECTION	STATEMENT OF PRINCIPLE IN THE TEXT	CURRENT STATE (AS OF 2026-10)	PRIMARY SOURCE	TO DO IN THE NEXT EDITION
A-5	Chapter 8, Section 5	The energy debate is given in its structure only, without figures	(not in the text)	—	In the body of the trend notes, describe only the two estimation methods (back-calculation from the hash rate vs. assumptions about hardware efficiency); figures only with their source and reference date
A-6	Chapter 10, Section 3	Proof-of-stake constants change with each fork	Fulu active (2025-12-03, epoch 411392): MIN_ACTIVATION_BALANCE 32 ETH, MAX_EFFECTIVE_BALANCE_ELECTRA 2,048 ETH, slashing share 1/4096, whistleblower 1/4096, proportional multiplier 3. Gloas not scheduled	consensus-specs master b834fa84; `widgets/data.json`	Recheck the constants when the next fork activates
A-7	Chapter 10, Section 3	Ethereum moved to PoS in September 2022	Fork chronology: Altair 2021-10-27 → Bellatrix 2022-09-06 → (TTD reached, estimated 2022-09-15) → Capella 2023-04-12 → Deneb 2024-03-13 → Electra 2025-05-07 → Fulu 2025-12-03	consensus-specs configs/mainnet.yaml	No change; check only
A-8	Chapter 5, Section 3	Taproot was activated in 2021	Core chainparams minimum activation height 709,632 ("Approximately November 12th, 2021")	Bitcoin Core v31.1 chainparams.cpp	No change
A-9	Chapter 17, Section 4	Proposer–builder separation runs on middleware and relays outside the protocol	ethereum.org PBS roadmap page (updated 2026-06-24): states that there is no finalized specification and that inclusion lists are under research; mev-boost README: states that it will be enshrined in the protocol in the future	https://ethereum.org/en/roadmap/pbs/ (source commit 26bdbe8e); https://github.com/flashbots/mev-boost/blob/7a60459f1e61ee4d3b256978cd34bef5fe5e95c6/README.md	When a specification is finalized and deployed, revise the sentence in Chapter 17, Section 4 and the paragraph in the same section on what the division of labor does not change

#	CHAPTER AND SECTION	STATEMENT OF PRINCIPLE IN THE TEXT	CURRENT STATE (AS OF 2026-10)	PRIMARY SOURCE	TO DO IN THE NEXT EDITION
A-10	Chapter 18, Section 1	Throughput ceiling = capacity per block ÷ block interval	Observed gas limit 60,000,000 (block 26,264,275, 2026-10-24T18:23Z, 3 RPC endpoints agree), slot 12,000ms (`SLOT_DURATION_MS`); the same configuration file has a separate value `SLOT_DURATION_MS_EIP8198: 10000` , and `EIP8198_FORK_EPOCH` at L69 is set to the maximum value (2^64−1, unspecified) (content not verified)	`widgets/data.json` ; consensus-specs b834fa84 `configs/mainnet.yaml` L74, L118	Re-observe the gas limit every half year; if 10-second slots are activated, recheck the ceiling calculation in Chapter 18, Section 1 and Chapter 21
A-11	Chapter 18, Section 2	Blob data is guaranteed only for a minimum availability period and is not kept permanently	`MIN_EPOCHS_FOR_BLOB_SIDECARS_REQUESTS: 4096` → 4,096 × 32 × 12 seconds ≈ 18.2 days (ethereum.org gives roughly 18 days)	consensus-specs b834fa84 `configs/mainnet.yaml` L194; https://ethereum.org/en/developers/docs/scaling/zk-rollups/	If the constant or the slot interval changes, recompute the number of days
A-12	Chapter 18, Section 3	In an optimistic rollup, withdrawing to the main chain requires waiting out the challenge period	ethereum.org: roughly 7 days (a general statement; may differ by rollup)	https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/ (source commit 3e3b0dec)	In the body of the trend notes, give each rollup's actual challenge period, with source and reference date
A-13	Chapter 18, Section 6	Each rollup has a different trust boundary (sequencer, data-availability layer, upgrade authority)	(not in the text; a task)	—	In the body of the trend notes, give the current state of each rollup (concentration of the sequencer, whether fraud proofs are permissioned, choice of DA, upgrade keys), with primary sources and reference dates. **TVL and market capitalization are price data and are not written**
A-14	Chapter 16, Section 5	Oracle manipulation happens at four layers: sources, node collusion, reading spot prices, and front-running of reports	(frequency not recorded; a task)	—	Once primary tallies by type of incident (with count, period, and method stated) are obtained, put them in the trend notes together with the judgment "it is common." Mark secondary tallies with 🟡

#	CHAPTER AND SECTION	STATEMENT OF PRINCIPLE IN THE TEXT	CURRENT STATE (AS OF 2026-10)	PRIMARY SOURCE	TO DO IN THE NEXT EDITION
A-15	Chapter 13, Section 3	The gas fee schedule changes with each fork	The values in the text follow Appendix G of the Shanghai edition of the Yellow Paper (2025-02-04); later forks, such as type-3 (blob) transactions, are not reflected	https://ethereum.github.io/yellowpaper/paper.pdf	Compare with the current list of EIPs and put changed values and new transaction formats in the trend notes; when a new edition of the Yellow Paper is published, update the edition cited in the Chapter 13 footnotes
A-16	Chapter 20, Section 2	The block subsidy halves every 210,000 blocks (Core `GetBlockSubsidy`, integer shift)	Observed height 972,958 (2026-10-24T18:17Z) ÷ 210,000 = 4 → subsidy 3.125 BTC (computed from the rule in the source code, not checked directly against a coinbase output). Next halving height 1,050,000	`data/hashrate_2026-10-25.json` (raw responses from Blockstream and mempool.space); Bitcoin Core v31.1 `src/kernel/chainparams.cpp` L84	After height 1,050,000, update to 1.5625 BTC. The reward is not converted into legal tender in the text
A-17	Chapter 21, Section 5	The blob fee market is a separate resource, a separate target, exponential updating, and burning	The constants in the text are the values in the original EIP-4844 (target 3 and maximum 6 blobs, 393,216/786,432 blob gas, update denominator 3,338,477). In the checking it was noted that after the fork active on the reference date (Fusaka), bpo1 and bpo2 change only blob parameters, and **the current target and maximum numbers of blobs on mainnet on the reference date were not checked in this edition** (a task)	https://eips.ethereum.org/EIPS/eip-4844	Check the current blob schedule (BLOB_SCHEDULE) values in consensus-specs and execution-specs as primary sources, and give "original value / current value" side by side in the Chapter 21 footnote
A-18	Chapter 24, Section 6	Redemption pressure can be transmitted to the short-term Treasury market through sales of reserve Treasuries	(size of the sector and issuers' Treasury holdings not recorded; a task) BIS WP1270, revised 2026-06: daily data 2021-01 to 2026-03; per inflow of USD 3.5 billion (2σ), the 3-month yield moves −0.71bp immediately and by up to −4bp within 10 days; the effect is larger under stress in the Treasury market and grows with the size of the sector	(abstract)	Put the size of the sector, as **outstanding issuance** (the term market capitalization is not used), and issuers' Treasury holdings, from issuers' reserve disclosures and U.S. Department of the Treasury data, in the body of the trend notes with a reference date. Price data are not used

#	CHAPTER AND SECTION	STATEMENT OF PRINCIPLE IN THE TEXT	CURRENT STATE (AS OF 2026-10)	PRIMARY SOURCE	TO DO IN THE NEXT EDITION
A-19	Chapter 26, Sections 2 and 6	In the Mt.Gox criminal case, the text of the judgments was checked through the first instance (Tokyo District Court (東京地裁), March 15, 2019) and the appeal (Tokyo High Court (東京高裁), June 11, 2020, appeal dismissed)	Whether a final appeal was filed and the date the judgment became final are **not verified** (as of 2026-10). The U.S. criminal proceedings (the indictment of Vinnik described in the trustee's report) were not checked against the original		When a decision of the Supreme Court of Japan (最高裁判所) is confirmed, correct "final appeal and finality not verified" in Chapter 26, Section 2 and in the table in Section 6
A-20	Chapter 26, Sections 5 and 6	The FTX criminal case: through the first-instance judgment (S.D.N.Y. 2024-03-28) and its affirmance on appeal (2d Cir. 2026-06-12)	Whether a petition for a writ of certiorari was filed is **not verified** (as of 2026-10)		When a disposition of the Supreme Court of the United States is confirmed, update the table in Chapter 26
A-21	27.3; Chapter 30, 2.1	Article 21(1), subparagraph 27 (income from virtual assets) of Korea's Income Tax Act (소득세법) has been promulgated but is not yet in force	Effective January 1, 2027 (deferred a third time by Act No. 20615; not in force as of 2026-10). The "current" text on the Korean Law Information Center (국가법령정보센터) displays it as if it were in force	Korean Law Information Center Open API (MST 280405) + text of the amendment to the addenda	After it takes effect on 2027-01-01, change "not in force" to "in force"; if a law deferring it further is promulgated, update the date
A-22	27.3; Chapter 30, 2.4	The Japanese definition of 暗号資産 (angō shisan, "crypto-asset") is in 資金決済法 第2条第14項 (Payment Services Act, Article 2(14))	The part of 令和8年法律第64号 (Act No. 64 of 2026) (promulgated 2026-07-23) that moves the definition to 金商法 第2条第49項 (Financial Instruments and Exchange Act, Article 2(49)) is **not in force** (e-Gov shows 2027-07-22; the cabinet order on the effective date not verified; as of 2026-10)	e-Gov API v2 `law_data/323AC0000000025_20270722_508AC0000000064`	When it takes effect, change the definitional-term row in 27.3 and Chapter 30, 2.4 to 金商法 第2条第49項, and 資金決済法 第2条第14項 to the definition of "電子決済手段サービス仲介業" (denshi kessai shudan sābisu chūkaigyō, "electronic payment instrument service intermediary business")

#	CHAPTER AND SECTION	STATEMENT OF PRINCIPLE IN THE TEXT	CURRENT STATE (AS OF 2026-10)	PRIMARY SOURCE	TO DO IN THE NEXT EDITION
A-23	27.3; Chapter 30, 2.3	The U.S. GENIUS Act (Pub. L. 119-27, 2025-07-18) has been enacted	Under §20 the effective date is the earlier of 18 months after enactment (at the latest 2027-01-18) or 120 days after the final regulations are issued; **whether the regulations have been issued is not verified** (as of 2026-10)	§20	When the date on which the primary Federal payment stablecoin regulators issue final regulations is confirmed, fix the effective date
A-24	27.3; Chapter 30, 2.5	UK SI 2026/102 (made 2026-02-04) is in force only for the part on FCA rule-making	Full commencement day **2027-10-25** (not yet in force as of 2026-10)	reg. 1(2)	After 2027-10-25, change "not yet in force" to "in force"; the FCA's final rules (PS26/9–13) are not cited in this edition
A-25	27.3, Korea	The Civil Execution Act (민사집행법) has no provision on virtual assets (as of the 2022 report)	Whether it has since been amended is **not verified** (a task as of 2026-10)	Lee Hye-jeong (이혜정) (2022)	Check the history of the Civil Execution Act on the Korean Law Information Center, and if a provision has been added, revise 27.3 and 28.1(a)
A-26	27.2 ⑨; 27.3, United States	UCC Article 12 (2022) is a model law	Adoption by the states **not verified** (a task as of 2026-10)		Check the ULC adoption table as a primary source and put it in the body of the trend notes
A-27	Chapter 29, Sections 4 and 6	Bitcoin's quantum transition consists of proposals in three strands: a new output type (BIP-360), a sunset for old signatures (BIP-361), and post-quantum signatures	BIP-360 "Pay-to-Merkle-Root (P2MR)" Status **Draft** (v0.12.1, Assigned 2024-12-18), BIP-361 "Post Quantum Migration and Legacy Signature Sunset" Status **Draft** (Informational, Assigned 2026-02-11, Requires "TBD Post Quantum Signature BIP"), BIP-54 Complete (no deployment section; A-3). None of these transition proposals has been activated on any mainnet (as of 2026-10; this is a different matter from the hash-based protective properties that already exist)	github.com/bitcoin/bips commit 100ba872 `bip-0360.mediawiki`, `bip-0361.mediawiki`, `README.mediawiki` L1286–1298	When the status changes to Final/Deployed/Rejected, update the sentences in Chapter 29, 4.1, 4.3, and Section 6 and the "actual structure" column for ⑤ in Appendix B

#	CHAPTER AND SECTION	STATEMENT OF PRINCIPLE IN THE TEXT	CURRENT STATE (AS OF 2026-10)	PRIMARY SOURCE	TO DO IN THE NEXT EDITION
A-28	Chapter 29, Sections 2 and 6	The resources Shor's algorithm needs for the ECDLP are estimated in logical qubits and gate counts, and the number of physical qubits is a function of five assumptions	Latest estimate: ≤1,200 logical qubits and ≤90 million Toffoli gates, fewer than 500,000 physical qubits under superconducting assumptions (Babbush et al. 2026-04, preprint, authors affiliated with Google and the EF); the official Ethereum page (source commit 643a7a45, 2026-10-23) says current hardware has thousands of noisy physical qubits. Expert survey (GRI and evolutionQ, "Quantum Threat Timeline Report 2025," published 2026-03, 26 respondents): asked how likely it is that a machine breaking RSA-2048 within 24 hours appears within 10 years, 19/26 answered more than 5% and 13/26 about 50% or more. This is opinion, not observation	; ethereum.org quantum-resistance page; https://globalriskinstitute.org/mp-files/pdf-quantum-threat-timeline-report-2025.pdf/ (accessed 2026-10-25)	When a peer-reviewed version is published or a demonstration on the scale of 1,000 logical qubits is reported, update the table in Chapter 29, Section 2. The survey is updated every year, so replace it with the latest edition
A-29	Chapter 29, 4.4 and Section 6	Ethereum is moving four areas (BLS, KZG, ECDSA, ZK) to post-quantum schemes, while account signatures are chosen account by account through account abstraction	EIP-8141 is under consideration for the Hegotá upgrade (the target year differs between two pages, 2027 on roadmap/security and the second half of 2026 on quantum-resistance; this book adopts neither); the target for the core post-quantum milestones is roughly 2029 (described as a planning target that may change); EF quantum team set up 2026-01; pq.ethereum.org roadmap I–*M* (updated 2026-06-27)	ethereum.org roadmap/security (source commit 290d753c, 2026-07-12) and quantum-resistance (commit 643a7a45, 2026-10-23); https://pq.ethereum.org/	When it is included in a fork and activated, update the sentence in Chapter 29, 4.4; if the 2029 target changes, update the year
A-30	Chapter 29, Section 3	Only coins whose public keys are exposed (by script type, reuse, or the mempool) are targets for Shor	Snapshot on the reference date (secondary tally 🟡, block 972,132, 2026-10-19): exposed 7,121,995 BTC = 35.4% of the supply issued on schedule (P2PK 1,715,077, P2TR 217,063, reuse 5,189,439). No primary reproduced value. Supplementary: Chaincode 2025-05 P2PK 1,720,747 (8.68%), Project Eleven 2025-01 6.26M; Deloitte (undated) 4M+ (25%); CoinShares 2026-02-06 ~1.6M (8%); BIP-361: 34% or more as of 2026-03-01; Babbush 2026 ~6.9M	`data/exposed_output_types_2026-10-25.json`, `data/README.md`	Replace the snapshot every half year; once a primary reproduced value is made by tallying UTXOs by type directly from a public node, remove the 🟡 mark

#	CHAPTER AND SECTION	STATEMENT OF PRINCIPLE IN THE TEXT	CURRENT STATE (AS OF 2026-10)	PRIMARY SOURCE	TO DO IN THE NEXT EDITION
A-31	Chapter 29, Section 6	(not in the text; a task) NIST's schedule for retiring ECDSA	The Ethereum page ("NIST anticipates deprecating ECDSA by 2030 and disallowing it by 2035") and BIP-360 ("according to NIST IR 8547, Elliptic Curve Cryptography (ECC) is planned to be disallowed within the US federal government after 2035") cite NIST IR 8547. The Ethereum page does not name the document, and in BIP-360 2030 is the CNSA 2.0 schedule and only 2035 is NIST IR 8547. This book has not opened the original of that document	(not verified)	Obtain the original of NIST IR 8547, put it in and add a line to Chapter 29, 4.2
A-32	Chapter 2, Section 11	The milestones after the white paper are treated from the original sources by the chapter responsible for each	Items from 2024 on recorded only as "institutional change": application of EU MiCA 2024-12-30 (Chapter 30), entry into force of Korea's Act on the Protection of Virtual Asset Users (가상자산 이용자 보호 등에 관한 법률) 2024-07-19 (Chapters 27 and 30), enactment of the U.S. GENIUS Act 2025-07-18 (A-23), Royal Assent to the UK Property (Digital Assets etc) Act 2025 on 2025-12-02 and SI 2026/102 made 2026-02-04 (A-24), promulgation of Japan's 令和8年法律第64号 (Act No. 64 of 2026) 2026-07-23 (A-22), publication of NIST FIPS 203–205 2024-08-13 (Chapter 29), BIP-360 and 361 Draft (A-27). **The approval and listing of spot exchange-traded funds are not recorded in any chapter, because this book could not obtain the primary sources (approval orders, listing notices)**: a task	The primary sources for each item are in the chapter concerned and in A-22 to A-24 and A-27	The table in Chapter 2 and Appendix D/W11 are closed at 2023. Later items are added only to this index. Spot exchange-traded funds are to be recorded only after official documents of the regulators (the original approval orders) are obtained, and their size and price are not to be written

APPENDICES

Appendix B. Common misconceptions: a verdict table

Twelve statements often heard about blockchains are set out in four columns: **verdict / why it looks that way / the actual structure / chapters**. There are three verdicts: **Holds** (the statement is true as it stands), **Holds in part** (one piece of the statement is true and another is false, so it is wrong if used as it stands) and **Does not hold** (the statement is not true). The "actual structure" column carries over verbatim, or summarizes, sentences that the relevant chapter wrote after checking the originals, and the original sources are in that chapter's footnotes. This table adds no new material. A verdict of "Does not hold" does not mean that the opposite statement is true. It means that the statement cannot be used as it stands.

#	STATEMENT	VERDICT	WHY IT LOOKS THAT WAY	THE ACTUAL STRUCTURE	CHAPTERS
①	A blockchain cannot be tampered with	**Holds in part**	Change one character in a past block and every block after it falls out of line, so it feels as though "it cannot be altered." Introductory phrases such as "an immutable ledger" reinforce the feeling	Tampering **shows when compared with** a hash and chain kept on record (hash chain), but the hash chain itself does not tell which of two histories is the original, and "shows" is not the same as "is prevented." A hash chain without proof of work can be re-threaded entirely at one hash per block (Chapter 7, NB07-1), and even with proof of work tampering is not impossible but **a cost**. Chapter 7: "The immutability of a blockchain is not a mathematical impossibility but **an economic cost**." That cost grows the further back the block lies, but it is finite, and once the share q exceeds 1/2, "is prevented" no longer holds (Chapter 9)	7·8·9
②	Smart contracts are contracts	**Holds in part**	The word "contract" is the name of a data type in the code, and the explanation that "the code performs automatically" sounds like the performance of a contract	On the technical side: the ten-line "contract" read in Chapter 13 "had no parties and no agreement"; it is a program that releases value when conditions are met. On the legal side: the primary source whose original this book checked (SEC Release No. 81207, 2017-07-25, p. 2) says only that "The automation of certain functions through this technology, 'smart contracts,' or computer code, does not remove conduct from the purview of the U.S. federal securities laws," and does not say whether code is a contract (28.1(c)). Code can be **a means of performing** a contract; whether it is a contract is decided outside the code. "Is the content of the contract the code or the intention?" is an open question in the box in Chapter 13	13·28.1(c)

#	STATEMENT	VERDICT	WHY IT LOOKS THAT WAY	THE ACTUAL STRUCTURE	CHAPTERS
③	Bitcoin is a Ponzi scheme	**Holds in part**	The pattern in which earlier entrants profit from the money of later entrants overlaps with trading on the secondary market, and the fact that the price rests on belief reads as a whiff of fraud	What the economics of Chapter 19 showed is **a listing of equilibria**: "An outside money with no dividend has both a zero-price equilibrium and positive-price equilibria, and the positive-price equilibria are held up by the belief that 'others will accept it' and are not pinned down to one." In the language of that literature, the value of fiat money itself is a "bubble" (a defined term), not a term of abuse. The same literature (van Oordt 2024, in the advanced box of Chapter 19) writes that "Investors who participate in a bubble equilibrium for a cryptocurrency with non-negative money growth experience Ponzi-scheme equivalent payoffs in the aggregate": one piece of the statement is true inside the model. The other piece is different. The issuance rules shown in Chapter 22 are written in code, and the design "resembles a company's articles but has no enforcer"; there is no **operator** that promises returns, raises funds and controls payments. What the jury found in the Terra case of Chapter 26 was a violation of the antifraud provisions of the securities laws, and the statements came not from the protocol but from the issuer on top of it. Whether the price went through recurring speculative phases is a statistical question for Chapter 25 and a separate one	19·22·25·26
④	Bitcoin encrypts nothing	**Holds**	The name "cryptocurrency" gives the impression that the content of transactions is encrypted, so this statement is not a misconception but one that **corrects** a misconception	Chapter 5: "Bitcoin's transactions and ledger are not encrypted." What is used is hashes (Chapter 4) and signatures (Chapter 5), not encryption. Every transaction is public (Chapters 1 and 11), and Chapter 12 read every byte of the genesis block as it stands. The scope, however, is "transactions and ledger"	4·5·12

#	STATEMENT	VERDICT	WHY IT LOOKS THAT WAY	THE ACTUAL STRUCTURE	CHAPTERS
⑤	When quantum computers arrive, Bitcoin is finished	**Holds in part**	It is true that Shor's algorithm breaks signatures, and that fact slides into "all of Bitcoin collapses"	What breaks is the **signatures** (coins whose public keys are exposed); hashes and proof of work withstand Grover's square-root speedup (Chapter 29, Section 1). By a secondary tally on the reference date, exposed coins of all types together come to about 1/3 of the supply, the firmest figure among them being the roughly 1.7 million BTC in P2PK (close to 9% of the total), and coins that are not exposed can also become targets while a spending transaction is out and waiting for confirmation; that window runs from a few minutes to a few hours and is not fixed (Chapter 29, Section 3). Post-quantum signature standards (FIPS 204 and 205) were published in 2024, but Bitcoin's transition proposals (BIP-360 and 361) are Drafts as of 2026-10 and have not been activated on either of the two mainnets (Chapter 29, Section 4; Appendix A-27). The problem is not "finished" but **the transition and the handling of the coins left behind**, and that handling is a question not of technology but of governance and law (Chapter 29, Section 5)	29
⑥	Faster computers will break the blockchain	**Does not hold**	The hash rate has grown by nearly fourteen orders of magnitude since 2009 (Chapter 8, NB08-3), which reads as "someday it will all be broken"	Chapter 8: "Mining is a lottery, not a puzzle, and even when machines get faster the winning interval is brought back to 10 minutes": difficulty is adjusted every 2,016 blocks. Chapter 8: the probability that the attacker finds the next block is the share q, and "Even if the total hash rate grows 10-fold, this probability stays the same as long as q is the same, and because difficulty adjustment brings the block interval back to 10 minutes, the shape of 'the race counted in blocks' also stays the same"; Chapter 9: the probability of a successful chase is a function of q and the number of confirmations z alone. The variable that breaks it is not absolute speed but **share**. The security of the hash function itself (preimage, collision) is a matter of the algorithm, not of speed (Chapter 4), and quantum speedup is taken up separately in ⑤	4·8·9
⑦	"6 confirmations " means final	**Does not hold**	Exchanges and wallets wait for 6 confirmations and then show "completed, " so it looks like a rule	Chapter 9: "'6 confirmations' is a convention, not a rule." The probability of reversal falls exponentially in z but never becomes 0, and at $q \geq 1 / 2$ it does not fall. The white paper's formula is optimistic, and the exact formula requires more confirmations for the same level of safety (6 rather than 5 at $q = 0.1$, and 32 rather than 24 at $q = 0.3$). There is no moment of "done," only a probability that gets smaller; where this fails to line up with the legal concepts of "performance" and "completion" is taken up in 28.1(c)	9·28.1(c)

#	STATEMENT	VERDICT	WHY IT LOOKS THAT WAY	THE ACTUAL STRUCTURE	CHAPTERS
⑧	Blockchains are anonymous	**Does not hold**	An address is a string, not a person's name, so it reads as "no one knows who it is"	Every transaction is **public** (Chapter 1: "transactions must be publicly announced"; Chapter 12 read every byte of the genesis block). Chapter 11: "In the UTXO model, privacy comes not from 'addresses being anonymous' but from 'the habit of switching addresses,' and that habit is broken by input merging and change patterns." It is not anonymous but **pseudonymous**, and §10 of the white paper itself writes "Some linking is still unavoidable with multi-input transactions …." The fact that there are output types in which the public key appears on the chain as it is (Chapter 29, Section 3) is evidence from the opposite side	1·11·12
⑨	Code is law	**Holds in part**	Smart contracts run as written and no one can stop them midway, so "the code is the rule" seems right	Chapter 16: "'Code is law' holds inside the chain, and whether a condition is true comes in from outside it." Even if an oracle feeds in a wrong value, the code does not deviate by a single line. Chapter 26 (The DAO): "The contract code was executed as written and betrayed its author's intention, and what reversed it was not code but the coordination of people": a majority took back, by a software update, a result the code had allowed. And the primary source writes that "The automation of certain functions through this technology, 'smart contracts,' or computer code, does not remove conduct from the purview of the U.S. federal securities laws" (SEC Release No. 81207, 2017-07-25, p. 2; 28.1(c)). True as a rule of execution inside the chain, false as a norm that keeps the law outside the chain from applying	16·26·28.1(c)
⑩	Proof of stake is a structure in which the rich get richer	**Holds in part**	Validators are chosen in proportion to their stake and rewarded in proportion too, so "those who have more get more"	Chapter 10: proportionality itself does not widen the **ratio** of the gap (NB10-1: if everyone grows at the same rate, the ratio stays the same). But the compounding of restaking rewards, economies of scale in running nodes, and the threshold of a 32 ETH minimum stake can work against small participants, and in proof of work, too, economies of scale in equipment and power work in the same direction. "Which system is more concentrated is a matter for measurement, and this book does not give those figures." Hence **Holds in part**	10

#	STATEMENT	VERDICT	WHY IT LOOKS THAT WAY	THE ACTUAL STRUCTURE	CHAPTERS
⑪	Stablecoins are stable	**Does not hold**	The name promises "stability," and the price stays near USD 1 for long stretches	Chapter 24: "A stablecoin is a token that carries a promise of redemption, and so it belongs not to the economics of outside money but to the economics of banking." "'Stability' is a property not of the price but of the balance sheet and the procedure." The collateral ratio is a measure of solvency and the redeemable ratio a measure of liquidity, and a run starts from the second; there is no deposit insurance and there is a secondary market, so closing the window does not stop a run. The two episodes of May 2022 (algorithmic) and March 2023 (fiat-collateralized) differ in structure, and this book does not say which coin is safe. Since the name does not guarantee stability, the statement as it stands does not hold	24·26
⑫	If it is decentralized, no one is responsible	**Does not hold**	On a network with no company and no representative, "whom do we hold responsible?" seems to have no answer	Having **no** responsible party is different from the responsible party being **invisible** (Chapter 23: a selfish miner can hide). Chapter 17: "the one who makes the block sets the order," and the rollups of Chapter 18 have someone who "decides the order" (the sequencer) and upgrade powers. Chapter 22: the record of governance tokens and The DAO is that majority voting, curators and splits were written in code but "there is no enforcer"; yet whoever holds the multisignature keys is "the de facto enforcer." Primary sources: the SEC (Release No. 81207, 2017-07-25) called The DAO "an unincorporated organization" and held it to be an issuer of securities, and the Court of Appeal of England and Wales (*Tulip Trading Ltd v van der Laan* [2023] EWCA Civ 83, 2023-02-03) held that whether Bitcoin developers owe fiduciary duties to holders is "a serious issue to be tried" (28.1(g)). It is not that no one is responsible; **who, in which position, is still open**	17·18·22·23·28.1(g)

Of the twelve statements, one **holds** (④), five **do not hold** (⑥, ⑦, ⑧, ⑪, ⑫) and six **hold in part** (①, ②, ③, ⑤, ⑨, ⑩). There are many "in part" verdicts because these statements usually carry one true technical fact over to a false conclusion: tampering shows (true) → cannot be tampered with (false); signatures break (true) → Bitcoin is finished (false); code runs as written (true) → code is law (false). What a lawyer should take from this table is less the verdicts themselves than **the habit of sorting out which piece is true**. When a party argues in litigation that "a blockchain cannot be tampered with, so this record is authentic," reading the true piece (tampering shows) apart from the false piece (impossible) is the starting point of 28.1(f).

Sources

- The "actual structure" in each row is carried over from the text, summaries and code output of the relevant chapter, and its original sources (the Nakamoto white paper, FIPS 180-4, Haber and Stornetta, Eyal and Sirer, Diamond and Dybvig, the SEC's 2017 report, *Tulip Trading*, BIP-360 and 361, Aggarwal et al., Babbush et al., and others) are in the footnotes of each chapter. This appendix adds no new material.
- The verdict framework is a reader's version, cut down to four columns, of the hypothesis verdict table from the research round before writing (five columns: "verdict / key evidence / evidence hierarchy / counter-evidence / observations that would change the verdict"). The verdicts of that round were used only for reference, and all the evidence was checked again against the chapters of the text and their original sources.
- There are no price, market capitalization or return data in any row.

APPENDICES

Appendix C. Concordance of technical and statutory terms

This table compresses into rows and columns what Chapter 30 read as sentences. The original texts of the provisions, their statute numbers and their status are carried over **word for word** from Sections 2 and 3 of Chapter 30, and this appendix adds no new provision and no new gloss (the working glosses on Korean and Japanese text are those of Chapter 30, copied word for word; the rule in Section 1 of Chapter 30). The reference date is 2026-10-25, and Appendix A (A-21 to A-24) tracks the provisions whose status changes. There are three tables: C.1, the defined terms of the five jurisdictions; C.2, the exclusion lists; C.3, which word of each jurisdiction each technical term reaches.

C.1 The defined terms of the five jurisdictions (Chapter 30, Section 2)

JURISDICTION	DEFINED TERM	PROVISION	STATUTE NUMBER · STATUS (2026-10-25)	SKELETON OF THE DEFINITION (EXCERPT FROM THE ORIGINAL)
Korea	"가상자산" (gasang jasan, "virtual asset")	Act on the Protection of Virtual Asset Users (가상자산 이용자 보호 등에 관한 법률), Article 2, subparagraph 1	Enacted by Act No. 19563 (promulgated July 18, 2023, in force July 19, 2024); current Act No. 21857 (in force October 2, 2026); **in force**	"경제적 가치를 지닌 것으로서 전자적으로 거래 또는 이전될 수 있는 전자적 증표(그에 관한 일체의 권리를 포함한다)" (Gloss: as something having economic value, an electronic token that can be traded or transferred electronically (including all rights relating to it))
Korea (cross-reference)	"가상자산"	Act on Reporting and Using Specified Financial Transaction Information (특정 금융거래정보의 보고 및 이용 등에 관한 법률), Article 2, subparagraph 3	Act No. 17113 (promulgated March 24, 2020, in force March 25, 2021) carried the text of the definition; cross-referencing wording from July 19, 2024	"「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산을 말한다" (Gloss: means a virtual asset under Article 2, subparagraph 1 of the Act on the Protection of Virtual Asset Users)
Korea (tax)	"가상자산소득" (gasang jasan sodeuk, "virtual asset income")	Income Tax Act (소득세법), Article 21(1), subparagraph 27	Inserted by Act No. 17757 (promulgated December 29, 2020); in force January 1, 2027; **not yet in force**	"「가상자산 이용자 보호 등에 관한 법률」 제2조제1호에 따른 가상자산(이하 "가상자산"이라 한다)을 양도하거나 대여함으로써 발생하는 소득(이하 "가상자산소득"이라 한다)" (Gloss: Income arising from transferring or lending virtual assets under Article 2, subparagraph 1 of the Act on the Protection of Virtual Asset Users (hereinafter "virtual assets") (hereinafter "virtual asset income"))

JURISDICTION	DEFINED TERM	PROVISION	STATUTE NUMBER · STATUS (2026-10-25)	SKELETON OF THE DEFINITION (EXCERPT FROM THE ORIGINAL)
EU	"crypto-asset"	Regulation (EU) 2023/1114 (MiCA), Art. 3(1)(5)	OJ L 150 (June 9, 2023); applies from 2024-12-30 under Article 149 (Titles III and IV from 2024-06-30); **applies**	"a digital representation of a value or of a right that is able to be transferred and stored electronically using distributed ledger technology or similar technology"
EU (subtypes)	"asset-referenced token," "e-money token," "utility token"	MiCA Art. 3(1)(6), (7), (9)	As above	(6) "purports to maintain a stable value by referencing another value or right or a combination thereof, including one or more official currencies"; (7) "purports to maintain a stable value by referencing the value of one official currency"; (9) "only intended to provide access to a good or a service supplied by its issuer"
United States (securities)	"investment contract"	"security" in Securities Act §2(a)(1), as defined in the *Howey* judgment of the Supreme Court of the United States	Case law (*SEC v. W. J. Howey Co.*, 328 U.S. 293, 298–299 (1946)); good law	"a contract, transaction or scheme whereby a person invests his money in a common enterprise and is led to expect profits solely from the efforts of the promoter or a third party"
United States (commodities)	"commodity"	7 U.S.C. § 1a(9), as quoted in the *CFTC v. McDonnell* decision	District court decision (E.D.N.Y. 2018); subsequent history in higher courts not verified	"all other goods and articles . . . in which contracts for future delivery are presently or in the future dealt in"
United States (tax)	"virtual currency" → "property"	IRS Notice 2014-21 §4 Q&A-1; Notice 2023-34 §3	Administrative interpretation; only a background sentence modified by Notice 2023-34; in effect	"For federal tax purposes, virtual currency is treated as property."

JURISDICTION	DEFINED TERM	PROVISION	STATUTE NUMBER · STATUS (2026-10-25)	SKELETON OF THE DEFINITION (EXCERPT FROM THE ORIGINAL)
United States (federal statute)	"digital asset," "payment stablecoin"	GENIUS Act §2(6), (22)	Pub. L. 119-27, 139 Stat. 419 (2025-07-18), enacted; under §20 the effective date is 2027-01-18 at the latest; whether the regulations have been published not verified	(6) "any digital representation of value that is recorded on a cryptographically secured distributed ledger"; (22)(A) "a digital asset— (i) that is, or is designed to be, used as a means of payment or settlement; and (ii) the issuer of which— (I) is obligated to convert, redeem, or repurchase for a fixed amount of monetary value, not including a digital asset denominated in a fixed amount of monetary value; and (II) represents that such issuer will maintain, or create the reasonable expectation that it will maintain, a stable value …"; (B) exclusion list in C.2
United States (private-law model act)	"controllable electronic record"	UCC Amendments (2022) §12-102(a)(1)	Model law; adoption by individual states not verified	"a record stored in an electronic medium that can be subjected to control under Section 12-105. The term does not include …" (exclusion list in Chapter 30, 2.3 (e))
Japan	「暗号資産」 (angō shisan, "crypto-asset")	Payment Services Act (資金決済に関する法律), 第2条第14項	平成21年法律第59号; renamed from 「仮想通貨」 to 「暗号資産」 by 令和元年法律第28号 (in force 2020-05-01); moved from 第5項 to 第14項 on 2023-06-01; **in force**. 令和8年法律第64号 (promulgated 2026-07-23), which moves the definition to 金商法第2条第49項, has been promulgated and is partly in force; **the part that transfers the definition is not yet in force as of 2026-10-25**	「物品等を購入し、若しくは借り受け、又は役務の提供を受ける場合に、これらの代価の弁済のために不特定の者に対して使用することができ、かつ、不特定の者を相手方として購入及び売却を行うことができる財産的価値（電子機器その他の物に電子的方法により記録されているものに限り、本邦通貨及び外国通貨、通貨建資産並びに電子決済手段（通貨建資産に該当するものを除く。）を除く。次号において同じ。）であって、電子情報処理組織を用いて移転することができるもの」 (Gloss: proprietary value (財産的価値) that can be used with unspecified persons (不特定の者) to pay the price when purchasing or borrowing goods, etc. or receiving the provision of services, and that can be bought and sold with unspecified persons as counterparties (limited to that which is recorded by electronic means on an electronic device or other object, and excluding Japanese currency and foreign currency, currency-denominated assets (通貨建資産) and electronic payment instruments (電子決済手段) (excluding those that fall under currency-denominated assets); the same applies in the following item), which can be transferred using an electronic data processing system (電子情報処理組織))

JURISDICTION	DEFINED TERM	PROVISION	STATUTE NUMBER · STATUS (2026-10-25)	SKELETON OF THE DEFINITION (EXCERPT FROM THE ORIGINAL)
United Kingdom (definition)	"cryptoasset"	Financial Services and Markets Act 2000, s. 417(1)	Inserted by FSMA 2023 s. 69(4)(a), in force 2023-08-29; **in force**	"any cryptographically secured digital representation of value or contractual rights that— (a) can be transferred, stored or traded electronically, and (b) that uses technology supporting the recording or storage of data (which may include distributed ledger technology)"
United Kingdom (regulated activities)	"qualifying cryptoasset," "qualifying stablecoin"	RAO arts. 88F, 88G (inserted by SI 2026/102)	SI 2026/102 made 2026-02-04; full commencement day 2027-10-25; **not yet in force** (only the parts for FCA rule-making are in force)	88F(2) "a cryptoasset which is— (a) fungible, (b) transferable, (c) not solely a record of value or contractual rights, including rights in another cryptoasset, and (d) not excluded by paragraph (4)"; 88G(2) "a qualifying cryptoasset where— (a) that cryptoasset seeks or purports to maintain a stable value in relation to a particular fiat currency ('the referenced fiat currency'), and (b) fiat currency … or other assets are held for the purpose of maintaining a stable value, in accordance with sub-paragraph (a)"
United Kingdom (property law)	(a confirmation, not a definition)	Property (Digital Assets etc) Act 2025 (c. 29), s. 1	Royal Assent 2025-12-02, in force the same day	"A thing (including a thing that is digital or electronic in nature) is not prevented from being the object of personal property rights merely because it is neither— (a) a thing in possession, nor (b) a thing in action."
International (reference)	"virtual asset"	FATF 2021 updated guidance, glossary	Recommendation; no legally binding force	"a digital representation of value that can be digitally traded, or transferred, and can be used for payment or investment purposes. Virtual assets do not include …" (the exclusion sentence is in Chapter 30, 2.6)

C.2 Exclusion lists (the table in Chapter 30, Section 3)

LEFT OUT	KOREA	EU	UNITED STATES (GENIUS)	JAPAN	UNITED KINGDOM (88F)
Legal tender, central bank digital currency	item (g) (money in electronic form issued by the Bank of Korea)	(8) "official currency" is a separate concept (no exclusion clause in the definition)	(B)(i) national currency, in the payment stablecoin definition only; no exclusion in (6) digital asset	本邦通貨及び外国通貨, 通貨建資産	(4)(c) currency … including a central bank digital currency
Electronic money, prepaid instruments	item (c)	(4)(c) funds (but EMTs are included)	— (deposits are left out separately by (B)(ii))	電子決済手段 (defined separately in 第5項)	(4)(b) electronic money
Securities, financial instruments	item (d) (electronically registered stocks, etc.), items (e) and (f)	(4)(a) financial instruments	(B)(iii) security	proviso: rights under 金商法 第29条の2第1項第8号 (電子記録移転権利)	(4)(a) specified investment cryptoasset
Use-restricted tokens	item (a) (the issuer restricted the places and purposes of use)	— (utility tokens are included, (9))	—	—	(4)(d) redeemable only with the issuer, limited network
Game outputs	item (b)	—	—	—	—
Non-fungible (NFT)	item (h) → Enforcement Decree §2(4) (what the Financial Services Commission (금융위원회) designates by public notice, e.g. tokens that can be used as means of payment, is brought back in by a proviso)	Art. 2(3) unique and not fungible	—	—	(2)(a) fungible requirement
Delegation	item (h) (Presidential Decree)	—	—	—	—

C.3 Which words the technical terms reach

The rows are the technical terms used in Parts 1 to 3 of this book, and the columns show which defined term or which exclusion item of each of the five jurisdictions the term reaches. "—" means that the provision of that jurisdiction has no word corresponding to that technical term; what Chapter 30 did not state is not stated here either. The last column is the **point of mismatch** that Chapter 30 identified.

TECHNICAL TERM (CHAPTER)	KOREA	EU	UNITED STATES	JAPAN	UNITED KINGDOM	POINT OF MISMATCH (CHAPTER 30)
Coins and tokens in general: the value of a UTXO output, an account balance (Chapter 11)	"가상자산" = "전자적으로 거래 또는 이전될 수 있는 전자적 증표" (Gloss: an electronic token that can be traded or transferred electronically)	"crypto-asset" = "digital representation of a value or of a right that is able to be transferred and stored electronically"	"digital asset" = "digital representation of value that is recorded on a cryptographically secured distributed ledger" (GENIUS); for tax, "property"; under securities and commodities law, according to the shape of the transaction	「暗号資産」 = 「財産的価値…であって、電子情報処理組織を用いて移転することができるもの」 (Gloss: proprietary value (財産的価値) … which can be transferred using an electronic data processing system (電子情報処理組織))	"cryptoasset" = "digital representation of value or contractual rights that … can be transferred, stored or traded electronically"	"Of the technology shown in Part 1, the provisions picked up **transferability** alone. Hashes, signatures, consensus rules and finality appear in none of the definitions. Only the EU, the United Kingdom and the US GENIUS Act put in technical words, 'distributed ledger technology' and 'cryptographically secured'; Korea and Japan are technology-neutral."
Blockchain, distributed ledger (Chapter 7)	— (technology-neutral)	"distributed ledger technology or similar technology"	"cryptographically secured distributed ledger" (GENIUS §2(6))	— (「電子情報処理組織」 (Gloss: electronic data processing system))	"technology supporting the recording or storage of data (which may include distributed ledger technology)"	Chapter 30, Section 3: "Only the EU, the United Kingdom and the US GENIUS Act put in technical words …"; the wording of the three provisions is as on the left
Legal tender, central bank digital currency (Chapter 19)	item (g) **excludes** it	"official currency" (8) is a separate concept	"national currency" is excluded from the payment stablecoin definition	「本邦通貨及び外国通貨」 (Gloss: Japanese currency and foreign currency) excluded	(4)(c) excluded	"'currency' is outside everywhere. … Law has kept the name 'currency' only for what it issues itself."

TECHNICAL TERM (CHAPTER)	KOREA	EU	UNITED STATES	JAPAN	UNITED KINGDOM	POINT OF MISMATCH (CHAPTER 30)
Stablecoins (Chapter 24)	(no definitional provision in Chapter 30; see the survey table in 27.3)	"asset-referenced token" (6), "e-money token" (7)	"payment stablecoin" §2(22)	Assets denominated in legal tender are defined separately as 「電子決済手段」 (第2条第5項, 「通貨建資産に限り」 (Gloss: limited to currency-denominated assets))	"qualifying stablecoin" 88G(2): "seeks or purports to maintain a stable value … and (b) … assets are held"	The defined terms of the three jurisdictions catch different things. EU (6) and (7) catch the **claim** of a stable value, and UK 88G(2) the claim plus reserve assets being held; only GENIUS §2(22)(A)(ii)(I) puts an **obligation to redeem** (the "promise to give it back" of Chapter 24) into the definition, and Japan took currency-denominated assets out altogether under a different word (wording in Chapter 30, Section 2)
Non-fungible tokens: the (contract address, number) pair of ERC-721 (Chapter 14)	Enforcement Decree §2(4) leaves them out but brings back in "특정 재화나 서비스의 지급수단으로 사용될 수 있는 전자적 증표 등 금융위원회가 정하여 고시하는 전자적 증표" (Gloss: electronic tokens determined and publicly notified by the Financial Services Commission, such as an electronic token that can be used as a means of payment for specific goods or services)	Art. 2(3) "unique and not fungible" excluded	—	—	88F(2)(a) "fungible" requirement	"Three jurisdictions left out non-fungible tokens, but at different levels." Whether an ERC-721 pair comes within the Financial Services Commission's notice as something "지급수단으로 사용될 수 있는" (Gloss: that can be used as a means of payment) depends on what the token points to and on the notice

TECHNICAL TERM (CHAPTER)	KOREA	EU	UNITED STATES	JAPAN	UNITED KINGDOM	POINT OF MISMATCH (CHAPTER 30)
What token standards do not fix: issuance, meaning (Chapter 14)	item (a) "발행인이 사용처와 그 용도를 제한한 것" (Gloss: one whose places of use and purposes have been restricted by the issuer) excluded	"utility token" (9) "access to a good or a service supplied by its issuer"	—	—	(4)(d) redeemable only with the issuer, limited network: excluded	The provisions presuppose **an issuer**. Chapter 30's question box asks: under "발행인이 사용처와 그 용도를 제한한 것" (item (a)), what happens to a token that has no "발행인" ("issuer")?
Securities character: token sales and investment contracts (Chapters 14, 22, 26)	item (d) "전자등록 주식등" ("electronically registered stocks, etc.") excluded	(4)(a) "financial instruments" excluded	if an "investment contract" (*Howey*), **inside** securities law; GENIUS (B)(iii) "security" excluded	proviso: 「電子記録移転権利」 excluded	(4)(a) "specified investment cryptoasset" excluded	"Korea, the EU, the United Kingdom and the GENIUS Act say 'if it is a security, it is outside this law,' while Japan leaves it out by bringing a concept of financial-instruments law, 「電子記録移転権利」, into a proviso. US securities law itself goes the other way: **if it is an 'investment contract,' it is inside this law**, so the same token becomes a security or not depending on the shape of the transaction"
Automated markets: an AMM without an operator (Chapter 15)	—	—	GENIUS §2(7) expressly excludes participation in liquidity pools from "digital asset service provider"	「不特定の者を相手方として購入及び売却を行うことができる」 (Gloss: can be bought and sold with unspecified persons as counterparties)	—	"The 'market without an operator' that the AMM of Chapter 15 and the MEV of Chapter 17 showed is a gap in jurisdiction, in that none of these names has an addressee there (28.1(e))"; Chapter 30's question box: how is the Japanese provision's 「不特定の者を相手方として」 (Gloss: with unspecified persons as counterparties) to be read for an AMM, where there is no counterparty?

TECHNICAL TERM (CHAPTER)	KOREA	EU	UNITED STATES	JAPAN	UNITED KINGDOM	POINT OF MISMATCH (CHAPTER 30)
Balances on an exchange's books: coins not on the chain (Chapter 26)	Is it a "전자적으로 거래 또는 이전될 수 있는 전자적 증표" (Gloss: an electronic token that can be traded or transferred electronically)?	—	—	Tokyo District Court: 「ブロックチェーン上に存在しないビットコインは…架空のビットコイン」 (Gloss: bitcoins that do not exist on the blockchain … fictitious bitcoins) (Chapter 26)	—	Chapter 30's question box: is a balance on an exchange's books (the 300,000 BTC that were not on the chain, Chapter 26) an "이전될 수 있는 전자적 증표" (Gloss: an electronic token that can be transferred), or a claim against the issuer (the exchange)?
The name "currency" itself (Chapters 19 and 30)	The Act does not call virtual assets "통화" (tonghwa, "currency") or "화폐" (hwapye, "money") (items (c) and (g) **exclude** "전자화폐" ("electronic currency") and "전자적 형태의 화폐" ("money in electronic form"))	"currency" only in "official currency" (8)	IRS: calls it "virtual currency" but taxes it as "property"; Notice 2023-34 deleted the sentence "it does not have legal tender status in any jurisdiction"	「仮想通貨」→「暗号資産」 (令和元年法律第28号)	—	"That choice has changed even within a single jurisdiction. Japan changed 「仮想通貨」 (kasō tsūka, 'virtual currency') to 「暗号資産」, the Supreme Court of Korea wrote '가상화폐' (gasang hwapye, 'virtual currency') in 2018 and '가상자산' in 2021, and the US Internal Revenue Service deleted in 2023 the statement that virtual currency 'does not have legal tender status in any jurisdiction.'"

Read the three tables on top of one another and the conclusion of Chapter 30 appears as it is. The common skeleton is "a digital representation of a value (or a right)" + "electronic transferability," the place where the definitions diverge is the exclusion list, and the only technical term that entered the provisions is transferability. Which column has many "—" entries shows which questions are left empty in which jurisdiction.

Sources

- Chapter 30, Section 2 (original text of the definitional provisions), Section 3 (table of exclusion lists), Section 4 (jurisdiction), Section 5 (changes of name) and the question box. The original texts of the provisions, the statute numbers, the dates of entry into force and the citations of judgments are all in the footnotes and "Sources for this chapter" of Chapter 30. This appendix adds no new provision, gloss or material.
- The survey table in 27.3 (agencies and procedures by jurisdiction) is outside the scope of this appendix. The agencies behind the names are in Chapter 27; the names themselves are in Chapter 30 and this appendix.

APPENDICES

Appendix D. Timeline, 1976–2023 (the still version of the clickable timeline W11)

This is a still version made with the same items and the same references as the table in Chapter 2, Section 1 (events that the Chapter 2 table groups in one row are split out here by date, and the chapter column lists every chapter that treats the event from its original text). The clickable timeline (W11) in the screen edition is drawn from the data of this table (`timeline` in `widgets/data.json`), and selecting an event takes you to the relevant chapter. The rules for years are the same as in Chapter 2, Section 1: when a work is cited with its conference, the year of the conference; when cited as a publication, the year of publication; for web documents, the date of posting. Dates that could not be fixed are not stated as certain, and that fact is recorded. Items from 2024 on date quickly and so are kept only in Appendix A (A-32). There is no price or market capitalization axis.

1. 29 milestones

DATE	MILESTONE	PRIMARY SOURCE (OPENED BY THIS BOOK)	CHAPTERS
1976-11	Diffie and Hellman, "New Directions in Cryptography": public-key cryptography and "the equivalent of a written signature"	*IEEE Trans. Inform. Theory* IT-22(6): 644–654	5
1980-04	Merkle, "Protocols for Public Key Cryptosystems": "tree authentication" to authenticate a list of public keys (detailed discussion deferred to another manuscript, [13])	*Proc. 1980 IEEE Symposium on Security and Privacy*, pp. 122–134, at pp. 126–127	6
1982-07	Lamport, Shostak and Pease, "The Byzantine Generals Problem"	*ACM TOPLAS* 4(3): 382–401	1·8
1982 (conference) / 1983 (published)	Chaum, "Blind Signatures for Untraceable Payments": blind-signature electronic cash	*Advances in Cryptology: Proceedings of Crypto 82*, pp. 199–203 (Springer, 1983)	1·2
1987 (conference) / 1988 (published)	Merkle, "A Digital Signature Based on a Conventional Encryption Function": tree signatures	*CRYPTO '87*, LNCS 293, pp. 369–378 (Springer, 1988)	6
1991	Haber and Stornetta, "How To Time-Stamp a Digital Document": a chain of time stamps	*J. Cryptology* 3(2): 99–111	4·7
1993	Bayer, Haber and Stornetta, "Improving the Efficiency and Reliability of Digital Time-Stamping": the chain turned into a tree	*Sequences II*, pp. 329–334 (Springer, 1993); the authors' typeset version is marked "March 1992"	6·7

DATE	MILESTONE	PRIMARY SOURCE (OPENED BY THIS BOOK)	CHAPTERS
1997-03-28	Back, Hashcash announcement (cypherpunks): a stamp whose cost is paid in computation	hashcash.org/papers/announce.txt; the 2002 paper gives "May 1997" (the discrepancy is discussed in a footnote to Chapter 2)	8
1998	Dai, "b-money": a picture in which money is created by computation and everyone holds the ledger	weidai.com/bmoney.txt (no date in the document; 1998 is the date in reference [1] of the white paper, with no month or day)	1·8
2005-12-29	Szabo, "Bit gold": proof of work, time stamps and a registry of ownership	unenumerated.blogspot.com/2005/12/bit-gold.html (posting date fixed by a Wayback snapshot of 2006-03-29; the date now displayed on the page is 2008-12-27)	8
2008-10-31	Nakamoto, "Bitcoin P2P e-cash paper": the white paper is released	cryptography@metzdowd.com 014810; white paper PDF	1·7·8·9
2009-01-03	The genesis block (nTime 1231006505 = 18:15:05 UTC)	`data/genesis_block.hex`; Bitcoin Core v31.1 `chainparams.cpp`	12
2014 (draft 2013)	Buterin, the Ethereum white paper (draft 2013, official version 2014)	Official PDF at ethereum.org; earliest Wayback capture 2013-12-19 (footnote to Chapter 2)	13
2014-02-28	Mt.Gox files a petition for civil rehabilitation (Tokyo District Court (東京地裁), 平成26年(再)第12号)	Debtor's announcement (Japanese and English)	26
2014-04-24	Order commencing bankruptcy proceedings against Mt.Gox (Tokyo District Court (東京地裁), 平成26年(フ)第3830号)	Original text of the order	26
2016-04-30 to 05-28	The DAO token sale	SEC Release No. 81207, p. 2	14·22·26
2016-06-17	The DAO attack (a recursive-call flaw)	Ethereum Foundation announcement of 2016-06-17; SEC Release No. 81207, p. 9	26
2016-07-20	Ethereum hard fork (block 1,920,000, "irregular state change")	Ethereum Foundation announcement of 2016-07-20; EIP-779; SEC Release No. 81207, p. 9	26
2017-07-25	SEC, "Report of Investigation … The DAO": "DAO Tokens are securities"	Release No. 81207	26·27
2019-03-15	Tokyo District Court (東京地裁), first-instance criminal judgment against Mt.Gox's representative director	Full text of the judgment (HTML)	26
2020-03	Adams, Zinsmeister and Robinson, "Uniswap v2 Core": a constant-product automated market	White paper PDF	15
2020-06-11	Tokyo High Court (東京高裁), criminal judgment on appeal against Mt.Gox's representative director (appeal dismissed)	Judgment PDF (courts.go.jp)	26
2021-08-05	Ethereum's London fork (EIP-1559, block 12,965,000, 12:33:42 UTC)	Block header (public RPC); go-ethereum `londonBlock`	21

DATE	MILESTONE	PRIMARY SOURCE (OPENED BY THIS BOOK)	CHAPTERS
2021-11	Bitcoin's Taproot activation (minimum activation height 709,632)	Bitcoin Core v31.1 `chainparams.cpp` (Appendix A-8)	5·29
2022-05	The collapse of Terra	BIS AER 2022 Box A (mechanism); judgments and verdict in Chapter 26	24·26
2022-09-15 (estimated)	Ethereum's transition to proof of stake (The Merge; estimated date the TTD was reached)	consensus-specs `configs/mainnet.yaml` (Appendix A-7)	10
2022-11-11	FTX files under Chapter 11 ("Petition Date"; West Realm Shires on 11-14)	Ray declaration ¶40	26
2023-02-03	Court of Appeal of England and Wales, *Tulip Trading Ltd v van der Laan* [2023] EWCA Civ 83	Judgment PDF	27·28
2023-06-09	EU MiCA published in the Official Journal (OJ L 150; applies from 2024-12-30)	Cellar PDF	27·30

Every date in this table comes from the primary source cited in the footnotes of the chapter responsible for it, and this appendix adds no new material. The marks "estimated," "not given" and "discrepancy" follow the text of those chapters.

2. Clickable timeline (screen edition)

This is the table above placed on a time axis. In the screen edition you select an item to see its source and use the "Chapter n →" button to go to the chapter that treats the event. The print edition shows a still image of the initial state.



Figure D-1. Clickable timeline, 1976–2023: the 29 milestones of the table above placed on a time axis. Select an item to see its source, and use the "Chapter n →" button to go to the chapter that treats the event from its original text. Dates are given at the precision confirmed (year, month or day), and estimated dates are marked separately. There is no price axis. — In the HTML edition you can change the values here (HTML edition, Appendix D, Section 2).

Sources

- "Sources for this chapter" in Chapter 2 and the footnotes of each responsible chapter (Chapters 1, 5, 6, 7, 8, 10, 12, 13, 14, 15, 21, 22, 24, 26, 27, 28 and 30). No new material.
- `timeline` in `widgets/data.json` (reference date 2026-10-25): the same items and the same sources as this table.

APPENDICES

Appendix E. Mathematical supplement

This appendix covers only what the [Advanced] boxes of the text say they "send to Appendix E." A sweep of the whole text for such places found one. The other topics listed as candidates at the design stage (modular arithmetic and finite fields, the elliptic-curve group, Poisson processes and random walks, 2×2 games and Nash equilibrium, unit roots and explosiveness tests) were each worked out in full during writing inside the [Advanced] boxes of the chapters concerned: Chapter 5 (elliptic curves and point addition), Chapter 9 (the negative binomial and the exact formula), Chapter 23 (the closed form and the critical share) and Chapter 25 (the double recursion and the wild bootstrap). So they are not repeated here.

Concordance: which box points here

CHAPTER AND BOX	THE SENTENCE THAT POINTS HERE	SECTION OF THIS APPENDIX
Chapter 4, "[Advanced] Inside SHA-256: a structure that chains compression functions together"	"The name and the detailed development of this design principle, which chains compression functions together, are left to Appendix E."	E.1
Chapter 18, Section 4, the [Intermediate] paragraph on proof systems	"The mathematics of the proofs is outside the scope of this book (Appendix E does not cover it either)." Not a request but **a declaration that it is out of scope**. This appendix does not cover it either	—

E.1 The Merkle–Damgård construction: chaining compression functions

[Advanced] Chapter 4 showed that FIPS 180-4 describes SHA-256 in two stages, preprocessing (padding, splitting into blocks, initial values) and hash computation (updating the state value block by block, the stage this appendix calls the compression function; FIPS 180-4 itself does not use the term),[1] and NB04-5 printed the intermediate values of that state as it passes from block to block. That design principle has a name: the **Merkle–Damgård construction**. The two people in the name each presented a paper at the same conference in 1989 (CRYPTO '89), Damgård's "A design principle for hash functions" and Merkle's "One-way hash functions and DES," and references [5] and [16] of Haber and Stornetta (1991) list the two.[2] This book did not open the originals of those two papers, so the construction itself is described in the words of a textbook draft whose original this book did open. Narayanan et al. write: "Recall that we require that our hash functions work on inputs of arbitrary length. Luckily, as long as we can build a hash function that works on **fixed-length** inputs, there's a generic method to convert it into a hash function that works on arbitrary-length inputs. It's called the Merkle-Damgard transform. SHA-256 is one of a number of commonly used hash functions that make use of this method. In common terminology, the underlying fixed-length collision-resistant hash function is called the compression function. It has been proven that if the underlying compression function is collision resistant, then the overall hash function is collision resistant as well."[3]

[Advanced] In symbols the construction is as follows. There is a compression function $f:\{0,1\}^n \times \{0,1\}^b \to \{0,1\}^n$: it takes an n-bit state and a b-bit block and outputs an n-bit state. For SHA-256, $n = 256$ and $b = 512$. Append padding bits and the length to the message M, cut it into b-bit blocks $M^{(1)}, \ldots, M^{(N)}$, start from a fixed initial value $H^{(0)}$ and repeat

$$H^{(i)} = f\left(H^{(i-1)},\, M^{(i)}\right), \qquad i = 1, \ldots, N$$

then take the last $H^{(N)}$ as the output. Reading the symbols: $H^{(i)}$ is the state value after the first i blocks have been taken in, and f is the 64-round compression function of NB04-5 in Chapter 4. A numerical example: when a two-block message was fed in in Chapter 4, the $H^{(1)}$ and $H^{(2)}$ that NB04-5 printed are exactly the two steps of this formula, and $H^{(2)}$ was equal to the value from `hashlib.sha256`.

[Advanced] Why collision resistance carries over can be shown by a short backtracking argument. Suppose two different messages $M \neq M'$ produce the same output. If their lengths differ, the last blocks (the blocks holding the length) differ, so at the last step $f(H^{(N-1)}, M^{(N)}) = f(H'^{(N'-1)}, M'^{(N')})$ the inputs differ while the outputs are equal, which is at once a collision of the compression function. If their lengths are the same, they have the same number of blocks, and we go back from the last step to find the first step at which the inputs part. At any step where $(H^{(i-1)}, M^{(i)}) \neq (H'^{(i-1)}, M'^{(i)})$ but $H^{(i)} = H'^{(i)}$, that is a collision of the compression function. Since the messages differ, such a step must exist. So "one collision of the whole function" mechanically gives "one collision of the compression function," and if the compression function is collision resistant, so is the whole. Putting the length into the padding (the reason the preprocessing of FIPS 180-4 appends the 64-bit

message length) supports the first branch of this argument.

[Advanced] Narayanan et al. point out one more thing: "You may have noticed that the block chain construction is similar to the Merkle-Damgard construction that we saw in the previous section. Indeed, they are quite similar, and the same security argument applies to both of them."[4] The chain of Chapter 7, in which each block holds the hash of the previous block header, has the same shape, "the state value passes to the next step," and what Narayanan et al. say goes as far as this: thanks to that shape, the same security argument (a collision of the final hash reduces to a collision of some block) applies to both. Chapter 7's observation that changing a block in the middle puts everything after it out of line is a result of that argument, not the argument itself. As Chapter 7 showed, however, that shape alone makes "re-threading" free; what makes it expensive is the proof of work of Chapter 8.

Sources

- NIST FIPS 180-4 (2015) §1 — the same place as in Chapter 4.
- Haber & Stornetta (1991), references [5] and [16] — bibliographic details of Damgård (CRYPTO '89; 1990) and Merkle (CRYPTO '89; 1990) (LNCS 435, Springer 1990). The two originals have not been obtained.
- Narayanan et al., *Bitcoin and Cryptocurrency Technologies*, the authors' public draft (2016-02-09), Chapter 1 — the description of the Merkle–Damgård transform and its similarity to a blockchain.
- The backtracking argument is this book's spelling-out of the textbook's statement that "it has been proven." No code.

Notes

1. NIST, *Secure Hash Standard (SHS)*, FIPS PUB 180-4 (August 2015), §1, Introduction (p. 3). Same as the original quoted in footnote 8 of Chapter 4.
2. Stuart Haber & W. Scott Stornetta, "How To Time-Stamp a Digital Document," *Journal of Cryptology* 3(2): 99–111 (1991), pp. 110–111, References. [5]: "I. Damgård. A design principle for hash functions. In Advances in Cryptology—Crypto '89 (ed. G. Brassard), pp. 416-427. Lecture Notes in Computer Science, vol. 435, Springer-Verlag, Berlin, 1990." [16]: "R. C. Merkle. One-way hash functions and DES. In Advances in Cryptology—Crypto '89 (ed. G. Brassard), pp. 428-446. Lecture Notes in Computer Science, vol. 435, Springer-Verlag, Berlin, 1990." The originals of the two papers are not among this book's source materials (a reserved item of Part 1). Footnote 9 of Chapter 4 cites [4] (Damgård, Eurocrypt '87) and [5] of the same reference list.
3. Arvind Narayanan, Joseph Bonneau, Edward Felten, Andrew Miller & Steven Goldfeder, *Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction* (Princeton University Press, 2016), the authors' public draft ("Draft — Feb 9, 2016"), Chapter 1. Emphasis added. Page numbers in the published edition not checked.
4. Narayanan et al. (2016 draft), Chapter 1.

APPENDICES

Appendix F. Glossary · list of interactive figures · list of notebooks · colophon

F.1 Glossary

These are the words that the text gives with their English at first appearance. The meaning is the sense in which this book uses the word, and the detailed definition is in the chapter(s) given. Statutory defined terms are listed separately in Appendix C.

TERM (KOREAN)	ENGLISH	MEANING IN THIS BOOK	CHAPTERS
이중지불	double spending	Spending the same digital money twice. The central difficulty of digital cash	1
신뢰받는 제3자	trusted third party	The one party that writes the ledger and decides what came "first" (a bank or mint)	1
시빌 공격	Sybil attack	A single attacker creating many fake participants to take a majority	1
비잔틴 장군 문제	Byzantine Generals Problem	The problem of the conditions under which a group that includes traitors can reach agreement	1·8
은닉 서명	blind signature	A scheme in which the signer signs without seeing the content: Chaum's electronic cash	1·2
해시캐시	Hashcash	A stamp whose cost is paid in computation and checked with a single hash	2·8
해시 함수	(cryptographic) hash function	A one-way function that turns an input of any length into a fixed-length fingerprint	4
역상 저항·제2역상 저항·충돌 저항	preimage / second-preimage / collision resistance	The three properties of a hash function	4
눈사태 효과	avalanche effect	The property that changing one bit of the input changes about half of the output	4
이중 SHA-256	double SHA-256	SHA-256 applied twice: Bitcoin's block hash and transaction identifier	4·12
머클–담고르 구성	Merkle–Damgård construction	A design that handles inputs of any length by chaining compression functions together	4·Appendix E
공개키·개인키	public key / private key	Cryptography that divides authority between two keys: verify with the public key, sign with the private key	5
디지털 서명	digital signature	A mark of authorization made with a private key and verified with a public key	5

TERM (KOREAN)	ENGLISH	MEANING IN THIS BOOK	CHAPTERS
타원곡선 이산로그 문제	ECDLP	The problem of finding k from $Q = [k]G$: the basis of signature security	5·29
secp256k1	—	The elliptic curve used by Bitcoin and Ethereum	5
ECDSA	Elliptic Curve Digital Signature Algorithm	The signature algorithm of Bitcoin and of Ethereum accounts	5
슈노르 서명	Schnorr signature	The signature that Bitcoin has also used since Taproot (2021) (BIP-340)	5
거래 가변성	transaction malleability	The former property that the representation of an authorized transaction could be changed, leaving its content and validity as they were, so as to change the transaction identifier (txid). Changing a signature (r, s) to $(r, n - s)$ is one route	5·26
주소	address	Usually an encoding of the hash of a public key (or script) (for P2TR, the public key itself)	5
P2PK · P2PKH · P2SH · P2WPKH · P2WSH · P2TR · P2MS	—	The standard types of Bitcoin output (locking script)	5·12·29
P2MR	Pay-to-Merkle-Root	Taproot with the key path removed: the output type proposed by BIP-360 (Draft)	29
머클 트리·머클 루트·머클 증명	Merkle tree / root / proof	Bundles transactions by hashing so that a single root vouches for them, and proves inclusion with a few branches	6
블록 헤더	block header	80 bytes: version, previous hash, Merkle root, time, target, nonce	7·12
논스	nonce	An arbitrary number varied and hashed in proof of work (in the account model, the transaction sequence number)	8·11
목표값·nBits·난이도	target / nBits / difficulty	The value a hash must not exceed, its compact representation, and a scale that is its inverse	8
작업증명	proof of work	The computation of finding a hash below the target: "one-CPU-one-vote"	8
해시율	hash rate	The number of hashes the network tries per second (estimated from the difficulty)	8·20
난이도 조정	difficulty adjustment	The rule that pulls the block interval back to 10 minutes every 2,016 blocks	8
채굴·채굴자	mining / miner	The work, and the people, of solving proof of work to make blocks	8
코인베이스	coinbase	The first transaction of a block: new coins and fees to the miner	12·20
블록 보조금·반감	block subsidy / halving	New coins per block, halved every 210,000 blocks	20
가장 긴 체인·누적 작업	longest chain / most accumulated work	The rule for choosing the canonical chain	7·8

TERM (KOREAN)	ENGLISH	MEANING IN THIS BOOK	CHAPTERS
포크(소프트·하드)	soft fork / hard fork	A rule change that old nodes also follow (a narrowing of the rules) / a rule change that splits from old nodes	7·23·26·29
확인 수	confirmations	The number of blocks stacked on a transaction, counting the block that contains it	9
확률적 최종성	probabilistic finality	The property that there is no moment of "done," only a probability of reversal that gets smaller	9
도박꾼의 파산	gambler's ruin	The probability model that §11 of the white paper uses for the attacker's chance of catching up	9
이기적 채굴	selfish mining	A strategy of withholding blocks and then releasing them to increase one's share	23
지분증명	proof of stake	Consensus that makes blocks through stake and voting	10
검증자	validator	A participant in proof of stake who puts up stake and proposes and attests to blocks	10
슬래싱	slashing	The penalty of cutting the stake of a validator who breaks the rules	10
UTXO	unspent transaction output	A transaction output not yet spent: the unit of state in Bitcoin	11
계정 모형	account model	A state model that keeps a balance and a nonce for each address: Ethereum	11
이더리움 가상머신	EVM	A state machine on which every node runs the same program	13
가스·가스 한도	gas / gas limit	The computational price set for each instruction, and the cap a transaction buys in advance	13
기본 수수료·우선 수수료·소각	base fee / priority fee / burning	The two parts of the fee under EIP-1559: the former is burned, the latter goes to the validator	13·21
스마트계약	smart contract	A program on the chain that releases value when conditions are met	13
ERC-20 · ERC-721 · ERC-165	—	Conventions on function names for tokens, non-fungible tokens and interface identification	14
대체불가 토큰	NFT, non-fungible token	A token identified by a (contract address, number) pair	14
자동화 시장조성자	AMM, automated market maker	A contract that sets prices by a formula (constant product) without an order book	15
상수곱·슬리피지·비영구 손실	constant product / slippage / impermanent loss	The AMM's formula, the worsening of the price with the size of the order, and the profit-and-loss structure of a liquidity provider	15
유동성 공급자	liquidity provider	A participant who puts two tokens into a pool and receives fees	15
오라클	oracle	A device that brings facts from outside the chain into it	16

TERM (KOREAN)	ENGLISH	MEANING IN THIS BOOK	CHAPTERS
최대 추출 가능 가치	MEV, maximum extractable value (formerly miner extractable value)	Value extracted from the power to decide the order, inclusion and exclusion of transactions	17
선행매매·후행매매·샌드위치	front-running / back-running / sandwich	Trading strategies that exploit order	17
제안자–빌더 분리	PBS, proposer–builder separation	A design that separates the party that builds a block from the party that proposes it	17
멤풀	mempool	The queue in which transactions wait before entering a block	17·29
롤업(낙관적·영지식)	optimistic / zero-knowledge rollup	A two-layer design that takes execution off the main chain and entrusts the data to it	18
시퀀서	sequencer	The party that sets the order in a rollup	18
데이터 가용성·블롭	data availability / blob	The problem of guaranteeing that data is available, and the unit of temporary storage for it	18
화폐의 세 기능	functions of money	Medium of exchange, unit of account, store of value	19
마팅게일	martingale	A process whose conditional expectation equals its current value: the price proposition of Chapter 19	19
합리적 버블	rational bubble	A price that departs from fundamental value without violating the no-arbitrage condition	19·25
보안 예산	security budget	The flow cost that honest participants spend on security, funded by rewards	20
EIP-1559	—	The fee market design of the London fork in 2021	21
거버넌스 토큰·DAO	governance token / decentralized autonomous organization	A token that carries voting rights, and an organization written in code	22
스테이블코인	stablecoin	A token with a redemption promise attached: fiat-collateralized, crypto-overcollateralized, algorithmic, synthetic	24
뱅크런·담보율	bank run / collateral ratio	A rush of redemption demands, and the measure of solvency	24
폭발적 근·GSADF	explosive root / generalized sup ADF	Econometric tests that locate bubble periods	25
논리 큐비트·물리 큐비트	logical / physical qubit	An error-corrected abstract qubit / a hardware element	29
쇼어·그로버 알고리듬	Shor's / Grover's algorithm	Discrete logarithms in polynomial time / search sped up to the square root	29
양자내성 암호	PQC, post-quantum cryptography	Cryptography believed to be secure even against quantum computers: FIPS 203·204·205	29

TERM (KOREAN)	ENGLISH	MEANING IN THIS BOOK	CHAPTERS
ML-DSA · SLH-DSA	—	Lattice-based signature standard (FIPS 204) · hash-based signature standard (FIPS 205)	29
장기 노출·단기 노출 (at-rest·on-spend)	long / short exposure attack	An attack on a public key that is already exposed / an attack on a public key in the mempool while the coin is being spent	29
결제 최종성	settlement finality	A point of irrevocability fixed by rules. In Bitcoin's proof of work there is no point at which the number of confirmations alone brings the probability of reorganization to 0 (Chapter 9), and "finalization" in proof of stake is finality under the protocol's rules, not the same thing as legal settlement finality (Chapter 10)	9·10·28.1(c)
가상자산 · crypto-asset · digital asset · 暗号資産 · cryptoasset	—	The defined terms of five jurisdictions	30·Appendix C

F.2 List of interactive figures

The twelve interactive figures of the screen edition. Their initial states are the same as the examples in the text, and every variable is a structural variable (no prices or returns). In the PDF a still image of the initial state appears in their place.

ID	NAME	CHAPTER · PLACE	VARIABLES	INITIAL STATE
W1	Hash avalanche	Chapter 4, after NB04-2	Input string, bit to flip	One letter changed from "hello"
W4	Merkle proof	Chapter 6, after NB06-1	Number of transactions n, leaf to prove	$n = 8$, leaf 5
W3	Chain tampering demonstration	Chapter 7, after NB07-1	Number and content of the block to tamper with	5-block chain, block 3 tampered with
W2	Hand miner	Chapter 8, after NB08-2	Difficulty in bits	16 bits
W5	Double-spending probability	Chapter 9, after NB09-2	q, z	$q = 0.1, z = 0...10$
W12	Block header anatomy	Chapter 12, after NB12-1	Field selection, nonce	Genesis block
W7	AMM slippage	Chapter 15, after NB15-1	Pool size x, y, order Δx	1,000 : 1,000, order 100
W10	Sandwich attack	Chapter 17, after NB17-1	Pool size, victim's order, attacker's input, slippage tolerance	The pool of Chapter 15, victim 100, attacker 100
W6	51% attack cost	Chapter 20, after NB20-3	Share q, unit cost of hashing, duration	Share 60%, $k + e = 12$
W9	Fee market	Chapter 21, after NB21-1	Gas used per block	Limit 60,000,000 · target 30,000,000
W8	Stablecoin bank run	Chapter 24, after NB24-2	Share demanding redemption, composition of reserve assets, collateral ratio	Type B′, demand 40%
W11	Clickable timeline	Chapter 2 · Appendix D	Click	1976–2023

F.3 List of notebooks

Colab notebooks that carry over the Python code of the text unchanged. The outputs are exactly the results of the editor's runs in Python 3.13 (conda environment `bcbook`), and the "Output" blocks in the text came from the same runs. There is no installation cell, and no external API is called. Cells that need the bundled data read the snapshots in `data/`, and if a file is missing they substitute constants with the same values. All the code is for study: do not use it with real assets, real private keys or real transactions.

NOTEBOOK	CHAPTER	CELLS	CONTENTS	EXTERNAL PACKAGES · DATA	OPEN
NB00	3	1	Environment check: Python version; `hashlib`, `secrets`, `struct`, `json` and `decimal` working	None	Colab
NB04	4	5	SHA-256 fingerprints, avalanche, forging a checksum vs. a hash, the birthday paradox, SHA-256 written by hand	None	Colab

NOTEBOOK	CHAPTER	CELLS	CONTENTS	EXTERNAL PACKAGES · DATA	OPEN
NB05	5	4	Group operations on a toy curve ($p = 97$), toy ECDSA and malleability, recovering a key from a reused k, checking the secp256k1 parameters	None	Colab
NB06	6	3	Merkle tree, proof and verification, table of proof lengths, a Bitcoin-style tree and CVE-2012-2459	None	Colab
NB07	7	2	Tampering with and re-linking a toy chain, two consistent chains	None	Colab
NB08	8	4	Hand miner, distribution of the number of tries, nBits → target → difficulty, simulation of the adjustment rule	None	Colab
NB09	9	5	Reproducing the white paper's formula and tables, the $P < 0.1\%$ table, the exact formula (finite negative-binomial sum), Monte Carlo, invariance to hash rate	None	Colab
NB10	10	2	Stake-weighted selection, immediate and correlation slashing penalties	None	Colab
NB11	11	2	UTXO state transition, account state transition	None	Colab
NB12	12	3	Parsing the genesis header, checking the double SHA-256, dissecting the coinbase transaction	`data/genesis_header.hex` · `genesis_block.hex`	Colab
NB13	13	3	Toy EVM (the fee schedule of Appendix G of the Yellow Paper), applying a transaction, an infinite loop and running out of gas	None	Colab
NB14	14	3	Imitating an ERC-20 ledger and the approve race condition, imitating an ERC-721 ledger, Keccak-256 implemented directly and identifiers recomputed	None	Colab
NB15	15	2	Constant-product pool (swaps, liquidity, slippage table), impermanent loss table and fee growth table	None	Colab
NB17	17	2	Sandwich simulation, the attacker's optimal input by the victim's slippage tolerance	None	Colab
NB19	19	2	Garratt–Wallace collapse probability, constructing the Schilling–Uhlig price path and checking the martingale	None	Colab
NB20	20	3	Bitcoin subsidy schedule, reproducing Budish's Proposition 3 and Table II, attack-cost calculator (in units of block rewards)	`data/hashrate_2026-10-25.json`	Colab
NB21	21	3	The EIP-1559 integer update formula and a check against the London fork block, first-price auction vs. 1559, the EIP-4844 blob fee path	None	Colab
NB23	23	2	Monte Carlo of the selfish-mining state machine, the 2×2 payoff table and the threshold	None	Colab

NOTEBOOK	CHAPTER	CELLS	CONTENTS	EXTERNAL PACKAGES · DATA	OPEN
NB24	24	3	The two equilibria of Diamond–Dybvig, reserve composition × redemption demand → depeg feedback, the algorithmic reflexive spiral	None	Colab
NB25	25	3	SADF and GSADF on synthetic data, Monte Carlo critical values and BSADF date-stamping, time-varying volatility and the wild bootstrap	`numpy` 2.5.4: the only external package in this book (reason in Chapter 25)	Colab
NB29	29	3	A general formula for the resources of Shor's algorithm and a comparison of four papers, recomputing the exposed amounts by type from the snapshot, a classifier of locking-script types	`data/exposed_output_types_2026-10-25.json`	Colab

Chapters 1, 2, 16, 18, 22, 26, 27, 28 and 30 and the appendices have no notebook, by design (chapters of concepts, surveys and provisions). The Solidity examples in Chapters 13 and 14 are for reading and were not put into the notebooks.

F.4 Colophon

Blockchain for Lawyers: A Reading Through Technology and Economics

- Edited by Bowon Kwon, Presiding Judge, Daegu District Court
- Edition: 1.0-en · published November 14, 2026 (reference date for figures 2026-10-25)
- Repository: AI-Assisted-Handbooks/blockchain-for-lawyers (https://github.com/BowonKwon/AI-Assisted-Handbooks)
- License: the text and the code of the interactive figures of this book are released under **Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0)**. https://creativecommons.org/licenses/by-nc/4.0/ — You may copy, adapt and share them for noncommercial purposes if you credit the source. Copyright in the originals quoted in the footnotes remains with their respective holders.
- The sources, reference dates and formulas of the bundled data (`data/`) are in the README of the data folder. The block data are raw responses from public nodes and explorers, and the tally of exposed output types is a third party's secondary tally (Chapter 29).
- The Solidity examples for reading in Chapter 13 are new code written for this book following the structure of the first example in the official Solidity documentation, and like the other code they are under CC BY-NC 4.0. The example code of the Solidity documentation (GPL-3.0) is not reproduced.
- This book states the editor's personal views and is unrelated to the institution the editor belongs to.
- Use of AI: the editor made this book with a generative AI system (Claude). Each chapter was checked against the originals several times, but erroneous statements may remain. Please confirm figures and quotations against the originals. This book will keep being

revised. This English edition was translated from the Korean original with the same AI system and checked against the Korean text and, where a quotation was originally in English, against that original wording.

- Errors: if you find a statement that differs from the original, please report it through the repository's Issues (https://github.com/BowonKwon/AI-Assisted-Handbooks/issues). Corrections are reflected with a new edition number and date.

Notice

This book is an academic self-study guide. It was written for studying the technology and economics of blockchains, and it does not recommend buying, selling or holding any particular crypto-asset, token or financial product. No sentence in this book should be read as an investment solicitation, investment advice or a price outlook.

The names of exchanges, projects and tokens in this book are used only to describe settled historical facts and are not recommendations or comparative evaluations. Every figure carries a reference date and a source and may have changed since. The code is meant to show principles and was not built to handle real assets. Do not use the code or the explanations in this book to manage real assets.

The editor is a sitting judge, but this book is the editor's personal study record and is unrelated to the views of the institution the editor belongs to. On ongoing cases and pending bills, it states only the facts and makes no evaluation. This book is not legal advice.

Sources

- The glosses in F.1 come from the text of each chapter. No new definitions were added.
- F.2 is taken from the places where the interactive figures are inserted in each chapter, and F.3 from the list in the notebook folder.
- The editor's name and affiliation, the license, the statement of no institutional connection and the notice in F.4 were set by the editor. The notice is given in full in this colophon, with short versions at the head of Chapter 3 and the opening of Chapter 26. The AI use notice and the "Errors" sentence use the same wording as the other self-study guides in the same repository.